

Arquitectura de Computadoras

Año 2023

Cursado y Promoción
Condiciones y fechas.

Aprobación de cursada

- Se realizarán 2 evaluaciones parciales de trabajos prácticos con dos fechas de recuperación.
- Las 2 evaluaciones parciales deben ser Aprobadas.
- Las evaluaciones se tomarán en aula y horario de cursada.
- Parcial 1: evaluará los Trabajos Prácticos 1, 2 y 3.
 - VIERNES 29 de SEPTIEMBRE
 - Recuperación 1: MARTES 17 de OCTUBRE
- Parcial 2: evaluará los Trabajos Prácticos 4, 5 y 6.
 - MARTES 14 de NOVIEMBRE
 - Recuperación 1: VIERNES 24 de NOVIEMBRE
- Recuperación 2 de Parcial 1 y/o de Parcial 2.
 - MARTES 05 de DICIEMBRE

Promoción de la asignatura.

- Se realizarán 2 evaluaciones de teoría que se tomarán en aula y horario de teoría de cada Turno.
- Evaluación Teórica Corta (ETC)
 - ✓ Condición:
 - ✓ Parcial 1 Aprobado en 1º fecha con el 70% de la nota máxima.
 - ✓ Fecha: MIÉRCOLES 11 de OCTUBRE
- Evaluación Teórica Promoción (ETP)
 - ✓ Condiciones:
 - ✓ Evaluación Corta de Teoría Aprobada.
 - ✓ Parcial 2 Aprobado en 1º fecha con el 70% de la nota máxima.
 - ✓ Fecha: MIÉRCOLES 22 de NOVIEMBRE

Arquitectura de Computadoras

CURSO 2022

Turno:

Clase 1

Resumen clase 1

Temas básicos – repaso conceptos de Organización

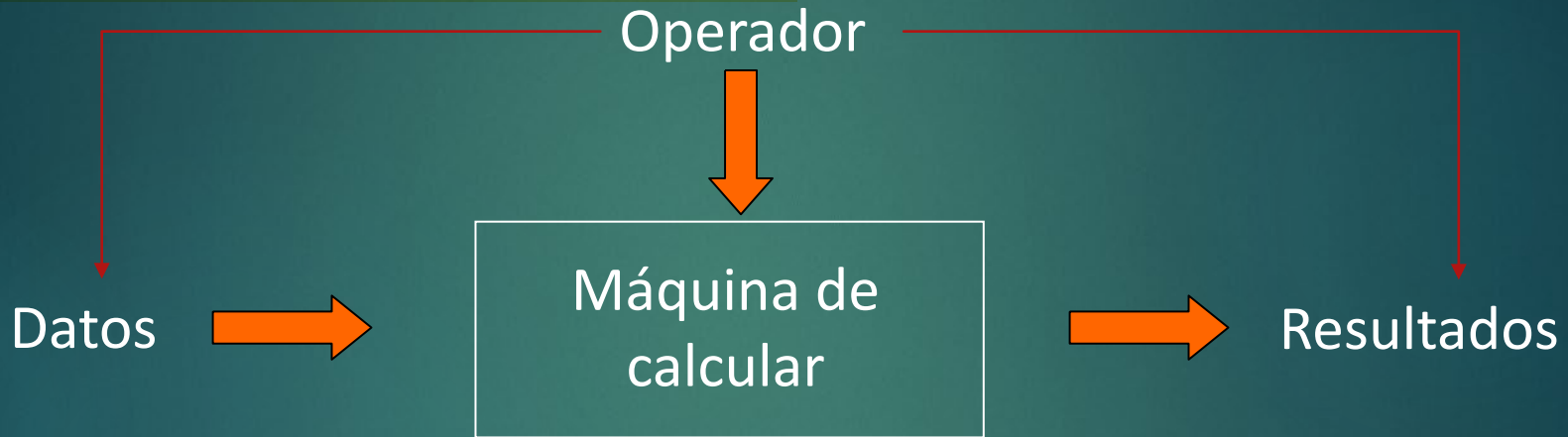
- Programas e instrucciones
- Modelo de von Neumann
- Lenguajes
- Instrucciones de máquina
- Ciclo de instrucción
- Simulador MSX-88
- Pila
- Subrutinas

Concepto de “programa”

- Se puede considerar como antecedente más cercano de los Sistemas de Cómputo a las máquinas de calcular.
- Las primitivas máquinas de calcular eran de tipo mecánico, capaces de realizar una sola operación (por ejemplo suma o resta) a la vez.
- Si el cálculo a resolver requería un solo tipo de operación (por ejemplo sumas o restas repetitivas) solo era necesario ingresar los datos de cada operación.
- Si, por el contrario, también se cambiaba la operación, se requería además modificar la máquina.
- En ambos casos, era necesaria la intervención de una persona (“operador”) para hacer esas tareas.

Concepto de “programa”

- Conceptualmente, esta forma de operación se puede asimilar al siguiente modelo:



- Este modelo en el que un “operador” tiene que actuar (modificar) sobre la máquina (es decir la “Unidad de Cálculo”) de acuerdo a la tarea a ejecutar, se puede considerar como un “modelo de Programación en hardware”, porque cuando se cambia la tarea, se requiere cambiar el medio físico que la ejecuta.

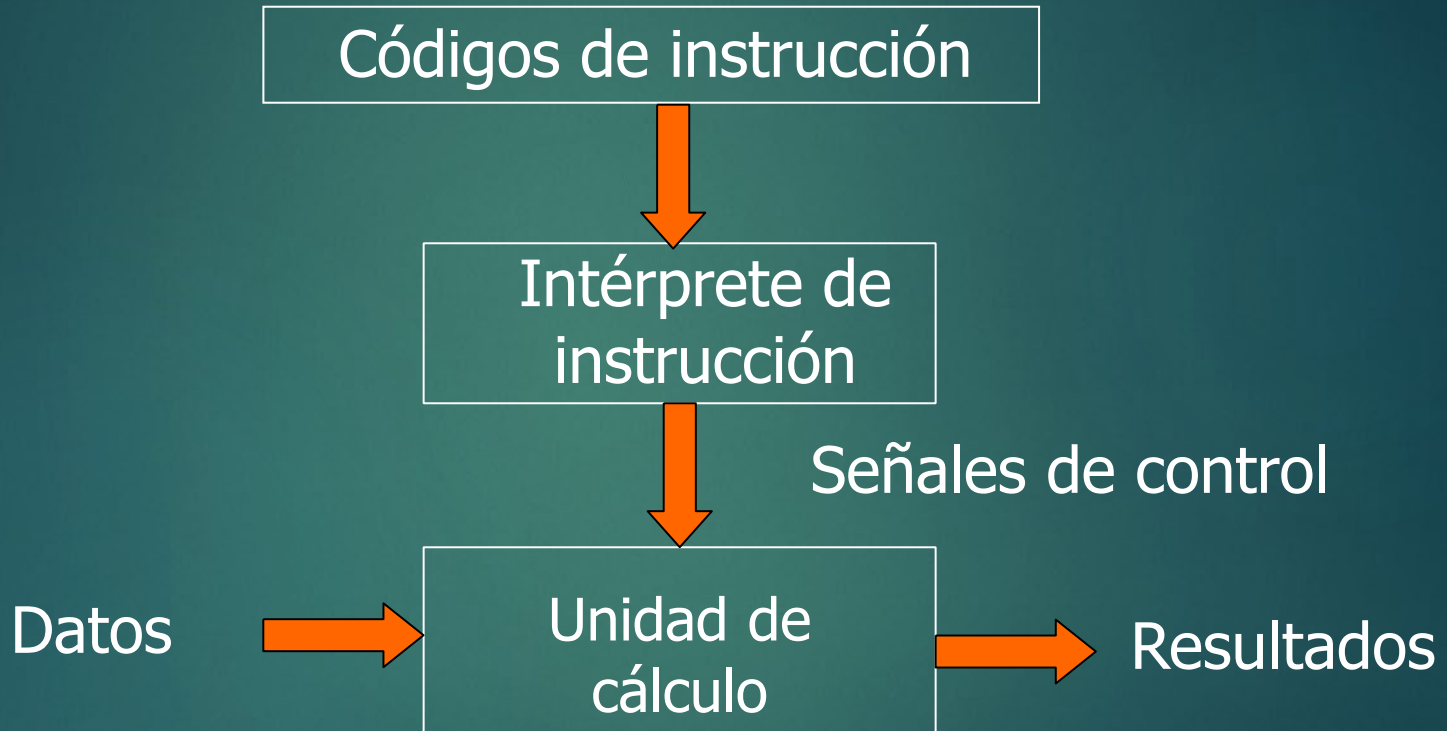
Concepto de “programa”

5

- En un Sistema de cómputo, la secuencia de tareas está almacenada en la máquina como instrucciones.
- Las instrucciones son interpretadas por la propia máquina en una unidad especial (Unidad de Control CU o Intérprete de instrucciones por hardware).
- Las instrucciones son convertidas en señales que comandan: la entrada de los datos, el tipo de operación, y la salida de los resultados.
- La Unidad de cálculo es universal y capaz de resolver distintos tipos de operaciones. El tipo de operación a resolver lo obtiene desde el Intérprete de instrucciones a partir de la instrucción leída.

Concepto de “programa”

- El modelo de operación cambia de la siguiente manera:



- Este nuevo modelo, en el que las operaciones se definen por medio de instrucciones almacenadas en memoria, se puede considerar como un “modelo de Programación en software”, porque el cambio de la tarea, solo requiere modificar la secuencia de instrucciones.

Concepto de “programa”

- En el modelo de Programación en software, la información requerida para ejecutar cada paso de la tarea es proporcionada por la secuencia de instrucciones a ejecutar.
- Cada instrucción es convertida en señales que son enviadas a las unidades funcionales.
- Cambiar la respuesta del Sistema implica únicamente modificar la secuencia de instrucciones (“programa”), que es fácilmente modificable (el “software”) debido a que está almacenada en una memoria dentro de la propia máquina.
- Por otra parte, las unidades funcionales (el “hardware”) no deben modificarse, su respuesta se modifica desde el programa (“software”).

Concepto de “programa”

- Cada instrucción contiene la información necesaria para resolver esa acción.
- Se requiere de un “intérprete por hardware” (la Unidad de control) que convierta la instrucción (en realidad decodifica) en el conjunto de señales requeridos por el hardware.
- Básicamente, un programa es una secuencia de instrucciones que realiza una tarea determinada.
- El programa puede ser modificado de una manera relativamente fácil (mucho más fácil que modificar el hardware).
- No hay que cambiar el hardware, es siempre el mismo. Cambia su comportamiento (y los resultados que produce) de acuerdo a la secuencia de acciones que ejecuta.

Concepto de “programa”

En definitiva, un Programa es:

- Una secuencia de acciones almacenadas en el sistema de cómputo.
- Cada acción es una instrucción (de máquina).
- Cada instrucción es convertida en señales.
- Diferentes señales de control se necesitan para cada operación.
- la Unidad de Control (CU) obtiene toda la información necesaria para resolver (ejecutar) la acción definida en la instrucción.

Modelo de von Neumann - IAS

- Las primeras máquinas (“Generación 1”) con características típicas de un Sistema de cómputo surgieron a principios de la década del 40.
- Impulsado por resolver cálculos matemáticos más o menos complejos en un tiempo relativamente pequeño.
- La Generación 1 eran máquinas destinadas a resolver básicamente problemas matemáticos (cuentas).
- El ámbito de aplicación era el de las áreas científica y militar.
- Usaban 2 tipos de tecnologías:
 - válvulas electrónicas (como las usadas en los primitivos televisores y radios, entre otros equipos electrónicos)
 - dispositivos electromecánicos (relés)

Modelo de von Neumann - IAS

11

- A mediados de la década del 40, había varios sitios donde se estaban desarrollando computadoras con características parecidas.
- Pero hubo una que sobresalió sobre el resto debido a que usaba conceptos novedosos no empleados en otros desarrollos.
- La máquina se conoció como IAS, y fué desarrollada por John von Neumann en la Universidad de Princeton (NJ).
- El Proyecto comenzó en 1942 y los últimos ajustes se completaron entre 1951 y 1952.

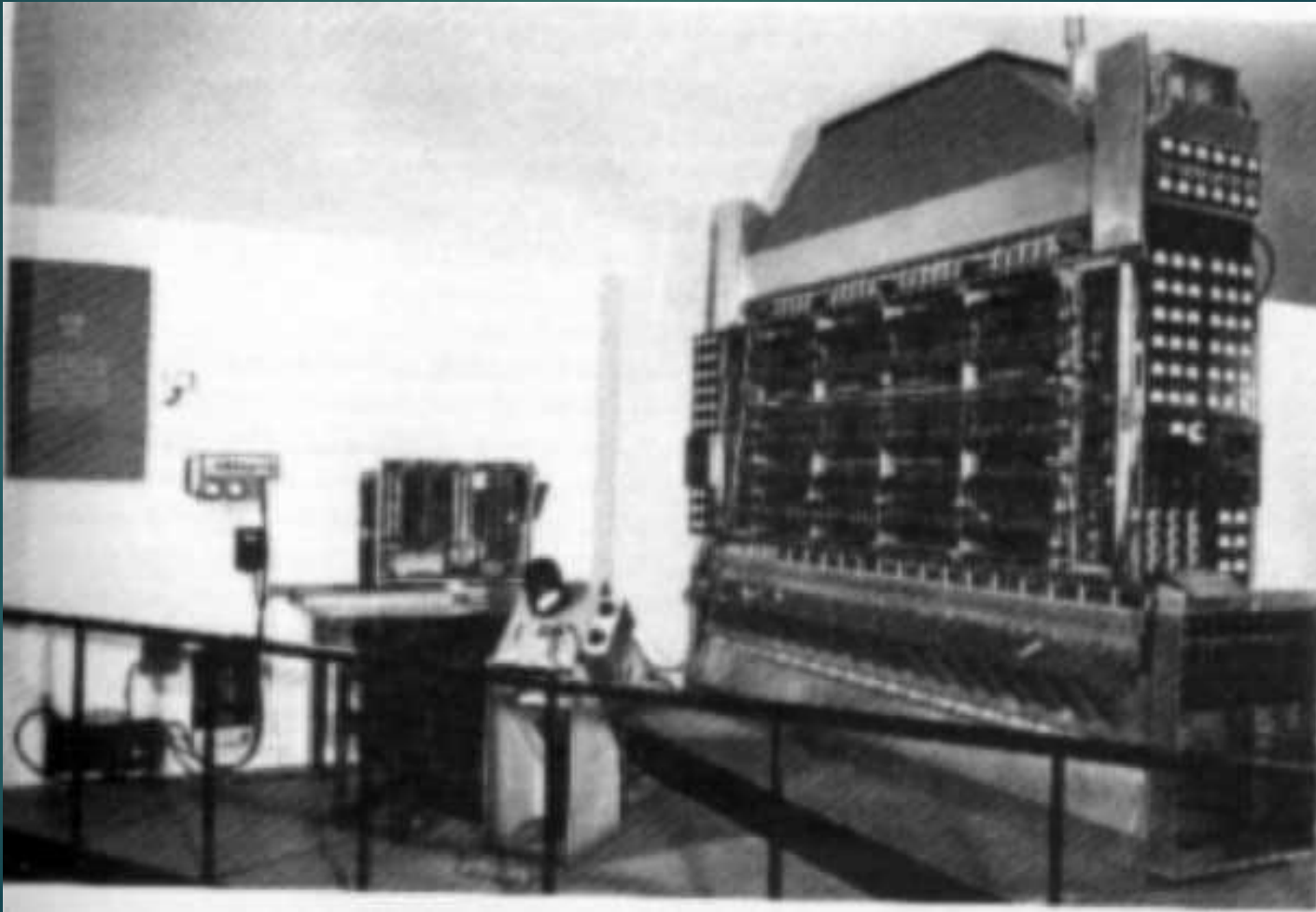
Modelo de von Neumann - IAS

- Una de las novedades más importantes de la IAS era que operaba en binario (en lugar de decimal).
- Tenía una memoria única (para instrucciones y datos), con 4096 palabras de 40 bits.
- El repertorio de instrucciones estaba compuesto por 21 instrucciones de 20 bits (incluyendo transferencia de datos, saltos, aritméticas, cálculo de dirección)
- El banco de registros de almacenamiento en la CPU estaba compuesto por:
 - Registro Buffer de Memoria (MBR)
 - Registro de Direcciones de Memoria (MAR)
 - Registros de Instrucción y Buffer de Instrucción
 - Registro Contador de Programa (Program Counter)
 - Registros Acumulador y Multiplicador/Cociente

Modelo de von Neumann - IAS

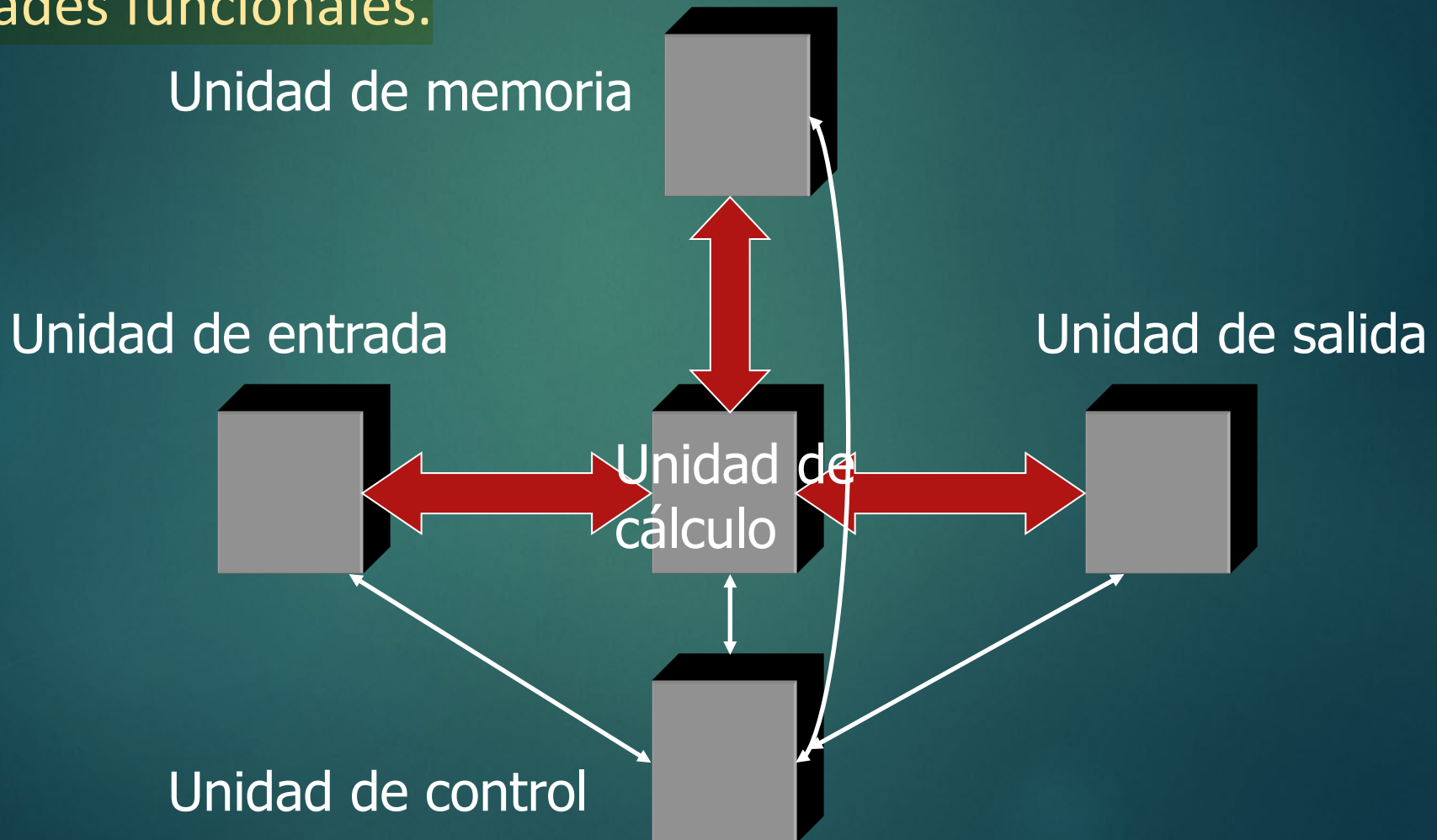
13

En la imagen siguiente se muestra una foto de la IAS.



Modelo de von Neumann de 5 subsistemas

El modelo de von Neumann de la IAS estaba basado en 5 Unidades funcionales.



Modelo de von Neumann de 5 subsistemas

Las 5 Unidades funcionales eran:

- Unidad de Control (UC)
- Unidad Aritmético-Lógica (ALU)
- Unidad de Memoria
- Unidad de Entrada (de datos)
- Unidad de Salida (de datos)

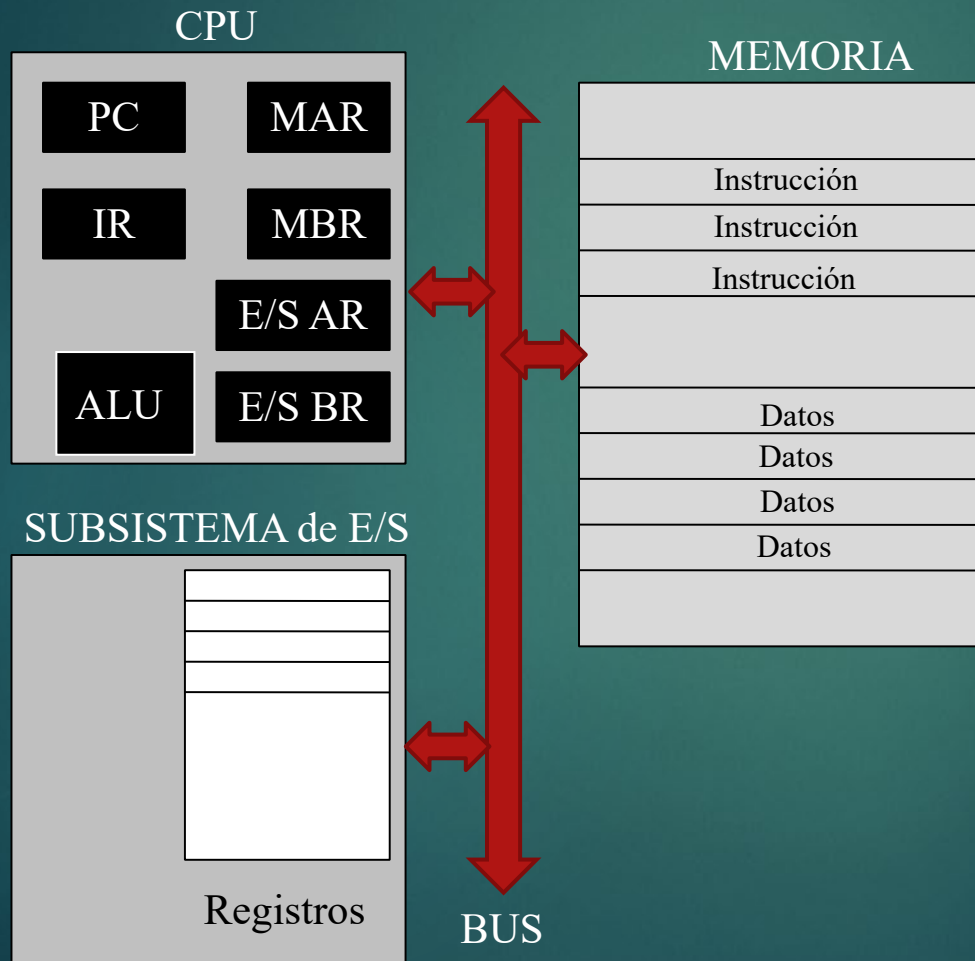
Este modelo de Arquitectura se puede simplificar en un modelo basado en 3 Unidades funcionales, modelo que perdura actualmente en muchas máquinas de propósito general.

Modelo de von Neumann de 3 Subsistemas

- La UNIDAD CENTRAL DE PROCESAMIENTO (CPU) constituida por:
 - Unidad de Control (UC)
 - Unidad Aritmético-Lógica (ALU)
 - Registros.
- La UNIDAD DE ENTRADA/SALIDA (E/S) para introducir y extraer los datos e instrucciones.
- La UNIDAD DE MEMORIA para almacenar temporalmente datos e instrucciones.

Modelo de von Neumann de 3 Subsistemas

Modelo de von Neumann con 3 subsistemas



- PC = Contador de programa
- IR = Registro de instrucción
- MAR = Registro de dirección de memoria
- MBR = Registro de datos de memoria
- E/S AR = Registro de dirección de E / S
- E/S BR = Registro de datos de E / S

Lenguajes

18

Lenguaje de máquina o absoluto

- El lenguaje que la máquina es capaz de “entender” y ejecutar se conoce como “lenguaje absoluto o de máquina”, y es del tipo binario.
- Las instrucciones están codificadas mediante un conjunto de bits almacenados en palabras de memoria (1 palabra o más).
- La representación de las instrucciones en memoria es puramente binaria.

Lenguajes

19

Lenguaje simbólico (Assembly)

- Dado que es prácticamente imposible para un programador tratar con las representaciones binarias de las instrucciones de máquina, se usan otras representaciones más “amigables”.
- Estas representaciones tienen distintos grados o niveles de abstracción.
- En el nivel siguiente al nivel de lenguaje de máquina, en cuanto a nivel de abstracción, está la representación simbólica (Lenguaje Assembly).
- En la representación simbólica, los campos que forman la instrucción se pueden representar con textos o números.

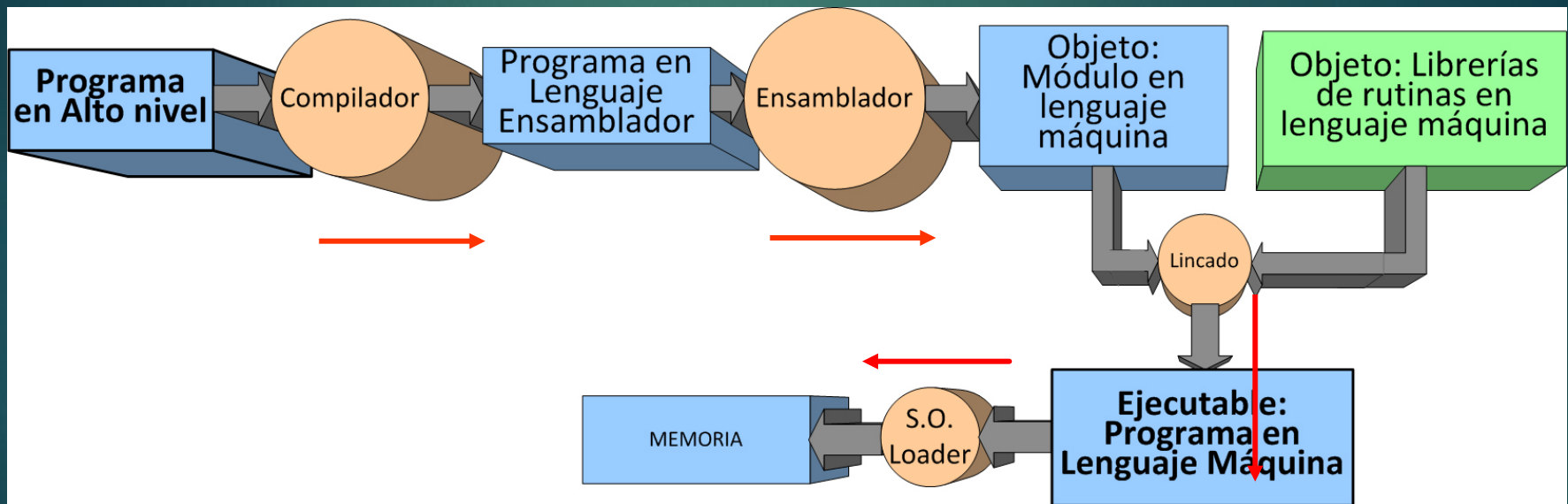
Lenguajes

Lenguaje simbólico (Assembly)

- En el lenguaje Assembly, los diferentes campos de la instrucción se pueden reemplazar por números o símbolos más fáciles de usar que el binario absoluto.
- Por ejemplo, el campo binario de la instrucción que contiene la información del tipo de instrucción (campo de código de operación COP) puede ser reemplazado, en el lenguaje Assembly por mnemónicos más sencillos de memorizar:
 - ADD (significa código de operación de) adición (suma)
 - SUB sustracción (resta)
 - MOV movimiento de datos
 - AND, OR, XOR operaciones lógicas

Lenguajes

El proceso de transformación (compilación) de un programa escrito en lenguaje de alto nivel (HLL) hasta el lenguaje de máquina típicamente es el siguiente.



Elementos de una instrucción de máquina

- Una instrucción de máquina es un código binario compuesto por una cantidad determinada de bits.
- Los bits son usados por la Unidad de Control (CU) para continuar y completar el proceso de ejecución de la instrucción.
- Para ello, el código debe contener suficiente “información” para que la Unidad de Control pueda resolverla.
- Las instrucciones tienen distinto grado de complejidad. Van desde las que no hacen nada, hasta las que operan sobre múltiples datos.

Elementos de una instrucción de máquina

- En general las instrucciones más complejas requieren de múltiples pasos o acciones. Las más sencillas se resuelven en pocos pasos.
- Un ejemplo de instrucción “relativamente compleja” es la que busca 2 operandos y produce un resultado. Para esta instrucción la CPU necesita definir:
 - 1: tipo de operación
 - 2: Lugar del 1er operando (donde está)
 - 3: Lugar del 2do operando (donde está)
 - 4: Lugar del resultado (donde guardarlo)
 - 5: donde está la próxima instrucción

Elementos de una instrucción de máquina

24

- Dado que hay diferentes datos que proveer, la instrucción está organizada en conjuntos de bits, comúnmente denominados campos, que proveerán esos datos.
- La cantidad de bits de un campo dependen de la cantidad de información que debe proveer.
- Por ejemplo, si un procesador dispone de 256 tipos de instrucciones, y un campo se usa para especificar el tipo de instrucción (conocido comúnmente como campo de Código de Operación COP), entonces ese campo requiere de al menos 8 bits.

Elementos de una instrucción de máquina

Los campos que típicamente pueden estar incluidos en una instrucción son:

- 1.- Campo de Identificación de la instrucción (Código de operación):
 - Especifica la operación a realizar (ej. sumar).
- 2.- Campos de Referencias a operandos:
 - Establece la información de referencia a el o los operandos fuentes (si es que se requieren). La información de referencia puede ser de distinto tipo (de acuerdo al modo de direccionamiento).
 - La operación puede involucrar uno ó más operando fuente (o de entrada). Por ejemplo en una suma se requiere especificar 2 operandos.

Elementos de una instrucción de máquina

➤ 3.- Campo de Referencia del resultado:

- Establece dónde almacenar el resultado.
- La operación puede involucrar uno o más resultados (de salida). Por ejemplo, en una suma se produce 1 resultado solamente.

➤ 4.- Campo de Referencia de la siguiente instrucción

- Provee a la CPU con información para determinar donde buscar la siguiente instrucción después de la ejecución de la instrucción anterior.
- En la mayoría de los casos, la próxima instrucción se ubica a continuación de la instrucción actual, es decir, en la siguiente posición de memoria. Esta situación se conoce como acceso secuencial a instrucciones consecutivas de memoria.

Taxonomía basada en formato de instrucción

- Una forma de clasificación de las máquinas está basado en el formato de instrucción.
- La clasificación identifica la cantidad de campos que contienen direcciones de memoria.
- Suponiendo el caso de una instrucción de suma de 2 operandos en memoria, y resultado almacenado en memoria, se necesitan:
 - 2 campos de direcciones para hacer referencia a los operandos.
 - 1 campo de dirección para hacer referencia a donde almacenar el resultado.
 - 1 campo de dirección de referencia de la ubicación de la próxima instrucción.

Taxonomía basada en formato de instrucción

En base a lo anterior, se pueden tener 5 tipos de maquinas básicos:

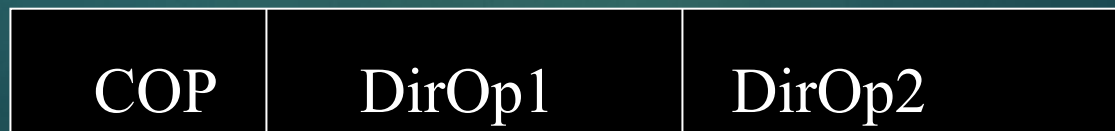
➤ Máquina de 4 direcciones:



➤ Máquina de 3 direcciones:



➤ Máquina de 2 direcciones:



Taxonomía basada en formato de instrucción

29

➤ Máquina de 1 dirección:



➤ Máquina de 0 dirección (máquina tipo Pila):



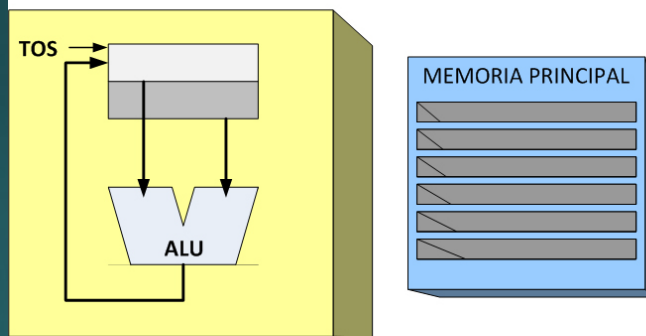
Taxonomía basada en los tipos de almacenamientos de datos

- Otra forma de clasificación de las máquinas está basada en los tipos de almacenamiento de los datos.
- Los datos pueden estar almacenados en 3 lugares físicos:
 - Memoria: cache, principal, virtual. En todos los casos se requiere para referenciar el dato, una dirección de memoria.
 - Registro de la CPU: la referencia es un número de registro
 - Dispositivo de E/S: la referencia es un número de registro de E/S

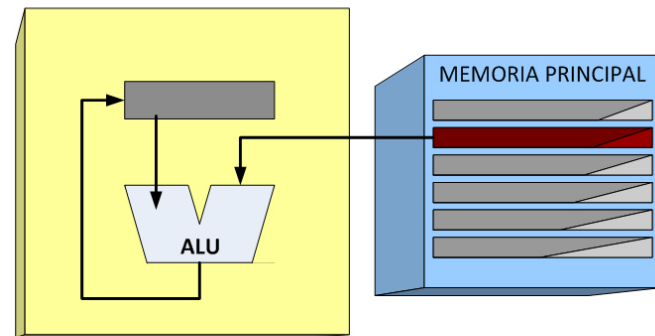
Taxonomía basada en los tipos de almacenamientos de datos

- Suponiendo una instrucción con referencias a 2 operandos y 1 resultado, podemos tener 4 tipos de máquinas básicas.

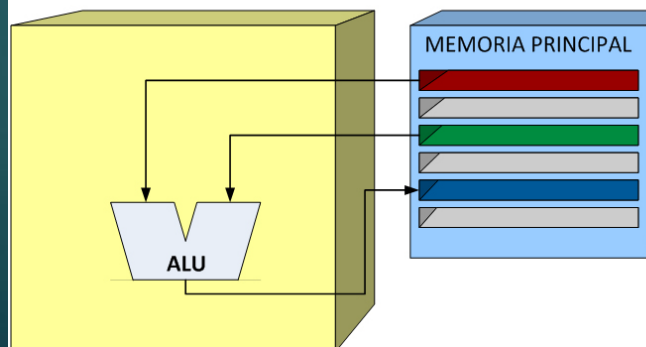
Almacenamiento tipo Pila



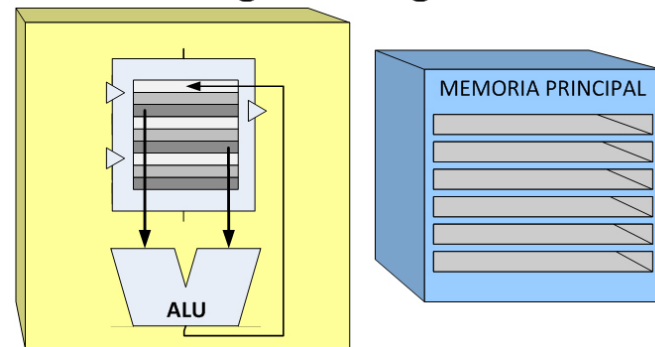
Almacenamiento tipo Acumulador



Almacenamiento tipo
Memoria - Memoria



Almacenamiento tipo
Registro-Registro



Decisiones en el diseño del conjunto de instrucciones

Cuando se diseña la arquitectura de un procesador son muchos los aspectos que se tienen en cuenta. Algunos de los más relevantes son:

- 1) Formato de instrucción
- 2) Tipo de datos
- 3) Repertorio de instrucciones
- 4) Cantidad y tipo de registros
- 5) Modos de direccionamiento

Decisiones en el diseño del conjunto de instrucciones

1) Formato de instrucción

Se refiere a la estructura que tendrán las instrucciones. 2 aspectos son relevantes en el diseño del formato de instrucción:

➤ Formato fijo o variable

- Formato fijo: todas las instrucciones tienen el mismo tamaño (bits). La captación y ejecución es más sencilla pero aumenta la cantidad de instrucciones de los programas.
- Formato variable: la longitud de las instrucciones varía con el tipo de instrucción y modo de direccionamiento. La captación y ejecución es más compleja pero disminuye la cantidad de instrucciones de los programas.

Decisiones en el diseño del conjunto de instrucciones

- Cantidad de campos de direcciones
 - A mayor cantidad de referencias de direcciones, se tienen instrucciones más complejas y largas, menos instrucciones por programa, y captación y ejecución de las instrucciones en forma más lenta.
 - A menor cantidad de referencias de direcciones, se tiene instrucciones más simples y cortas, más instrucciones por programa, captación y ejecución de las instrucciones más rápida.

Decisiones en el diseño del conjunto de instrucciones

2) Tipos de datos

Una decisión relevante, referida a las características del procesador y del repertorio de instrucciones, es la cantidad y tipo de datos que puede soportar.

Los tipos de datos más comunes son:

- Numérico: Con/sin signo, punto fijo / punto flotante, BCD, etc.
- Caracter: ASCII, EBCDIC, etc.
- Datos lógicos: manipulación de bits

Decisiones en el diseño del conjunto de instrucciones

3) Repertorio de instrucciones

En el diseño del repertorio de instrucciones, se deben considerar, de manera prioritaria:

- La cantidad de instrucciones (operaciones)
- Tipos de instrucciones
- Características de las instrucciones (sencillas, complejas, etc.)

Decisiones en el diseño del conjunto de instrucciones

4) Administración y uso de Registros

Dado que los registros disponibles en un procesador son un recurso escaso y tienen un fuerte impacto en la velocidad de ejecución de los programas, deben ser diseñados en forma asumamente cuidadosa, en particular:

- Tamaño y cantidad de registros
- Tipos de registros: como va a ser el uso y a que instrucciones se van a aplicar.

Decisiones en el diseño del conjunto de instrucciones

5) Modos de direccionamiento

Un aspecto importante en el diseño del repertorio de instrucciones es el referido a los modos de direccionamiento.

Las decisiones de diseño de los modos de direccionamiento se orientan a:

- Cantidad de modos de direccionamiento
- Tipos de modos de direccionamiento

Decisiones en el diseño del conjunto de instrucciones

Los diseños de las arquitecturas de procesadores se han orientado hacia 2 estrategias:

- Aumentar la complejidad y prestaciones del procesador
- Simplificar y optimizar el Procesadores
- El primer camino conduce a procesadores identificados genéricamente como tipo CISC (Computadores de Conjunto Complejo de Instrucciones)
- El segundo corresponde a las familias de procesadores RISC (Computadores de Repertorio de Instrucciones Reducido)

Decisiones en el diseño del conjunto de instrucciones

- Los procesadores CISC básicamente tienen:
 - Repertorio de instrucciones muy amplio
 - Muchos modos de direccionamiento
 - Formato de instrucción variable
- Los procesadores RISC se caracterizan por tener:
 - Repertorio de instrucciones simplificado
 - Pocos modos de direccionamiento
 - Formato de instrucción fijo

Técnicas de almacenamiento

41

- Las técnicas de almacenamiento se refieren a la forma en que los datos se guardan en memoria, particularmente en los casos en que ocupan más de una palabra de memoria.

➤ Ejemplo:

Supongamos que tenemos el siguiente dato de 32 bits (4 bytes):

98765432H (32 bits)

y supongamos tener una memoria organizada en palabras de 8 bits (1 byte por palabra).

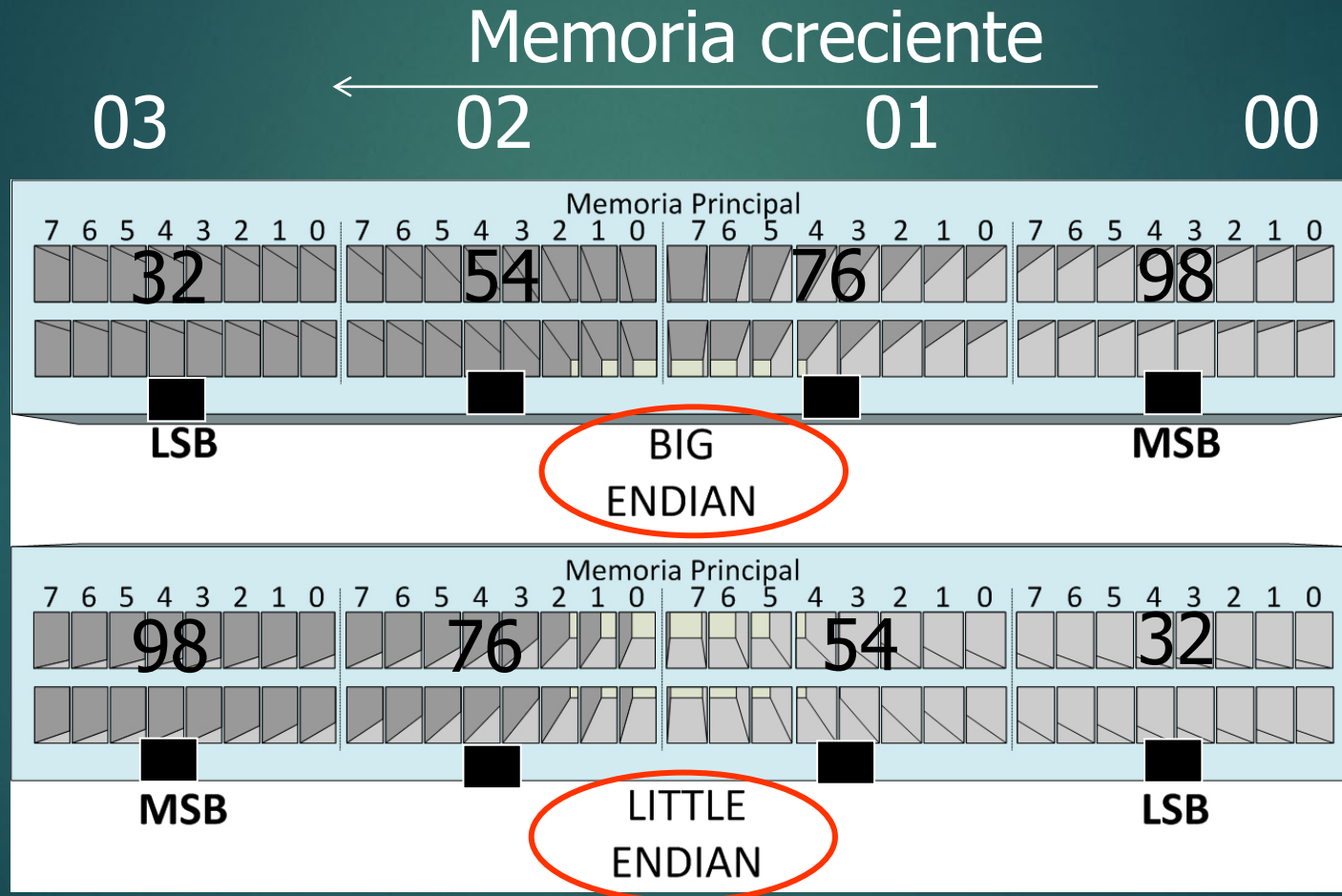
- Existen 2 formas de almacenar el dato (en múltiples palabras de memoria) de acuerdo al orden en que se almacenan las diferentes partes del dato:

Técnicas de almacenamiento

- Las 2 formas se llaman:
 - Big-endian: el byte más significativo se almacena en la dirección con valor numérico más bajo (es decir en la dirección mas baja)
 - Little-endian: el byte mas significativo se almacena en la dirección con valor numérico más alto (es decir, en la dirección más alta).
- En la figura siguiente se muestran las 2 formas.

Técnicas de almacenamiento

Almacenamiento Big-endian y Little-endian



Técnicas de almacenamiento

44

Es decir que, físicamente, en la memoria quedan almacenados así:

Dir. de memoria	Big-endian	Little-endian
xxx00	98	32
xxx01	76	54
xxx02	54	76
xxx03	32	98

Técnicas de almacenamiento

45

- Ejemplo de procesadores con almacenamiento Little-endian:
 - Intel 80x86
 - Pentium
 - VAX
- Ejemplo de procesadores con almacenamiento Big-endian:
 - IBM S/370
 - Motorola 680x0 (antiguos procesadores de las Mac)
 - La mayoría de los procesadores RISC
- Notar que las 2 formas de almacenamiento son incompatibles entre sí.

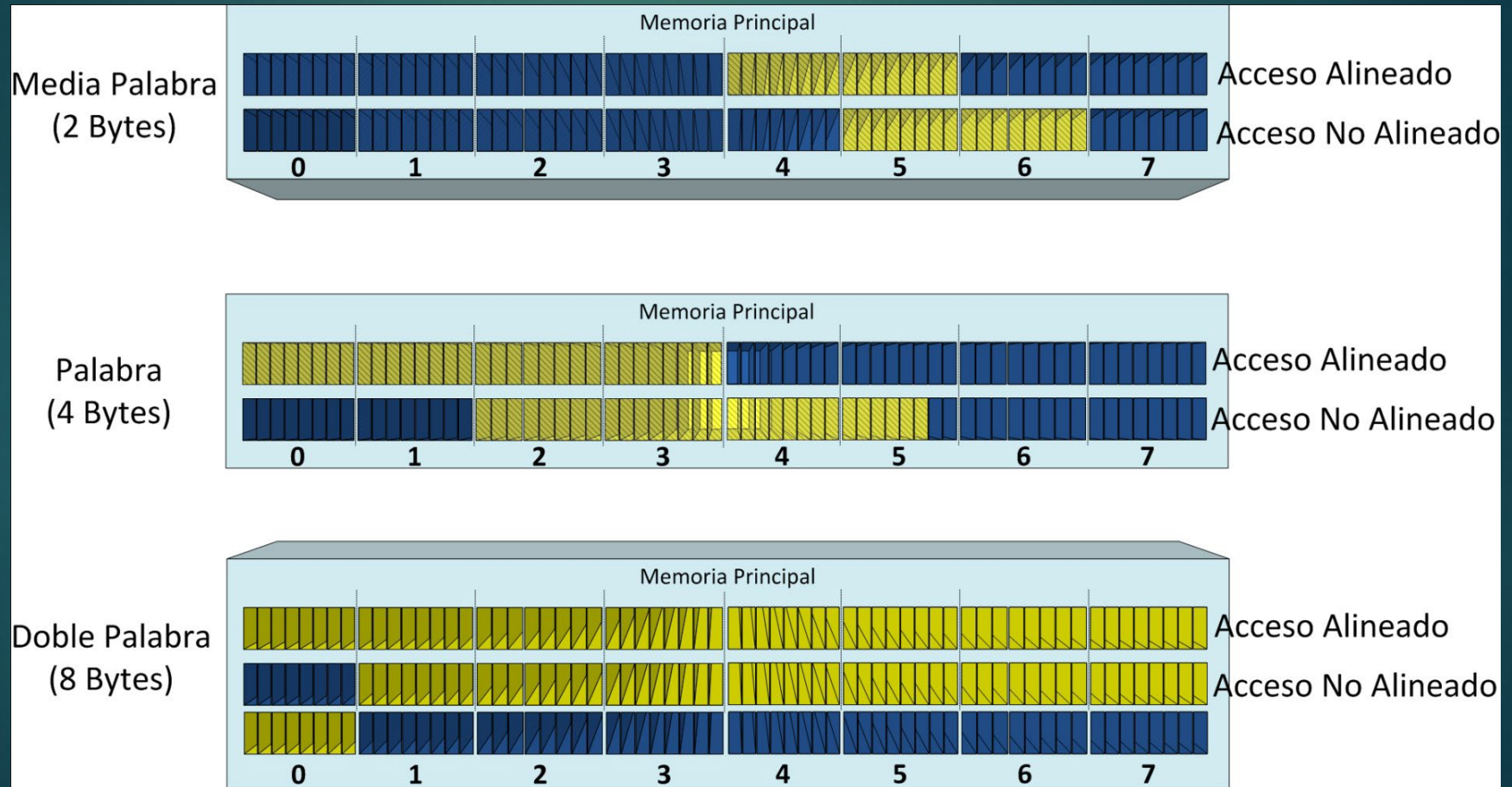
Técnicas de almacenamiento

46

- El otro aspecto relacionado con el almacenamiento de los datos, es el referido a las direcciones de memoria donde se guardan los datos, particularmente en los casos en que ocupan más de una palabra de memoria.
- Existen 2 formas de almacenar los datos múltiples en memoria:
 - Alineados: si un dato de n bytes empieza en una dirección de memoria múltiplo de 2^m , donde $m = \log_2 n$, con $n > 1$. En otras palabras, los m bits menos significativos son 0.
 - No alineados: si no cumple la condición anterior.
- En la figura siguiente se muestran las 2 técnicas para los casos de datos de 2, 4 y 8 bytes.

Técnicas de almacenamiento

Almacenamiento alineado y desalineado



Técnicas de almacenamiento

48

- En los ejemplos anteriores se puede apreciar que:
 - Si el dato es de 2 bytes, el almacenamiento alineado ocurre cuando se almacena a partir de una dirección múltiplo de 2.
 - Si el dato es de 4 bytes, el almacenamiento alineado ocurre cuando se almacena a partir de una dirección múltiplo de 4.
 - Si el dato es de 8 bytes, el almacenamiento alineado ocurre cuando se almacena a partir de una dirección múltiplo de 8.
- Algunos procesadores permiten únicamente acceso alineado. Otros procesadores admiten ambos tipos de accesos, alineados y no alineados. Otros, solo alineados.
- En general, los accesos no alineados son más lentos que los accesos alineados.

Repertorio de instrucciones

Los tipos de instrucciones más comunes que se pueden encontrar en el repertorio de instrucciones de un procesador pertenecen a alguna de las siguientes categorías:

- Transferencias de datos
- Aritméticas
- Lógicas y de rotación y desplazamiento
- Conversión
- Entrada/Salida
- Control de flujo
- Control del sistema

Repertorio de instrucciones

Instrucciones de Transferencia de datos

- Son las más típicas. En general se requiere especificar:
 - Ubicación del operando fuente (usando alguno de los modos de direccionamiento)
 - Ubicación del operando destino (usando alguno de los modos de direccionamiento)
 - Tamaño de los datos a ser transferidos
- En el lenguaje Assembly, los mnemónicos de las instrucciones de movimiento de datos pueden ser:
 - Diferentes mnemónicos para diferentes instrucciones. Por ejemplo: TFR Reg-Reg, LOAD Reg-Mem, STORE Mem-Reg
 - Un solo mnemónico y diferentes formatos. Por ejemplo: MOV destino, fuente ; MOV reg1,reg2

Repertorio de instrucciones

Instrucciones Aritméticas

- Son también muy comunes.
- Las instrucciones típicas provistas en un procesador convencional son: Add, Subtract, Multiply y Divide.
- Un aspecto importante de estas instrucciones es el tipo de dato que puede manejar: números enteros sin/con signo, números en punto flotante, etc.
- Otras instrucciones aritméticas pueden ser:
 - Increment o Decrement (operando +/- 1)
 - Negate: cambia el signo del operando (en Ca2).
 - Absolute: toma el valor absoluto del operando.
 - Shift left/right: desplazamiento de bits a izq/der un lugar

Repertorio de instrucciones

52

Instrucciones Lógicas y de rotación y desplazamiento

- Las instrucciones lógicas realizan operaciones que manipulan bits individualmente o en conjunto
- Las más comunes son las funciones de tipo booleana, es decir: AND, OR, XOR, NOT
- Las instrucciones de rotación y desplazamiento son, por ejemplo: Rotate (rota a izq/der), Rotate through carry (rota a través del carry a izq/der), Shift (desplaza a izq/der).

Repertorio de instrucciones

Instrucciones de conversion de tipo

- Son instrucciones poco comunes orientadas a cambiar formatos de datos.
- Algunos ejemplos:
 - Conversión de binario a decimal
 - Conversión de EBCDIC a ASCII

Repertorio de instrucciones

54

Instrucciones de ENTRADA/SALIDA

- Algunos procesadores tienen instrucciones específicas de Entrada/Salida. Los que no tienen este tipo de instrucciones, usan las mismas instrucciones de movimiento de datos con la memoria, para realizar las transferencias de E/S.
- Las instrucciones de E/S típicamente son pocas, con acciones específicas. Por ejemplo: IN, OUT
- Si disponen de instrucciones específicas de E/S, se requiere disponer, asimismo, de un espacio de direccionamiento separado para E/S y Memoria (típicamente en procesadores de la familia Intel 80x86, por ejemplo).

Repertorio de instrucciones

Instrucciones de control (de flujo de programa)

- Son instrucciones que pueden modificar el valor contenido en el registro PC.
- Las instrucciones más comunes de este tipo son:
 - Salto Incondicional: JMP destino
 - Salto Condicional: JZ destino
 - Salto con retorno (llamada a subrutina): CALL subrut
 - Retorno de subrutina: RET
 - Interrupción por software

Repertorio de instrucciones

56

Instrucciones de control de Sistema (control de la CPU)

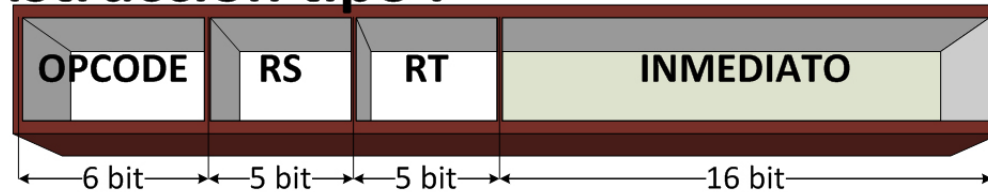
- Modifican algún estado de operación de la CPU.
- Las instrucciones más comunes de este tipo son:
 - Habilidad o inhibición de interrupciones
 - Modo de operación: supervisor/usuario
 - Halt: suspensión de actividades

Formato de instrucción fijo

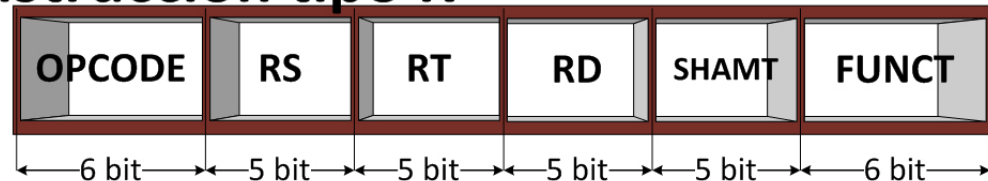
- Los formatos de las instrucciones pueden ser fijos o variables.
- En los formatos de instrucción de longitud fija, el tamaño de la instrucción es el mismo para todas las instrucciones.

Por ejemplo, el formato de instrucción de un procesador tipo RISC es el siguiente (solo 3 tipos de instrucciones todas de 32 bits).

Instrucción tipo I



Instrucción tipo R



Instrucción tipo J

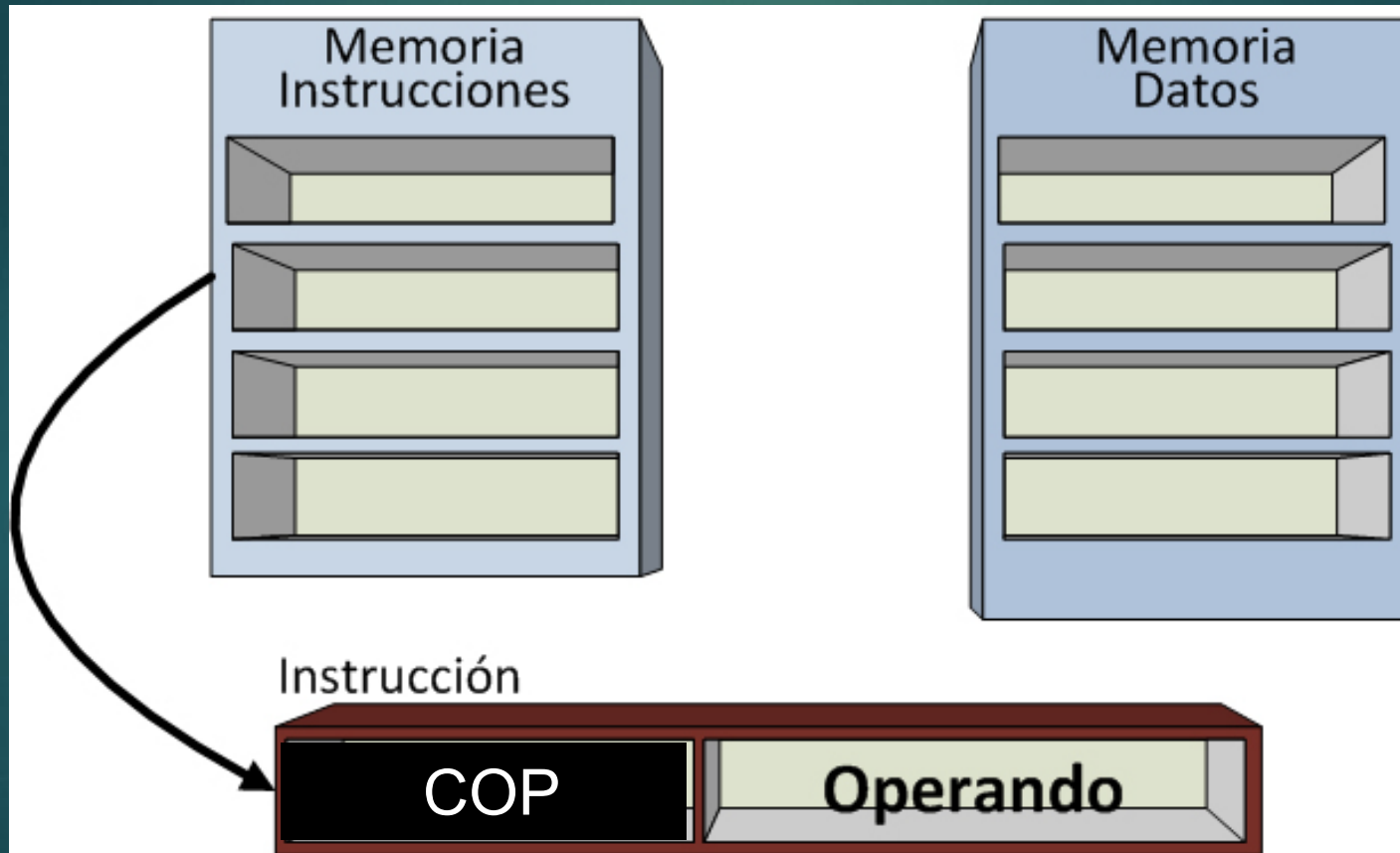


Modos de direccionamiento

- Los modos de direccionamiento son las diferentes formas de especificar el lugar donde reside un operando, donde se va a guardar el resultado, o donde buscar la próxima instrucción a ejecutar.
- En general, los procesadores disponen de una variedad más o menos amplia de modos de direccionamiento. Algunos de los más típicos son:
 - Inmediato
 - Directo: a memoria (“absoluto”), a Registro
 - Indirecto: a memoria (en desuso), con registro
 - Base: base, base Indexado
 - Relativo al PC
 - Pila (o relativo al SP)

Modos de direccionamiento

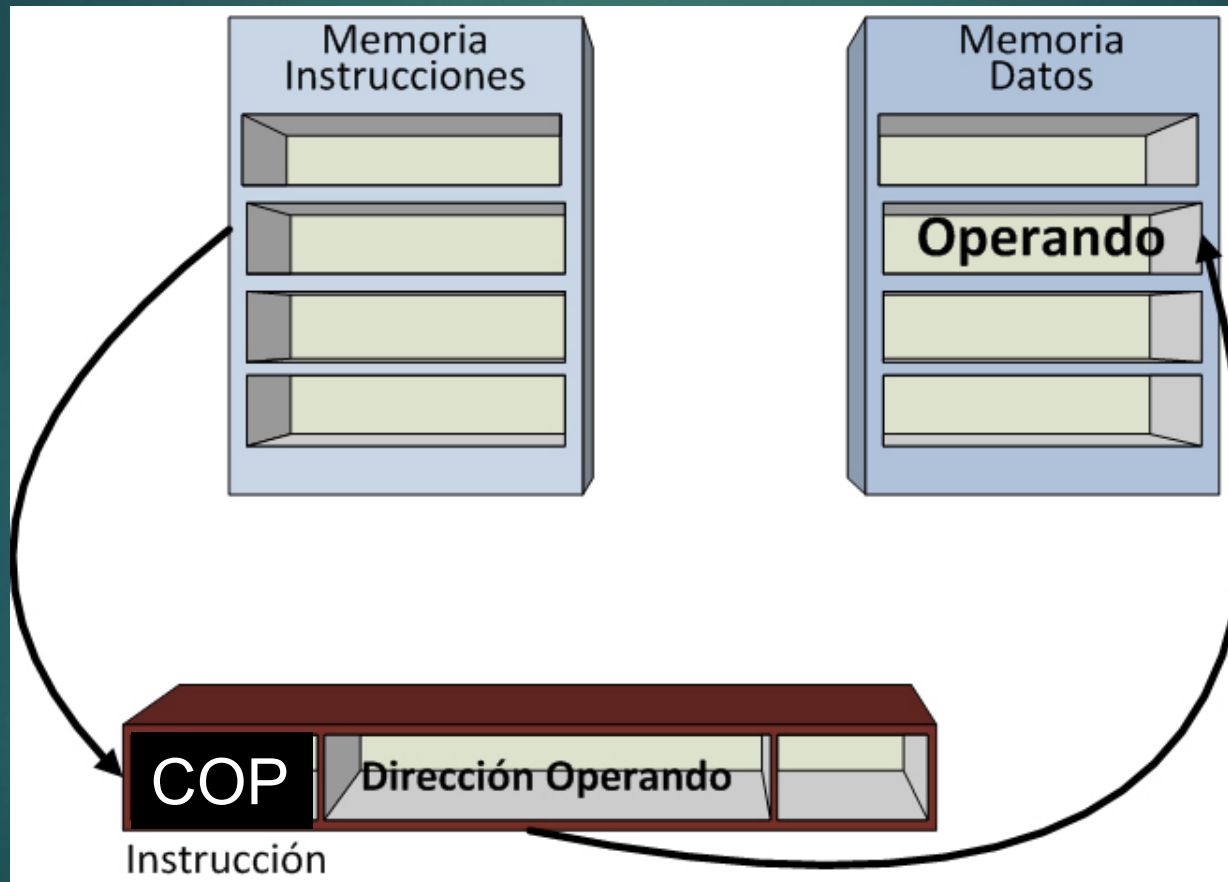
➤ Esquema del modo de direccionamiento Inmediato



COP: Código de operación

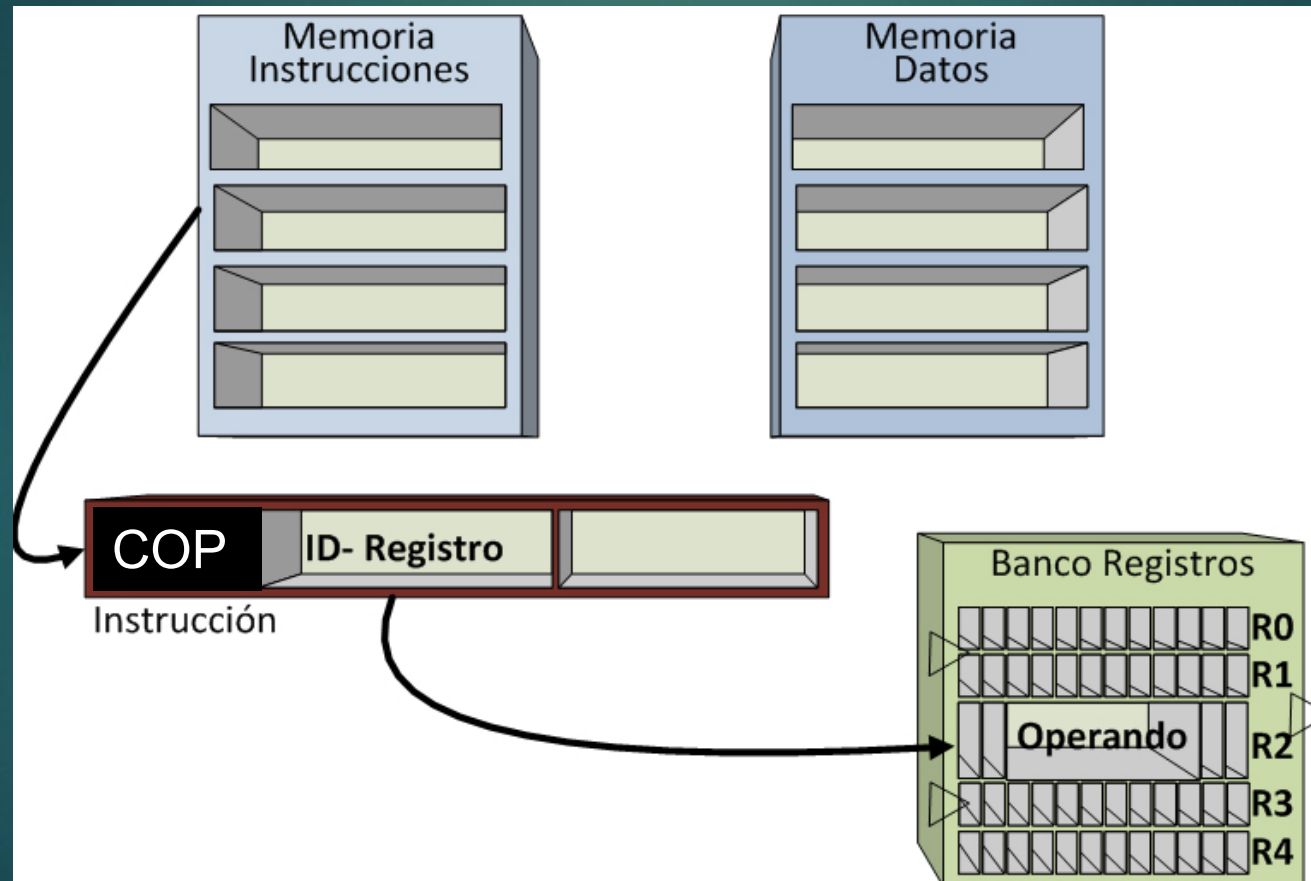
Modos de direccionamiento

- Esquema del modo de direccionamiento Directo a memoria o absoluto



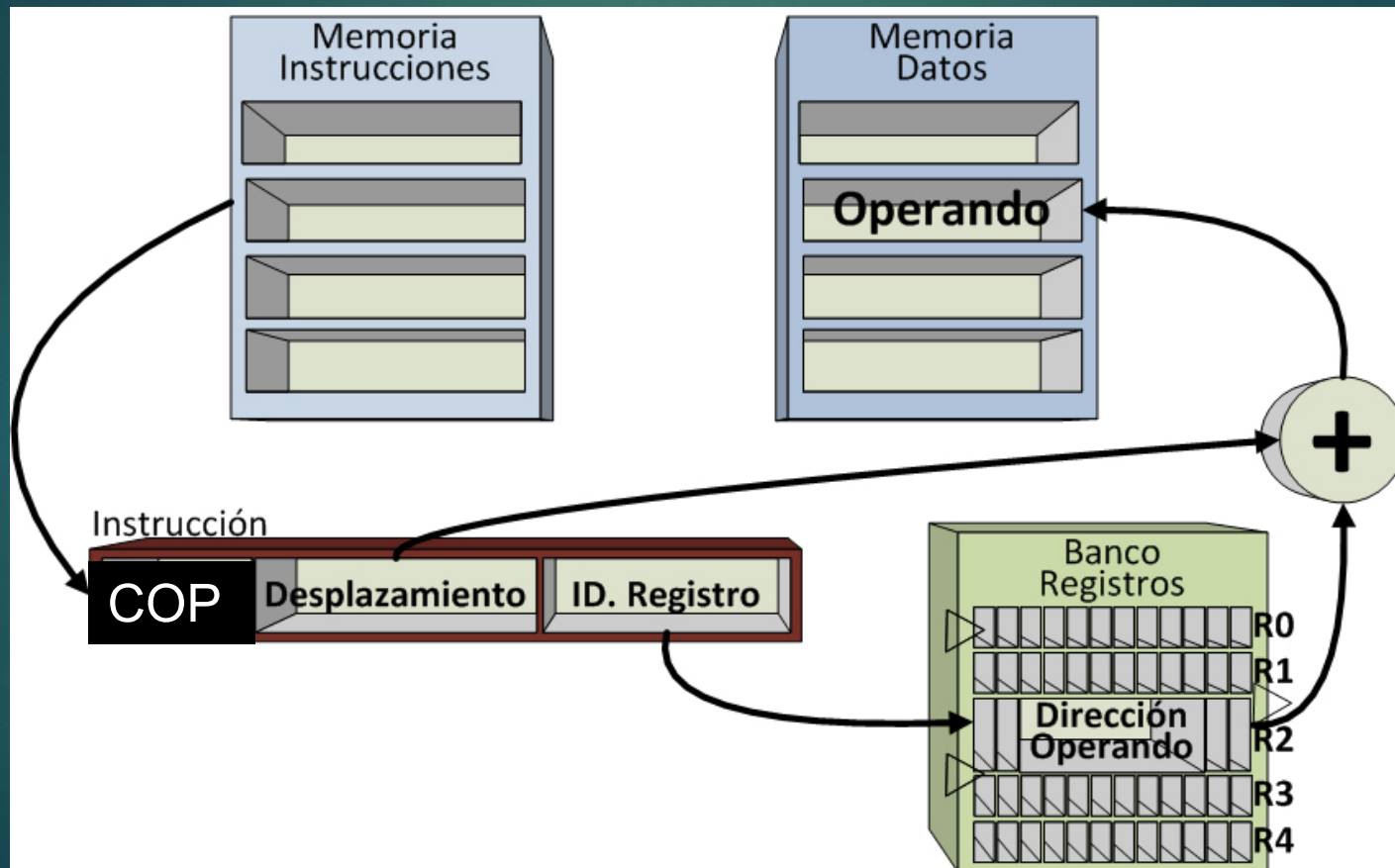
Modos de direccionamiento

➤ Esquema del modo de direccionamiento Directo a Registro



Modos de direccionamiento

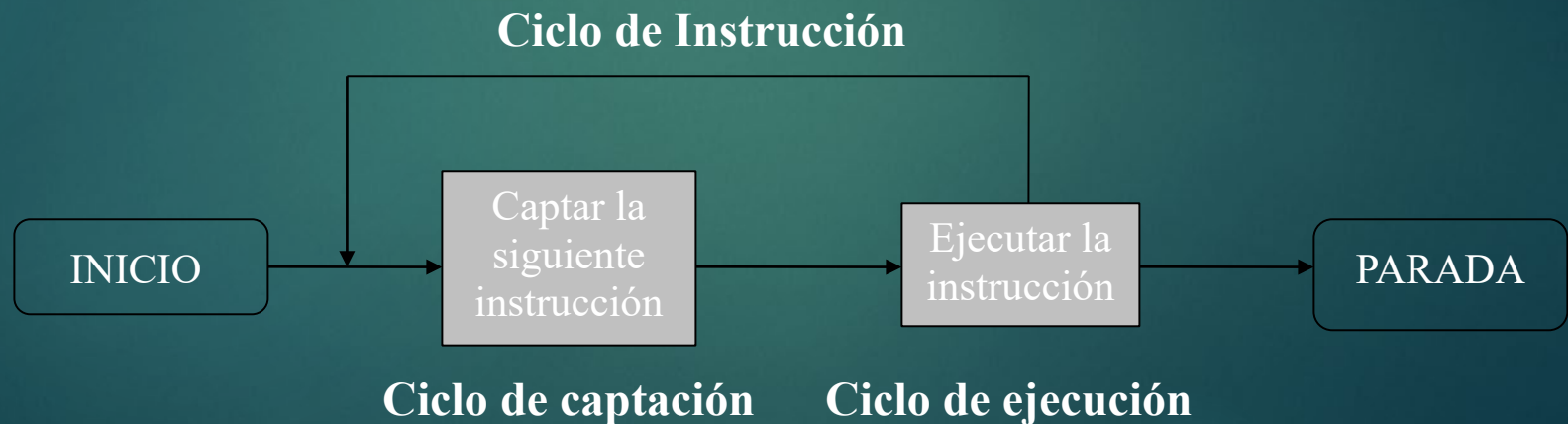
- Esquema del modo de direccionamiento Indirecto con desplazamiento (Base con desplazamiento)



Ciclo de instrucción básico

63

- El ciclo de instrucción es la secuencia de acciones que realiza el procesador para resolver las instrucciones.
- En su forma más sencilla, se puede descomponer en 2 acciones básicas:
 - Captación de la instrucción
 - Ejecución de la operación



Ciclo de instrucción básico

➤ Ciclo de captación de la instrucción

El ciclo de captación, o lectura de la instrucción, requiere las siguientes acciones:

- La dirección de la instrucción que se debe captar se carga en el registro Contador de Programa (PC).
- La UC envía al bus de direcciones el valor almacenado en el PC.
- La UC lee la instrucción desde la Memoria, y la almacena en un registro temporal denominado Registro de Instrucción (IR).
- El PC se incrementa en 1 (se prepara para apuntar a la próxima instrucción).
- La UC decodifica (“interpreta”) la instrucción captada.

Ciclo de instrucción básico

➤ Ciclo de ejecución de la instrucción

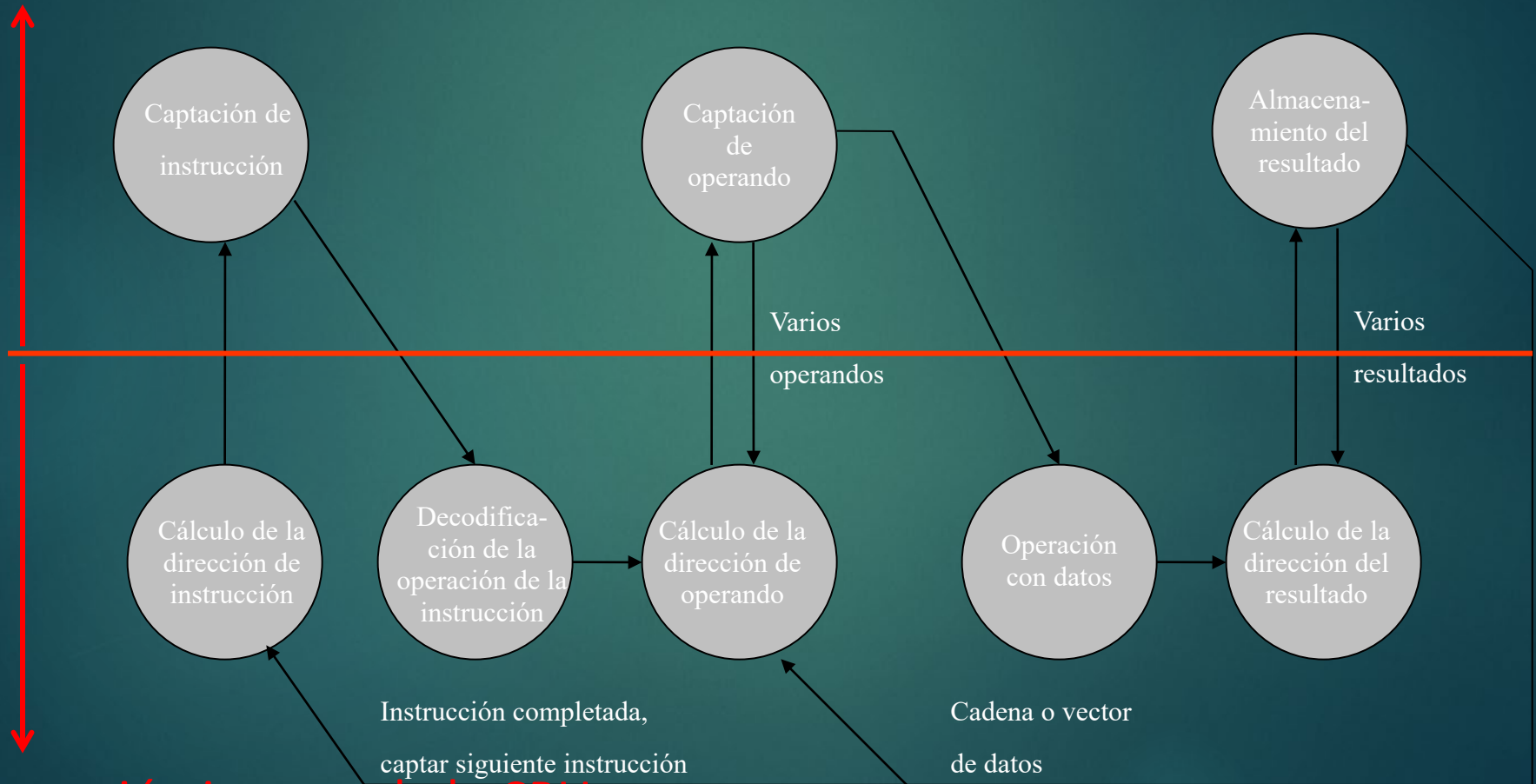
El ciclo de ejecución puede ser muy variado dependiendo del tipo de instrucción a ejecutar. Algunas de las posibles acciones en este ciclo son:

- Acciones procesador – memoria: por ejemplo transferencia de datos CPU - Memoria.
- Acciones procesador - E/S: por ejemplo, transferencias de datos entre la CPU y un dispositivo de E/S.
- Acciones de procesamiento de datos: operaciones aritméticas o lógicas con los datos.
- Acciones de control: por ejemplo alteración de la secuencia de ejecución (salto, salto a subrutina).
- Acciones combinadas de las anteriores

Ciclo de instrucción detallado

Modelo de ciclo de instrucción detallado

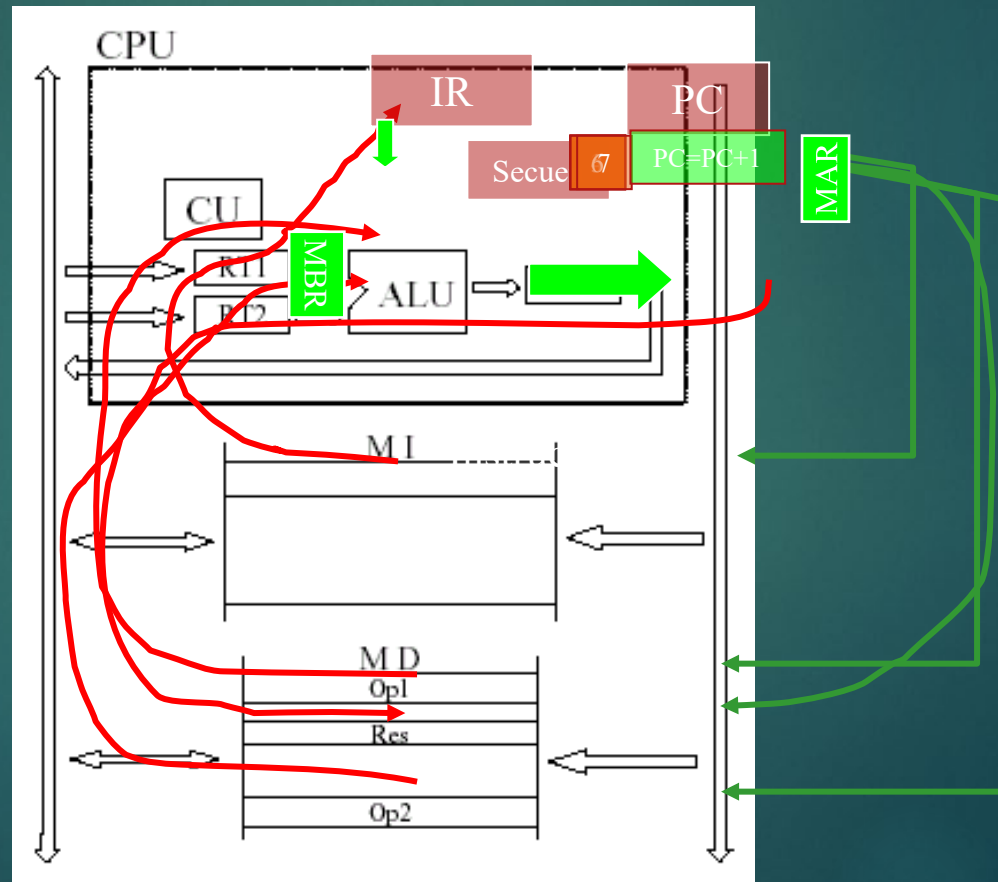
Acceso de CPU a memoria o E/S



Operación interna de la CPU

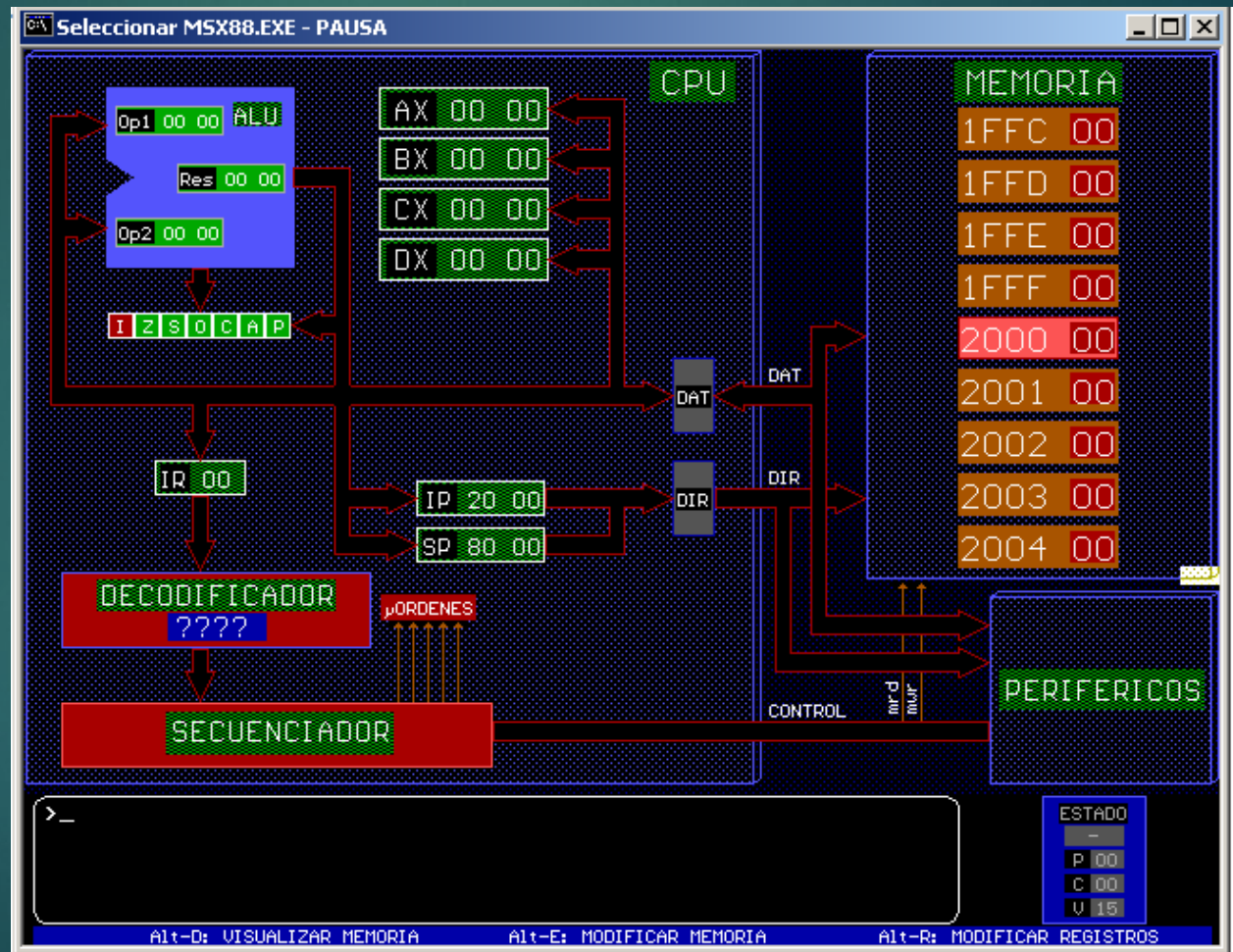
Análisis de la ejecución de una instrucción a nivel de hardware

La figura siguiente representa la secuencia de acciones de un procesador para resolver una instrucción de 2 operandos en memoria.



Simulador MSX88

Pantalla del simulador MSX88



Repertorio de instrucciones del MSX88

Instrucciones de movimiento de datos del MSX88

1	MOV <i>dest,fuente</i>	Copia <i>fuente</i> en <i>dest</i>	$(dest) \leftarrow (fuente)$
2	PUSH <i>fuente</i>	Carga <i>fuente</i> en el tope de la pila	$(SP) \leftarrow (SP) - 2; [SP+1:SP] \leftarrow (fuente)$
2	POP <i>dest</i>	Desapila el tope de la pila y lo carga en <i>dest</i>	$(fuente) \leftarrow [SP+1:SP]; (SP) \leftarrow (SP) + 2$
2	PUSHF	Apila los flags	$(SP) \leftarrow (SP) - 2; [SP+1:SP] \leftarrow (flags)$
2	POPF	Desapila los flags	$(flags) \leftarrow [SP+1:SP]; (SP) \leftarrow (SP) + 2$
3	IN <i>dest,fuente</i>	Carga el valor en el puerto <i>fuente</i> en <i>dest</i>	$(dest) \leftarrow (fuente)$
4	OUT <i>dest,fuente</i>	Carga en el puerto <i>dest</i> el valor en <i>fuente</i>	$(dest) \leftarrow (fuente)$

1. *dest/fuente* son: *reg/reg*, *reg/mem*, *reg/op.inm*, *mem/reg*, *mem/op.inm*.

mem puede ser una etiqueta (dir.directo) o [BX] (dir.indirecto).

2. *dest* y *fuente* solo pueden ser registros de 16 bits.

3. *dest/fuente* son: *AL/mem*, *AX/mem*, *AL/DX*, *AX/DX*.

4. *dest/fuente* son: *mem/AL*, *mem/AX*, *DX/AL*, *DX/AX*.

mem debe ser dirección entre 0 y 255. Puede ser un operando inmediato o una etiqueta.

Repertorio de instrucciones del MSX88

Instrucciones aritméticas y lógicas del MSX88

1	ADD <i>dest,fuente</i>	Suma <i>fuente</i> y <i>dest</i>	$(dest) \leftarrow (dest) + (fuente)$
1	ADC <i>dest,fuente</i>	Suma <i>fuente</i> , <i>dest</i> y flag <i>C</i>	$(dest) \leftarrow (dest) + (fuente) + C$
1	SUB <i>dest,fuente</i>	Resta <i>fuente</i> a <i>dest</i>	$(dest) \leftarrow (dest) - (fuente)$
1	SBB <i>dest,fuente</i>	Resta <i>fuente</i> y flag <i>C</i> a <i>dest</i>	$(dest) \leftarrow (dest) - (fuente) - C$
1	CMP <i>dest,fuente</i>	Compara <i>fuente</i> con <i>dest</i>	$(dest) - (fuente)$
5	NEG <i>dest</i>	Negativo de <i>dest</i>	$(dest) \leftarrow CA2(dest)$
5	INC <i>dest</i>	Incrementa <i>dest</i>	$(dest) \leftarrow (dest) + 1$
5	DEC <i>dest</i>	Decrementa <i>dest</i>	$(dest) \leftarrow (dest) - 1$
	AND <i>dest,fuente</i>	Operación <i>fuente</i> AND <i>dest</i> bit a bit	$(dest) \leftarrow (dest) \text{ AND } (fuente)$
1	OR <i>dest,fuente</i>	Operación <i>fuente</i> OR <i>dest</i> bit a bit	$(dest) \leftarrow (dest) \text{ OR } (fuente)$
1	XOR <i>dest,fuente</i>	Operación <i>fuente</i> XOR <i>dest</i> bit a bit	$(dest) \leftarrow (dest) \text{ XOR } (fuente)$
1	NOT <i>dest</i>	Complemento a 1 de <i>dest</i>	$(dest) \leftarrow CA1(dest)$

1. *dest/fuente* son: *reg/reg*, *reg/mem*, *reg/op.inm*, *mem/reg*, *mem/op.inm*.

5. *dest* solo puede ser *mem* o *reg*.

mem puede ser una etiqueta (dir.directo) o [BX], siendo (BX) una dirección de memoria (dir.indirecto).

Repertorio de instrucciones del MSX88

Instrucciones de control de programa del MSX88

6	CALL <i>etiqueta</i>	Llama a subrutina cuyo inicio es <i>etiqueta</i>	
6	RET	Retorna de la subrutina	
6	JZ <i>etiqueta</i>	Salta si el último valor calculado es cero	Si $Z=1$, $(IP) \leftarrow mem$
6	JNZ <i>etiqueta</i>	Salta si el último valor calculado no es cero	Si $Z=0$, $(IP) \leftarrow mem$
6	JS <i>etiqueta</i>	Salta si el último valor calculado es negativo	Si $S=1$, $(IP) \leftarrow mem$
6	JNS <i>etiqueta</i>	Salta si el último valor calculado no es negativo	Si $S=0$, $(IP) \leftarrow mem$
6	JC <i>etiqueta</i>	Salta si el último valor calculado produjo carry	Si $C=1$, $(IP) \leftarrow mem$
6	JNC <i>etiqueta</i>	Salta si el último valor calculado no produjo carry	Si $Z=1$, $(IP) \leftarrow mem$
6	JO <i>etiqueta</i>	Salta si el último valor calculado produjo overflow	Si $O=1$, $(IP) \leftarrow mem$
6	JNO <i>etiqueta</i>	Salta si el último valor calculado no produjo overflow	Si $O=0$, $(IP) \leftarrow mem$
6	JMP <i>etiqueta</i>	Salto incondicional a <i>etiqueta</i>	$(IP) \leftarrow mem$

6. *mem* es la dirección de memoria llamada *etiqueta*.

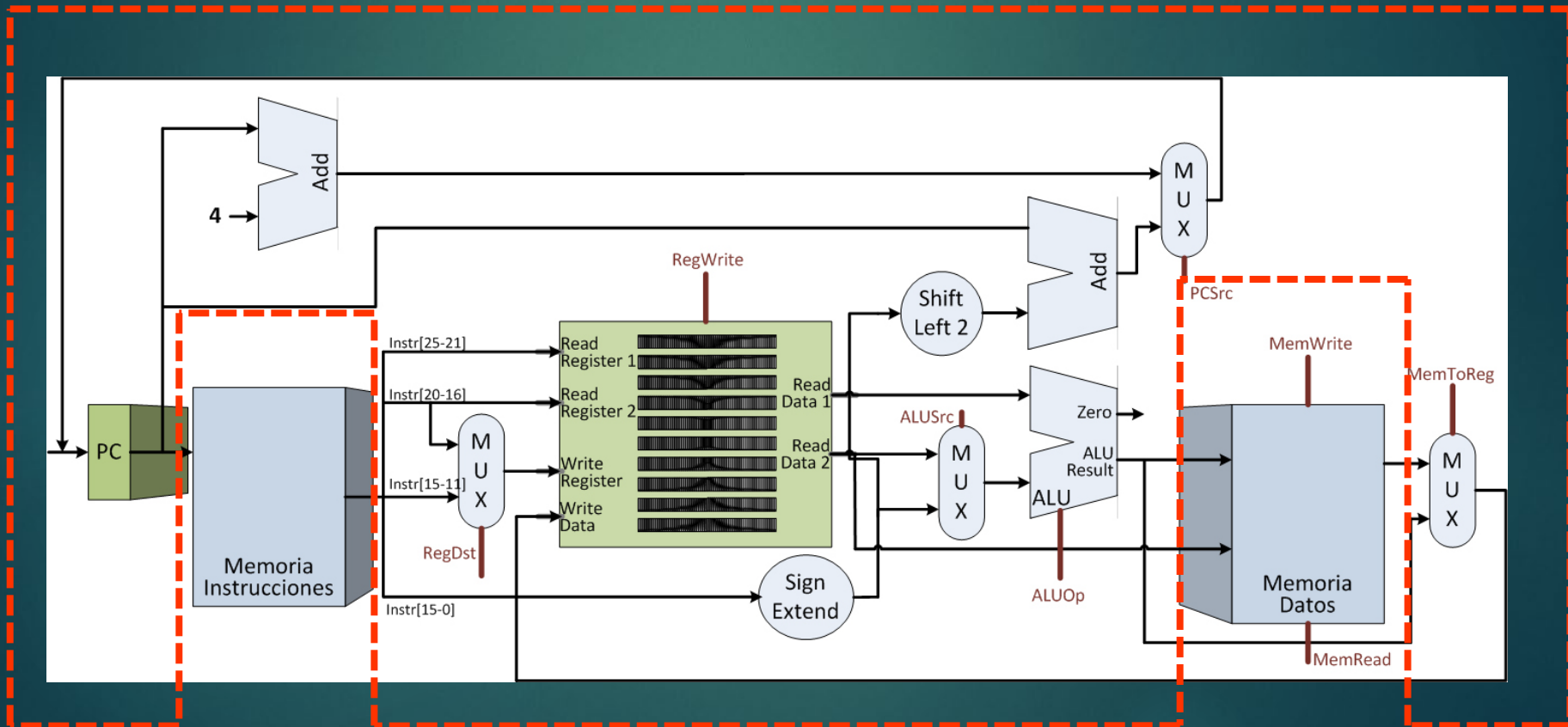
Modelo de Organización del nanoMIPS

- Para el análisis de algunos temas de las próximas clases, se va a emplear una máquina de características muy distintas de las usadas hasta ahora.
- La máquina se identifica con el nombre de nanomips.
- Es un modelo simplificado de una familia de máquinas tipo RISC de uso comercial ,conocidas como MIPS.
- En la siguiente figura se muestra, en forma simplificada, la sección del nanoMIPS encargada del flujo de los datos e instrucciones (es decir, la trayectoria de los datos).

Modelo de Organización del nanoMIPS

- Principales bloques funcionales encargados del flujo de los datos e instrucciones del nanoMIPS.

CPU

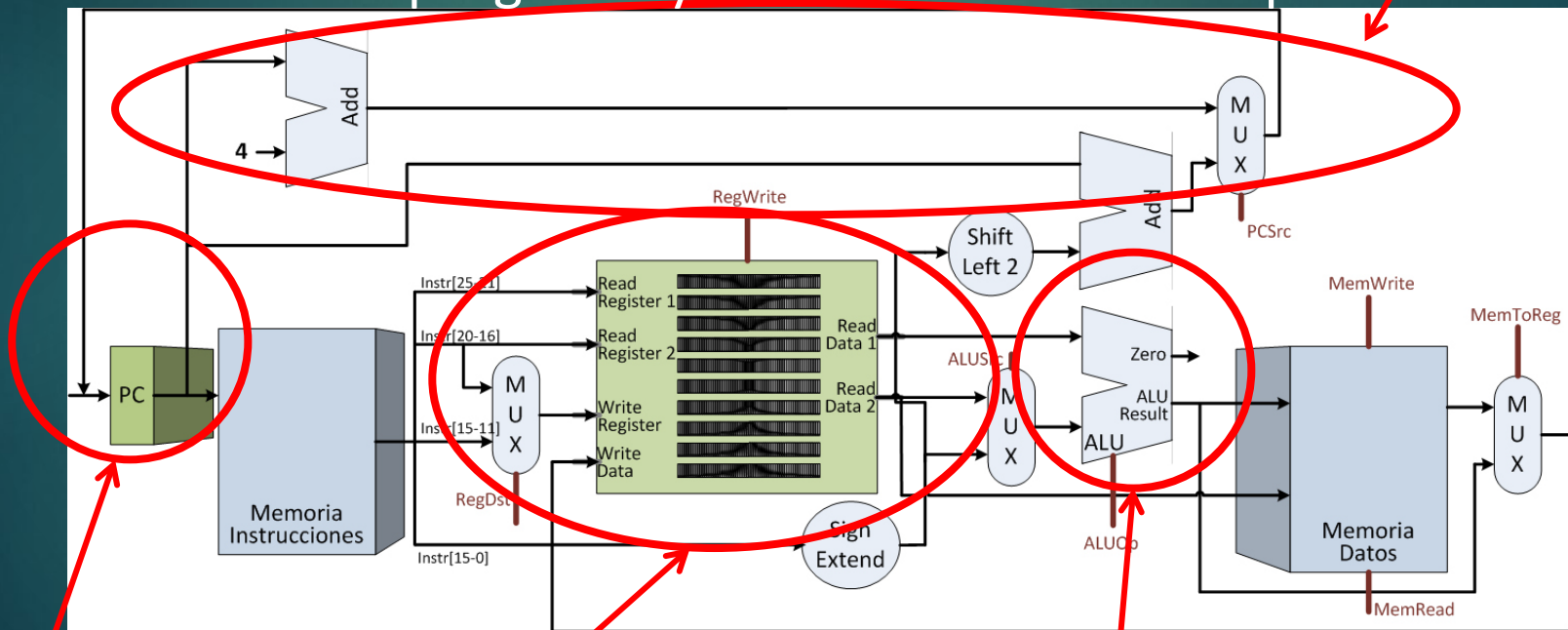


Modelo de Organización del nanoMIPS

➤ Las principales unidades del nanoMIPS son:

- Banco de registros
- ALU de datos
- Contador de programa y cálculo de la dir. de la próxima instruct.

CALCULO
PROXIMO VALOR
CONTADOR DE
PROGRAMA



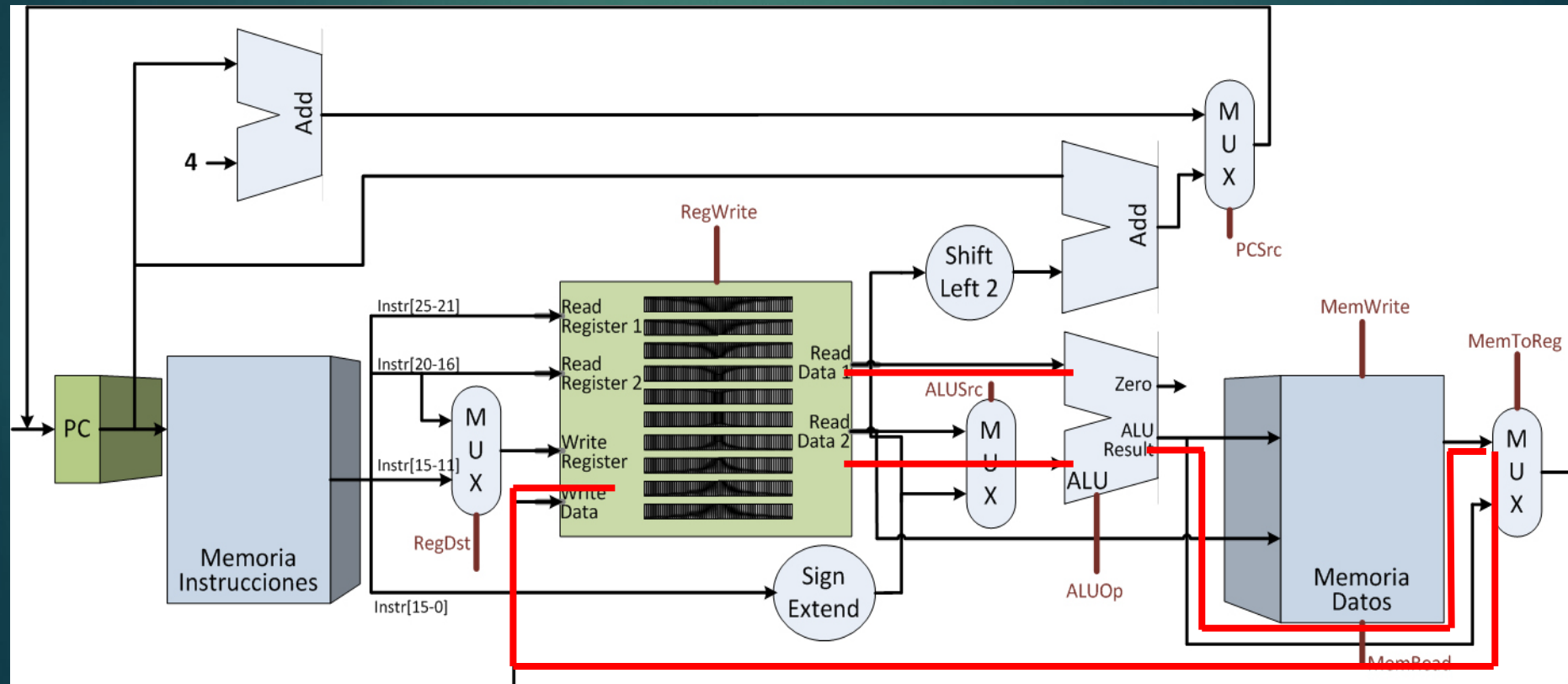
CONTADOR DE
PROGRAMA

REGISTROS

ALU PARA
DATOS

Modelo de Organización del nanoMIPS

- En la figura siguiente se muestra un ejemplo de la trayectoria de los datos para un operación entre 2 registros y el resultado guardado en otro registro.



Modelo de Organización del nanoMIPS

76

- En la figura anterior, se ha marcado con la línea roja, la trayectoria de los datos, desde que salen de los 2 registros hasta que entran al tercero (caso típico de una operación aritmética de 2 operandos y 1 resultado).
- El camino de los datos desde los 2 registros, pasando por la ALU, y almacenándose en un tercer registro, ha sido diseñada para optimizar el tiempo de ejecución.
- Esa operación puede resolverse en un solo ciclo de reloj.
- Notar que la máquina dispone de 2 unidades de memoria separadas: instrucciones y datos.
- Esta arquitectura de memorias separadas de instrucciones y datos se conoce como “Arquitectura tipo Harvard”.

PILA

77

- La Pila es una estructura ordenada de datos tipo LIFO (Last In- First Out).
- El acceso a la estructura se hace únicamente desde el tope (o cabeza) de la estructura.
- El acceso se hace mediante un registro específico denominado Puntero de Pila (SP, Stack Pointer).
- Sobre la Pila se pueden realizar 2 operaciones básicas, inversas entre sí:
 - Almacenar o “apilar”
 - Mnemónico en Assembly PUSH
 - Extraer o “desapilar”
 - mnemónico en Assembly POP

PILA

- Las operaciones de apilado y desapilado requieren 2 acciones que se ejecutan secuencialmente:
 - **1- Movimiento de datos:** que pueden ser del tipo
 - Registro a memoria
 - Memoria a registro
 - Memoria a memoria
 - **2- Operación del puntero:** antes y/o después del movimiento de datos (Stack Pointer – SP)
- Al haber movimiento de datos y de puntero durante el apilado o desapilado, hay que tener en cuenta:
 - Dónde apunta el puntero de pila SP
 - Cómo se mueve el SP, porque determina como “crece” la pila (que área de memoria va ocupando)

PILA

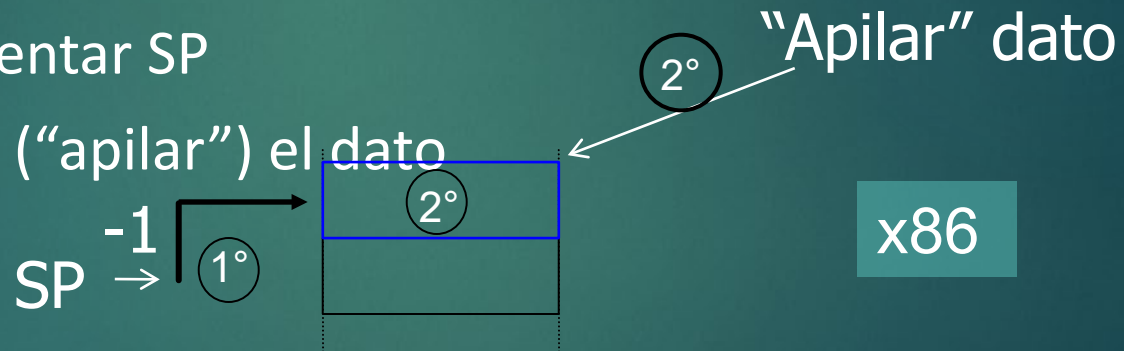
Manejo de la Pila en procesadores de la familia i80x86

En los procesadores de la familia Intel, la secuencia del movimiento del dato a la pila y la actualización del SP es la siguiente.

➤ PUSH:

1° -> decrementar SP

2° -> guardar ("apilar") el dato



x86

Es decir, primero se decrementa el SP, y luego se transfiere el dato al "tope" de la pila (lugar a donde "apunta" el SP).

Así, los datos se van almacenando en posiciones decrecientes de memoria. La pila "crece" hacia posiciones de memoria inferiores.

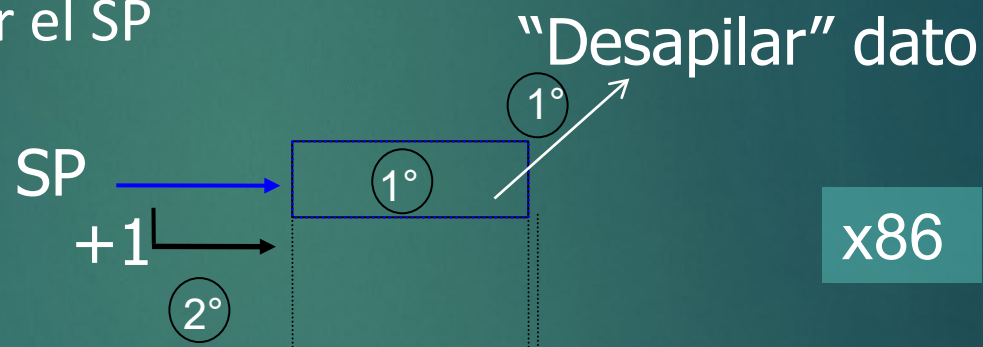
PILA

Manejo de la Pila en procesadores de la familia i80x86

➤ POP:

1° -> extraer (“desapilar”) el dato

2° -> incrementar el SP



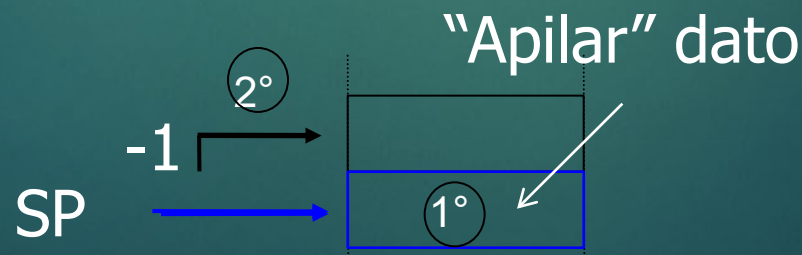
Es decir, primero se extrae el dato desde el “tope” de la pila (lugar a donde “apunta” el SP) , y luego se incrementa el SP. La operación de POP es exactamente inversa de la de PUSH.

El SP queda apuntando a la dirección donde está el último dato guardado (tope de la pila), el dato extraído permanece en la memoria pero no forma parte de la estructura de datos de la pila (copia no válida)

PILA

Manejo de la Pila en procesadores de la familia M68xxx

- Algunos procesadores administran la pila de una manera distinta.
- Por ejemplo, en los procesadores de la familia Motorola, la secuencia del movimiento del dato a la pila y la actualización del SP es la siguiente.
- PUSH:
 - 1° -> guardar (“apilar”) el dato
 - 2° -> decrementar SP



68xxx

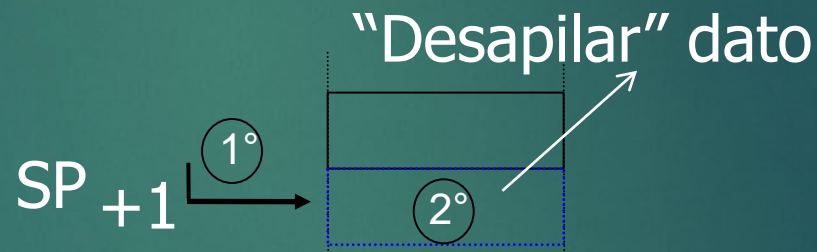
PILA

Manejo de la Pila en procesadores de la familia M68xxx

➤ POP:

1° -> incrementar el SP

2° -> extraer (“desapilar”) el dato



68xxx

Igual que antes, los datos se van almacenando en posiciones decrecientes de memoria. Es decir, la pila “crece” hacia posiciones de memoria inferiores.

Además el PUSH y el POP son exactamente inversas entre sí.

Pero ahora el SP apunta siempre a la primer posición libre, por encima del tope de la pila.

Instrucciones de uso de la PILA

Instrucciones para la PILA en Lenguaje Assembly

- Para operar con la Pila, los procesadores disponen de instrucciones de carga (PUSH) y descarga de datos (POP).
- En el ejemplo siguiente se muestra un programa donde se usa la Pila para intercambiar datos entre los registros AX y CX usando las instrucciones de PUSH y POP.
- Notar que después de ejecutado el programa, la Pila queda exactamente como estaba previamente. Es decir, el SP va a quedar con el mismo valor que tenía cuando comenzó el programa.

Instrucciones de uso de la PILA

Ejemplo de uso de instrucciones para la PILA en Lenguaje Assembly

```
ORG 2000H
MOV BX, 3000H
MOV AX, [BX]
MOV BX, 3002H
MOV CX, [BX]
PUSH AX
PUSH CX
POP AX
POP CX
HLT

ORG 3000H
datos DB 55h, 33h, 44h, 22h
END
```


Instrucciones de uso de la PILA

- En el ejemplo anterior se ejecutan en forma consecutiva 2 instrucciones PUSH, y a continuación 2 instrucciones POP.
- En el PUSH AX, el SP se decrementa en 2 y la CPU almacena AX (3355H) en el tope de la pila (el dato de 16 bits se almacena en 2 palabras consecutivas de la pila).
- Idem con PUSH CX (2244H).
- En el POP AX, se carga AX desde el tope de la PILA, que contenía 2244H y el SP se incrementa en 2.
- Idem con POP CX (3355H).
- Al final del programa, el SP contiene el valor que tenía antes de la ejecución del programa.

Instrucciones de uso de la PILA

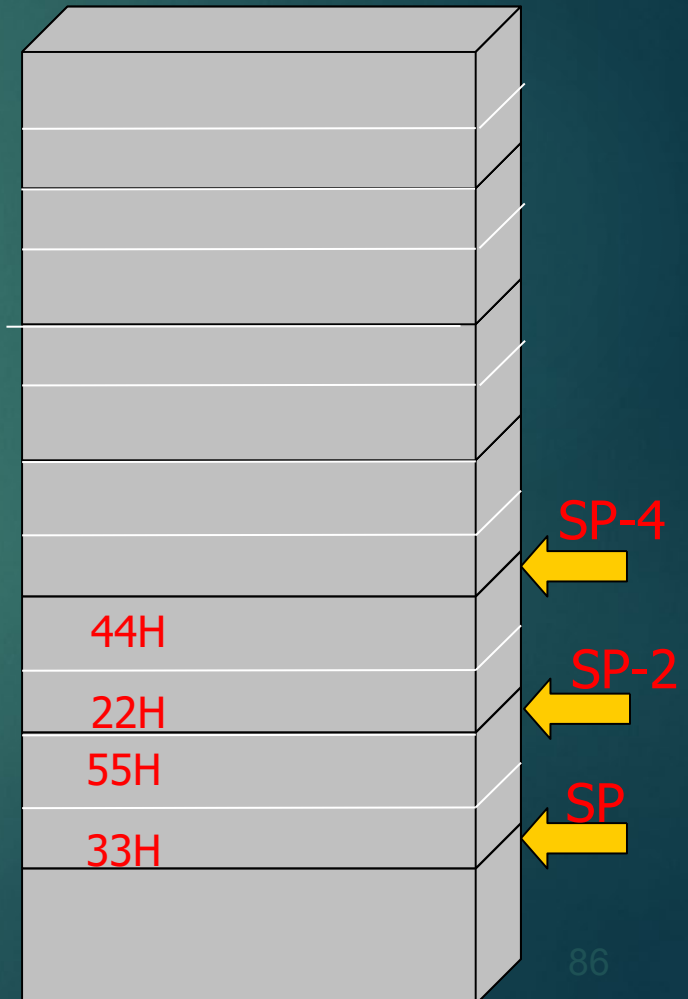
Movimiento de datos y SP durante la ejecución del programa

Ejemplo:

	ORG	2000H
	MOV	BX, 3000H
	MOV	AX, [BX]
	MOV	BX, 3002H
	MOV	CX, [BX]
	PUSH	AX
	PUSH	CX
	POP	AX
	POP	CX
	HLT	
	ORG	3000H
datos	DB	55h, 33h, 44h, 22h
	END	



MEMORIA



Subrutinas

➤ DEFINICIÓN

Una subrutina es una sección de código, que recibe el control en un punto de entrada, y lo devuelve en un punto de salida.

➤ OBJETIVO

El objetivo de la subrutina es realizar una tarea definida, para lo cual se le transfiere el control. Una vez finalizada la tarea lo devuelve al programa que la invocó en el punto donde la invocó.

Subrutinas

➤ VENTAJAS

El uso de subrutina tiene muchas ventajas. Entre otras:

- **Economía de programa:** el código puede ser usado varias veces
- **Modularidad:** se puede subdividir el programa en unidades pequeñas, más fácilmente verificables.

➤ OBSERVACIONES

Para realizar su trabajo, se requieren pasar parámetros entre el programa que invoca y la subrutina invocada.

Subrutinas

➤ Invocación

Una subrutina se invoca mediante una instrucción específica, disponible en el repertorio de instrucciones del procesador.

Por ejemplo, un mnemónico típico usado para invocar una subrutina es la instrucción: CALL

➤ Terminación

La subrutina termina mediante una instrucción específica de retorno. Por ejemplo: RET

➤ Ejemplo

A continuación se muestra una típica sección de código Assembly con un programa (principal) invocando una subrutina (también llamado Procedimiento).

Subrutinas

En Programa principal:

...

CALL Nombre (del procedimiento)

Dir. Ret

Prox. Instrucción

...

En Procedimiento:

Nombre Proc

...

...

RET

Nombre Endp



Cuerpo del procedimiento

Subrutinas

91

- En el ejemplo anterior, el programa principal invoca la subrutina en la instrucción `CALL Nombre` (del procedimiento).
- Al invocarla, le transfiere el control. Es decir, el PC se carga con la dirección de comienzo de la subrutina, y la CPU comienza a ejecutar las instrucciones de la subrutina.
- Cuando la subrutina complete su tarea, la última instrucción que ejecuta es la de `RET`.
- En ese momento el PC se carga con la dirección de la instrucción siguiente al `CALL` en el programa principal. De esa manera el programa principal continúa con su tarea en el punto siguiente a la invocación de la subrutina.

Subrutinas

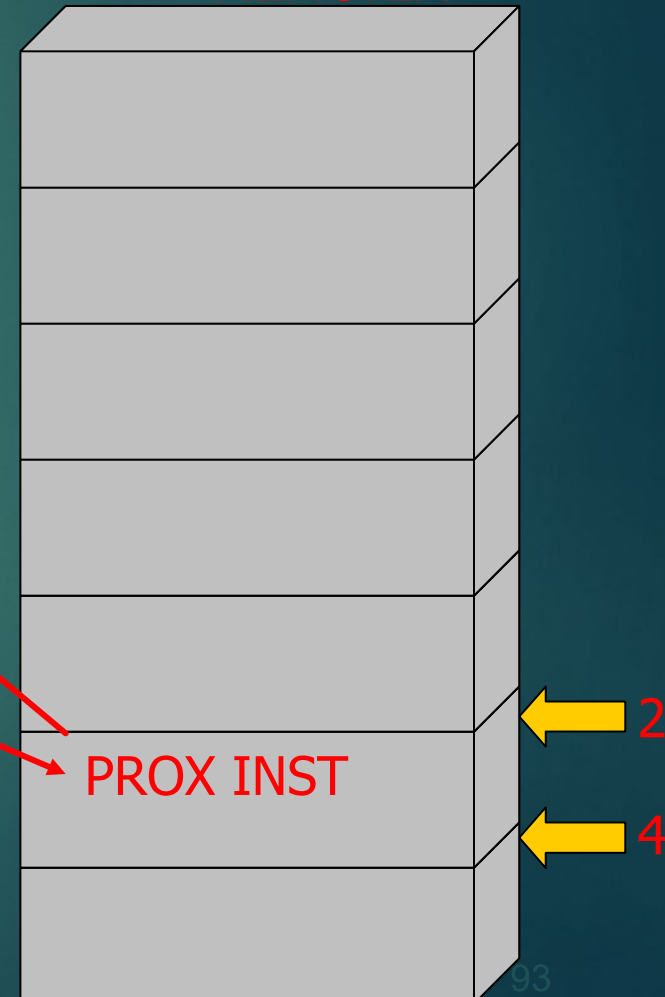
92

- Para poder recuperar la dirección de la instrucción siguiente al CALL en el programa principal, esta dirección debe haber sido guardada previamente.
- El lugar donde se guarda esta dirección es la Pila. Cada vez que se ejecuta una instrucción de CALL, se guarda en la Pila el valor actual del PC.
- Cuando se ejecuta una instrucción RET, el PC se carga con la dirección almacenada en el tope de la Pila.
- Mediante este mecanismo, al terminar la subrutina, se restablece en el PC la dirección de la instrucción siguiente al CALL.
- Este mecanismo se muestra en la figura siguiente, que corresponde al programa del ejemplo previo.

Subrutinas

Movimiento de la Pila durante la ejecución del programa

MEMORIA



Ejemplo:

ORG 2000H

MOV AX, 0001H

CALL SUB1 ← 1

PROX INST **MOV BX, 0002H**

...

...

ORG 3000H

SUB1

PROC ← 3

...

RET ← 4

SUB1

ENDP

A modo de ejemplo, se necesita analizar el comportamiento de la Pila y los registros para el siguiente ejercicio.

```
SUB1:    ORG 1000H
         NEG     AX
         RET     ←
         ←
         ORG 2000H
         MOV     BX, 0
         MOV     AX, dato
         PUSH    AX
         CALL    SUB1 ←
         POP     BX
         HLT
         ORG 3000H
dato:    DB      55H,00H
         END
```

Subrutinas

Análisis del ejemplo anterior

- El punto de entrada del programa principal es 2000H.
- La instrucción PUSH AX apila el registro AX.
- A continuación se invoca la subrutina SUB1. Antes de transferir el control a SUB1 se apila la dirección de la instrucción siguiente (es decir, la de POP BX).
- Se transfiere el control a la subrutina SUB1.
- La subrutina SUB1 termina en la instrucción RET, que desapila el valor almacenado en el tope de la pila, con el que se carga el PC. El nuevo valor del PC es el que corresponde a la dirección de memoria donde está la instrucción POP BX.
- La instrucción POP BX carga en BX el dato desde el tope de la pila.

Subrutinas

Movimiento de la Pila del ejemplo anterior

El programa ensamblado queda:

1000H NEG AX

1001H RET

...

...

2000H MOV BX, 0

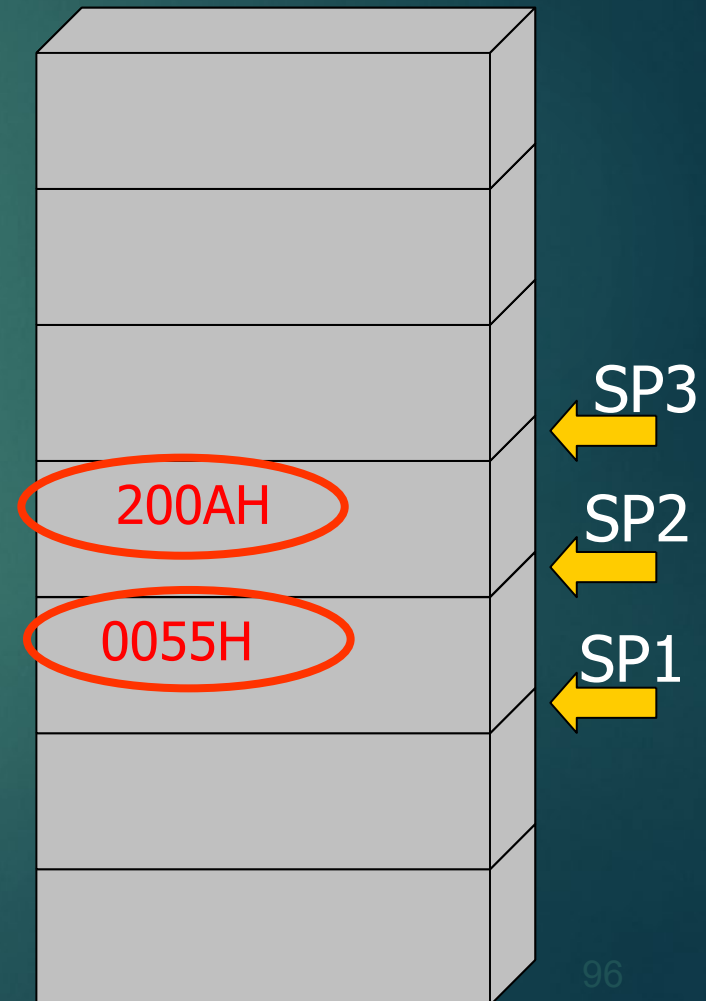
2003H MOV AX, 0055H

2006H PUSH AX

2007H CALL 1000H

200AH POP BX

200BH HLT



Detalles de uso de subrutinas

- Para que una subrutina realice una tarea, requiere de datos.
- Los datos (argumentos o parámetros) se los proporciona el programa que invoca a la subrutina.
- A su vez, la subrutina produce 1 o más resultados. Los resultados deben ser devueltos al programa que invocó la subrutina.
- Existen 3 métodos para pasar parámetros a subrutinas.

Detalles de uso de subrutinas

➤ 1.- Vía registros

- Los parámetros se pasan a través de los registros de la CPU.
- Es un método sencillo, pero limitado por el número de registros disponibles.
- Dado que se van a modificar los contenidos de los registros, es importante documentar los registros a usar.

➤ 2.- Vía memoria

- Los parámetros se transfieren a través de un área definida de memoria (RAM).
- Difícil de estandarizar, debido a las dificultades en asignar un área de memoria.

Detalles de uso de subrutinas

➤ 3.- Vía pila (stack)

- Los datos se pasan a través de la Pila.
- Es el método más ampliamente usado porque presenta varias ventajas respecto de los otros métodos.
- La principal ventaja es que es independiente de la memoria y registros.
- Los registros no tienen que ser modificados en las subrutinas.
- Pero hay que manejar correctamente, porque la Pila va a ser usada por el usuario (en la invocación y en la subrutina) y por el sistema.
- Recordar que en el x86, el SP apunta al último lugar usado (último dato entrado)

Detalles de uso de subrutinas

Programa principal con llamada a Subrutina

- Para invocar una subrutina con pasaje de parámetros vía Pila, en el programa que invoca se hace lo siguiente:

...



Push Parámetro 1

Push Parámetro 2

CALL SUB1 (del procedimiento)

DirRet Prox. Instrucción

Detalles de uso de subrutinas

Programa principal con llamada a Subrutina

- Previo a la invocación de la subrutina (es decir previo al CALL) se deben apilar los parámetros requeridos por la subrutina (en el ejemplo, se apilan 2 parámetros).
- Tener en cuenta que las 2 instrucciones de Push modifican el SP. El programa principal deberá restablecer a partir de Prox. Instrucción (luego del retorno de la subrutina) el valor correcto del SP (por ejemplo con 2 instrucciones POP).
- El movimiento de los datos y la Pila, se muestran en la filmina siguiente.

Detalles de uso de subrutinas

Movimiento de la Pila durante la invocación a la subrutina

Procedimiento

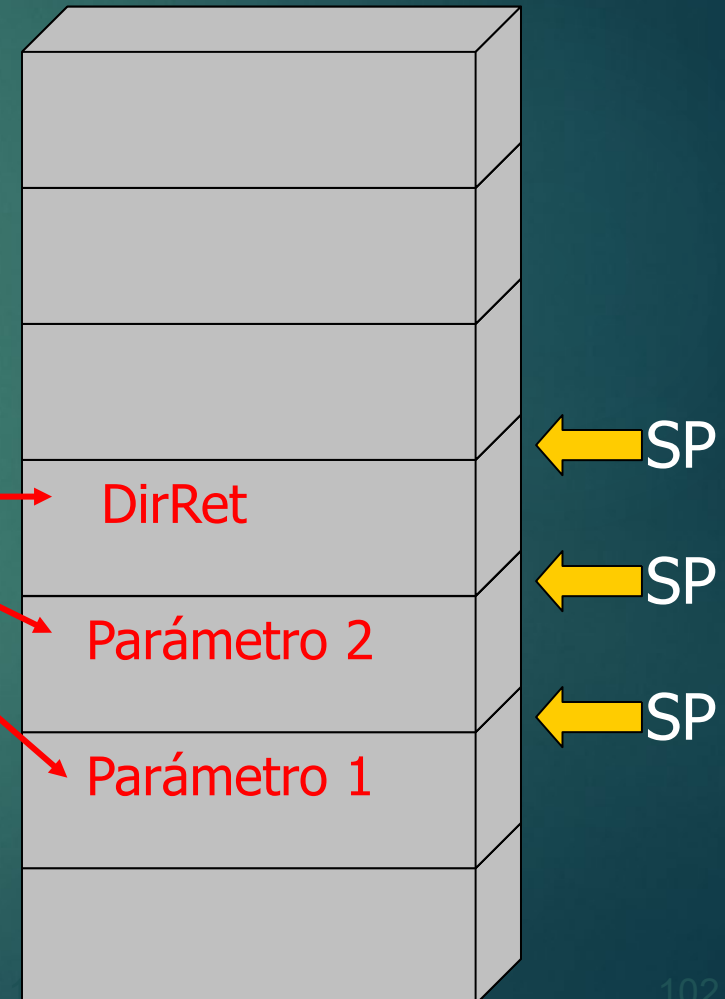
.....

Push Parametro 1

Push Parámetro 2

Call SUB1

DirRet Prox. Inst.



Detalles de uso de subrutinas

Estructura de la subrutina

- Para la descripción que sigue se va a usar como referencia el procesador (80x86).
- Hay que tener en cuenta que el 80x86 no dispone de un modo de direccionamiento relativo al SP (direccionamiento Base). Algunas operaciones deben hacerse debido a esta “limitación” del procesador.
- Para el acceso a la estructura de datos de la Pila se requiere un registro base (dado que no se puede usar el SP), típicamente el BP.
- La subrutina no debería modificar los contenidos de los registros al momento de su invocación. Si la subrutina usa un registro lo debería salvaguardar al comienzo, y restaurarlo al final.

Detalles de uso de subrutinas

Estructura de la subrutina

- La Subrutina requiere de una serie de acciones de manera que quede correctamente estructurada.
- La parte de la subrutina que ejecuta la acción específica se llama cuerpo de la subrutina.
- Previo al cuerpo de la subrutina, se requiere una serie de acciones de “preparación”.
- Posterior al cuerpo de la subrutina, se deben ejecutar otras acciones más para restablecer y adecuar la subrutina, y devolver el control al programa que la invocó en las condiciones correctas.

Detalles de uso de subrutinas

Estructura de la subrutina

1. Salvar el estado de BP (viejo BP)
2. Apuntar con BP al tope de la pila (BP=SP)
3. Reservar espacio para datos locales (opcional)
4. Salvar valores de otros registros (opcional)
5. Acceder a parámetros
6. Escribir sentencias a ejecutar ← cuerpo
7. Restaurar los registros modificados (opcional)
8. Restaurar el valor original de SP y BP
9. Retornar parámetro (opcional)
10. Regresar (correctamente del procedimiento)

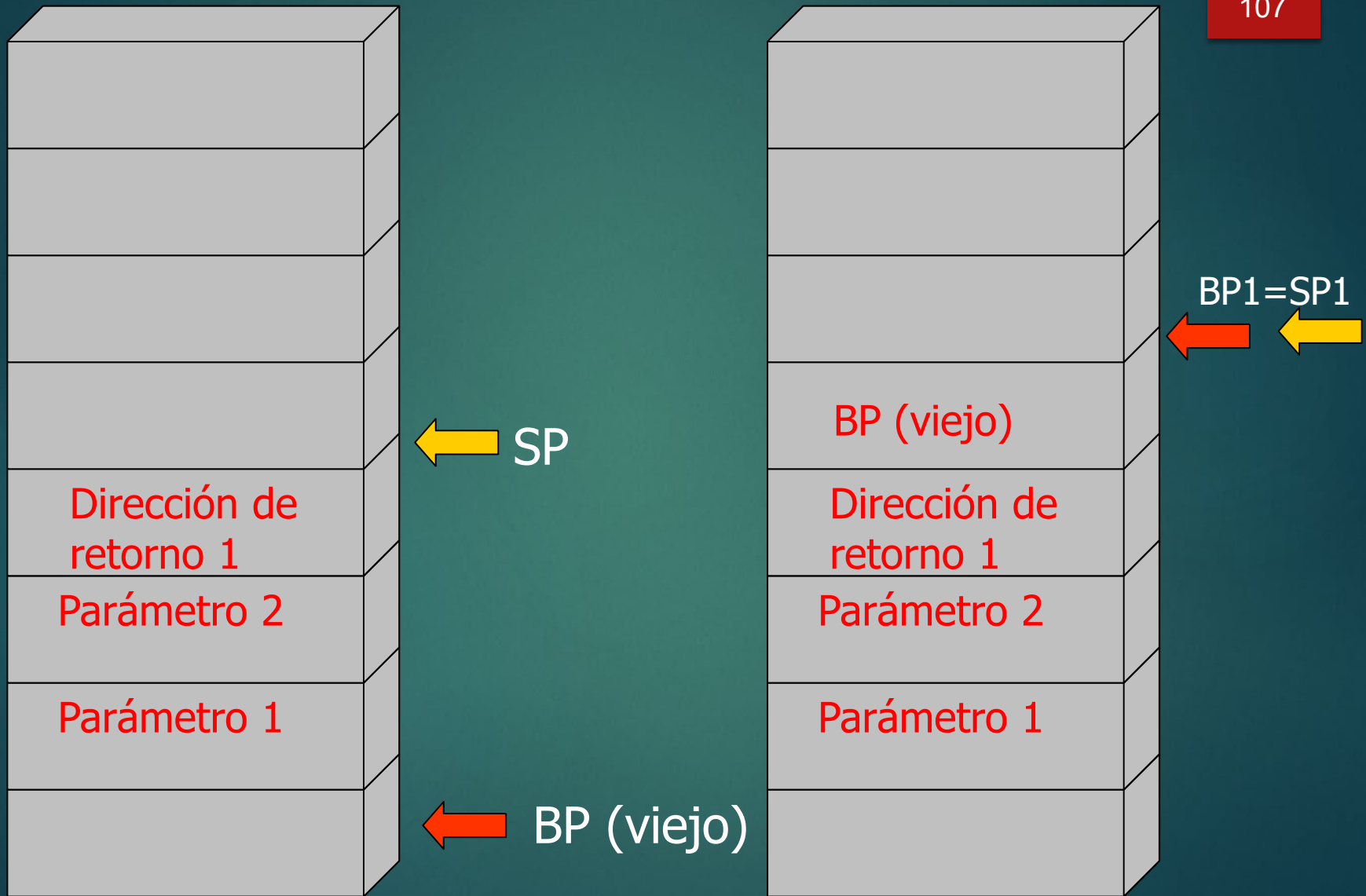
Detalles de uso de subrutinas

Estructura de la subrutina

- | | |
|--------------------------|-------------------|
| 1. Salvar el BP (viejo): | <i>push BP</i> |
| 2. Inicializa BP: | <i>mov BP, SP</i> |

- Tal como se mencionó anteriormente, se usará el puntero base BP para el acceso a la estructura de datos que se formará en la Pila.
- Como se modificará el BP, siguiendo el concepto de que los registros no deben ser alterados al finalizar la subrutina, la primer instrucción tiene como objetivo salvaguardar el valor actual del BP.
- Una vez hecho esto, la segunda instrucción inicializa el puntero BP para que apunte a la estructura de datos que se formará en la Pila.

Detalles de uso de subrutinas



SP y BP antes de invocar SUBR 1

SP y BP después del paso 2 en SUBR1

Detalles de uso de subrutinas

108

- BP es el puntero al área de la pila asignada al procedimiento (normalmente llamado "Frame pointer"). Para acceder a los datos en el área de la Pila asignada al procedimiento se deberá sumar un desplazamiento fijo al BP.
- De esta manera se establece a BP como puntero de referencia, y es usado para acceder a los parámetros y datos locales en la pila.
- SP no puede ser usado para éste propósito porque no es un registro base ó índice(es decir, no hay un direccionamiento base o indexado que use el SP). El valor de SP puede cambiar, pero típicamente BP permanece 'quieto'.

Detalles de uso de subrutinas

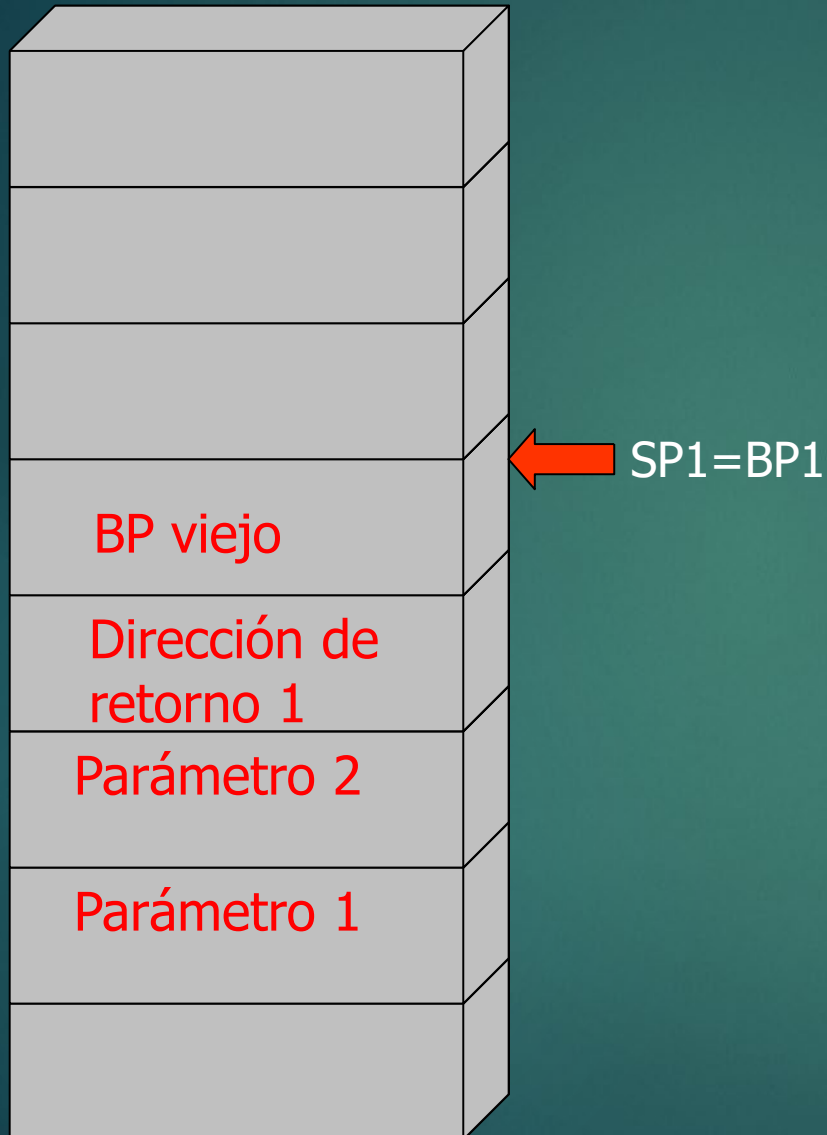
Estructura de la subrutina

*3. Reserva espacio para
variables locales*

sub SP,2

- El tercer paso en la subrutina consiste en reservar espacio para almacenar variables locales (variables temporales usadas por la subrutina y que al finalizar se deben descartar).
- Eso se puede hacer, por ejemplo moviendo el SP una cantidad determinada de bytes.
- En el ejemplo anterior, se reservan 2 bytes para variables locales, decrementando el SP en 2.
- El sistema puede utilizar al SP sin escribir sobre el área de trabajo (o frame) del procedimiento.

Detalles de uso de subrutinas



Pila y punteros después del paso 2



Pila y punteros después del paso 3

Detalles de uso de subrutinas

Estructura de la subrutina

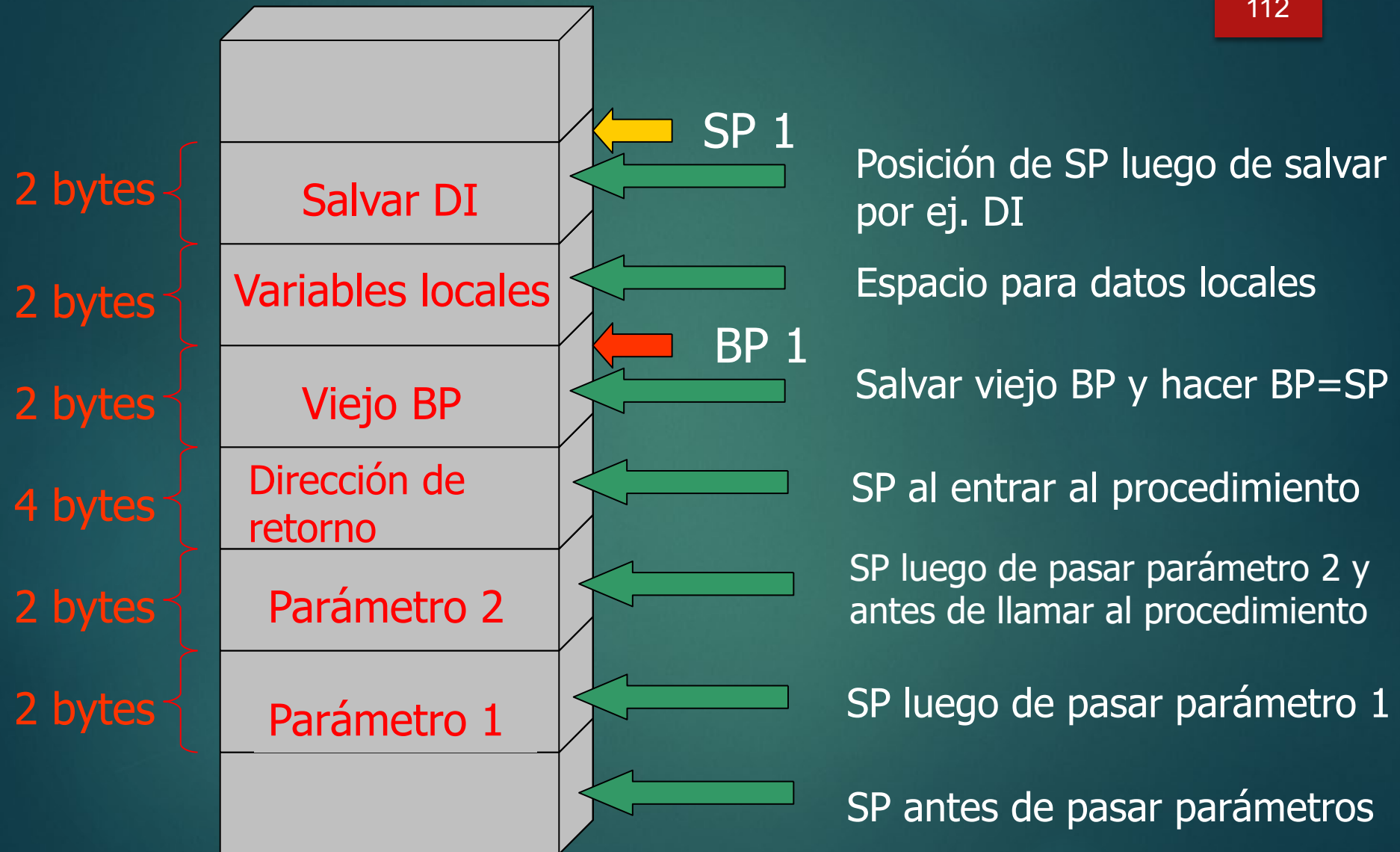
4. Salva otros registros

usados en la subrutina

push DI

- En el caso que la subrutina necesite usar registros de la CPU, es aconsejable salvarlos antes, para restablecerlos previo a retornar de la subrutina.
- En el paso 4 se salva el ó los registros que sean necesarios. En este ejemplo se salva el registro DI.
- No necesitan salvarse los registros que no son modificados.
- Normalmente los registros son salvados después de establecer el puntero (frame pointer) y reservar el área para los datos locales.

Detalles de uso de subrutinas



Pila y punteros después del paso 4

Detalles de uso de subrutinas

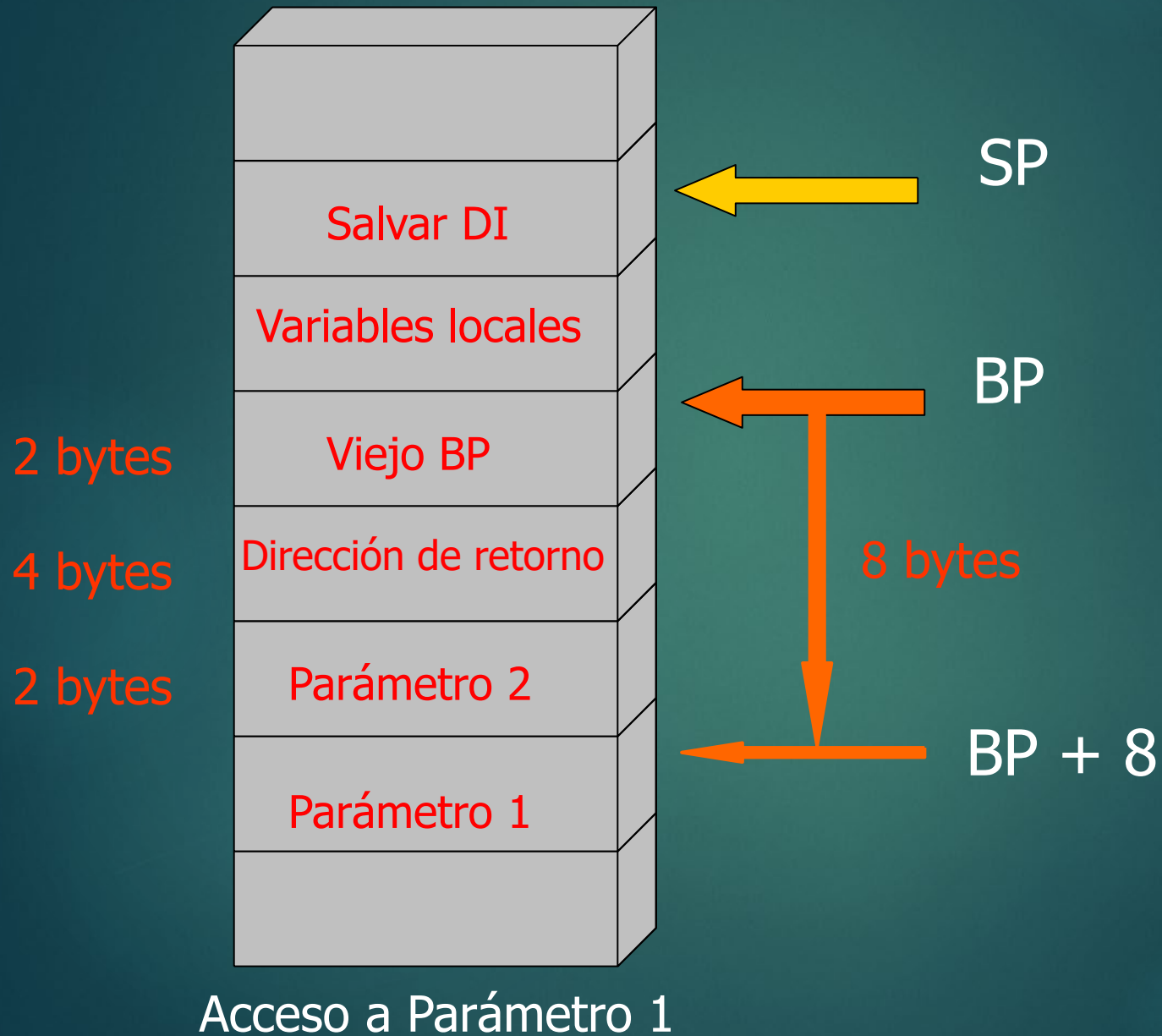
Estructura de la subrutina

5. Acceder a un parámetro

`mov CX, [ BP + 8 ]`

- En general el acceso a un parámetro se hace sumando un desplazamiento a BP (BP no se modifica), correspondiente a un direccionamiento base con offset.
- El offset que se suma al BP se obtiene sumando todos los bytes intercalados entre la posición actual del BP y la posición del parámetro a buscar.
- Por ejemplo, para acceder al Parámetro 1, el offset es de:
 - $4+2+2=8$ bytesy el desplazamiento es BP+8.

Detalles de uso de subrutinas



Detalles de uso de subrutinas

115

Estructura de la subrutina

6. Cuerpo de la subrutina

....

....

- En el paso 6 se escriben las instrucciones propias de la tarea que desarrolla la subrutina, y que constituyen el cuerpo del procedimiento.

Detalles de uso de subrutinas

Estructura de la subrutina

7. Restaurar los registros modificados *pop DI*

- En el paso 7 se restauran los valores originales en los registros usados durante la subrutina.
- En el ejemplo, solo se usó el DI (que fué salvado en el paso 4), por lo que solo ese registro es recuperado.

Detalles de uso de subrutinas

Estructura de la subrutina

8. Restaurar SP y BP

mov SP, BP

pop BP

- En el paso 8 se restauran los valores originales de SP y BP.
- La primer instrucción (*mov SP, BP*) restablece el valor de SP que pudo haberse movido cuando se reservó espacio para las variables locales (paso 3). Notar que el BP no se movió durante el acceso a la estructura de datos en la Pila.
- La segunda instrucción (*pop BP*), restaura el valor original (el valor “viejo”) que tenía el BP antes de ingresar a la subrutina.

Detalles de uso de subrutinas

Estructura de la subrutina

9. *Retornar parámetros*

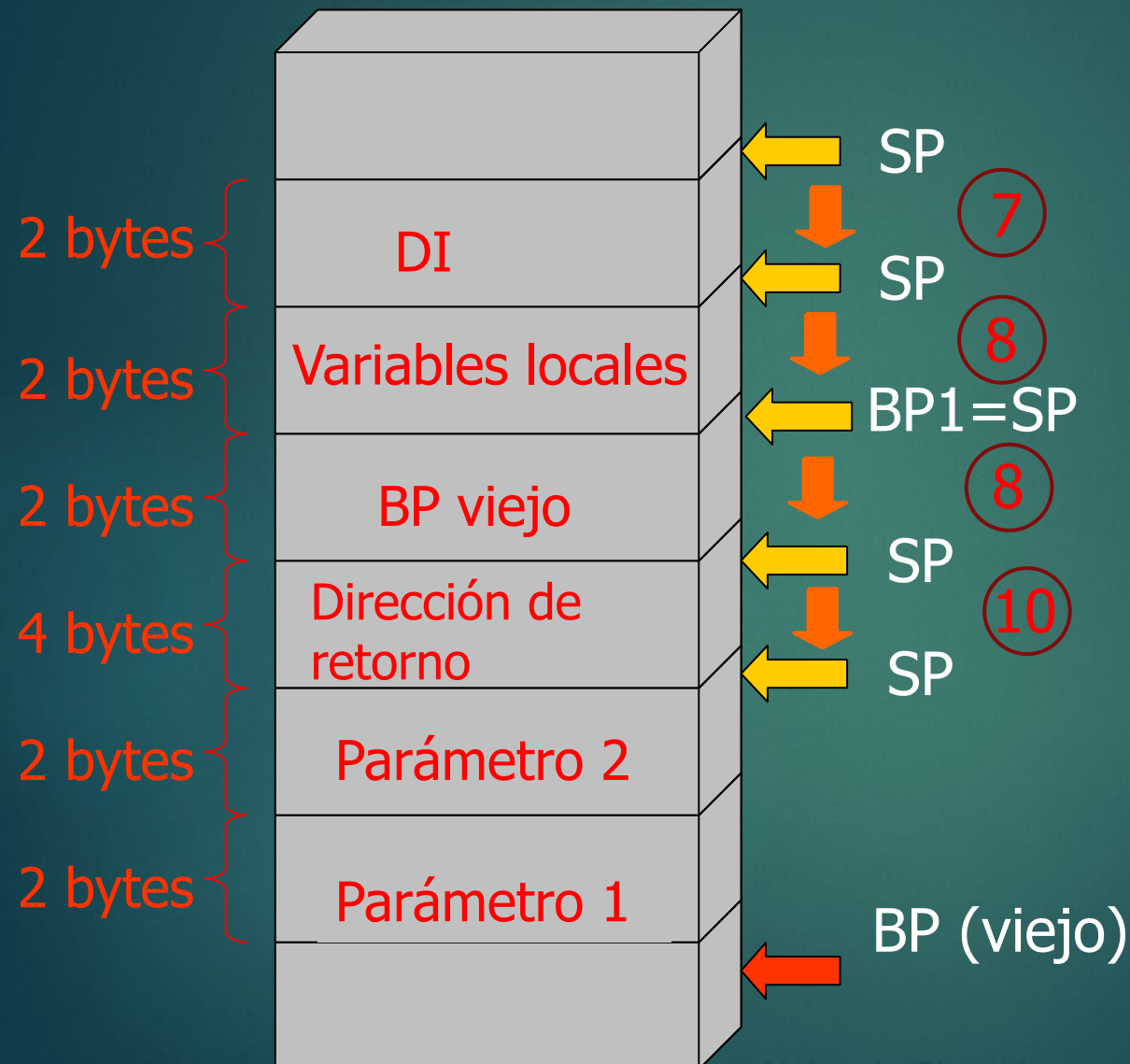
....

- En el paso 9 se actualizan los resultados obtenidos por la subrutina a ser devueltos al programa que la invoca.

10.- *Retornar* *ret*

- El ultimo paso de la subrutina es la instrucción de retorno de subrutina Ret. De esta manera se le devuelve el control al programa principal (que invoca la subrutina) en la instrucción siguiente a la instrucción que invoca a la subrutina (CALL).

Detalles de uso de subrutinas



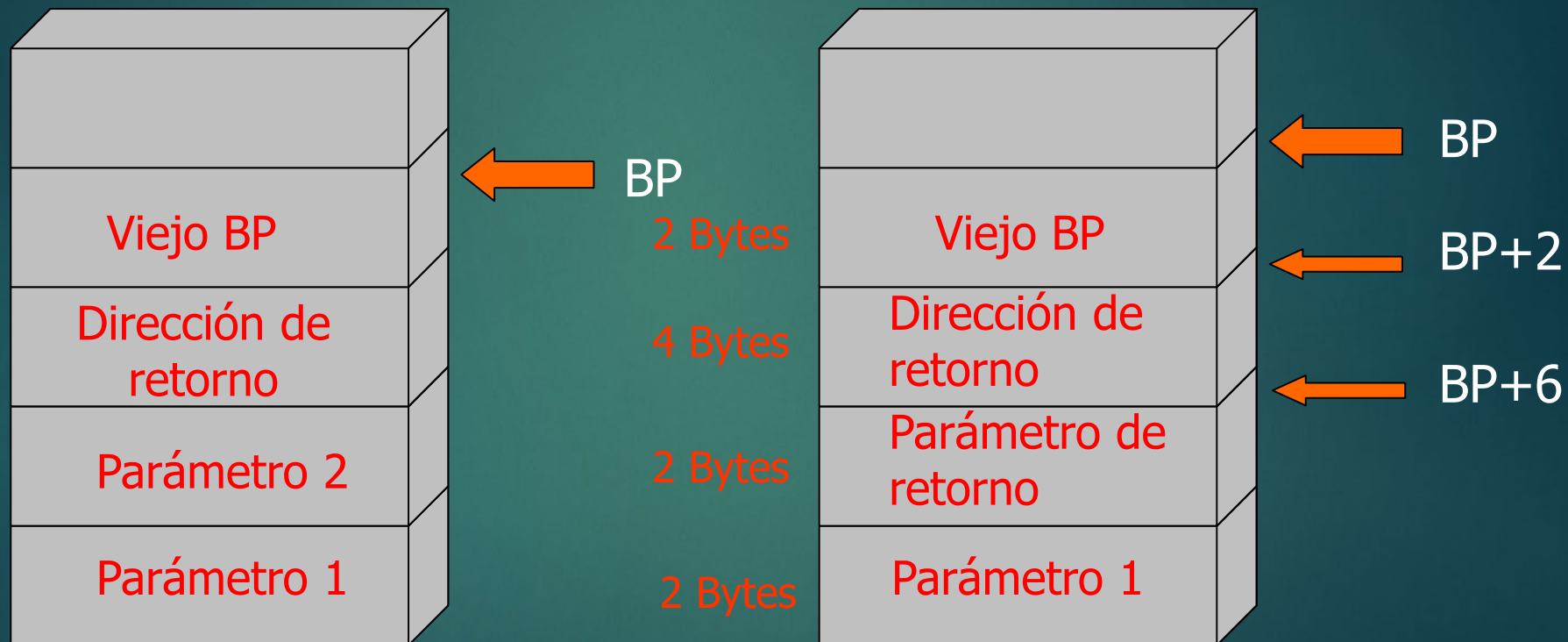
Pila y punteros pasos 7, 8, 9 y 10

Detalles de uso de subrutinas

Estructura de la subrutina – Parámetros de retorno

- Las subrutinas realizan un trabajo determinado. Ese trabajo puede producir un resultado, que debe ser devuelto al programa que la invoca.
- Típicamente la subrutina devuelve 1 o más resultados.
- El resultado se retorna por las mismas posiciones de memoria donde la subrutina recibió los parámetros.
- En el ejemplo, el resultado debe guardarse en la dirección de memoria con un offset de 6 respecto del valor actual de BP, es decir: $BP+6$
- Notar que en $BP+2$ está la dirección de retorno que NO debe ser alterada de ninguna manera.

Detalles de uso de subrutinas



Pila sin parámetros de retorno

Pila con parámetros de retorno

Anidamiento de subrutinas

122

- Una subrutina puede invocar a otra subrutina. Esto se conoce como “anidamiento de subrutinas”.
- Cada subrutina tiene asociado su propio espacio de memoria en la Pila, tal como se vio anteriormente.
- La cantidad de anidamientos posibles de subrutinas depende de varios factores, pero básicamente del espacio de memoria para disponer de la estructura de datos de cada subrutina. Es por eso que la cantidad de anidamiento es muy grande (casi “infinita”).
- Dado que la información se guarda en una estructura tipo LIFO, cada vez que se termina una subrutina de un nivel de anidamiento dado, se retorna a la subrutina del nivel anterior. En otras palabras, cuando se termina una subrutina se devuelve el control a la subrutina que la invocó.

Anidamiento de subrutinas

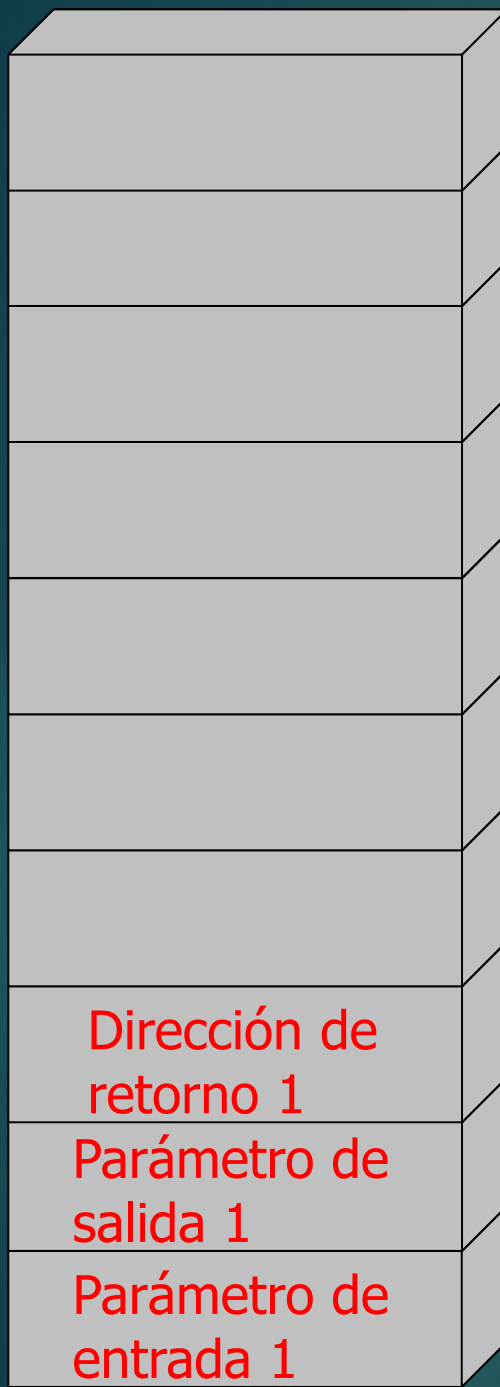
- En el ejemplo que sigue se muestra un programa (principal) PPIO que comienza en la dirección 1000.
- El PPIO invoca una subrutina SUBR1 (nivel de anidamiento 1) que comienza en la dirección 3000H.
- La subrutina SUBR1 invoca a subrutina SUBR2 (nivel de anidamiento 2) que comienza en la dirección 4000H.
- Cuando termina SUBR2, de forma correcta, retorna a la tarea que la invocó, es decir ejecuta en SUBR1 la instrucción siguiente del CALL SUBR2 (es decir ejecutará POP BX).
- Cuando termina SUBR1, retorna a PPIO, a la instrucción siguiente del CALL SUBR1 (es decir ejecutará POP BX).

Anidamiento de subrutinas

124

	ORG 1000H	ORG 3000H	ORG 4000H
PPIO:	MOV BX, 0	SUBR1: NEG AX	SUBR2: INC AX
	MOV AX, dato	PUSH AX	RET
	PUSH AX	CALL SUBR2	
	CALL SUBR1	POP BX	
	POP BX	RET	
	HLT		
	ORG 2000H		
dato:	DB 55H,00H		
	END		

Anidamiento de subrutinas

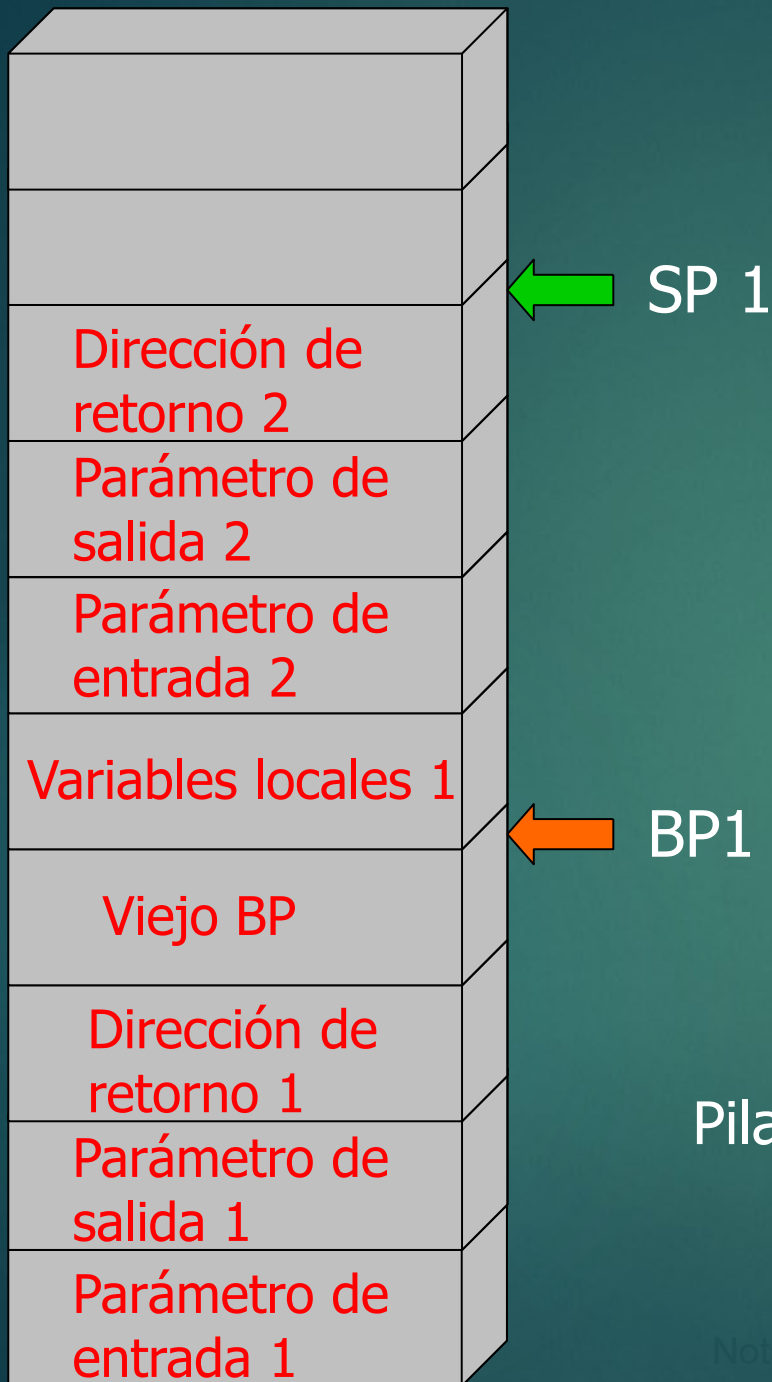


← SP

Pila y punteros al invocar la subrutina 1

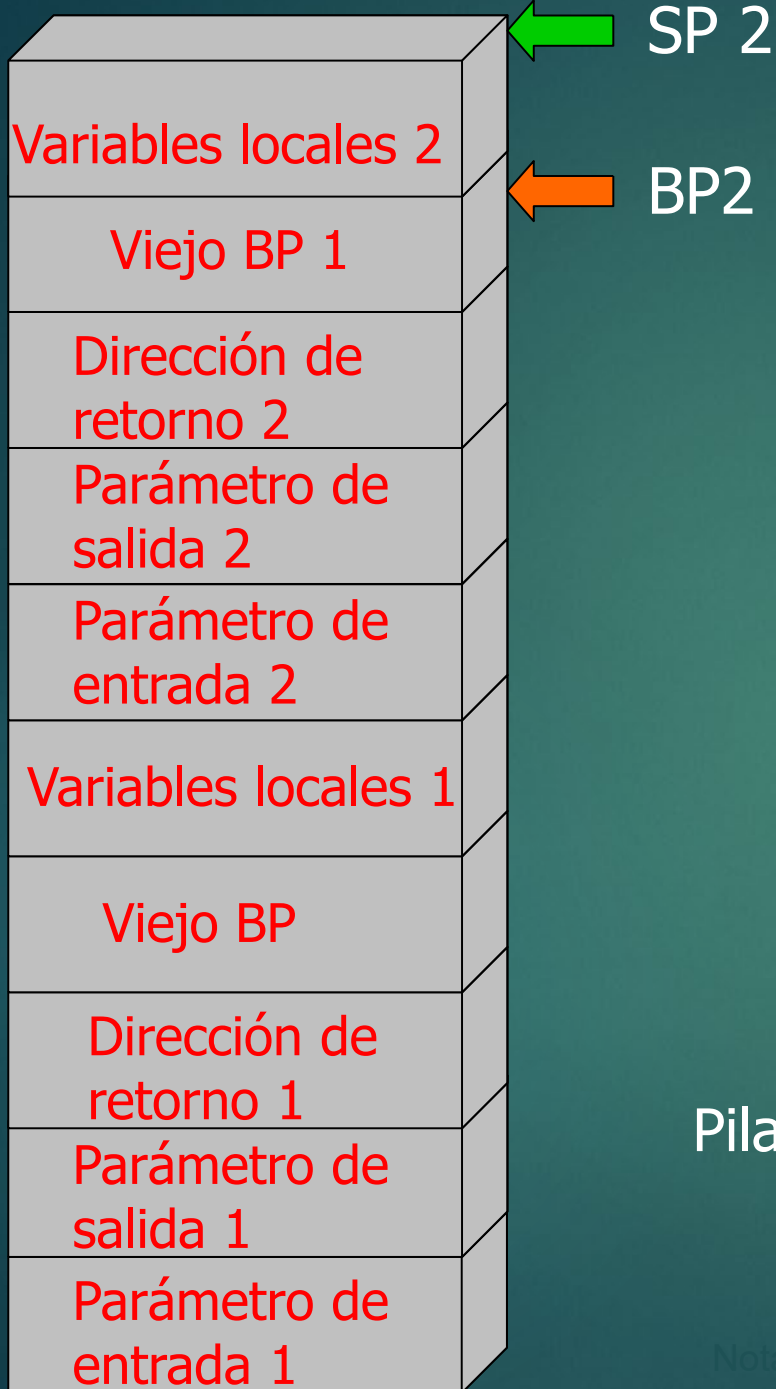
← BP

Anidamiento de subrutinas



Pila y punteros al invocar la subrutina 2

Anidamiento de subrutinas



Pila y punteros en la subrutina 2

Ejemplo para simulador MSX88

- A continuación se muestra un programa de ejemplo para ser cargado y ejecutado en el simulador MSX88.
- El programa principal que invoca una subrutina empieza en la dirección 2000H.
- La subrutina a ser invocada empieza en la dirección 3000H.
- Como ejercicio, analizar la secuencia de acciones realizada en el programa principal y en la subrutina, y comparar con los pasos previstos en las estructuras de programa principal y subrutina vistos.
- Tener en cuenta que la declaración del procedimiento se hace directamente así: *nombre: instrucción*
- Además, dado que el MSX88 no dispone del registro BP, para el acceso a la estructura de datos de la Pila se usa el BX en su reemplazo.

Ejemplo para simulador MSX88

129

ORG 1000HNUM1 **DW** 5HNUM2 **DW** 3HRES **DW** ?**ORG 2000H**

MOV AX,NUM1

PUSH AX

MOV AX,NUM2

PUSH AX

MOV AX,OFFSET RES

PUSH AX

MOV DX,0

CALL MUL

POP AX

POP AX

POP AX

HLT

END**ORG 3000H**

MUL: PUSH BX

MOV BX,SP

PUSH CX

PUSH AX

PUSH DX

ADD BX,6

MOV CX,[BX]

ADD BX,2

MOV AX,[BX]

SUMA: ADD DX,AX

DEC CX

JNZ SUMA

SUB BX,4

MOV AX,[BX]

MOV BX,AX

POP DX

POP AX

POP CX

POP BX

RET

Bibliografía

- ***Organización y Arquitectura de Computadoras – Diseño para optimizar prestaciones***, Stallings W., Editorial Prentice Hall (5º edición).
- ***Organización de Computadoras***, Tanenbaum A., Editorial Prentice Hall (4º edición).
- ***Arquitectura de Computadores - Un enfoque cuantitativo***, Hennessy & Patterson., Editorial Mc Graw Hill (1º edición).
- ***Diseño y evaluación de arquitecturas de computadoras***, Beltrán M. y Guzmán A., Editorial Prentice Hall (1º edición).

Referencias

- Organización y Arquitectura de Computadoras. W. Stallings, 5ta Ed.
 - Repertorios de instrucciones
 - Capítulo 9: características y funciones
 - Capítulo 10: modos de direccionamiento y formatos
 - Apéndice 9A: Pilas
 - Ciclo de instrucción:
 - Capítulo 3 apartado 3.2.
 - Capítulo 11 apartados 11.1. y 11.3.
 - Organización de los registros
 - Capítulo 11 apartado 11.2.
 - Formatos de instrucciones
 - Capítulo 10 apartado 10.3. y 10.4.
- Simulador MSX88

Máquina de 4 direcciones

COD OP	dirOP1	dirOP2	dirRES	dirProxInstr
--------	--------	--------	--------	--------------

$$A = \frac{(B + C) \times D}{E}$$

Programa

I0 ADD dirB, dirC, dirA, dirI3
I1 **HALT**
I2 DIV dirA, dirE, dirA, dirI1
I3 MUL dirA, dirD, dirA, dirI2

Máquina de 3 direcciones

COD OP	dirOP1	dirOP2	dirRES
--------	--------	--------	--------

$$A = \frac{(B + C) \times D}{E}$$

CPU necesita registro PC para ejecución secuencial

Programa

I0 ADD dirB, dirC, dirA
I1 MUL dirA, dirD, dirA
I2 DIV dirA, dirE, dirA
I3 **HALT**

Máquina de 2 direcciones

COD OP	dirOP1/dirRES	dirOP2
--------	---------------	--------

$$A = \frac{(B + C) \times D}{E}$$

CPU con registro PC y nuevas operaciones

Programa

I0 **MOV** dirA, dirB
I1 ADD dirA, dirC
I2 MUL dirA, dirD
I4 DIV dirA, dirE
I5 **HALT**

Máquina de 1 dirección

COD OP	dirOP2
--------	--------

$$A = \frac{(B + C) \times D}{E}$$

CPU con registro PC y registro Acumulador
y nuevas operaciones

Programa

I0 **LOAD** dirB
I1 ADD dirC
I2 MUL dirD
I4 DIV dirE
I5 **STORE** dirA
I6 **HALT**

Máquina de 0 direcciones

COD OP

$$A = \frac{(B + C) \times D}{E}$$

CPU con registro PC, Pila y
nuevas operaciones

Programa

I0	PUSH dirE
I1	PUSH dirD
I2	PUSH dirC
I3	PUSH dirB
I4	ADD
I5	MUL
I6	DIV
I7	POP dirA
I8	HALT

Arquitectura de Computadoras

CURSO 2022

Turno:
Clase 2

Resumen clase 2

Interrupciones

- Concepto de interrupción
- Características de las interrupciones
- Tratamiento de las interrupciones
- Controlador programable de interrupciones PIC
- Interrupciones del 8086
- Interrupciones del MSX88

Concepto de Interrupción

3

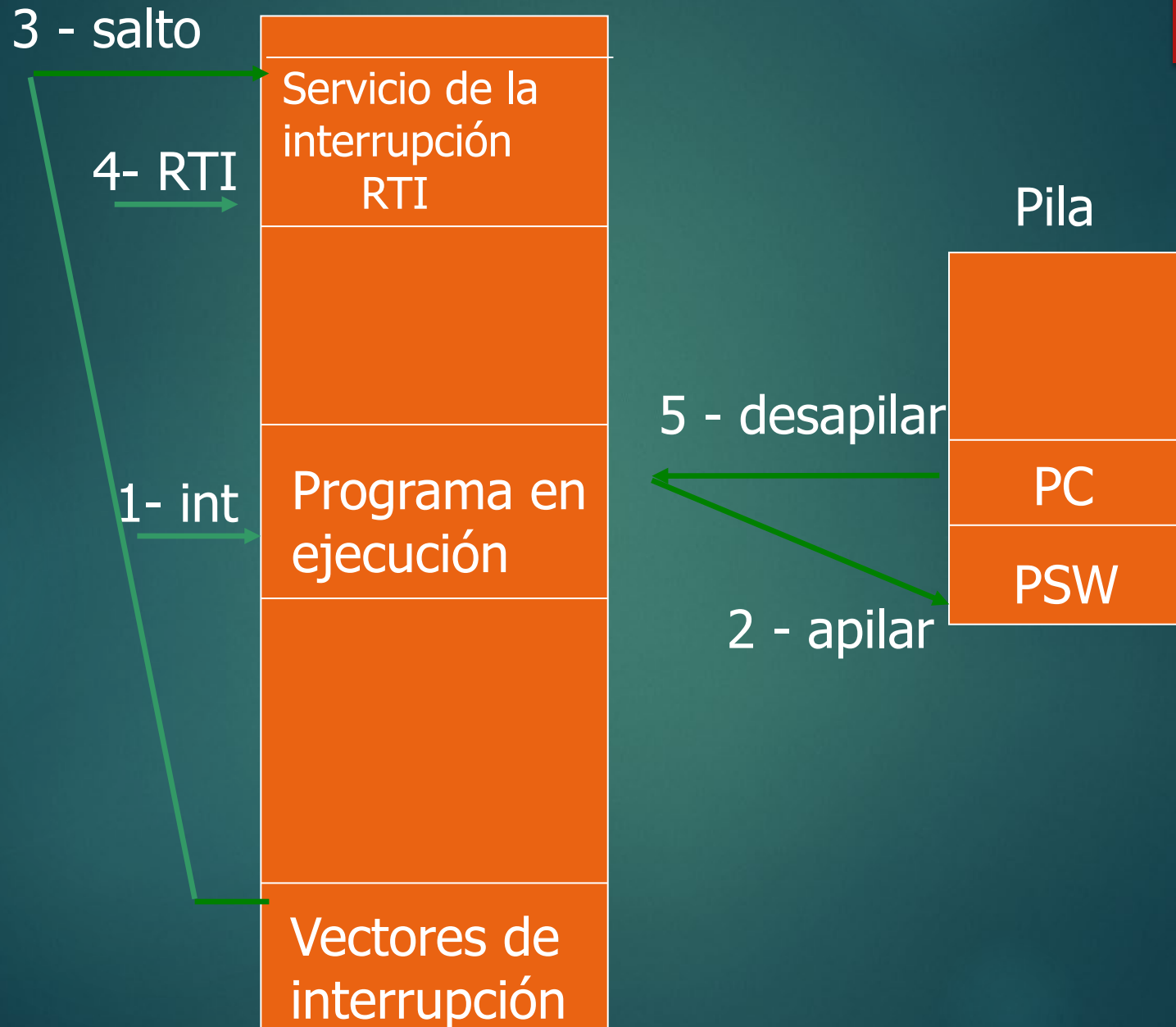
- En condiciones “normales”, la CPU lee y ejecuta una instrucción a continuación de la otra de manera ininterrumpida (el “bucle interno del ciclo de instrucción”).
- Una interrupción es un mecanismo que permite alterar ese proceso de “ejecución normal” de la CPU.
- Este mecanismo permite que la CPU suspenda la tarea que está haciendo y responda a una solicitud de atención para resolver (ejecutar) otra tarea (servicio de la interrupción).
- Una vez completado el servicio de la interrupción, el procesador retoma la tarea suspendida, en el punto donde se detuvo (de una manera similar al llamado a subrutina).

Concepto de Interrupción

4

- De acuerdo al concepto de interrupción, el procesador conmuta de una tarea en ejecución a otra a ejecutarse por efecto de la presencia de un evento (la interrupción). Por lo tanto se requiere de 3 acciones:
- **1)** Detener (suspender) la tarea que está ejecutando el procesador (suspender, no terminar ni abortar).
- **2)** Bifurcar (saltar) a otra tarea, asociada a la solicitud de interrupción (comúnmente conocida como servicio interrupción).
- **3)** Restablecer la tarea suspendida en las condiciones en las que se encontraba en el momento en el que se la detuvo.

Operación de una Interrupción



Concepto de Interrupción

6

- La forma en que opera la interrupción se puede ver en la figura anterior.
- **1.-** La CPU recibe, mientras está ejecutando una tarea (el programa en ejecución), un pedido de interrupción.
- **2.-** La CPU salva todo o parte del estado de la CPU correspondiente a la tarea a ser suspendida. Al menos salva el Contador de programa (PC) y el registro de estado (PSW), típicamente en la Pila del sistema . Esto lo hace porque lo necesita para restaurar la tarea suspendida.
- **3.-** La CPU busca, en un área de memoria definida, la dirección de comienzo (el “vector”) del servicio de la interrupción, y comienza a ejecutar dicho servicio.

Concepto de Interrupción

7

- **4.-** Cuando la CPU termina el servicio de la interrupción, tiene que retornar al programa interrumpido. Esto se hace mediante una instrucción especial de Retorno de interrupción (RTI).
- **5.-** La ejecución de la instrucción RTI desapila exactamente lo apilado cuando atendió la interrupción (es decir, todo o parte del estado de la CPU). Como mínimo recupera el PC y el PSW. De esta manera retoma la tarea suspendida (el programa en ejecución) en el punto en que fue interrumpida.

Características de las interrupciones

- El origen de una interrupción es la ocurrencia de un evento que requieren la intervención de la CPU.
- Existen 2 tipos de eventos:
 - Interno: es decir, debido a la ocurrencia de una situación dentro del Sistema de cómputo. Algunos eventos internos que pueden producir interrupción son: un error asociado a la ejecución de una instrucción, desbordamiento aritmético (“overflow”), división por cero, temporizados propios del sistema, fallo del hardware, error de paridad en la memoria, pérdida de energía.
 - Externo: asociado a operaciones de E/S con periféricos. Algunos eventos típicos que pueden generara interrupción son: finalización de una transferencia, error en la transferencia, dispositivo indisponible, etc.

Características de las interrupciones

Interrupciones múltiples

- La necesidad de administrar eventos de distinto origen requiere, en la mayor parte de los casos, administrar varias interrupciones.
- Dado el origen diverso de las interrupciones, hay algunas que son más importantes que otras.
- En general, los procesadores son capaces de manejar varias interrupciones de distinta características y orígenes.

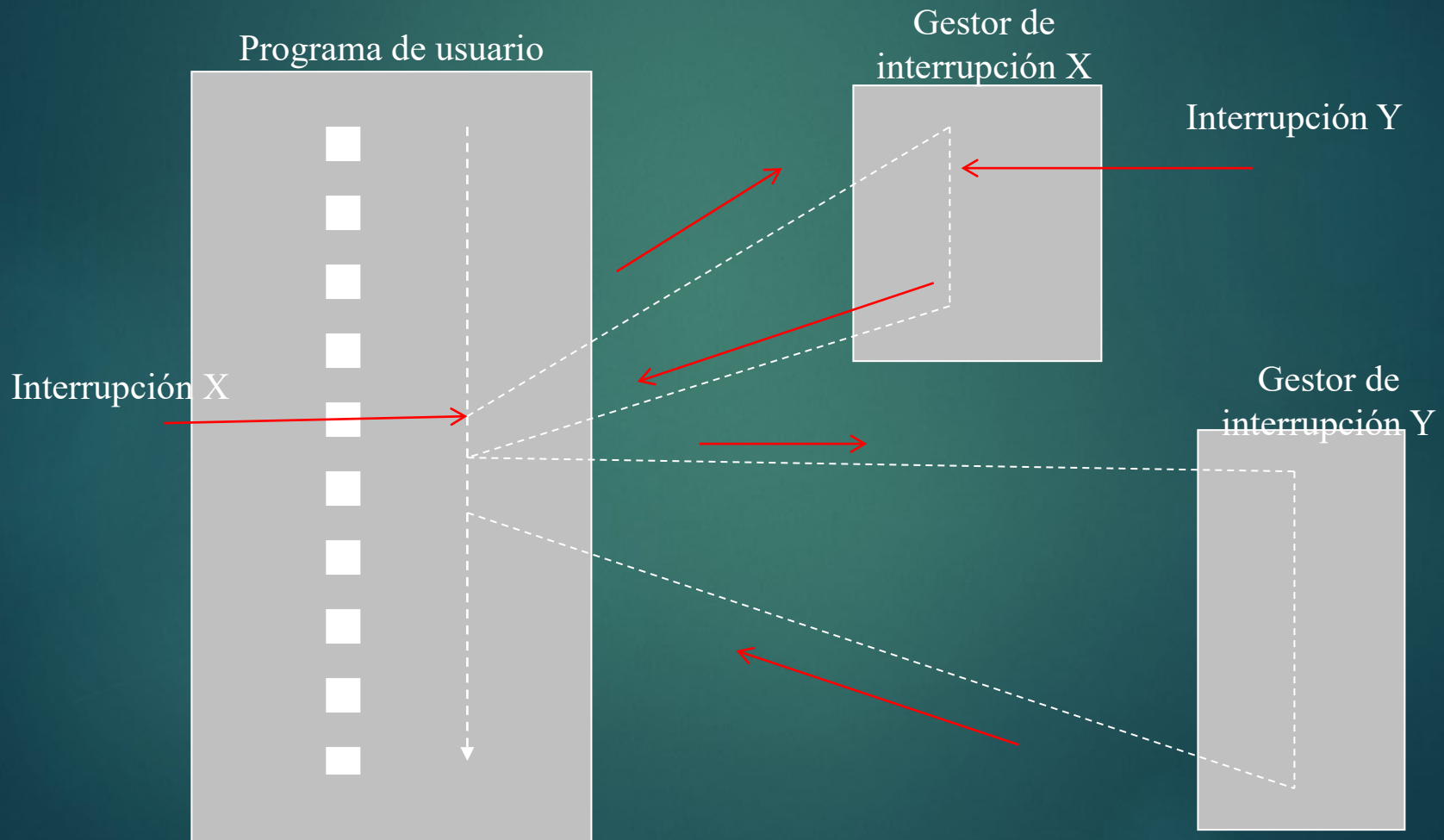
Prioridades

Prioridades en interrupciones múltiples

- Las interrupciones más importantes deben tener mayor “prioridad” que las menos importantes.
- Cuanto mayor sea su prioridad, mayor es la urgencia para ser atendida, incluso si hay una interrupción en curso.
- Si las interrupciones son de igual prioridad, se procesan en el orden en que llegan.
- Si las interrupciones tienen distinta prioridad, las interrupciones de mayor prioridad pueden interrumpir a las de menor prioridad. La inversa no vale.

Prioridades

➤ Procesamiento de interrupciones de igual prioridad.



Prioridades

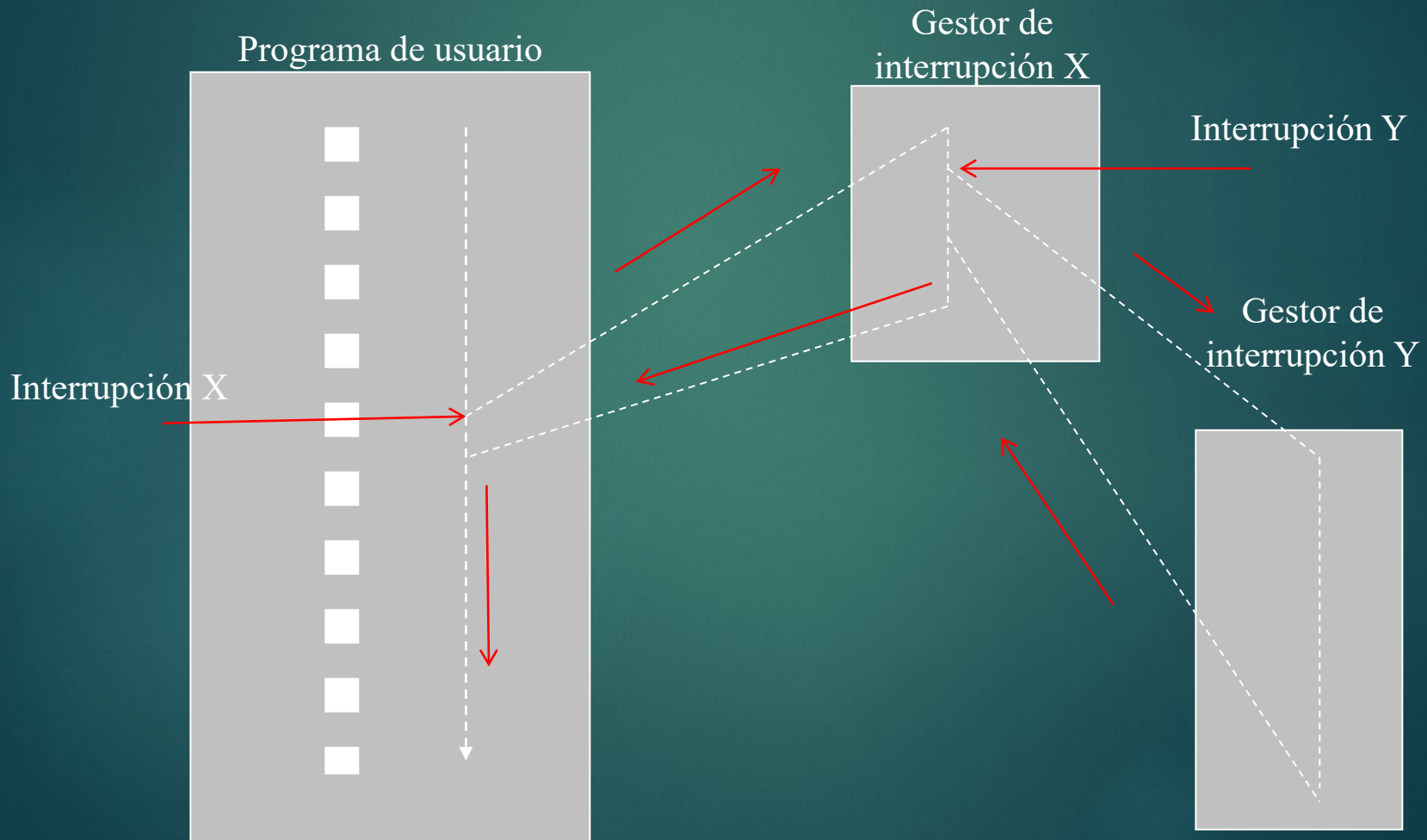
Procesamiento de interrupciones de igual prioridad

- Las interrupciones se atienden según el orden de llegada.
- Cuando llega una interrupción y es atendida, se inhabilita el resto de las interrupciones de igual o menor nivel de prioridad.
- Si llega una nueva interrupción quedará pendiente.
- El procesador ejecutará el servicio de la interrupción atendida.
- Al finalizar el servicio se habilitarán nuevamente las interrupciones.
- La interrupción pendiente será atendida.

Prioridades

13

- Procesamiento de interrupciones de distinta prioridad.



Prioridades

14

Procesamiento de interrupciones de distinta prioridad

- Una interrupción de prioridad más alta puede interrumpir en cualquier momento a una interrupción de prioridad menor.
- Cuando se ha gestionado la interrupción de prioridad más alta, el procesador vuelve a las interrupciones previas (de menor prioridad).
- Terminadas todas las rutinas de gestión de interrupciones se retoma el programa del usuario.

Tipos de interrupciones

15

Interrupciones enmascarables y no-enmascarables

- Los sistemas de cómputo pueden responder de manera diferente a los distintos tipos de interrupciones, principalmente asociado a la prioridad que tienen.
- Típicamente tienen 2 categorías de interrupciones.
 - No Enmascarables: son interrupciones que no pueden ser ignoradas, es decir, se atienden indefectiblemente, y están asociadas a eventos críticos, peligrosos o de alta prioridad.
 - Enmascarables: pueden ser, eventualmente, “ignoradas”. Para ello el procesador permite realizar algunas acciones que pueden inhibir la atención de la interrupción. Las interrupciones enmascarables generalmente están asociadas a operaciones menos críticas, por ejemplo de E/S.

Tipos de interrupciones

16

Interrupciones por hardware y por software

- Desde el punto de vista de la forma en que se invocan, las interrupciones pueden ser 2 de tipos:
 - Por Hardware: son generadas por señales físicas asociadas a eventos externos o internos. De acuerdo al origen de la señal de interrupción, pueden ser:
 - Externas
 - Internas
 - Por Software: son producto de la ejecución de instrucciones con efecto similar a una interrupción por hardware.

Tipos de interrupciones

17

Interrupciones por hardware externas

- Son conocidas como Interrupt request.
- El origen de estas señales de pedido de interrupción proviene típicamente de dispositivos conectados al subsistema de E/S.
- Se consideran las “verdaderas” interrupciones porque son aleatorias en relación al proceso en ejecución (es decir, pueden ocurrir en “cualquier instante de tiempo”).
- El sistema de cómputo tiene que ser capaz de manejar estos eventos externos “no planeados” ó “asincrónicos”.
- Pueden o no estar relacionadas con el proceso en ejecución en ese momento.

Tipos de interrupciones

18

Interrupciones por software

- Son conocidas como Software interrupt.
- Son instrucciones explícitas que tienen un efecto similar a una interrupción por hardware.
- Como normalmente el SO administra los servicios de las Interrupciones, las Interrupciones por software permiten invocar los servicios del SO asociados a las Interrupciones (en otras palabras, son “llamadas” a funciones del SO).
- El SO define los lugares donde se cargan los servicios de las Interrupciones. El usuario no conoce, a priori, esos lugares, pero usa los servicios invocándolos a través de las interrupciones que maneja el SO.

Tipos de interrupciones

19

Interrupciones por hardware internas

- Son conocidas como Trap o excepciones.
- Son señales creadas dentro del sistema de cómputo en respuesta a situaciones propias del proceso en ejecución y no vinculadas con operaciones de E/S, por tal motivo no son estrictamente aleatorias.
- Algunos eventos que pueden producir un Trap son:
 - Condiciones excepcionales: overflow en ALU de punto flotante.
 - Fallas de programa: tratar de ejecutar una instrucción no definida.
 - Fallas de hardware: error de paridad de memoria.
 - Accesos no alineados o a zonas de memoria protegidos

Tipos de interrupciones

20

Interrupciones por software

- Hay sistemas que no permiten hacer una llamada directa a una función del SO, por estar en una zona reservada.
- Si no se pudieran usar las interrupciones por software como mecanismo de invocación de esos servicios que administra el SO, cuando se necesita administrar una tarea por interrupción, se debería:
 - **Opción 1:** escribir el servicio (bastante complicado)
 - **Opción 2:** el programa que requiere el servicio debería buscar entre todas las llamadas a funciones del BIOS y SO la que necesita, y reemplazar en el código la dirección de esa función invocada (también muy complicado).

Tratamiento de las interrupciones

21

Gestión de las interrupciones

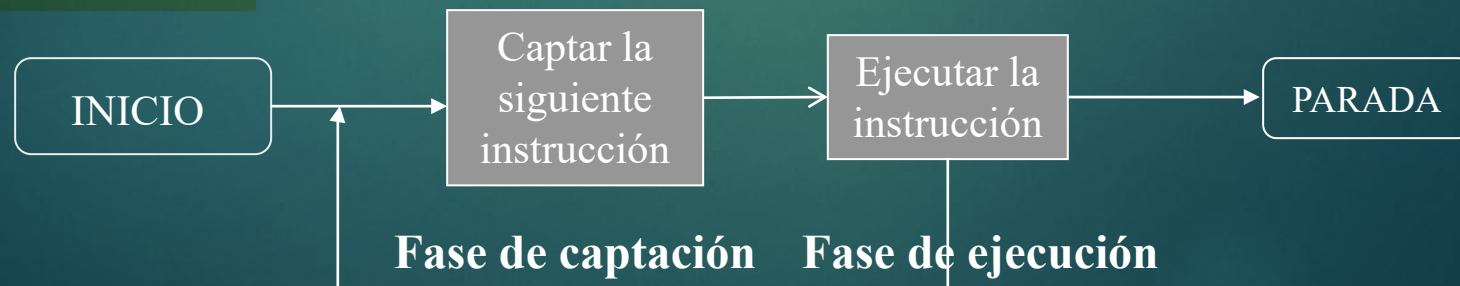
➤ El uso de interrupciones requiere de la gestión ordenada de las siguientes acciones básicas:

1. Detectar el pedido de interrupción
2. Detener la tarea que se estaba ejecutando
3. Salvar el estado de la tarea que se estaba ejecutando
4. Obtener la dirección de comienzo del servicio de la interrupción y bifurcar a dicho servicio
5. Ejecutar el servicio de la interrupción
6. Retornar y restaurar el estado en que estaba la tarea interrumpida.
7. Continuar con la ejecución normal de la tarea interrumpida (en el punto en el que se detuvo)

Tratamiento de las interrupciones

Detección del pedido de interrupción

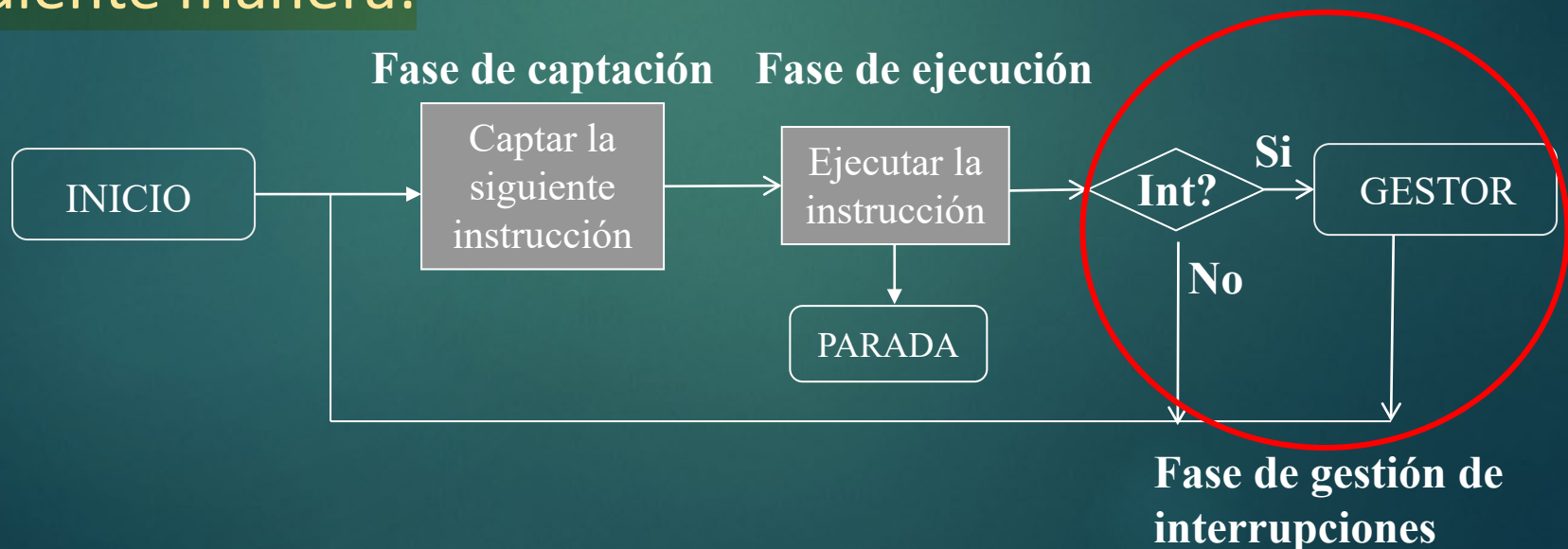
- El primer paso en la administración de las interrupciones consiste en detectar el pedido de atención.
- Para ejecutar esta operación, el procesador examina, en cada ciclo de instrucción, la presencia de interrupciones.
- De acuerdo a lo visto previamente, el ciclo de instrucción podía interpretarse como un bucle de ejecución interna, infinito, compuesto por 2 fases: búsqueda de la instrucción y ejecución.



Tratamiento de las interrupciones

Detección del pedido de interrupción

- Para poder implementar la tarea de detección de interrupciones, en el ciclo de instrucción anterior se agrega, además de las fases de captura y ejecución, la fase de gestión de interrupciones.
- El ciclo completo de la instrucción queda modificado de la siguiente manera:



Tratamiento de las interrupciones

24

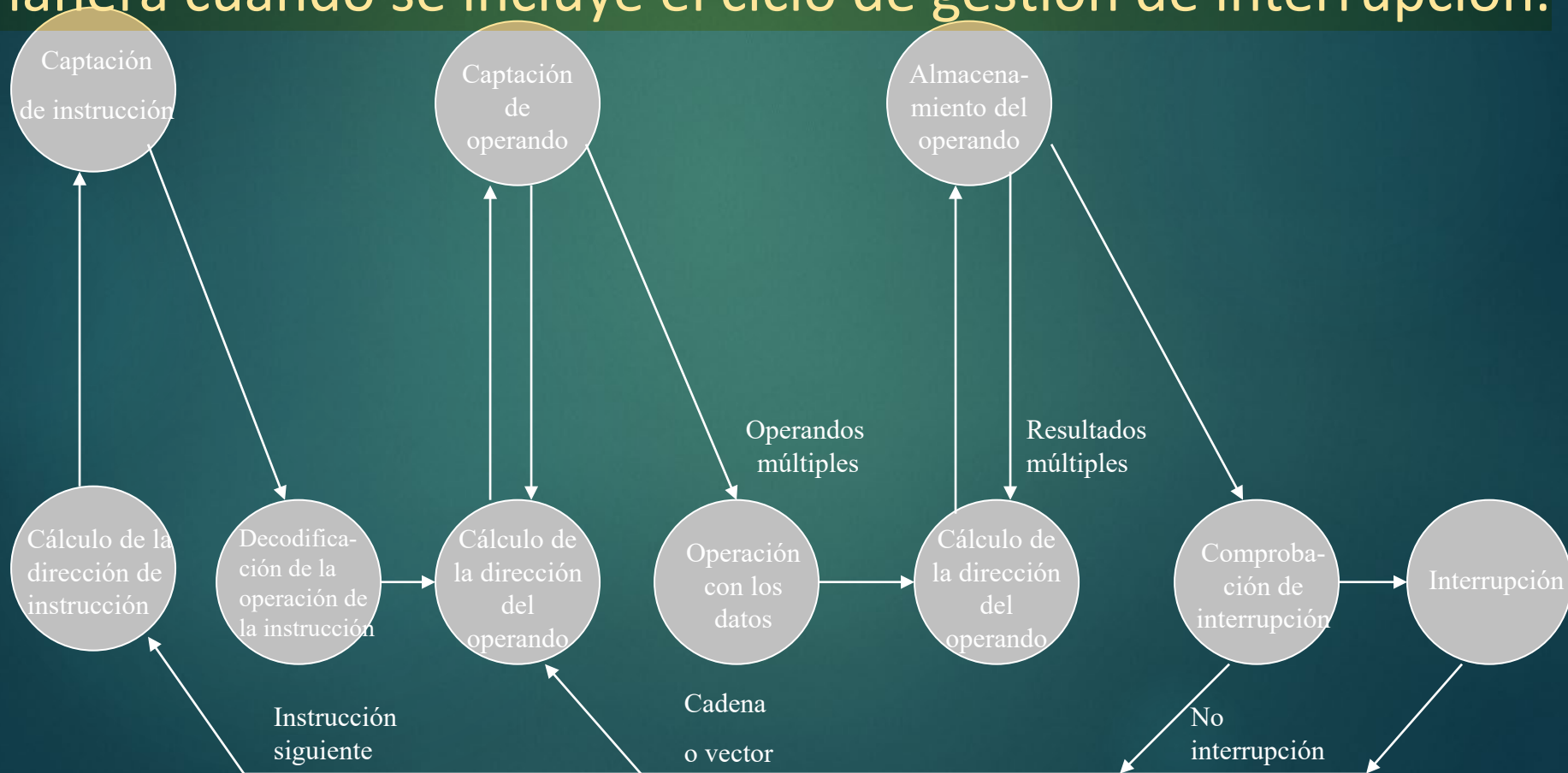
Detección del pedido de interrupción

- La etapa de gestión de interrupciones en el ciclo de instrucción, determina la presencia o ausencia de pedido de interrupciones.
- La presencia de un pedido de interrupción se manifiesta mediante una o más señales discretas (bits) comúnmente llamadas bandera (o “Flag”) que la CPU examina.
- El estado de estos flags asociados a interrupciones están en algún registro (especial) de la CPU.
- Dependiendo del estado 0 o 1 del flag (es decir, del pedido de interrupción) se tienen 2 posibles caminos.

Tratamiento de las interrupciones

Detección del pedido de interrupción

El ciclo de instrucción detallado se modifica de la siguiente manera cuando se incluye el ciclo de gestión de interrupción.



Tratamiento de las interrupciones

26

Almacenamiento del proceso a ser interrumpido

- Si no hay pedido pendiente (Flag inactivo) se inicia el ciclo de captación de la siguiente instrucción (proceso “normal” de ejecución).
- Si hay algún pedido de interrupción pendiente, el procesador guarda en la pila del Sistema, el “estado del proceso”.
- Existen 2 estrategias de guardado del estado del proceso:
 - Guardar solo la próxima instrucción a ejecutar y algún registro crítico (por ejemplo el registro de estado).
 - Guardar todos los registros del procesador.
- El objetivo de esta operación es el de restablecer el estado del procesador al terminar el servicio de la interrupción.

Tratamiento de las interrupciones

27

Bifurcación al servicio de la interrupción

- Obtiene la dirección donde comienza la rutina de la interrupción y carga el PC con este valor, bifurcando de esta manera, al servicio de la interrupción.
- Existen varias técnicas para obtener la dirección donde comienza el servicio de la interrupción. En general se dispone de un área de memoria reservada, donde están estas direcciones. Tener en cuenta que son varias direcciones porque el procesador es capaz de atender varias interrupciones, y habrá una dirección por interrupción.
- Esta área de memoria se llama área de vectores de interrupciones.

Tratamiento de las interrupciones

28

Detección del pedido de múltiples interrupciones

- Dentro del proceso de gestión de las interrupciones, lo primero que tiene que hacer la CPU es detectar el pedido de interrupción. Cuando hay múltiples fuentes de interrupción, hay varias formas para identificar el origen del pedido. Los más comunes son:
 - **Opción 1:** 1 señal física de entrada a la CPU por cada Interrupción.
 - **Opción 2:** 1 única señal física de entrada a la CPU para todas las interrupciones e identificación por software.
 - **Opción 3:** 1 única señal física de entrada a la CPU para todas las interrupciones e identificación por hardware.

Tratamiento de las interrupciones

29

Detección del pedido de múltiples interrupciones

- **Opción 1:** Se tiene 1 señal física de entrada a la CPU por cada interrupción, es decir que hay múltiples líneas de pedido de interrupción en la CPU. Dado que disponer de líneas en la CPU para interrupción es costoso, la cantidad de señales para ser usadas en interrupciones se acota normalmente a un número reducido (por ejemplo 3 o 4).
- Cada dispositivo que puede provocar interrupción tiene una entrada física de interrupción conectada directamente a la CPU.
- La implementación es bastante sencilla.
- Restringida por la cantidad de líneas disponibles en la CPU.

Tratamiento de las interrupciones

30

Detección del pedido de múltiples interrupción

- **Opción 2:** Hay 1 sola entrada física de pedido de interrupción a la que están conectados todos los dispositivos.
- Para poder identificar la fuente de la solicitud, la CPU debe “preguntar” a cada dispositivo si ha producido el pedido de interrupción. Este método de consulta se conoce como encuesta o polling, y es básicamente un programa que ejecuta la CPU dentro de los servicios de interrupción.
- Al ser un esquema de detección por software, el tener que consultar uno por uno a todos los dispositivos lo hace relativamente lento e ineficiente.

Tratamiento de las interrupciones

31

Detección del pedido de múltiples interrupción

- **Opción 3:** Hay 1 sola entrada física de pedido de interrupción a la que están conectados todos los dispositivos.
- Para poder identificar la fuente de la solicitud, la CPU recibe a continuación, típicamente a través del bus de datos, un número que identifica la fuente de la interrupción (conocido como “vector de la interrupción”).
- El vector es provisto por el periférico que generó el pedido, o por algún dispositivo que se ocupe de generar el número dependiendo de la interrupción a ser atendida.

Controlador de interrupciones PIC

32

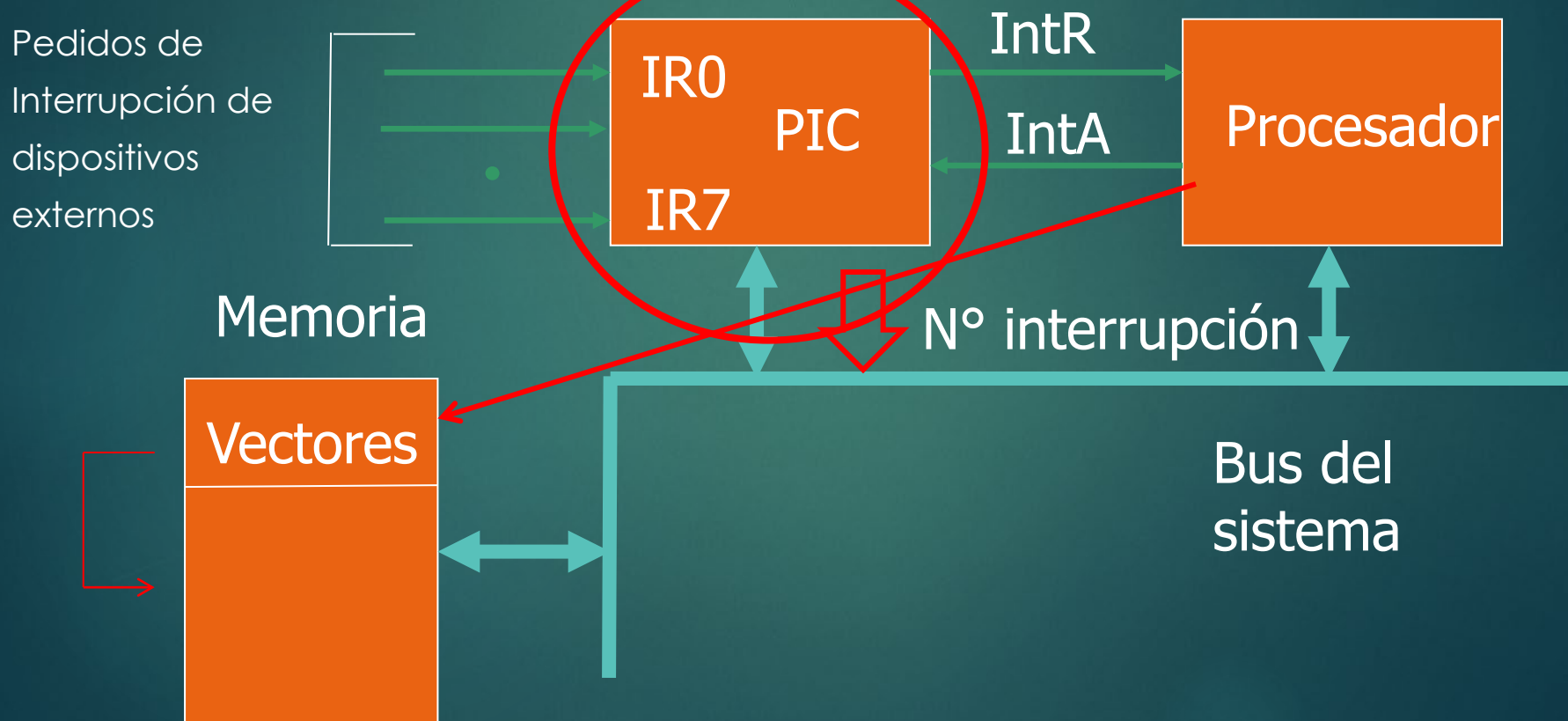
Interrupciones vectorizadas con el PIC

- La opción 3 se conoce como Interrupciones vectorizadas.
- El escenario que se tiene es el siguiente:
 - El procesador tiene una única entrada de pedido de interrupciones.
 - Hay varios “productores” de interrupciones.
 - Un “dispositivo especial” administra las necesidades propias de la interrupción. En la familia Intel, este dispositivo se conoce como Controlador Programable de Interrupciones (‘PIC’). El PIC se encarga, entre otras cosas, de generar el vector, administrar prioridades, habilitar interrupciones, etc.

Controlador de interrupciones PIC

Interrupciones vectorizadas con el PIC

La figura siguiente representa el **esquema de conexionado del PIC en un sistema de interrupciones vectorizadas (i80x86)**.



Controlador de interrupciones PIC

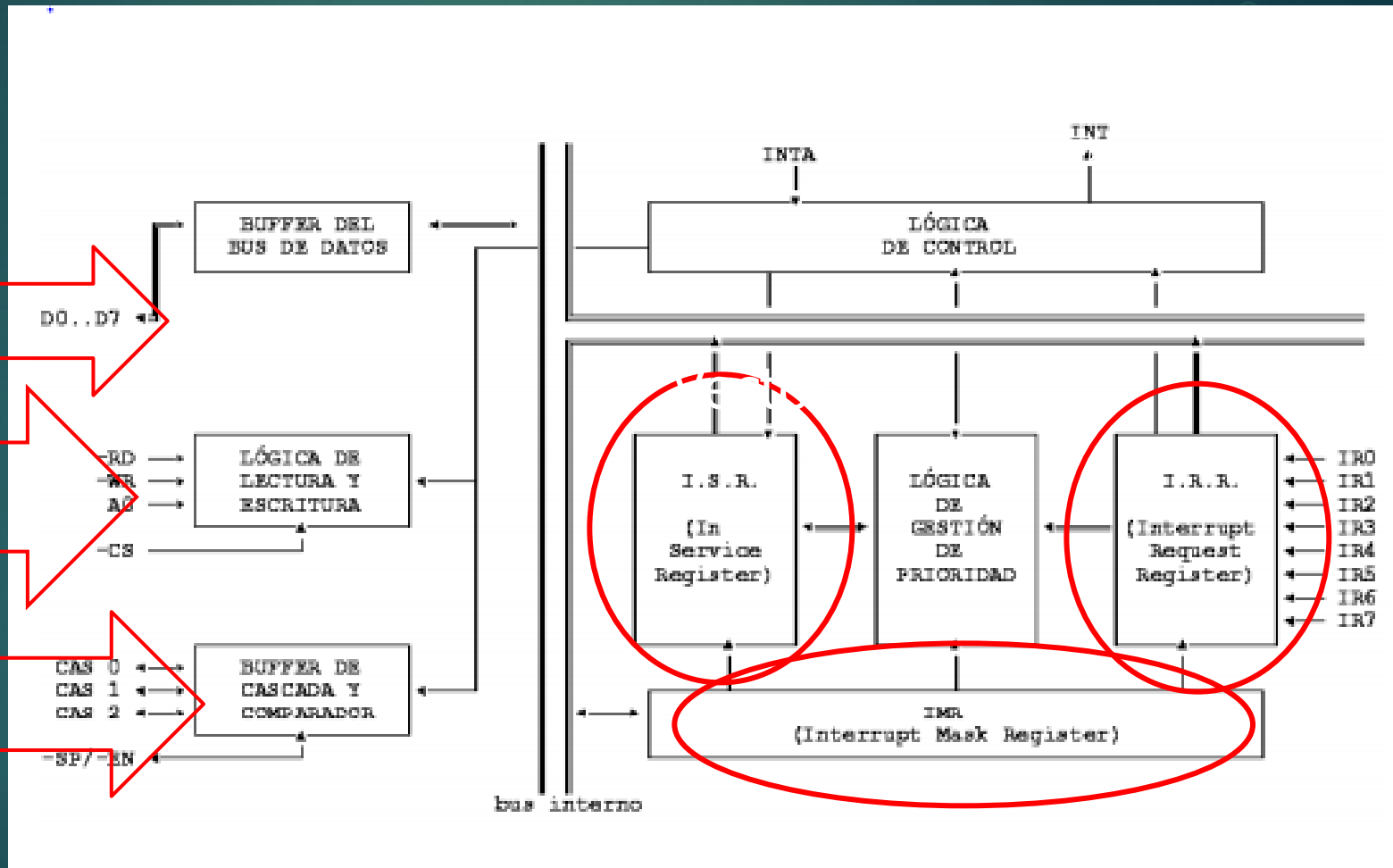
34

Interrupciones vectorizadas con el PIC

- El PIC recibe los pedidos de interrupción, típicamente de periféricos que piden atención.
- El PIC solicita atención a la CPU con la única señal de pedido de interrupción IntR (Interrupt request)
- Cuando la CPU está lista para atender la interrupción, le avisa al PIC mediante la señal IntA (Interrupt acknowledge).
- El PIC genera en el bus de datos el número de la interrupción (vector) a ser atendida. La CPU lee ese número y busca en la memoria el vector correspondiente al servicio de esa interrupción.
- Dado que se hace por hardware, es mucho más rápido.

Controlador de interrupciones PIC

La estructura interna de PIC se puede apreciar en la figura siguiente.



BUS del SISTEMA

Controlador de interrupciones PIC

36

Interrupciones vectorizadas con el PIC

- El PIC internamente tiene 3 registros principales.
 - El ISR que identifica la interrupción en servicio
 - El IRR que contiene los pedidos de interrupción provenientes de los periféricos.
 - El IMR que se usa para habilitar/deshabilitar los pedidos de interrupción. Esta funcionalidad se conoce como enmascaramiento de interrupciones.

Interrupciones del i8086

- A modo de ejemplo se van a describir las características más importantes de las interrupciones en el 8086.
- El 8086 tiene 2 interrupciones por hardware: INTR y NMI
 - La interrupción NMI es del tipo no-enmascarable.
 - La interrupción INTR es del tipo enmascarable. La INTR tiene asociado un flag IF que determina si la INTR va a ser atendida o no (de ahí el concepto de interrupción enmascarable).
- El Procesador dispone de una señal de reconocimiento de interrupción INTA.

Banderas:

OF	DF	IF	TF	SF	ZF	AF	PF	CF
-	-	x	x	-	-	-	-	-

Interrupciones del i8086

38

- El 8086 tiene 1 instrucción de interrupción por software: INT n (con n entre 0 y 255).
- La instrucción de retorno de interrupción es IRET.
- Dispone de 2 banderas relacionadas con las interrupciones:
 - Bandera IF para habilitar/deshabilitar la INTR (si IF=0 no se atiende la INTR)
 - Bandera TF para habilitar/deshabilitar el modo "Trace" (single-step). El modo "trace" o "single-step" es un mecanismo implementado mediante una interrupción (la interrupción de Trace), que habilita a la CPU ejecutar de a 1 instrucción por vez.

Interrupciones del i8086

39

- El esquema de manejo de las interrupciones es vectorizado.
- El área de memoria donde están los vectores de las interrupciones está ubicado en las posiciones más bajas de memoria (normalmente tipo RAM)

es decir: 0000-03FF (= 1024 bytes)

- En total hay 256 vectores, identificados como 0 a 255 (o 00-FFH), correspondiente a 256 interrupciones distintas.
- Cada vector ocupa 4 bytes: 2 para el registro de segmento de código (CS) y 2 para el Contador de Programa (IP).

Interrupciones del i8086

40

La imagen siguiente muestra una parte de la cartilla de interrupciones del 8086 (vectores 0 a 22H) y su uso tentativo.

Interrupción		Uso
Hex	Decimal	
00h	0	Generada por la CPU cuando se intenta hacer una división por cero
01h	1	Utilizada par ir paso a paso por los programas (como DEBUG)
02h	2	Interrupción no enmascarable
03h	3	Utilizada para establecer puntos de ruptura en programas (igual que con DEBUG)
04h	4	Generada cuando operaciones aritméticas dan operaciones de desbordamientos
05h	5	Invoca la rutina de servicio de imprimir pantalla de la ROM BIOS
06h	6	Reservada para DOS
07h	7	Reservada para DOS
08h	8	Generada por el tic-tac del reloj del hardware
09h	9	Generada por acción del teclado
0Ah a 0Dh	10 a 13	Reservadas para BIOS
0Eh	14	Señala atención al diskette (por ejemplo, para señalar operación completada)
0Fh	15	Utilizada para controlar la impresora
10h	16	Invoca servicios de video de la ROM BIOS
11h	17	Invoca el servicio de lista de equipamiento de ROM BIOS
12h	18	Invoca servicio de tamaño de memoria de la ROM BIOS
13h	19	Invoca servicios de disco de la ROM BIOS
14h	20	Invoca servicios de comunicaciones de la ROM BIOS
15h	21	Invoca servicios del sistema de la ROM BIOS
16h	22	Invoca los servicios estándar del teclado de la ROM BIOS
17h	23	Invoca los servicios de la impresora de la ROM BIOS
18h	24	Activa el lenguaje BASIC de la ROM
19h	25	Invoca la rutina cargadora de la secuencia de arranque de la ROM BIOS (invocarla equivale a hacer un RESET)
1Ah	26	Invoca los servicios de hora y fecha de la ROM BIOS
1Bh	27	Interrupción de la ROM BIOS para Ctrl-Break
1Ch	28	Interrupción generada con cada pulso de reloj
1Dh	29	Apunta a la tabla de parámetros de control del video
1Eh	30	Apunta a la tabla de parámetros de la unidad de disco
1Fh	31	Apunta a los caracteres gráficos del CGA
20h	32	Invoca al servicio de terminación de programa del DOS
21h	33	Invoca a todos los servicios de llamada a función DOS
22h	34	Dirección de la rutina de terminación del programa del DOS

Interrupciones del MSX88

- Como ya se mencionó anteriormente, el procesador usado en el simulador MSX88 es una versión simplificada del 8086, por lo que presenta algunas diferencias respecto de la CPU verdadera.
- Interrupciones por hardware
 - Línea INT (interrupción enmascarable) y línea de reconocimiento de interrupción INTA
 - Línea NMI (interrupción no-enmascarable)
- Interrupciones por software
 - Instrucción: INT xx
 - Retorno de interrupción: IRET

Interrupciones del MSX88

42

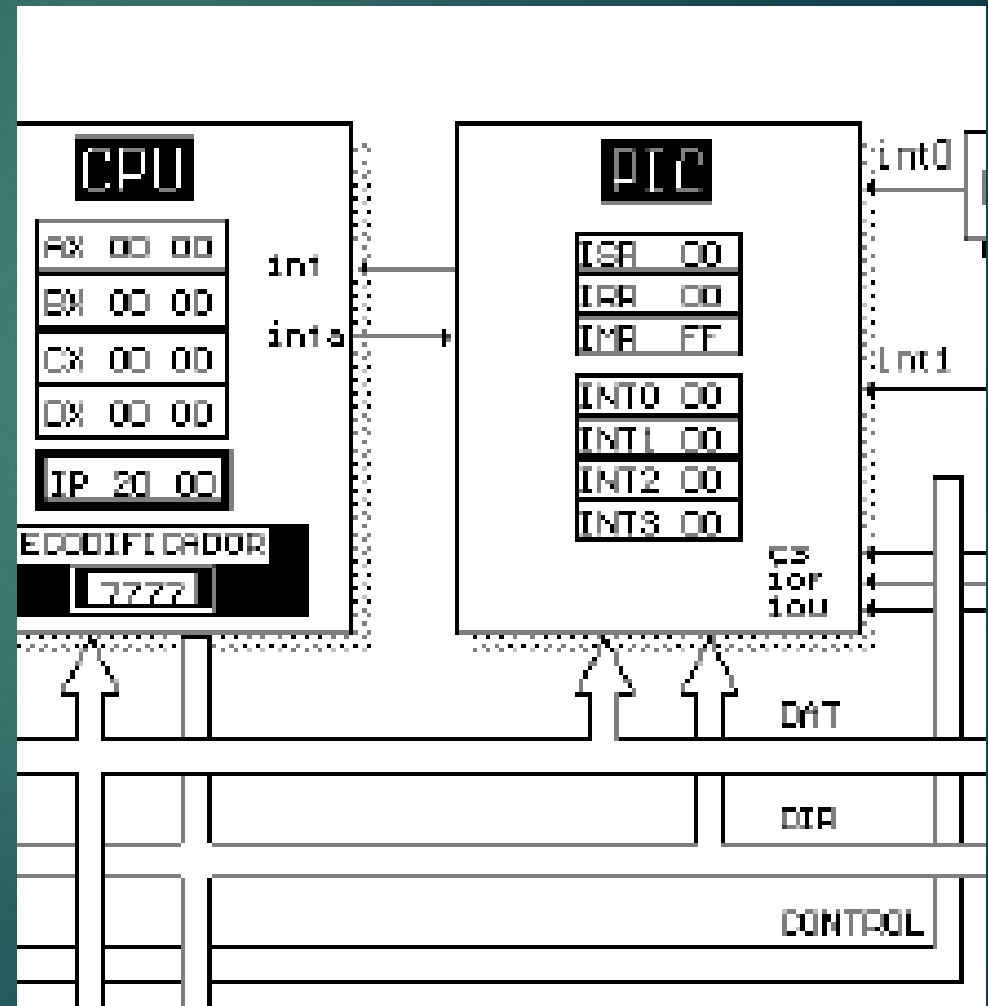
- Los vectores de las interrupciones están en la parte más baja de la memoria.
- Cada entrada (o vector) es una palabra doble (4 bytes), que contiene la dirección del procedimiento que brinda el servicio. La parte alta del vector es 0.
 - Ej: 0000yyyy, donde yyyy es la dirección lógica/física.
- El MSX88 tiene preasignados los siguientes vectores:
 - Tipo 0 – finaliza ejecución de programa
 - Tipo 3 – punto de parada para depuración/seguimiento
 - Tipo 6 – lectura de entrada std. Requiere el uso de BX.
 - Tipo 7 – escritura de salida std. Requiere BX y AL.

Interrupciones del MSX88

La imagen siguiente muestra un **esquema interno del PIC**.

Registros internos PIC

- **INT0...INT7**
 - Vector de cada int.
- **IRR**: petición de int
 - Indica con bit en 1
- **ISR**: int en servicio
 - Indica con bit en 1
- **IMR**: máscara de int
 - enmascara con 1



Interrupciones del MSX88

44

- Los registros internos del PIC se sitúan a partir de la dirección 20H (del espacio de direcciones de E/S).
- Son accedidos con operaciones lectura y escritura en el espacio de E/S, es decir, mediante las instrucciones IN y OUT.

20H	EOI
21H	IMR
22H	IRR
23H	ISR
24H	INT0
...	...
2BH	INT7

Interrupciones del MSX88

Las Interrupciones de hardware preasignadas en el MSX88 son:

- **INT0** – tecla F10 - Produce una interrupción cada vez que se presiona F10
- **INT1** – Timer - Conectada a la salida del Timer
- **INT2** – Handshake - Conectada a una salida para Handshake
- **INT3** – DMA - Conectada a la salida del puerto a Impresora
- **INT4 a INT7** no usadas

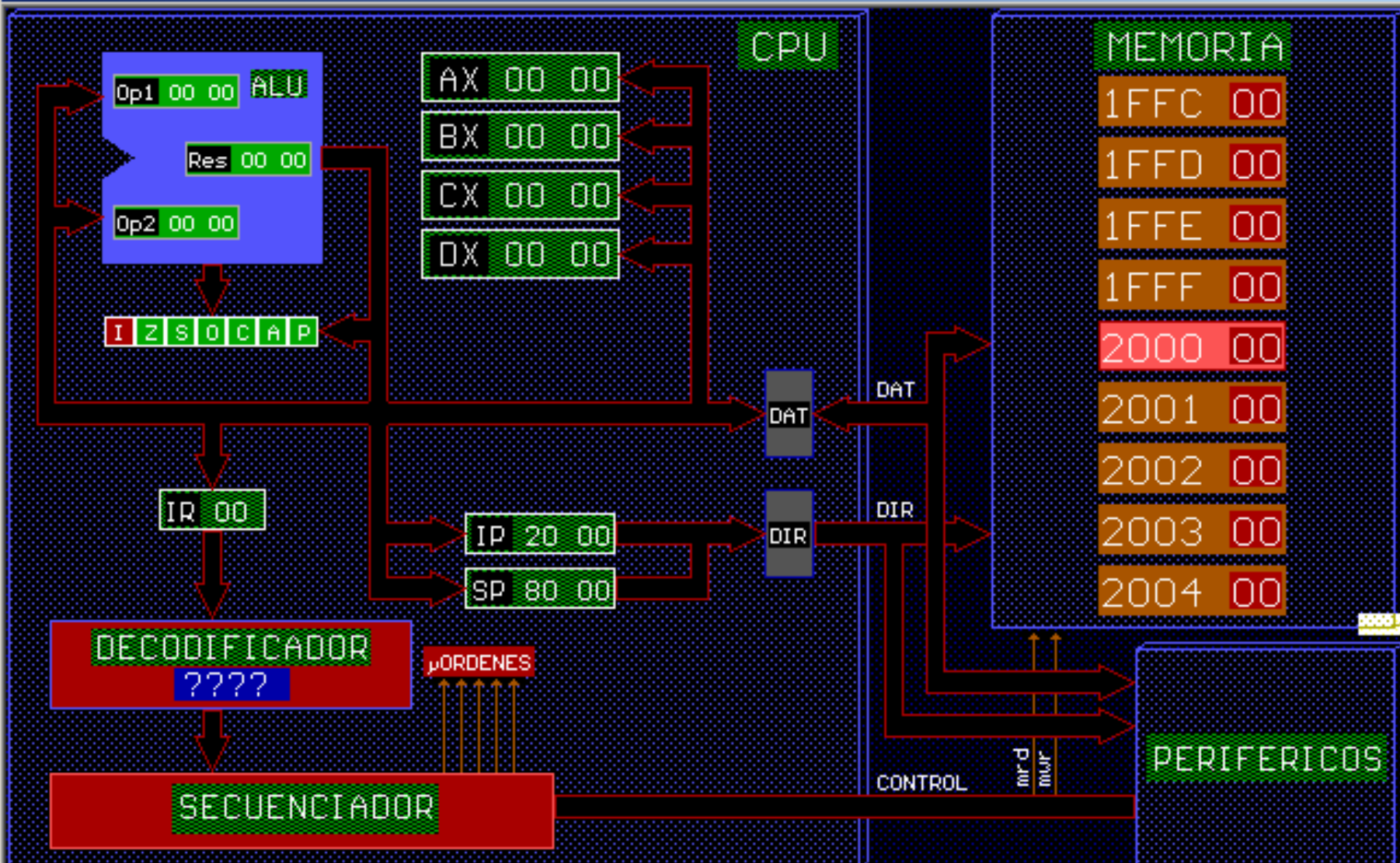
Referencias

- William Stallings, Capítulo 3.
- MSX88, Manual de usuario.
- Lecturas recomendadas
 - “Interrupciones en la arquitectura INTEL IA32”, Blázquez, J.M. 2004.

Arquitectura de Computadoras

Anexo Clase 2

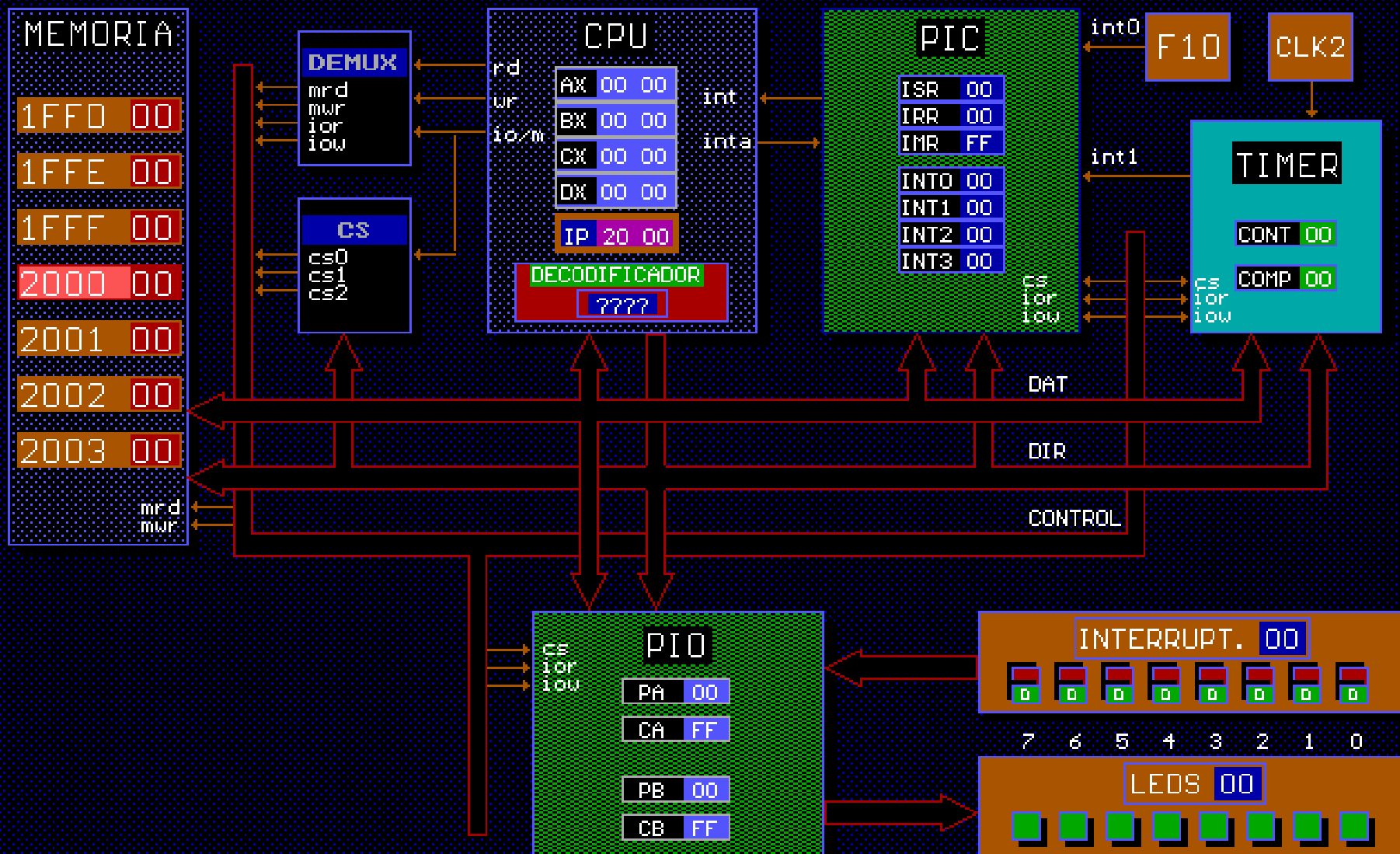
**Interrupciones. Ejercicios de
práctica y el simulador MSX88**



>_

ESTADO

-
P 00
C 00
V 15



>p1
>_

ESTADO
-
P 01
C 00
U 15

Operaciones de lectura/escritura

- Para direccionar memoria se utilizan las líneas del bus de control **mrd y mwr (memory read y memory write)**.

Estas líneas del bus de control son activadas por la unidad de control al decodificar una instrucción mov. Activa una u otra según la operación sobre la memoria.

- Para direccionar un dispositivo periférico se utilizan las líneas del bus de control **ior e iow (input-output read e input-output write)**.

Estas líneas del bus de control son activadas por la unidad de control al decodificar una instrucción in u out. Activa una u otra según la operación sobre el dispositivo de e/s.

Interrupción por software

```
      ORG 1000H
MSJ   DB "ARQUITECTURA DE COMPUTADORAS"
      DB "FACULTAD DE INFORMATICA"
      DB "UNLP"
FIN   DB ?
```

```
      ORG 2000H
Parámetros { MOV BX, OFFSET MSJ ; Puntero al comienzo del mensaje
             MOV AL, OFFSET FIN-OFFSET MSJ ; Cantidad de
                                             ; caracteres en el mensaje
```

```
      INT 7
      INT 0
      END
```

Lectura de datos desde el teclado

```
ORG 1000H
MSJ DB "INGRESE UN NUMERO : "
FIN DB ?
```

```
ORG 1500H
NUM DB ?
```

```
ORG 2000H
MOV BX, OFFSET MSJ ; Puntero a MSJ
MOV AL, OFFSET FIN - OFFSET MSJ ; Cantidad de caracteres
INT 7
MOV BX, OFFSET NUM ; Puntero a NUM
INT 6
MOV AL, 1; BX quedó cargado apuntando a NUM
INT 7
MOV CL, NUM
INT 0
END
```

```
PIC    EQU    20H
EOI     EQU    20H
N_F10   EQU    10
```

Interrupción con tecla F10

```
ORG 40
IP_F10  DW     RUT_F10
```

```
ORG 2000H
CLI
MOV AL, 0FEH
OUT PIC+1, AL
MOV AL, N_F10
OUT PIC+4, AL
MOV DX, 0
STI
LAZO:  JMP LAZO
```

```
ORG 3000H
RUT_F10: PUSH AX
          INC DX
          MOV AL, EOI
          OUT EOI, AL
          POP AX
          IRET

          END
```

ORG 40

IP_F10 DW RUT_F10

- ORG 40: vamos a instalar la interrupción en el lugar 10 de la tabla de vectores.
- Como cada entrada ocupa 4 bytes, la dirección es $4 * 10$.
- Aquí va la dirección de la primera instrucción del servicio que atiende a la interrupción. Esta dirección tiene una etiqueta RUT_F10.
- vemos que es 3000h.

MOV AL, 0FEH
OUT PIC+1, AL

➤ Estas dos instrucciones cargan en el registro IMR el valor FEh, poniendo el bit 0 en 0 y los restantes bits en 1, enmascarando todas las interrupciones menos la INT0 que corresponde a la tecla F10.

➤ 0FEh, lleva un 0 al principio para indicar que el resto de las letras corresponden a un número y no al nombre de una variable.

MOV AL, N_F10
OUT PIC+4, AL

- Estas dos instrucciones escriben, en el registro INT0 del PIC, el valor de la posición en la tabla de vectores, en éste registro se buscará dicha posición para la interrupción producida por F10.
- Recordemos que se accede al registro en la dirección 24h (PIC+4).

MOV DX, 0

- En el registro DX vamos a contar cuántas veces fué presionada la tecla F10.
- Acá se inicializa la cuenta en 0.

STI y CLI

- Las instrucciones activan y desactivan interrupciones.
- Cuando se activa la bandera I permite que por el terminal INT del procesador, se 'ingresen' interrupciones.
- Cuando se desactiva el bit, se ignora los cambios en el terminal INT.
- INT es un terminal del procesador.

MOV AL, EOI OUT EOI, AL

- La CPU debe indicarle al controlador PIC la culminación del servicio a cada **interrupción de hardware**.
- Por lo tanto al final de la rutina de servicio de interrupción se deberá escribir en el registro de comandos EOI, un comando (número) que indique el mencionado final atención.
- La dirección del registro coincide con el valor a escribir (OUT 20H, 20H)

IRET

- La instrucción IRET es una instrucción especial para retorno
 - se utiliza para retornar de las interrupciones de software y de hardware.
- La instrucción IRET es similar a una instrucción RET, por utilizar la pila, pero recupera una copia del registro de estado y la dirección de retorno.
- La instrucción IRET extrae 6 bytes de la pila: 4 para la dirección de retorno y 2 para el registro de estado.

TIMER (temporizador)

- Posee dos registros de 8 bits.
- **COMP**: registro de comparación que determina el módulo de la cuenta del timer.
- **CONT**: registro contador, muestra la cuenta de los pulsos de la señal aplicada a la entrada del periférico. Cuando coincide su valor con el del registro COMP provoca una señal de salida.
- Direcciones de registros: 10H y 11H
- Frecuencia: 1 Hz

Arquitectura de Computadoras

CURSO 2022

Turno:
Clase 3

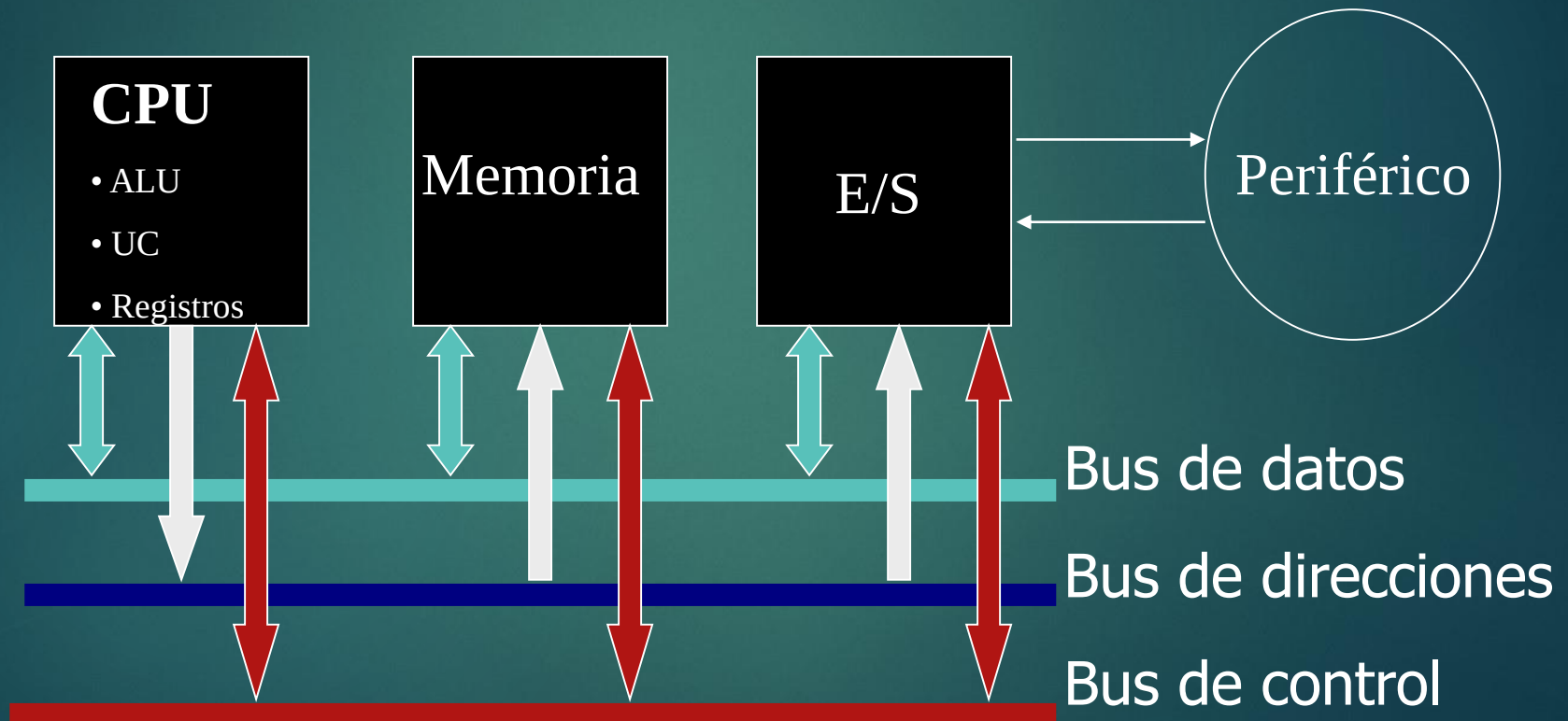
Resumen clase 3

Subsistema Entrada/salida

- Concepto básicos del Subsistema de E/S
- Módulo de E/S
- Tipos de puertos de E/S
- Organización interna de los puertos de E/S
- Acceso al subsistema de E/S
- Gestión de las transferencia de E/S
- Eficiencia en las técnicas de gestión de transferencias de E/S
- DMA – DMAC y canales de E/S

SUBSISTEMA E/S

En la primera clase de este curso se presentó el **modelo de Arquitectura de von Neumann** basado en 3 subsistemas: CPU, Memoria y E/S.



SUBSISTEMA E/S

El subsistema de E/S comprende los dispositivos que están conectados al bus del sistema y proveen los servicios de transferencia de datos con los Periféricos.

Debido a la gran variedad de Periféricos con los que se requiere intercambiar información, el subsistema de E/S tiene que ser lo suficientemente flexible para permitir:

- Transmisión de diferentes cantidades de datos
- Rango de velocidades de transmisión muy amplio.
- Diferentes formatos de dato y tamaño de palabra.

En general, todos los Periféricos son más lentos que la CPU y la Memoria. Los dispositivos que forman parte del subsistema de E/S permiten descongestionar el trabajo de la CPU.

SUBSISTEMA E/S

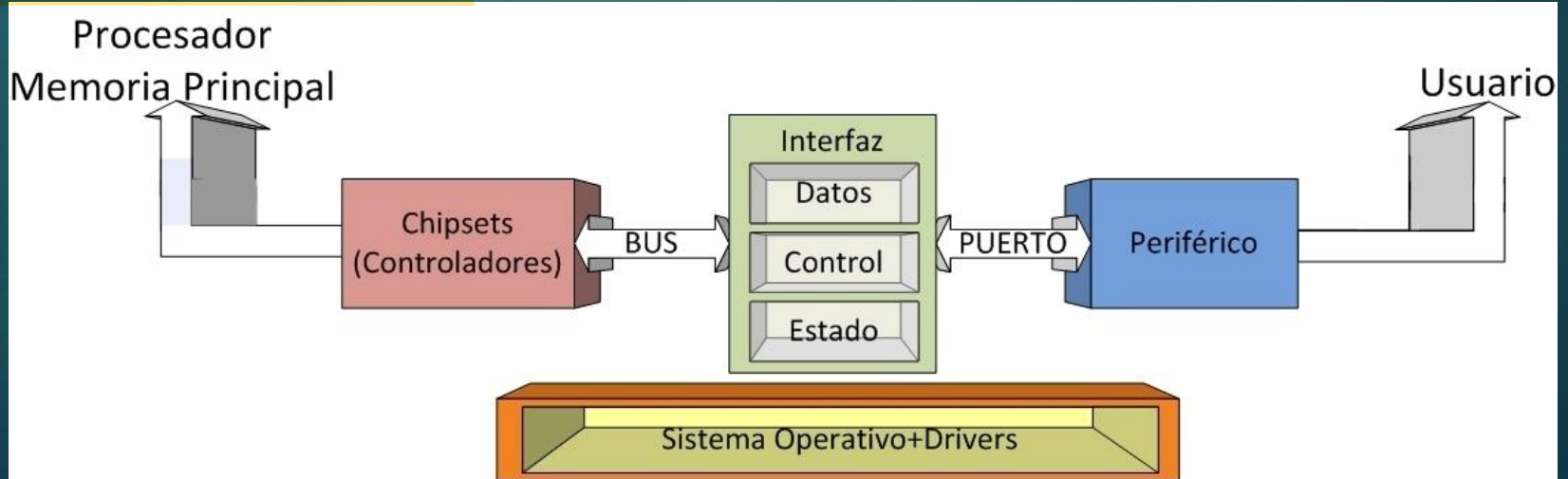
Dispositivos periféricos

- Existe una variedad muy grande de equipos periféricos. Algunos de los tipos más comunes son:
 - **Comunicación hombre-máquina:** monitor/pantalla, mouse, teclado
 - **Almacenamiento:** disco duro, CD, DVD
 - **Impresión:** impresora, escáner
 - **Comunicación con dispositivos remotos:** modem, placa de red
 - **Multimedia:** micrófono, parlantes
 - **Automatización y control:** sensores, alarmas, adquisición de datos

SUBSISTEMA E/S

Módulo de interfaz de E/S

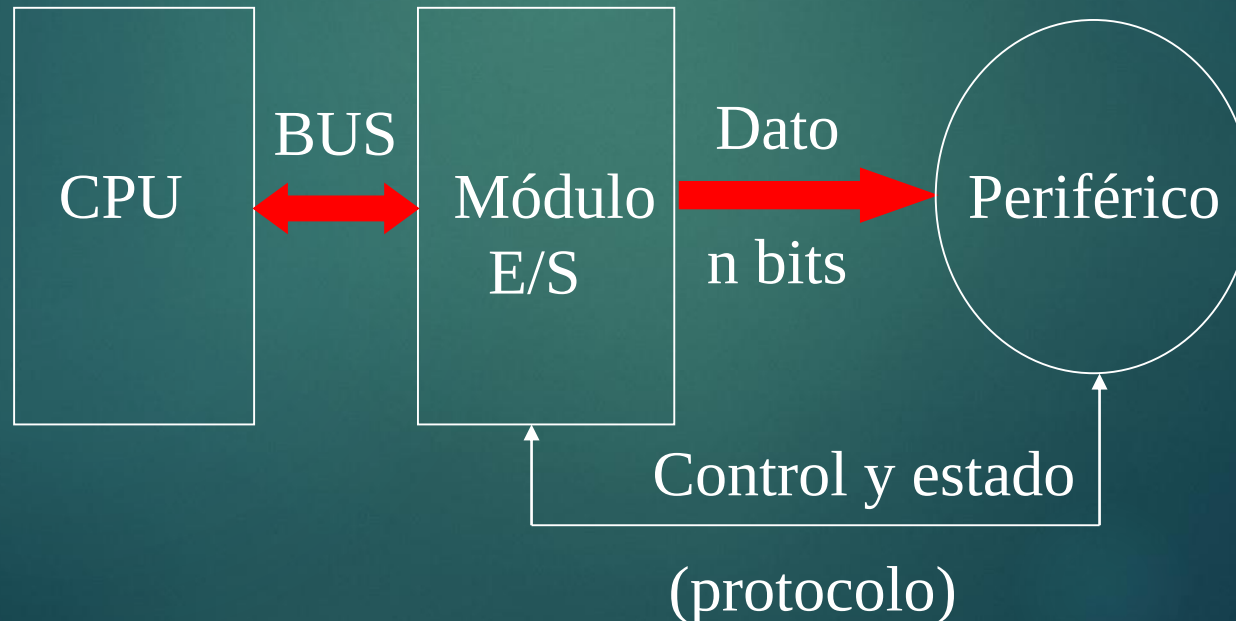
- Son los dispositivos más sencillos para implementar las transferencias de E/S con periféricos.
- Los puertos de E/S conectan la interfaz entre el procesador/memoria y un periférico.
- Son, por lo común, administrado por el SO a través de drivers específicos.



Módulo de E/S

Módulo de E/S

- El **esquema básico de transferencia de datos usando un Módulo de E/S sencillo** es el que se muestra en la figura siguiente.



Módulo de E/S

Módulo de E/S

- En la figura anterior se puede observar el Módulo de E/S comunicado con la CPU a través del Bus del Sistema.
- La conexión con el Periférico provee 2 tipos de informaciones:
 - 1.- Datos: Información útil a transferir
 - 2.- Control y Estado: información que permite realizar la transferencia (en lo posible libre de errores). Por ejemplo: sentido de la transferencia (de entrada, de salida), operación (de lectura, de escritura), estado del periférico (listo, no-listo, en falla).

Módulo de E/S

Módulo de E/S

➤ El Módulo de E/S debe ejecutar 2 tipos de comunicaciones:

1.- Hacia el Periférico (comunicaciones externas con la periferia):

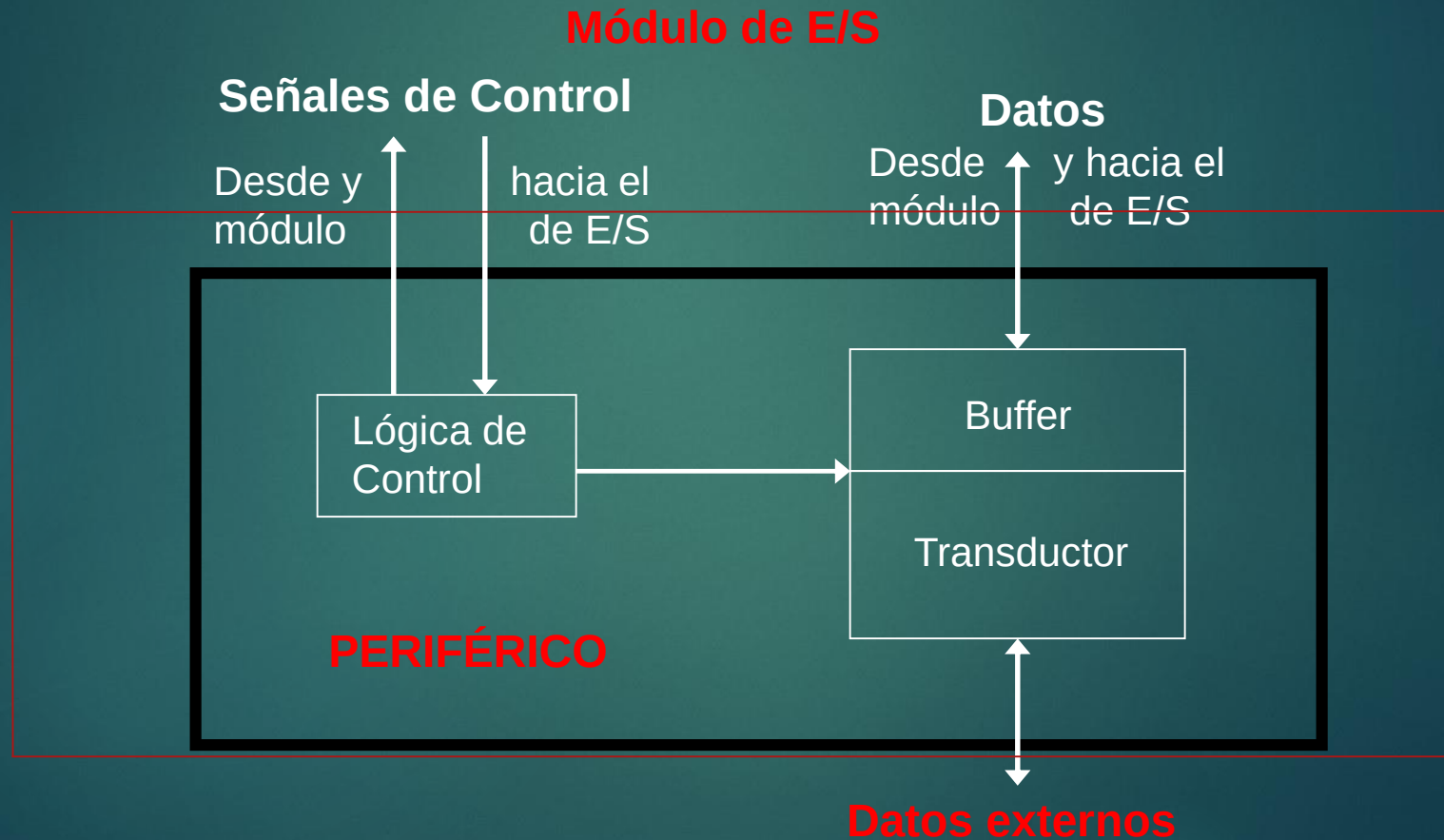
- Transferir datos con el periférico (incluye adaptación eléctrica).
- Controlar y temporizar uno o más dispositivos externos.
- Almacenar temporalmente datos (“buffer”).
- Detectar errores.

2.- Hacia el Bus (internas con CPU y Memoria):

- Interpretar las órdenes que recibe de CPU y transmitirlas al periférico.
- Transferir datos con la CPU (registros) y Memoria.
- Informar a la CPU del estado del periférico.

Módulo de E/S

Esquema simplificado de un Periférico conectado



Módulo de E/S

Módulo de E/S

- En el esquema anterior se puede apreciar que **un periférico tiene 2 bloques funcionales que manejan la comunicación entre el periférico y el módulo de E/S:**

1.- Hacia el exterior del periférico:

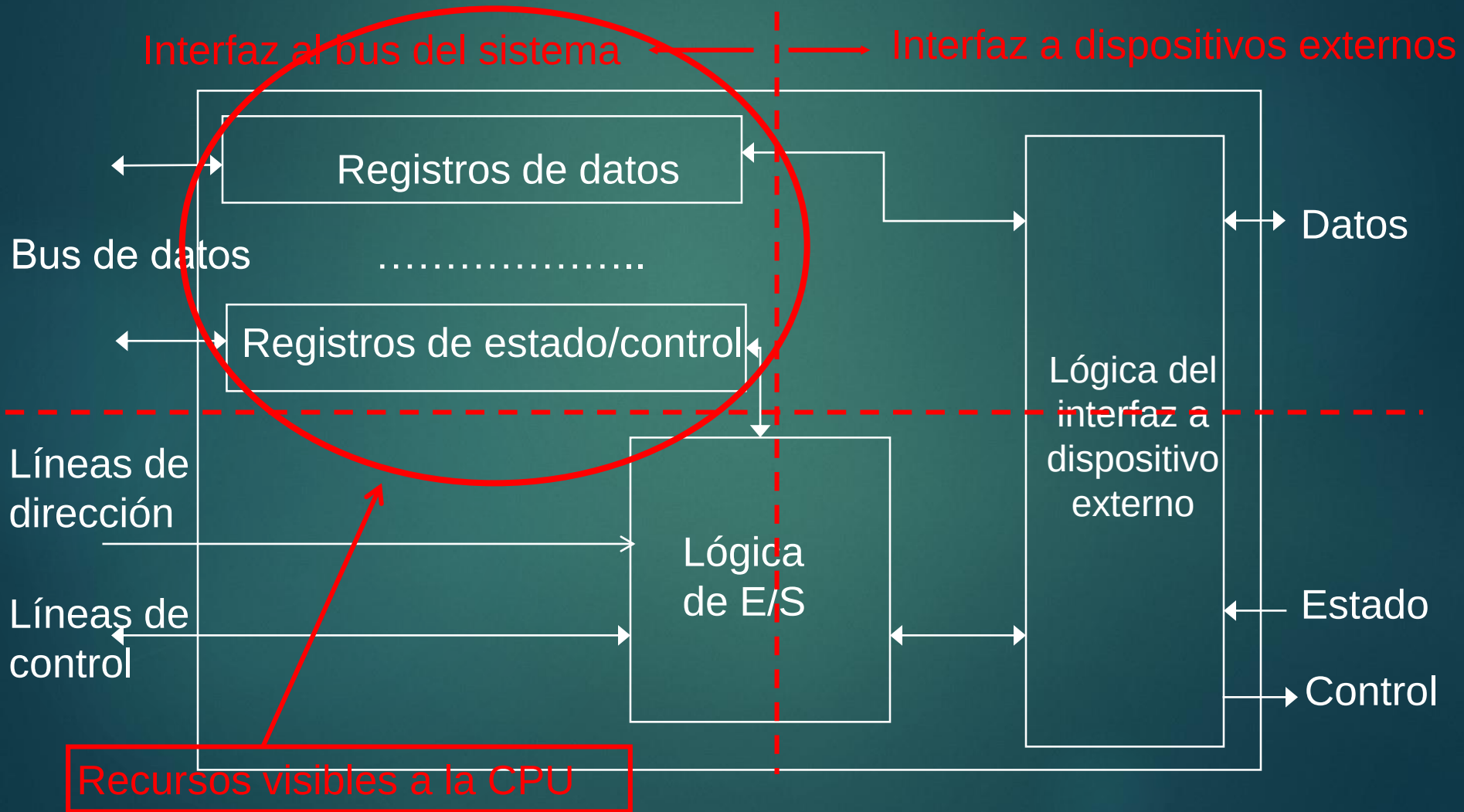
- Sección de manipulación de datos (buffer/trasductor). Almacena y convierte los datos a intercambiar con el periférico
- Sección de control y estado (Lógica de control): recibe y genera las señales de control y estado del periférico

2.- Hacia el Módulo de E/S :

- Sección de manipulación de datos
- Sección de manipulación de señales de control y de estado

Módulo de E/S

Esquema detallado de un Módulo de E/S



Módulo de E/S

13

Esquema detallado de un Módulo de E/S

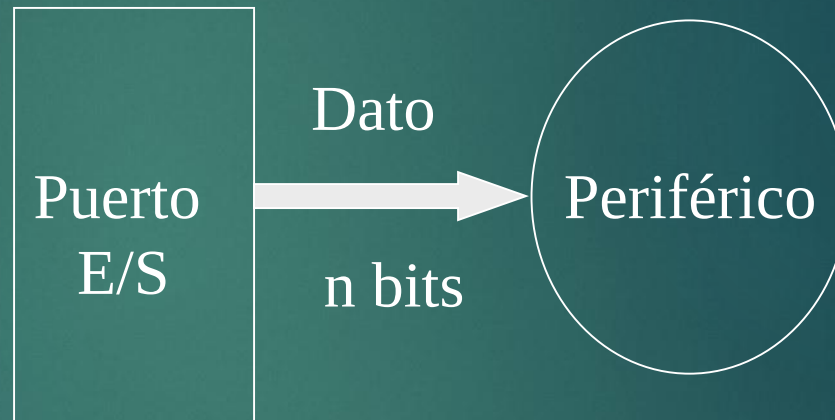
- En el esquema anterior se pueden apreciar los bloques funcionales que manejan la comunicación hacia el periférico y hacia el bus del sistema.
- Hacia el sistema los recursos que son “visibles” a la CPU (y al programador) son básicamente registros. Existen 2 tipos de registros:
 - Registros de datos: que contienen la información útil recibida desde, o transmitida a, el periférico.
 - Registros de control y estado: que controlan las características de la transferencia, y almacenan señales de estado de la comunicación (falla, no listo, etc.).

Puerto de E/S

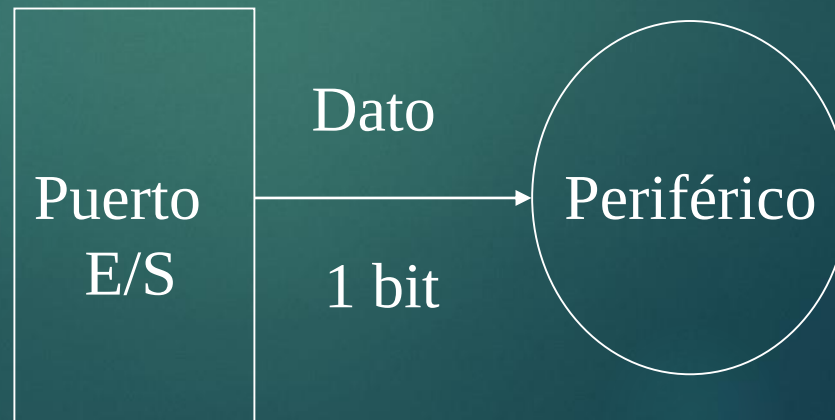
Tipos de Puerto de E/S básicos

Existen 2 tipos básicos: serie y paralelo.

PARALELO



SERIE



Puerto de E/S

15

Tipos de Puerto de E/S básicos

- Puerto paralelo: hay varias líneas de datos (n) que transfieren n bits simultáneamente entre el puerto de E/S y el periférico.
 - Ejemplos: impresora paralelo, scanner
- Se requiere disponer de una conexión mediante un cable que incluya al menos los n bits de datos, lo que lo hace un método bastante costoso.

Puerto de E/S

Tipos de Puerto de E/S básicos

- **Puerto serie:** hay 1 línea de dato para la transferencia entre el puerto de E/S y el periférico.
 - Ejemplos: impresora serie, red Ethernet, mouse, teclado, etc.
- Se requiere disponer de una conexión mediante un cable sencillo. El costo es mucho menor.
- Los datos deben serializarse (transmitirse de 1 bit por vez, uno a continuación del otro) lo que en teoría sería mucho más lento que transmitir de a n bits simultáneamente, como lo hace el puerto paralelo.

CPU y E/S

Subsistema de E/S

Las Puertas de E/S son un tipo particular de dispositivo de E/S. Hay otros dispositivos más complejos que proveen de otras prestaciones, además de la tarea básica de implementar la transferencia de los datos, por ejemplo:

- Ocultar las propiedades particulares del dispositivo periférico a la CPU: temporizados, formatos, electromecanismos, etc.
- Manejar múltiples dispositivos simultáneamente.
- Controlar varias funciones del dispositivo

CPU y E/S

18

Registros de un Puerto de E/S

Desde el punto de vista de la CPU, una operación de E/S requiere acceder a los registros internos del Módulo de Interfaz de E/S. Los registros pueden ser de lectura y/o escritura.

- Dentro de Módulo de E/S hay 2 tipos de registros:
 - de DATOS: interviene en la transferencia de entrada o de salida del dato a intercambiar entre el Sistema de cómputo y el periférico.
 - de CONTROL: registros que controlan y registran el funcionamiento del módulo, la transferencia, y el periférico.

CPU y E/S

19

Registros de un Puerto de E/S

- de Datos: la transferencia de un dato entre el Sistema de cómputo y el periférico consiste básicamente en:
 - Operación de entrada: lectura de un registro de dato de entrada (es decir un registro escrito por el periférico y leído por la CPU)
 - Operación de salida: escritura de un registro de dato de salida (es decir, un registro escrito por la CPU y leído por el periférico).
- de Control y Estado: la supervisión y/o control de la transferencia requiere de:
 - Control: adecuar la configuración del módulo para ajustar formatos, sincronizaciones, etc.
 - Estado: registrar el estado operativo del módulo y del periférico.

CPU y E/S

Acceso al Subsistema de E/S

Desde el punto de vista de la CPU, el Subsistema de E/S está compuesto por un conjunto de registros a los que accede para una operación de entrada o de salida.

Existen 2 técnicas de acceso a estos registros:

- Espacio de E/S compartido con memoria (memory-mapped)
- Espacio de E/S separada de la memoria

CPU y E/S

21

Espacio de E/S compartido con memoria (memory-mapped)

- En esta técnica los registros de los dispositivos de E/S y memoria comparten un único espacio de direcciones.
- Los registros de la E/S se comportan idéntico a una memoria de lectura/escritura.
- No hay instrucciones específicas para E/S, se usan las mismas instrucciones de movimiento de datos a memoria.

Ej: Mov Reg_dato, AL

Donde Reg_dato es la dirección de un registro de salida del módulo de E/S (dirección idéntica a la correspondiente a una posición de memoria).



CPU y E/S

22

Espacio de E/S separado de memoria (modelo Intel)

- En esta técnica los registros de los dispositivos de E/S y la memoria están en diferentes espacios de direcciones.
- Dado que el bus de direcciones es compartido por la memoria y el subsistema de E/S, se requieren señales de control adicionales para identificar a donde está accediendo la CPU: a la memoria, o a la E/S.
- Hay instrucciones específicas de E/S, distintas de las instrucciones de acceso a la memoria.



CPU y E/S

Espacio de E/S separado de memoria (modelo Intel)

- El procesador dispone de instrucciones específicas para acceder a los registros que están en el subsistema de E/S.
- Cuando se ejecutan estas instrucciones específicas, en el bus de control se identifica el acceso al mapa de direcciones de E/S. Para el resto de las instrucciones en el bus de control se identifica el acceso a la Memoria.

Ejemplos:

- 1) Entrada: IN dest, fuente
Donde dest es AL o AX (8 o 16 bits) y fuente un número de 8 bits sin signo (0 y 255) o DX (número entre 0 y 65535).
- 2) Salida: OUT dest, fuente
Donde fuente es AL o AX (8 o 16 bits) y dest un número de 8 bits sin signo (0 y 255) o DX (número entre 0 y 65535).

Gestión de la transferencia

Desde el punto de vista de la gestión para transferir datos entre el Sistema de cómputo y el periférico, existen 3 estrategias básicas de implementación:

- E/S Programada y espera de respuesta
 - E/S Programada y administrada por interrupción
 - E/S con acceso directo a memoria (DMA)
- Las 2 primeras opciones requieren intervención directa de la CPU, es decir, que la CPU participa en la transferencia de todos los datos (byte o word) a transferir.

Gestión de la transferencia

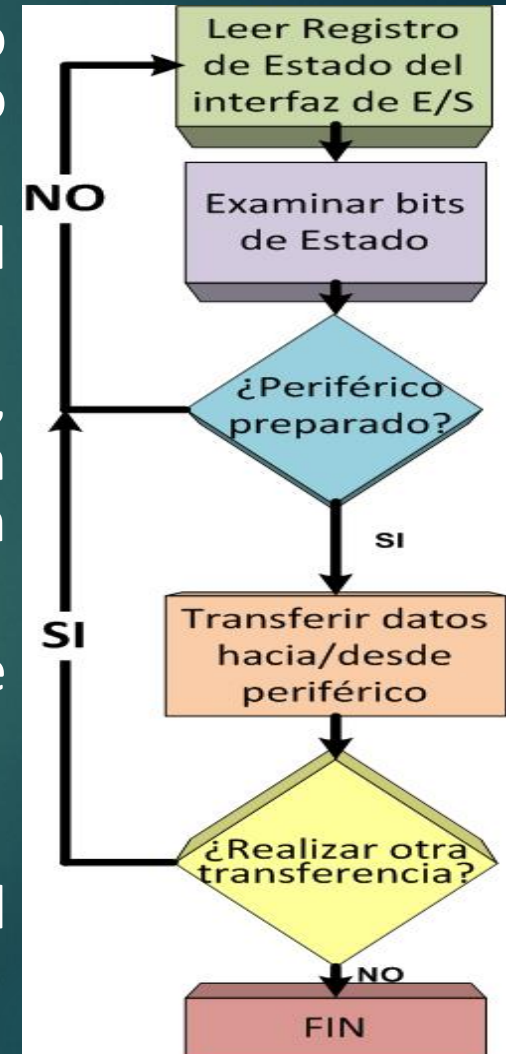
E/S Programada y espera de respuesta

- La CPU interviene directamente en la transferencia de cada unidad de información (byte, word) con el módulo.
- Es decir que la CPU tiene control casi directo sobre la operación de E/S. Entre otras acciones hace:
 - Comprueba el estado del dispositivo
 - Envía los comandos requeridos (por ejemplo de lectura, escritura)
 - Realiza la transferencia de todos los datos (de a uno)
- En cada dato que es transferido, la CPU espera que el módulo E/S termine la operación, típicamente que el periférico “acepte” el dato.
- Durante la espera, la CPU permanece ociosa (no deseable).

Gestión de la transferencia

La secuencia de acciones que ejecuta la CPU son :

1. La CPU verifica el estado de periférico (preparado/no-preparado) leyendo un registro del módulo de interfaz.
2. Examina el estado del periférico chequeando el bit (o bits) que identifican dicho estado.
3. Si el dispositivo no está listo (por ejemplo bit=0), la CPU vuelve al paso 1. Este lazo significa que la CPU “espera” hasta que el periférico se pone en “preparado”, es decir, listo para la transferencia.
4. Cuando el dispositivo está listo, la CPU transfiere 1 dato hacia o desde el módulo de interfaz.
5. Si hay más datos que transferir vuelve al paso 1.
- 6.- Si se completó la transferencia, termina el servicio de E/S.



Gestión de la transferencia

E/S Programada y administrada por interrupción

- La CPU interviene directamente en la transferencia de cada unidad de información (byte, word) con el módulo.
- Cada vez que el módulo está listo (o completó una transferencia) avisa a la CPU con un pedido de interrupción.
- Ahora la CPU no tiene que ejecutar el lazo de comprobación del estado del módulo (pasos 1, 2 y 3 de la figura anterior). Solo inicia la transferencia al recibir el pedido de interrupción del periférico.
- Durante el tiempo que el periférico no está listo, la CPU no tiene que esperar, puede seguir ejecutando otra tarea.

Gestión de la transferencia

28

E/S Programada y administrada por interrupción

A continuación se describen la **secuencia de acciones para una transferencia de entrada de 1 dato (por ejemplo, 1 caracter) administrado por interrupción.**

- La CPU inicia la operación de lectura (entrada) enviando una orden de lectura (READ) al módulo de E/S.
- El módulo de E/S solicita el dato al periférico.
- El periférico busca el dato, mientras la CPU continúa con sus tareas.
- Cuando el módulo E/S tiene el dato enviado por el periférico emite un pedido de interrupción a la CPU.
- La CPU detecta el pedido, interrumpe el proceso, y bifurca al servicio de la interrupción.
- Durante la interrupción, la CPU lee el dato desde el módulo de E/S.
- La CPU retorna a la tarea interrumpida.

Gestión de la transferencia

29

Interrupciones en operaciones de E/S

Cuando hay varios dispositivos periféricos, con esta forma de administración de las transferencias de E/S se requiere poder identificar la fuente de interrupción.

Como se vió durante el análisis de Interrupciones, existen varias estrategias distintas para identificar la fuente de la interrupción:

- 1) Diferentes líneas de interrupción
- 2) Una sola línea de interrupción y encuesta por software
- 3) Una sola línea de interrupción con conexión en cadena (daisy chain) tipo “hard poll” (encuesta por hardware)
- 4) Una sola línea de interrupción y vectorizado

Gestión de la transferencia

30

1) Diferentes líneas de interrupción

- Se dispone de una línea de interrupción por cada dispositivo.
- Es sencilla de implementar.
- Hay una limitación en la cantidad de dispositivos a conectar debido a la cantidad restringida de señales de interrupción que puede manejar la CPU.

2) Una sola línea de interrupción y encuesta por software

- Se dispone de 1 sola línea de interrupción para todos los dispositivos.
- Cuando ocurre el pedido de interrupción, la CPU tiene que consultar a cada dispositivo (es decir, a cada módulo de E/S) para determinar quien fue el demandante.
- Este esquema de encuesta por software (también conocido como “polling”) puede resultar sumamente lento.

Gestión de la transferencia

31

3) Una sola línea de interrupción con conexión en cadena

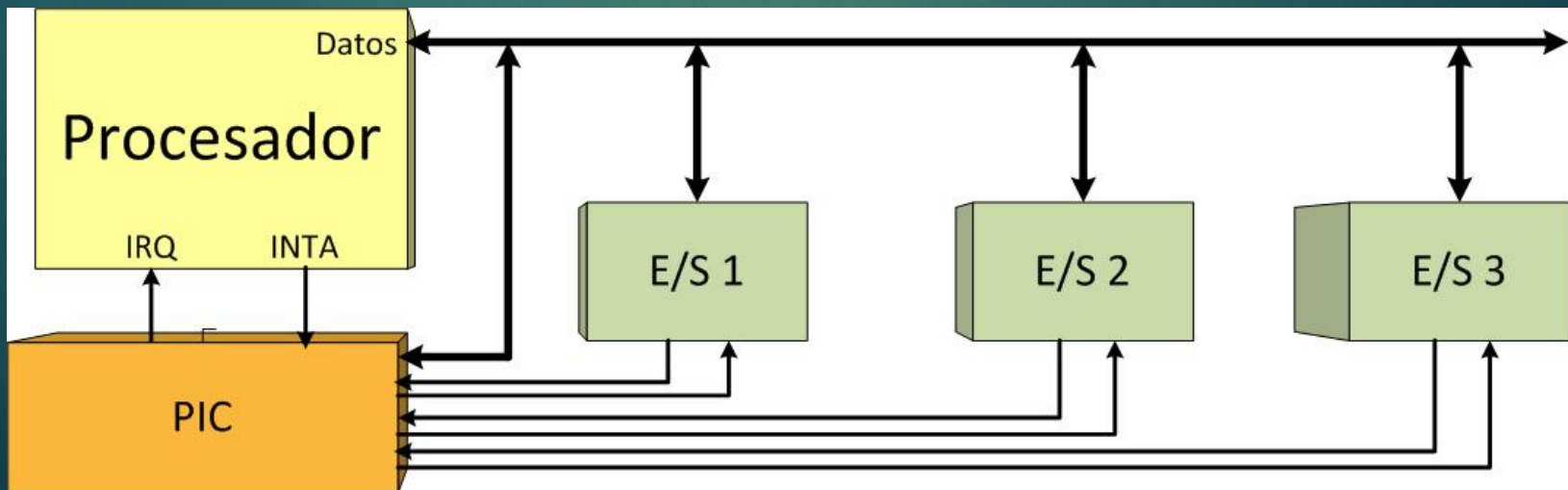
- Se dispone de 1 sola línea de interrupción INTR para todos los dispositivos.
- La línea de respuesta de la CPU INTA (reconocimiento de interrupción) se conecta encadenadamente a todos los módulos (conexión tipo “margarita” o “Daisy chain”).
- Una vez enviada la confirmación de parte de la CPU, el módulo que está más adelante (más próximo a la CPU) en la conexión Daisy chain responderá colocando un vector (palabra), en el bus, que lo identifica.
- Si hay otros pedidos de interrupción más abajo del que respondió, deberán esperar la terminación del servicio del que respondió.

Gestión de la transferencia

32

4) Una sola línea de int. y vectorizado

- Se dispone de 1 sola línea de interrupción INTR para todos los dispositivos.
- Un controlador dedicado (PIC) provee el vector que identifica la fuente de interrupción.
- Las líneas de interrupción tienen un orden de prioridad, las líneas con más prioridad pueden interrumpir a las líneas con menor prioridad.
- Si existe un maestro del bus, solo él puede interrumpir.



Gestión de la transferencia

33

En general:

- Las operaciones de E/S administradas por interrupción son más eficientes que las programadas con espera.
- Ambas técnicas requieren la intervención directa de la CPU.
- Al tener que intervenir la CPU en la transferencias de los datos se presentan 2 problemas:
 - La velocidad de transferencia depende de la capacidad de la CPU de atender estas tareas. Aunque la CPU es muy rápida, puede ser que en determinadas circunstancias no sea capaz de administrar varias transferencias simultáneamente.
 - La CPU puede permanecer ocupada mucho tiempo durante la operación, sin poder hacer otras tareas.
- Además, si el volumen de datos a transferir es grande, el tiempo de ocupación de la CPU crece también.

Gestión de la transferencia

34

Para analizar el comportamiento de la CPU en transferencias de E/S, vamos a analizar la respuesta de la CPU en 2 casos:

- 1) Transferencia hacia un periférico lento (impresora de 20 páginas/minuto)
- 2) Transferencia hacia un periférico rápido (disco de 10Mb/s)

Para los 2 casos, se analizará la respuesta del sistema para transferencias de E/S manejadas por la CPU con espera de respuesta y administrada por interrupción.

Gestión de la transferencia

Sobre la CPU

- Se tiene un procesador con un reloj de 200 MHz (período del reloj = 5 ns).
- El procesador, en promedio, necesita 2 ciclos de reloj por instrucción. Este parámetro se conoce como CPI. En este caso $CPI=2$
- Es decir, que una instrucción tarda, en promedio:

$$\text{Ciclo de instrucción} = 2 \times 5 \text{ ns} = 10 \text{ ns} = 10^{-8}$$

- El procesador tiene, por lo tanto, una capacidad de procesamiento de:

Nº de instrucciones por segundo = $1 / \text{tiempo de ciclo de instrucción}$

Nº instrucciones por segundo = $1/10^{-8}$

Nº Instrucciones por segundo = 100 millones de instrucciones por segundo (100 Mips)

Gestión de la transferencia

36

Caso 1: Transferencia a una Impresora

- Se tiene que imprimir (operación de salida) un archivo de 10 Kbytes en una impresora láser de 20 páginas por minuto.

- Considerando que:

1 página tiene (estimado) 3.000 letras (es decir, 3000 caracteres)

y:

1 carácter = 1 byte

- La impresora imprime:

$20 \text{ ppm} = 20 \text{ pag/min} \times 3000 \text{ car/pag} = 60.000 \text{ caracteres por minuto}$

Es decir, que la velocidad de transferencia de la impresora es de :

$V_t = 60000 \text{ car./min} = 60000 \text{ car./60 seg.} = 1000 \text{ car./s} = 1 \text{ Kbyte/s}$

Gestión de la transferencia

37

Caso 1: Transferencia a una impresora

a) E/S con espera de respuesta

- La CPU entra en un bucle y envía un nuevo byte cada vez que la impresora está preparada para recibirlo
- Si la impresora tarda 1 seg en imprimir 1 Kbyte (ya que la tasa de transferencia es de 1 kbyte/seg), necesitará:

Tiempo total de transferencia = 10 seg para 10 Kbytes.

- Conclusión:

La CPU está ocupada con la operación de E/S durante 10 seg.

- Observación:

Ya que la velocidad de la CPU es de 100MIPs, en ese tiempo la CPU podría haber ejecutado:

Nºinstrucciones en 10 seg. = 1000 millones de instrucciones

Gestión de la transferencia

38

Caso 1: Transferencia a una impresora

b) E/S administrada con interrupciones

- La impresora genera una interrupción cada vez que está preparada para recibir un nuevo byte.
- Si la gestión de la interrupción (que se llama ATI) requiere 10 instrucciones (entre las que se incluyen salvar contexto, comprobar estado, transferir byte, restaurar contexto, y retornar), entonces:

Para transferir 10 Kbytes se requiere ejecutar 10.000 veces la ATI.

- El tiempo que dura el servicio de la transferencia es, aproximadamente, el requerido por las 10.000 ATI. Recordando que 1 instrucción tiene un tiempo de ejecución de 10^{-8} seg

$$\begin{aligned}\text{Tiempo total de transferencia} &= 10.000 \text{ ATI} = 10.000 \times 10 \text{ instr} = \\ &= 10.000 \times 10 \times 10^{-8} \text{ seg} = 100.000 \times 10^{-8} \text{ seg} = 10^{-3} \text{ seg} = 0,001 \text{ seg}\end{aligned}$$

- Conclusión:

La CPU está ocupada con la operación de E/S durante 0,001 seg.

Gestión de la transferencia

39

Conclusiones Caso 1 - Transferencia a una impresora:

- De los resultados anteriores en el uso de la CPU para el caso de una transferencia con un dispositivo relativamente lento, usando las 2 técnicas de E/S, con espera y administrada por interrupción, se pueden sacar las siguientes conclusiones.
- La transferencia dura 10 segundos, porque es el tiempo que tarda la impresora en imprimir los 10.000 caracteres.
- El tiempo de uso de la CPU en ambos casos es:
 - Caso a) con espera: 10 seg.

La CPU está los 10 segundos ocupada administrando la transferencia.

- Caso b) administrada por interrupción: 0,001 seg

De los 10 segundos que dura la transferencia, la CPU le ocupa solo 0,001 segundo en administrarla.

- Es decir, que la E/S por interrupciones reduce en 10.000 veces el tiempo que la CPU está ocupada gestionando la impresora, y por lo tanto, es mucho más eficiente.

Gestión de la transferencia

Caso 2: Transferencia a un disco

- Se tiene que transferir un archivo de memoria a disco de 10 Mbytes

- Considerando que:

El disco posee una velocidad de transferencia de 10 MB/s (1 byte cada 10^{-7} seg ó 100 nanoseg)

significa que:

Tiempo total de transferencia= 1 segundo.

Gestión de la transferencia

41

Caso 2: Transferencia a un disco

a) E/S con espera de respuesta

- La CPU entra en un bucle y envía un nuevo byte cada vez que el disco está preparado para recibirlo
- Si el disco transfiere 10 Mbytes/seg, necesita 1 seg para recibir un archivo de 10 Mbytes.

Tiempo total de transferencia = 1 seg para 10 Mbytes.

- Conclusión:

La CPU está ocupada con la operación de E/S durante 1 seg.

- Observación:

Ya que la velocidad de la CPU es de 100MIPs, en ese tiempo la CPU podría haber ejecutado:

Nºinstrucciones en 1 seg. = 100 millones de instrucciones

Gestión de la transferencia

42

Caso 2: Transferencia a un disco

b) E/S administrada con interrupciones

- El disco genera una interrupción cada vez que está preparado para recibir un nuevo byte
- Si la gestión de la interrupción (que se llama ATI) requiere 10 instrucciones (entre las que se incluyen salvar contexto, comprobar estado, transferir byte, restaurar contexto, y retornar), entonces:

Para transferir 10 Mbytes tenemos que ejecutar 10^7 veces la ATI

- El tiempo que dura el servicio de la transferencia es, aproximadamente, el requerido por las 10.000.000 ATI. Recordando que 1 instrucción tiene un tiempo de ejecución de 10^{-8} seg

$$\begin{aligned}\text{Tiempo total de transferencia} &= 10.000.000 \text{ ATI} = 10^7 \times 10 \text{ instr} = \\ &= 10^7 \times 10 \times 10^{-8} \text{ seg} = 1 \text{ seg}\end{aligned}$$

- Conclusión:

La CPU está ocupada con la operación de E/S durante 1 seg.

Gestión de la transferencia

43

Conclusiones Caso 2 – Transferencia a un disco:

- De los resultados anteriores en el uso de la CPU para el caso de una transferencia con un dispositivo relativamente rápido, usando las 2 técnicas de E/S, con espera y administrada por interrupción, se pueden sacar las siguientes conclusiones.
- La transferencia dura 1 segundo, porque es el tiempo que tarda el disco en transferir los 10Mbytes.
- El tiempo de uso de la CPU en ambos casos es:
 - Caso a) con espera: 1 seg.

La CPU está 1 segundo ocupada administrando la transferencia.

- Caso b) administrada por interrupción: 1 seg

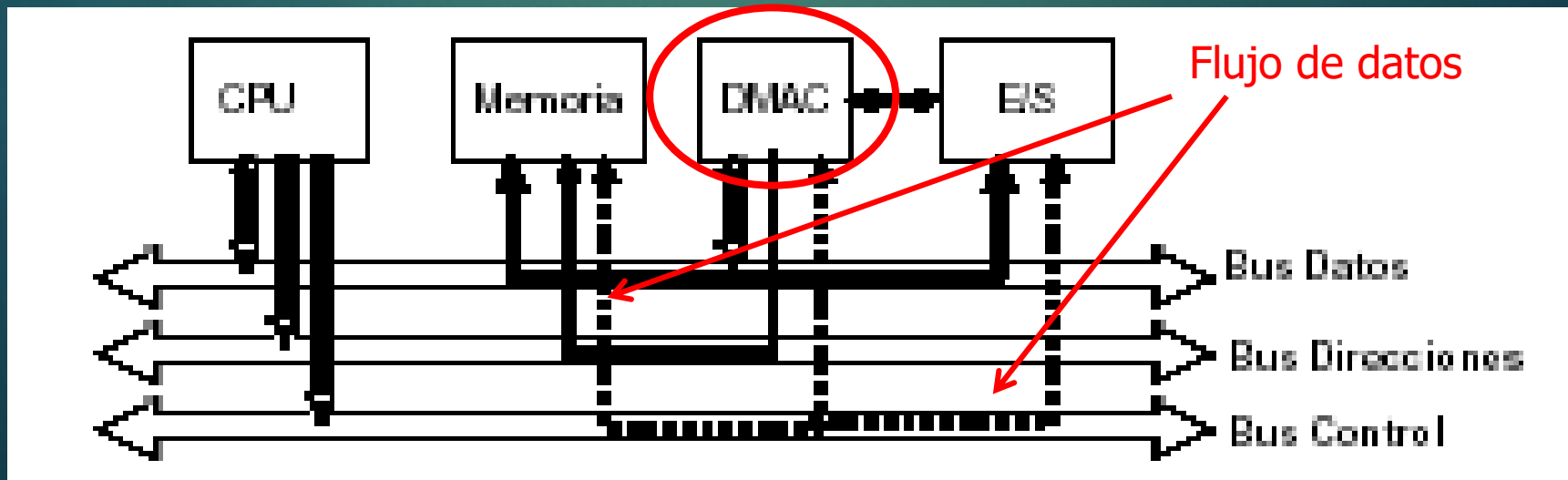
La CPU está 1 segundo ocupada administrando la transferencia.

- Es decir, que no hay diferencia entre las 2 técnicas. En ambas la CPU está ocupada 1 segundo el 100% del tiempo.
- Si la velocidad del dispositivo fuera mayor, la CPU no podría hacerla.

Acceso directo a memoria (DMA)

44

- El Acceso Directo a Memoria (DMA) es una técnica de transferencia de datos entre periférico y Memoria sin intervención directa de la CPU.
- Comúnmente llevada a cabo por un “Controlador de DMA” (o DMAC) específico encargado de llevar a cabo la transferencia.
- Físicamente está ubicado como se muestra a continuación.



Acceso directo a memoria (DMA)

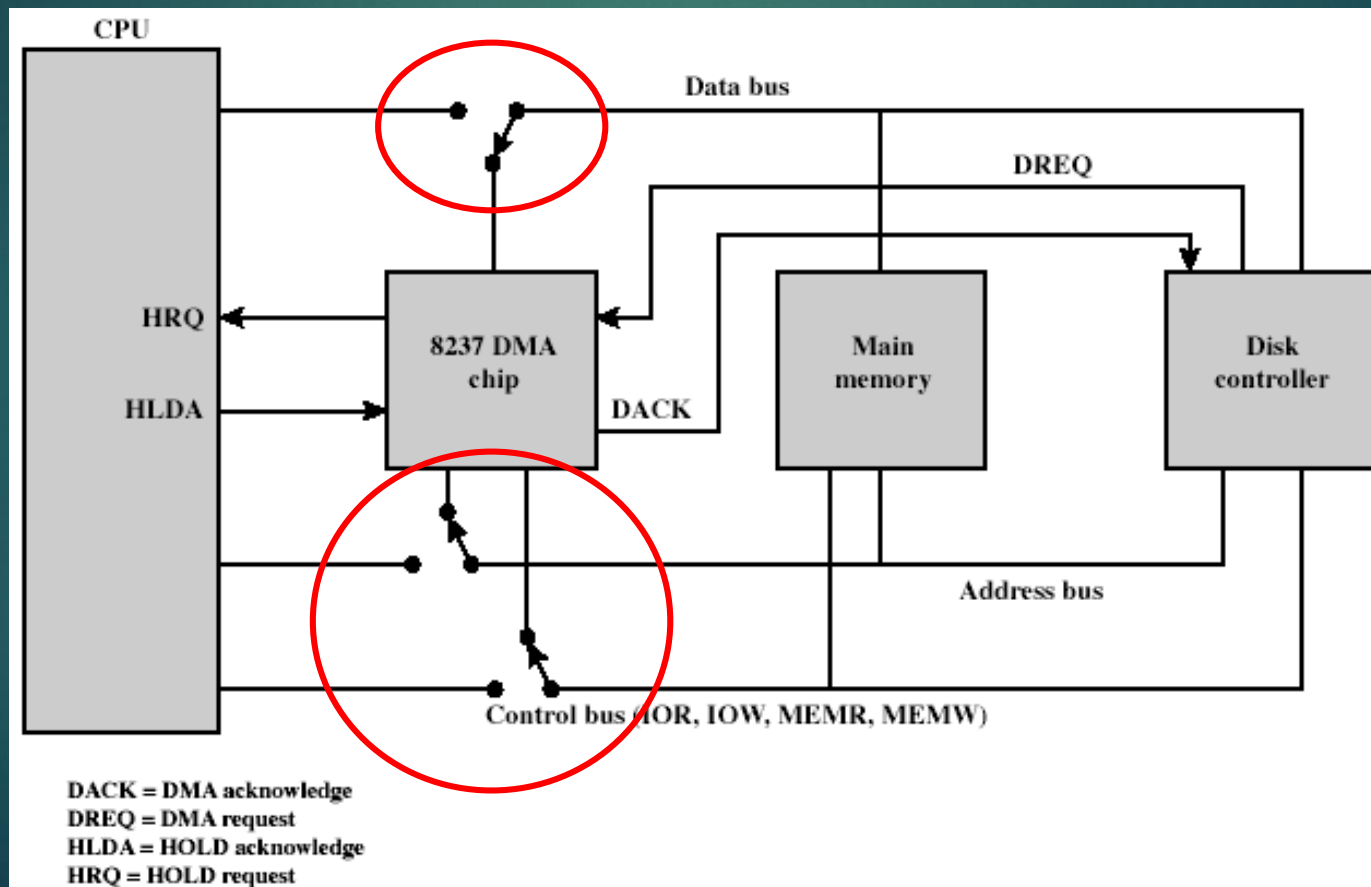
45

- Dado que la transferencia por DMA requiere el uso del Bus, tanto el DMAC como la CPU pueden tomarlo. El DMAC y la CPU “compiten” por el uso del Bus.
- Cuando el DMAC toma el Bus, actúa como “master” durante la transferencia por DMA, y debe ser capaz de:
 - Solicitar el uso del bus mediante las señales y la lógica de arbitraje necesarias
 - Especificar la dirección de memoria sobre la que se realiza la transferencia
 - Generar las señales de control del bus
 - Especificar el tipo de operación (lectura/escritura)
 - Generar las señales de sincronización de la transferencia

Acceso directo a memoria (DMA)

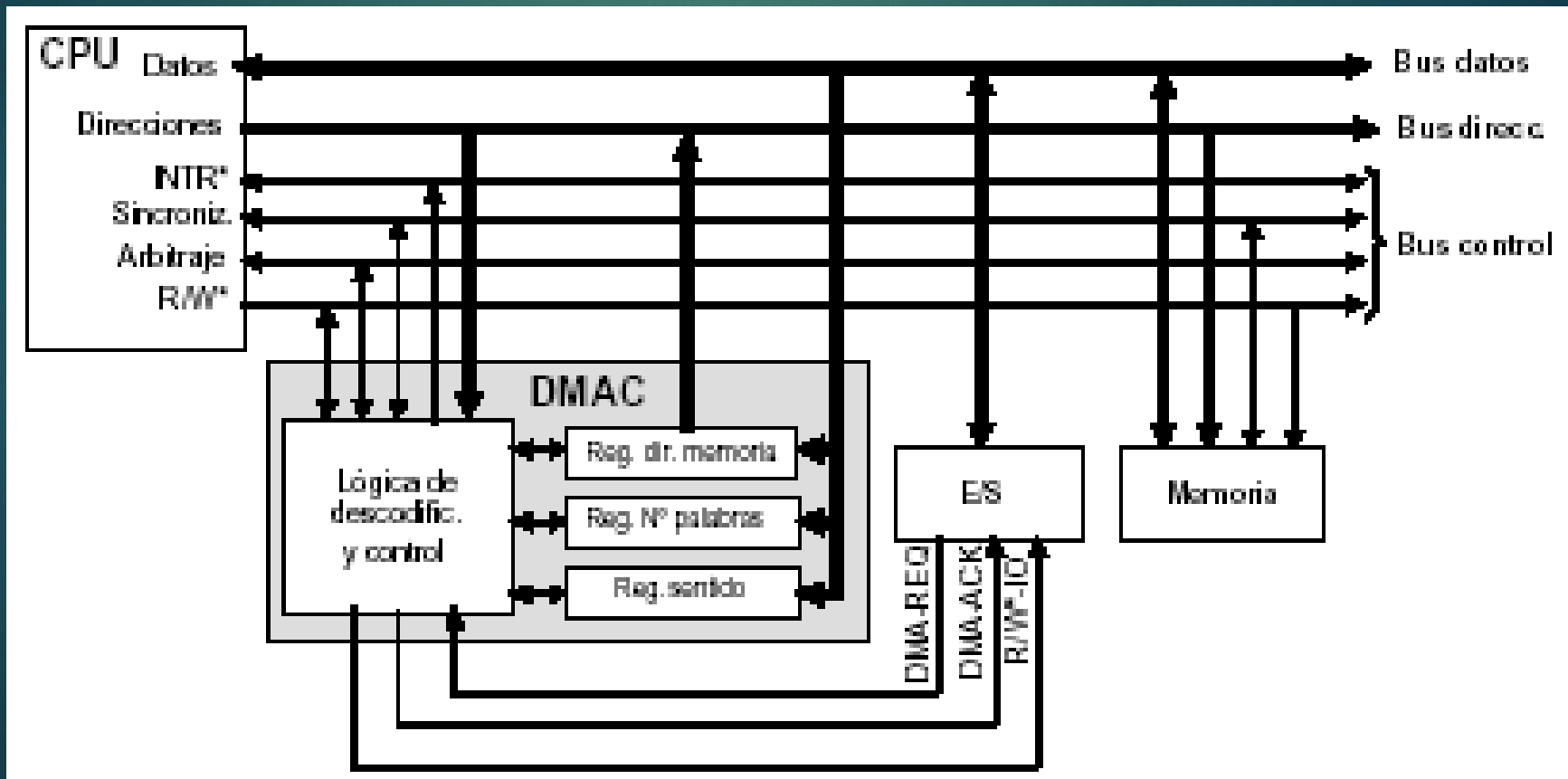
46

- Cuando la CPU entrega el bus al DMAC (8237 DMA chip en la figura), se desconecta lógicamente del mismo, y es el DMAC el que toma el control del bus.



Acceso directo a memoria (DMA)

- En la figura siguiente se muestra un detalle de la forma en que se conecta al Bus del sistema y de su estructura interna.



Acceso directo a memoria (DMA)

- El proceso de transferencia requiere realizar una serie de acciones o fases relativamente complejas. Las principales son:
 - 1.- Fase de Inicialización
 - 2.- Fase de ejecución de la transferencia
 - 3.- Fase de finalización y análisis de la transferencia

Acceso directo a memoria (DMA)

➤ 1.- Fase de inicialización

En la fase de inicialización la CPU debe configurar el módulo de E/S y el DMAC con los parámetros de la transferencia.

- Inicialización interfaz de E/S:
 - Tipo de transferencia (lectura/escritura)
 - Configuración del periférico
 - Otra información de control para el periférico (por ejemplo si es un disco se especifica el número de pista, sector, etc.)
- Inicialización DMAC
 - Nº de bytes o palabras a transferir
 - Tipo de transferencia (lectura/escritura)
 - Dirección de memoria inicial para la transferencia
 - Otra información la transferencia.

Acceso directo a memoria (DMA)

50

➤ 2.- Fase de ejecución de la transferencia

- Cuando el periférico está listo, pide al DMAC iniciar la transferencia mediante una señal física.
- Cuando el DMAC recibe el pedido del periférico, pide el control del bus mediante alguna señal especial a la CPU. La CPU típicamente dispone de algunas señales destinadas a implementar las transferencias por DMA.
- Cuando reconoce el pedido de DMA, la CPU entrega (libera) el bus y se “desconecta” (ya no controla el bus) lógicamente del mismo.
- La CPU avisa al DMAC que liberó el bus mediante otra señal especial.
- Al liberar la CPU el bus, el DMAC toma el control del bus y ejecuta la transferencia hasta terminarla.

Acceso directo a memoria (DMA)

51

- El DMAC avisa al periférico que puede iniciar la transferencia.
- El periférico comienza a transferir los datos, a través de bus, con la memoria, de a uno por vez.
- La transferencia implica que:
 - Bus master: DMAC + Periférico - Bus slave: Memoria
- Después de la transferencia de cada palabra se actualizan los registros del DMAC:
 - Nº de bytes faltantes (o cuenta de los que se transfirieron)
 - Próxima dirección de memoria a donde guardar el dato (anterior o posterior de la corriente)
- Cuando el número de bytes faltantes es igual a 0 significa que transfirió todos los datos y terminó la transferencia.

Acceso directo a memoria (DMA)

52

➤ 3.- Finalización de la transferencia

- Una vez que termina la fase 2, el DMAC libera el bus y le avisa a la CPU por medio de una señal física.
- La CPU retoma el control del bus.
- El DMAC suele activar, además, una señal de interrupción para indicar a la CPU la finalización de la operación de E/S solicitada.
- La CPU, mediante la interrupción, verifica el resultado de la transferencia vía los registros internos del DMAC. Algunos resultados a verificar son:
 - Transferencia OK/fallida?
 - Errores? Tipo de errores?
 - Estado periférico

Acceso directo a memoria (DMA)

53

Ventajas e inconvenientes del DMA

- La principal ventaja es la eficiencia, dado que la CPU se libera de tener que controlar la transferencia de los datos. Solo prepara la transmisión, y verifica el resultado de la misma.
- La principal desventaja se origina en el uso del bus. Como las transferencias por DMA pueden tener mayor prioridad que la CPU, se puede degradar el rendimiento de la CPU si el DMAC hace uso intensivo del bus.

Acceso directo a memoria (DMA)

54

Ventajas e inconvenientes del DMA

- Sin embargo, no necesariamente la CPU necesite todo el tiempo el bus porque:
 - En el caso de Computadoras con memoria caché:
 - La mayor parte del tiempo, la CPU lee instrucciones de la cache, por lo que no necesita usar el bus de memoria.
 - El DMAC puede aprovechar estos intervalos en los que la CPU está leyendo instrucciones de la cache (y por tanto no usa el bus de memoria) para realizar las transferencias.
 - En el caso de computadores sin cache
 - El procesador no utiliza el bus en todas las fases de la ejecución de una instrucción.
 - El DMAC puede aprovechar las fases de ejecución de una instrucción en las que la CPU no utiliza el bus.

Acceso directo a memoria (DMA)

55

Técnicas de transferencia por DMA

Hay varias formas distintas de implementar las transferencias por DMA. Vamos a ver 2:

- Por ráfagas (burst)
- Por robo de ciclo (cycle-stealing)

Acceso directo a memoria (DMA)

Método de transferencia de DMA por ráfaga

- Es la vista hasta ahora. El DMAC solicita el control del bus a la CPU.
- Cuando la CPU concede el bus, el DMAC no lo libera hasta haber finalizado la transferencia de todo el bloque de datos completo.
- VENTAJAS:

La transferencia se realiza de forma muy rápida, limitada por la velocidad del periférico.

- DESVENTAJAS:

Durante el tiempo que dura la transferencia la CPU no puede utilizar el bus con memoria, lo que puede degradar el rendimiento del sistema.

Acceso directo a memoria (DMA)

57

Método de transferencia de DMA por robo de ciclo

- El DMAC solicita el control del bus a la CPU.
- Cuando la CPU concede el bus al DMAC, se realiza la transferencia de una única palabra y después el DMAC libera el bus.
- El DMAC solicita el control del bus tantas veces como sea necesario hasta finalizar la transferencia del bloque completo. El uso del bus se reparte entre la CPU y el DMAC.
- VENTAJAS:

No se degrada tanto el rendimiento del sistema y de la CPU.

- DESVENTAJAS:

La transferencia puede tardar un poco más de tiempo.

Acceso directo a memoria (DMA)

58

Método de transferencia de DMA por robo de ciclo

- Notar que dado que la toma y liberación del bus por parte de la CPU no es una interrupción, es decir que el procesador no debe guardar el contexto (es decir, no está interrumpiendo su tarea).
- Si bien el trabajo de la CPU es más lento (que si no estuviera presente la transferencia por DMA), no será tanto como si ella tuviera que estar desconectada del bus todo el tiempo.
- En general, para transferencia de E/S de múltiples palabras, la técnica por robo de ciclo es la más eficiente, ya que permite implementar la transferencia por DMA al mismo tiempo que la CPU continúa trabajando en su tarea.

Canales de E/S

59

- Las transferencias de E/S se pueden dividir, en función de la capacidad para interactuar con los periféricos, en varios niveles:
 - Nivel 1: CPU + Módulo de Interfaz de E/S o controlador: la CPU controla directamente los periféricos e interfaz, y administra la transferencia por programa (con espera).
 - Nivel 2: CPU + Módulo de Interfaz E/S o controlador con interrupción: la CPU controla directamente los periféricos y administra la transferencia con programa e interrupciones.
 - Nivel 3: DMA (DMAC + Módulo de E/S): la CPU no interviene directamente, solo prepara y supervisa la transferencia.
 - Nivel 4: Canal de E/S básico (Procesador básico + módulo de E/S): la CPU interviene mínimamente.
 - Nivel 5: Canal de E/S inteligente (Procesador inteligente + módulo de E/S): la CPU no interviene, excepto situaciones especiales.

Canales de E/S

60

- En el nivel más alto de la escala de transferencias de E/S están los canales de E/S, que representan una extensión al concepto de DMA.
- Los canales de E/S tienen la habilidad de ejecutar programas de servicios de E/S, lo que les permite tener un completo control de la transferencia de datos. La CPU no ejecuta las instrucciones de E/S, las realiza el procesador incluido en el canal. El programa que ejecuta el procesador interno del canal está almacenado en la memoria principal.
- La CPU solamente interviene para iniciar la transferencia, y dar la orden de ejecutar el programa de E/S que está en memoria. El programa de servicio de E/S especifica dispositivos, áreas de memoria a usar, prioridades y acciones ante errores.

Canales de E/S

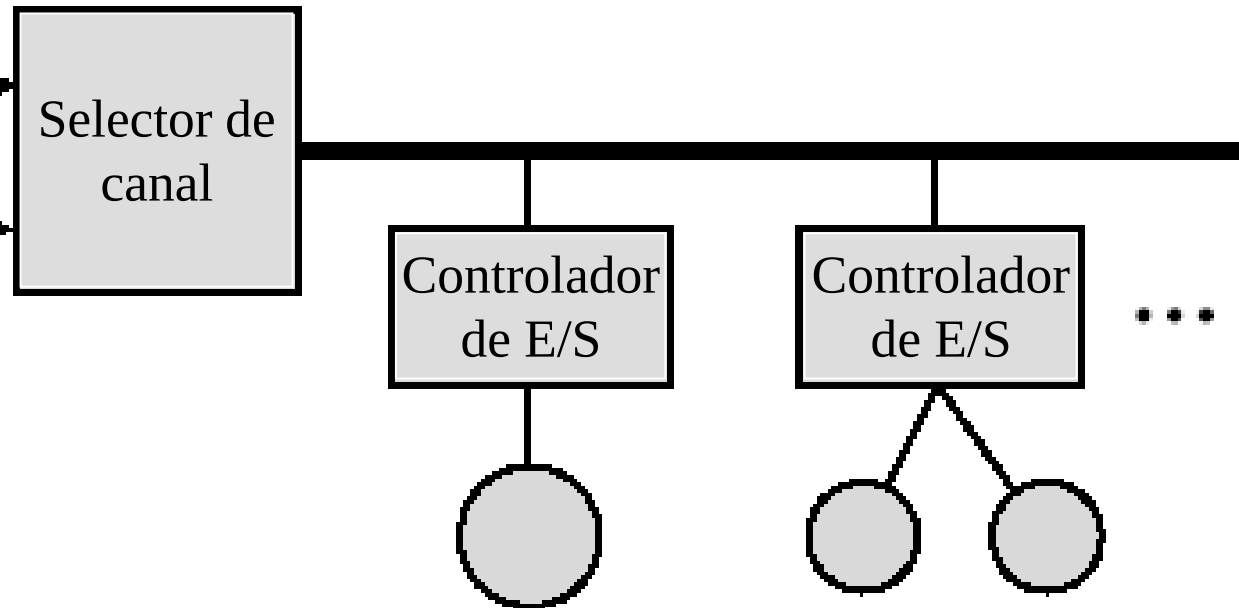
- Hay 2 tipos básicos de canales de E/S:
 - Selector
 - Multiplexor
- Canal selector de E/S
 - El canal controla varios dispositivos de alta velocidad, de a uno por vez.
 - El canal selecciona un dispositivo y efectúa la transferencia sobre el dispositivo seleccionado.
 - Cada dispositivo tiene asociado un controlador o módulo de E/S que lo maneja.
 - Por lo tanto el canal de E/S ocupa el lugar de la CPU en el control del módulo de E/S.

Canales de E/S

- En la imagen siguiente se puede apreciar el esquema de funcionamiento de un Canal selector de E/S.
- Solo puede atender 1 dispositivo a la vez.

Datos y direcciones
canal a mem.ppal.

Señales de control
CPU.- Selector



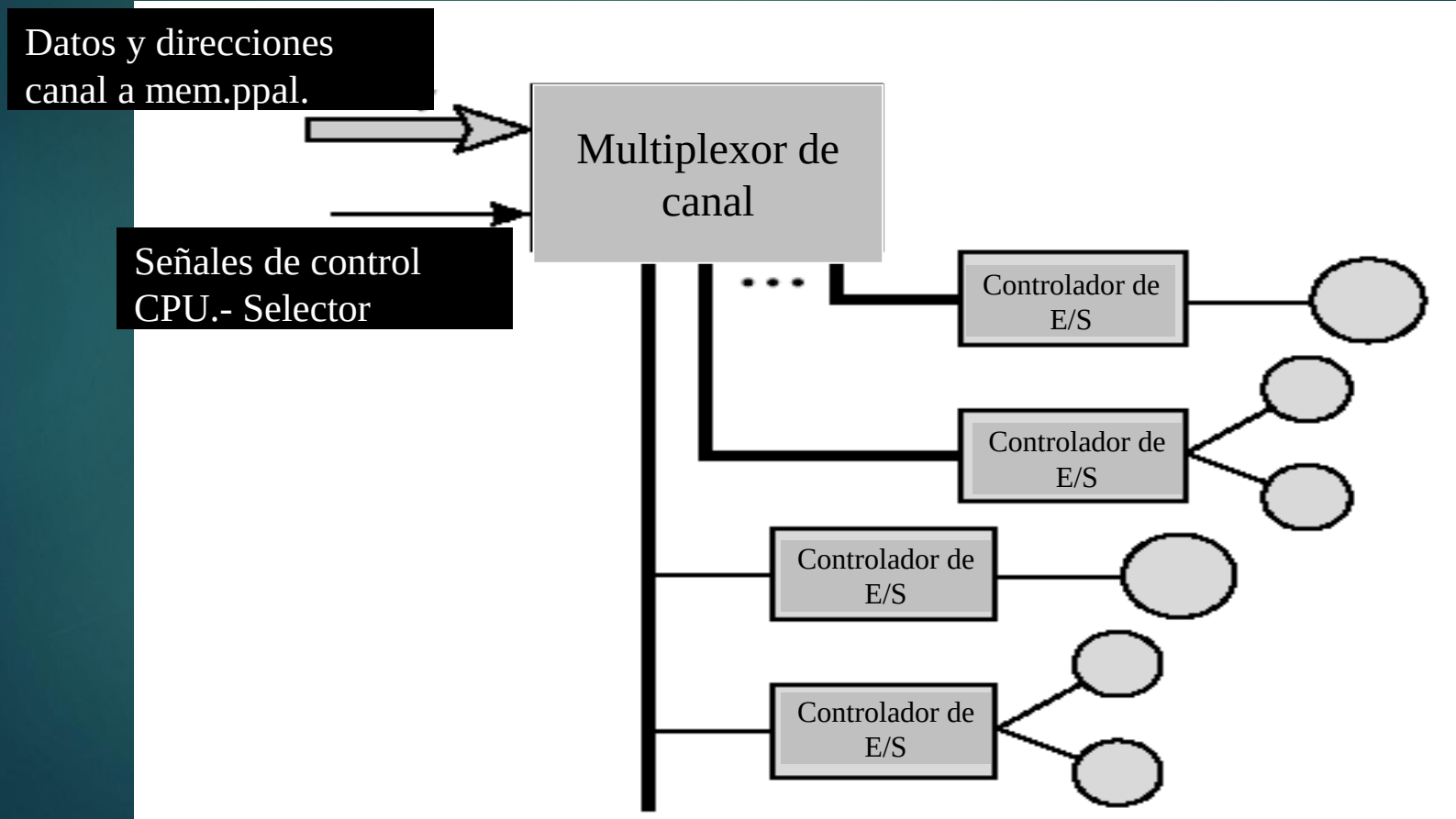
Canales de E/S

➤ Canal Multiplexor de E/S

- El canal controla varios dispositivos de alta velocidad, incluso simultáneamente.
- El canal multiplexa la atención entre los dispositivos seleccionados. El multiplexado puede ser:
 - Multiplexor de bytes: acepta y transmite de a caracteres.
 - Multiplexor de bloques: intercala bloques de datos desde distintos dispositivos.

Canales de E/S

- En la imagen siguiente se puede apreciar el esquema de funcionamiento de un Canal multiplexor de E/S.
- Puede atender varios dispositivos a la vez.



Referencias

- W. Stallings, 5º Ed - Capítulo 6.
- <http://www.pcguide.com/ref/mbsys/res/irq/func.htm>

Arquitectura de Computadoras

CURSO 2022

Turno:
Clase 4

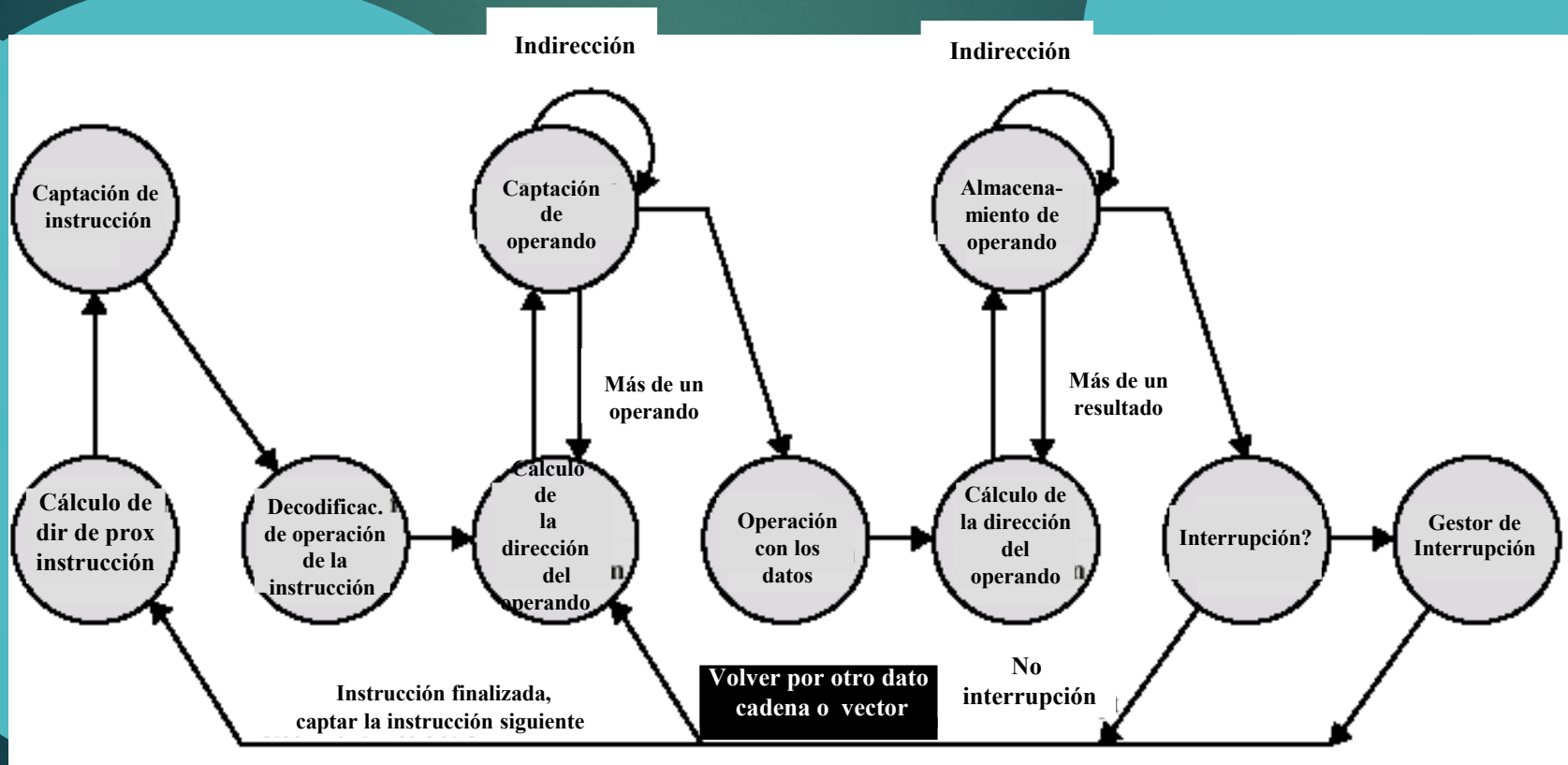
Resumen clase 4

Segmentación del cauce

- Análisis del nanoMips
 - Introducción
 - Formato de instrucción
 - Modos de direccionamientos
 - Repertorio de instrucciones
 - Rutas de datos, instrucciones y control
 - Diagrama de estados
- Segmentación de instrucciones
- Segmentación en el nanoMIPS
- Conflictos y soluciones en la segmentación

Diagrama de estados del ciclo de instrucción

En la figura siguiente se muestra el **diagrama de estados de una CPU convencional**. Este diagrama de estados va a servir de referencia para el procesador que se va a describir a continuación (**nanoMIPS**).



NanoMIPS

4

Introducción al nanoMIPS

- Para algunos de los temas a tratar a continuación, es mejor usar un modelo de procesador más sencillo del que se usó hasta ahora (8086 / MSX88).
- Se va a usar un modelo de procesador simplificado basado en el procesador MIPS, que es un procesador comercial tipo RISC.

NanoMIPS

5

Introducción al nanoMIPS

El nanoMIPS tiene estas características principales:

- Palabra de memoria de 32 bits
- Espacio de direcciones de 64 bits (virtual)
- 32 (31 efectivos) Registros de propósito general de 64 bits (R0..R31).
- El registro R0 es 0 (es decir, si referenciamos al registro R0 nos devuelve 0).
- Como son 32 registros, se requieren 5 bits para identificarlo.

NanoMIPS

6

Formato de instrucción

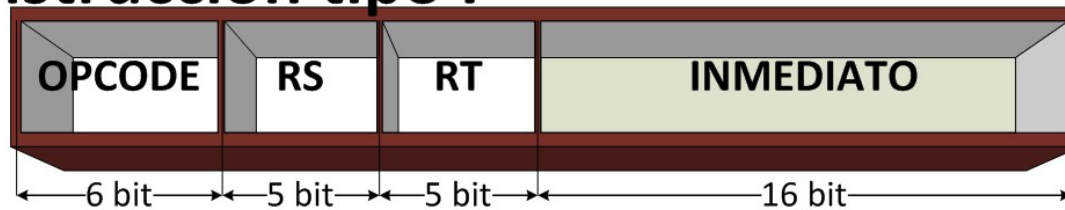
El nanoMIPS dispone de 3 tipos de Instrucciones:

- Tipo I: instrucciones de acceso a memoria, únicamente las de carga y almacenamiento (“LOAD” y “STORE”).
- Tipo R: instrucciones aritméticas y lógicas, únicamente registro a registro (no admite operaciones con memoria).
- Tipo J: instrucciones de salto
- El acceso a la memoria se hace únicamente con las instrucciones tipo I. Por esta razón a este tipo de máquinas se las identifica como máquina tipo “LOAD/STORE”.
- El formato es muy regular, porque:
 - Las instrucciones tienen todas el mismo tamaño (32 bits).
 - Los campos con referencias son siempre los mismos.

NanoMIPS

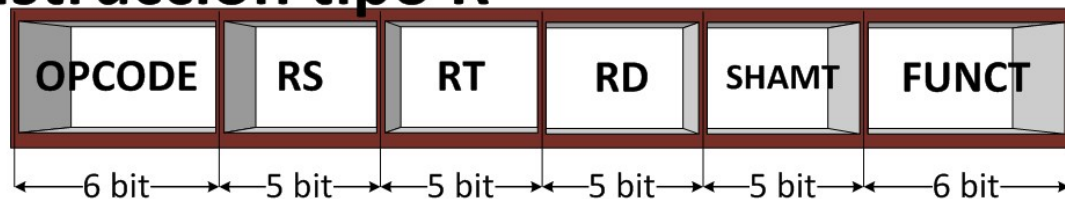
Formato de instrucción

Instrucción tipo I



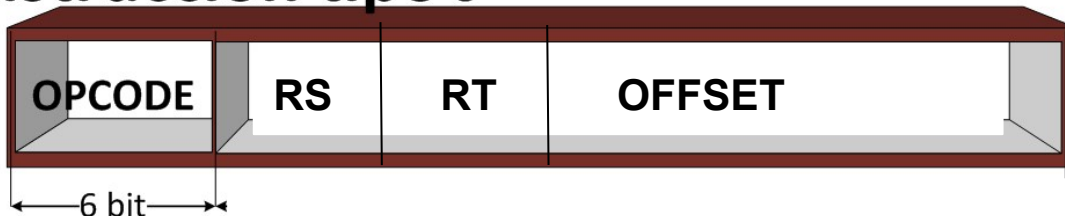
Load/store

Instrucción tipo R



Aritm./lógicas

Instrucción tipo J



Salto cond./incond.

NanoMIPS

Modos de direccionamiento

El nanoMIPS dispone formalmente de 2 modos de direccionamiento, pero si se consideran 2 casos especiales, se obtienen 2 modos más (4 en total).

- Inmediato: dato de 16 bits.
- Indirecto con desplazamiento: suma de registro más un desplazamiento (distinto de 0).
- Indirecto vía registro: si el desplazamiento es 0 el direccionamiento es indirecto vía registro.
- Directo absoluto: Si el desplazamiento es vía registro R0 (que vale 0), entonces se obtiene directo absoluto.

NanoMIPS

Repertorio de instrucciones

Para analizar el comportamiento de este procesador, se va a considerar un repertorio básico de 8 instrucciones: 2 de movimiento de datos con memoria, 5 aritméticas y lógicas, y 1 de salto condicional.

Instrucción		Pseudocódigo	Descripción
I	LW	LW RT, desp(RS)	Carga registro RT desde memoria
	SW	SW RT, desp(RS)	Almacena en memoria desde registro RT
	ADD	ADD RD, RS, RT	Suma palabras en registros RS y RT, resultado en RD
	SUB	SUB RD, RS, RT	Resta palabras en registros RS y RT, resultado en RD
R	AND	AND RD, RS, RT	AND de palabras en registros RS y RT, resultado en RD
	OR	OR RD, RS, RT	OR de palabras en registros RS y RT, resultado en RD
	SLT	SLT RD, RS, RT	Pone 1 en RD si RS es menor o igual que RT
J	BEQ	BEQ RS, RT, destino	Salta a 'destino' si RS es igual a RT

NanoMIPS

Repertorio de instrucciones

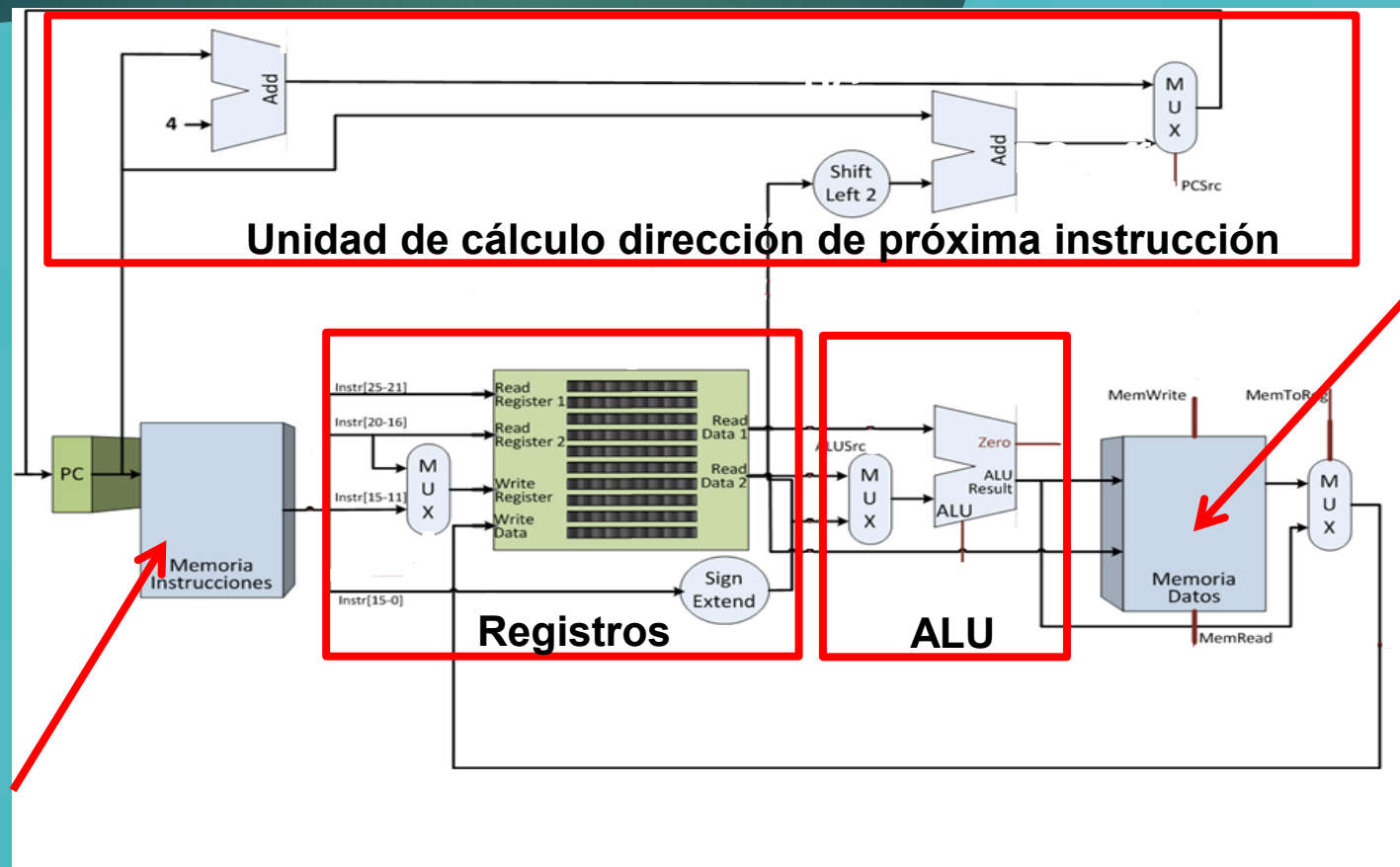
- **Instrucciones tipo I:** Load/store
 - RS: registro base
 - RT: Registro de fuente o destino de la transferencia
 - Desp: offset
- **Instrucciones tipo R:** Aritmético/lógicas
 - RS: operando 1
 - RT: operando 2
 - RD: resultado
- **Instrucciones tipo J:** de salto
 - RS: operando 1
 - RT: operando 2
 - Destino: es un desplazamiento respecto del PC (16 bits)

NanoMIPS

11

Ruta de datos e instrucciones

En la imagen siguiente se muestra la ruta de los datos e instrucciones dentro y fuera de la CPU



NanoMIPS

12

Ruta de datos e instrucciones

La imagen anterior muestra la ruta de los datos e instrucciones del nanoMips. **Se pueden identificar:**

➤ **Banco de registros:**

- Está compuesto por 32 registros (el R0=0)
- Se pueden leer 2 registros al mismo tiempo (doble entrada)
- Se puede escribir en 1 solo registro

➤ **ALU:**

- Es una unidad de cálculo típica de 2 entradas y 1 salida
- En la entrada se pueden seleccionar diferentes fuentes de datos

➤ **Unidad de cálculo de dirección de próxima instrucción:**

- Calcula el próximo valor con el que se carga el PC
- Puede ser el de la instrucción consecutiva (PC+4), o sumando un desplazamiento (instrucción de salto con desplazamiento).

NanoMIPS

13

Ruta de datos e instrucciones

También se puede apreciar que existen 2 bancos de memoria:

➤ Memoria de instrucciones:

- Solo contiene el programa a ejecutar
- Se accede únicamente para lectura, en la fase de búsqueda de la instrucción, y accedida solo a través del PC.

➤ Memoria de datos:

- Contiene los datos a leer o escribir
- Solo se accede a través de las instrucciones LOAD y STORE

NanoMIPS

14

Ruta de datos, instrucciones y control

La Unidad de control (CU) es una unidad relativamente sencilla que captura la instrucción y a partir de ella genera las señales de control requeridas por los demás bloques operativos.

En particular, la CU necesita el campo del código de operación (OPCODE) para determinar que tareas deberá realizar para completar la instrucción corriente.

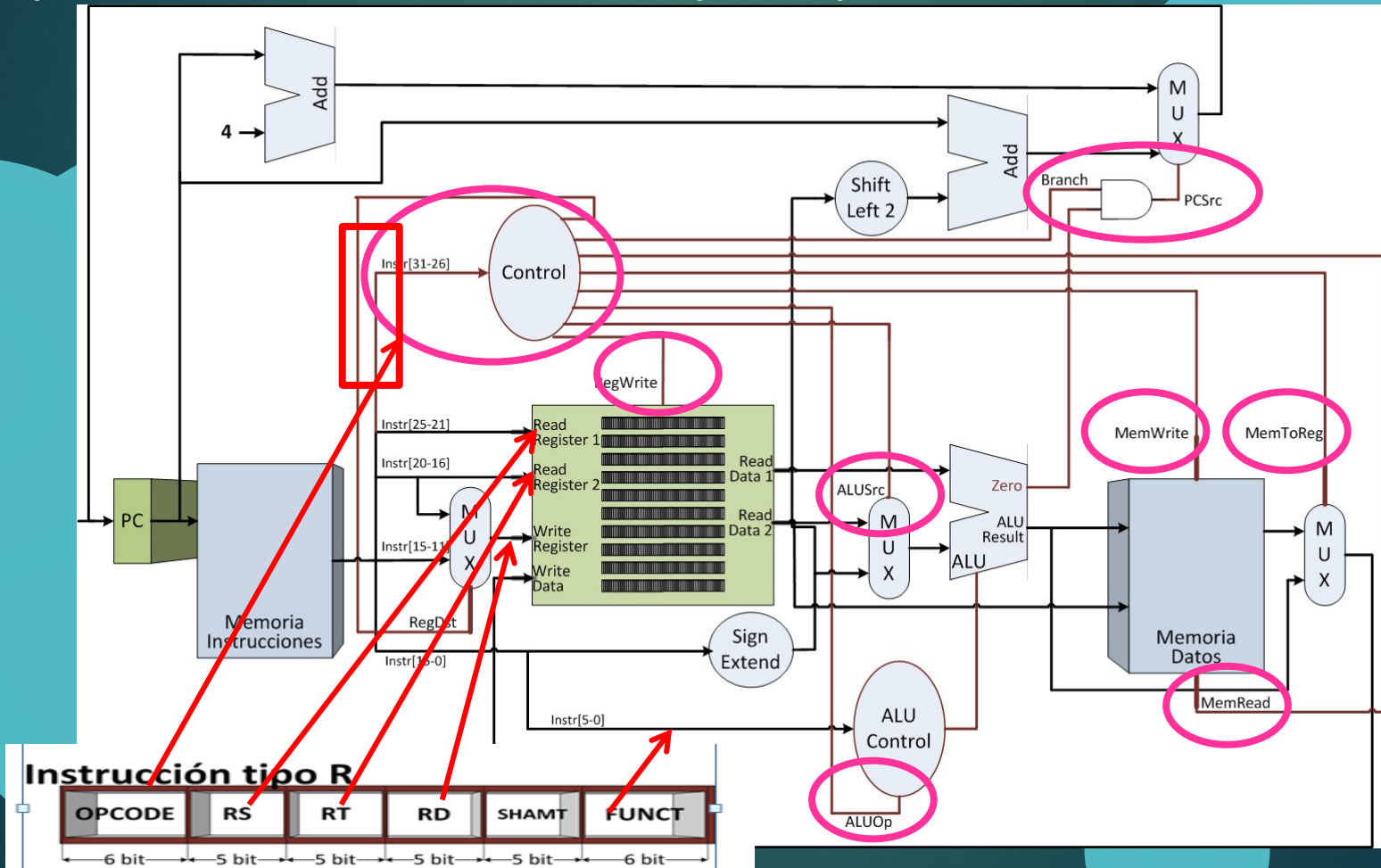
El resto de los campos se usan para las identificaciones restantes. Por ejemplo:

- El campo RS selecciona 1 registro en el banco de registros
- Idem los campos RT y RD (cuando corresponda)
- El resto de los campos se usan para completar las órdenes a dar a las Unidades funcionales restantes (por ejemplo la ALU).

NanoMIPS

Ruta de datos, instrucciones y control

En la imagen siguiente se muestra la asociación entre los campos de una instrucción tipo R y su uso dentro de la CPU.



NanoMIPS

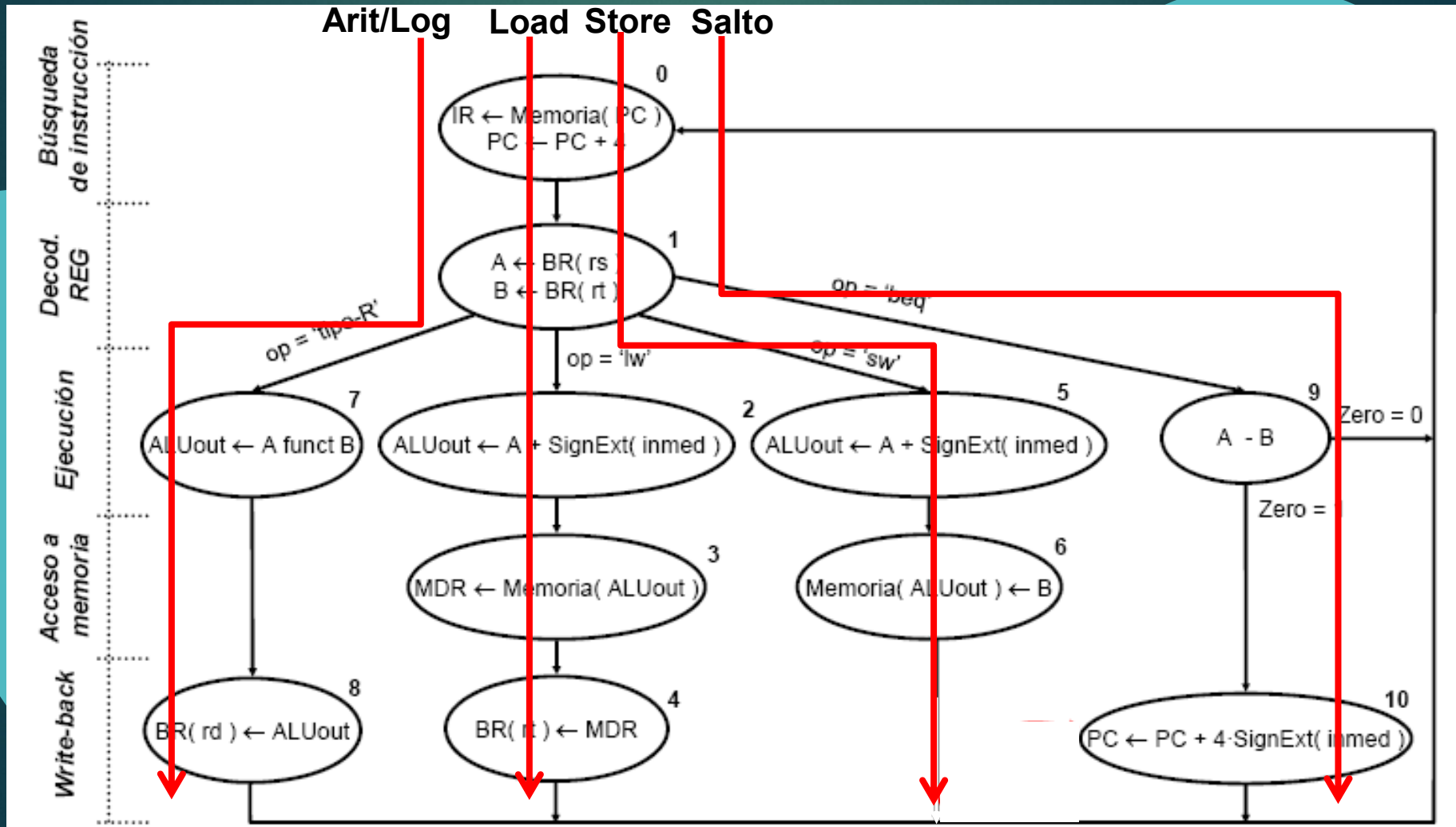
16

Ruta de datos, instrucciones y control

En la imagen anterior se puede apreciar que la Unidad de control ("Control"), a partir de los bits del campo OPCODE (bits 31-26) genera las señales de control para las demás unidades, por ejemplo RegWrite, ALUSrc, MemWrite, MemToReg, etc.

NanoMIPS

Diagrama de estados



NanoMIPS

18

Diagrama de estados

Tiene una secuencia de instrucción compuesta por 5 estados.

Fase 1 - Búsqueda de instrucción (F, Fetch):

- ocurre en todas las instrucciones
- Busca y lee la instrucción en la memoria (de instrucciones)
- Actualizar el PC (sumar 4 al valor actual del PC): cada instrucción ocupa 4 bytes, por lo que la próxima instrucción consecutiva está en la dirección actual + 4.

Fase 2 - Decodificación (D, Decode) y acceso a registros:

- ocurre en todas las instrucciones
- Se decodifica la instrucción (del campo CODOP)
- Se accede al banco de registros
- Opcional: Extensión del signo del offset para cálculo de la dirección efectiva

NanoMIPS

19

Diagrama de estados

Fase 3 - Ejecución (X, Execute)

- ocurre en todas las instrucciones
- Se ejecuta la operación en la ALU

Fase 4 - Acceso a memoria (M, Memory Access)

- ocurre solo en las instrucciones de LOAD y STORE
- Se accede a memoria

Fase 5 - Almacenamiento en Registro (W, Writeback)

- ocurre en las instrucciones que almacenan un dato en un registro
- También es en esta fase cuando puede calcular el desplazamiento a sumar al PC en instrucciones de salto.

NanoMIPS

Diagrama de estados

En la imagen anterior se pueden apreciar las 5 fase de ejecución:

Instrucción LOAD:

- la secuencia de fases es F-D-X-M-W(indicado como 0-1-2-3-4)
- Requiere las 5 fases

Instrucción STORE:

- la secuencia de fases es F-D-X-M (indicado 0-1-5-6)
- Usa 4 fases, no tiene fase W

Instrucción Aritmético/lógica:

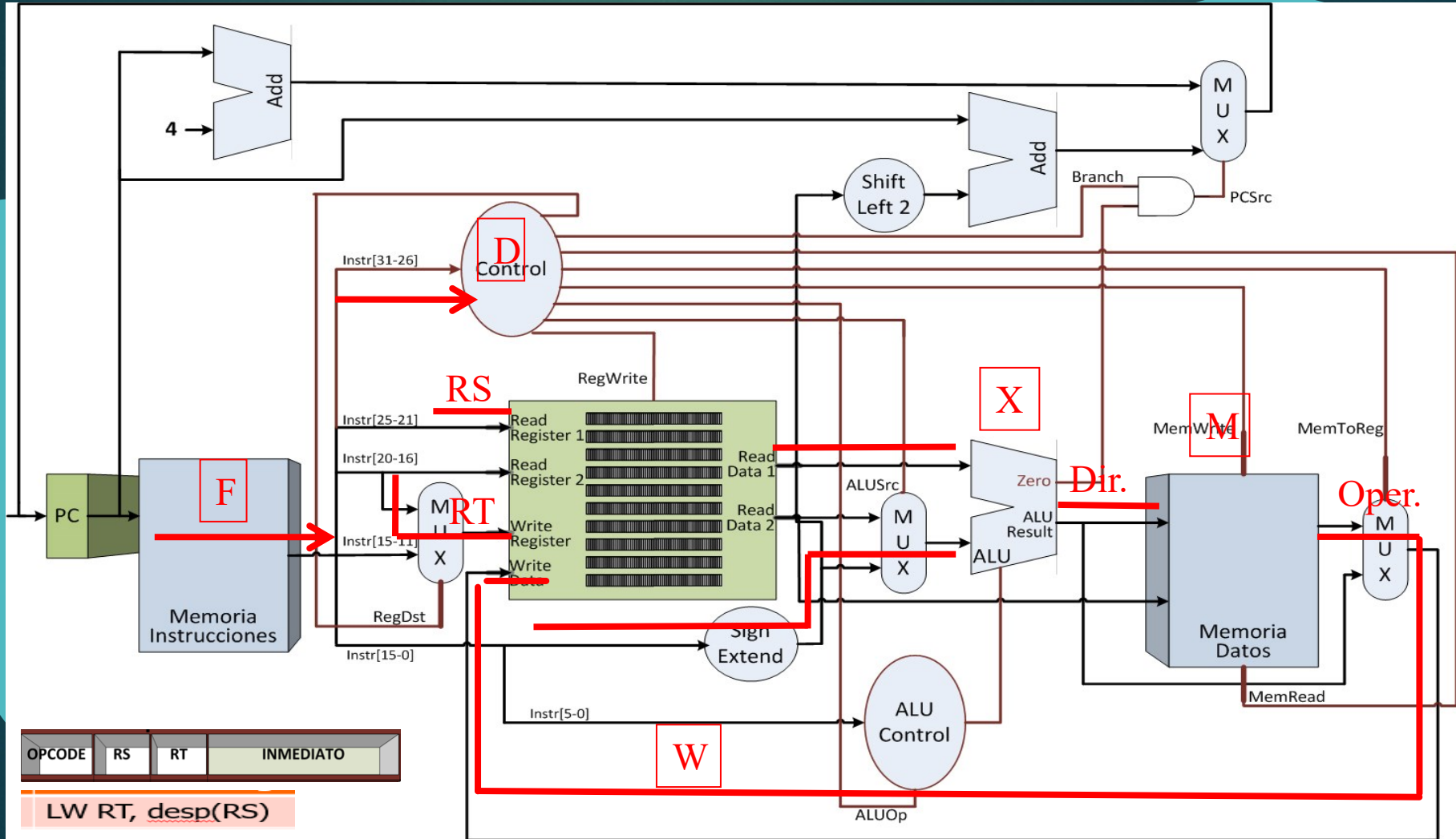
- la secuencia de fases es F-D-X-W (indicado 0-1-7-8)
- Usa 4 fases, no tiene fase M

Instrucción de salto:

- la secuencia de fases es F-D-X-W (indicado 0-1-9-10)
- Usa 4 fases, no tiene fase M

NanoMIPS

Secuencia de estados para una instrucción de LOAD



NanoMIPS

22

Diagrama de estados

En la imagen anterior se puede ver el **proceso de ejecución de una instrucción de LOAD**.

Fase F: Lee la instrucción.

- Los bits 26 a 31 son leídos por la Unidad de Control.
- Los bits 21 a 25 identifican el registro RS (registro puntero)
- Los bits 16 a 20 identifican el registro RT (destino o de escritura)
- Los bits 0 a 15 son el desplazamiento a sumar al RS

Fase D: la UC decodifica la instrucción y se accede al registro RS.

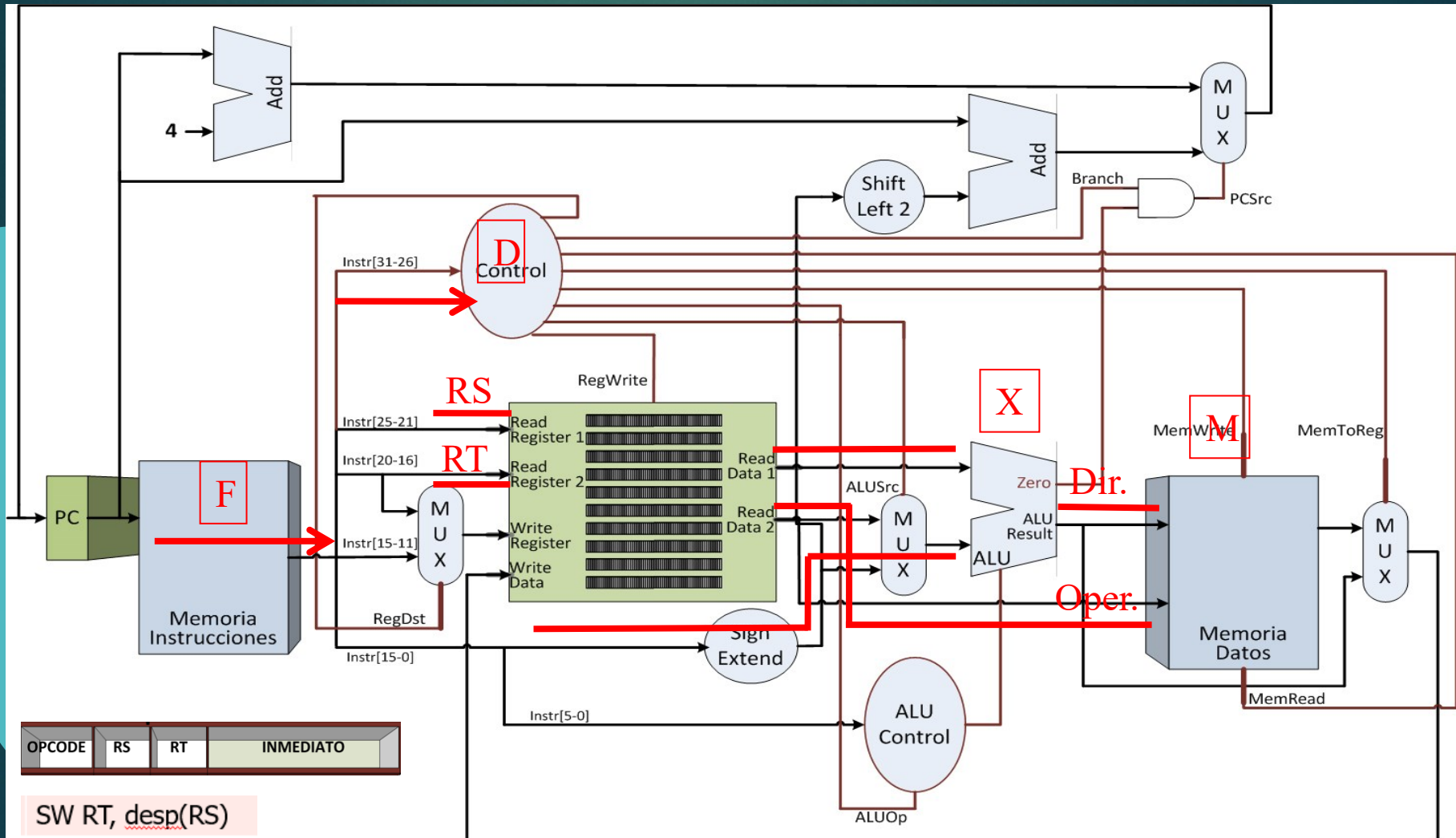
Fase X: se calcula la dirección efectiva donde está el operando a cargar en RT, como el valor de RS más el desplazamiento (inmediato).

Fase M: con la dirección efectiva, se accede a la memoria de datos, y se lee el dato a cargar en RT.

Fase W: el dato de la memoria se carga en el registro identificado con el campo RT de la instrucción.

NanoMIPS

Secuencia de estados para una instrucción de STORE



NanoMIPS

24

Diagrama de estados

En la imagen anterior se puede ver el **proceso de ejecución de una instrucción de STORE**.

Fase F: Lee la instrucción.

- Los bits 26 a 31 son leídos por la Unidad de Control.
- Los bits 21 a 25 identifican el registro RS (registro puntero)
- Los bits 16 a 20 identifican el registro RT (destino o de escritura)
- Los bits 0 a 15 son el desplazamiento a sumar al RS

Fase D: la UC decodifica la instrucción y se accede al registro RS.

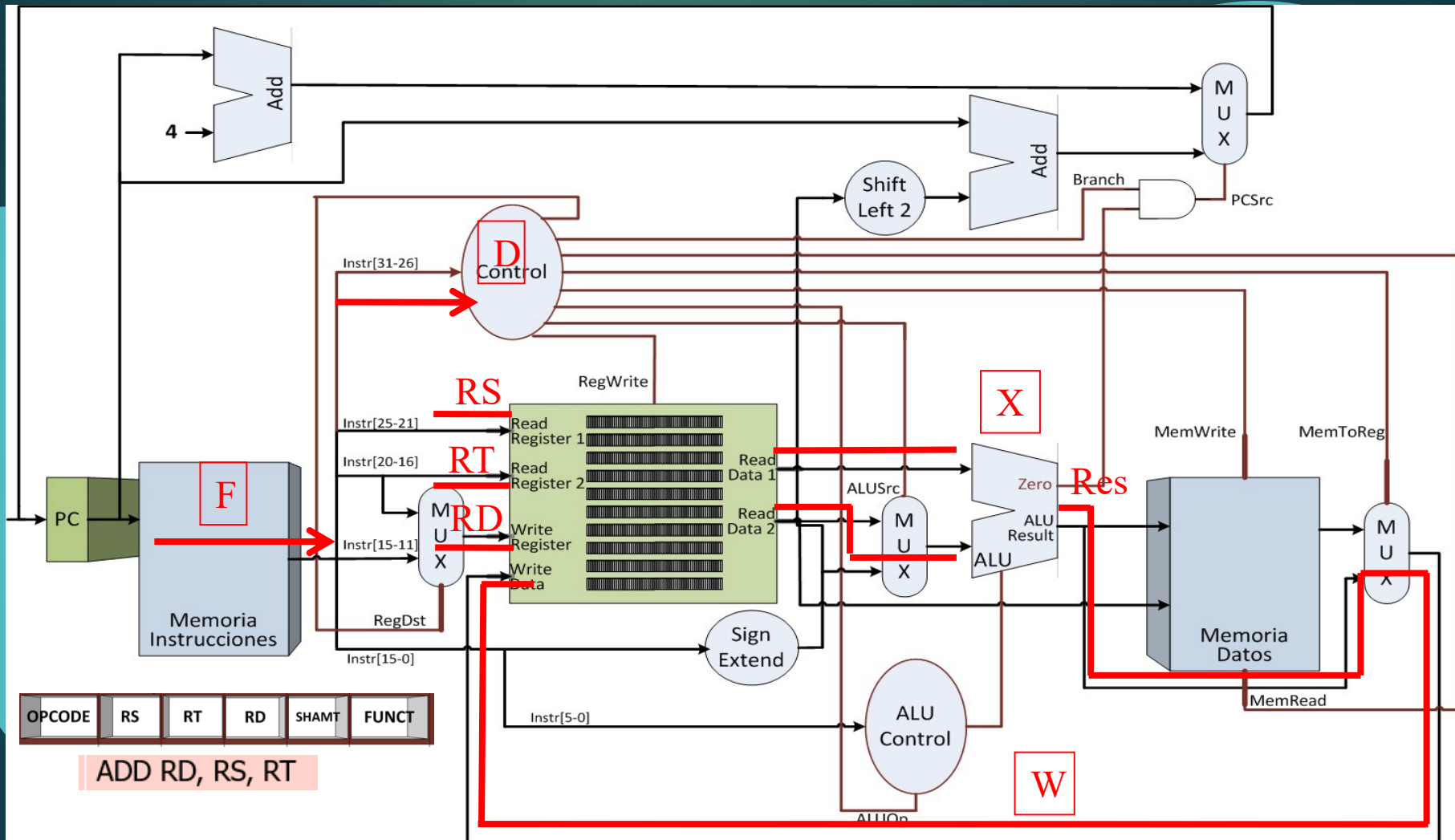
Fase X: se calcula la dirección efectiva, donde se va a almacenar el operando cargado en RT, como el valor de RS más el desplazamiento (inmediato).

Fase M: con la dirección efectiva, se accede a la memoria de datos, y se carga el dato almacenado en RT.

Fase W: no hay.

NanoMIPS

Secuencia de estados para una instrucción de ADD



NanoMIPS

26

Diagrama de estados

En la imagen anterior se puede ver el **proceso de ejecución de una instrucción de ADD**.

Fase F: Lee la instrucción.

- Los bits 26 a 31 son leídos por la Unidad de Control.
- Los bits 21 a 25 identifican el registro RS (1er operando)
- Los bits 16 a 20 identifican el registro RT (2do operando)
- Los bits 11 a 15 identifican el registro de destino RD

Fase D: la UC decodifica la instrucción y se accede a los registros RS y RT.

Fase X: se suman los contenidos de los registros RS y RT

Fase M: no hay

Fase W: se escribe el resultado en RD.

NanoMIPS

27

Ciclo de instrucción – Ejecución MONOCICLO

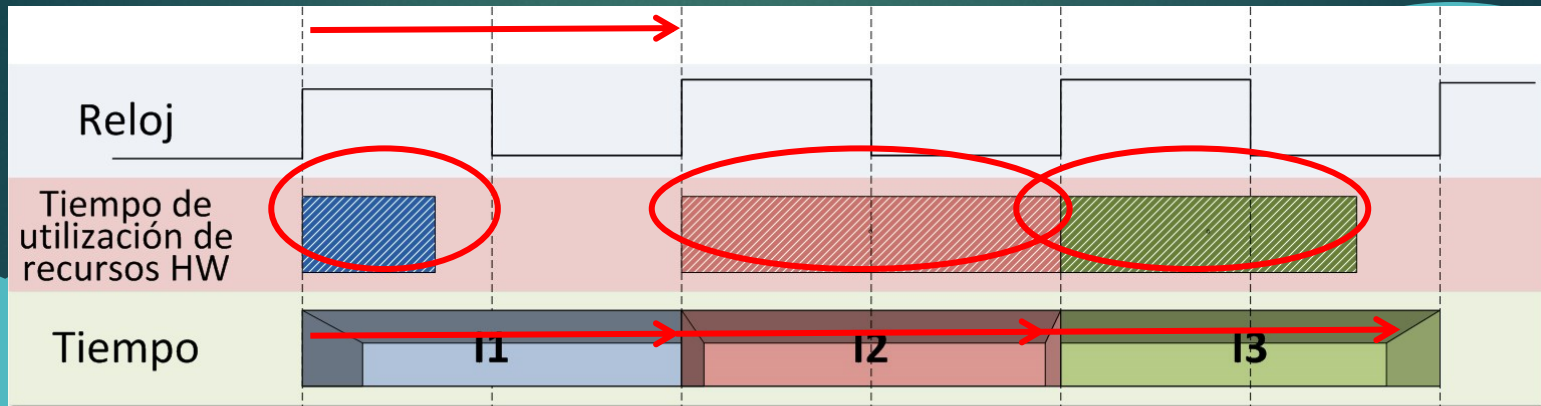
En las descripciones anteriores, cada vez que el PC envía una dirección a la memoria de instrucciones, se accede a una nueva instrucción.

Como no hay registros sincrónicos intercalados en las trayectorias de datos e instrucciones, el dato fluye por las unidades funcionales (registros, ALU, memoria, MUX, etc.) hasta que se complete la ejecución de la instrucción.

No se inicia un nuevo ciclo de instrucción hasta que el PC se carga con un nuevo valor (instrucción consecutiva o la de salto). Si el PC está sincronizado con un reloj (el de la CPU) significa que en cada ciclo de reloj se inicia una nueva instrucción. Este tipo de forma de operación se llama Ejecución MONOCICLO, y se muestra en la imagen siguiente.

NanoMIPS

nanoMIPS con ejecución Monociclo



- El período del reloj se hace con el tiempo necesario para completar la instrucción más lenta.
- En las instrucciones más rápidas el procesador “espera” el fin del ciclo para continuar (Ej: I1, I3).
- La eficiencia depende del tiempo de resolución de la instrucción más lenta (I2).

NanoMIPS

29

Ciclo de instrucción – Ejecución MULTICICLO

En la ejecución MONOCICLO el ciclo de reloj se hace en función de la instrucción “mas larga”, es decir la instrucción de LOAD.

Las instrucciones que son “mas cortas” tienen que esperar un tiempo hasta que se complete el período de reloj.

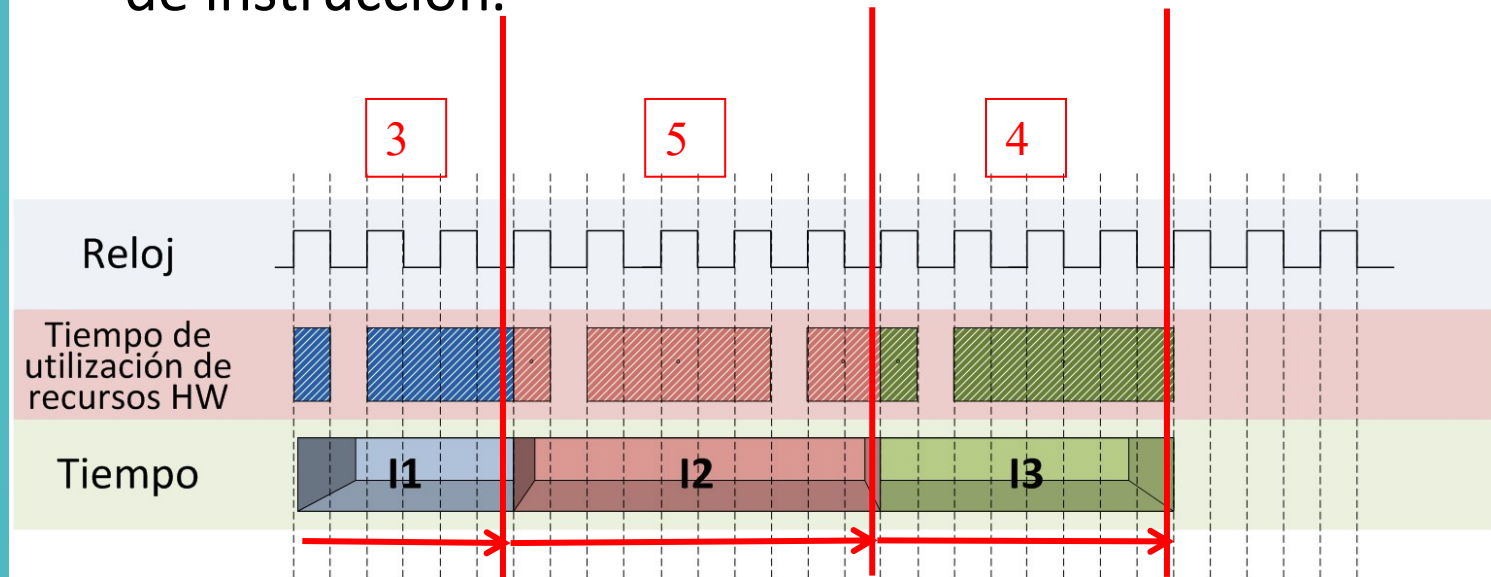
Si la tasa de instrucciones cortas es mucho mayor que la de las largas, se puede perder mucho tiempo inútilmente.

Una solución es fijar un ciclo de reloj más pequeño, y disponer que cada instrucción ocupe varios períodos de reloj.

Este tipo de forma de operación se llama Ejecución MULTICICLO, y se muestra en la imagen siguiente.

Ciclo de instrucción – Ejecución MULTICICLO

- En la ejecución MULTICICLO cada instrucción requiere varios ciclos de reloj.
- Las instrucciones más rápidas usan menos ciclos de reloj que las más lentas.
- El ciclo de instrucción es variable y se ajusta al tipo de instrucción.



Segmentación y solapamiento del cauce (Pipelining)

31

La segmentación del cauce (Pipelining) consiste en:

- Descomponer el proceso de ejecución de las instrucciones en fases o etapas.
- Las fases o etapas son ejecutadas por unidades separadas y capaces de operar simultáneamente.
- Las instrucciones se van ejecutando a medida que se liberan las unidades.
- Las instrucciones no necesitan esperar la terminación de la previa para comenzar a resolverse.
- La segmentación del cauce (pipelining) es una forma particularmente efectiva de organizar el hardware de la CPU para realizar más de una tarea al mismo tiempo. Explota el paralelismo en el flujo secuencial de instrucciones.

Segmentación y solapamiento del cauce (Pipelining)

32

Para entender el concepto de segmentación de una tarea repetitiva se puede analizar un “proceso” de una lavandería que requiere 3 etapas:

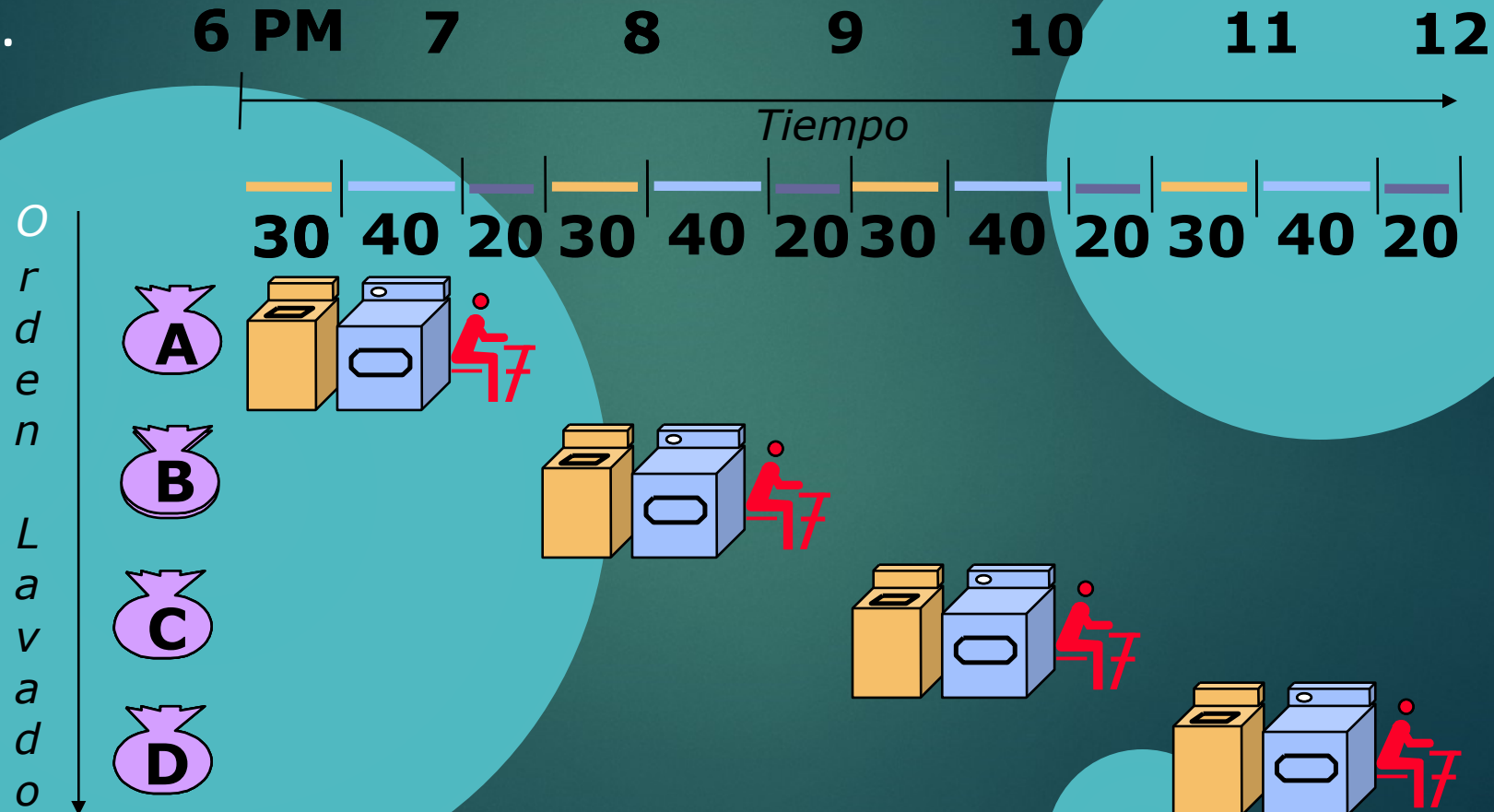
- Lavado, que requiere 30 minutos
- Secado, que requiere 40 minutos
- Despacho, que requiere 20 minutos

Las 3 etapas se ejecutan en 3 sectores (estaciones) separados. La primera forma de operar es ejecutar el proceso en forma puramente secuencial, cuando se termina con un encargo recién se empieza con el siguiente.

Este modelo secuencial se muestra en la figura siguiente.

Segmentación y solapamiento del cauce (Pipelining)

Proceso con ejecución “secuencial”: cada tarea (A, B, C, D) require en total 90 minutos (30+40+20). Tiempo total = 6 horas.



Segmentación y solapamiento del cauce (Pipelining)

34

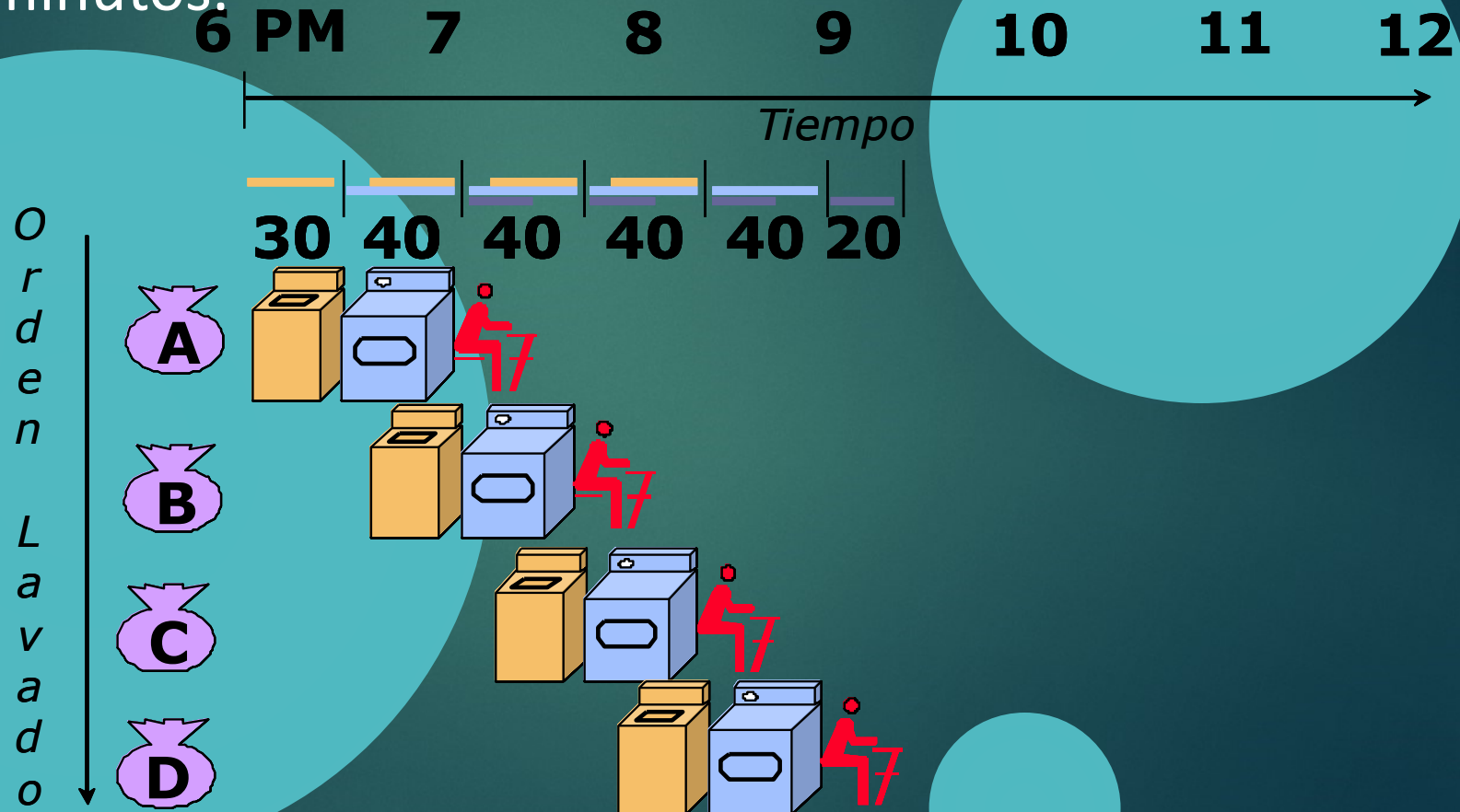
Como las unidades funcionales Lavado, Secado y Despacho son unidades distintas, pueden operar en forma simultánea.

Si se superponen (solapan) las tareas de las 3 unidades funcionales es posible obtener una mejora sustancial en el tiempo total de ejecución de los 4 encargues (A, B, C y D).

Esa situación se muestra en la figura siguiente.

Segmentación y solapamiento del cauce (Pipelining)

Proceso con ejecución “solapada”: cada tarea (A, B, C, D) requiere en total 90 minutos (30+40+20). Tiempo total = 3 horas 30 minutos.



Segmentación y solapamiento del cauce (Pipelining)

36

- En el ejemplo anterior se puede observar que cada tarea lleva exactamente el mismo tiempo: $30+40+20 = 90$ minutos.
- La mejora se obtiene debido al solapamiento de las tareas de las distintas unidades funcionales.
- Solapando las tareas A, B, C y D, se consigue realizarlas en 3 horas 30 minutos, en lugar de las 6 horas que se tardaba operándolas en forma secuencial.
- Se puede aplicar este concepto al proceso repetitivo de ejecutar una secuencia de instrucciones.

Segmentación y solapamiento del cauce (Pipelining)

37

Supongamos que se tiene una máquina que resuelve las instrucciones en 3 fases:

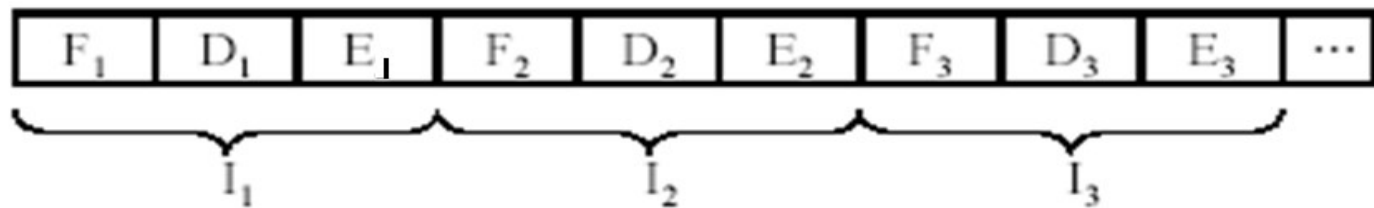
- Búsqueda o Captación (F, Fetch)
 - Acceso a memoria (búsqueda de la instrucción)
 - Incremento del PC
- Decodificación (D, Decode)
 - Decodificación de la instrucción
 - Obtención de los operandos
- Ejecución (E, Execute)
 - Si es procesamiento: ejecución en la ALU
 - Si es acceso a memoria: obtención de la dirección efectiva
 - Si es salto: cálculo del destino y decisión de salto (s/n)

Segmentación y solapamiento del cauce (Pipelining)

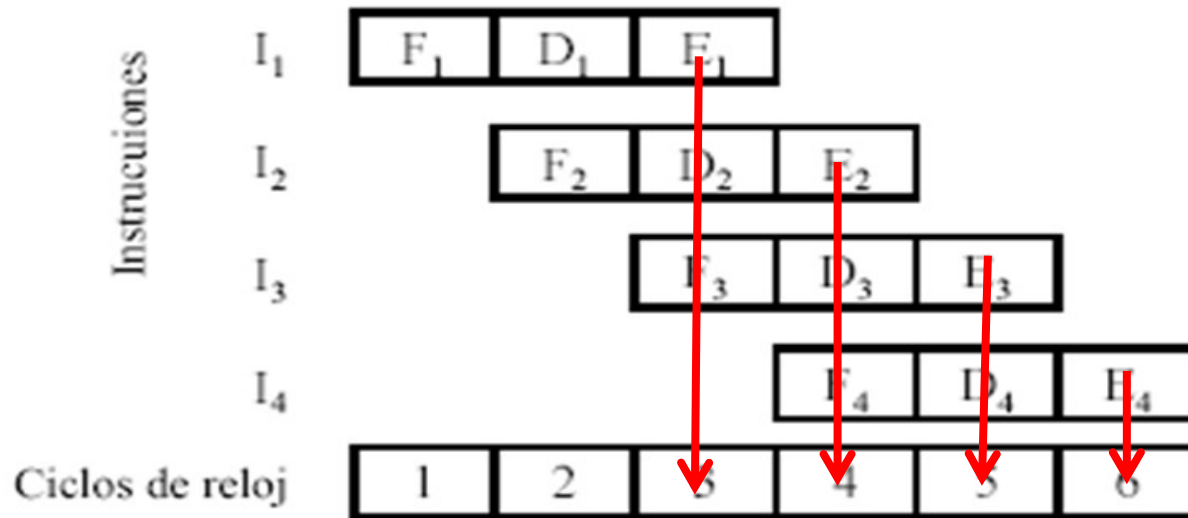
38

La secuencia de tareas se puede hacer secuencialmente o segmentadamente.

- Ejecución secuencial:



- Ejecución segmentada (tres etapas)



Segmentación y solapamiento del cauce (Pipelining)

39

- En el procesador con ejecución secuencial, se tardan 9 ciclos de reloj para completar 3 instrucciones.
- En el procesador segmentado de la figura anterior, de 3 etapas, se tardan 5 ciclos de reloj para completar las mismas 3 instrucciones.
- En ambos casos, cada instrucción requiere 3 ciclos de reloj.

Segmentación y solapamiento del cauce (Pipelining)

Si la máquina tiene 4 fases para resolver una instrucción:

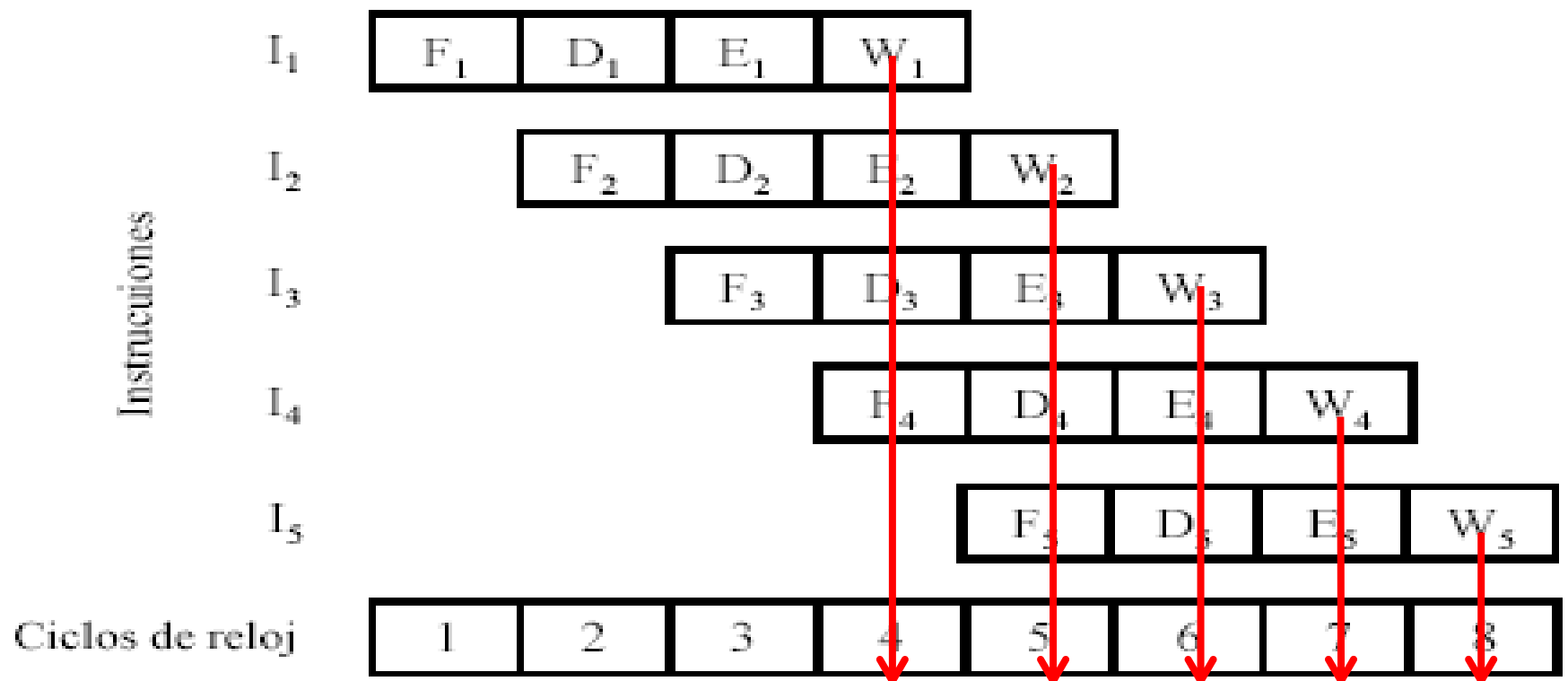
- Búsqueda o Captación (F, Fetch)
- Decodificación (D)
- Ejecución (E)
- Escritura o actualización (W)

La segmentación queda como se muestra en la figura siguiente.

Segmentación y solapamiento del cauce (Pipelining)

Ejecución segmentada en una máquina de 4 etapas F, D, E, W, de una secuencia de 5 instrucciones.

Ejecución segmentada (4 etapas)



Segmentación y solapamiento del cauce (Pipelining)

42

- En el procesador segmentado de 4 etapas de la figura anterior, se tardan 8 ciclos de reloj para completar 5 instrucciones.
- En el procesador con ejecución secuencial se tardarían $5 \times 4 = 20$ ciclos de reloj para completar las 5 instrucciones.
- En ambos casos, cada instrucción requiere 4 ciclos de reloj.

Segmentación y solapamiento del cauce (Pipelining)

Conclusiones:

- El comportamiento de un procesador ejecutando una secuencia de instrucciones se comporta de una forma similar a una línea de montaje en una planta de manufactura.
- Cada instrucción pasa, durante su ejecución, por varias etapas, en cada una se realiza solo una parte del todo.
- La entrada de una nueva instrucción puede hacerse antes de que se terminen las anteriores.
- Por lo tanto, varias instrucciones están siendo manipuladas simultáneamente, cada una en un estado de ejecución distinto.

Segmentación y solapamiento del cauce (Pipelining)

44

Conclusiones:

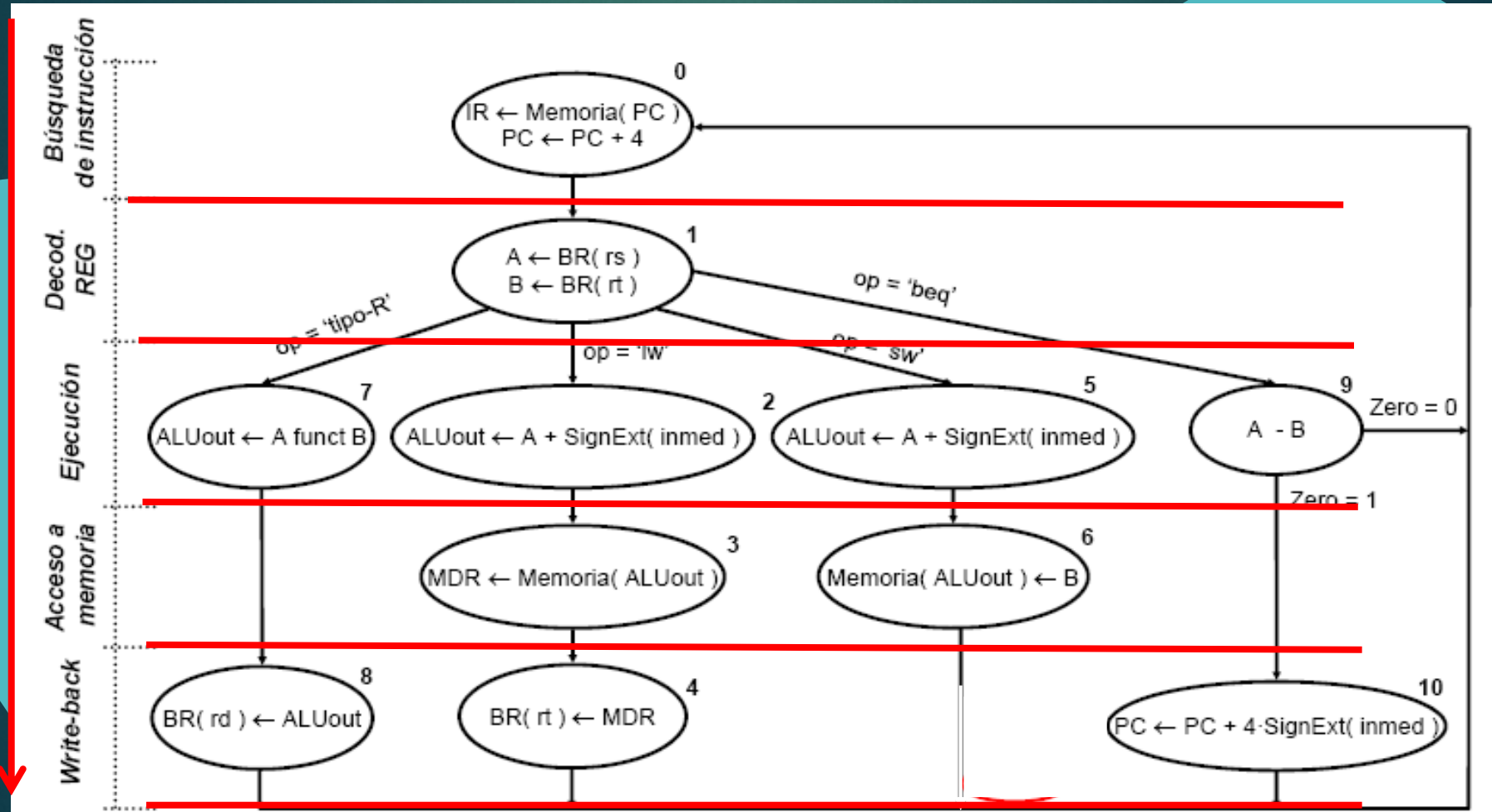
- Dado que el procesador tarda menos tiempo en resolver un conjunto de instrucciones, la segmentación mejora las prestaciones (a nivel de diseño del hardware).
- Como el programador no interviene en la segmentación y solapamiento de las instrucciones, la segmentación es “invisible” (por ahora) al programador.
- Se puede usar tanto en procesadores RISC como CISC.
- El diseño de procesadores segmentados tiene gran dependencia del repertorio de instrucciones.

Segmentación y solapamiento del cauce en el nanoMIPS

- Se puede aplicar el concepto de Segmentación en el nanoMIPS.
- De acuerdo a lo visto, el proceso de ejecución de las instrucciones en el nanoMIPS está compuesto por 5 fases o etapas.
- En la imagen siguiente se puede apreciar la segmentación de las fases de ejecución de una instrucción.

Segmentación y solapamiento del cauce en el nanoMIPS

Ejecución segmentada en 5 etapas en el nanoMIPS



Segmentación y solapamiento del cauce en el nanoMIPS

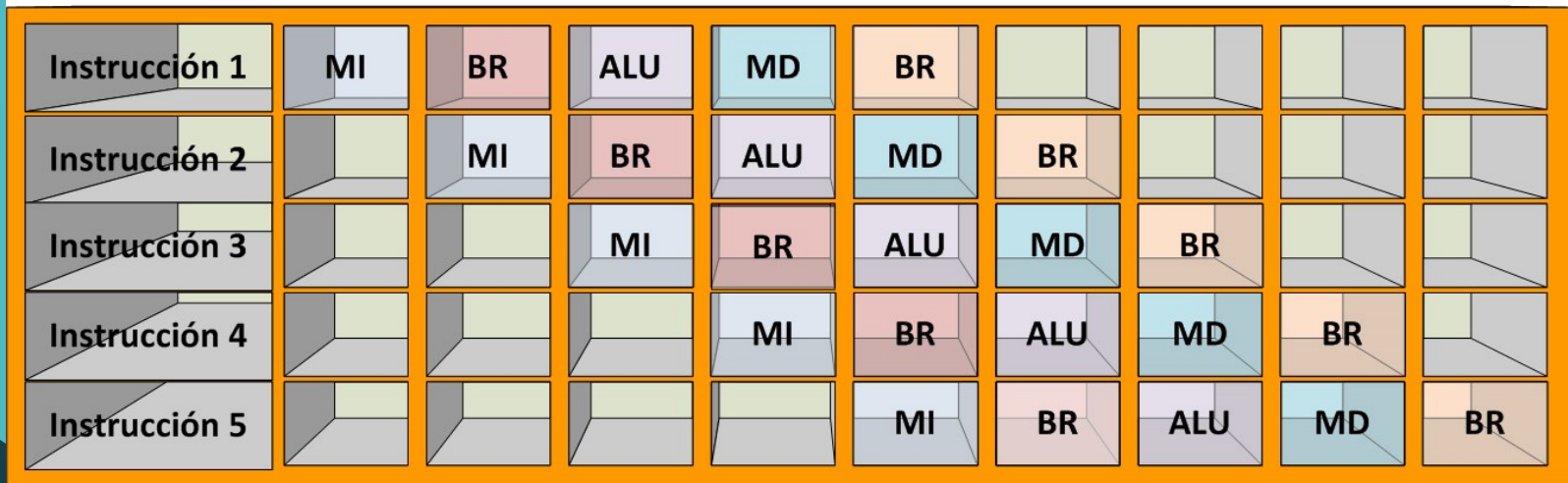
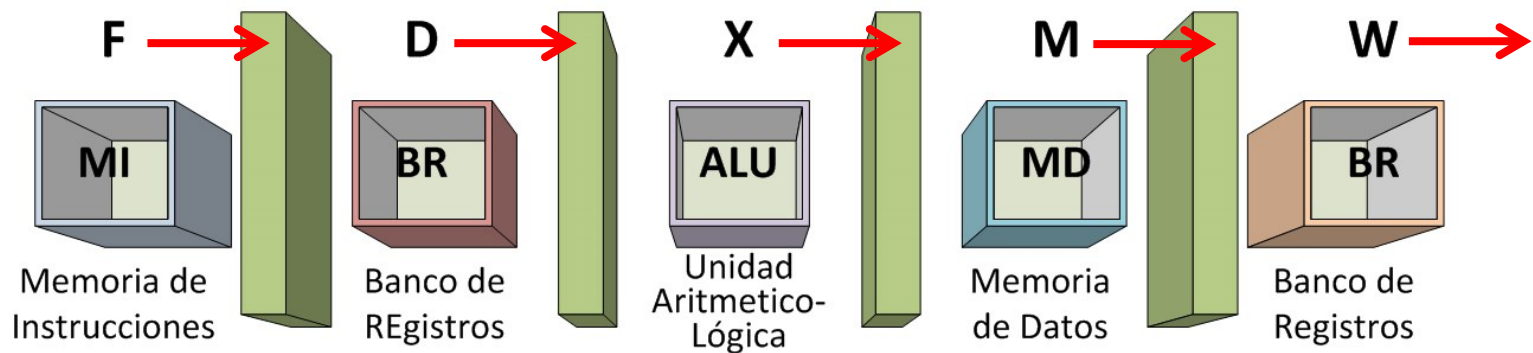
- Los 5 segmentos del nanoMIPS son:
 - **F**: búsqueda de la instrucción en la MI
 - **D**: decodificación y acceso a registros en el BR
 - **X**: ejecución de la ALU
 - **M**: acceso a la memoria de datos MD
 - **W**: escritura del resultado en el banco de registros BR
- Y los recursos de la máquina son:
 - **MI**: memoria de instrucciones
 - **BR**: banco de registros
 - **ALU**: ALU
 - **MD**: memoria de datos

Segmentación y solapamiento del cauce en el nanoMIPS

- Para poder aplicar el concepto de segmentación en el nanoMIPS, hay que dividir (segmentar) el cauce de los datos en etapas.
- La forma de hacer esto es usando registros sincrónicos entre cada etapa.
- Los datos avanzan, de etapa a etapa, en cada ciclo de reloj, que es cuando el registro (de separación de la etapa) copia a la salida lo que tiene en su entrada.
- Los segmentos en que queda dividido el cauce de datos, corresponden a las 5 fases de ejecución de la instrucción: F, D, X, M y W.
- Este proceso se puede apreciar en la siguiente figura.

Segmentación y solapamiento del cauce en el nanoMIPS

Ejecución segmentada en 5 etapas en el nanoMIPS, de un secuencia de 5 instrucciones.



Segmentación y solapamiento del cauce en el nanoMIPS

- En la figura anterior se puede apreciar el **proceso de ejecución de una secuencia de 5 instrucciones**.
- Se considera que cada instrucción requiere 5 ciclos de reloj, uno por cada segmento (F-D-X-M-W).
- En cada ciclo se incorpora una nueva instrucción, sin haber completado las anteriores.
- La primera instrucción se termina en el período de reloj 5, hasta ese ciclo no se había resuelto ninguna instrucción.
- A partir del quinto ciclo, se termina 1 instrucción por ciclo.
- Notar que en el quinto ciclo el procesador está ejecutando 5 instrucciones simultáneamente, pero en distintas fases.

Segmentación y solapamiento del cauce en el nanoMIPS

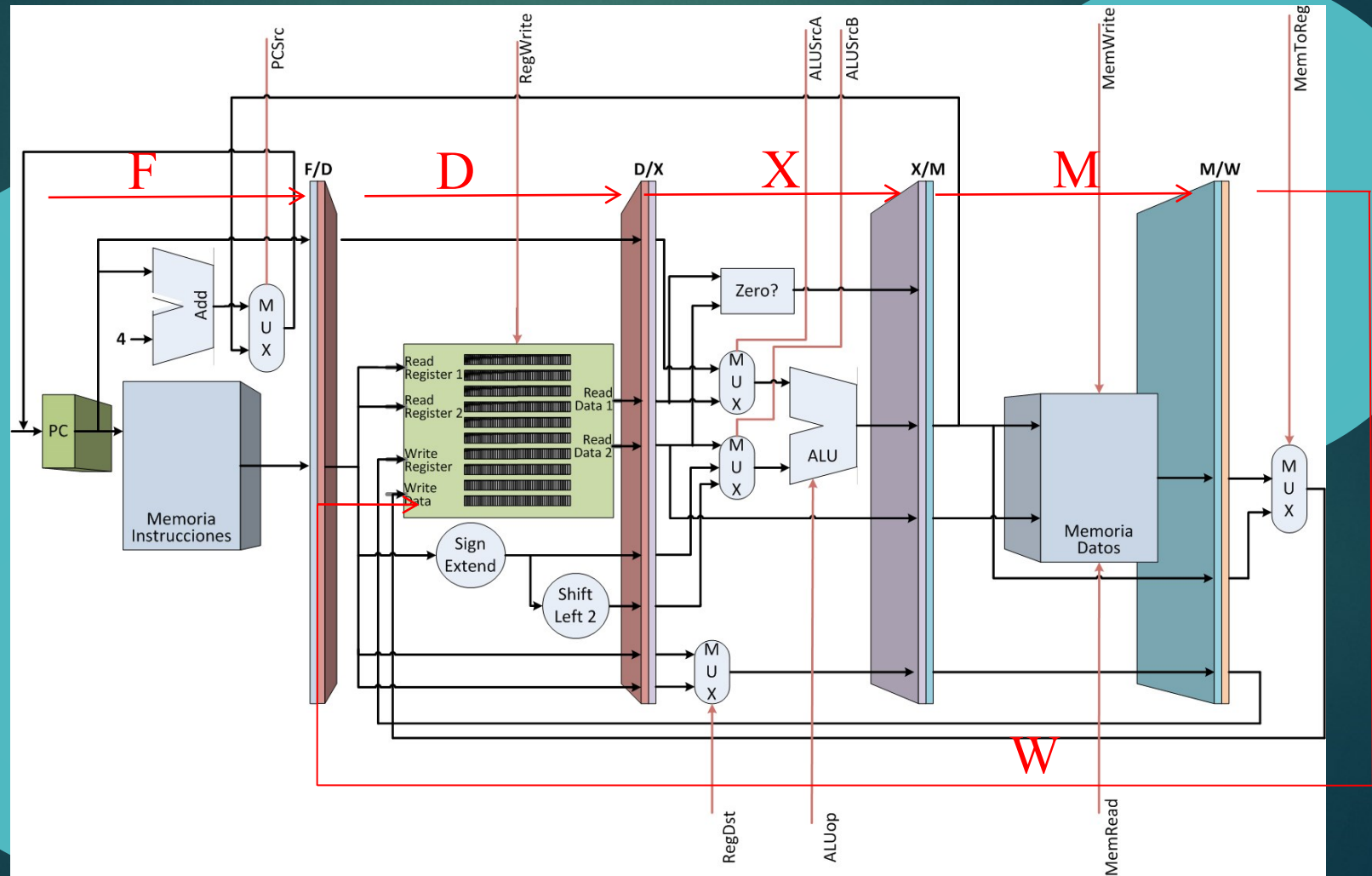
51

- Para implementar la segmentación del cauce se requiere intercalar registros entre cada etapa.
- Los registros (sincrónicos) separan las unidades funcionales, para que puedan operar con distintas instrucciones.
- En cada ciclo de reloj el dato avanza de una unidad funcional a la siguiente en el cauce, hasta completar las fases requeridas por a instrucción.
- El cauce de datos queda segmentado de la siguiente manera.

Segmentación y solapamiento del cauce en el nanoMIPS

52

Segmentación del cauce de datos.



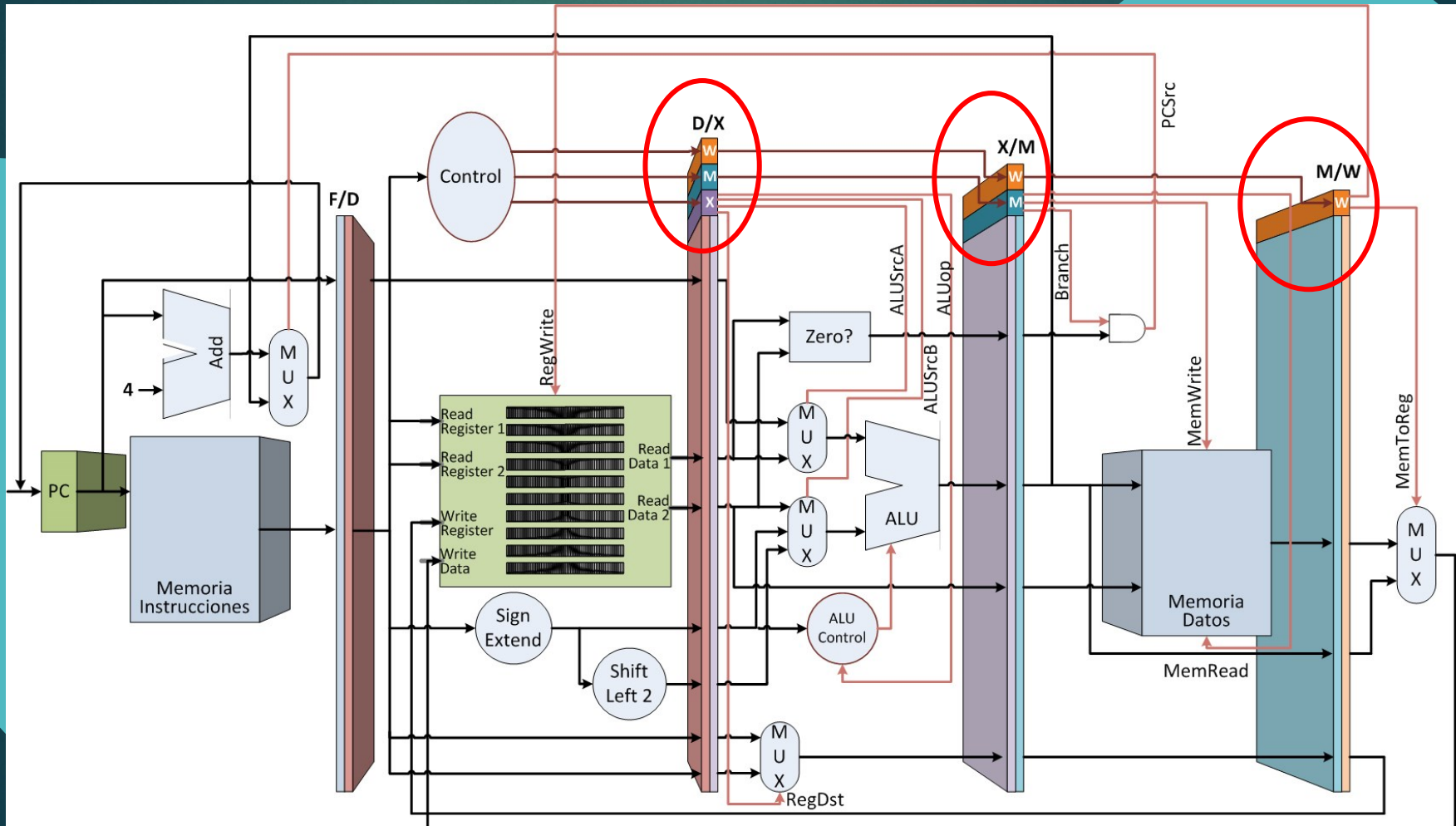
Segmentación y solapamiento del cauce en el nanoMIPS

53

- Así como se segmenta el cauce de los datos, también se debe hacer lo mismo con las señales que controlan las unidades funcionales: MI, BR, ALU, MD, BR.
- Eso es así porque cada Unidad funcional está operando con diferentes instrucciones (en diferentes estados de ejecución).
- Por ejemplo, mientras la unidad funcional BR (Banco de registros) está buscando los operandos de la instrucción I1, la unidad funcional MI (Memoria de instrucciones) estará buscando la instrucción I2.
- Las señales de control se segmentan igual que el cauce de datos, usando registros sincrónicos entre cada etapa, como se muestra en la figura siguiente.

Segmentación y solapamiento del cauce en el nanoMIPS

Segmentación del cauce de datos y de control.



Análisis de la Segmentación

55

- El máximo rendimiento teórico se obtiene cuando se completa una instrucción en cada ciclo de reloj. En esas condiciones todas las unidades funcionales están trabajando simultáneamente con distintas instrucciones.
- Si K es el número de etapas del cauce, entonces:

$$\text{Vel. procesador segmentado} = K \times \text{Vel. secuencial}$$
- El incremento potencial de la segmentación del cauce es proporcional al número de etapas del cauce.
- Notar que no se mejora la velocidad de ejecución de la instrucción, la segmentación incrementa la productividad (throughput), es decir la cantidad de instrucciones resueltas en un período de tiempo determinado.

Análisis de la Segmentación

56

Para el análisis anterior se han hecho **algunas suposiciones:**

- Todas las tareas o segmentos duran la misma cantidad de ciclos de reloj (tiempo).
- Todas las instrucciones siempre pasan por todas las mismas etapas.
- Todos las etapas pueden ser manejadas en paralelo (no hay conflictos para usarlas simultáneamente).
- No se consideraron instrucciones de salto.

Para hacer un análisis más preciso se requiere estudiar el comportamiento de la segmentación, pero en condiciones mas realistas.

Análisis de la Segmentación

57

Las primeras correcciones a hacer son:

1.- No todas las instrucciones necesitan todas las etapas.

- Ej: en el nanoMips la instrucción SW RT, inmed(RS) no utiliza W
- Ej: en el MSX88: un MOV AX,mem ; no requiere X

2.- No todas las etapas pueden ser manejadas en paralelo.

- Ej: si la memoria no estuviera dividida y fuera una sola, los segmentos F (búsqueda de la instrucción) y M (acceso a memoria) accederían ambos a la memoria (única) de datos e instrucciones.

3.- Los programas tienen instrucciones de salto.

Análisis de la Segmentación

58

Los conflictos que aparecen si se tienen en cuenta las correcciones anteriores, se denominan riesgos.

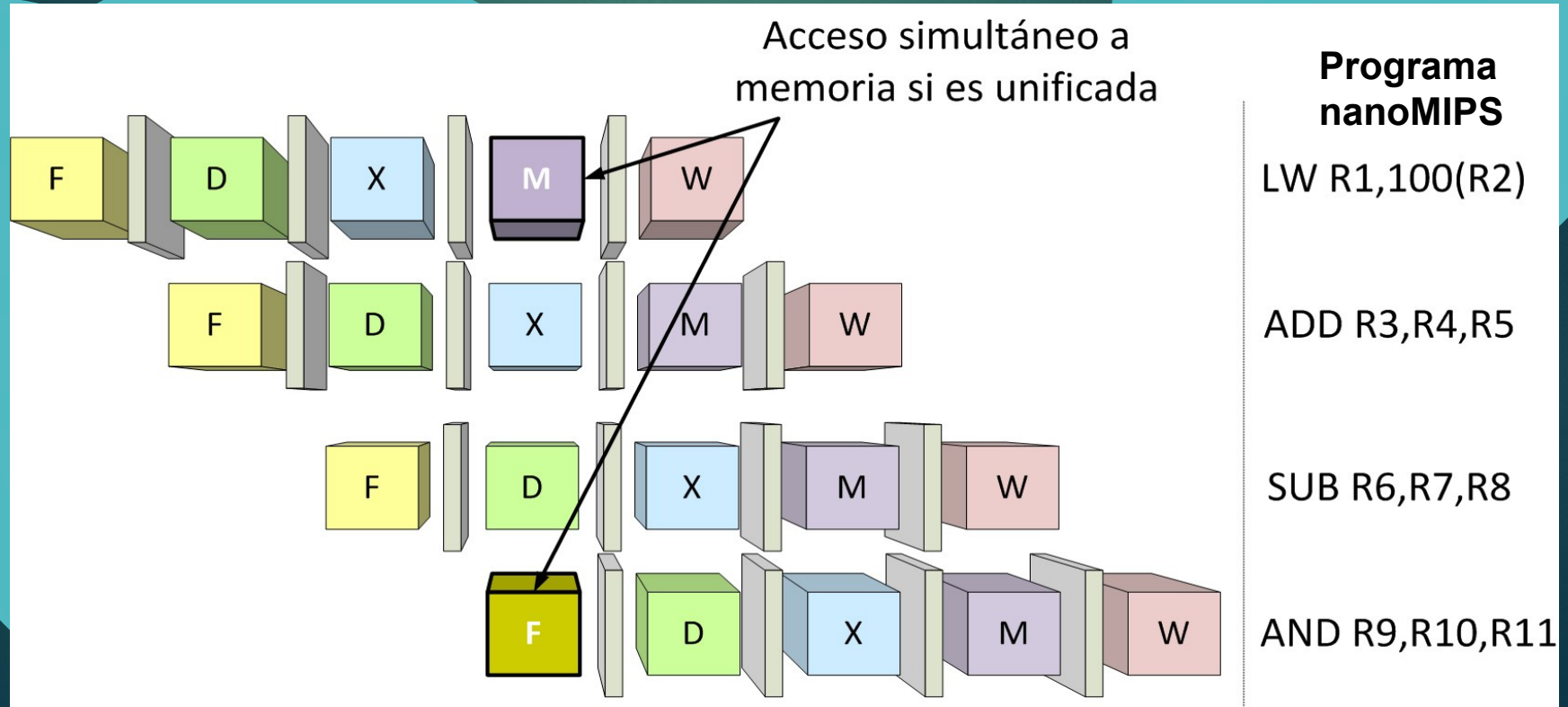
Existen 3 tipos de riesgos:

- 1.- Estructurales: son conflictos provocados por el uso de los “recursos”. Los recursos típicamente son: memoria, ALU, registros.
- 2.- Por dependencia de datos: son conflictos originados entre 2 o más instrucciones que comparten un mismo dato. Por ejemplo, una instrucción produce un resultado que lo necesita otra, ambas instrucciones dentro del cauce de datos.
- 3.- Por dependencia de control: son conflictos que ocurren cuando la ejecución de una instrucción depende de cómo se ejecute otra (ej.: un salto y los 2 posibles caminos).

Análisis de la Segmentación

Ejemplo de conflicto por uso de un recurso: la Memoria

Si la memoria del nanoMIPS fuera una sola (instrucciones y datos) habría un conflicto cada vez que se solapan los ciclos M y F.



Análisis de la Segmentación

60

Riesgos Estructurales

- En la imagen anterior se muestra la situación de un conflicto entre las fases F y M si la memoria fuera una sola.
- Por esta razón, la memoria del nanoMIPS está dividida en MI y MD, de manera de reducir los conflictos por accesos a memoria.
- Otra opción para evitar el conflicto por el acceso a un recurso es retardar (retrasar) la ejecución de la tarea los ciclos de reloj necesarios hasta que desaparece el conflicto. Pero al perder ciclos de reloj se pierde tiempo y baja la performance del procesador. Y además, el retraso puede generar nuevos conflictos que también deben resolverse de alguna manera.
- Esta solución se muestra en la figura siguiente.

Análisis de la Segmentación

Ejemplo de resolución del conflicto por uso de un recurso: la Memoria

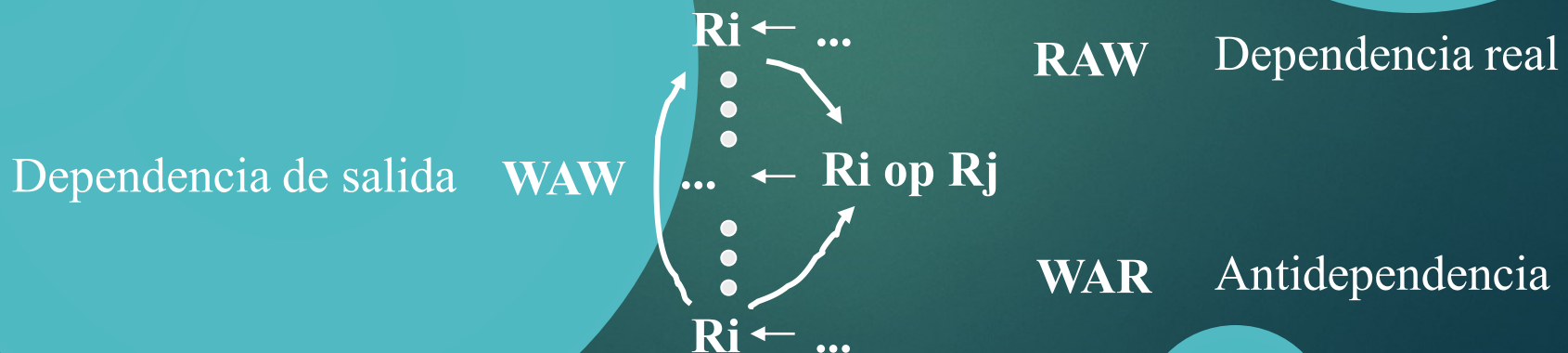
Se retrasa el inicio de la instrucción AND R9,R10,R11 en 1 ciclo (ciclo de parada) para evitar el conflicto con la instrucción LW R1,100(R2).



Análisis de la Segmentación

2.- Por dependencia de datos:

- Hay riesgos de conflictos por dependencia de datos, cuando un dato es usado en 2 o más segmentos del cauce.
- Los operandos fuente o destino de una instrucción no están disponibles en el momento en que se necesitan en una etapa determinada del cauce.
- Se pueden dar 3 tipos de dependencias:



Análisis de la Segmentación

63

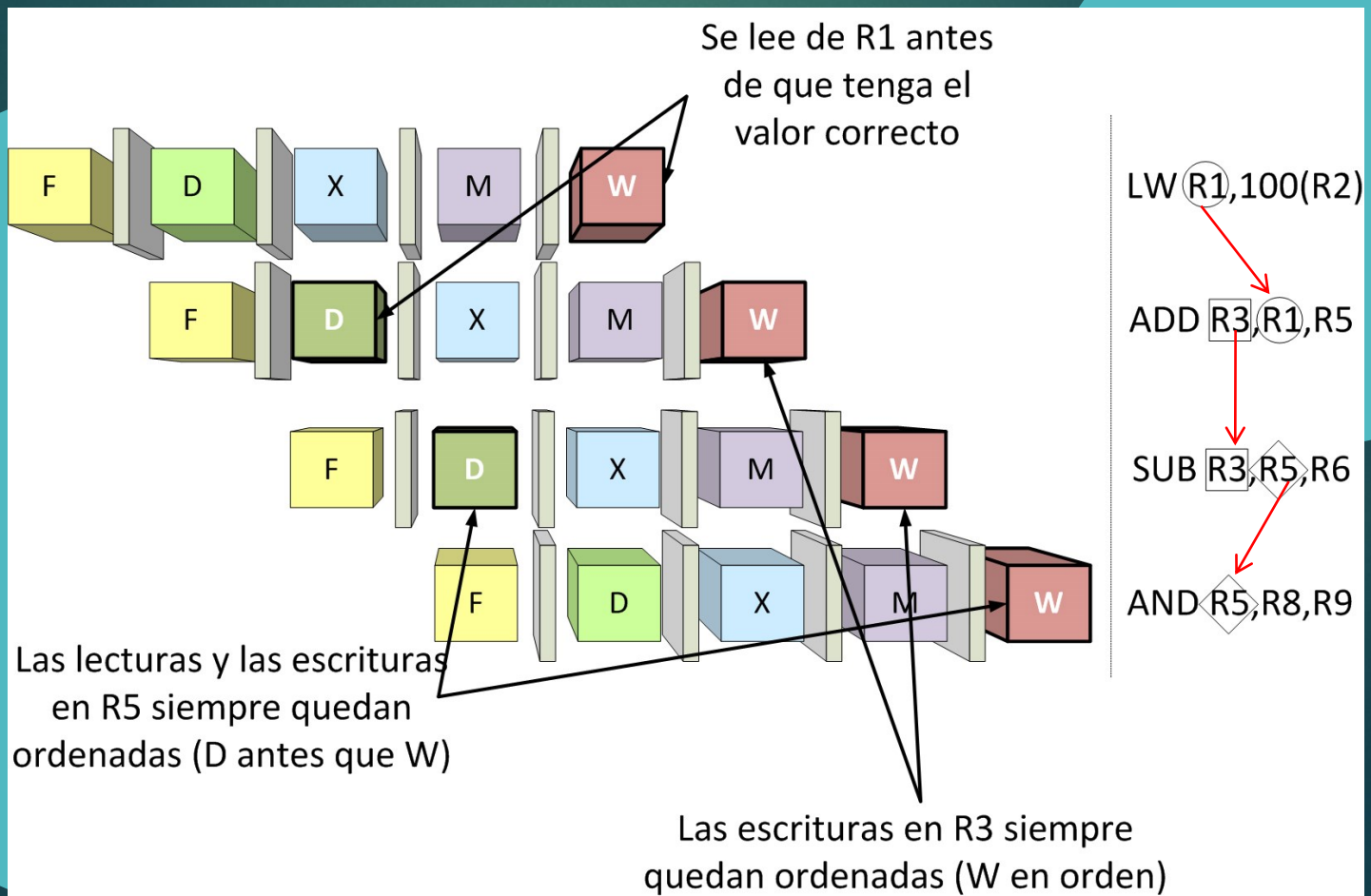
Los 3 tipos de dependencias de datos son:

- Lectura después de Escritura (RAW, dependencia verdadera)
 - Una instrucción escribe un dato que otra lee posteriormente
- Escritura después de Escritura (WAW, dependencia en salida)
 - una instrucción escribe un dato que otra escribe posteriormente
 - sólo ocurre si se permite que las instrucciones se adelanten unas a otras (alteración en la secuencia de ejecución de instrucciones)
- Escritura después de Lectura (WAR, antidependencia)
 - una instrucción lee un dato que otra escribe posteriormente
 - no se puede dar en nuestro cauce simple

Análisis de la Segmentación

Ejemplo de conflicto por dependencia de datos

Se muestra una secuencia de 4 instrucciones con los “3 tipos de dependencias”.

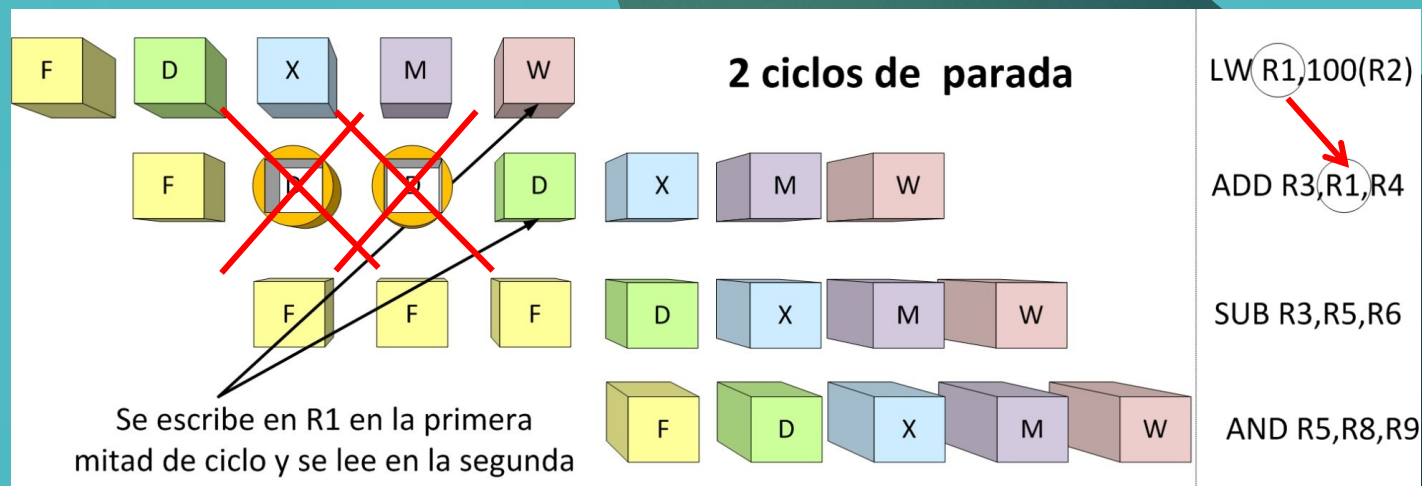


Análisis de la Segmentación

Ejemplo de resolución del conflicto por dependencia de datos

Se retrasa la decodificación D de la instrucción ADD R3,R1,R4 3 ciclos para esperar tener el dato disponible de la instrucción LW R1,100(R2).

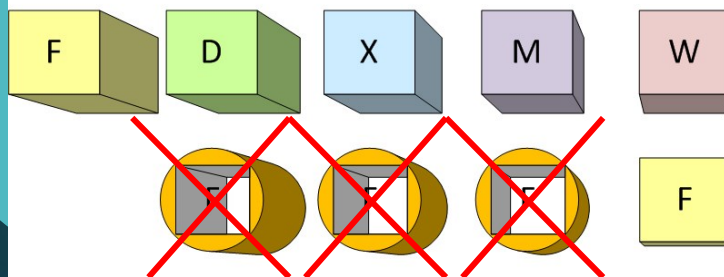
Se considera que la escritura W de la instrucción LW R1,100(R2) se hace en el primer medio ciclo y se puede leer por la instrucción ADD R3,R1,R4 en el segundo semiciclo.



Análisis de la Segmentación

3.- Por dependencia de control:

- Son riesgos que pueden ocurrir cuando se va a ejecutar una instrucción de salto condicional.
- Una instrucción tiene que calcular el nuevo valor que modifica el valor del PC. La próxima instrucción no puede comenzar hasta que no se resuelva el salto.
- Una forma de resolver el conflicto es retrasar varios ciclos (3 ciclos) la próxima instrucción hasta que se resuelva el cálculo de la instrucción de salto, como se puede apreciar en la figura siguiente.



3 ciclos de parada

BEQ R1,R2,etiqueta

Siguiente o etiqueta

Referencias

- W. Stallings, 5º Ed - Capítulo 11.
- Hwang & Briggs – Computer Architecture & Parallel Processing – Chapter 10.
- M. Pardo y A. Sacristán - Diseño y evaluación de arquitecturas de computadoras.

Arquitectura de Computadoras

CURSO 2022

Turno:
Clase 5

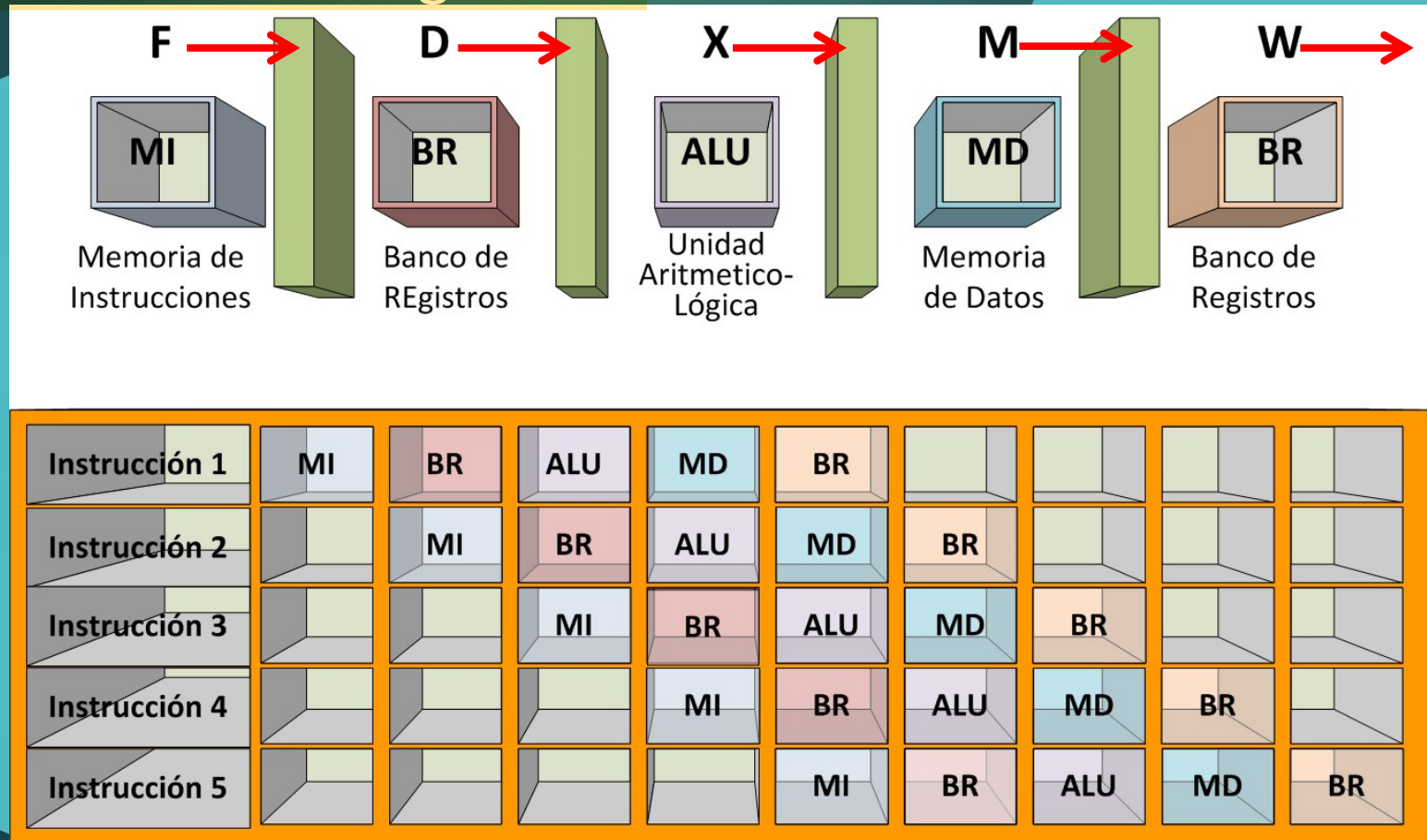
Resumen clase 5

Correcciones en procesadores segmentados

- Conflictos en la segmentación del cauce del nanoMips: estructurales, por dependencia de datos, por instrucciones de salto
- Soluciones a conflictos por dependencia de datos
 - Por hardware: ciclos de parada, adelantamiento
 - Por software: reordenamiento
- Soluciones a conflictos por instrucciones de salto
 - Por hardware: predictivas estáticas y dinámicas
 - Por software
- Segmentación del 80486
- Ejecución de la segmentación en un nanoMIPS multifuncional

Segmentación y solapamiento del cauce en el nanoMIPS

De acuerdo a lo presentado en la clase 4, la ejecución segmentada en 5 etapas en el nanoMIPS, de una secuencia de 5 instrucciones es la siguiente:



Problemas de la Segmentación

La imagen anterior representa una situación ideal. En la clase anterior se identificaron los tipos de conflictos que aparecen en un procesador segmentado real. **Estos riesgos pueden producir atascos en el flujo de las instrucciones.**

- **1.- Estructurales:** provocados por conflictos en el uso de los “recursos”. Los recursos típicamente son: memoria, ALU, registros.
- **2.- Por dependencia de datos:** son conflictos originados entre 2 o más instrucciones que comparten un mismo dato. Por ejemplo, una instrucción produce un resultado que lo necesita otra, ambas instrucciones dentro del cauce de datos.
- **3.- Por dependencia de control:** Ejecución de instrucciones que alteran la secuencia normal de ejecución, es decir, instrucciones de salto (condicional e incondicional).

Soluciones a los problemas de la Segmentación

Solución elemental

- La solución mas elemental para superar los atascos, producto de los 3 tipos de riesgos que se pueden presentar en un procesador segmentado, es agregar puntos de parada en el cauce hasta que el conflicto desaparezca.
- El gran problema que tiene este tipo de solución es que el rendimiento cae fuertemente por la cantidad de tiempos muertos que se insertan, inclusive puede llegar a ser hasta inútil aplicar la segmentación.
- Hay varias técnicas que permiten reducir y, en algunos casos, eliminar los efectos de los riesgos.
- A continuación se presentan algunas de las soluciones usadas en las máquinas con segmentación del cauce.

Soluciones a los problemas de la Segmentación

Solución a los conflictos estructurales por uso de los recursos

La solución a estos conflictos consiste básicamente en “agregar más hardware”. Algunas estrategias usadas son:

➤ 1.- Duplicación de recursos de hardware

Ej: agregar sumadores o restadores además de la ALU.

➤ 2.- Separación del recurso en conflicto

Ej: separar memorias de instrucciones y datos.

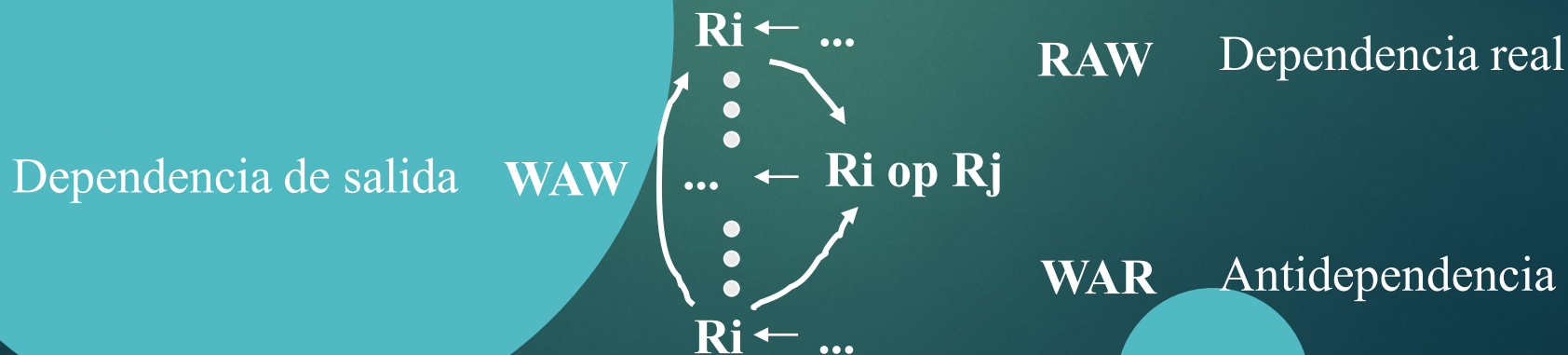
➤ 3.- Subdividir el acceso al recurso

Ej: en el acceso a los registros (banco de registros) la escritura de un registro se puede hacer en el primer semiciclo de reloj, y la lectura en el segundo semiciclo, con lo que en un ciclo se pueden hacer las 2 operaciones, siempre y cuando la velocidad del recurso lo soporte.

Soluciones a los problemas de la Segmentación

Solución a los conflictos por dependencia de datos

- Recordemos que **existen 3 tipos de dependencias de datos en un proceso de segmentación:**
 - **RAW:** lectura después de escritura
 - **WAR:** escritura después de lectura
 - **WAW:** escritura después de escritura
- Esas situaciones se muestran en el siguiente esquema:



Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos

Las soluciones se pueden implementar de 2 maneras:

➤ 1.- por Hardware

- Retardos en las etapas del cauce, es la solución ya vista.
- Adelantamiento de operandos (forwarding) entre etapas del cauce.

➤ 2.- por Software

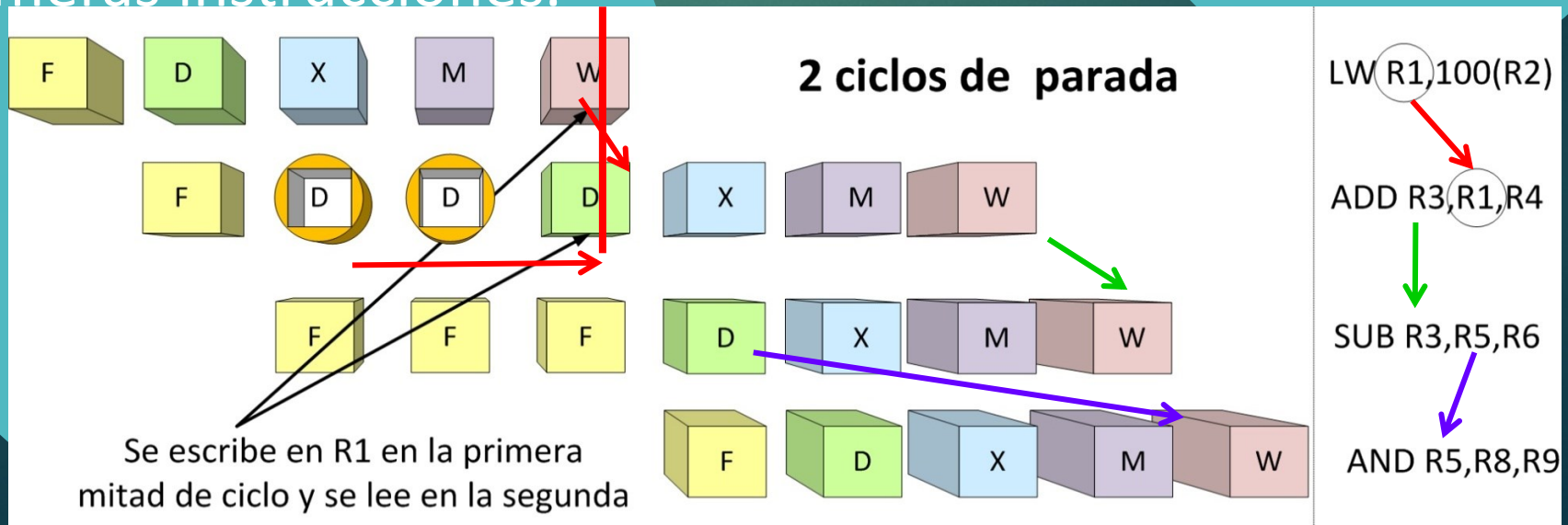
- Introducción de NOP (equivalente a insertar retardos), equivalente a introducir puntos de parada.
- Reordenación de código.

A continuación se van a analizar las diferentes técnicas.

Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con ciclos de parada

La solución más sencilla para eliminar conflictos por dependencia de datos es agregando retardos o ciclos de parada en la ruta de datos. En el siguiente ejemplo, hay una dependencia del tipo RAW con el registro R1 entre las 2 primeras instrucciones.



Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con ciclos de parada

- En el ejemplo anterior, el conflicto surge entre las instrucciones:
LW R1,100(R2)
ADD R3,R1,R4
- La instrucción ADD requiere, en la etapa D (decodificación), el valor de R1, que se carga en la instrucción previa LW en la etapa W (escritura en BR). La etapa D de ADD no se puede ejecutar hasta que se ejecute la etapa W de LW.
- Si se considera que R1 se escribe en el primer medio ciclo de la etapa W, y se lee en el segundo semiciclo de la etapa D, entonces agregando 2 ciclos de parada, como se muestra en la imagen anterior, se resuelve el conflicto.

Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con ciclos de parada

- Está claro que este tipo de solución permite resolver los conflictos por dependencia de datos.
- El gran problema de esta solución es que los ciclos de parada agregados reducen fuertemente la performance del procesador segmentado.

Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

- Este método consiste en pasar directamente (“adelantar”), desde una unidad funcional a otra, el resultado obtenido en una instrucción a las instrucciones siguientes que lo necesitan como operando.
- Si el dato que necesita la instrucción $i+1$ ya está calculado por la instrucción i , se puede llevar (adelantar) a la entrada de la etapa de la instrucción $i+1$ que lo necesita, sin esperar la etapa final de escritura de la instrucción i .
- Esta solución puede ser relativamente sencilla de implementar si se identifican todos los adelantamientos y se pueden adelantar los datos a las unidades que lo necesitan.

Soluciones a los problemas de la Segmentación

13

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

Consideremos esta solución para el caso del nanoMIPS.

En la siguiente secuencia de instrucciones hay dependencia del tipo RAW entre la escritura del registro R1 en la instrucción LW y su lectura en la instrucción SW.

```
LW R1, 100(R2)
```

```
SW R1, 0(R10)
```

En la imagen siguiente se muestra la relación entre las etapas de la segmentación y los ciclos donde se ejecutan la escritura de R1 (en la etapa 4, fase W, de la instrucción de LOAD) y la lectura de R1 (en la etapa 6, fase M, de la instrucción STORE).

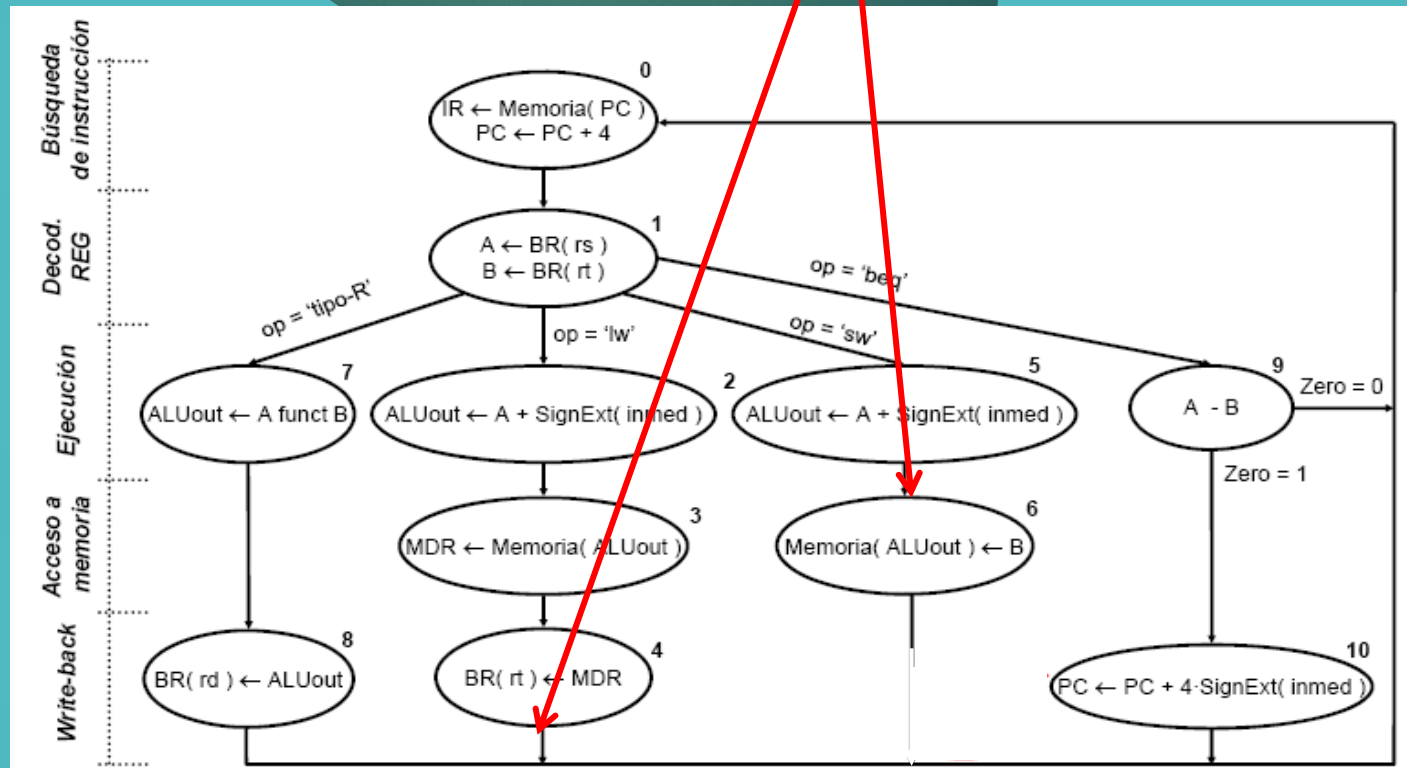
Soluciones a los problemas de la Segmentación

LW R1, 100(R2)

F D X M W

SW R1, 0(R10)

F D X M W



Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

- En la imagen anterior se puede apreciar que no se puede ejecutar el ciclo M de la instrucción SW hasta que se resuelva el ciclo W de la instrucción LW.
- En este caso, el dato que se va a cargar en R1 ya está disponible en la etapa previa (M) de la instrucción LW, aunque aún no se ha cargado en el R1 (etapa W).
- Si hubiera un camino que adelantara el dato desde la salida de la unidad M hasta la entrada de la misma unidad, no habría que esperar el ciclo de W de la instrucción LW, para ejecutar el ciclo M de la instrucción SW.
- Esta situación se muestra en la figura siguiente.

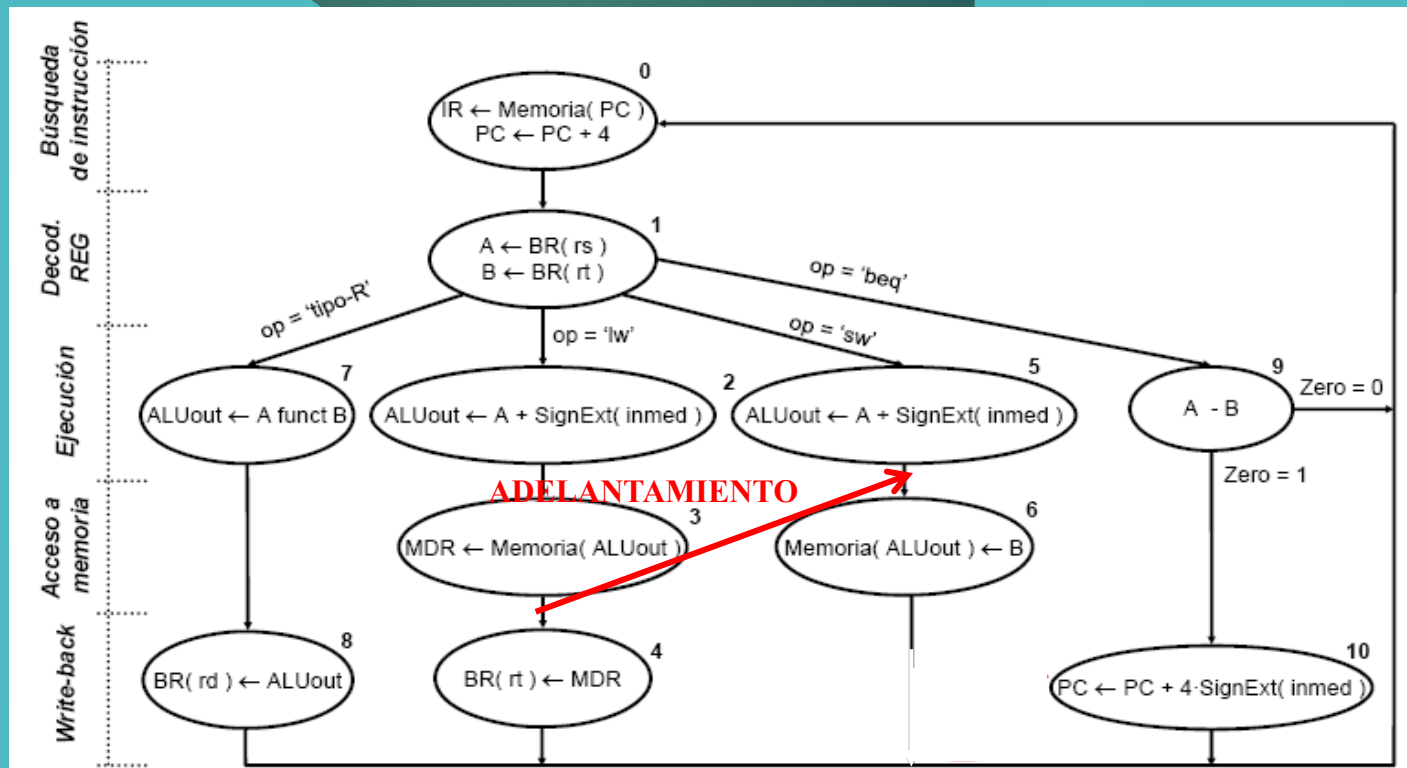
Soluciones a los problemas de la Segmentación

LW R1, 100(R2)

F D X M W

SW R1, 0(R10)

F D X M W



Soluciones a los problemas de la Segmentación

17

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

- Se puede hacer el mismo análisis para todas las secuencias de instrucciones, a fin de determinar los posibles casos de adelantamiento y su impacto en la mejora de la segmentación.
- En procesadores sencillos como el nanoMIPS este análisis es relativamente fácil. Si el procesador es más complejo el análisis puede resultar muy complicado.
- En el caso del nanoMIPS se pueden identificar 8 tipos de conflictos por dependencia de datos (del tipo RAW únicamente), y sus posibles soluciones con adelantamiento se muestran en la figura siguiente.

Soluciones a los problemas de la Segmentación

CASOS a considerar:

- 1) LW seguido de SW
- 2) LW seguido de Aritmética-Lógica
- 3) LW seguido de BEQ
- 4) Aritmético-Lógica seguida de SW
- 5) Aritmético-Lógica seguida de Aritmético-Lógica.
- 6) Aritmético-Lógica seguida de BEQ
- 7) LW seguido de LW
- 8) Aritmético-Lógica seguida de LW



LW R1, 100(R2)

SW R1, 0(R10)

LW R1, 100(R2)

ADD R3, R1, R8

LW R1, 100(R2)

BEQ R1, R3, etiqueta

ADD R1, R2, R3

SW R1, 0(R10)

ADD R1, R2, R3

SUB R8, R1, R6

ADD R1, R2, R3

BEQ R1, R7, etiqueta

LW R1, 0(R7)

LW R10, 100(R1)

ADD R1, R2, R3

LW R10, 100(R1)

Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

En la figura anterior, se han identificado **8 casos de conflictos por dependencia del tipo RAW:**

- **1)** LW seguido de SW
- **2)** LW seguido de Aritmética-Lógica
- **3)** LW seguido de BEQ
- **4)** Aritmético-Lógica seguida de SW
- **5)** Aritmético-Lógica seguida de Aritmético-Lógica.
- **6)** Aritmético-Lógica seguida de BEQ
- **7)** LW seguido de LW
- **8)** Aritmético-Lógica seguida de LW

Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

Para cada caso se considera:

- A la derecha está remarcada la dependencia en la secuencia de instrucciones
- A la izquierda se indica donde se debe implementar el adelantamiento.

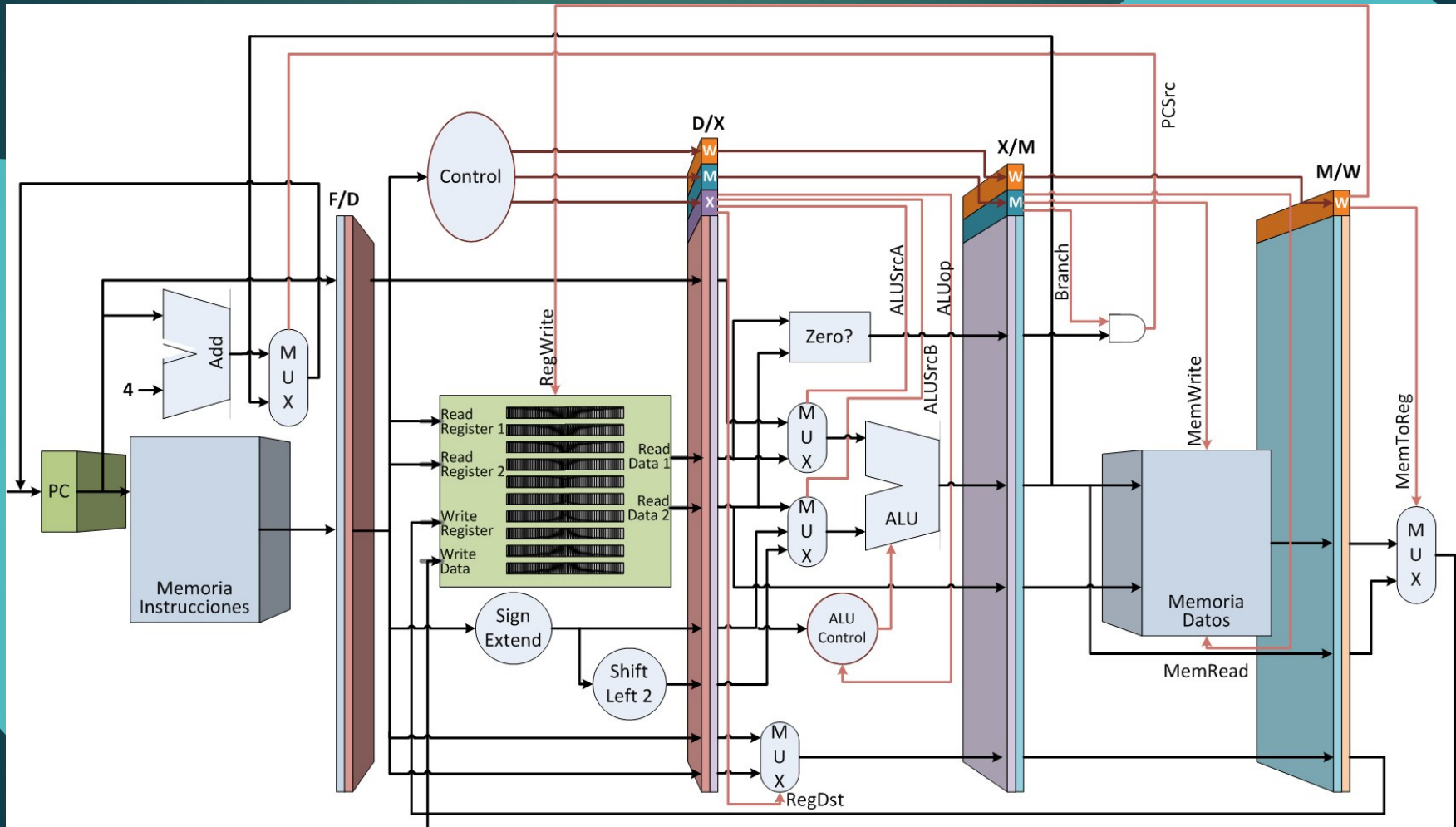
Por ejemplo, en el caso 1 de LW seguido de SW, el adelantamiento debería hacerse entre la salida de la M (memoria de datos) y la propia entrada de M.

A continuación se describe la forma de implementar estos adelantamientos, y su impacto en la organización interna de la máquina.

Soluciones a los problemas de la Segmentación

21

El modelo del nanoMIPS segmentado era el siguiente.



Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

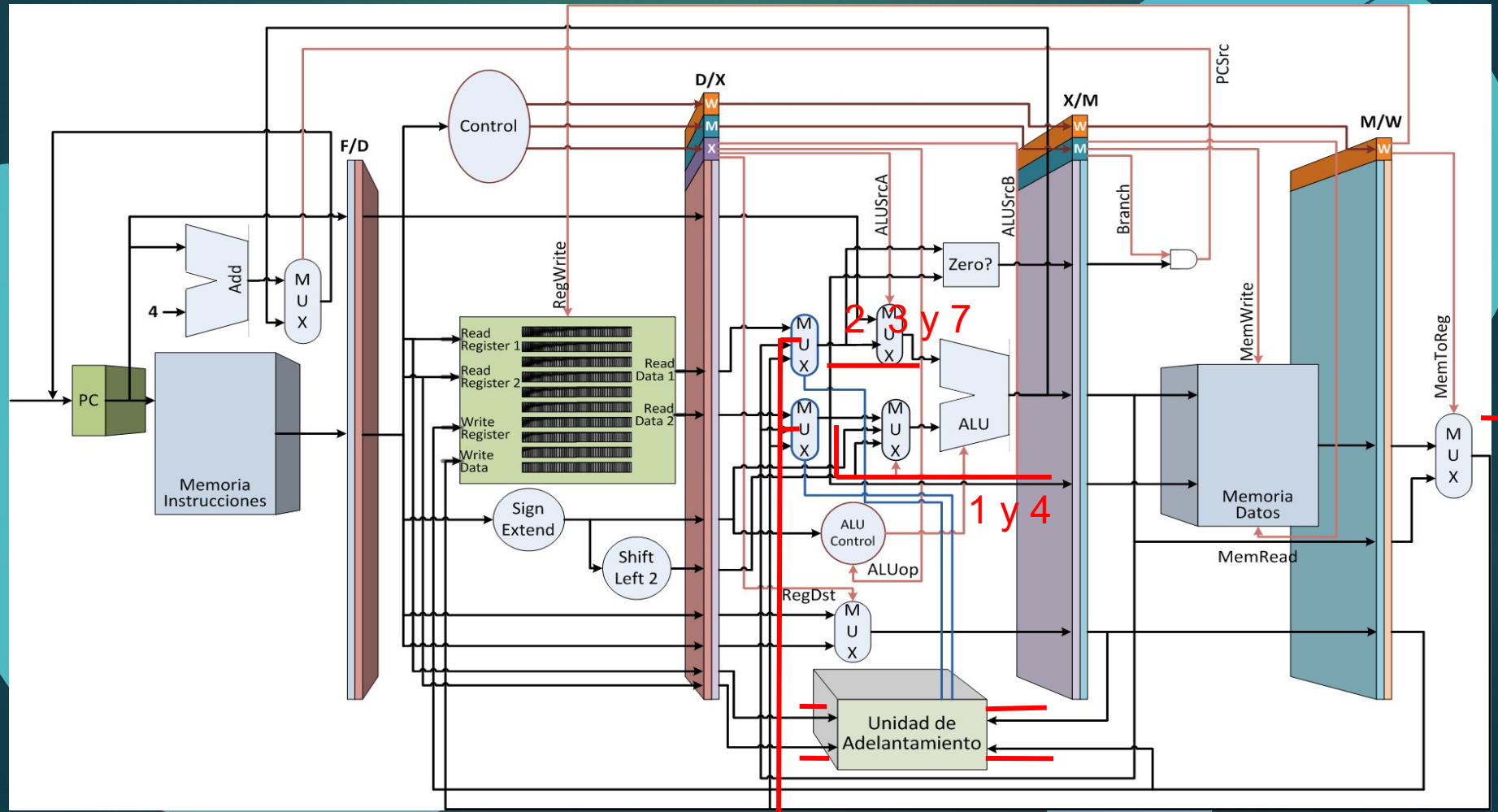
Los 8 casos encontrados, se pueden agrupar en 3 problemas de adelantamiento:

- 1) En los casos 1 y 4, adelantamiento desde la salida de M a la entrada de M.
- 2) En los casos 2, 3 y 7, adelantamiento desde la salida de M a la entrada de X.
- 3) En los casos 5, 6 y 8, adelantamiento desde la salida de X a la entrada de X.

Para solucionar el primer grupo (casos 1 y 4) se requiere prever un camino de datos directo desde la salida de M a la entrada de M, y para el segundo grupo (casos 2, 3 y 7) se requiere prever un camino de datos directo desde la salida de M a la entrada de X. Eso se puede resolver usando multiplexores (MUX) como se muestra a continuación.

Soluciones a los problemas de la Segmentación

Solución de adelantamiento en casos 1, 2, 3, 4 y 7:



Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

En la imagen anterior, la trayectoria de los datos desde la M hasta el BR (banco de registros) tiene una derivación que termina en multiplexores (MUX), que permiten seleccionar distintas alternativas de fuentes de datos a X (ALU) y a M (memoria de datos).

Cuando una unidad auxiliar, identificada como unidad de adelantamiento detecta en los campos de las instrucciones que se están ejecutando, las dependencias 1,2,3,4 y 7, selecciona en los MUX asociados a ella, las entradas necesarias para adelantar los datos requeridos (por X o M).

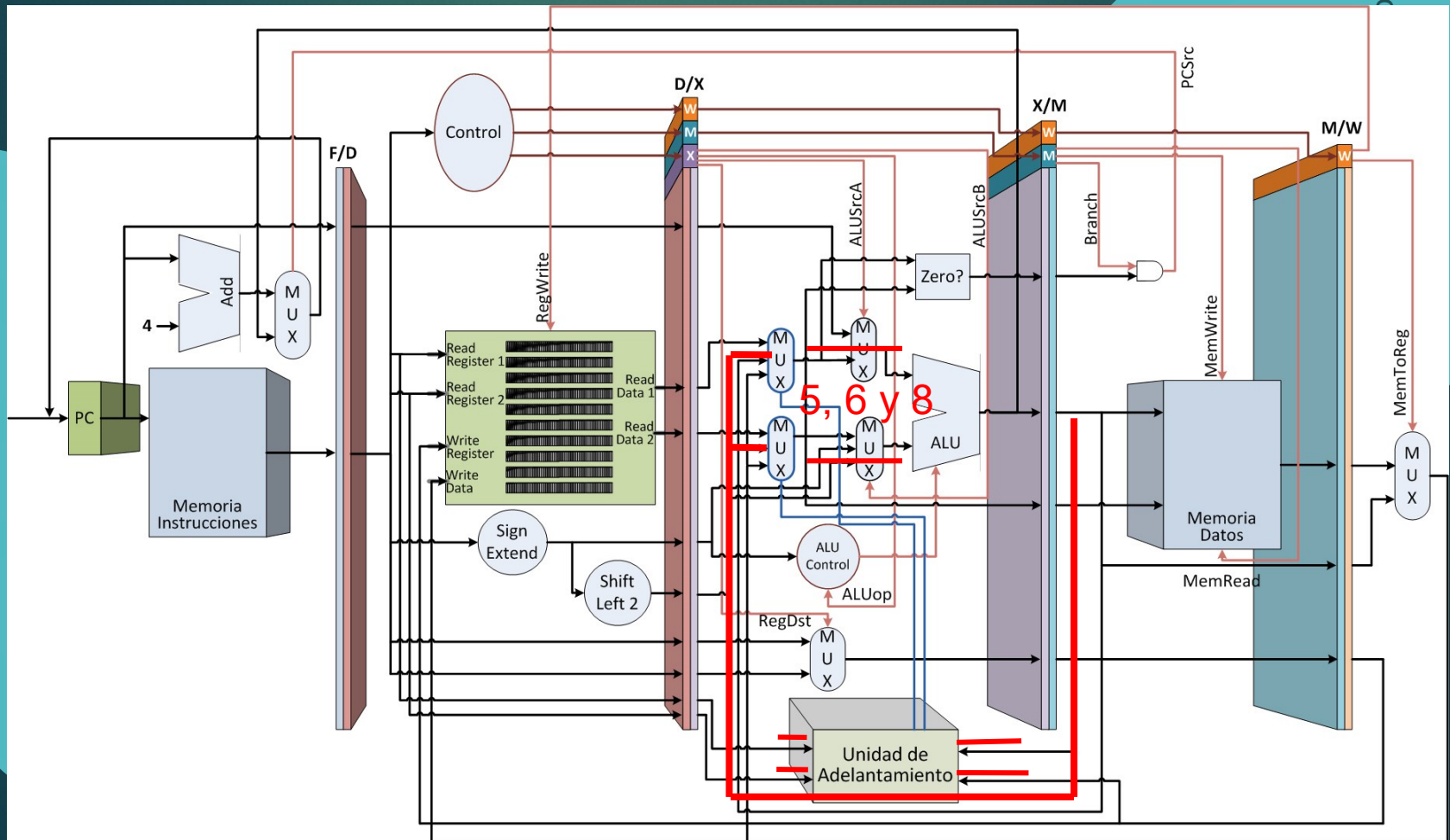
Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

Para solucionar el tercer grupo (casos 5, 6 y 8) se requiere prever un camino de datos directo desde la salida de X a la entrada de X. Eso se puede resolver usando multiplexores (MUX) como se muestra a continuación.

Soluciones a los problemas de la Segmentación

Solución de adelantamiento en casos 5, 6 y 8:



Soluciones a los problemas de la Segmentación

27

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

En la imagen anterior, la trayectoria de los datos desde la X hasta M o el BR (banco de registros) tiene una derivación que termina en los mismos multiplexores (MUX), que permiten seleccionar distintas alternativas de fuentes de datos a X (ALU) y a M (memoria de datos).

Cuando la unidad de adelantamiento detecta en los campos de las instrucciones que se están ejecutando, las dependencias 5, 6 y 8, selecciona en los MUX asociados a ella, las entradas necesarias para adelantar los datos requeridos (por X).

Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos con forwarding o adelantamiento

Conclusión:

En los análisis anteriores se puede apreciar que existen soluciones a los conflictos por dependencia de datos que permiten reducir o eliminar el impacto de estos conflictos en la performance del procesador.

Los cambios introducidos no son complicados de implementar, y sus resultados justifican ampliamente su uso.

Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos por software

- Otra técnica que se usa para evitar los atascos por dependencia de datos, es a través del software.
- Esta solución es realizada por el compilador, por lo que, al igual que las técnicas por hardware, es “transparente” al programador. Hay 2 posibles soluciones:
 - Introducción de instrucciones NOP (es decir, retardos) entre las instrucciones que tienen dependencia de datos, pero genera retardos que reducen la performance del procesador (equivalente al uso de puntos de parada).
 - Reordenamiento de instrucciones, para tratar de aumentar la separación entre las instrucciones con dependencia de datos, pero hay que tener cuidado con modificar el comportamiento del programa.

Soluciones a los problemas de la Segmentación

Solución a conflictos por dependencia de datos mediante reordenamiento de instrucciones:

```
1 LW R2,0(R1)
2 LW R3, 100(R0)
3 ADD R6, R2, R3
4 SUB R4, R6, R7
5 LW R10, 0(R0)
6 OR R8, R1, R10
7 SW R8, 100(R0)
8 ADD R9, R12, R10
9 BEQ R9, R0, etiq
```

```
LW R2, 0(R1)
LW R3, 100(R0)
LW R10, 0(R0)
ADD R6, R2, R3
OR R8, R1, R10
SUB R4, R6, R7
ADD R9, R12, R10
SW R8, 100(R0)
BEQ R9, R0, etiq
```


Soluciones a los problemas de la Segmentación

31

Solución a conflictos por dependencia de datos mediante reordenamiento de instrucciones:

En la imagen anterior se muestra a la izquierda el programa original (de 9 instrucciones), y a la derecha, el programa reordenado por el compilador al detectar conflictos de dependencia de datos.

Por ejemplo, hay dependencias entre las instrucciones 2 y 3 (registro R3), y entre la 3 y la 4 (registro R6). La instrucción 4 no se puede reubicar porque tiene dependencia con la 3, pero la instrucción 5 no depende de ninguna de las anteriores. Entonces, reubicando la instrucción 5 a continuación de la 2, se elimina la dependencia sin alterar la lógica del programa.

El compilador continúa con este proceso intentando reordenar las instrucciones para evitar los conflictos. En caso de no poder resolverlos, deberá insertar instrucciones de NO-OPERACIÓN (NOP) para eliminar los que persisten luego del reordenado.

Soluciones a los problemas de la Segmentación

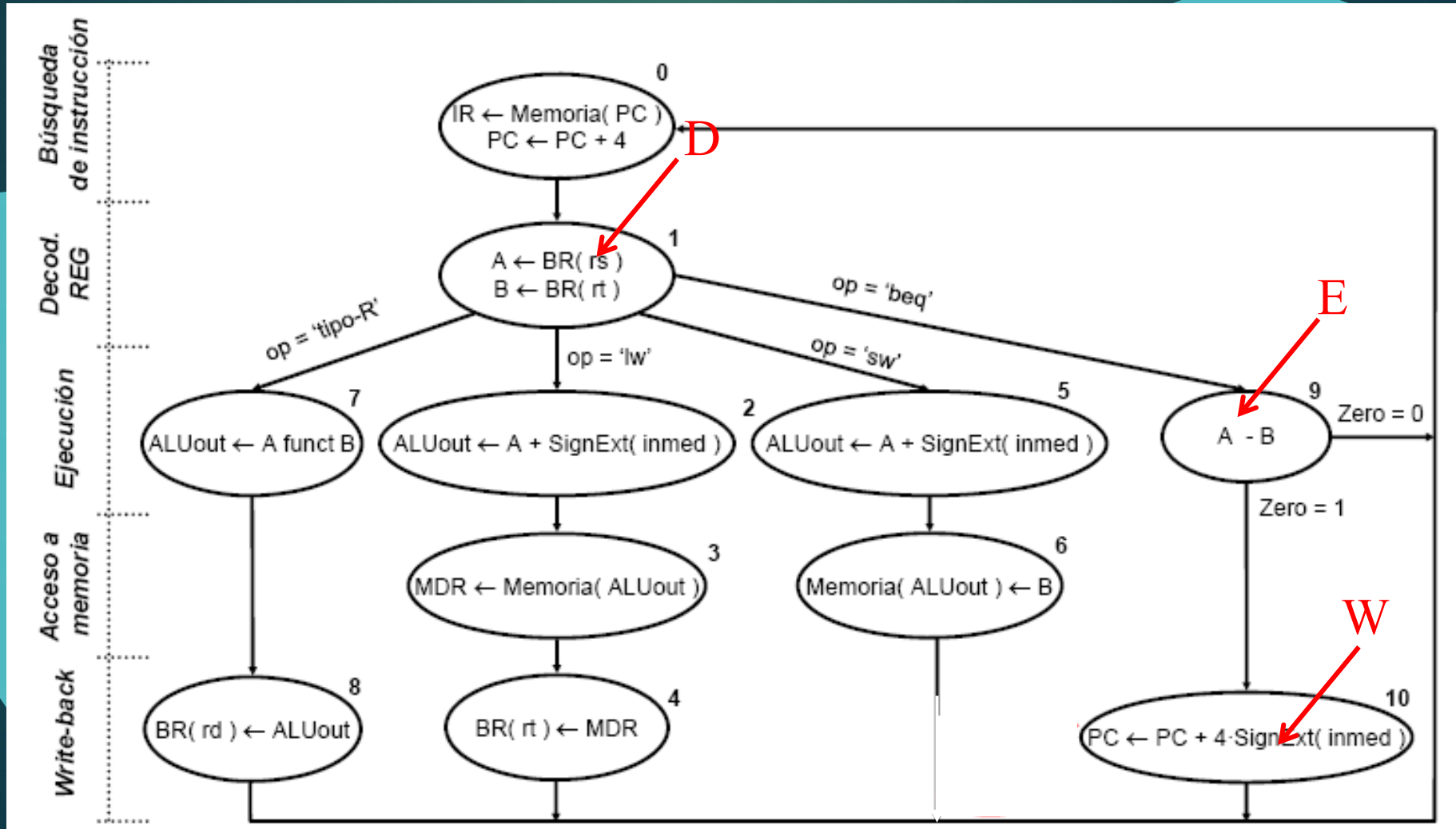
Solución a los conflictos por dependencia de control

El tercer tipo de riesgo que puede haber en la segmentación es la presencia de instrucciones de salto, que pueden ser de 2 tipos:

- **Incondicional:** donde la dirección de destino se debe determinar lo más pronto posible, dentro del cauce, para reducir la penalización.
- **Condicional:** introduce el riesgo adicional por la dependencia entre la condición de salto y el resultado dependiente de una instrucción previa.
 - Por ejemplo: en la instrucción de salto (condicional) BEQ en el nanoMIPS, recién se calcula la dirección de salto durante la fase W. Esta situación se muestra en la figura siguiente.

Soluciones a los problemas de la Segmentación

Instrucción de salto condicional BEQ en el nanoMIPS.

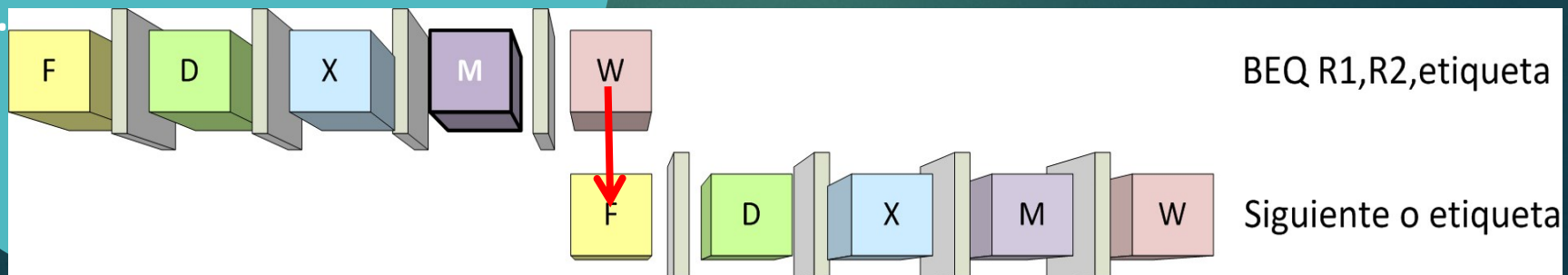


Soluciones a los problemas de la Segmentación

Solución a los conflictos por dependencia de control

Igual que antes, la forma más elemental de resolver el conflicto es agregando tiempos muertos (ciclos de parada) hasta que desaparece el problema.

En el caso del nanoMIPS, se requerirían 3 ciclos de reloj para empezar la instrucción siguiente a la de salto, considerando que se puede calcular el salto en el primer semiciclo del ciclo W de la instrucción BEQ, y se puede leer el resultado en el segundo semiciclo del ciclo F de la instrucción a donde se salta.



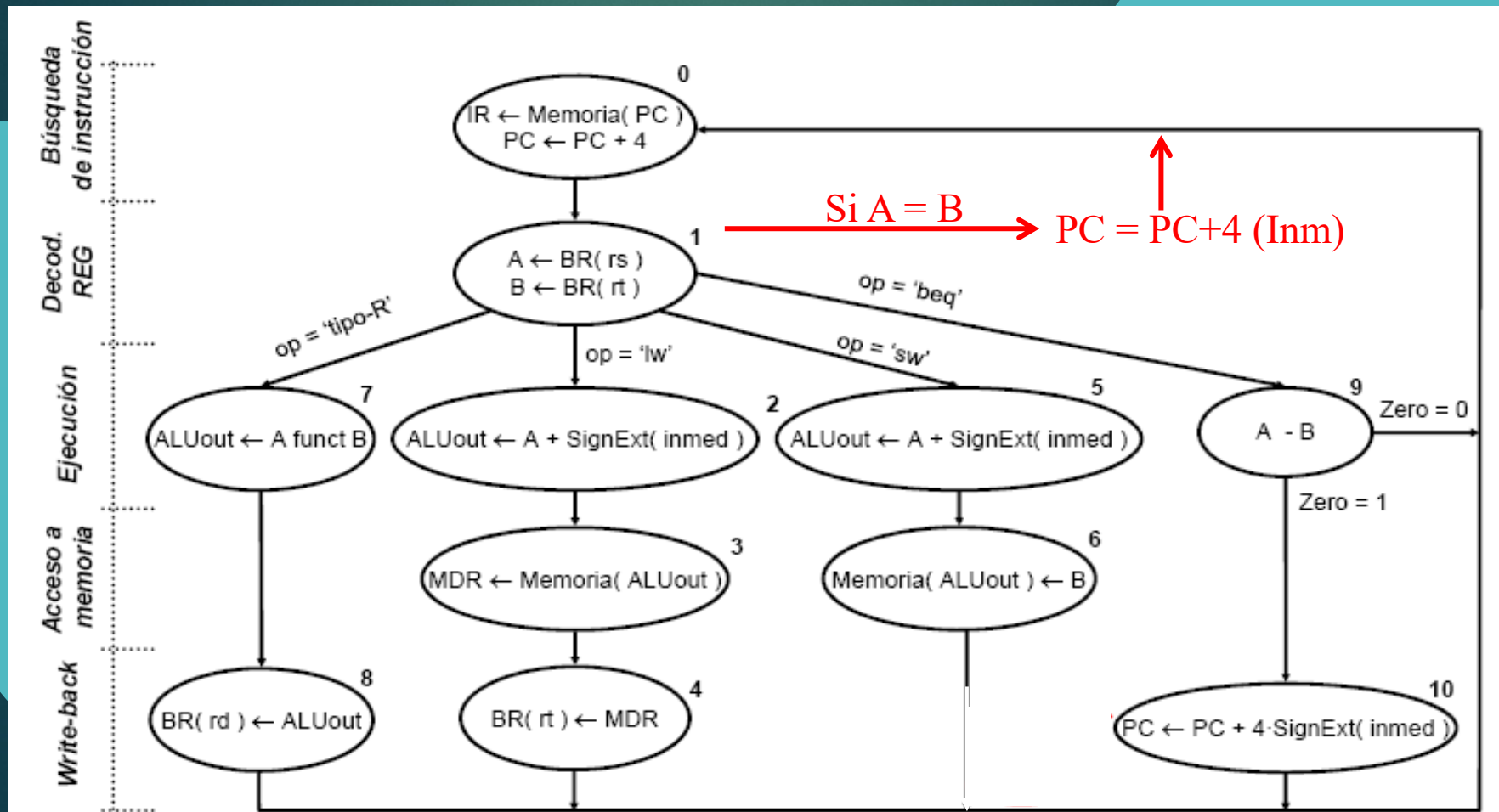
Soluciones a los problemas de la Segmentación

Solución a los conflictos por dependencia de control – penalización con adelantamiento

- Se puede mejorar la situación anterior si se modifica la ruta de datos para reducir la cantidad de ciclos muertos, adelantando la resolución de los saltos a la etapa D, donde se decodifica y se accede a los registros.
- El objetivo del adelantamiento es evaluar tempranamente la condición de salto, usando, por ejemplo, un restador extra a la salida del banco de registros, en lugar de esperar a la etapa X asociada a la ALU, para determinar la condición de igualdad de la instrucción BEQ.
- Además se puede agregar un sumador adicional para calcular la dirección de salto, en lugar de esperar a la etapa X con la ALU, para adelantar la obtención del salto.

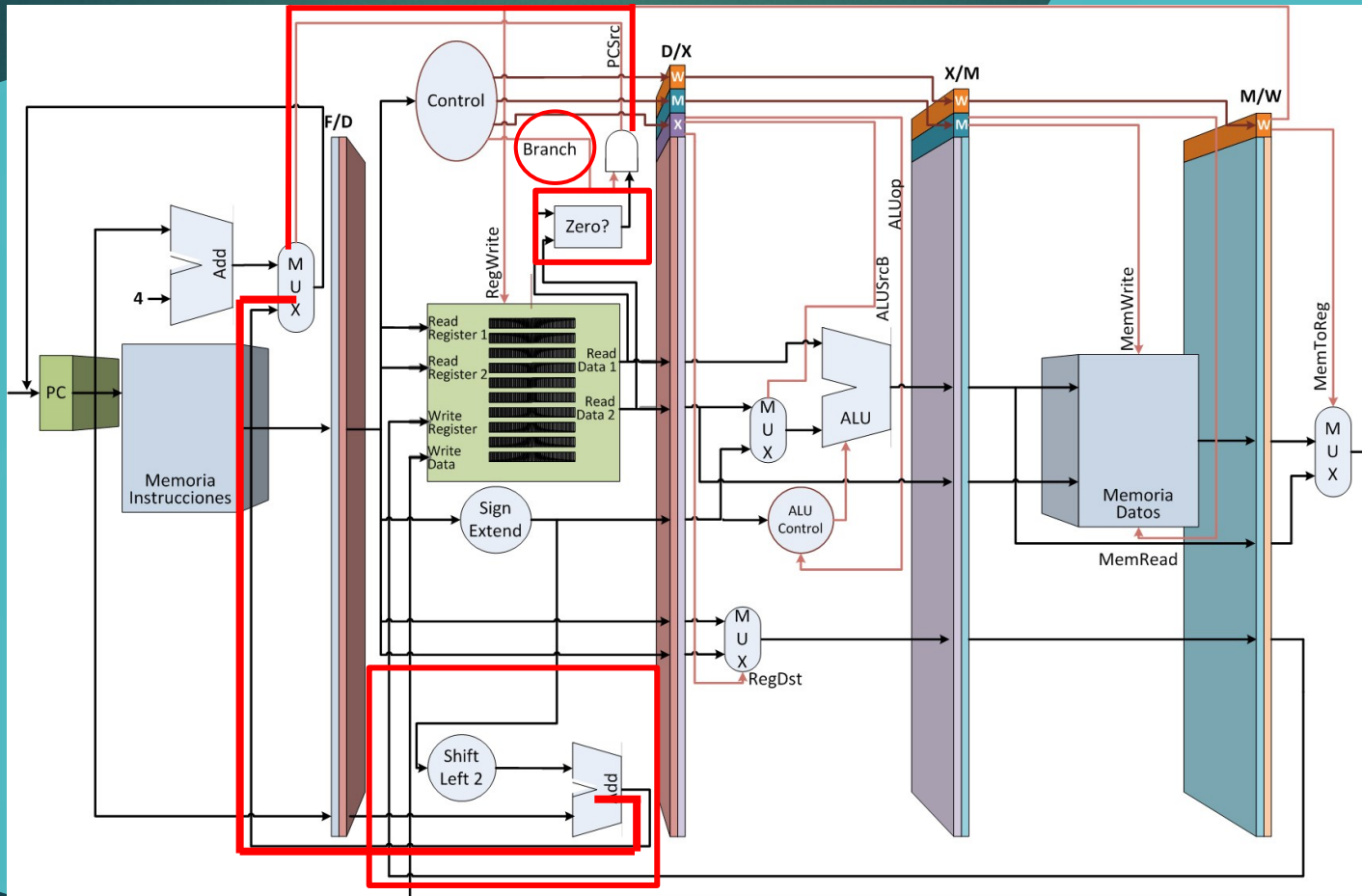
Soluciones a los problemas de la Segmentación

Adelantamiento en el nanoMIPS a la resolución de la instrucción BEQ y cálculo de la próxima instrucción



Soluciones a los problemas de la Segmentación

Adelantamiento en el nanoMIPS a la resolución de la instrucción BEQ y cálculo de la próxima instrucción



Soluciones a los problemas de la Segmentación

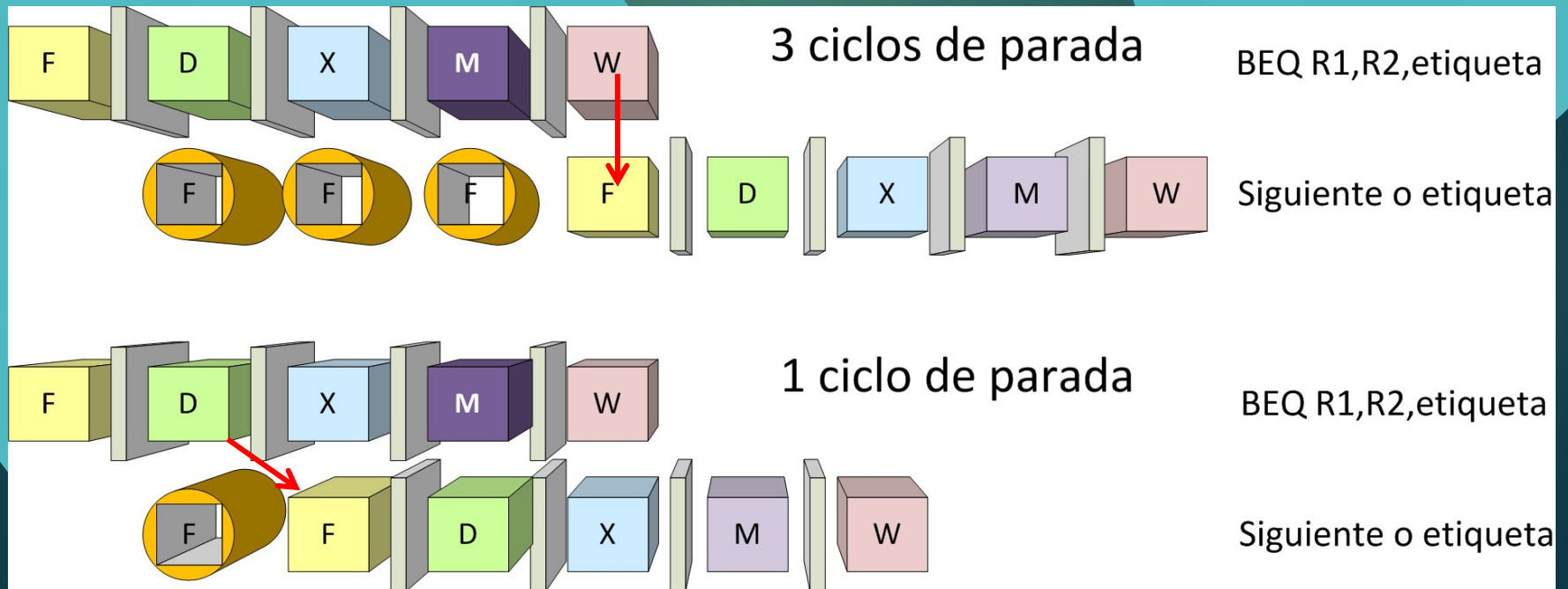
Adelantamiento en el nanoMIPS a la resolución de la instrucción BEQ y cálculo de la próxima instrucción

- Se puede ver en la imagen anterior que a la salida del BR se ha agregado una lógica para detectar en forma temprana la condición de igualdad en una instrucción de BEQ, que controla un MUX.
- El MUX selecciona el valor con el que se va a cargar el PC.
- Una de las entradas al MUX es el valor actual del PC con el desplazamiento incluido en la instrucción de BEQ.
- El hardware agregado determina:
si: la instrucción es BEQ, y $A=B$
entonces: $PC = PC + \text{desplazamiento}$
donde desplazamiento se calcula con el modo de direccionamiento.

Soluciones a los problemas de la Segmentación

Adelantamiento en el nanoMIPS a la resolución de la instrucción BEQ y cálculo de la próxima instrucción

Con la ruta de datos modificada, la respuesta del nanoMIPS a una instrucción de salto BEQ se pierde 1 solo ciclo de reloj, en lugar de los 3 que se perdían originalmente.



Soluciones a los problemas de la Segmentación

Técnicas de tratamiento de saltos

En máquinas más complejas, la solución antes presentada para el nanoMIPS puede no ser tan sencilla de implementar. En general las técnicas para mitigar el impacto, en la performance de procesadores segmentados, de las instrucciones de salto pueden ser de 2 tipos:

- Técnicas por hardware: que consiste en resolver los conflictos a nivel de hardware mediante predicción de saltos.
- Técnicas por software: que intentan resolver los conflictos por software a nivel del compilador.

Soluciones a los problemas de la Segmentación

Técnicas de resolución de instrucciones de salto por hardware

Estas técnicas se basan en usar estrategias para predecir el resultado de la instrucción de salto. Pueden ser de 2 tipos:

- **Técnicas estáticas:** no tienen en cuenta información previa de la ejecución del programa.
- **Técnicas dinámicas:** tienen en cuenta la historia previa del programa en ejecución.

Soluciones a los problemas de la Segmentación

42

Técnicas de predicción de saltos por hardware - estática

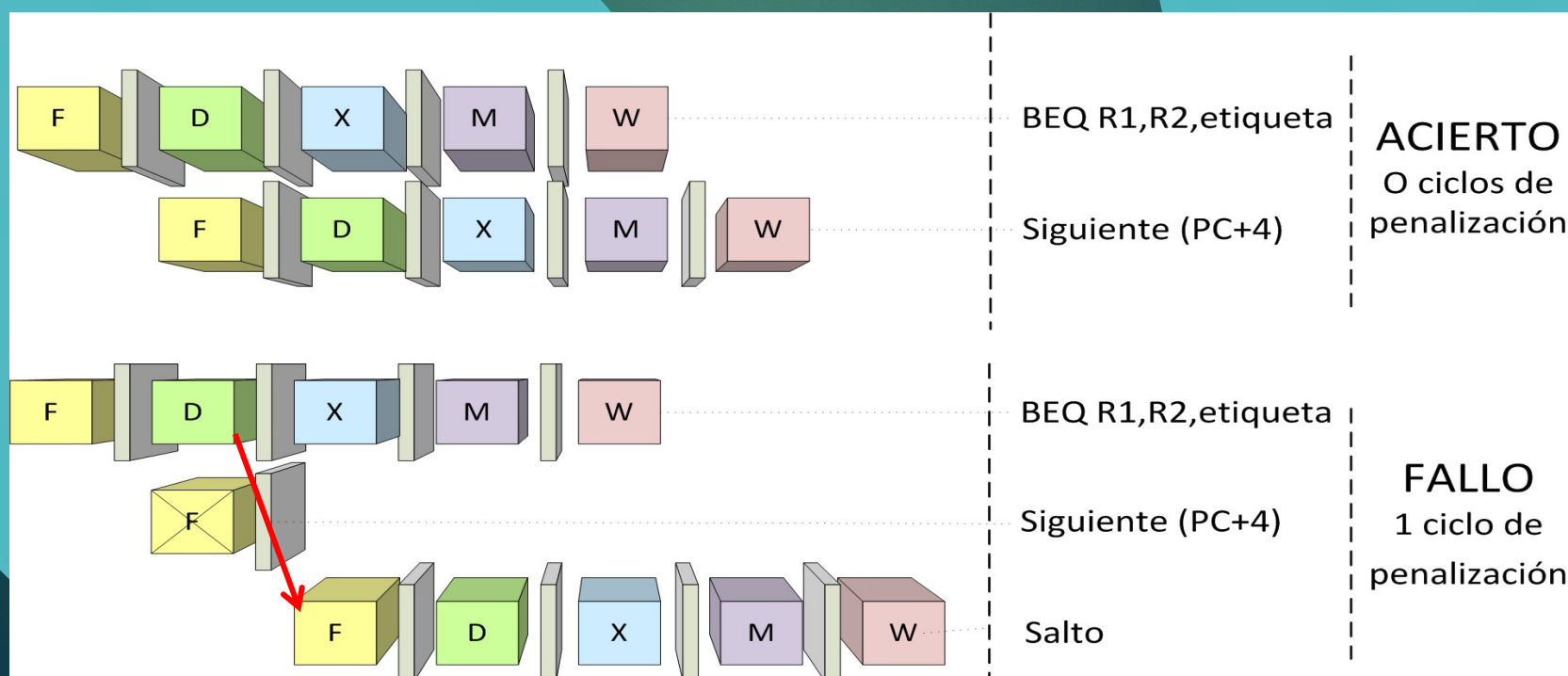
Básicamente se presume una condición (salta o no salta) y se ejecuta en base a esa predicción.

- **Predcir que nunca se salta:** asume que el salto no se producirá, y por lo tanto siempre capta la siguiente instrucción.
- **Predcir que siempre se salta:** asume que el salto se producirá, y por lo tanto siempre capta la instrucción destino del salto.

Soluciones a los problemas de la Segmentación

Técnicas de predicción de saltos por hardware - estática

En el nanoMIPS siempre capta la próxima instrucción, es decir predice que no va a saltar. Si no salta no se pierde ningún ciclo, si salta se pierde 1 ciclo.



Soluciones a los problemas de la Segmentación

44

Técnicas de predicción de saltos por hardware - estática

En el nanoMIPS siempre capta la próxima instrucción, es decir predice que no va a saltar. Si no salta no se pierde ningún ciclo, si salta se pierde 1 ciclo.

➤ **Ventajas:**

- Sencilla de implementar
- Pequeños cambios en la Unidad de control
- Ruta de datos sin cambios

➤ **Desventajas:**

- La instrucción a descartar puede afectar el estado del procesador (registros y/o memoria).
- En el nanoMIPS el ciclo que se descarta es de búsqueda de la instrucción (ciclo F) que no modifica el procesador.

Soluciones a los problemas de la Segmentación

Técnicas de predicción de saltos por hardware - dinámica

- Está técnica se basa en el análisis de datos previos (la “historia”) sobre como se comporta el programa, para predecir la acción.
- Algunos ejemplos de técnicas de predicción por hardware son:
 1. Conmutador Saltar/no saltar
 2. Tabla de historia de saltos (branch history table)
 3. Predicción según el código de operación
 4. Varios cauces (uno por cada opción de salto) (Multiple stream)
 5. Precaptación del destino del salto (Prefetch branch target)
 6. Buffer de bucles

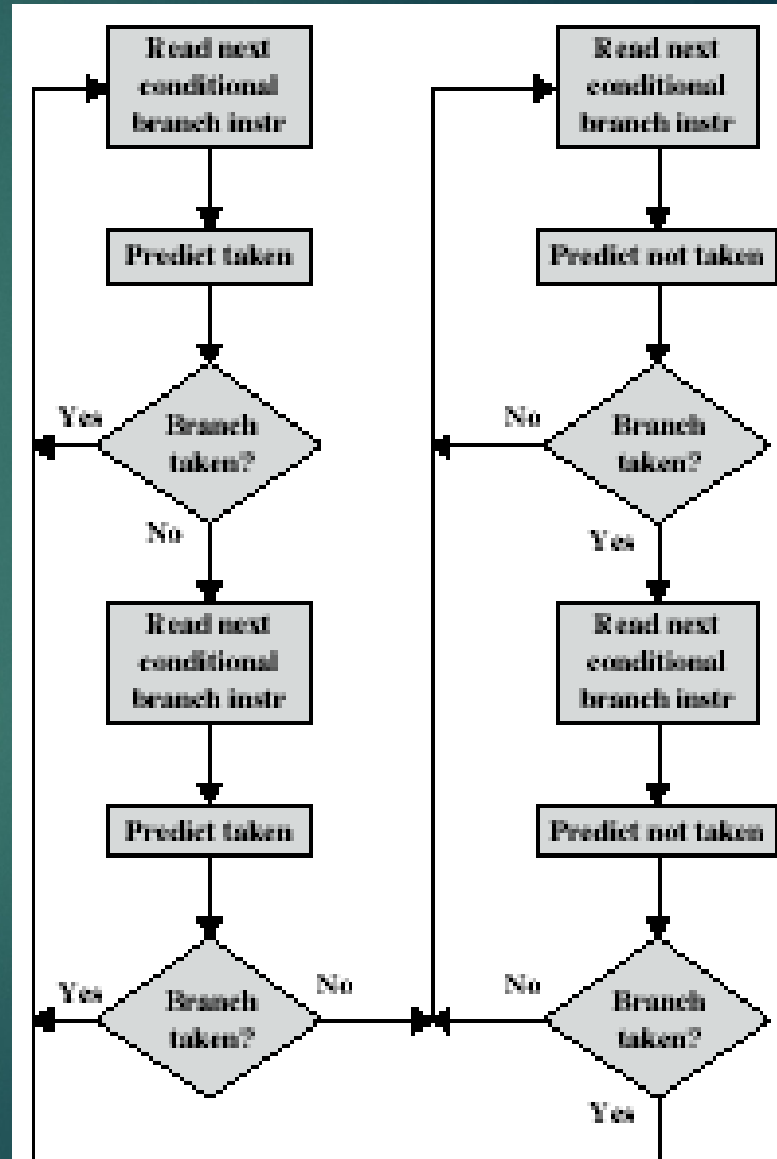
Soluciones a los problemas de la Segmentación

46

Conmutador Saltar/no saltar

Estando en uno de las 2 condiciones (predecir que salta o predecir que no salta), se requieren 2 predicciones fallidas consecutivas para conmutar al otro estado.

Esta predicción tiene un buen comportamiento en lazos donde el resultado de una consulta (por ejemplo un contador que tiene que llegar a 0) se puede repetir muchas veces antes de que cambie el resultado de la consulta.



Soluciones a los problemas de la Segmentación

Tabla de historia de saltos (BHT branch history table)

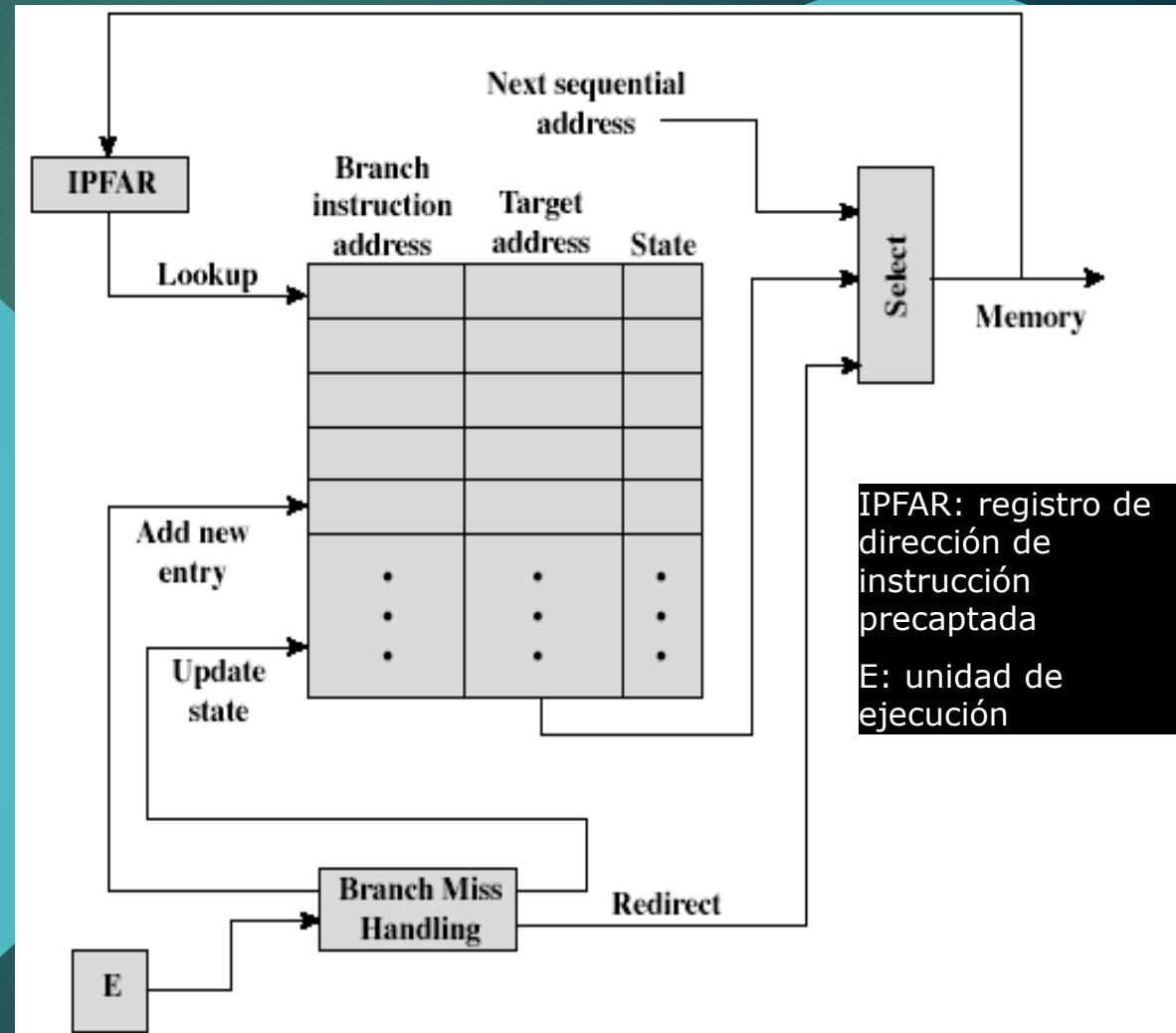
Se implementa usando una pequeña cache asociada a la etapa de búsqueda (F). La cache tiene 3 campos:

- Dirección de una instrucción que es del tipo bifurcación
 - Dirección del destino ó Instrucción destino
 - N bits de estado (historia de uso)
- Cada vez que se precapta una instrucción, se busca en la cache BHT si la instrucción es de ramificación.
- Se toma una decisión predictiva en función de los bits de la historia de uso.
- Si la predicción es saltar, se capta la dirección de salto.
- Si la predicción falla, se actualiza la tabla
- Si la instrucción de ramificación no estaba en la BHT, se la carga como nueva entrada.

Soluciones a los problemas de la Segmentación

BHT

Tabla caché para almacenar instrucciones de ramificación con las direcciones de destino para el caso de predecir el salto.



Soluciones a los problemas de la Segmentación

49

Predicción según el código de operación

- Se basa en la suposición de que algunas instrucciones de salto tienen mayor probabilidad de saltar o de no saltar.
- Por ejemplo, si la instrucción de salto involucra un lazo que se repite muchas veces, la condición que se usa para repetir el lazo ocurrirá muchas más veces que la que se usa para salir del lazo (que ocurrirá 1 sola vez).
- La tasa de acierto puede llegar a alcanzar un 75%

Soluciones a los problemas de la Segmentación

Varios cauces (uno por cada opción de salto- Multiple stream)

- En esta solución se usan cauces distintos (duplicando hardware) para ejecutar simultáneamente los cauces, el que corresponde a la opción de no saltar y el que corresponde a la instrucción de salto.
- Luego, cuando se determina el resultado de la ramificación, se debe utilizar el cauce correcto y descartar el incorrecto.
- Como se están captando simultáneamente 2 cauces distintos, aumentan las búsquedas, lo que puede provocar retardos en el acceso al bus y a los registros.
- Si hay nuevos saltos en los cauces en ejecución, se necesita disponer de más cauces para contemplar esta situación, lo que aumenta la complejidad del hardware.

Soluciones a los problemas de la Segmentación

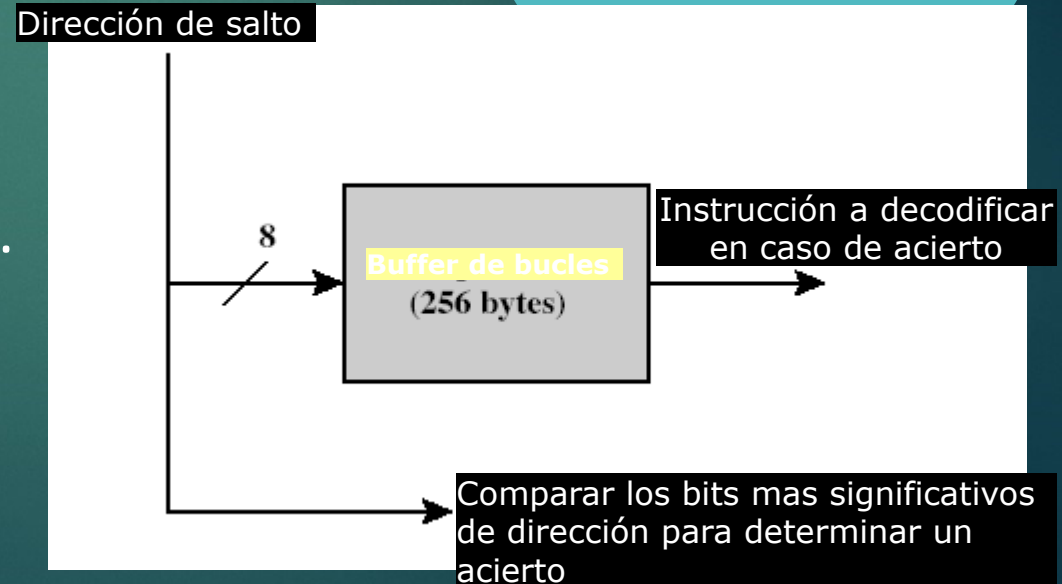
Precaptación del destino del salto (Prefetch branch target)

- Se precapta la instrucción destino del salto, además de las instrucciones siguientes a la bifurcación.
- La instrucción se guarda hasta que se ejecute la instrucción de bifurcación.
- El IBM 360/91 usa este método.

Soluciones a los problemas de la Segmentación

Buffer de bucles

- Se usa una memoria muy rápida, gestionada por la etapa de captación de instrucciones, que contiene las últimas instrucciones recientemente buscadas.
- Si hay una instrucción de salto, el hardware comprueba si está en el Buffer de bucles. Si es así, la próxima instrucción se busca desde el buffer.
- Muy eficaz para pequeños bucles y saltos, si el buffer es capaz de contener todo el bucle. Funciona parecido a la cache, solo que contiene instrucciones consecutivas, únicamente.



Soluciones a los problemas de la Segmentación

53

Técnicas de resolución de instrucciones de salto por software

- Las técnicas de resolución por software de conflictos debido a las instrucciones de salto se basan en tratar de realizar trabajo útil mientras el salto se resuelve.
- Cuando hay una instrucción de salto, se necesita 1 ciclo (o más) de tiempo para determinar si se va a ejecutar el salto o no. Ese tiempo se llama hueco o ranura de retardo de salto (Branch delay slot).
- Hay máquinas que captan y ejecutan siempre la instrucción siguiente a una instrucción de ramificación (en lugar de descartarla).
- El compilador puede tratar de insertar, en dichos huecos, instrucciones útiles que en lo posible no dependan del salto. De esta manera se elimina el conflicto y se hace trabajo útil.
- Si no es posible insertar instrucciones, se necesita agregar instrucciones del tipo NO-OPERACION (NOP) para evitar el conflicto.

Soluciones a los problemas de la Segmentación

Técnicas de resolución de instrucciones de salto por software

- La forma de llenar los huecos que quedan luego de las instrucciones de salto es mediante el reordenamiento de las instrucciones. Ese trabajo lo hace el compilador.
- Las instrucciones a reordenar pueden provenir de:
 - **Caso 1:** Instrucciones antes del salto
 - **Caso 2:** Instrucciones del destino del salto
 - **Caso 3:** Instrucciones a continuación del salto
- La mejor solución sería reordenar instrucciones anteriores a la de salto, porque siempre deben ejecutarse y van a ser útiles.
- El reordenamiento con instrucciones del destino del salto o a continuación del salto deben ser tal que no modifiquen el estado del proceso aun cuando falle la predicción.

Soluciones a los problemas de la Segmentación

Proceso de reordenamiento de instrucciones en máquinas con delay-slot (casos 1, 2 y 3):

Caso 1	Caso 2	Caso 3
Antes:	Antes:	Antes:
ADD R1,R2,R3 si R2=0 entonces retardo ... MULT r6,r7,r8	SUB R4,R5,R6 MULT r6,r7,r8 ... ADD R1,R2,R3 si R1=0 entonces retardo	ADD R1,R2,R3 si R1=0 entonces retardo SUB R4,R5,R6 ... MULT R14,R15,R16 Ins+1
Despues:	Despues:	Despues:
si R2=0 entonces ADD R1,R2,R3 ... MULT r6,r7,r8	SUB R4,R5,R6 MULT r6,r7,r8 ... ADD R1,R2,R3 si R1=0 entonces SUB R4,R5,R6	ADD R1,R2,R3 si R1=0 entonces SUB R4,R5,R6 ... Ins-1 MULT R14,R15,R16 Ins+1

Soluciones a los problemas de la Segmentación

56

Técnicas de resolución de instrucciones de salto por software

- En el ejemplo anterior:
 - Caso 1: el hueco se rellena con una instrucción anterior al salto. Sirve siempre, es la mejor opción (no siempre es posible).
 - Caso 2: Como la instrucción ADD R1,R2,R3 se requiere ejecutar antes de la instrucción de salto, no se puede reubicar después del salto. El compilador busca una instrucción a reubicar, en el lugar a donde puede llegar a saltar, por ejemplo SUB R4,R5,R6. La copia en el hueco, y corrige el lugar a donde salta (MULT r6,r7,r8). Si la instrucción de salto efectivamente salta, entonces la instrucción insertada en el hueco (SUB R4,R5,R6) sirve y se adelantó trabajo. Si la instrucción no salta, la instrucción insertada (SUB R4,R5,R6) se ejecuta pero no sirve y el resultado no debe alterar la lógica. En otras palabras, el valor que se cargue en R4 (en la instrucción SUB R4,R5,R6) no sirve y no debe alterar la lógica del programa.

Soluciones a los problemas de la Segmentación

57

Técnicas de resolución de instrucciones de salto por software

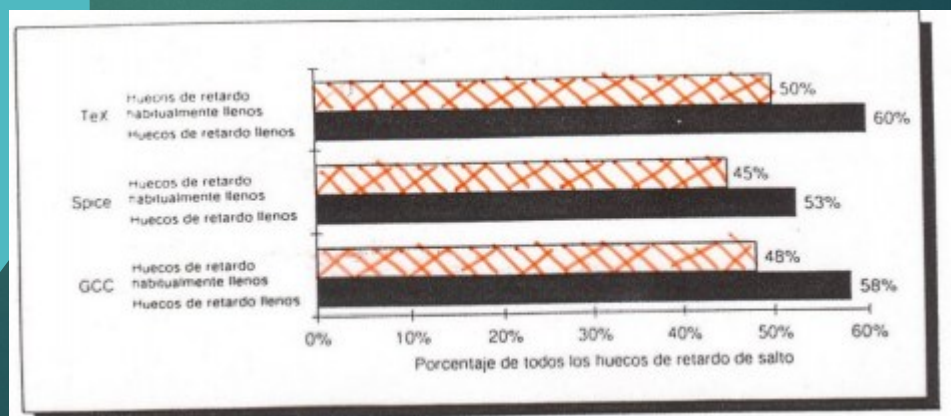
- Caso 3: Como la instrucción ADD R1,R2,R3 se requiere ejecutar antes de la instrucción de salto, no se puede reubicar después del salto. El compilador busca una instrucción a reubicar, a continuación de la de salto, por ejemplo SUB R4,R5,R6. Si la instrucción no salta, entonces la instrucción insertada en el hueco (SUB R4,R5,R6) sirve y se adelantó trabajo. Si la instrucción salta, la instrucción insertada (SUB R4,R5,R6) se ejecuta pero no sirve y el resultado no debe alterar la lógica. En otras palabras, el valor que se cargue en R4 (en la instrucción SUB R4,R5,R6) no sirve y no debe alterar la lógica del programa.

Soluciones a los problemas de la Segmentación

58

Técnicas de resolución de instrucciones de salto por software

- En la imagen inferior se muestran estudios realizados en 3 máquinas con delay-slot. Se observa que:
 - Aproximadamente el 60% de los huecos se rellenan con instrucciones diferentes a NOP (barra oscura en la imagen inferior)
 - Del 60% de huecos rellenos con instrucciones (los 3 casos vistos antes), entre el 80 y 85% corresponde a instrucciones útiles (barras rayadas).
 - En definitiva, el 50% de los huecos (retardos) se aprovechan con instrucciones útiles.



Segmentación en el i80486

Como ejemplo de procesador convencional (tipo CISC) se puede analizar el procesador de Intel 80486, que tiene un cauce compuesto por 5 etapas, 2 de ellas de decodificación:

- Fetch (F)
 - Decode 1 (D1)
 - Decode 2 (D2)
 - Execute (EX)
 - Write back (WB)
- En las figuras siguientes se muestran 3 casos de secuencias de instrucciones y el comportamiento del cauce para cada secuencia.

Segmentación en el i80486

Secuencia de instrucciones sin penalización

3 instrucciones de movimiento entre memoria y registro sin retrasos.

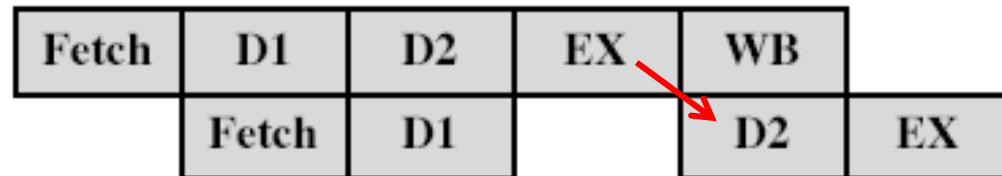
Fetch	D1	D2	EX	WB			MOV Reg1, Mem1
	Fetch	D1	D2	EX	WB		MOV Reg1, Reg2
		Fetch	D1	D2	EX	WB	MOV Mem2, Reg1

No hay retardo en el cauce por carga de un dato desde memoria.

Segmentación en el i80486

Secuencia de instrucciones con penalización de 1 ciclo

Retardo de 1 ciclo en el cauce por carga que utiliza un puntero (con adelantamiento desde la etapa EX a la D2).



MOV Reg1, Mem1

MOV Reg2, (Reg1)

(b) Pointer Load Delay

Segmentación en el i80486

Secuencia de instrucciones con penalización de 2 ciclos

Retardo de 2 ciclos en el cauce por temporizado en una instrucción de bifurcación



(c) Branch Instruction Timing

Procesador segmentado nanoMIPS

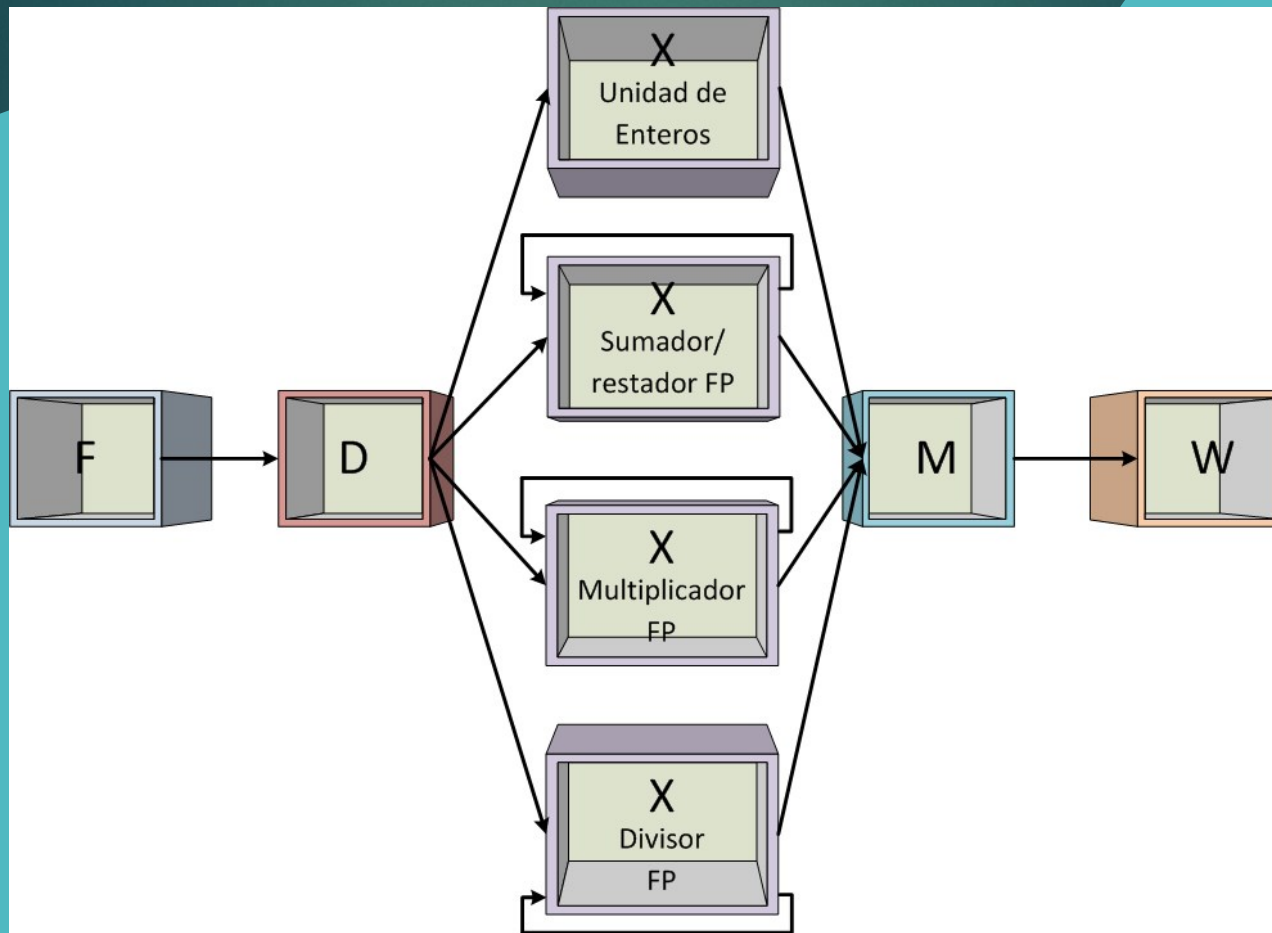
multifuncional

63

- En general es imprescindible que los procesadores dispongan de operaciones en punto flotante. El modelo visto del nanoMIPS no las tiene.
- Si se quiere agregar operaciones en punto flotante en el nanoMIPS, se requiere realizar las siguientes modificaciones:
 - Agregar instrucciones en PF (Ej: ADD.D, LD.D)
 - Agregar registros de PF (Ej: F1, F2, etc.)
 - Agregar hardware para las operaciones aritméticas de: Suma, Resta, Multiplicación y División
- Es posible compatibilizar la segmentación con las operaciones en punto flotante. La primera solución es la que se muestra en la figura siguiente.

Procesador segmentado nanoMIPS multifuncional

Esquema del nanoMIPS segmentado con Unidades de punto flotante



Procesador segmentado nanoMIPS multifuncional

- El problema que tiene el esquema anterior es que los tiempos de ejecución de las unidades de punto flotante son muy distintos de los de la unidad de enteros.
- Hasta ahora habíamos considerado que todas las etapas del nanoMIPS tenían la misma duración, pero ahora las unidades funcionales de PF pueden requerir varios ciclos para completarse.
 - Por ejemplo, la división en punto flotante lleva muchos mas ciclos que una suma de números enteros
- **Conclusión: cuando se incorpora punto flotante, no todas las etapas del cauce van a durar lo mismo.**

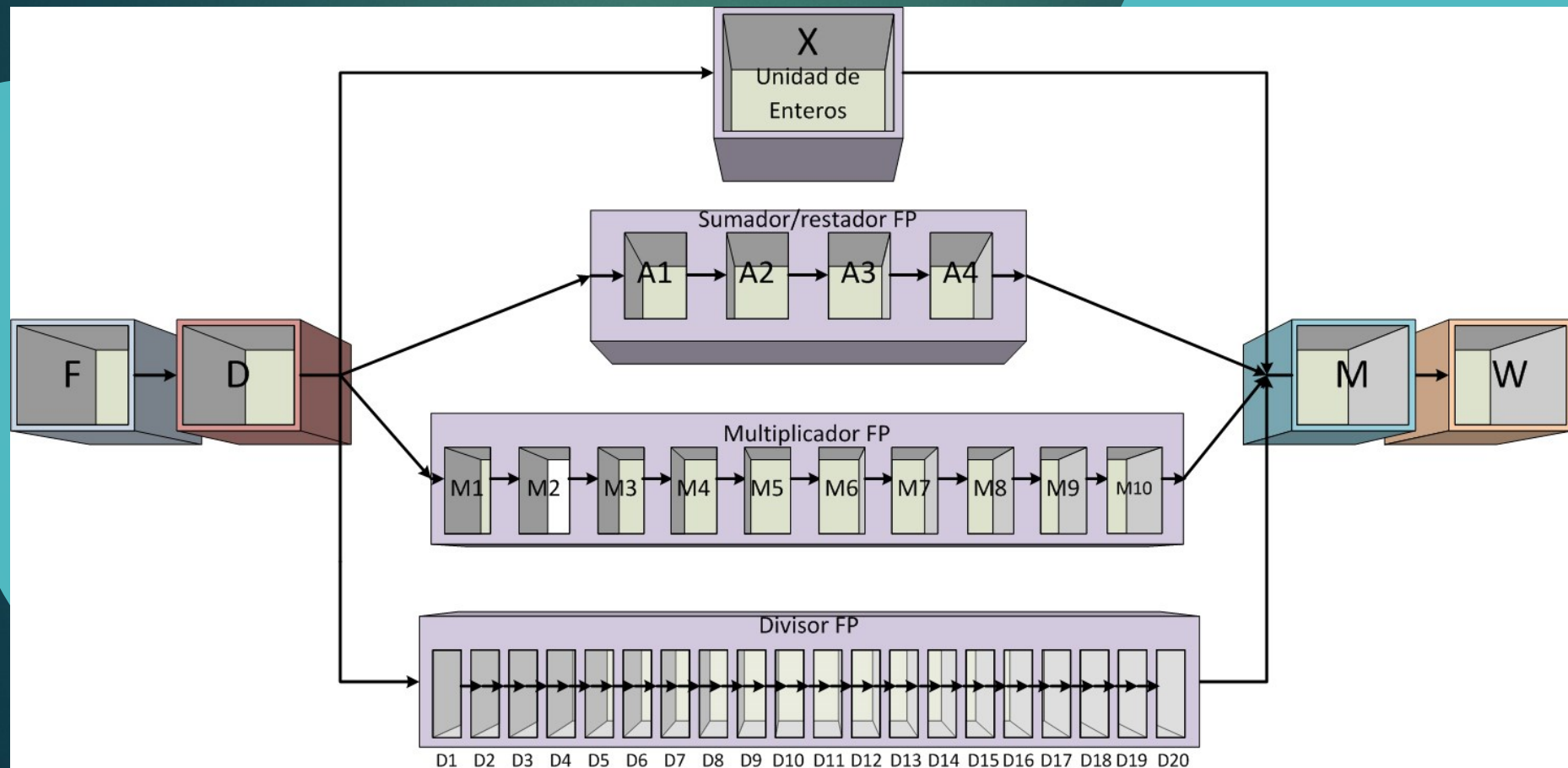
Procesador segmentado

nanoMIPS multifuncional

- Si las unidades funcionales en punto flotante tardan varios ciclos, entonces una secuencia de 2 instrucciones consecutivas en punto flotante requeriría retardar la segunda instrucción hasta que la primera se complete.
- En el siguiente ejemplo se muestra cual es la situación:
ADD.D F1, F2, F3
ADD.D F4, F5, F6
- No se puede empezar la segunda instrucción ADD.D F4,F5,F6 hasta que no se termine la primera.
- Hay una solución a este problema: segmentar las unidades funcionales de punto flotante.
- Esta solución se muestra en la figura siguiente.

Procesador segmentado nanoMIPS multifuncional

Esquema del nanoMIPS segmentado con Unidades de punto flotante segmentadas



Procesador segmentado nanoMIPS multifuncional

- Segmentando las unidades de punto flotante se pueden iniciar nuevas instrucciones en PF sin haber completado a actual.
- En la figura anterior se ha segmentado:
 - La unidad de suma en PF en 4 etapas
 - La Unidad de multiplicación en PF en 10 etapas
 - La unidad de división en PF en 20 etapas.

Referencias

- W. Stallings, 5º Ed - Capítulo 11.
- Hwang & Briggs – Computer Architecture & Parallel Processing – Chapter 10.

Arquitectura de Computadoras

CURSO 2022

Turno:

Clase 6

Resumen clase 6

Procesadores RISC y CISC

- Hitos relevantes en la evolución de los sistemas de cómputo
- Procesadores CISC
- Estudios sobre el comportamiento de los programas
- Procesadores RISC
- Ventana de registros: concepto, implementación
- Optimización por software
- Ventajas y desventajas de los procesadores CISC
- Comparación entre procesadores RISC y CISC
- Ejemplos de procesadores RISC

Evolución histórica

Algunos hitos significativos en la evolución de los sistemas de cómputo

- **Familia de computadoras:** el concepto de familia de computadoras fué introducido por IBM a partir de su modelo System/360 en 1964, basado en la disponibilidad de un conjunto de máquinas con una arquitectura común y prestaciones variables de acuerdo a las necesidades.
- **Arquitectura e implementación:** son 2 conceptos separados que introdujo DEC en su máquina PDP-8, referidos a las características visibles al programador y las prestaciones del sistema.

Evolución histórica

Algunos hitos significativos en la evolución de los sistemas de cómputo

- **Unidad de control microprogramada:** idea propuesta por Wilkes en 1951 para el desarrollo sistemático de las unidades de control, e introducida por IBM en la línea S/360 en 1964.
- **Memoria cache:** pequeña memoria de alta velocidad para balancear la capacidad de la CPU y la velocidad de transferencia de la Memoria principal. Introducida En 1968 en la IBM S/360 Modelo 85.
- **Memoria de estado sólido:** desarrollo, en 1970, de memorias semiconductoras en reemplazo de las memorias magnéticas, lentas y complicadas de manejar, usadas hasta entonces.

Evolución histórica

Algunos hitos significativos en la evolución de los sistemas de cómputo

- **Microprocesador** : en 1971 aparece el primer microprocesador fabricado por Intel, el i4004.
- **Segmentación de cauce**: Introduce el paralelismo en la naturaleza secuencial de los programas.
- **Procesadores múltiples**: desarrollo de sistemas con múltiples procesadores para aumentar la potencia de cómputo.
- **Procesadores RISC**: diseño conceptualmente opuesto a los procesadores CISC.

Procesadores CISC

Evolución de los procesadores

- El desarrollo de la tecnología de fabricación de circuitos integrados de muy alta escala de integración (VHSI) permitió incorporar cada vez mas funcionalidades dentro de un chip.
- La incorporación de nuevas funcionalidades se orientó principalmente en mejorar la eficiencia en la ejecución de programas.
- Un aspecto de esta mejora consistió en brindar un mayor soporte a los lenguajes de alto nivel (HLL), sobre todo los más complejos.

Procesadores CISC

Evolución de los procesadores

- Dar mayor soporte a los HLL tiende a facilitar el proceso de compilación (y también el trabajo de elaboración de compiladores), sobre todo considerando que el costo del “software es típicamente mucho más caro que el hardware”.
- La “distancia” entre un HLL y el lenguaje de máquina se denomina GAP semántico, y básicamente es la relación entre la cantidad de sentencias de un programa en HLL y la cantidad de instrucciones de máquina que se requieren para resolverlo.
- El mayor soporte tiende a reducir el “GAP semántico”, agregando instrucciones de máquina más complejas que resuelvan las sentencias del HLL más directamente.

Procesadores CISC

Procesadores CISC (Complex Instruction set computer)

- Los conceptos anteriores sentaron las bases de diseño de los procesadores “convencionales”:
 - Repertorios de instrucciones grandes.
 - Formato de instrucción complejo y de longitud variable.
 - Muchos modos de direccionamiento.
 - Capacidad de implementar sentencias de HLL con muy pocas instrucciones de máquina. Esta idea consiste básicamente en resolver una sentencia de HLL “por hardware”.
- De los conceptos anteriores surge el modelo de procesador conocido como CISC, es decir, procesador con repertorio de instrucciones complejo

Procesadores CISC

Procesadores CISC (Complex Instruction set computer)

- Con el uso masivo de la tecnología VHSI y el desarrollo de procesadores tipo CISC, se empezaron a realizar diferentes estudios sobre el comportamiento de los procesadores tipo CISC.
- Los estudios se orientaron a:
 - 1. Uso del repertorio de instrucciones
 - 2. Uso de los operandos
 - 3. Secuencia de ejecución de instrucciones
- Los estudios se realizaron de 2 maneras distintas:
 - Estáticos
 - Dinámicos

Procesadores CISC

Procesadores CISC (Complex Instruction set computer)

- Los estudios sobre operaciones estaban orientados a determinar el uso de las instrucciones y, por lo tanto, su interacción con la memoria.
- Los estudios sobre operandos básicamente intentaban determinar cuales eran los tipos de datos usados y su frecuencia de uso.
- Los estudios sobre el secuenciamiento de la ejecución de instrucciones tenían como objetivo determinar el impacto de disponer de un repertorio de instrucciones largo y complejo, en la Unidad de Control (que debe resolver las instrucciones), y en el cauce de datos que tiene relación con la eficiencia en la ejecución de una secuencia de instrucciones.

Procesadores CISC

Procesadores CISC (Complex Instruction set computer)

- Los estudios estáticos computaban cantidades basados en el código objeto ejecutable. Es una forma sencilla de obtener métricas, pero tienen el problema de que no tienen en cuenta el flujo del programa ejecutado, y pueden resultar bastante equivocados. Por ejemplo, la cantidad de instrucciones que componen un lazo no tiene nada que ver con las veces que se ejecuta.
- Los estudios dinámicos computan cantidades en base a la ejecución de diferentes programas (típicamente programas de prueba o “benchmarks”). Son más realistas porque los resultados tienen en cuenta la situación real por la que pasan los procesadores.

Procesadores CISC

Estudio sobre el uso de instrucciones

- Este estudio permitió medir el uso de las instrucciones, y su interacción con la memoria. Se efectuó sobre VAX, PDP-11 y MC68000.
- Las operaciones (instrucciones) se dividieron en:
 - Asignación: movimiento de datos, aritméticas y lógicas
 - Sentencias condicionales: IF, LOOP
 - Sentencias incondicionales: GOTO
 - Llamadas y retornos de subrutinas: CALL
 - Otras
- Los resultados obtenidos se pueden ver en la siguiente tabla.

Procesadores CISC

Estudio sobre el uso de instrucciones - Resultados

	Aparición dinámica		Instruc. máquina (Ponderadas)		Referencias a memoria (Ponderadas)	
	Pascal	C	Pascal	C	Pascal	C
Assign	45	38	13	13	14	15
Loop	5	3	42	32	33	26
Call	15	12	31	33	44	45
If	29	43	11	21	7	13
GoTo	-	3	-	-	-	-
Otras	6	1	3	1	2	1

Procesadores CISC

14

Estudio sobre el uso de instrucciones - Resultados

- En la tabla anterior se muestran los resultados para programas escritos en Pascal y C.
- **Los resultados son medidos de 3 maneras (columnas):**
 - **Aparición dinámica:** cuenta la cantidad de sentencias del HLL ejecutadas.
 - **Instrucciones de máquina:** computa las instrucciones de máquina para las sentencias del HLL ponderadas al tiempo de ejecución de las mismas.
 - **Referencias a memoria:** computa la cantidad de accesos a memoria.

Procesadores CISC

Estudio sobre el uso de instrucciones - Resultados

- **Se puede apreciar del estudio mostrado que:**
 - Las sentencias de asignación son las más numerosas, seguidas por las de llamadas a procedimientos y las sentencias condicionales (primer columna).
 - Cuando se hace la ponderación basado en la cantidad de instrucciones de máquina que resuelven las sentencias de alto nivel, las sentencias condicionales y llamadas a procedimientos son las que requieren más instrucciones de máquina (segunda columna).
 - Cuando se hace la ponderación basado en la cantidad de accesos a memoria, siguen predominando las sentencias condicionales y llamadas a procedimiento (tercer columna).

Procesadores CISC

16

Estudio sobre el uso de instrucciones - Resultados

- Los accesos a memoria tienen mucho impacto en el tiempo de ejecución del programa.
- Por lo tanto, los llamados a procedimiento consumen mucho tiempo y tiene que hacerse lo más eficiente posible para optimizar los accesos a memoria.

Procesadores CISC

17

Estudios sobre llamadas a procedimientos - Resultados

- En varios estudios se analizó el comportamiento en llamadas y retornos a procedimientos.
- En un alto porcentaje (>98%) se pasan menos de 6 datos.
- En un alto porcentaje (>92%) la cantidad de variables locales es menor a 6.
- En general el nivel de anidamiento es menor a 7.
- Conclusiones:
 - La mayoría de los programas tienen una secuencia corta de llamadas seguida por la secuencia de retornos.
 - La mayoría de las variables son locales.
 - Las referencias a operandos están muy localizadas.

Procesadores CISC

18

Estudios sobre uso de operandos - Resultados

La siguiente tabla muestra algunos resultados sobre el uso de operandos

	Pascal	C	Promedio
Constantes enteras	16	23	20
Variables escalares	58	53	55
Matrices/estructuras	26	24	25

Además, en otros estudios se observó:

- Uso de variables locales (80%) del procedimiento.
- Uso de punteros del tipo variable local y escalar para manejo de estructuras de datos.
- Uso intensivo de registros (no reflejado en este estudio).

Procesadores CISC

19

Conclusiones finales

Las conclusiones finales a las que se arribaron luego de muchos estudios fueron:

- 1) Optimización: se necesita optimizar el tiempo de ejecución de las instrucciones más usadas y las que consumen más tiempo.
- 2) Simplificación: de las instrucciones, para resolverlas más rápidamente.
- 3) Ajuste: del cauce de datos para resolver las instrucciones más usadas lo más rápido posible.
- 4) Registros: usar los registros en forma intensiva, para manejo de operandos. Disponer de un banco de registros significativo para reducir los accesos a memoria.

Procesadores RISC

20

- Los estudios realizados sobre el comportamiento de los programas, condujeron a un nuevo enfoque sobre el diseño de procesadores, a los que se identificó genéricamente como procesadores RISC (Reduced Instruction Set Computer).
- Los principales referentes de este nuevo enfoque fueron Patterson y Hennesey (“Arquitectura de computadoras – Un enfoque estructurado”).
- Conceptualmente, los primeros procesadores RISC surgieron a mediados de los 80 en las Universidades de Berkeley (David Patterson, proyecto RISC 1) y Stanford (John Hennesey, proyecto MIPS-XMP).

Procesadores RISC

21

- Los primeros RISC tenían, entre otras, las siguientes características relevantes:
 - Repertorio de instrucciones reducido y básico.
 - Formato de instrucción fijo y sencillo (todas las instrucciones tienen la misma cantidad de bits o bytes).
 - Unidad de control “cableada” (no microprogramada).
 - Diseño del cauce optimizado para ejecutar 1 instrucción por ciclo (uso preciso de la segmentación)
 - Máquina tipo LOAD-STORE: acceso a memoria solo con instrucciones de movimiento de datos. Instrucciones aritméticas y lógicas solo entre registros.
 - Uso intensivo de registros (de uso general) con optimización por software.

Procesadores RISC

22

Comparación entre CISC y RISC

En la tabla siguiente se muestra una comparativa entre 3 procesadores CISC y 2 procesadores RISC.

	Complex Instruction Set (CISC) Computer			Reduced Instruction Set (RISC) Computer	
Characteristic	IBM 370/168	VAX 11/780	Intel 80486	SPARC	MIPS R4000
Year developed	1973	1978	1989	1987	1991
Number of instructions	208	303	235	69	94
Instruction size (bytes)	2-6	2-57	1-11	4	4
Addressing modes	4	22	11	1	1
Number of general-purpose registers	16	16	8	40 - 520	32
Control memory size (Kbits)	420	480	246	—	—
Cache size (KBytes)	64	64	8	32	128

Procesadores RISC

- En la tabla comparativa anterior se puede apreciar:
 - Repertorio de instrucciones: los CISC tienen muchas más instrucciones que los RISC.
 - Tamaño de instrucción: en los CISC es muy variable, mientras que en los RISC es fija.
 - Modos de direccionamiento: los CISC tienen una buena variedad de modos de direccionamiento, mientras que los RISC son mínimos.
 - Banco de registros: en los RISC son amplios, y más limitados en los CISC.
 - Memoria de control: típicamente microprogramada en los CISC, lógica “cableada” en los RISC.

Procesadores RISC - Ventana de registros

Registros en procesadores RISC

- Un aspecto esencial de los procesadores RISC es disponer de un banco de registros amplio, para reducir los accesos a memoria, y simplificar las instrucciones.
- La optimización del banco de registros se puede hacer de 2 maneras:
 - **Por Hardware:** agregando más registros
 - **Por Software:** optimizando la asignación de registros a las variables que se usen más en un período de tiempo dado.

Procesadores RISC - Ventana de registros

Solución por hardware de los registros en procesadores RISC

Tener un banco de registros amplio tiene ventajas y desventajas:

➤ Ventajas:

- Reducción de accesos a memoria.
- Disponibilidad para más variables escalares locales.

➤ Desventajas:

- Más bits en la instrucción para identificar el registro.
- En llamadas y retornos de subrutina hay que considerar que se requiere tiempo para pasar parámetros, asignar espacio para las variables locales, y devolver los resultados.

Procesadores RISC - Ventana de registros

26

Llamadas y retornos de subrutinas - Variables

- Por los estudios realizados, los llamados y retornos de subrutina son los que más tiempo consumen.
- Por lo tanto, es necesario hacer muy eficiente el manejo de este tipo de instrucciones.
- Los estudios realizados permiten inferir:
 - En general, las subrutinas usan pocas variables locales.
 - Cada llamada recibe y pasa pocos argumentos.
 - Las subrutinas usan pocas variables globales.

Procesadores RISC - Ventana de registros

27

Llamadas y retornos de subrutinas – Variables

En general, cada subrutina necesita, para operar, 4 tipos de datos:

- Datos de entrada: argumentos de entrada
- Datos propios: datos (variables) locales
- Datos de salida: para pasar argumentos
- Datos globales (vistos por todas las subrutinas)

Los estudios realizados permiten inferir:

- En general, las subrutinas usan pocas variables locales.
- Cada llamada recibe y pasa pocos argumentos.
- Las subrutinas usan pocas variables globales.

Procesadores RISC - Ventana de registros

Llamadas y retornos de subrutinas – Variables y registros

Si la subrutina usara registros para manipular todos los tipos de datos mencionados anteriormente, necesitaría disponer de la siguiente estructura de registros (asumiendo 8 variables por tipo de dato).

R0-R7

REGISTROS PARA DATOS
GLOBALES

R8-R15

REGISTROS PARA
PARAMETROS RECIBIDOS

R16-R23

REGISTROS PARA
VARIABLES LOCALES

R24-R31

REGISTROS PARA
ARGUMENTOS A PASAR

Procesadores RISC - Ventana de registros

Llamadas y retornos de subrutinas – Variables y registros

Cada tipo de dato requiere un banco de registros:

- Datos de entrada: registros de parámetros
- Datos propios: registros locales
- Datos de salida: registros temporales
- Datos globales: registros globales

Procesadores RISC - Ventana de registros

30

Llamadas y retornos de subrutinas – Variables y registros

- Si cada subrutina administra “su” banco de registros (que le “pertenece”), excepto los globales que son comunes a todos, la cantidad real de registros de cada subrutina se limita a un número no excesivamente grande (24 en el ejemplo anterior), y por lo tanto, requiere pocos bits para identificarlo (5 bits).
- Los registros globales son comunes a todas las subrutinas, y accesibles por todas ellas.

Procesadores RISC -

Ventana de registros

Por otra parte, los problemas que aparecen con el uso de subrutinas son:

- Cada vez que una subrutina invoca otra subrutina (anidamiento de subrutinas) debe pasarle los argumentos, lo que produce un gasto de tiempo considerable.
- Cuanto más registros tenga la subrutina, más tiempo se consume en pasar la información.
- Cuantos mas registros necesite localmente, mas tiempo se requiere en reservarlos.

Para solucionar los problemas en el uso de Procedimientos y pasaje de parámetros se puede usar una estrategia conocida como Ventana de registros.

Procesadores RISC - Ventana de registros

La solución de usar una Ventana de registros consiste en:

1°) Asignar a cada subrutina un banco de registros propio:

R0-R7



REGISTROS PARA DATOS
GLOBALES

R8-R15



REGISTROS PARA
PARAMETROS RECIBIDOS

R16-R23



REGISTROS PARA
VARIABLES LOCALES

R24-R31



REGISTROS PARA
ARGUMENTOS A PASAR

Por ejemplo, los registros R8 a R31 son los registros accesibles por la subrutina.

Los registros R0 a R7 son globales y accesibles por todas las subrutinas.

Procesadores RISC -

Ventana de registros

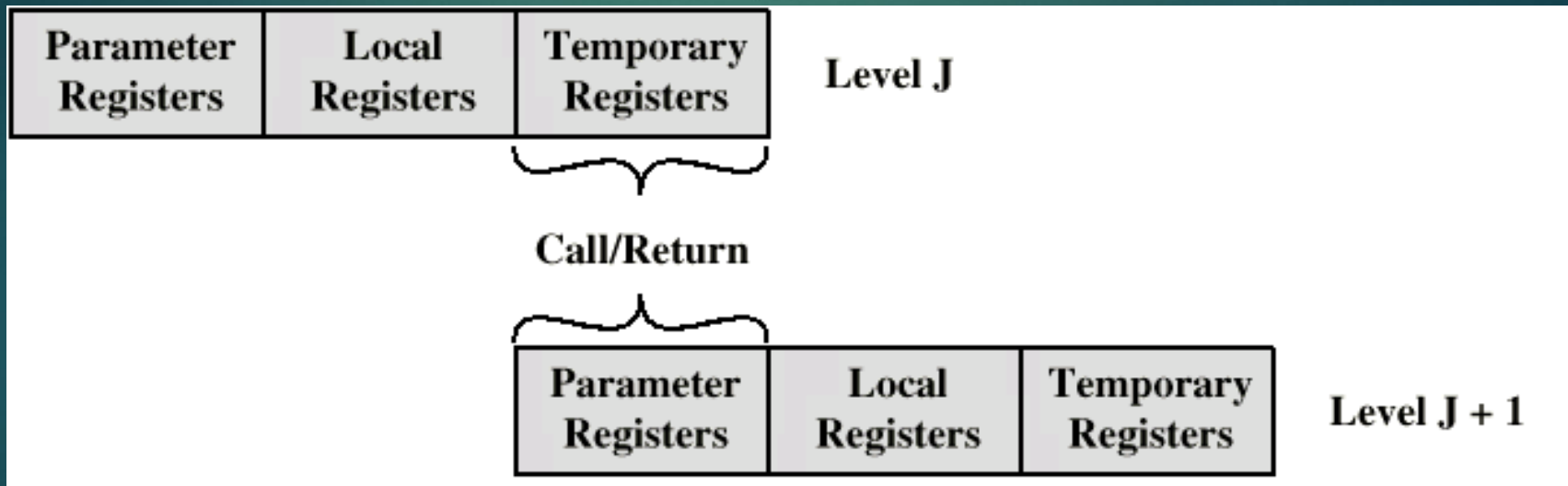
33

2°) Superponer los registros donde recibe los parámetros una subrutina con los registros donde pasa los argumentos la subrutina que la llama. Es decir:

- La subrutina de nivel $j+1$ recibe los parámetros de la subrutina de nivel j en los registros R8 a R15. La subrutina de nivel j le pasa los parámetros a la subrutina de nivel $j+1$ en los registros R24 a R31,
- Entonces si se solapan los registros R24 a R31 de la subrutina j con los R8 a R15 de la $j+1$, se pueden pasar los parámetros directamente entre la subrutina j y la $j+1$ a través del mismo grupo de registros (físico).
- Esta situación se muestra en la figura siguiente.

Procesadores RISC - Ventana de registros

Esquema de solapamiento de los registros para pasar
parámetros desde una subrutina j que llama, a otra $j+1$ que es
invocada:



Los registros temporarios del nivel j y los registros de parámetros del nivel $j+1$ son físicamente los mismos.

En cambio, los registros locales son físicamente distintos para cada subrutina y solo accesibles por ella.

Procesadores RISC -

Ventana de registros

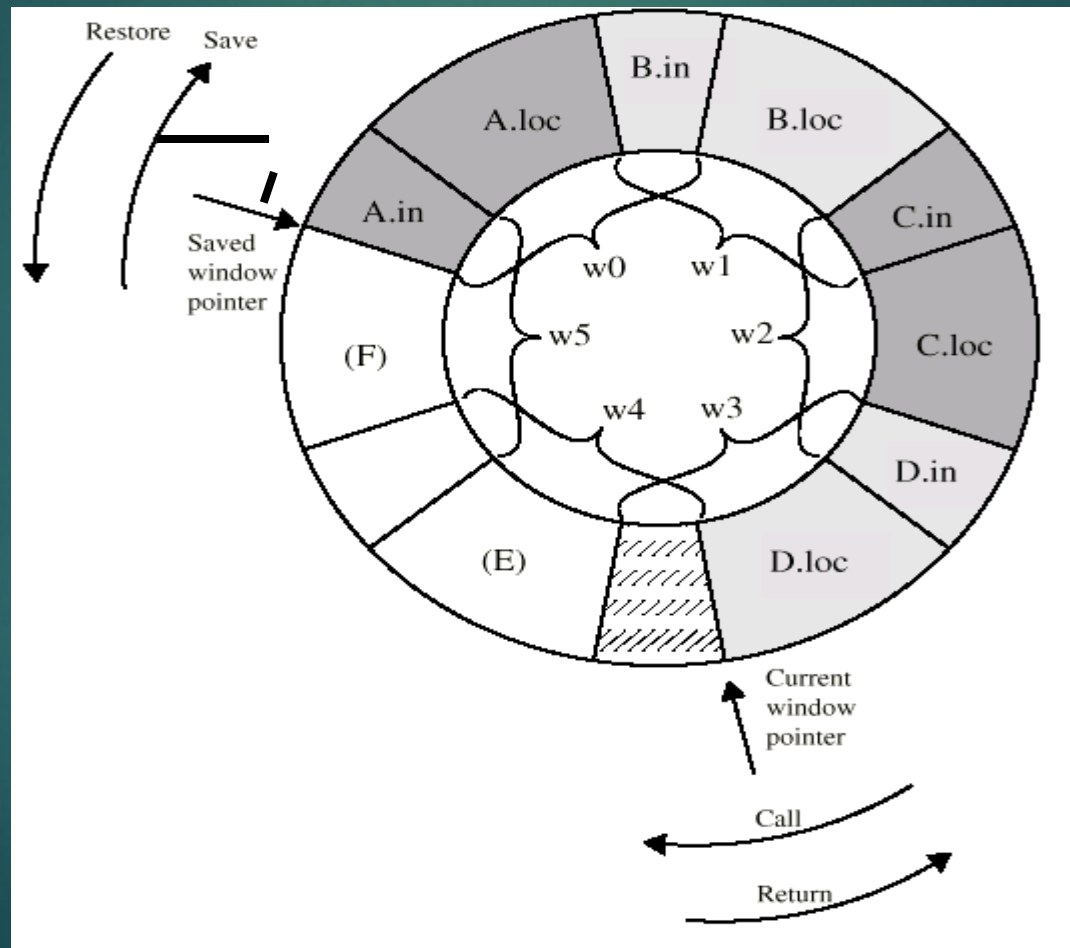
35

- De esta manera, cada subrutina accede a una ventana de registros, de los cuales los primeros y los últimos están solapados (superpuestos), y son usados para pasaje de parámetros.
- El banco de registros se implementa como un buffer de tipo circular, con una capacidad determinada de “ventanas”, basada en la “profundidad” de anidamientos admitidas por el procesador (basada en los estudios antes analizados, podría ser, por ejemplo, del orden de 7)
- El esquema del banco de registros de ventanas solapadas se muestra en la figura siguiente.

Procesadores RISC - Ventana de registros

36

Implementación de la Ventana de registros con un buffer circular



Procesadores RISC -

Ventana de registros

En la imagen anterior se muestra el buffer circular usado en un esquema de ventana de registros.

- En el nivel 0 (w0), los registros accesibles son A_{in} , A_{loc} y B_{in} .
- En el nivel 1 (w1), los registros accesibles son B_{in} , B_{loc} y C_{in} .
- Los registros B_{in} son accesibles por la rutina de nivel 0 (por donde pasa los parámetros) y la rutina de nivel 1 (por donde recibe los parámetros).
- Cada invocación de una subrutina mueve un puntero que apunta al siguiente banco de registros. El movimiento del puntero es tal que quedan superpuestos (solapados) los registros usados para pasaje de parámetros.
- La cantidad de registros accesibles por cada subrutina es la misma, pero cambian los registros físicos a los que accede.

Procesadores RISC - Ventana de registros

38

Variables globales

Para el manejo de variables globales (es decir compartidas por todas las subrutinas) se pueden usar 2 soluciones:

1. El compilador asigne posiciones de memoria a las variables: es algo ineficiente para variables globales a las que se accede frecuentemente porque requiere accesos a memoria que son inherentemente lentos.
2. Incorporar en el procesador un conjunto de registros para variables globales accesible en todos los niveles. Este banco de registros no está mostrado en la ventana de registros.

Procesadores RISC - Ventana de registros

Ventana de registros vs. Memoria cache

<u>Banco de registros amplio</u>	<u>Cache</u>
➤ Todos los datos son escalares y locales ➤ Acceso individual a variables ➤ Variables globales asignadas por el compilador ➤ Salvaguarda/restauración basadas en la profundidad de anidamiento ➤ Direccionamiento de registro	➤ Datos escalares locales reciente-mente usados ➤ Acceso a bloques de memoria ➤ Variables locales y globales usadas recientemente ➤ Salvaguarda/restauración basadas en el algoritmo de reemplazo ➤ Direccionamiento de memoria

Procesadores RISC - Técnicas por software

Técnicas de optimización por software

- También se puede mejorar las prestaciones en los procesadores RISC, optimizando el uso de los registros.
- Pero los lenguajes HLL no tienen referencias explícitas a los registros.
- Como la asignación de registros se hace en la compilación, el uso optimizado de registros es responsabilidad del compilador (no del programador).
- Las estrategias de optimización tratan de asignar los pocos registros a las variables más usadas o las que más tiempo permanecerán en el registro.

Procesadores RISC -

Técnicas por software

Técnicas de optimización por software

- Una estrategia consiste en suponer que se disponen de infinitos registros (llamados “virtuales”). A cada variable del programa se le asigna un registro virtual (que son ilimitados).
- El compilador mapea (asigna) el número ilimitado de registros simbólicos a un número fijo de registros reales.
- Los registros simbólicos que no se solapan pueden compartir el registro real. Si se agotan los registros reales, algunas de las variables se asignan a posiciones de memoria.
- En la optimización se usa una técnica denominada ‘coloreado de grafos’.

Procesadores RISC

42

Características relevantes de los procesadores RISC

- Formatos de instrucción sencillos (fijo).
- Modos de direccionamiento sencillos.
- Diseño de la Unidad de control del tipo “cableado” (sin microcódigo).
- Operaciones registro a registro.
- Instrucciones a memoria únicamente LOAD y STORE.
- Una instrucción por ciclo (segmentación eficiente).
- Típicamente máquinas tipo Harvard.
- Mayor tiempo/esfuerzo de compilación.

Procesadores RISC y CISC

43

Ejemplos de procesadores RISC y CISC

Processor	Number of instruction sizes	Max instruction size in bytes	Number of addressing modes	Indirect addressing	Load/store combined with arithmetic	Max number of memory operands	Unaligned addressing allowed	Max Number of MMU uses	Number of bits for integer register specifier	Number of bits for FP register specifier
AMD29000	1	4	1	no	no	1	no	1	8	3 ^a
MIPS R2000	1	4	1	no	no	1	no	1	5	4
SPARC	1	4	2	no	no	1	no	1	5	4
MC88000	1	4	3	no	no	1	no	1	5	4
HP PA	1	4	10 ^a	no	no	1	no	1	5	4
IBM RT/PC	2 ^a	4	1	no	no	1	no	1	4 ^a	3 ^a
IBM RS/6000	1	4	4	no	no	1	yes	1	5	5
Intel i860	1	4	4	no	no	1	no	1	5	4
IBM 3090	4	8	2 ^b	no ^b	yes	2	yes	4	4	2
Intel 80486	12	12	15	no ^b	yes	2	yes	4	3	3
NSC 32016	21	21	23	yes	yes	2	yes	4	3	3
MC68040	11	22	44	yes	yes	2	yes	8	4	3
VAX	56	56	22	yes	yes	6	yes	24	4	0
Clipper	4 ^a	8 ^a	9 ^a	no	no	1	0	2	4 ^a	3 ^a
Intel 80960	2 ^a	8 ^a	9 ^a	no	no	1	yes ^a	—	5	3 ^a

a RISC that does not conform to this characteristic.

b CISC that does not conform to this characteristic.

Procesadores RISC y CISC

44

Ventajas y desventajas de los procesadores CISC

- **Ventaja:** El compilador es mas sencillo por disponer de un repertorio amplio de instrucciones y modos de direccionamiento
- **Desventaja:** las instrucciones de máquina complejas son difíciles de aprovechar por el compilador, es decir las usa poco nada.
- **Desventaja:** la optimización es más difícil de realizar.
- **Ventaja:** los programas son más pequeños, tiene menos instrucciones, lo que probablemente implique que ocupa menos memoria. Sin embargo, la memoria hoy día es muy barata, por lo que esta ventaja es muy relativa.

Procesadores RISC y CISC

45

Ventajas y desventajas de los procesadores CISC

- El número de bits de memoria que ocupa no tiene porqué ser más pequeño al tener menos instrucciones
- **Desventaja:** al tener repertorio de instrucciones más amplio, los campos de código de operación son más largos y aumentan el tamaño de la instrucción.
- Las referencias a registros necesitan menos bits.

Procesadores RISC y CISC

46

Velocidad de ejecución

- El objetivo principal en el desarrollo de los procesadores es mejorar la velocidad de ejecución de los programas.
- Se supone que aumentar la complejidad del procesador debería mejorar su velocidad en ejecutar los programas.

Procesadores RISC y CISC

47

Velocidad de ejecución

- Pero:
 - Los procesadores CISC casi no usan las instrucciones más complejas.
 - El repertorio de instrucciones complejo exige una Unidad de control más compleja y lenta. En particular la memoria de control en la Unidad de control (microprogramada) es muy grande y “lenta”.
 - Una Unidad de control más lenta aumenta el tiempo de ejecución de las instrucciones simples, es decir, penaliza las instrucciones que podrían hacerse más rápido.
 - En definitiva, no está comprobado que la tendencia hacia CISC fuera la apropiada.

Procesadores RISC y CISC

48

Comparación entre RISC y CISC

- Se han realizado numerosas mediciones comparativas entre procesadores RISC y CISC.
- Pero las comparaciones tienen varios problemas:
 - No existe un par de máquinas RISC y CISC directamente comparables.
 - No hay un conjunto de programas de prueba definitivo.
 - EN los análisis, es muy difícil separar los efectos del hardware de los del compilador.
 - En muchos casos, las comparaciones se realizan usando prototipos o simulaciones en un “ambiente” controlado, y pocas veces con productos comerciales.
 - La mayoría de las máquinas son una mezcla de ambas.

Procesadores RISC y CISC

49

- Por otra parte es necesario definir qué parámetros se van a medir. En general, las evaluaciones pueden ser:
 - **Cuantitativas:** básicamente comparando los tamaño de los programas y su velocidad de ejecución
 - **Cualitativa:** revisión de soporte de lenguajes de alto nivel y uso óptimo de los recursos VLSI.

Conclusiones:

- No existe una marcada diferencia de performance entre uno y otro
- No está clara la barrera que separa uno u otro estilo.
- Muchos diseños incluyen características de ambos criterios, por ejemplo PowerPC y Pentium II.

Procesadores RISC

50

Procesador RISC SPARC (Sun Microsystems)

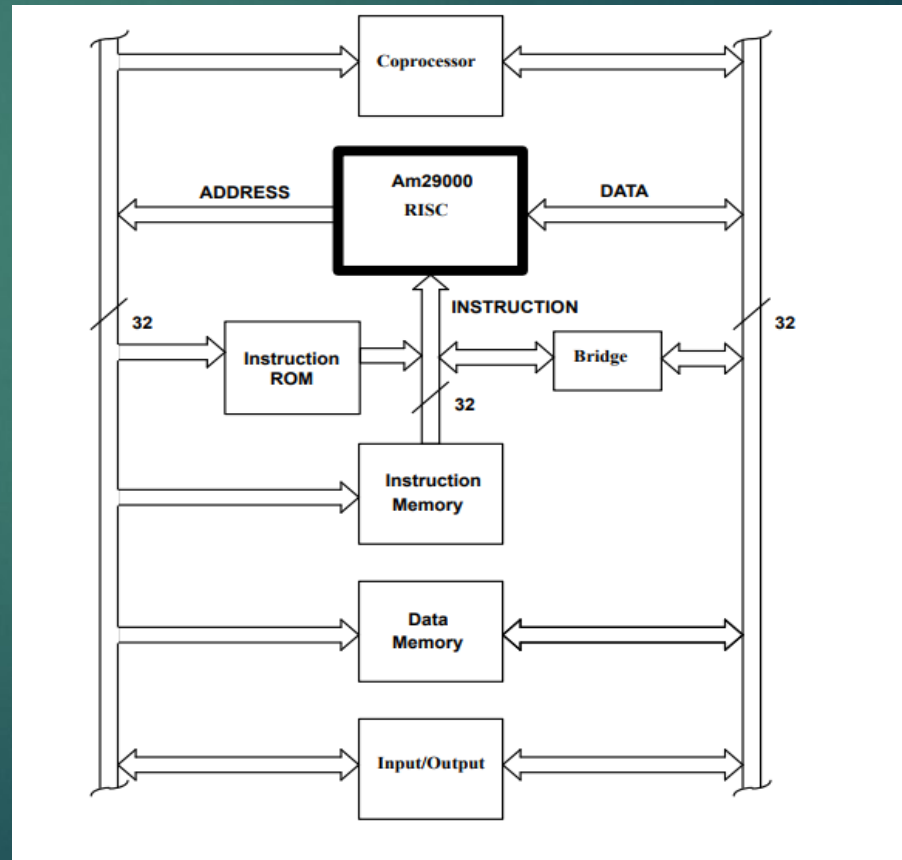
- La primer generación del SPARC se empezó a comercializar en 1987. Tenía una frecuencias de reloj de 16 a 50 MHz, y un diseño del tipo escalar (los procesadores escalares se verán más adelante).
- La segunda generación, conocida como SUPER SPARC, fue liberada en 1992. Tenía una frecuencias de reloj de 33 a 50 MHz, y una arquitectura super escalar.
- La tercera generación se conoció como ULTRA SPARC, y fué liberada a mediados de 1996. Disponía de una arquitectura super escalar de 4 etapas y de 64 bits, 5 unidades de coma flotante, y velocidades de reloj entre 250 y 300 MHz.

Procesadores RISC

51

Procesador AMD 29000

- El AMD 29000 fué introducido en 1988. Tenía memorias de datos e instrucciones separadas (como el nanoMIPS), y un ancho de bus de 32bits.



Procesadores RISC

52

Otros procesadores RISC comerciales

- **PA-RISC:** Hewelett Packard
- **Power PC:** Apple, Motoroloa e IBM (Power Macintosh)
- **MIPS**

Referencias

- Organización y Arquitectura de Computadoras, William Stallings, Capítulo 12, 5ta ed.
- Diseño y evaluación de arquitecturas de computadoras, M. Beltrán y A. Guzmán, Capítulo 1, 1er ed.
- Arquitectura de computadores – Un enfoque estructurado, Hennessy y Patterson.

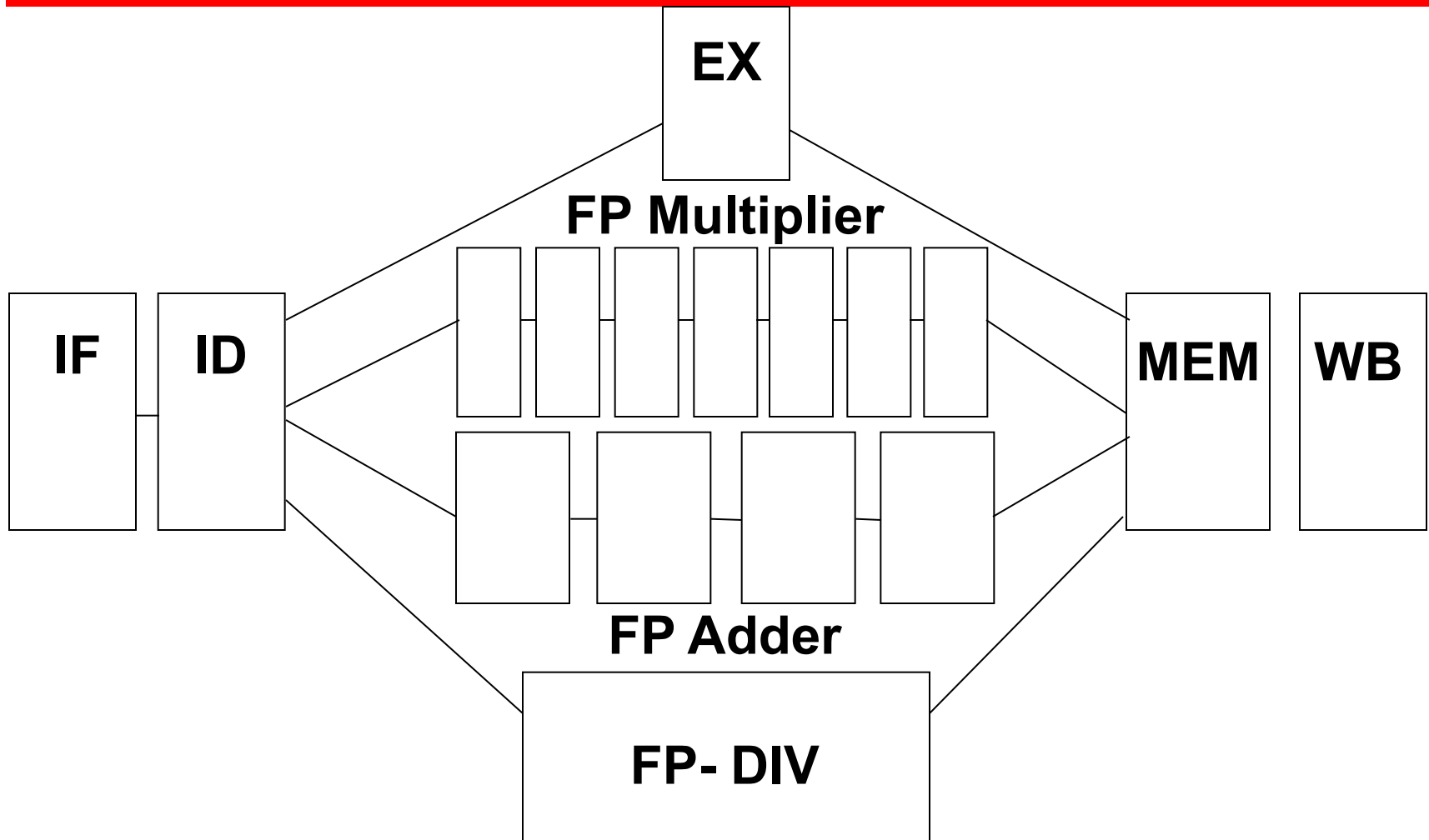
Arquitectura de Computadoras

Simulador
WINMIPS64

Procesador MIPS

- 32 registros de uso general: r0 .. r31 (64 bits)
 - excepto r0 siempre igual a 0
- 32 registros de punto flotante: f0 .. f31 (64 bits)
- 2^{30} palabras de memoria (32 bits c/u)
- Instrucciones de 1 palabra de longitud (32 bits)
- Acceso a memoria limitado a 2 instrucciones
 - LOAD (carga de memoria en un registro)
 - STORE (almacena un registro en memoria)

Segmentación en el MIPS



Segmentación en MIPS (2)



⌘ Búsqueda (IF)

- ☒ Se accede a memoria por la instrucción
- ☒ Se incrementa el PC

⌘ Decodificación / Búsqueda de operandos (ID)

- ☒ Se decodifica la instrucción
- ☒ Se accede al banco de registros por los operandos
- ☒ Se calcula el valor del operando inmediato con extensión de signo (si hace falta)
- ☒ Si es un salto, se calcula el destino y si se toma o no

⌘ Ejecución / Dirección efectiva (EX)

- ☒ Si es una instrucción de proceso, se ejecuta en la ALU
- ☒ Si es un acceso a memoria, se calcula la dirección efectiva
- ☒ Si es un salto, se almacena el nuevo PC

⌘ Acceso a memoria / terminación del salto (MEM)

- ☒ Si es un acceso a memoria, se accede

⌘ Almacenamiento (WB)

- ☒ Se almacena el resultado (si lo hay) en el banco de registros

Directivas al assembler (MIPS64)

- .data** - comienzo de segmento de datos
- .text** - comienzo de segmento de código
- .code** - comienzo de segmento de código (= .text)
- .org <n>** - dirección de comienzo
- .space <n>** - deja n bytes vacios
- .asciiz <s>** - entra string ascii terminado en cero
- .ascii <s>** - entra string ascii

donde <n> es un número como 24 y <s> denota un string como "fred".

Directivas al assembler (2)

- .word <n1>,<n2>..** - entra word(s) de dato (64-bits)
- .byte <n1>,<n2>..** - entra bytes
- .word32 <n1>,<n2>..** - entra número(s) de 32 bit
- .word16 <n1>,<n2>..** - entra número(s) de 16 bit
- .double <n1>,<n2>..** - entra número(s) en floating-point

donde <n1>,<n2>.. son números separados por comas.

E/S del MIPS64

E/S mapeada en memoria.

Dirección de CONTROL= 0x10000 y DATA=0x10008

Si CONTROL = 1, Set DATA con Entero S/S para sacar

Si CONTROL = 2, Set DATA con Entero C/S para sacar

Si CONTROL = 3, Set DATA con Punto Flotante para sacar

Si CONTROL = 4, Set DATA con dirección comienzo de string para sacar

Si CONTROL = 5, Set DATA+5 con coordenada X, DATA+4 con coordenada Y
y DATA con color RGB para sacar

Si CONTROL = 6, limpia la pantalla terminal

Si CONTROL = 7, limpia la pantalla gráfica

Si CONTROL = 8, leer DATA (sea un entero o pto fte) del teclado

Si CONTROL = 9, leer un byte de DATA, sin eco de caracter.

Instrucciones Load/Store

- LD R1, offset(R2) ; Load Doubleword (64 bits)
 - LB R1, offset(R2) ; Load Byte
 - LBU R1, offset(R2) ; Load Byte s/signo
 - LH R1, offset(R2) ; Load Halfword (16 bits)
 - LHU R1, offset(R2) ; Load Halfword s/signo
 - LW R1, offset(R2) ; Load Word (32 bits)
 - LWU R1, offset(R2) ; Load Word s/signo
- SD R1, offset(R2) ; Store Doubleword
 - SB R1, offset(R2) ; Store Byte
 - SH R1, offset(R2) ; Store Halfword
 - SW R1, offset(R2) ; Store Word

Instrucciones ALU inmediatas

- DADDI R1,R2,7 ; R1= R2 + Inmediato
 - DADDUI R1,R2,7 ; R1= R2 + Inmediato s/signo
- SLTI R1,R2,7 ; si R2<Inmediato then R1=1
- ANDI R1,R2,7 ; R1= R2 And Inmediato
- ORI R1,R2,7 ; R1= R2 Or Inmediato
- XORI R1,R2,7 ; Exclusive Or Immediate

Instrucciones ALU en registros

- DADD R1,R2,R3 ; R1= R2 Add R3
 - DADDU R1,R2,R3 ; R1= R2 Add R3 s/signo
- DSUB R1,R2,R3 ; R1= R2 Subtract R3
 - DSUBU R1,R2,R3 ; R1= R2 Subtract R3 s/signo
- SLT R1,R2,R3 ; Si $R2 < R3$ then $R1=1$
- AND R1,R2,R3 ; R1= R2 And R3
- OR R1,R2,R3 ; R1= R2 Or R3
- XOR R1,R2,R3 ; R1= R2 Exclusive Or R3

Instrucciones ALU en registros (2)

de desplazamiento:

- DSLL R1,R2,4 ; Shift Left Logical
 - DSLLV R1,R2,R3 ; idem anterior Variable
- DSRL R1,R2,4 ; Shift Right Logical
 - DSRLV R1,R2,R3 ; idem anterior Variable
- DSRA R1,R2,4 ; Shift Right Arithmetic
 - DSRAV R1,R2,R3 ; idem anterior Variable

Instrucciones Punto Flotante

Movimiento:

- L.D F1, offset(R0) ; Load Double precision float
- S.D F1, offset(R0) ; Store Double precision float

- MTC1 F1, R1 ; Move Word a Floating Point
- MOV.D F1, F2 ; Move Floating Point

Instrucciones Punto Flotante (2)

Aritméticas:

- ADD.D F1, F2, F3 ; Floating Point Add
- DIV.D F1, F2, F3 ; Floating Point Divide
- MUL.D F1, F2, F3 ; Floating Point Multiply
- SUB.D F1, F2, F3 ; Floating Point Subtract

Instrucciones Punto Flotante (3)

Conversión:

- CVT.L.D F1,F2 ; Floating Point a entero
; (64bits)
- CVT.W.D F1,F2 ; Floating Point a entero
; (32bits)

Instrucciones de control de flujo

Salto incondicional:

- J offset ; Jump a offset
- JAL offset ; Jump and Link a offset
- JR R1 ; Jump a dir. en Registro

Instrucciones de control ... (2)

Salto condicional que compara 2 registros:

- BEQ R1, R2, offset ; si $R1 = R2$ saltar a offset
 - BNE R1, R2, offset ; si $R1 \neq R2$ saltar a offset

Salto condicional que compara con cero:

- BEQZ R1, offset ; si $R1 = 0$ saltar a offset
 - BNEZ R1, offset ; si $R1 \neq 0$ saltar a offset

Otras instrucciones

- NOP ; No Operación
- HALT ; Detiene el Simulator

Formato Instrucciones Tipo-R

aritmético-lógicas

Op.	Rs	Rt	Rd	Shamnt	funct
6	5	5	5	5	6

Ej: DADD R8, R17, R18 $R8 = R17 + R18$

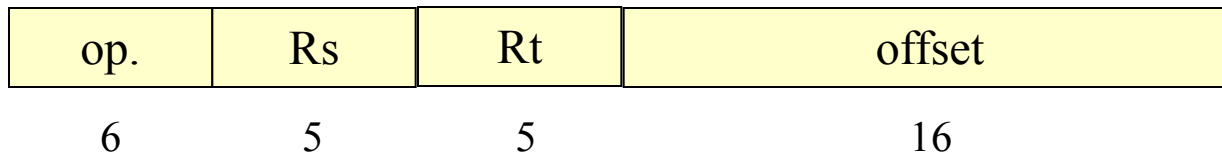
op	17	18	8	0	funct
6	5	5	5	5	6

Ej: SLT R1, R2, R3 if $R2 < R3$ then $R1 = 1$ else $R1 = 0$

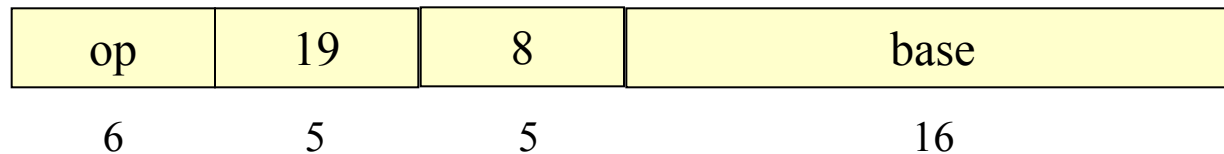
op	2	3	1	0	funct
6	5	5	5	5	6

Formato Instrucciones Tipo-I

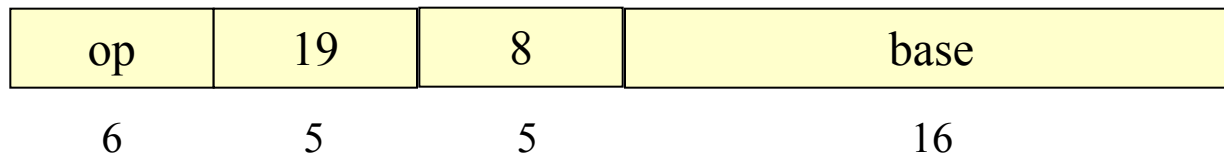
inmediatas



Ej: LD R8, base (R19) $R8 = M[\text{base} + R19]$



Ej: SD R8, base (R19) $M[\text{base} + R19] = R8$

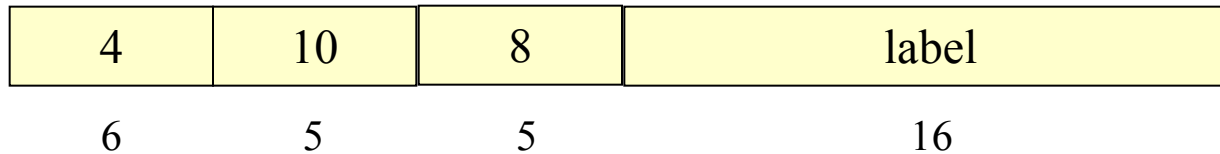


Formato Instrucciones Tipo-I (2)

ramificación o salto condicional

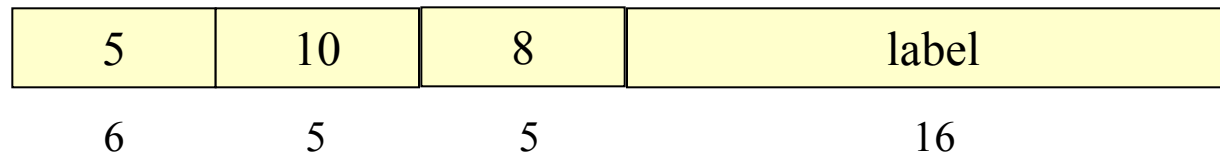
Ej: BEQ R8, R10, label

if R8 = R10 goto label



Ej: BNE R8, R10, label

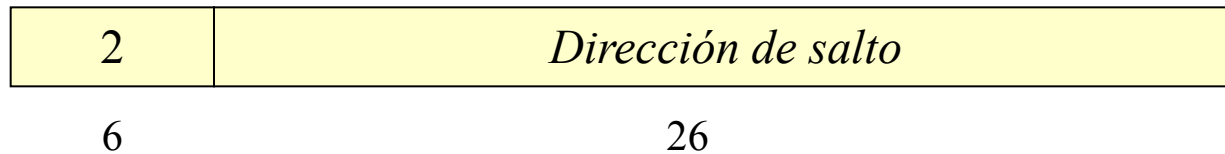
if R8 $\neq$ R10 goto label



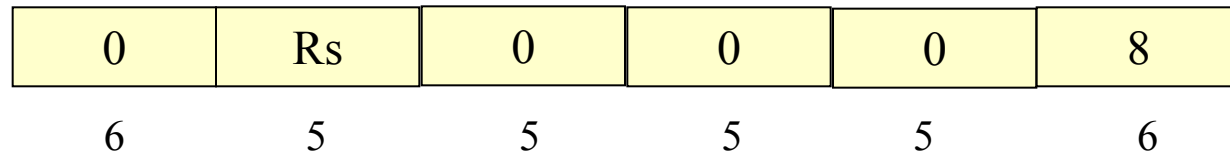
Formato Instrucciones de control

instrucciones de salto

Ej: $J \text{ dir-de-salto}$ $PC = \text{dir-de-salto}$



Ej: $JR \ R3$ $PC = R3$



Llamadas a procedimientos

EL MIPS no tiene pila de hardware, almacena la dirección de retorno **siempre** en R31

- SALTO A SUBROUTINA (Jump And Link)

JAL dir-de-salto

$R31 = PC$

J dir-de-salto

- RETORNO DE SUBROUTINA

JR R31

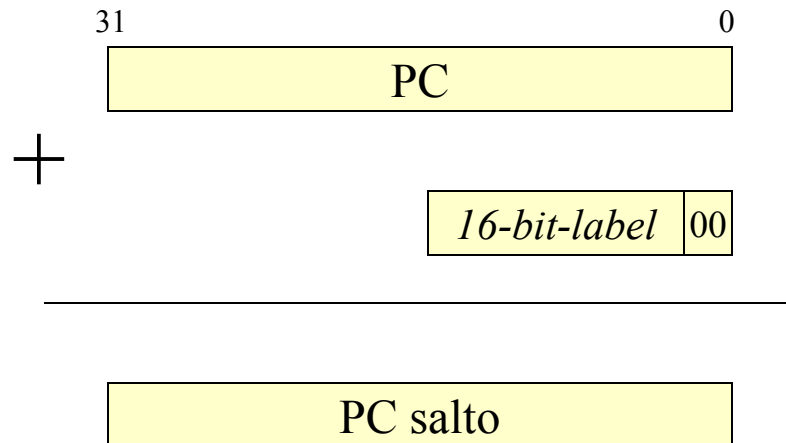
$PC = R31$

Comparación de los saltos

- **BEQ**: salto corto dentro de página

BEQ R4, R5, *16-bit-label*

If (R4==R5) PC = PC + (*16-bit-label* << 2)



Rango de salto → **64K palabras**

RAM

2^{30}

2^{14} páginas

0

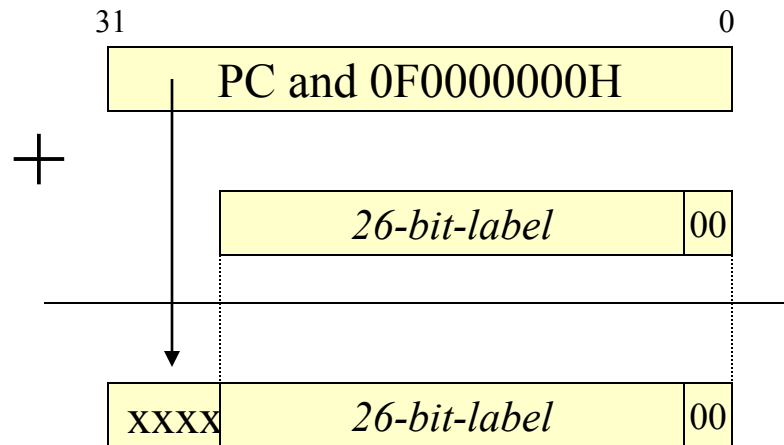
23

Comparación de los saltos (2)

- **J**: salto largo dentro de bloque

J 26-bit-label

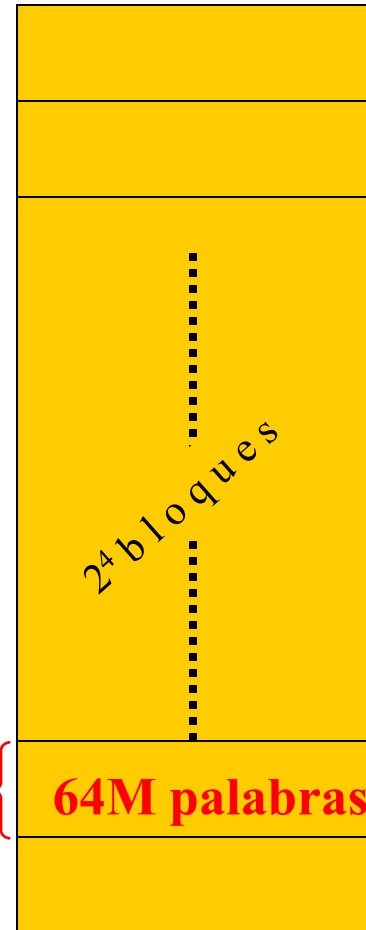
$$PC = (PC \text{ and } 0F0000000H) + (26\text{-bit-label} \ll 2)$$



Rango de salto

RAM

2^{30}



0

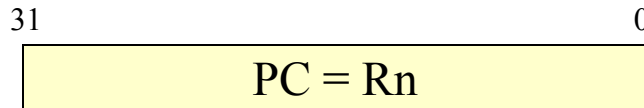
24

Comparación de los saltos (3)

- **JR:** salto largo a toda la memoria

JR R_n

PC = R_n



Rango de salto



Nombres de registros (MIPS)

- \$0 **zero** ,siempre retorna 0
- \$1 **at** ,reservado para uso por el ensamblador
- \$2,\$3 **v0,v1** ,valor retornado por subrutina
- \$4-\$7 **a0-a3** ,argumentos para una subrutina
- \$8-\$15 **t0-t7** ,temporarios para subrutinas
- \$16-\$23 **s0-s7** ,variables de subrutinas. Preservar sus valores
- \$24,\$25 **t8,t9** ,temporarios para subrutinas
- \$26,\$27 **k0,k1** ,usados por manejador de interrupciones/trap
- \$28 **gp** ,puntero global (acceso a var static/extern)
- \$29 **sp** ,puntero de pila
- \$30 **s8/fp** ,noveno registro de variable o frame pointer
- \$31 **ra** ,retorno de subrutina

Ejemplo 1

; C=A+B

.data

A: .word 10

B: .word 8

C: .word 0

.text

```
main: ld    r4, A(r0)      ; A en r4
      ld    r5, B(r0)      ; B en r5
      dadd  r3, r4, r5      ; r3 = r4+r5
      sd    r3, C(r0)      ; resultado en C
      halt
```

Ejemplo 2

for i =1 to 1000 do
A[i]:=B[i]+5;

```
.data
base_B: .word 1,2,3,4,5,6, ...,1000
base_A: .space 1000
.text
DADDI R2, R0, 1           ; variable I = 1 (en R2)
DADDI R5, R0, 5           ; R5 = 5
DADDI R10, R0, 1001       ; límite del FOR (en R10)
ciclo: LD      R1, base_B(R2) ; R1 = B[I]
DADD  R1, R1, R5          ; R1 = B[I] + 5
SD    R1, base_A(R2)      ; A[I] = R1
DADDI R2, R2, 1           ; I = I + 1
BNE   R2, R10, ciclo      ; I <> 1001 => ir a ciclo
HALT
```

Ejemplo 3

.data
busca: .word 7
vect: .word 1,4,8,10,7
largo: .word 5

```
.text
dadd R10,R0,R0 ; registro R10 puesto en '0'
dadd R1,R0,R0 ; registro R1 elegido como indice
ld R2,largo(R0) ; calculamos la dimension del vector vect.
dsll R2,R2,3 ; multiplico R2 x 8
ld R3,busca(R0) ; elemento buscado
loop: ld R4,vect(R1) ; elemento del vector a comparar
      beq R3,R4,found ; salgo de loop si son iguales
      daddi R1,R1,8 ; R1++ (8 byte)
      slt R5,R1,R2 ; comparo (resultado en R5)
      bnez R5,loop ; continuo el ciclo?
      j end ; el valor buscado no se encontró
found: daddi R10,R0,1 ; coloco TRUE en R10
end: halt ; comando winmips de cierre
```

Arquitectura de Computadoras

CURSO 2022

Turno:

Clase 7

Resumen clase 7

Memoria

- Subsistema Memoria
- Organización jerárquica de la memoria
- Memoria caché , concepto.
- Organización de la caché.
- Consideraciones de la caché
- Análisis cuantitativo
- Políticas de asignación
- Políticas de reemplazo y escritura
- Ejemplos prácticos de procesadores con caché

Subsistema de Memoria

En el diseño de la Memoria existe un compromiso entre:

- Capacidad
- Velocidad
- Costo

Una Memoria ideal sería aquella que:

- Es infinitamente grande
- El tiempo de acceso sumamente pequeño (casi 0)
- El costo relativamente bajo

Sistema de Memoria

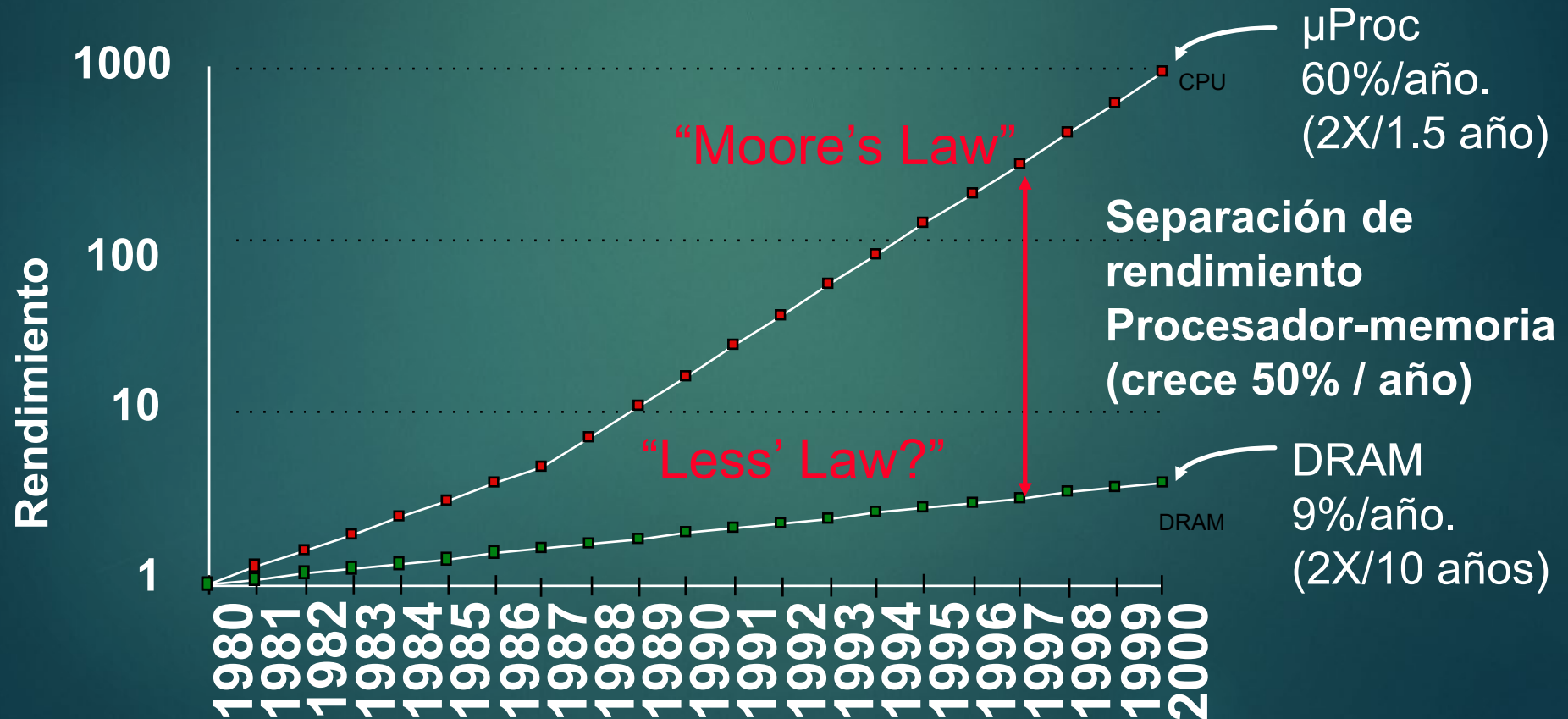
4

Durante años se ha cumplido que:

- La velocidad del procesador aproximadamente se ha duplicado cada 18 meses (casi sin variar su precio), medido en la cantidad de instrucciones ejecutadas por segundo.
- Se ha cuadruplicado el tamaño de la memoria cada 36 meses (al mismo precio). Pero la velocidad aumenta a razón de un 10% anual.
- El desbalance entre la velocidad del procesador y la de la memoria ha generado una brecha que ha crecido a lo largo del tiempo.

Sistema de Memoria

Esta brecha se puede apreciar en el siguiente diagrama temporal de las velocidades de CPU y memoria, para el periodo 1980-2000.



Subsistema de Memoria

En la figura anterior se pueden ver las **curvas de performance de la CPU y la memoria:**

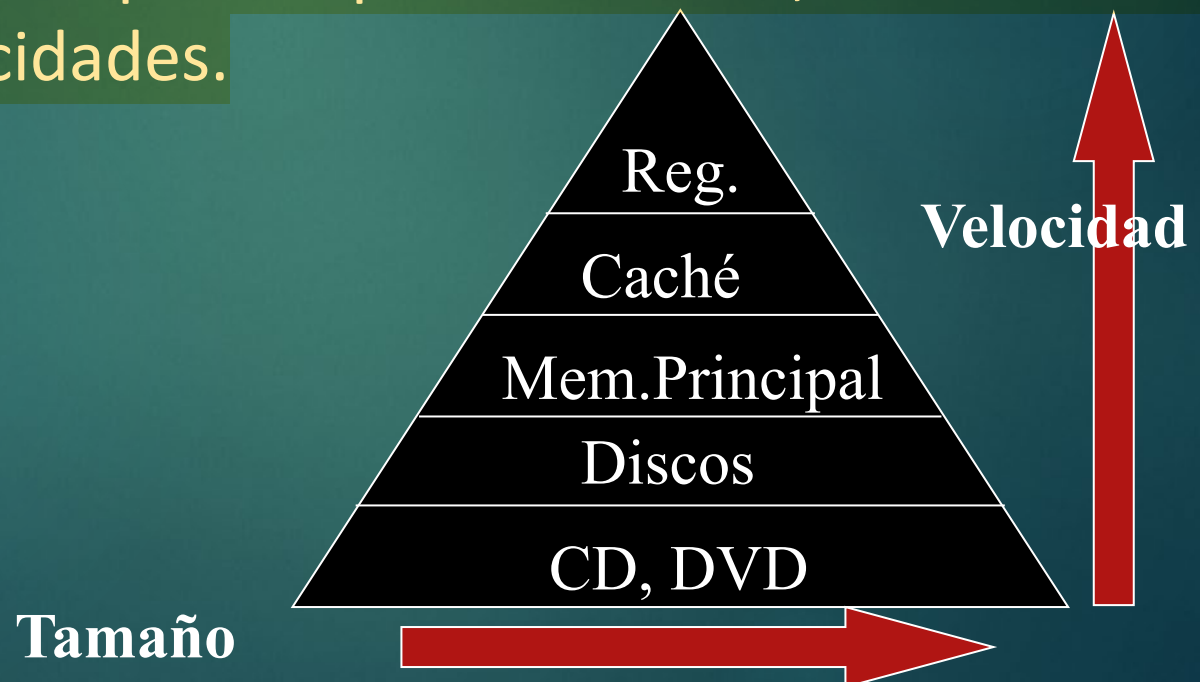
- La velocidad del procesador creció según una curva (conocida como ley de Moore).
- La velocidad de la memoria creció más lentamente.
- Estas diferencias crean un desbalance entre ambos subsistemas.
- Para compensar este desbalance sin impactar fuertemente en el costo del subsistema, se implementa la Memoria como una organización jerárquica compuesta por varios tipos de dispositivos.

Sistema de Memoria

- En una computadora típica hay distintos tipos de memorias, desde las rápidas y caras (Ej.: registros) hasta las lentas y baratas (Ej.: discos).
- En las computadoras actuales los diferentes tipos de memorias actúan coordinadamente y no separadas.
- Esa interacción permite un comportamiento global equivalente al que tendría con una memoria única, grande y rápida.

Jerarquía de Memoria

- La forma en que se organizan coordinadamente los distintos tipos de memoria se conoce con el nombre de Jerarquía de Memoria
- La Jerarquía de memoria se puede pensar como una pirámide de múltiples capas o niveles, de diferentes tamaños y velocidades.



Jerarquía de Memoria

9

Jerarquía de Memoria es:

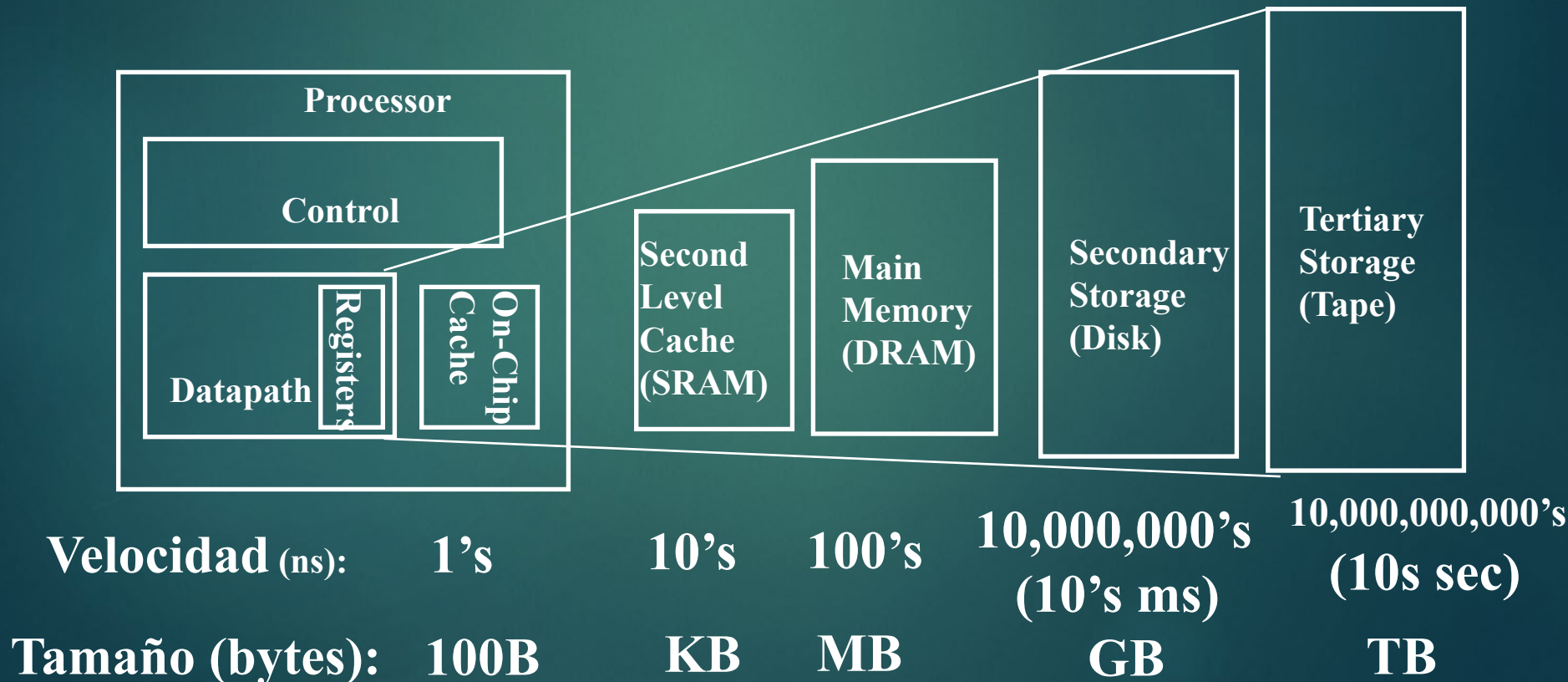
- Un método de administración del almacenamiento de la información (Memoria) estructurado en varios niveles ubicados físicamente en distintos lugares, con tecnologías, costos, tamaños y velocidades distintas.
- De esta manera los programadores “creen” disponer de cantidades casi “ilimitadas” de memoria a un costo accesible, y con velocidades cercanas a las de una memoria “ultrarápida”.

Jerarquía de Memoria

- Niveles principales de la Jerarquía de Memoria:
 - Registros
 - Memoria cache (RAM de muy alta velocidad)
 - Memoria principal (RAM de alta velocidad)
 - Memoria virtual o secundaria (medios de almacenamiento magnético/óptico)
- A medida que nos alejamos de la CPU, los niveles son más grandes, más lentos y más baratos que los niveles previos (o superiores) en la jerarquía, de ahí la forma de pirámide.

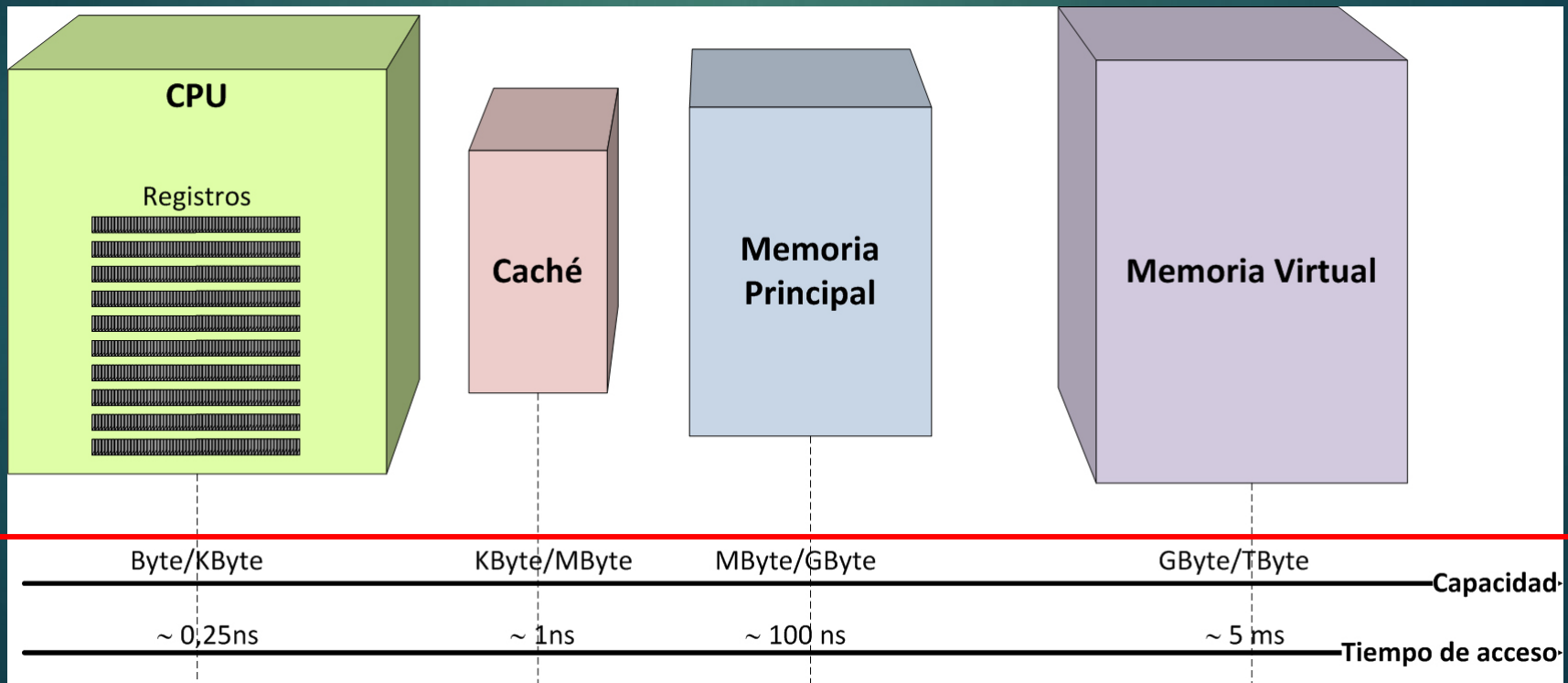
Jerarquía de Memoria

Comparación de ubicación física, tamaños y velocidades de los niveles jerárquicos de la memoria.



Jerarquía de Memoria

Comparación de ubicación física, tamaños y velocidades de los niveles jerárquicos de la memoria.



Jerarquía de Memoria

Los principales objetivos de una Jerarquía de memoria son:

- **Maximizar tamaño:** idealmente disponer de una “capacidad ilimitada”, equiparada al tamaño del nivel más grande.
- **Optimizar velocidad:** simular que se dispone de un banco de memoria “ultrarápida”, próximo a la velocidad del nivel más rápido
- **Minimizar el costo total:** implementar una memoria a un costo cercano al del nivel más lento.

Jerarquía de Memoria

Para que la memoria se comporte como una jerarquía (integrada) debe cumplir las siguientes propiedades:

- Inclusión: los datos almacenados en un nivel han de estar también almacenados en los niveles inferiores a él.
- Coherencia: las copias de la misma información en los distintos niveles deben contener los mismos valores.

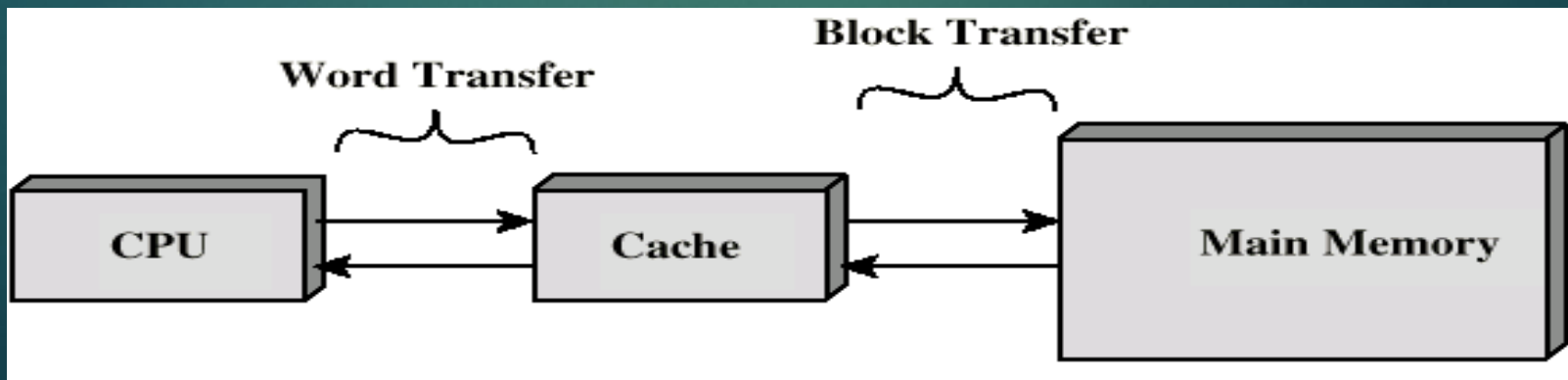
Jerarquía de Memoria

El manejo de la jerarquía de memoria es administrado por:

- **Nivel registros:** el compilador. Se puede decir que el programador no interviene en la administración de este recurso porque en los lenguajes de programación no son visibles (con algunas excepciones)
- **Cache:** la administración se hace por hardware
- **Memoria principal:** la administración la pueden hacer:
 - Hardware
 - Sistema operativo
 - Programador (archivos)

Memoria Cache

- En la Jerarquía de memoria ya se vieron anteriormente los niveles de Registros y Memoria Principal. Ahora se va a analizar el nivel de la Memoria cache.
- La memoria Caché es una memoria pequeña y muy rápida, que se ubica entre la memoria principal y la CPU.
- Puede localizarse en un chip separado o dentro de la CPU, o en ambos lugares.
- Contiene algunos sectores (bloques) de la MP.

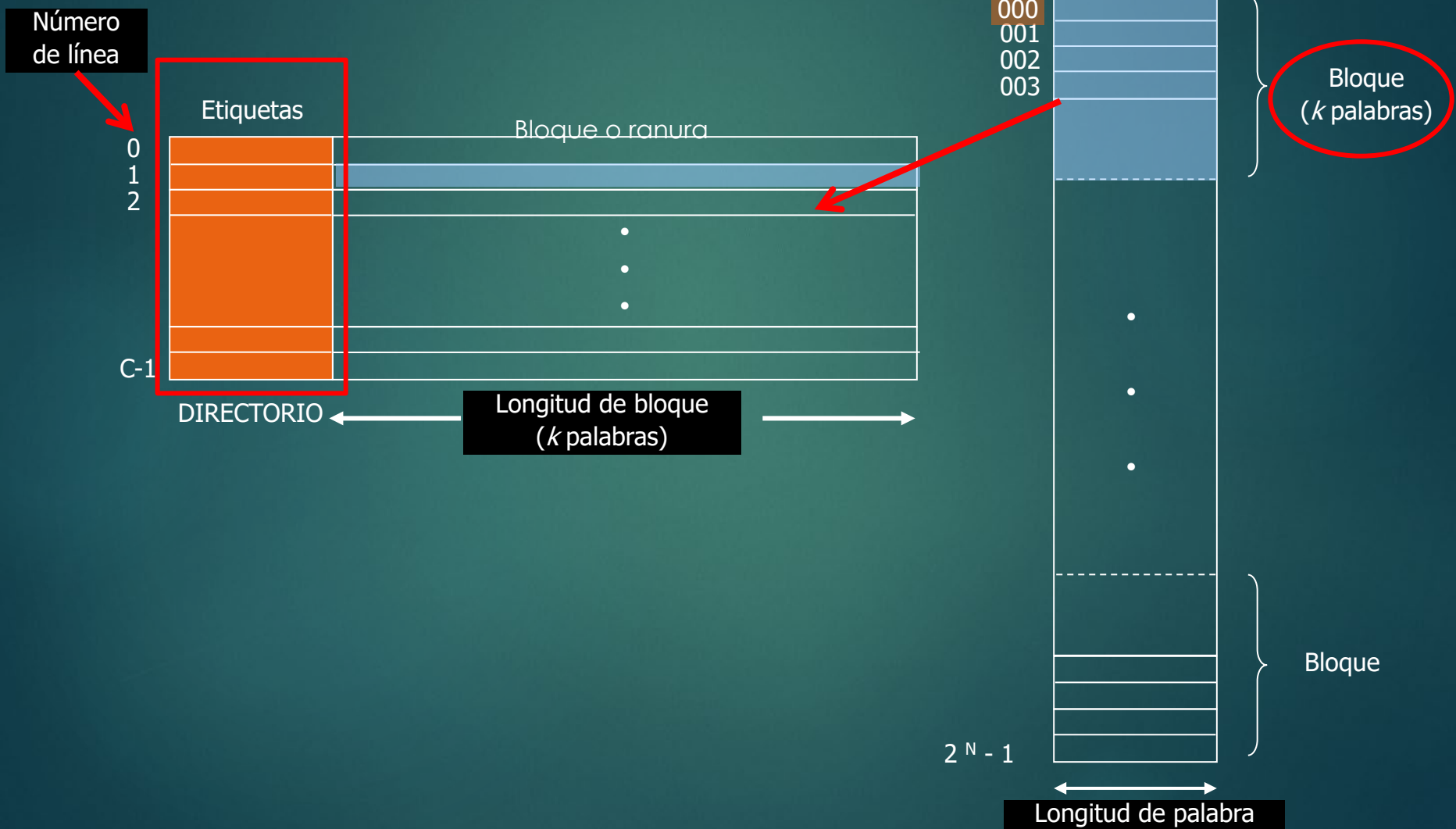


Memoria Cache

- La información contenida en la Caché se organiza en bloques (también llamadas ranuras) de longitud fija (Ej.: 8 bytes, 16 bytes, 32 bytes, etc.).
- En los bloques de la Caché se copian algunos bloques (de idéntico tamaño) de la Memoria principal.
- La cantidad de bloques copiados depende del tamaño de la memoria Caché y del bloque.
- Cada ranura tiene asociada una etiqueta para identificar el bloque de la Memoria que tiene copiado.
- El conjunto de etiquetas forma el directorio de la cache.

Memoria Cache

Relación entre bloques y ranuras



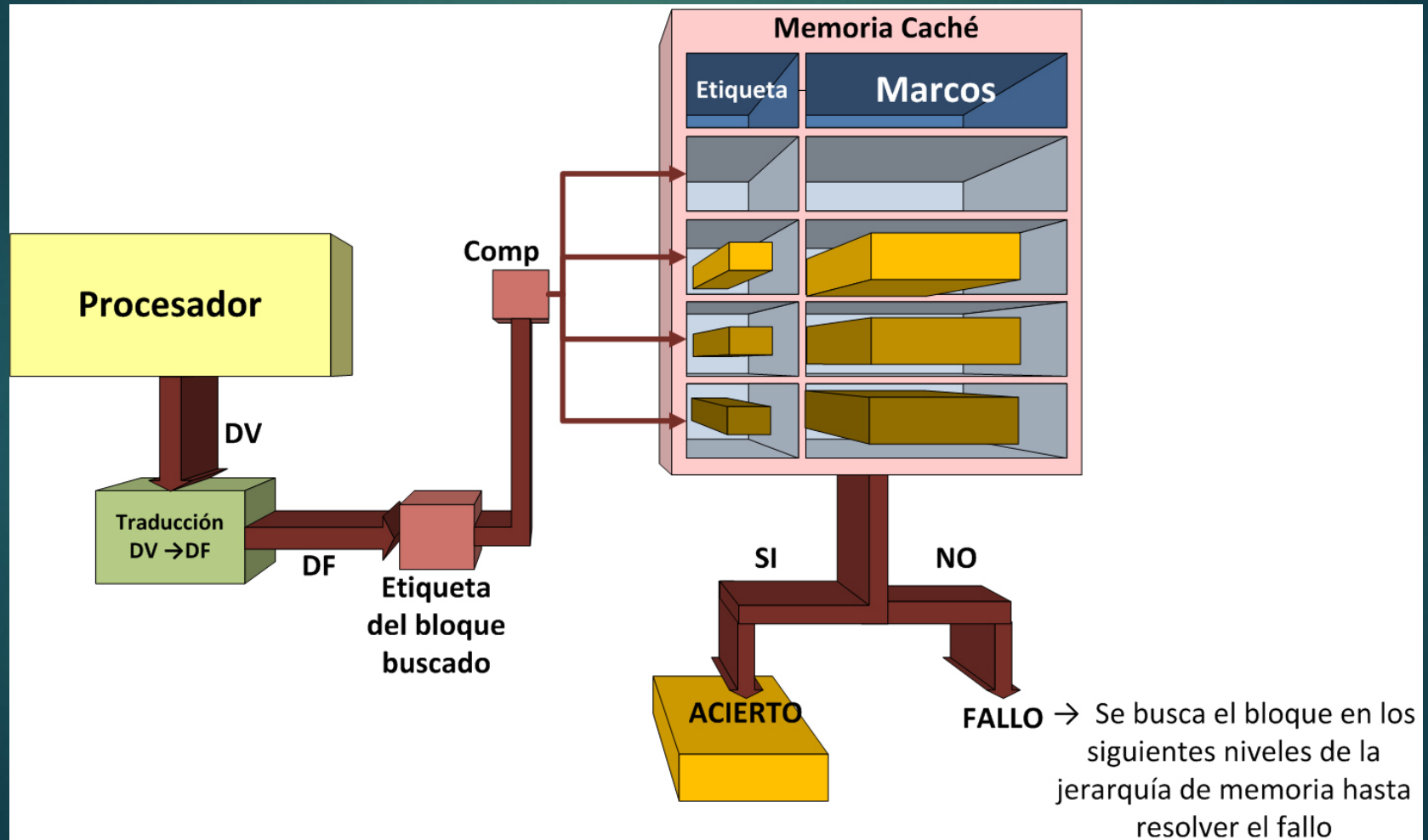
Memoria Cache

Funcionamiento:

- Cuando la CPU necesita un dato, genera 1 dirección de memoria.
- La cache “intercepta” esa dirección y determina si tiene ese dato. Pueden ocurrir 2 situaciones:
 - ACIERTO: si lo tiene, se lo envía a la CPU (a la velocidad de la cache)
 - FALLO: Si no lo tiene, se trae el bloque que contiene esa dirección desde la memoria principal, y la cache entrega el dato requerido a la CPU.

Memoria Cache

Gráficamente, el proceso completo de búsqueda de información cuando hay una memoria caché es el siguiente.



Memoria Cache

- Si el dato que busca la CPU está en la cache, la velocidad del acceso depende del tiempo de acceso de la cache (relativamente muy corto).
- Si el dato no está en la cache, la velocidad del acceso depende del tiempo de acceso de la memoria principal (relativamente largo).
- Así, la eficiencia del uso de la cache depende de la cantidad de veces que “acierta” a la cache.
- La cantidad de veces que se acierta (la “tasa de aciertos”) no necesariamente tiene que ser proporcional al tamaño de la caché, que es miles de veces mas chico que el de la Memoria principal. La razón de esto tiene que ver con el comportamiento de los programas.

Memoria Cache

Principios en los que se basa la memoria caché

Los programas tiene un comportamiento basado en 2 principios (que son de carácter empírico).

➤ Principio de localidad temporal de las referencias:

Es altamente probable que los elementos de memoria referenciados recientemente (datos o instrucciones) vuelvan a ser referenciados en el corto tiempo.

➤ Principio de localidad espacial de las referencias:

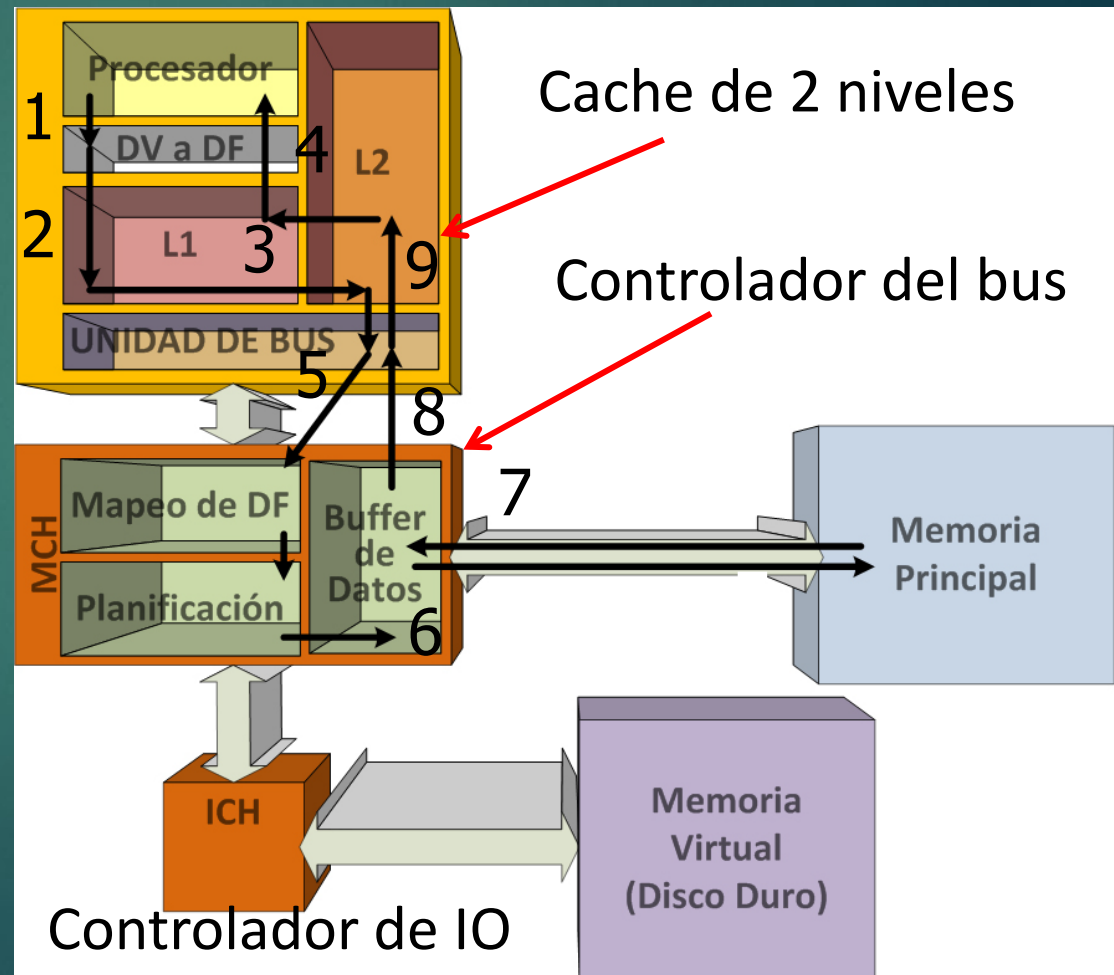
Es altamente probable que los próximos elementos de memoria referenciados estén en las proximidades de los últimos referenciados.

Memoria Cache

Desde el punto de vista físico, el **proceso de acceso a los datos de parte de la CPU** se muestra en la figura siguiente.

El camino 1-2-3-4 corresponde a un acierto.

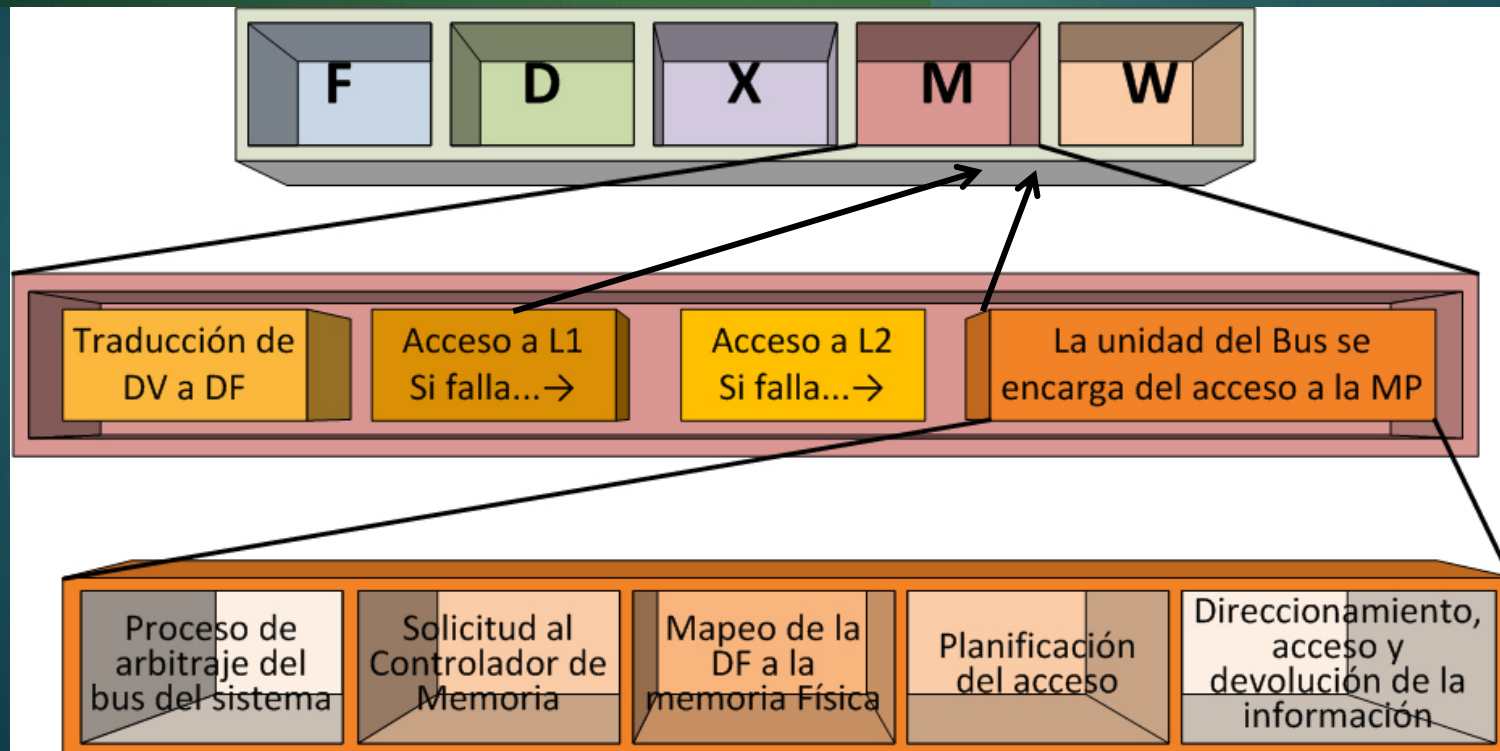
El camino 1-2-5-6-7-8-9-3-4 corresponde a un fallo.



Memoria Cache

Acceso a memoria en un procesador segmentado

En un procesador con segmentación del cauce, se dispone de un ciclo de reloj para el acceso a memoria. Ese tiempo debe ser suficiente para el acceso a la memoria caché. En caso de fallo el acceso a la memoria principal requiere de varios ciclos extra.



Memoria Cache

Tal como se vio, cuando la CPU busca un dato se pueden dar 2 situaciones:

- **Acierto (se conoce como “hit”):** se encuentra en la caché el dato solicitado
- **Fallo (se conoce como “miss”):** no se encuentra en la caché el dato solicitado
- Cuando ocurre un fallo, el bloque que contiene la palabra accedida se copia de la memoria principal a una línea de caché.
- Los fallos de caché se gestionan mediante hardware y causan que el procesador se detenga hasta que el dato esté disponible.
- Esta acción requiere un tiempo determinado.

Memoria Cache

- El tiempo para servir un fallo depende de la latencia y ancho de banda de la memoria principal.
- La latencia es el tiempo necesario para completar un acceso a memoria (depende de la memoria).
- El ancho de banda es la velocidad a la cual se puede transferir el dato, es decir, es la cantidad de información por unidad de tiempo que puede transferirse desde/hacia la memoria (depende de la velocidad del bus).

Memoria Cache

El tiempo de acceso promedio de la CPU es el promedio del tiempo que tarda en obtener los datos buscados en memoria, compuestos por accesos a la caché y accesos a la memoria principal.

$$t_{\text{CPU}} = (1 - \text{TF}) t_{\text{accesoMC}} + \text{TF} \times \text{PF}$$

TF es la “tasa de fallos”, que se obtiene como:

TF = Tasa de Fallos = número de fallos / número total de accesos

PF es la “penalización por fallo”, es decir el tiempo “gastado” en acceder a la Memoria principal (t_{accesoMP})

- Para mejorar las prestaciones hay que reducir el t_{accesoMC} , TF y PF.

Organización de la cache

Consideraciones sobre la cache

Para el diseño de la caché hay que tener en cuenta varias consideraciones.

1. Tamaño de la memoria caché

- Debe ser suficientemente grande para contener la mayor cantidad posible de información.
- Pero no demasiado grande porque el tamaño tiene impacto en la velocidad (es decir, en el t_{accesoMC}) y en el costo.

Organización de la cache

29

Consideraciones sobre la cache

2. Tamaño del bloque o ranura

- El tamaño del bloque es muy importante en la tasa de aciertos.
- El bloque o ranura debe ser suficientemente grande para aprovechar al máximo las referencias cercanas. Al aumentar el tamaño mejora la tasa de aciertos, hasta un cierto tamaño del bloque. Después de eso, aumentar el tamaño no mejora la tasa de aciertos.
- Por otra parte, al aumentar el tamaño del bloque hay menos bloques de la memoria principal en la caché, lo que tiende a aumentar la tasa de fallos, y la penalización por fallos porque son más palabras a transferir entre la memoria principal y la caché.
- Existe un valor óptimo para el tamaño del bloque.

Organización de la cache

Consideraciones sobre la cache

3. Costo

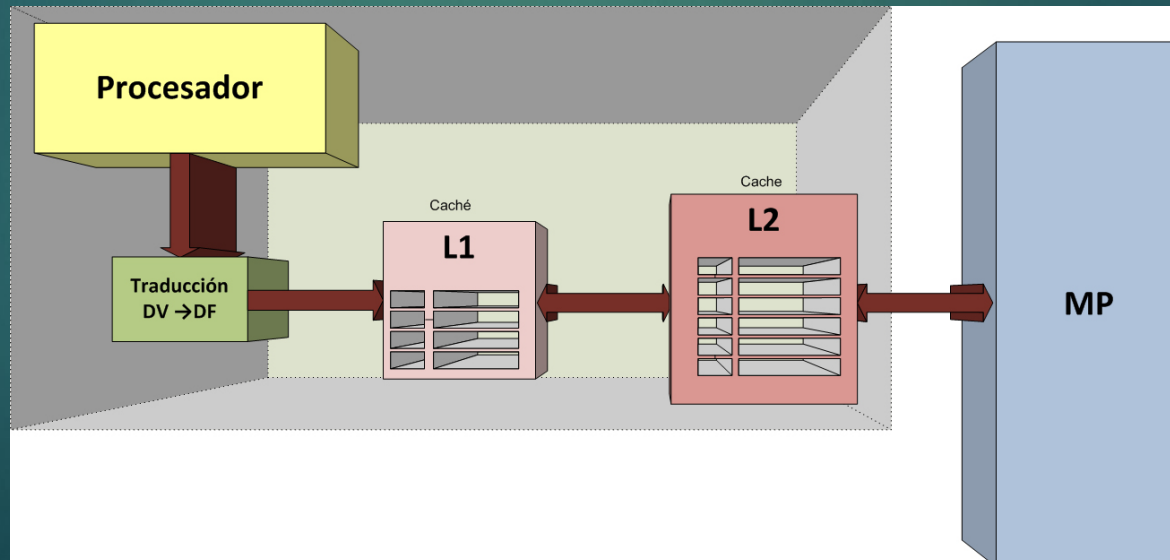
- El costo crece fuertemente con el tamaño de la memoria.
- Y este costo es significativamente grande cuando la caché está incluida en el chip que contiene el procesador.

Organización de la cache

Consideraciones sobre la cache

4. Niveles de la caché

- La memoria caché puede ser una sola (1 nivel) o estar dividida en varias unidades (múltiples niveles). Los múltiples niveles de caché por lo general tienen distintos tamaños.
- En la figura siguiente, la caché está dividida en 2 niveles, el L1 muy pequeño y ultrarápido, y el L2 un poco más grande y lento.



Organización de la cache

Consideraciones sobre la cache

5. Separación de la caché de instrucciones y operandos

- El mecanismo de acceso a las instrucciones es distinto al del acceso de los datos. Es por ello que las estrategias para obtener una alta tasa de aciertos es distinta en un caso que en el otro.
- Teniendo en cuenta esto, es posible mejorar la tasa de aciertos general si la caché se divide en una caché de instrucciones y una de datos, con distintas características.
- Por ejemplo, ambas caches pueden tener distintos tamaños y políticas de acceso y reemplazo.

Organización de la cache

Análisis cuantitativo de los fallos en la caché

- Para entender el impacto en la velocidad de ejecución de un programa, de los fallos en los accesos a la Caché, supongamos el siguiente problema.
- Se tiene un procesador con:
 - Frecuencia de reloj de 2 GHz
 - CPI=1 (ciclos de reloj por instrucción)
 - Caches de instrucciones y datos separadas,
 - Cuánto tarda en ejecutar 100 instrucciones?

Organización de la cache

Análisis cuantitativo de los fallos en la caché

Si $F = 2 \text{ GHz}$

Entonces el ciclo (período) del reloj es:

$$T = 1/f = 1 / 2 \times 10^9 \text{ seg} = 0,5 \times 10^{-9} \text{ seg} = 0,5 \text{ nseg}$$

es decir:

$$1 \text{ Instrucción} = 1 \text{ ciclo de reloj} = 0,5 \text{ nseg}$$

Organización de la cache

Análisis cuantitativo de los fallos en la caché - Caso 1

Se tienen 2 memorias caches ideales: CI (caché de instrucciones) y CD (caché de datos):

- Los tiempos de accesos de ambas caché son igual a 0.

$$t_{\text{accesoCI}} = t_{\text{accesoCD}} = 0$$

- Las tasas de fallos son 0 (la CPU siempre “acierta” a las caché).

$$TF_{\text{CD}} = TF_{\text{CI}} = 0$$

- El tiempo total requerido por la CPU para ejecutar 100 instrucciones es:

$$t_{\text{total}} = t_{\text{cpu}} + t_{\text{mem}} \quad \text{donde:}$$

- t_{cpu} es el tiempo de ejecución de la instrucción
- t_{mem} es el tiempo de acceso a las instrucciones y datos

Organización de la cache

Análisis cuantitativo de los fallos en la caché - Caso 1

Los tiempos de accesos a la memoria se pueden descomponer en accesos a instrucciones y accesos a datos:

$$t_{\text{mem}} = t_{\text{mem_inst}} + t_{\text{mem_datos}} \quad \text{por lo tanto:}$$

$$t_{\text{total}} = t_{\text{cpu}} + t_{\text{mem_inst}} + t_{\text{mem_datos}}$$

En general, se puede considerar que:

$$t_{\text{mem_inst}} = N^{\circ} \text{ accesosIns}(t_{\text{accesoCI}} + TF_{\text{CI}} \cdot PF_{\text{MI}})$$

$$t_{\text{mem_datos}} = N^{\circ} \text{ accesosDat}(t_{\text{accesoCD}} + TF_{\text{CD}} \cdot PF_{\text{MD}})$$

donde PF_{MI} y PF_{MD} son las penalizaciones por falla de acceso a las memorias caché de datos e instrucciones.

En el caso ideal que estamos considerando:

$$t_{\text{accesoCI}} = t_{\text{accesoCD}} = TF_{\text{CI}} = TF_{\text{CD}} = 0$$

Organización de la cache

Análisis cuantitativo de los fallos en la caché - Caso 1
es decir:

$$t_{\text{mem}} = t_{\text{mem_inst}} + t_{\text{mem_datos}} = 0$$

y, por lo tanto:

$$t_{\text{total}} = t_{\text{cpu}} + t_{\text{mem_inst}} + t_{\text{mem_datos}} = t_{\text{cpu}}$$

Finalmente:

$$t_{\text{total}} = t_{\text{cpu}} = nI \times \text{CPI} \times T = 100 \times 1 \times 0,5 \text{ ns} = 50\text{ns}$$

El tiempo total gastado en ejecutar 100 instrucciones corresponde únicamente al tiempo neto ocupado por la CPU, dado que no hay pérdidas de tiempo por accesos a las memorias caché.

Organización de la cache

Análisis cuantitativo de los fallos en la caché - Caso 2

Ahora se tienen 2 memorias caches con tiempos de accesos despreciables (es decir, 0) pero con tasas de fallas mas “reales”:

- Cache de instrucciones con:
 - $t_{\text{accesoCI}} = 0$
 - TF = 4% (la tasa de fallos a la caché de instrucciones es de 0,04)
 - PF = 100ns (cuando no accede a la caché de instrucciones se requieren 100nseg para resolver el acceso a la memoria principal)
- Cache de datos con:
 - $t_{\text{accesoCD}} = 0$
 - TF = 6%
 - PF = 115ns

Organización de la cache

39

Análisis cuantitativo de los fallos en la caché - Caso 2

➤ Suponemos que el 25% de las instrucciones acceden a datos.

Se puede calcular nuevamente el tiempo requerido para resolver 100 instrucciones.

$$t_{\text{total}} = t_{\text{cpu}} + t_{\text{mem_inst}} + t_{\text{mem_datos}}$$

Los tiempos requeridos para los accesos a las instrucciones y a los datos son:

$$t_{\text{mem_inst}} = N^{\circ} \text{ accesosIns} (t_{\text{accesoCI}} + TF_{\text{CI}} \cdot PF_{\text{MI}}) = 100 (0 + 0,04 \times 100 \text{nseg})$$

$$t_{\text{mem_datos}} = N^{\circ} \text{ accesosDat} (t_{\text{accesoCD}} + TF_{\text{CD}} \cdot PF_{\text{MD}}) = 25 (0 + 0,06 \times 115 \text{nseg})$$

Por lo tanto:

$$t_{\text{total}} = t_{\text{cpu}} + t_{\text{mem_inst}} + t_{\text{mem_datos}} = 50 \text{nseg} + 400 \text{nseg} + 172 \text{nseg} = 622,5 \text{nseg}$$

Organización de la cache

Análisis cuantitativo de los fallos en la caché - Conclusiones

- En el análisis anterior se ha considerado que:
 - Tasas de fallo típicos de instrucciones y datos
 - Los tiempos de accesos a las memorias caché se han despreciado respecto de las penalizaciones por fallo
- Las penalizaciones por fallo en los accesos a las cache de instrucciones y datos producen un tiempo de ejecución más de 10 veces superior al requerido por la CPU. Es decir, el tiempo gastado por accesos a la memoria principal (caso de fallos a accesos a las caché) es mucho mayor del propio de la CPU.

Diseño de la caché

En el diseño de la caché se deben definir:

- **Organización:** tamaño de la caché, cantidad y tamaño de bloques
- **Política de asignación:** es decir, el tipo de función de correspondencia entre los bloques de la Memoria Principal y los bloques o ranuras de la Caché.
- **Política de reemplazo:** se refiere a los algoritmos para reemplazar bloques en la memoria Caché.
- **Política de escritura:** son los mecanismos de escritura en Memoria Principal.

Políticas de asignación

Las políticas de asignación son las funciones (de mapeo) que definen la forma en que se van a asignar los bloques de la Memoria Principal en la Memoria Caché.

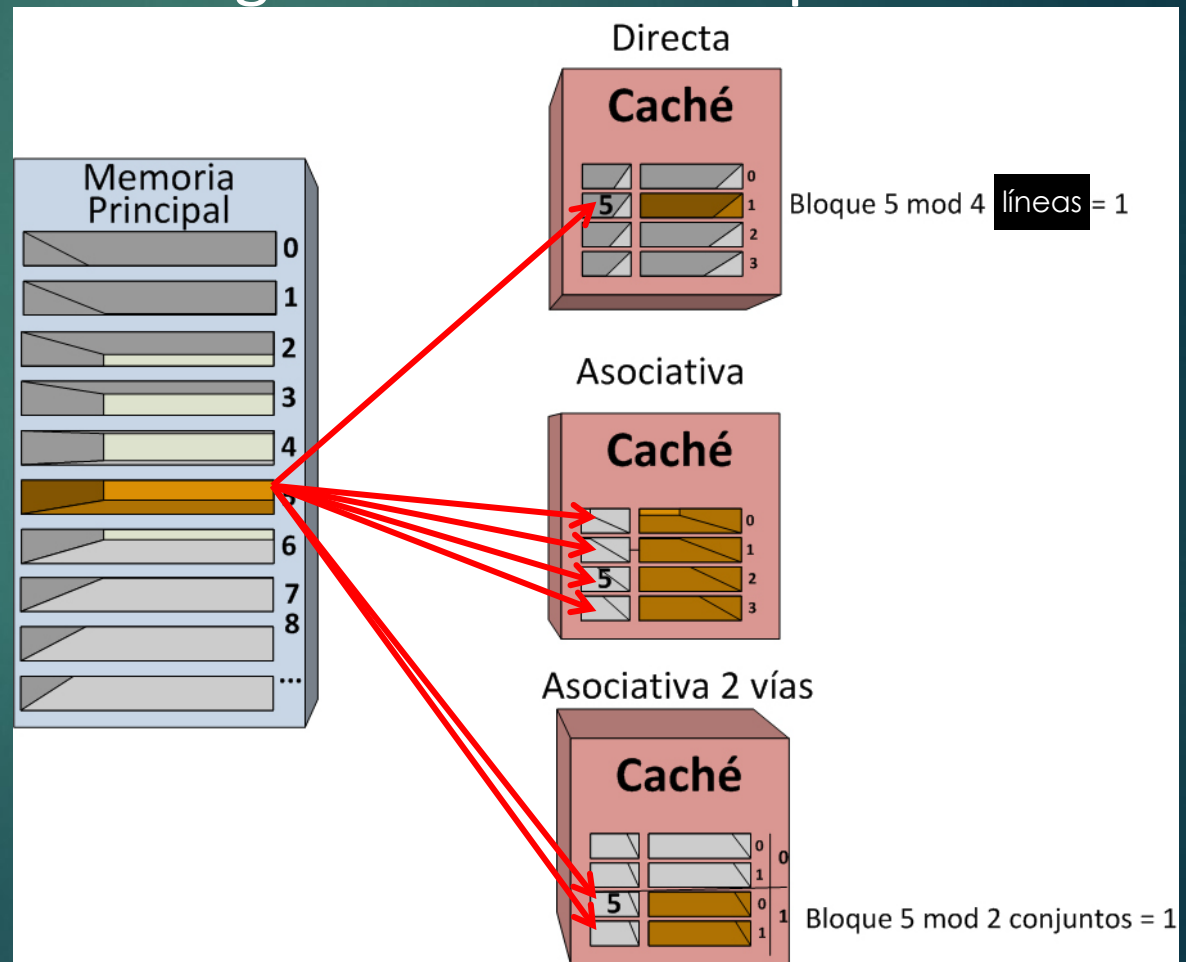
Las políticas más empleadas son:

- Correspondencia totalmente asociativa: Un bloque puede almacenarse libremente en cualquier lugar de la caché.
- Correspondencia directa: Un bloque sólo puede estar almacenado en un lugar fijo de la caché.
- Correspondencia asociativa por conjuntos: Un bloque puede almacenarse en un conjunto restringido de lugares en la caché.

Políticas de asignación

Políticas de asignación típicas

Posibles alternativas de asignación del bloque 5 de la Memoria Principal



Políticas de asignación

Política de asignación Asociativa

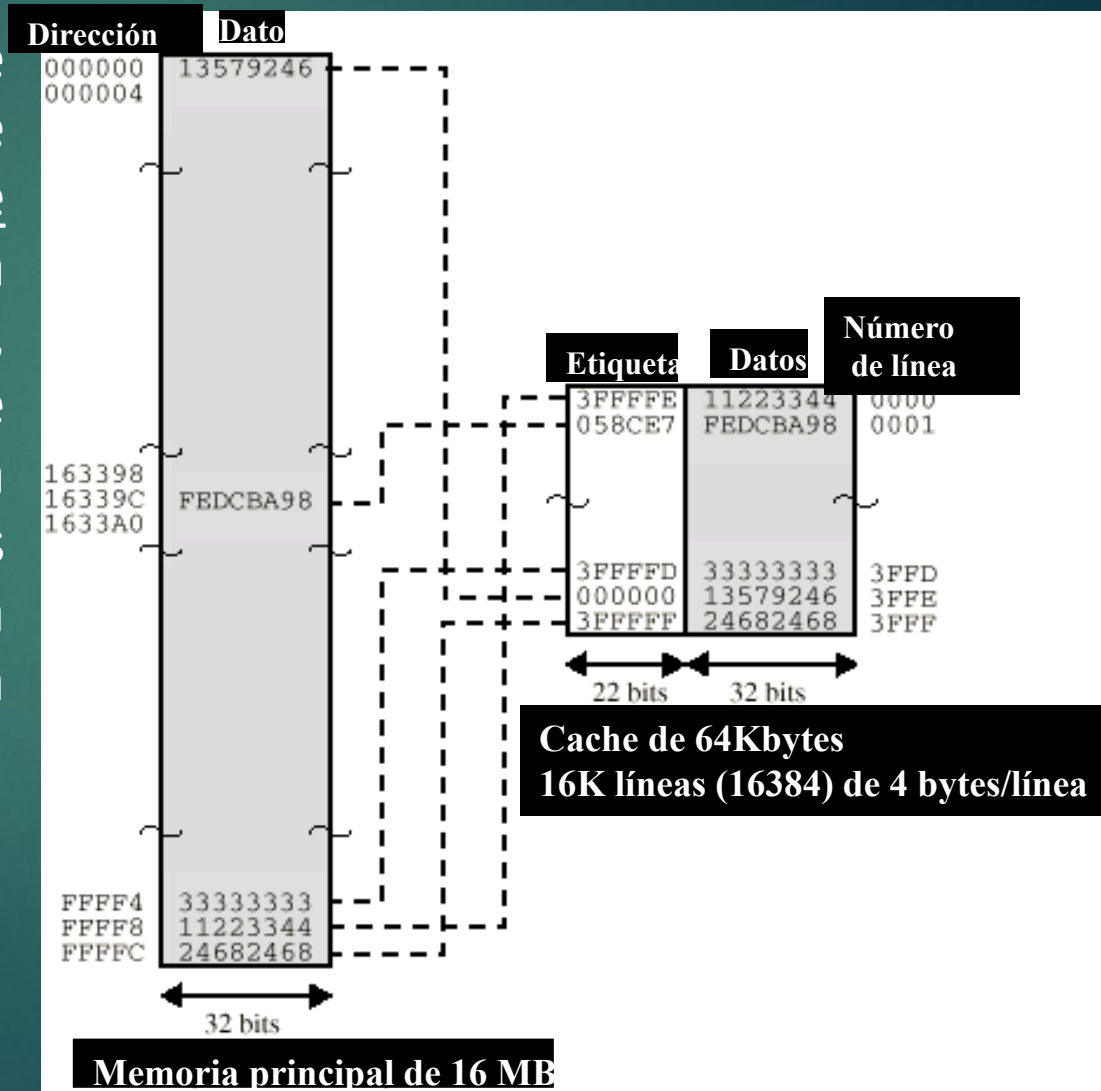
En la imagen de al lado se muestra un ejemplo de asignación completamente asociativa de una memoria de 16MB y caché de 64KB, con bloques de 4 bytes. De los 24 bits que forman la dirección, los 22 bits más significativos se usan para identificar el bloque, y 2 para la palabra dentro del bloque.

Dirección a memoria

22	2
----	---

Bloque o línea

Palabra



Políticas de asignación

Política de asignación Asociativa

En la figura anterior, se puede apreciar que:

- **La dirección de memoria está compuesta de 2 campos**
 - **Tag:** es el número de bloque en la memoria principal
 - **Word:** es el número de palabra dentro del bloque
- El tag es comparado simultáneamente con todas las etiquetas de la caché, que identifican qué bloques de la MP están asignados a la caché. Las etiquetas de la memoria caché forman el “directorio” de la caché.
- Si la comparación da un acierto, el dato se busca en la caché. Si la comparación da una falla, el dato se trae de la memoria principal.

Políticas de asignación

Política de asignación Asociativa - Conclusiones

- Un bloque de memoria principal puede colocarse en cualquier línea de la cache.
- La etiqueta identifica unívocamente un bloque de memoria.
- Todas las etiquetas de las líneas se examinan para buscar una coincidencia. Para esa búsqueda se requiere una memoria del tipo asociativa (es decir, una memoria de acceso por contenido CAM) para implementar el directorio.
- La implementación del directorio de la caché es compleja y costosa.
- Esta política permite una libertad absoluta para la asignación y reemplazo de los bloques.

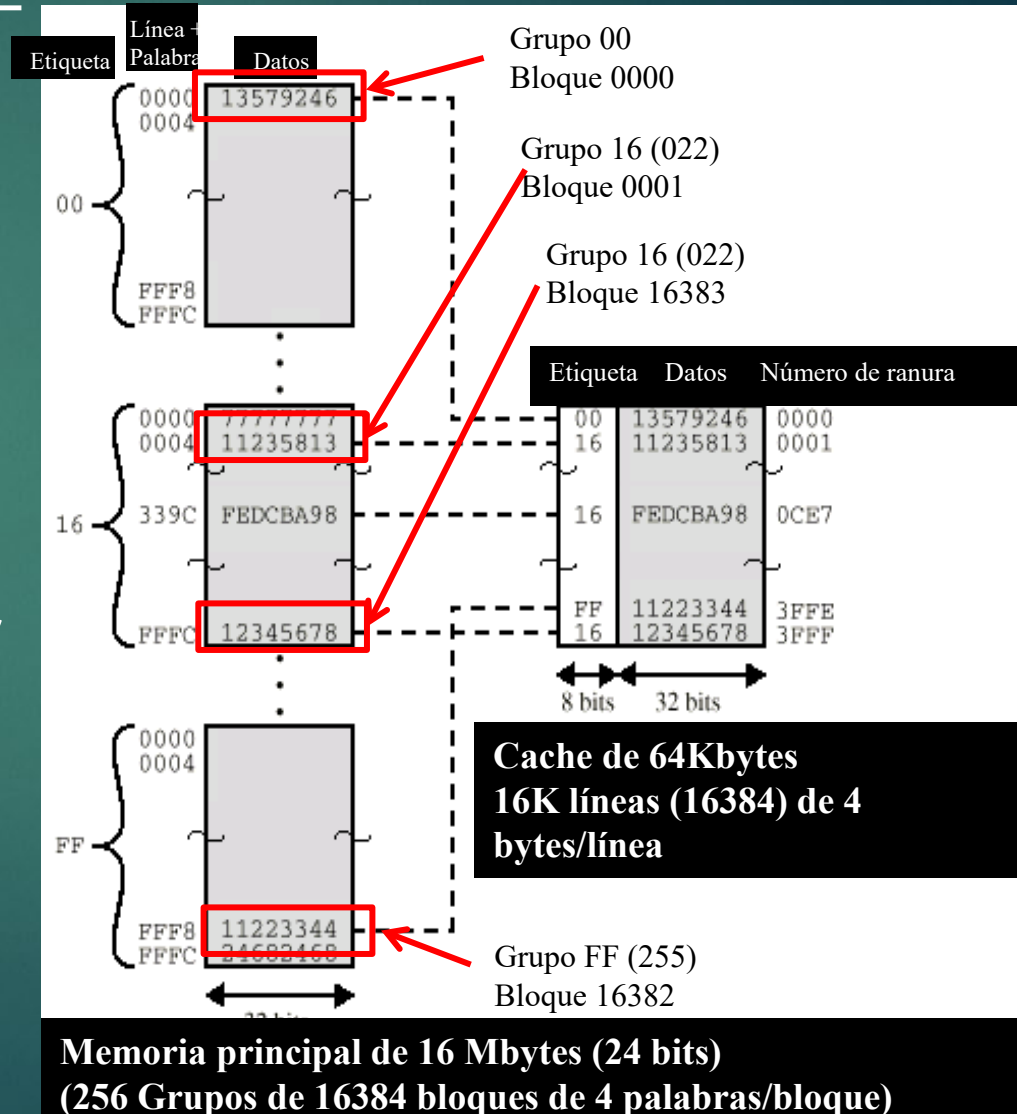
Políticas de asignación

Política de asignación Directa

En la imagen de al lado se muestra un ejemplo de asignación directa de una memoria de 16MB y caché de 64KB, con bloques de 4 bytes. De los 24 bits que forman la dirección, los 8 más significativos son el N° de grupo, los siguientes 14 el bloque dentro del grupo y los 2 restantes la palabra dentro del bloque.

Dirección a memoria

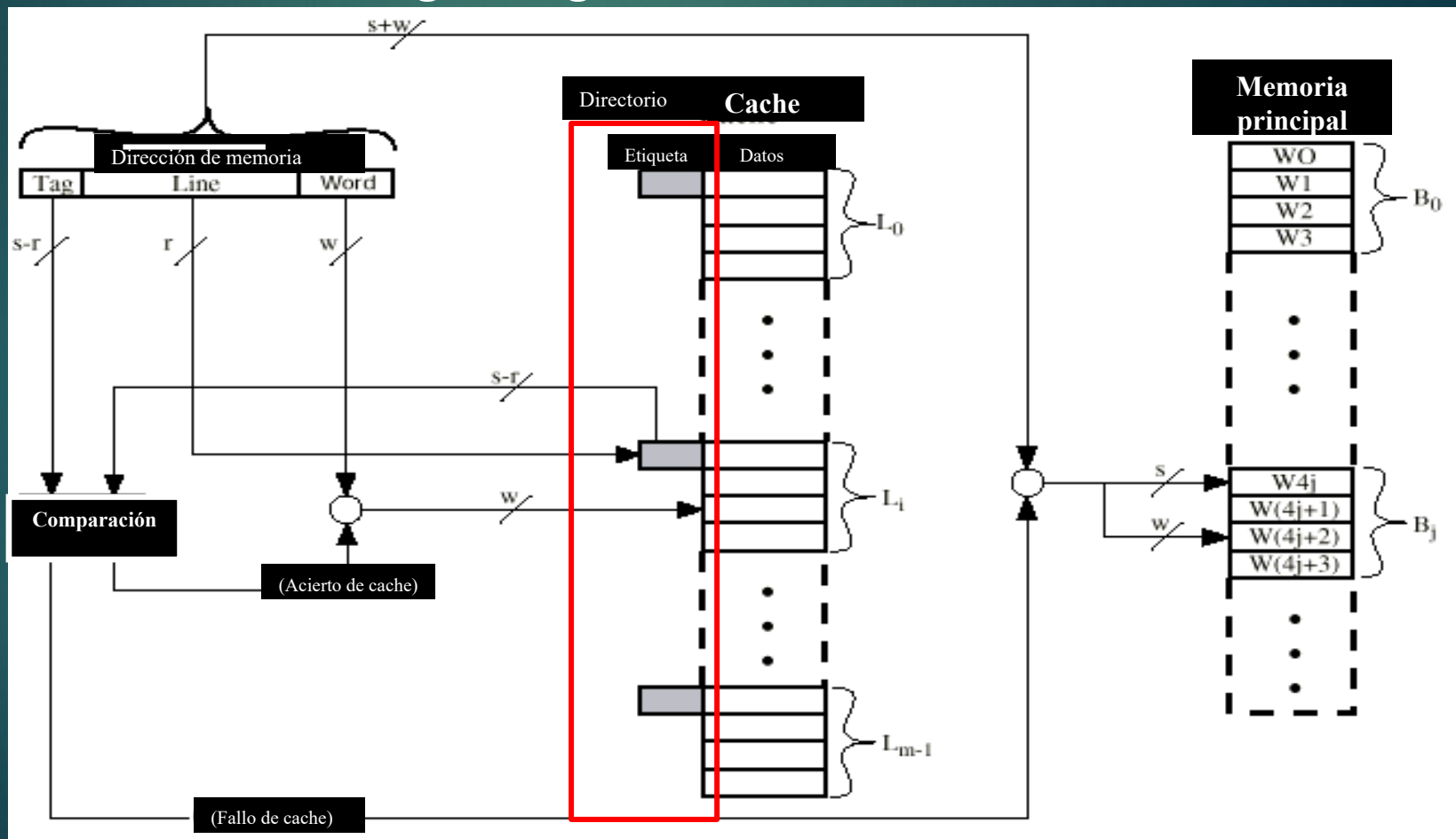
8	14	2
Grupo	Bloque o línea	Palabra



Políticas de asignación

Política de asignación Directa

En general, el esquema de operación en un esquema de asignación directa, se muestra en la figura siguiente.



Políticas de asignación

Política de asignación Directa

En la figura anterior, se puede apreciar que:

- **La dirección de memoria está compuesta de 3 campos:**
 - **Tag:** es el número de grupo en la memoria principal
 - **Line:** es el número de bloque dentro del grupo
 - **Word:** es el número de palabra dentro del bloque
- El tag es comparado con la etiqueta de la caché correspondiente al número de línea. Hay tantas líneas en la caché como bloques por grupo en la memoria principal. Las etiquetas de la memoria caché forman el “directorio” de la caché.
- Si la comparación da un acierto, el dato se busca en la caché. Si la comparación da una falla, el dato se trae de la memoria principal.

Políticas de asignación

Política de asignación Directa - Conclusiones

- Un bloque de memoria principal puede colocarse en una única línea de la cache.
- $N^{\circ} \text{ línea caché} = N^{\circ} \text{ bloque ref. "en módulo"} \ N^{\circ} \text{ líneas caché}$
- La etiqueta solo contiene el número de grupo de la memoria principal asignado a esa línea de la caché.
- Esta política de asignación es muy simple de implementar y poco costosa.
- Tiene un rendimiento aceptable, aunque a veces puede ser malo. Por ejemplo, si un programa accede a dos bloques que se corresponden a la misma línea (diferentes bloques de memoria principal) de forma repetida, las pérdidas de cache (desaciertos) serán muy grandes.

Políticas de asignación

Política de asignación Asociativa por conjuntos

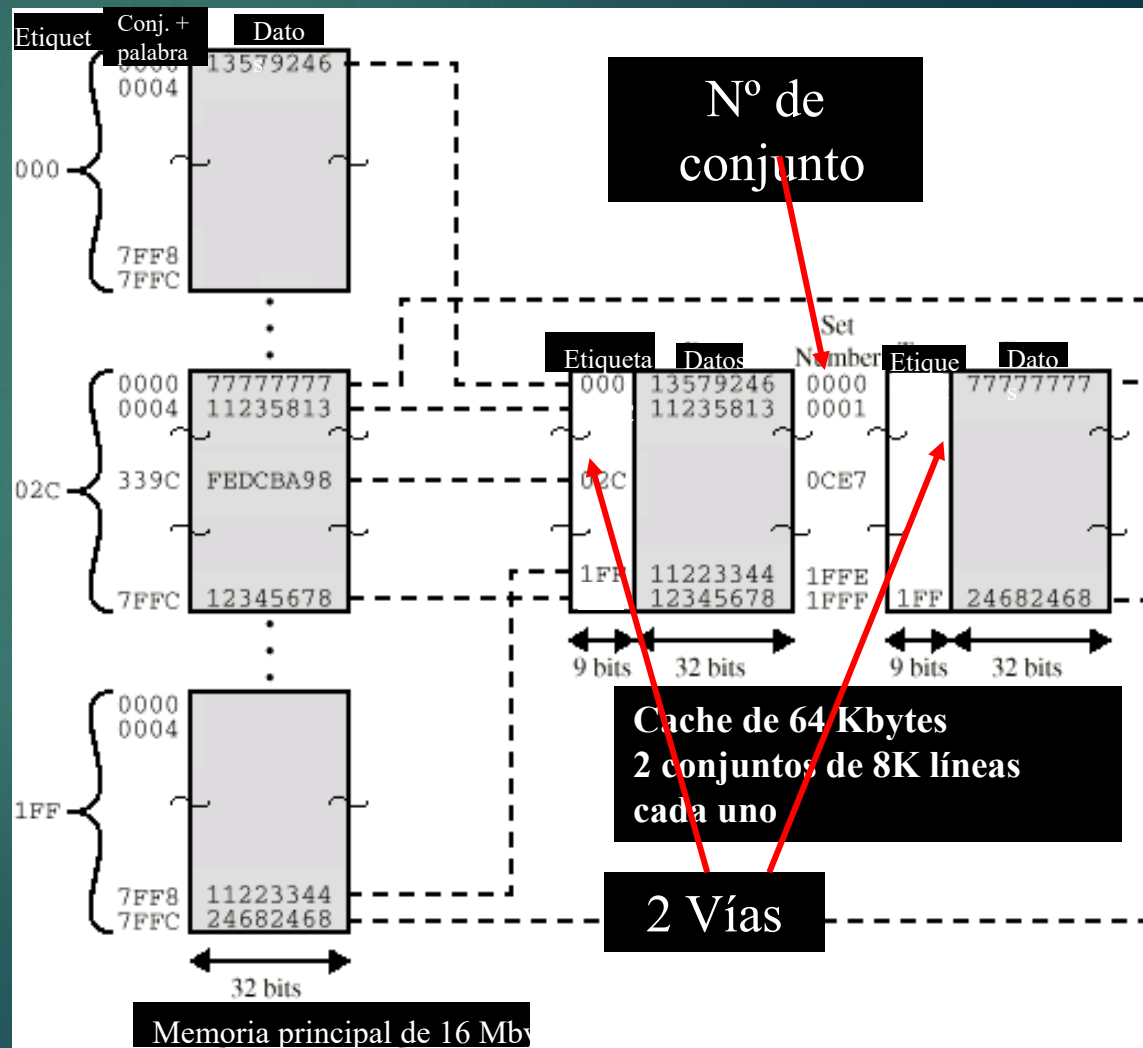
En la imagen de al lado se muestra un ejemplo de asignación asociativa por conjuntos de 2 vías de una memoria de 16MB y caché de 64KB, con bloques de 4 bytes. De los 24 bits que forman la dirección, los 9 más significativos son el N° de grupo, los siguientes 13 el conjunto dentro del grupo, y los 2 restantes la palabra dentro del bloque.

9	13	2
---	----	---

Grupo

Conjunto

Palabra



Diseño de la caché

Política de asignación Asociativa por conjuntos - Conclusiones

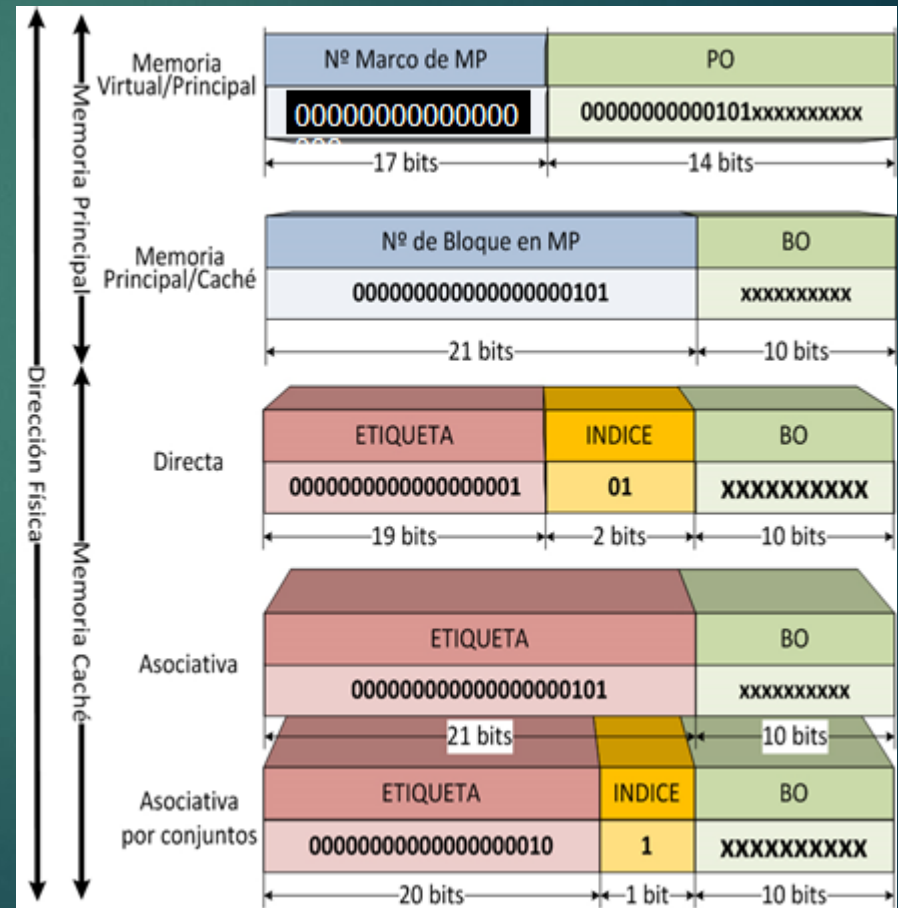
- Un bloque de memoria principal puede colocarse en bloques determinados de la cache. La cache se divide en un número de conjuntos N (N vías, con $N=2, 4, 8 \dots$ etc.)
- Cada conjunto contiene un número de líneas o ranuras.
- Un bloque determinado corresponderá a alguna línea o ranura de un conjunto determinado.
- En general, la función asociativa por conjuntos combina lo mejor de las otras correspondencias (asociativa y directa).

Políticas de asignación

Políticas de asignación – Comparación

El siguiente ejemplo compara los resultados de implementar las 3 políticas de asignación.

- MP de 2GB (31 bits)
- Bloques de 1KB (10 bits)
- MC de 4KB
- MC de 4 bloques
- Directa: 2 bits para N° de bloque/grupo + 19 bits N° grupo
- Asociativa: 21 bits N° de bloque
- Asociativa de 2 conjuntos: 1 bit N° de bloque/grupo + 20 bits N° grupo



Políticas de reemplazo

Políticas de reemplazo

- Cuando hay un fallo de acceso a la memoria cache, se debe traer un bloque desde la Memoria principal y almacenar un bloque existente en la memoria cache.
- El lugar a donde va a ser ubicado el bloque a traer desde la memoria principal requiere reemplazar un bloque existente.
- Las diferentes estrategias de reemplazo de los bloques se conocen como Políticas de reemplazo de bloque.

Políticas de reemplazo

57

Políticas de reemplazo - Correspondencia directa

- En la función de mapeado directo, la asignación de bloques de la memoria principal a bloques de la cache es fija, no hay elección.
- Sólo hay una posible línea para cada bloque.
- Por lo tanto si se requiere traer un nuevo bloque desde la memoria principal, indefectiblemente reemplazará el que está usando esa ranura actualmente.

Políticas de reemplazo

Políticas de reemplazo - Correspondencia asociativa y asociativa por conjuntos

- Para las asignaciones asociativa y asociativa por conjuntos, hay varios algoritmos de reemplazo:
 1. LRU
 2. FIFO
 3. LFU
 4. Aleatorio
- Todos los algoritmos deben implementarse en hardware (por razones de velocidad).

Políticas de reemplazo

1.- Menos usado recientemente (LRU)

- Se reemplaza el bloque que lleva más tiempo sin utilizarse.
- Requiere controles de tiempos.
- Aprovecha la localidad temporal.
- Válido en Asociativas por conjunto, donde por cada ranura se agrega un bit de USE para identificar el usado recientemente.

2.- Primero en entrar - primero en salir (FIFO).

- Se reemplaza el bloque que entró antes en la caché.
- Requiere controles de acceso para identificar el orden en que ingresaron.

Políticas de reemplazo

3.- Menos frecuentemente usado (LFU)

- Se sustituye aquella línea que ha experimentado menos referencias.
- Requiere controles de uso.

4.- Aleatoria

- Se sustituye una línea al azar.

Políticas de escritura

Políticas de escritura

- Cuando la CPU tiene que almacenar (“escribir”) un resultado en la memoria puede hacerlo tanto en un acierto (la dirección donde se va a guardar el dato está en un bloque de la cache) como en un fallo (la dirección donde se va a guardar el dato NO está en un bloque de la cache).
- En cualquiera de las 2 situaciones, se debe evitar inconsistencia de información entre las memorias principal y cache, durante los procesos de escrituras. Es decir que, aún escribiendose el dato en la cache, el correspondiente bloque de la memoria principal debe ser actualizado en algún momento.
- Además, a veces un módulo E/S puede tener acceso directo a la memoria principal y requerir información que fué modificada en la cache. En arquitecturas complejas (procesadores paralelo) múltiples CPU pueden tener caches individuales.

Políticas de escritura

Políticas de escritura

- Las políticas de escritura son distintas en aciertos que en fallos.
- Cuando la dirección de memoria a donde tiene que escribir un dato está en la memoria caché, se usan las políticas de escritura en acierto.
- Cuando la dirección de memoria a donde tiene que escribir un dato no está en la memoria caché, se usan las políticas de escritura en fallo.

Políticas de escritura

Políticas de escritura en aciertos

Hay 2 estrategias:

- **Write-through (Escritura inmediata)**
 - Se actualizan simultáneamente la posición de la caché y de la memoria principal.
 - Aumenta el tráfico con la memoria principal.
 - Pueden haber retrasos durante múltiples escrituras.
- **Write-back (Post-escritura).**
 - La información sólo se actualiza en la caché y se escribe la memoria cuando se reemplaza el bloque. El bloque requiere de un bit de “sucio” para indicar cuando se lo escribió.
 - Como la memoria principal se actualiza en el reemplazo, puede contener información errónea en algún momento.

Políticas de escritura

Políticas de escritura en fallos

Hay 2 estrategias:

➤ No-write allocate

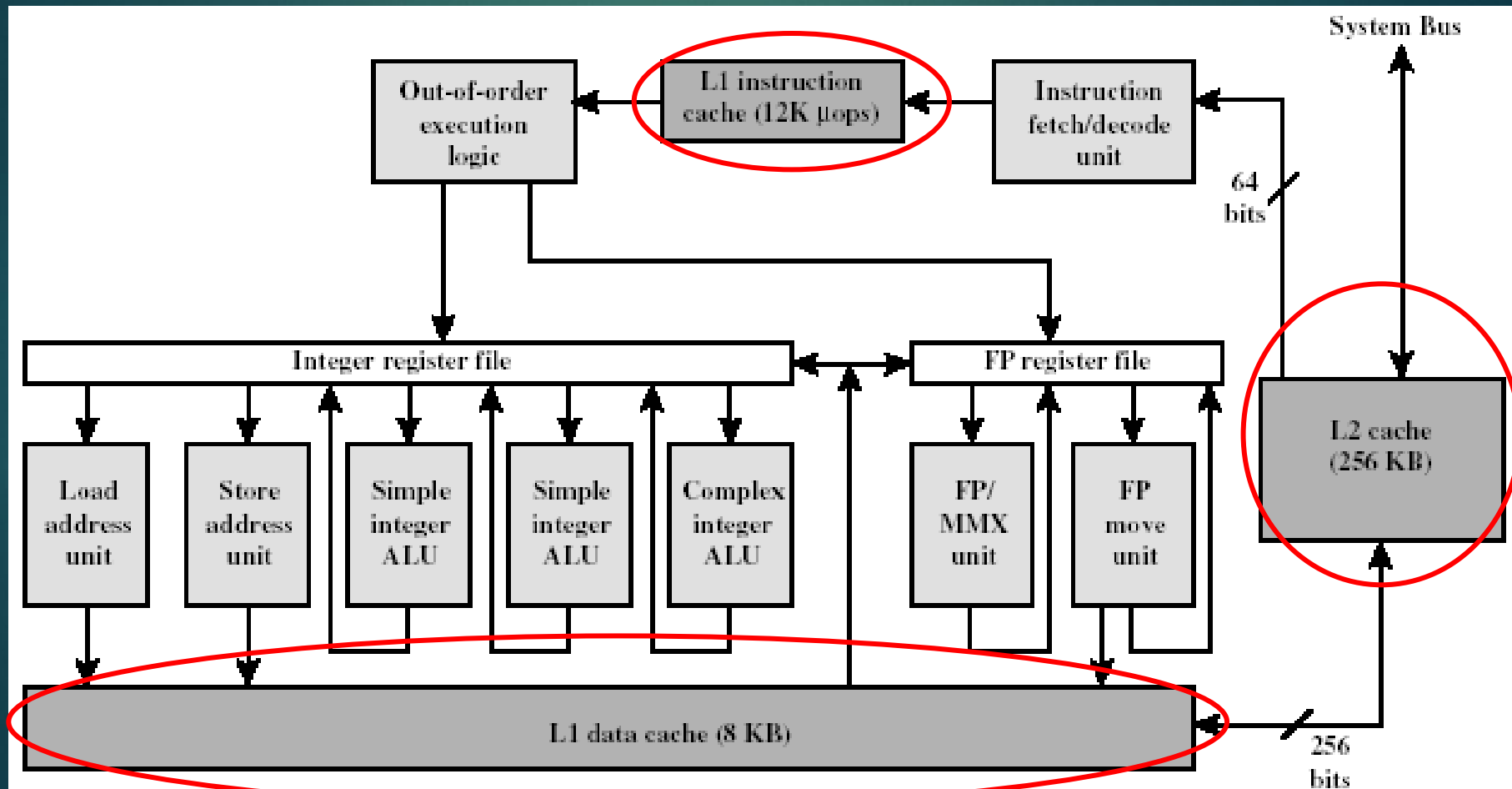
- Se escribe directamente en la memoria principal. La caché se usa solo en las lecturas. El bloque no se lleva a la memoria caché mientras no se lo tenga que leer.
- Habitual con write-through.

➤ Write allocate

- El bloque requerido primero se copia en la cache, y luego se escribe (en la cache).
- Habitual con write-back.

Caché de Pentium 4

Ejemplo práctico de memoria caché - Pentium 4



Caché de Pentium 4

Caché de Pentium 4

- 2 niveles de caché: L1 y L2
- Ambas caché integradas en el chip del procesador.
- Ancho de banda de las transferencias: 48 GBytes/s.
- La arquitectura admite un tercer nivel de caché (L3) en el mismo chip (para aplicaciones en servidores).

Caché de Pentium 4

Caché de Pentium 4

- Cache L1: integrada por 2 cache, una para datos y otra para instrucciones
 - Cache de datos
 - Tamaño: 8 KB
 - 128 Bloques de 64 bytes
 - Organización: Asociativa por conjuntos de 4 vías.
 - Política de Escritura: Write-through (inmediata)
 - Velocidad: acceso a los datos enteros en dos ciclos de reloj.
 - Caché de instrucciones:
 - Tamaño: 12 kB
 - Almacena segmentos de caminos de ejecución de instrucciones decodificadas (trazas).

Caché de Pentium 4

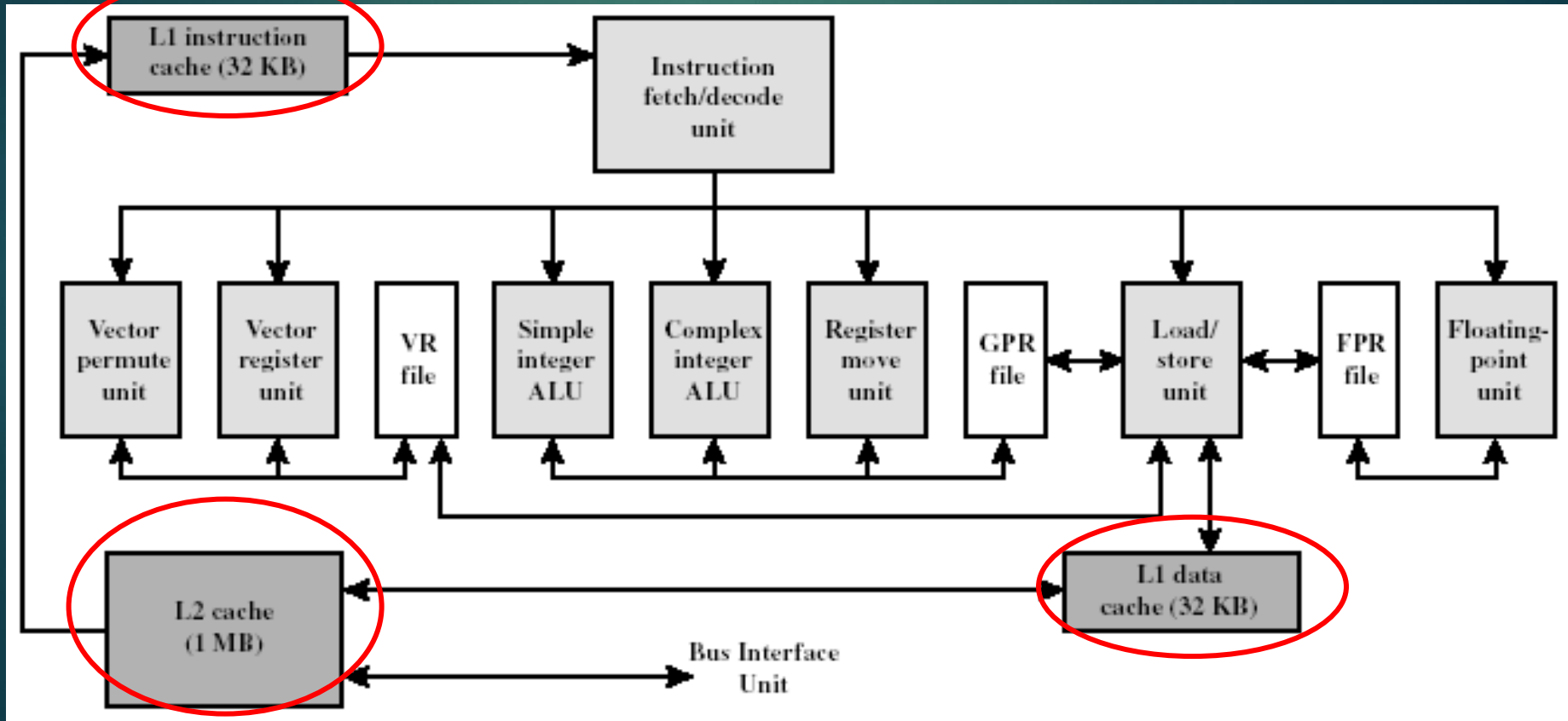
Caché de Pentium 4

- Cache L2 (interna) unificada para datos e instrucciones
 - Capacidad: 256 KBytes.
 - Bloques de 128 bytes.
 - Función de mapeo: Asociativa por conjuntos de 8 vías.
 - Política de escritura: Post-escritura.
 - Latencia de acceso de 7 ciclos de reloj.

Power PC G3

Ejemplo práctico de memoria caché - Power PC G3

El Power PC tiene también 2 niveles de caché L1 y L2 (1MB). La L1 dividida en cache de instrucciones (32Kb) y de datos (32Kb).



Referencias

- Organización y Arquitectura de Computadoras, William Stallings, Capítulo 4, 5ta ed.
- Diseño y evaluación de arquitecturas de computadoras, M. Beltrán y A. Guzmán, Capítulo 2 Apartados 2.1 a 2.4, 1er ed.

Arquitectura de Computadoras

Anexo Clase 7

Buses del Sistema

Estructuras de interconexión

- Todas las unidades han de estar interconectadas.
- Existen distintos tipos de interconexiones para los distintos tipos de unidades:
 - Memoria
 - Módulo de E/S
 - Procesador

Interconexión de la memoria

- Recibe y entrega datos.
- Recibe direcciones (ubicación de trabajo).
- Recibe señales de control
 - Leer
 - Escribir
 - Temporizar

Interconexión del módulo E/S

- E/S es funcionalmente similar a la memoria
 - Recibe y entrega datos del/al procesador
 - Envía y recibe datos al/del periférico
 - Recibe direcciones (ubicación del periférico)
 - Recibe señales de control del procesador
 - Envía señales de control al periférico
 - Envía señales de control al procesador
 - Interrupción

Interconexión del procesador

- Lee instrucciones y datos.
- Escribe datos (los procesados).
- Envía señales de control a otras unidades.
- Recibe (y utiliza) señales de interrupción.

Buses

- Existe una serie de sistemas de interconexión.
- Las estructuras sencillas y múltiples son las más comunes.
 - Ejemplo: control/dirección/bus de datos (PC)
 - Ejemplo: unibus (DEC-PDP)

¿Qué es un bus?

- Es un camino de comunicación entre dos o más dispositivos.
- Normalmente, medio de transmisión.
- Suele agruparse:
 - Varios caminos de comunicación o líneas con función común.
 - un dato de 8 bits puede transmitirse mediante ocho líneas del bus.

Bus de datos

- Transmite datos.
 - Recuerde que a este nivel no existe diferencia alguna entre “datos” e “instrucciones”.
- El ancho del bus es un factor clave a la hora de determinar las prestaciones.
 - 8, 16, 32, 64 bits.

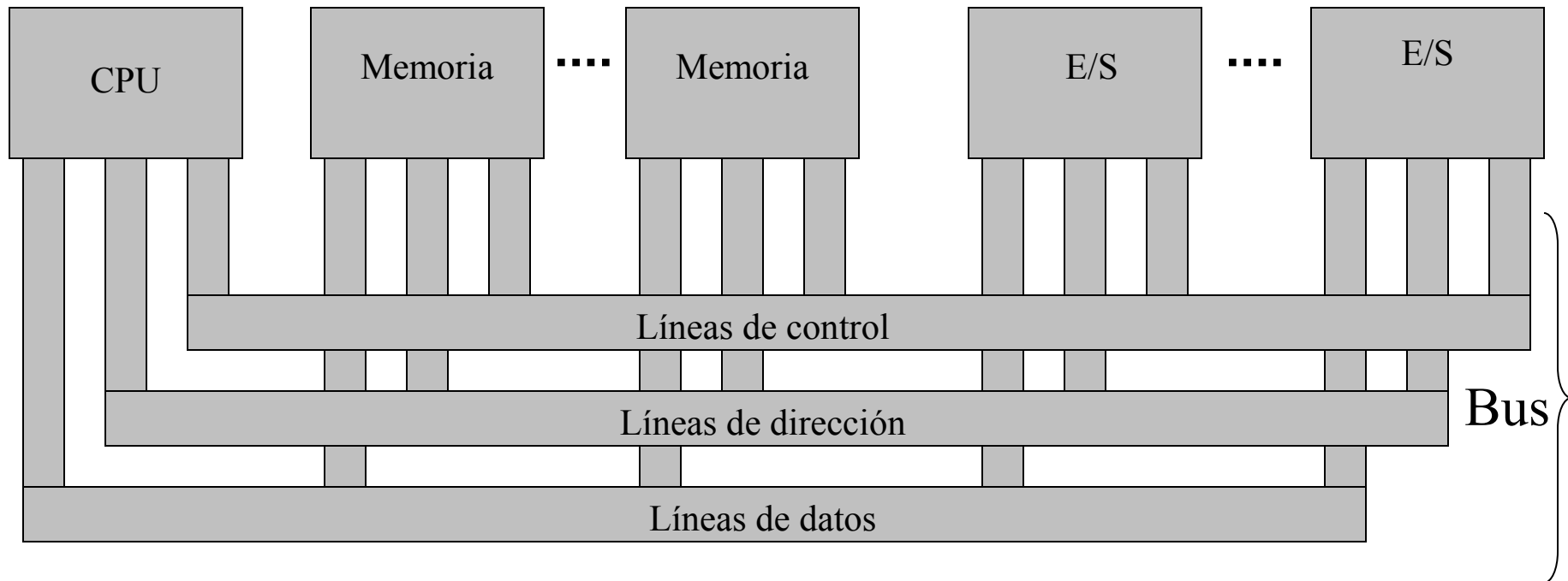
Bus de dirección

- Identifica la fuente o destino de un 'dato'.
 - cuando el procesador desea leer una palabra de una determinada parte en la memoria.
- El ancho del bus de direcciones determina la máxima capacidad de memoria posible en el sistema.
 - MSX88 tiene un bus de dirección de 16 bits, lo que define un espacio para direcciones de 64K lugares

Bus de control

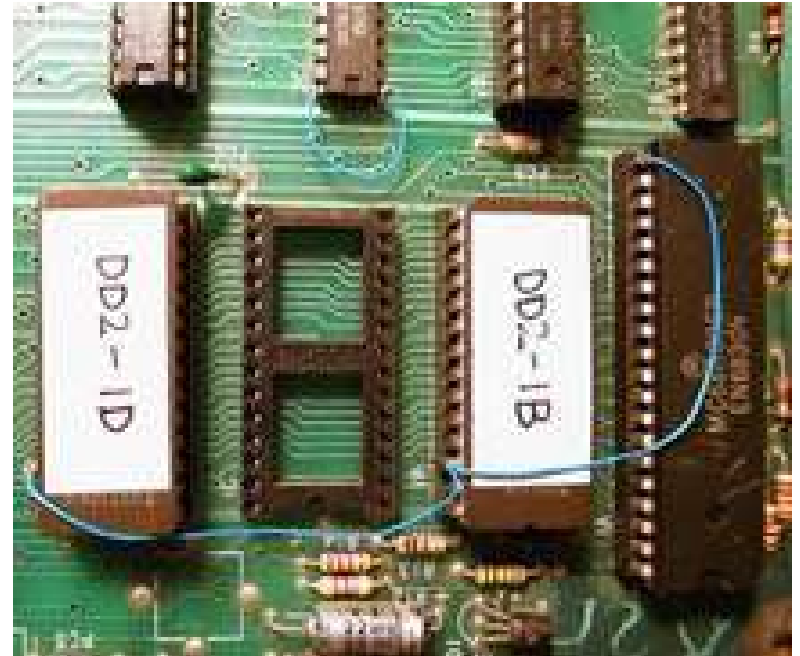
- Transmite información de señales de control y temporización
 - Señal de escritura/lectura en memoria.
 - Petición de interrupción.
 - Señales de reloj.

Interconexión mediante un bus



Cómo son ???

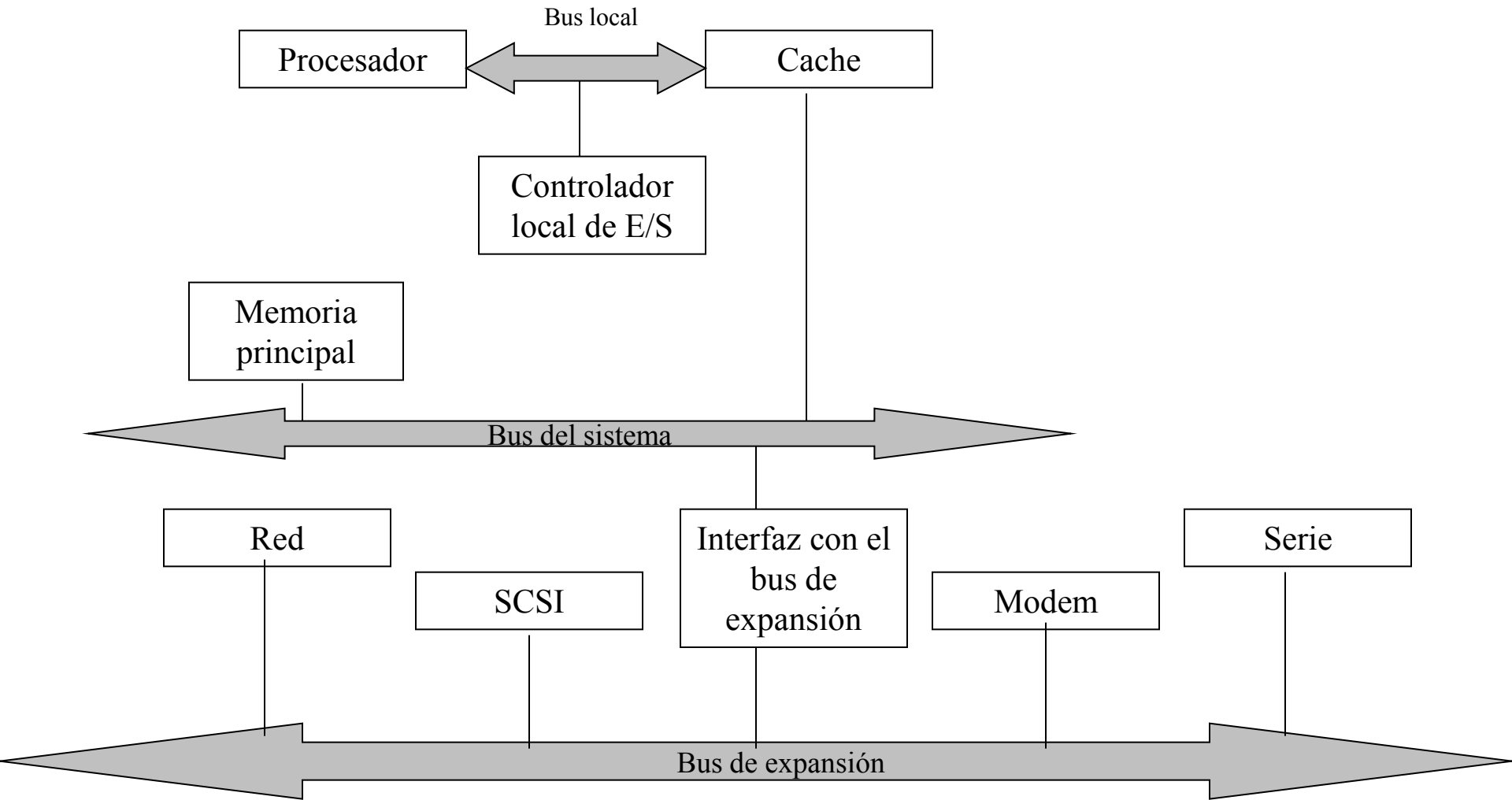
- Es un conjunto de conductores eléctricos paralelos.
- Líneas de metal.
- Poseen conectores para colocar 'tarjetas'

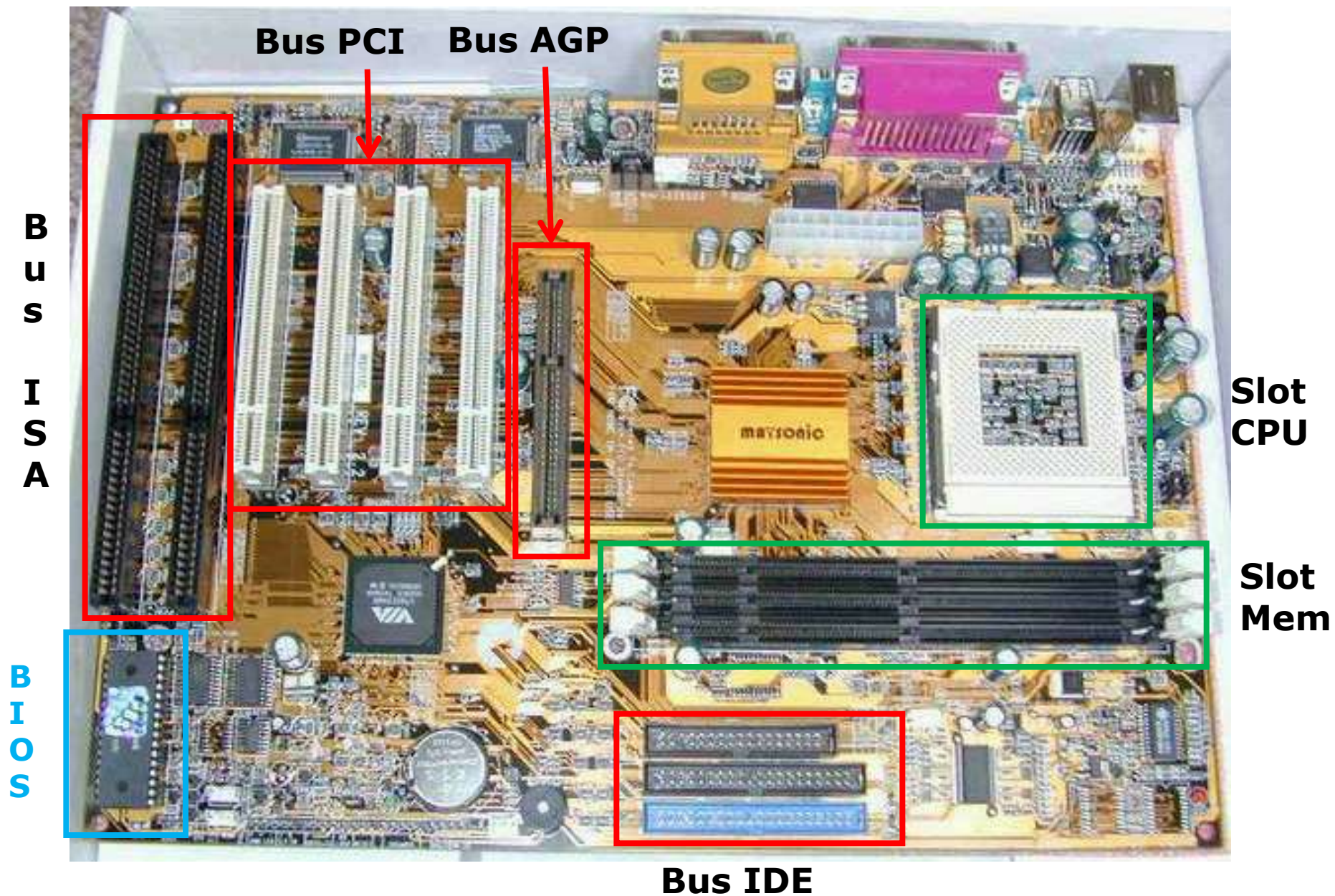


Problemas de un único bus

- Conectar gran número de dispositivos a un bus producen Retardos de propagación
 - Si el control del bus pasa de un dispositivo a otro, puede afectar sensiblemente a las prestaciones.
- La mayoría de los sistemas utilizan varios buses para solucionar estos problemas.
 - Jerarquía de buses

Arquitectura de bus tradicional





Tipos de buses

➤ Dedicados

- Uso de líneas separadas para direcciones y para datos.
 - 16 líneas de direcciones
 - 16 líneas de datos
 - 1 línea de control de lectura ó escritura (r/w)

➤ Multiplexados

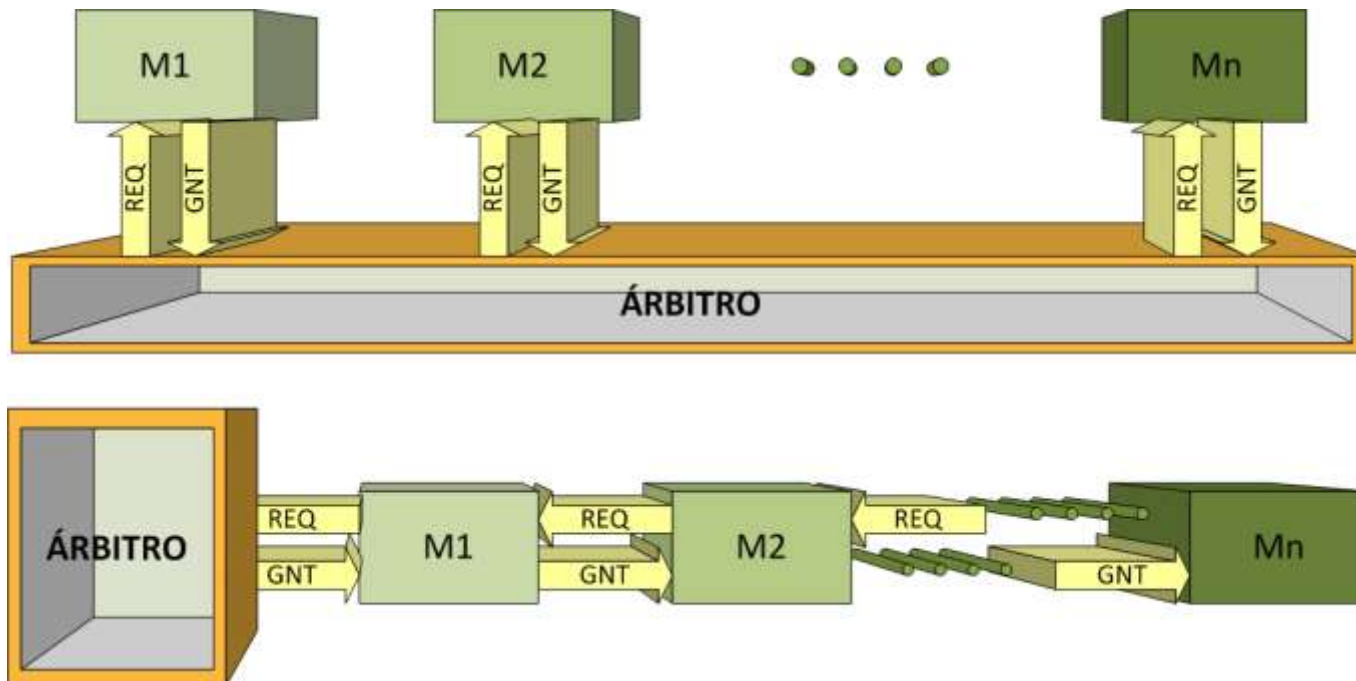
- Uso de las mismas líneas.
 - 16 líneas de direcciones ó datos
 - 1 línea de control de lectura ó escritura (r/w)
 - 1 línea de control para definir direcciones ó datos (a/d)
- Menos líneas pero mas circuitería. ¿Prestaciones?

Arbitraje del bus

- El control del bus puede necesitar más de un módulo.
 - Ejemplo: CPU y el controlador DMA
- Sólo una unidad puede transmitir a través del bus en un instante dado.
- Los métodos de arbitraje se pueden clasificar como centralizados o distribuidos.

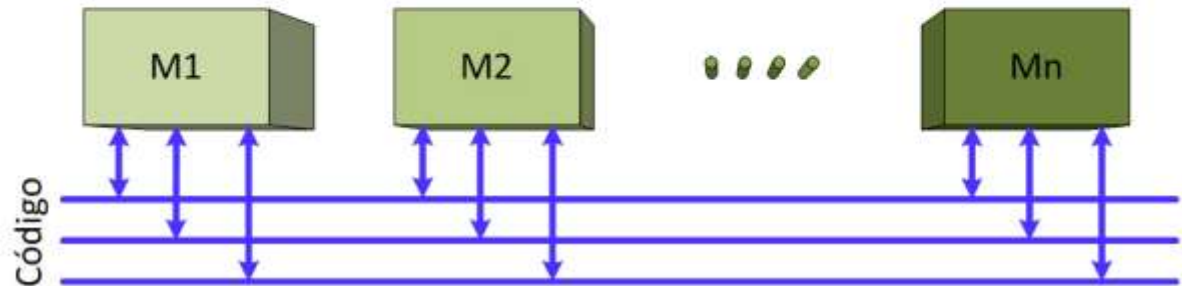
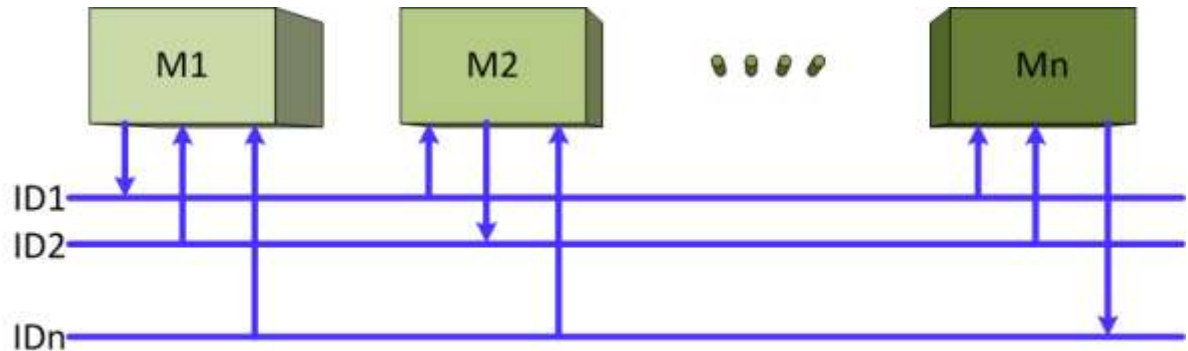
Arbitraje centralizado

- Un único dispositivo hardware es responsable de asignar tiempos en el bus: **Controlador del bus ó Árbitro**
 - Puede estar en un módulo separado o ser parte del procesador.



Arbitraje distribuido

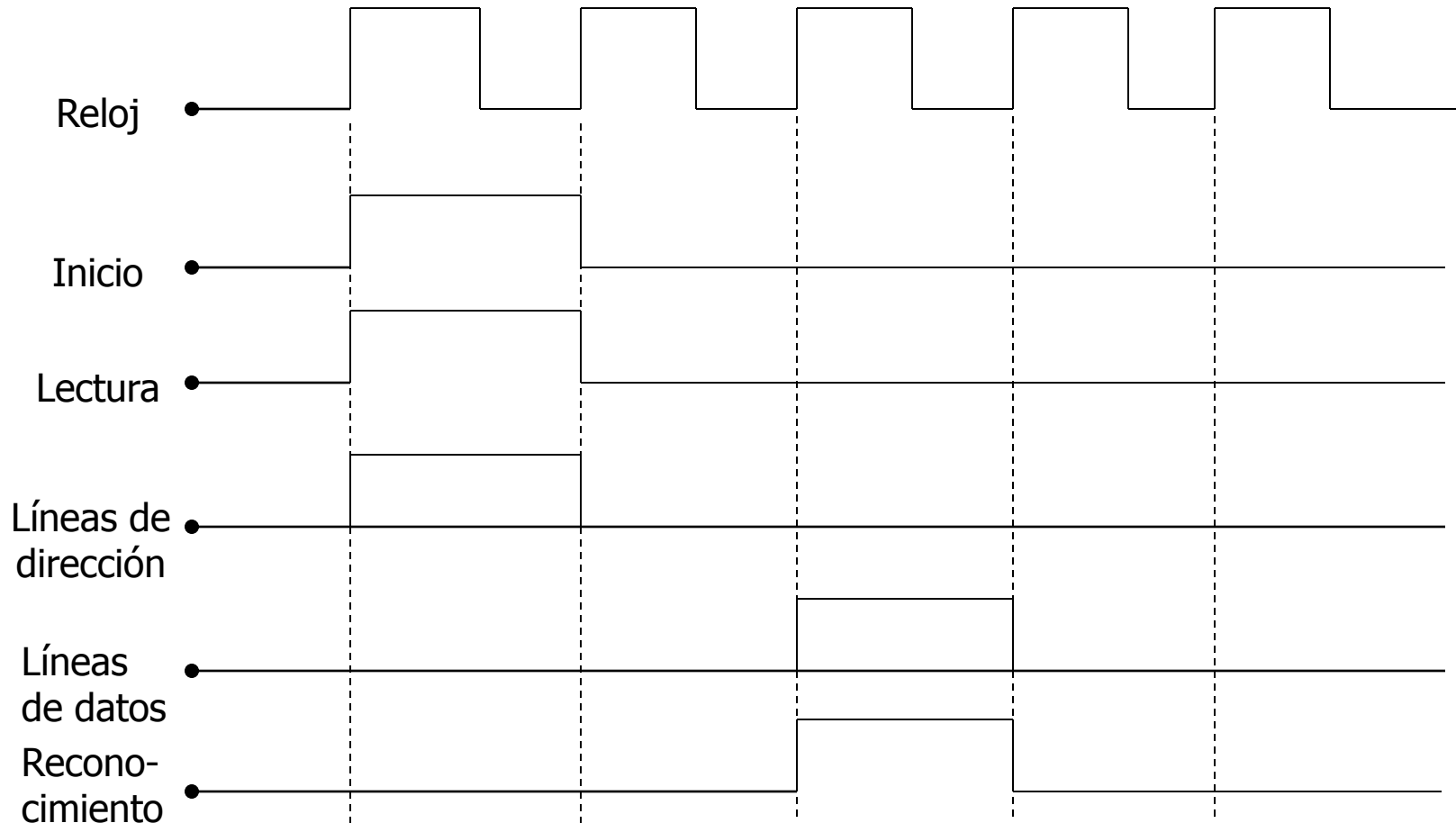
- Cada módulo puede controlar el acceso al bus.
- Cada módulo dispone de lógica para controlar el acceso.



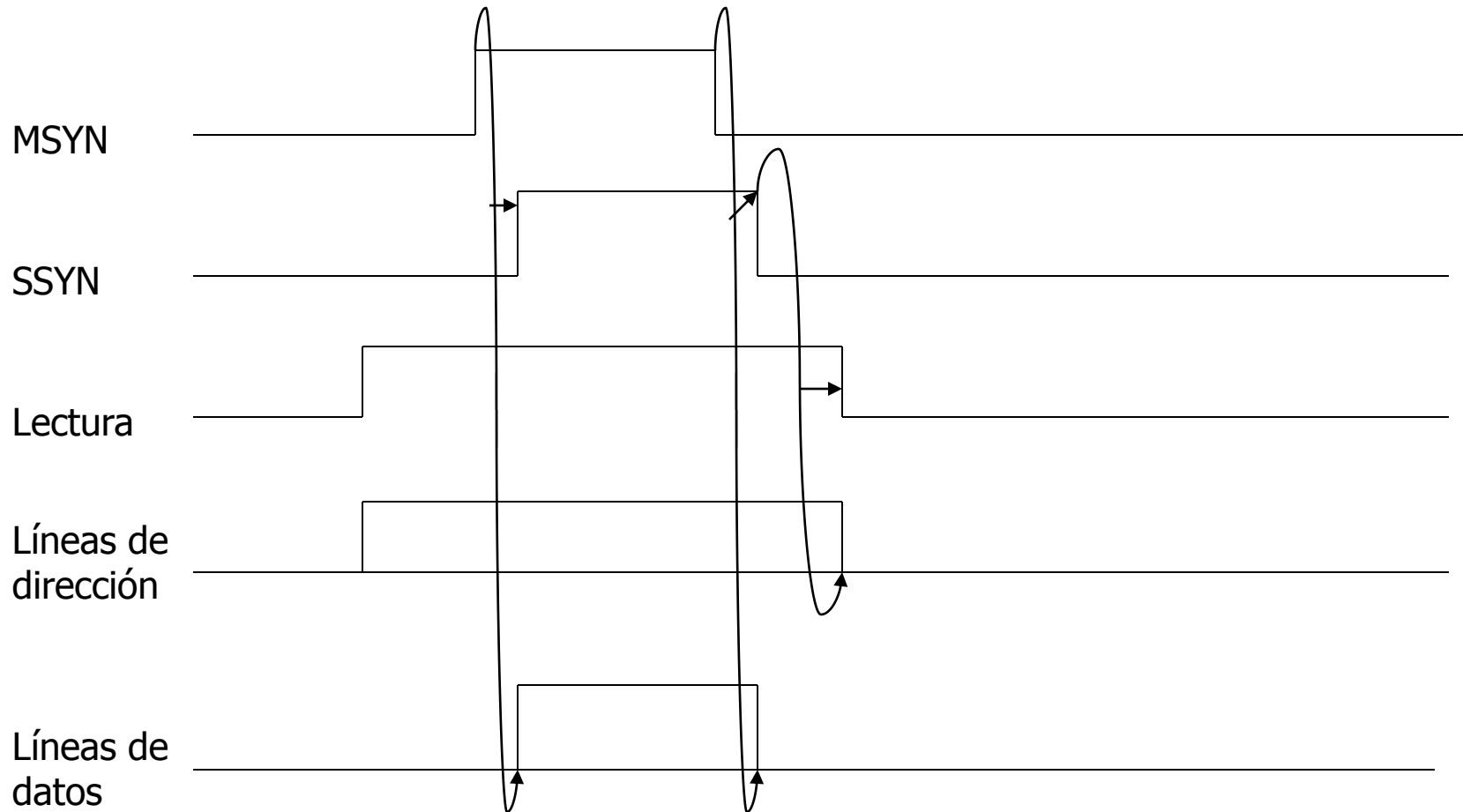
Temporización

- Forma de coordinar los eventos en el bus.
- Temporización síncrona
 - La presencia de un evento está determinada por un reloj.
 - El bus incluye una línea de reloj.
 - Un intervalo desde un “uno” seguido de otro a “cero” se conoce como ciclo de bus.
 - Todos los dispositivos del bus pueden leer la línea de reloj.
 - Suele sincronizar en el flanco de subida.
 - La mayoría de los eventos se prolongan durante un único ciclo de reloj.

Temporización síncrona

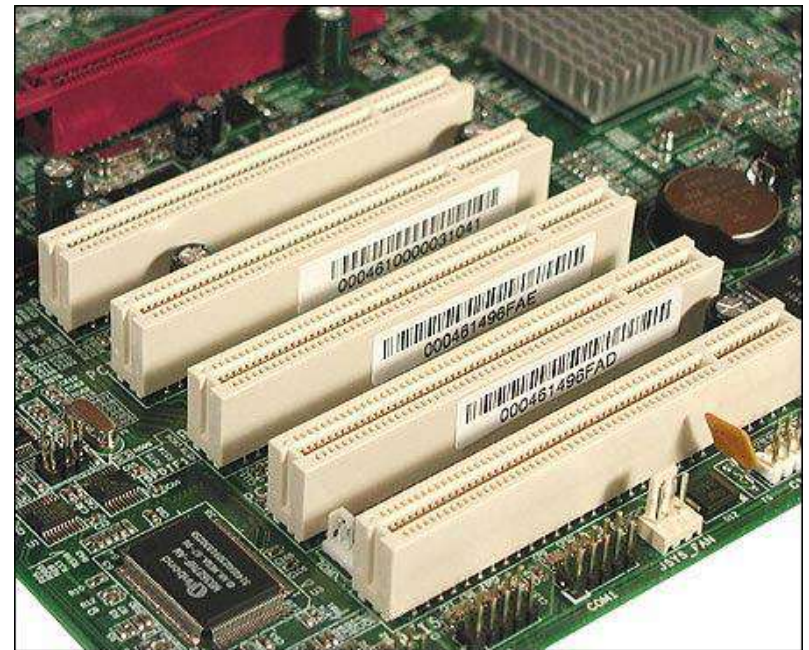


Temporización asíncrona



Bus PCI

- Interconexión de Componente Periférico.
- Intel cedió sus patentes al dominio público.
- 32 o 64 bits.
 - 32 bit a 33MHz = 133 MB/s
 - 64 bit a 66MHz = 528 MB/s
- Comandos
 - Transacción maestro - esclavo.
 - Maestro toma control del bus.
 - Determina tipo de transacción.
 - lectura ó escritura
 - Fase de direccionamiento.
 - Una o más fases de datos.



Líneas de señal PCI

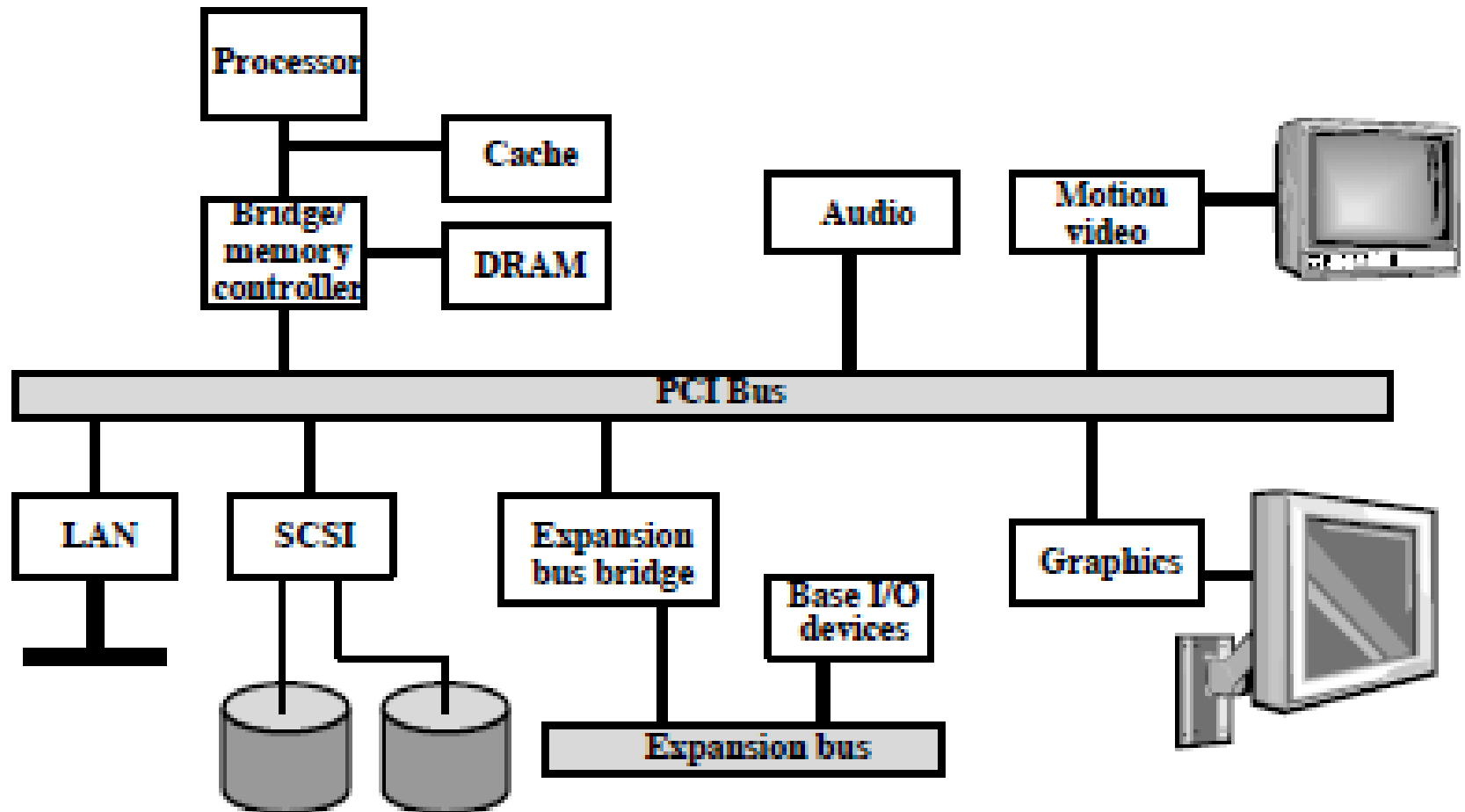
➤ 49 líneas obligatorias

- **Líneas del sistema**
 - Incluyen reloj y reset.
- **Terminales de direcciones y datos**
 - 32 líneas multiplexadas para direcciones y datos.
 - Líneas para interpretar y validar eventos.
- **Terminales de control de la interfaz**
 - Temporización y Coordinación
- **Terminales de arbitraje**
 - Líneas no compartidas.
 - Conexión directa al árbitro del bus PCI.
- **Terminales para señales de error**

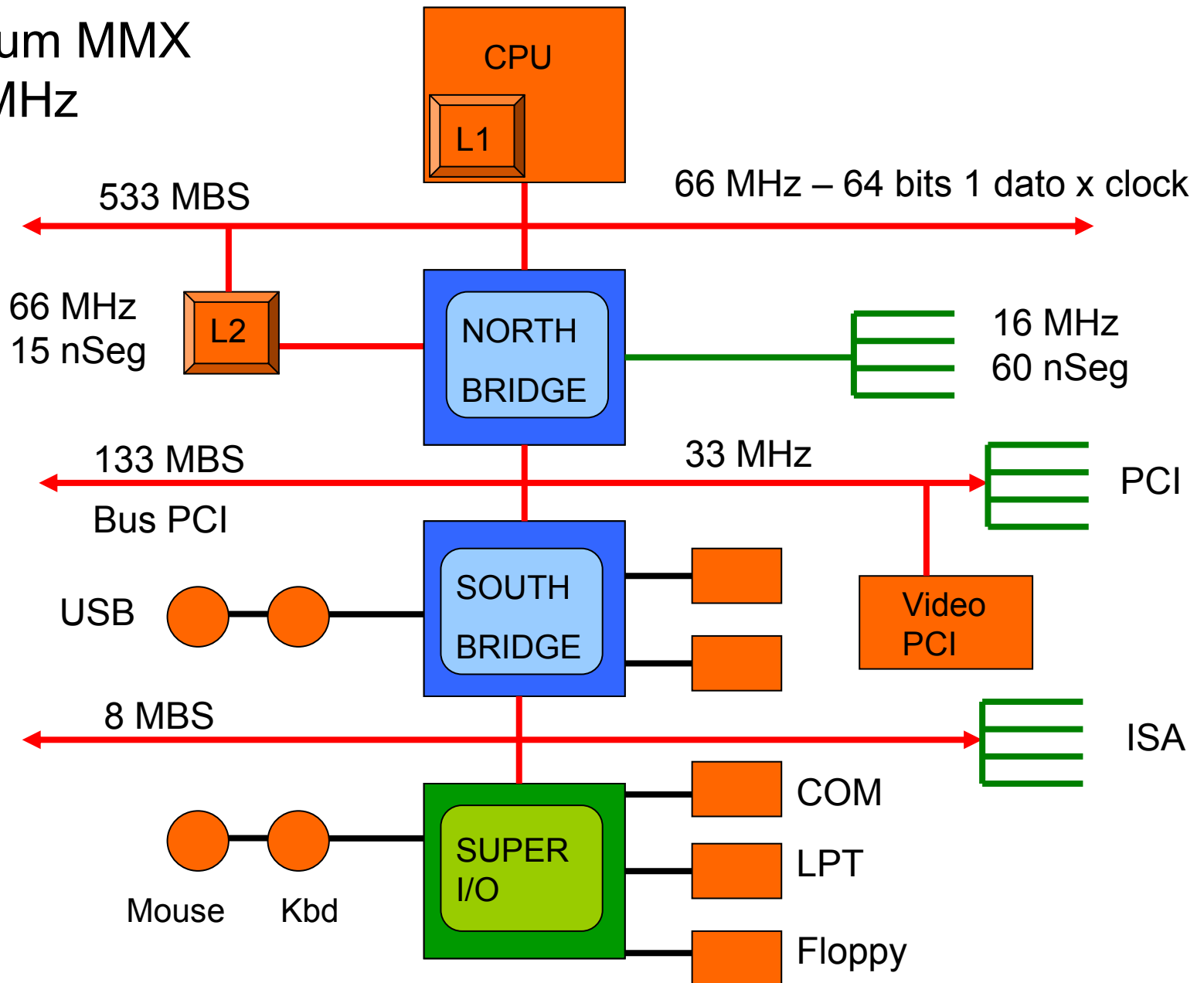
➤ 51 líneas opcionales

- **Extensión a 64 bits**
 - 32 líneas adicionales.
 - Líneas multiplexadas.
 - 2 líneas para transferir a 64 bits.

Sistema con Bus PCI típico



Pentium MMX 266 MHz



Pentium MMX 266 MHz

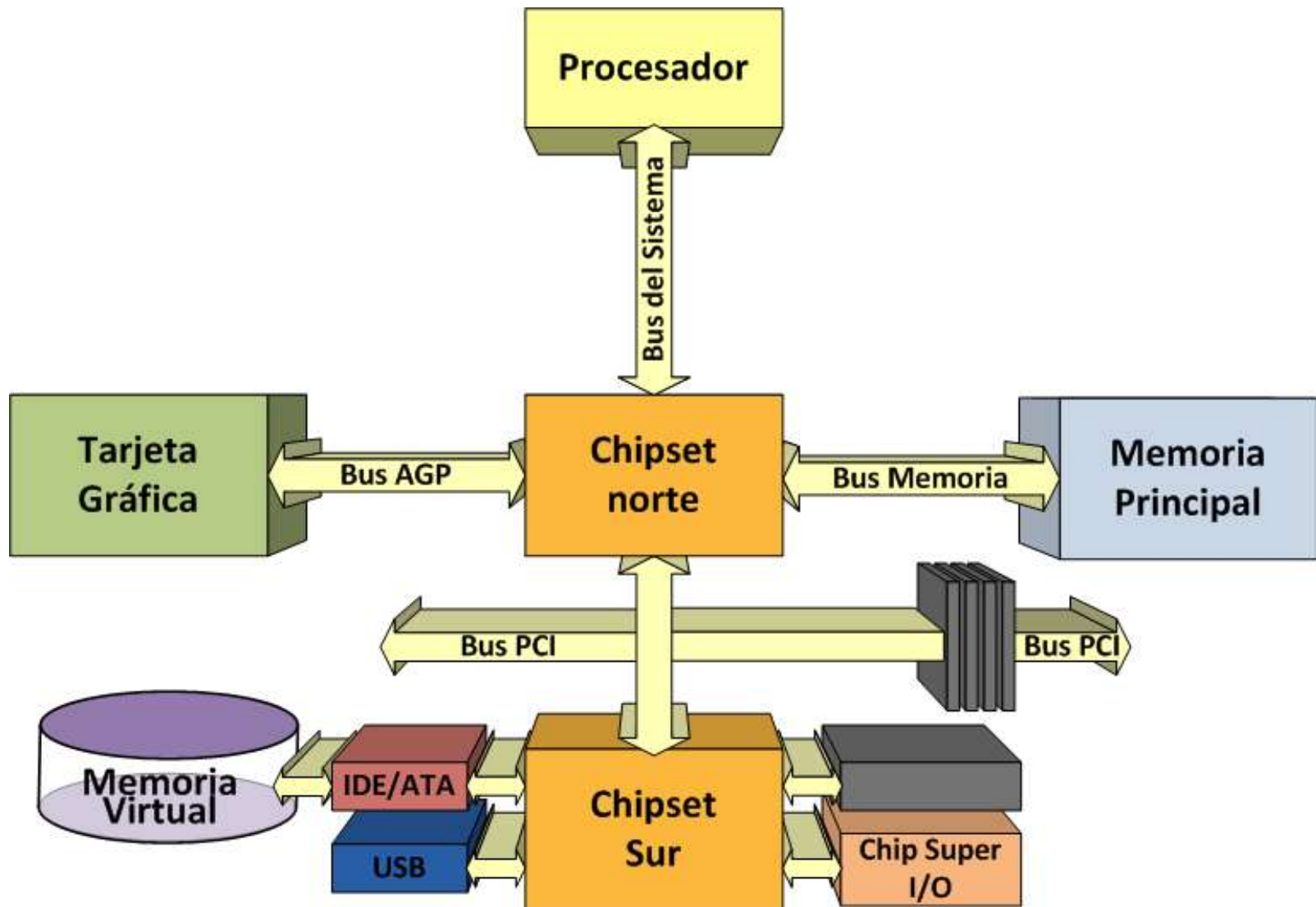
✓ FSB = Front Side Bus

66,66 MHz x 64 bits x 1 dato.clock =
533 MBytes/seg ←

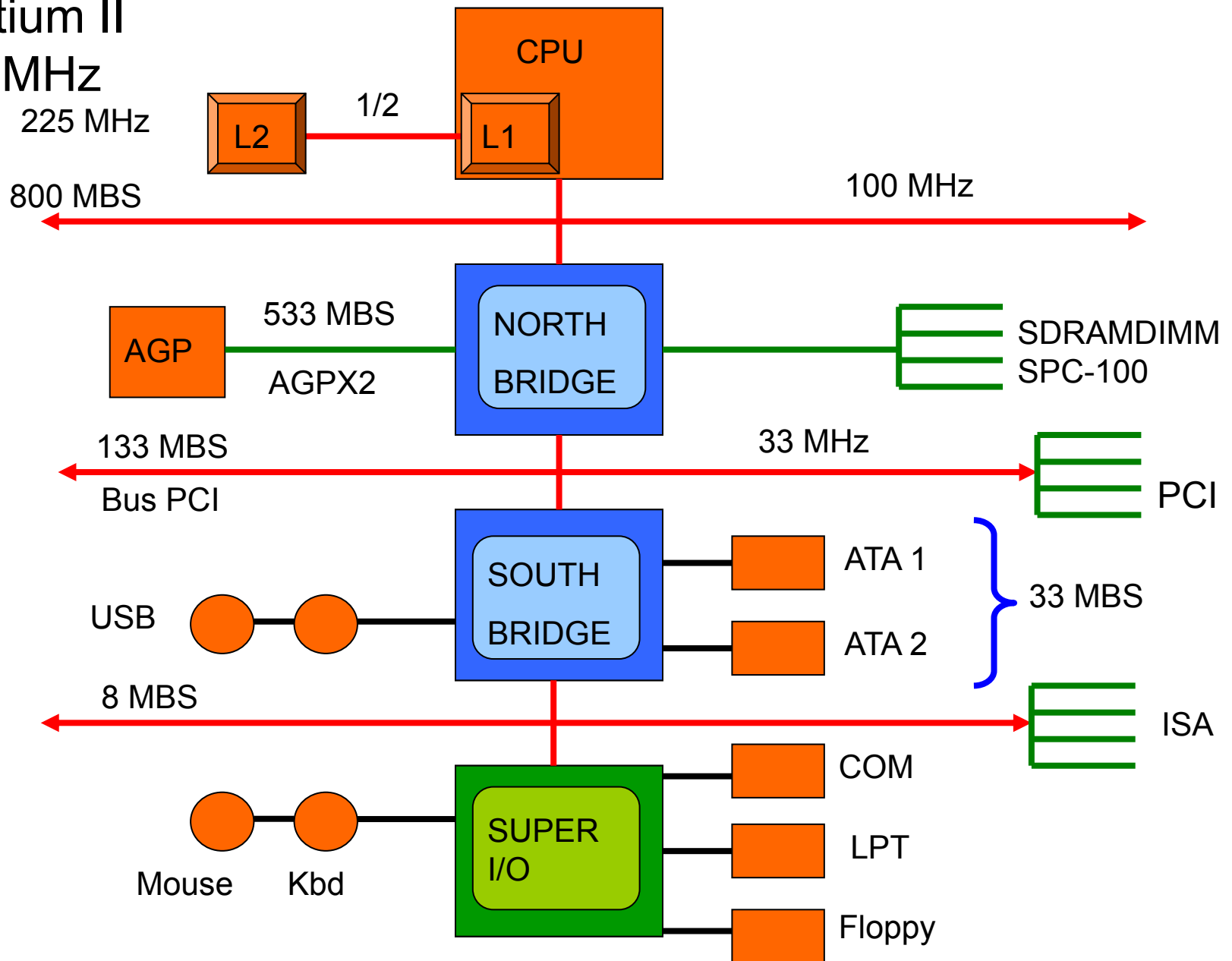
✓ Bus PCI

33,33 MHz x 32 bits x 1 dato.clock =
133 MBytes/seg ←

Evolución de jerarquía de bus (1)



Pentium II 450 MHz



Pentium II 450 MHz

- ✓ FSB = Front Side Bus

$100 \text{ MHz} \times 64 \text{ bits} \times 1 \text{ dato.clock} = 800 \text{ MBytes/seg}$

- ✓ Bus AGP

$66,66 \text{ MHz} \times 32 \text{ bits} \times 2 \text{ datos.clock} = 533 \text{ MBytes/seg}$

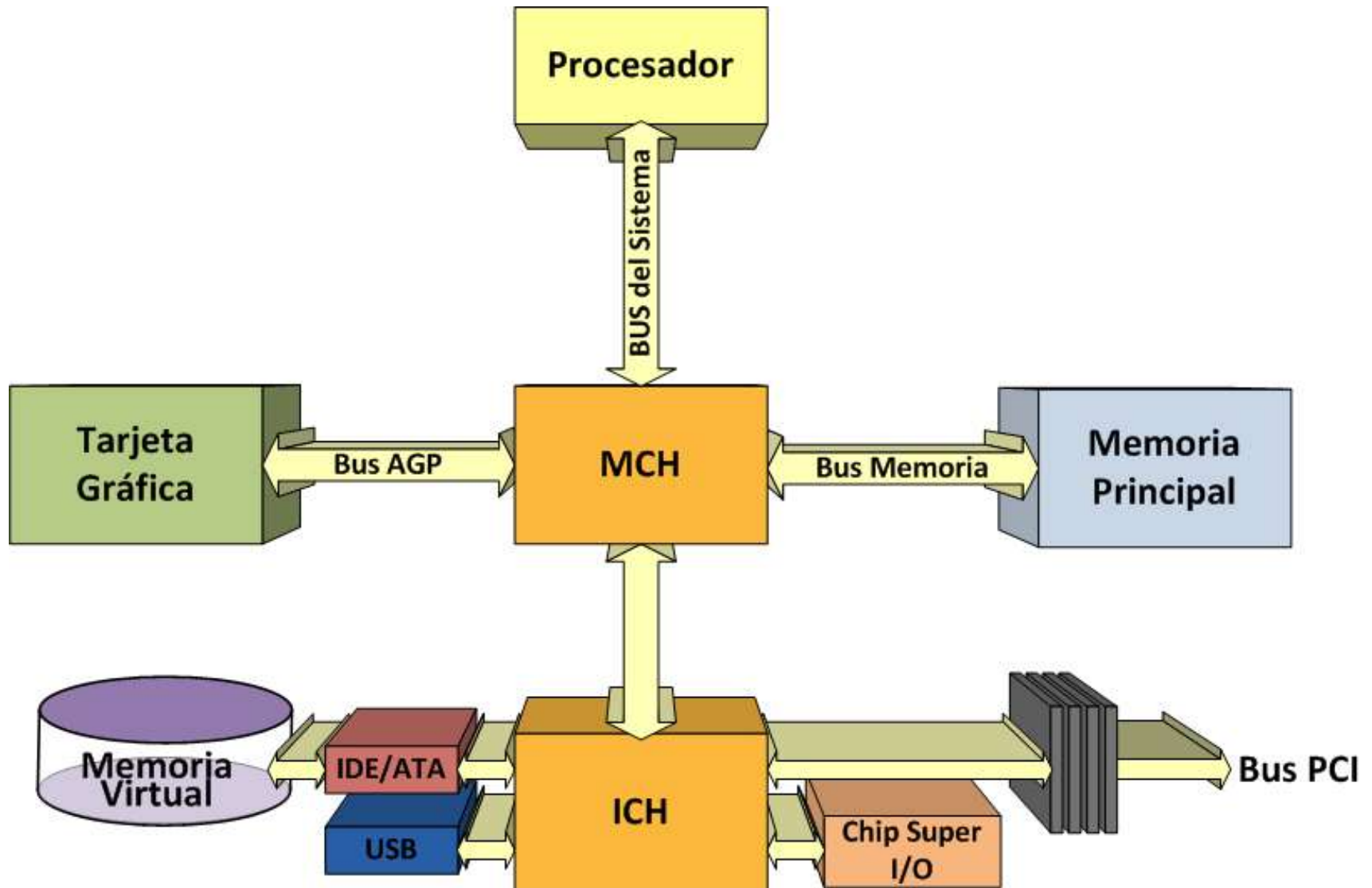
- ✓ ATA-UDMA

$8,33 \text{ MHz} \times 16 \text{ bits} \times 2 \text{ datos.clock} = 33 \text{ MBytes/seg}$

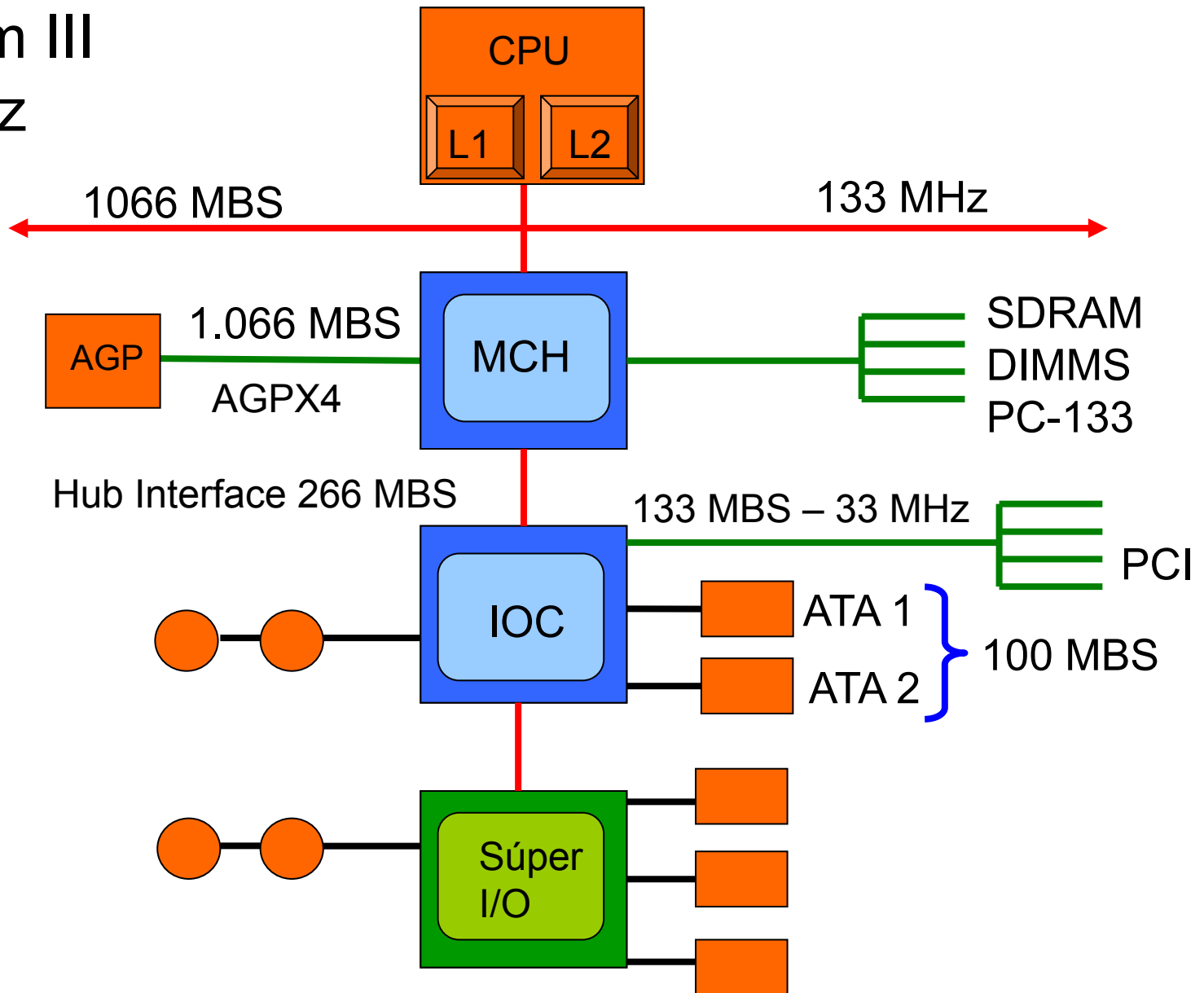
- ✓ PC100 SDRAM DIMM

$100 \text{ MHz} \times 64 \text{ bits} \times 1 \text{ dato.clock} = 800 \text{ MBytes/seg}$

Evolución de jerarquía de bus (2)



Pentium III 1,4 GHz



Pentium III 1,4 GHz

➤ MCH = Memory Controller Hub

➤ IOC = I/O Controller

✓ FSB = Front Side Bus

$133,33 \text{ MHz} \times 64 \text{ bits} \times 1 \text{ dato.clock} = 1066 \text{ MBytes/seg}$

✓ Bus AGP

$66,66 \text{ MHz} \times 32 \text{ bits} \times 4 \text{ datos.clock} = 1066 \text{ MBytes/seg}$

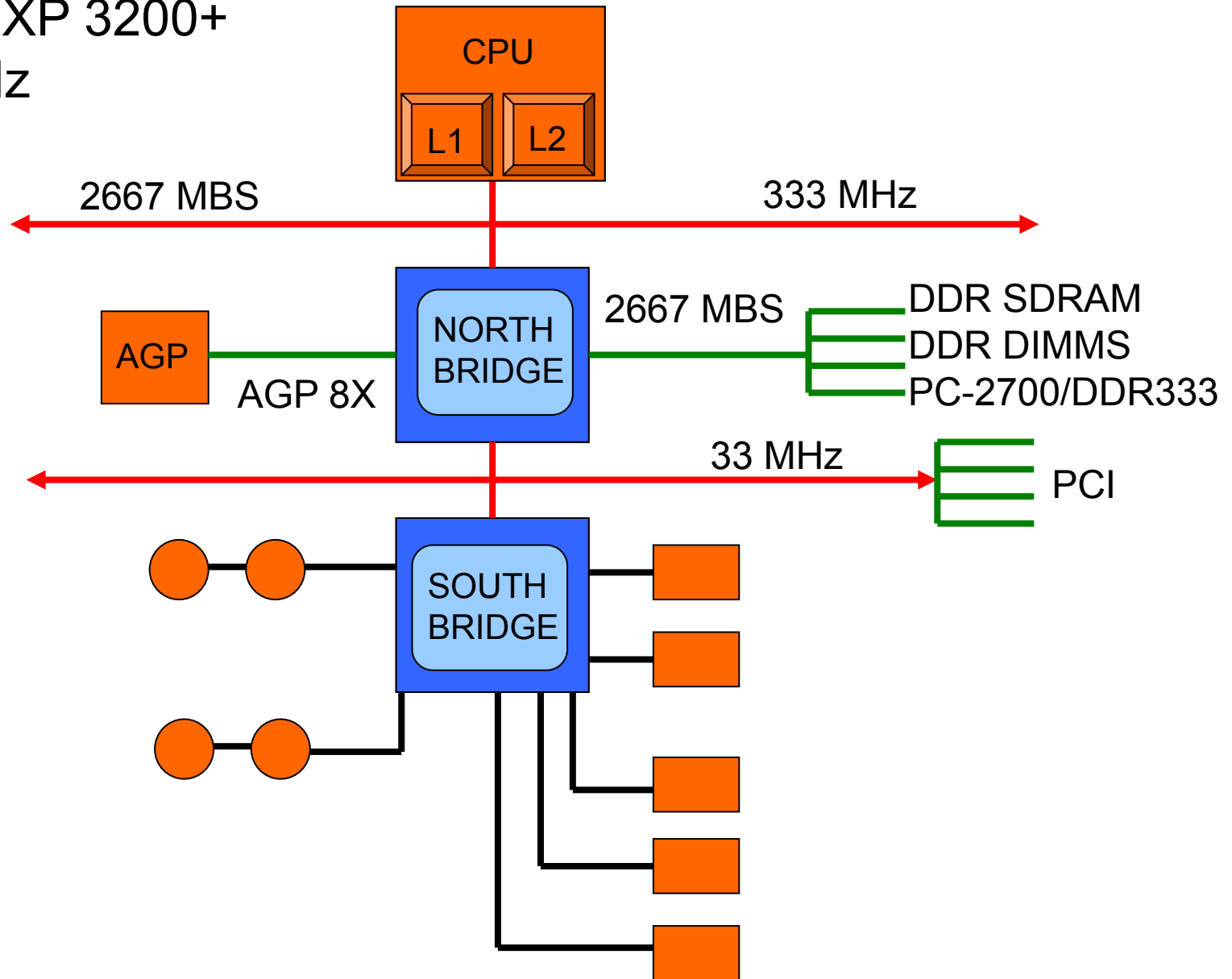
✓ ATA-UDMA

$25 \text{ MHz} \times 16 \text{ bits} \times 2 \text{ datos.clock} = 100 \text{ MBytes/seg}$

✓ PC133 SDRAM DIMM

$133,33 \text{ MHz} \times 64 \text{ bits} \times 1 \text{ dato.clock} = 1066 \text{ MBytes/seg}$

Athlon XP 3200+
2,2 GHz



Athlon XP 3200+ 2,2 GHz

- ✓ FSB = Front Side Bus

$166,66 \text{ MHz} \times 64 \text{ bits} \times 2 \text{ datos.clock} = 2667 \text{ MBytes/seg}$

- ✓ Bus AGP

$66,66 \text{ MHz} \times 32 \text{ bits} \times 8 \text{ datos.clock} = 2133 \text{ MBytes/seg}$

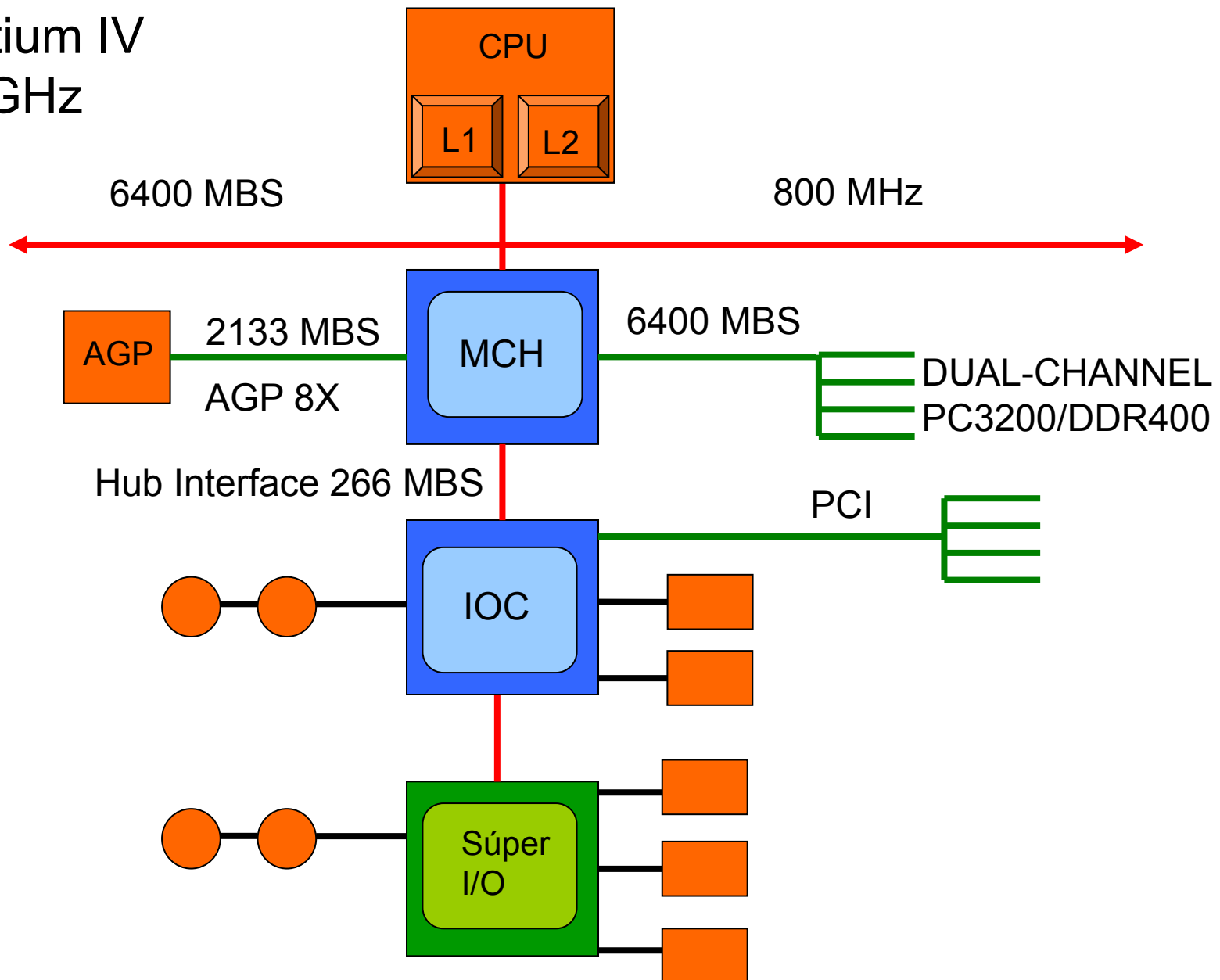
- ✓ ATA-UDMA

$25 \text{ MHz} \times 16 \text{ bits} \times 2 \text{ datos.clock} = 100 \text{ MBytes/seg}$

- ✓ PC2700 DDR DIMM (DDR 333)

$166,66 \text{ MHz} \times 64 \text{ bits} \times 2 \text{ datos.clock} = 2667 \text{ MBytes/seg}$

Pentium IV 3,6 GHz



Pentium IV 3,6 GHz

- ✓ FSB = Front Side Bus

$200 \text{ MHz} \times 64 \text{ bits} \times 4 \text{ datos.clock} = 6400 \text{ MBytes/seg}$

- ✓ Bus AGP

$66,66 \text{ MHz} \times 32 \text{ bits} \times 8 \text{ datos.clock} = 2133 \text{ MBytes/seg}$

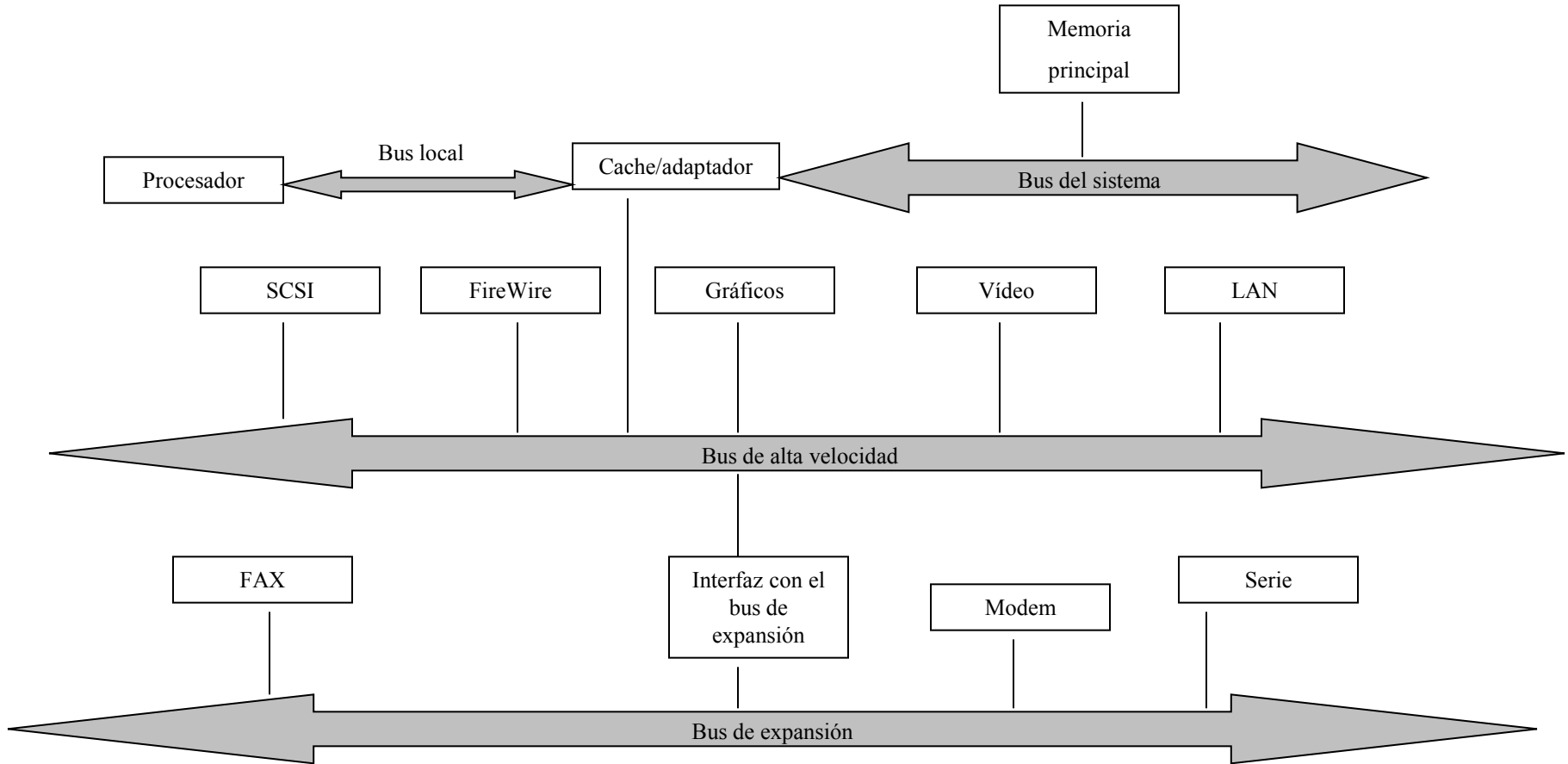
- ✓ ATA-UDMA

$25 \text{ MHz} \times 16 \text{ bits} \times 2 \text{ datos.clock} = 100 \text{ MBytes/seg}$

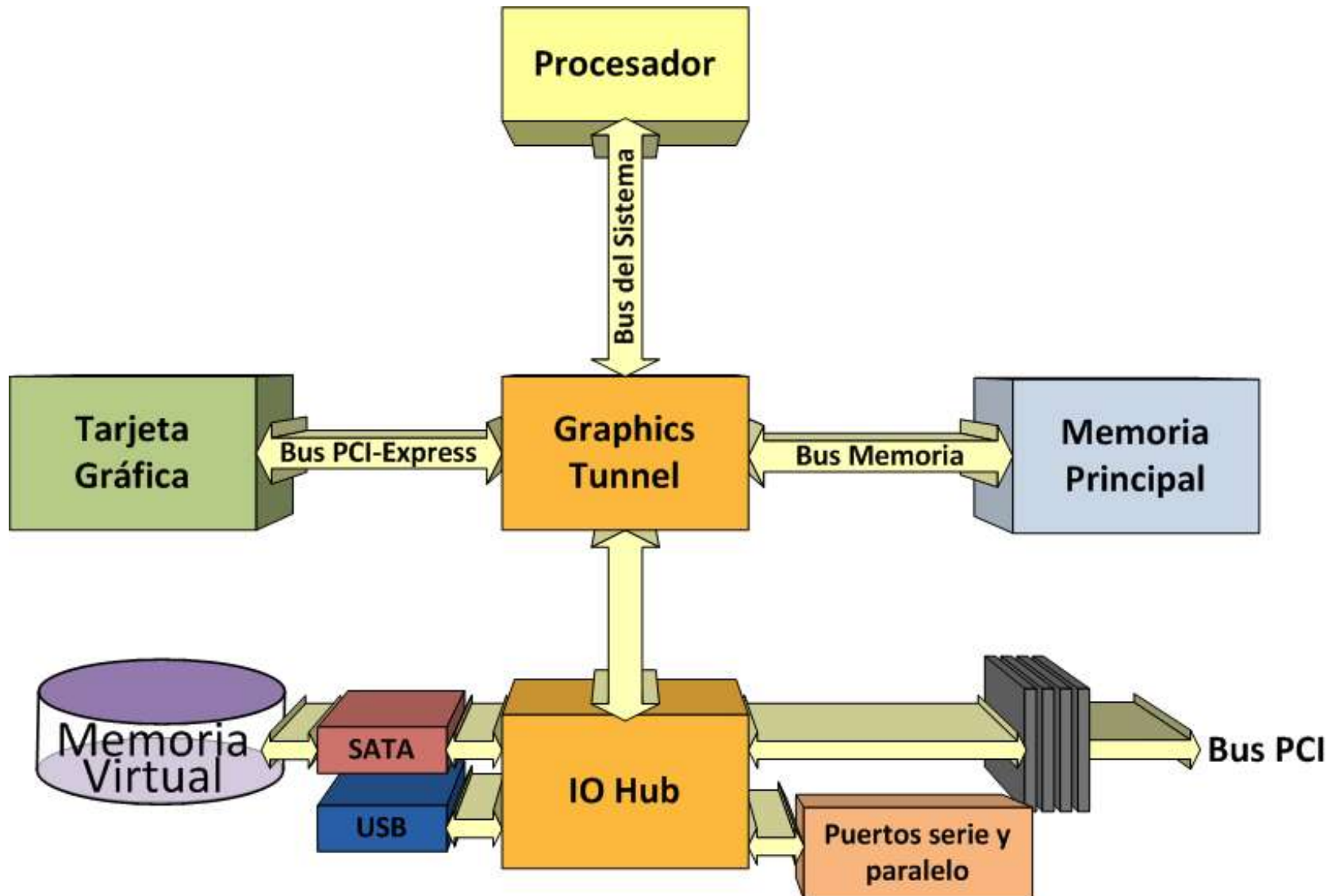
- ✓ PC3200 DDR DIMM (DDR400)

$200 \text{ MHz} \times 64 \text{ bits} \times 2 \text{ dato.clock} = 3200 \text{ MBytes/seg}$

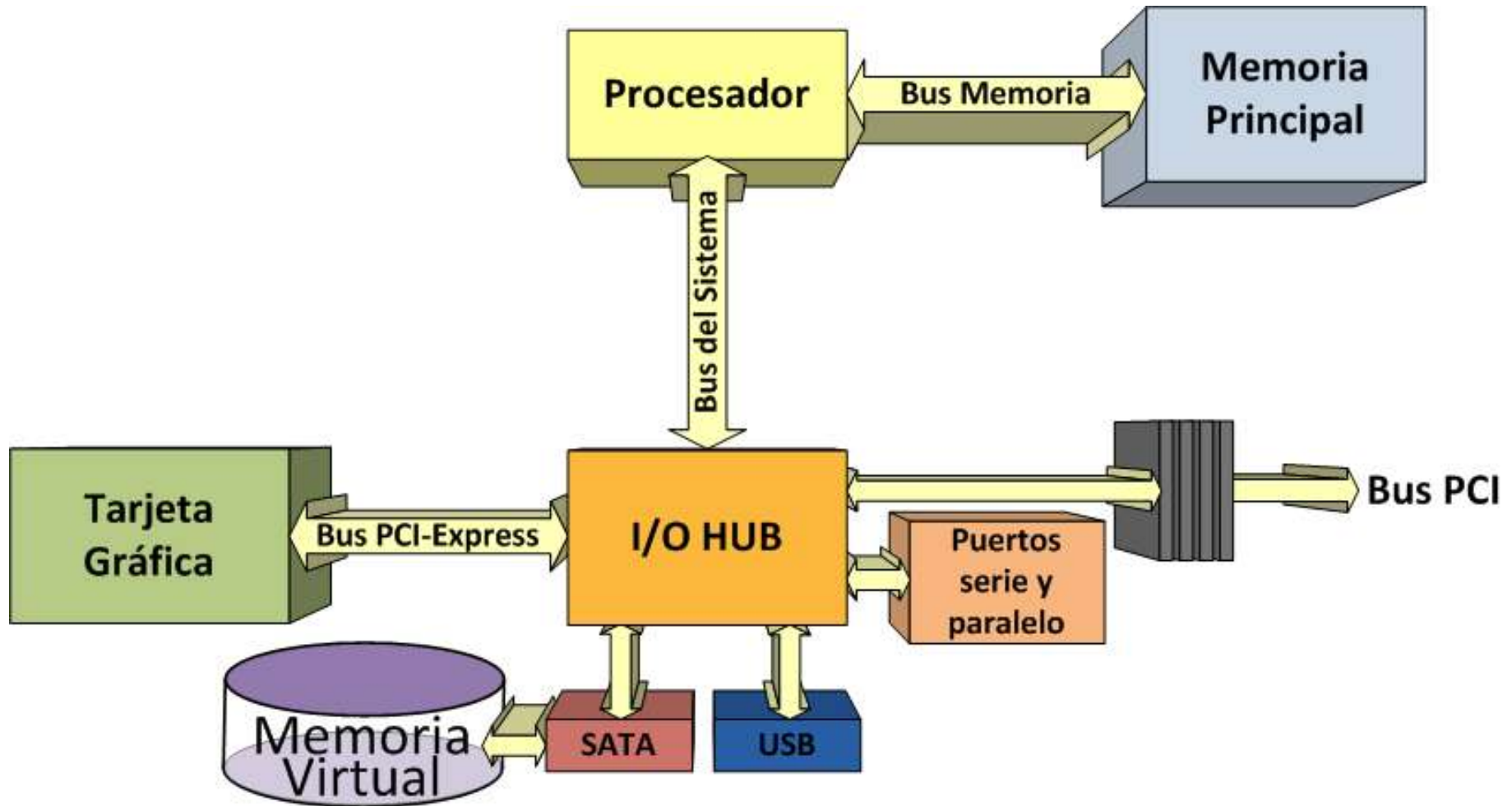
Bus de altas prestaciones



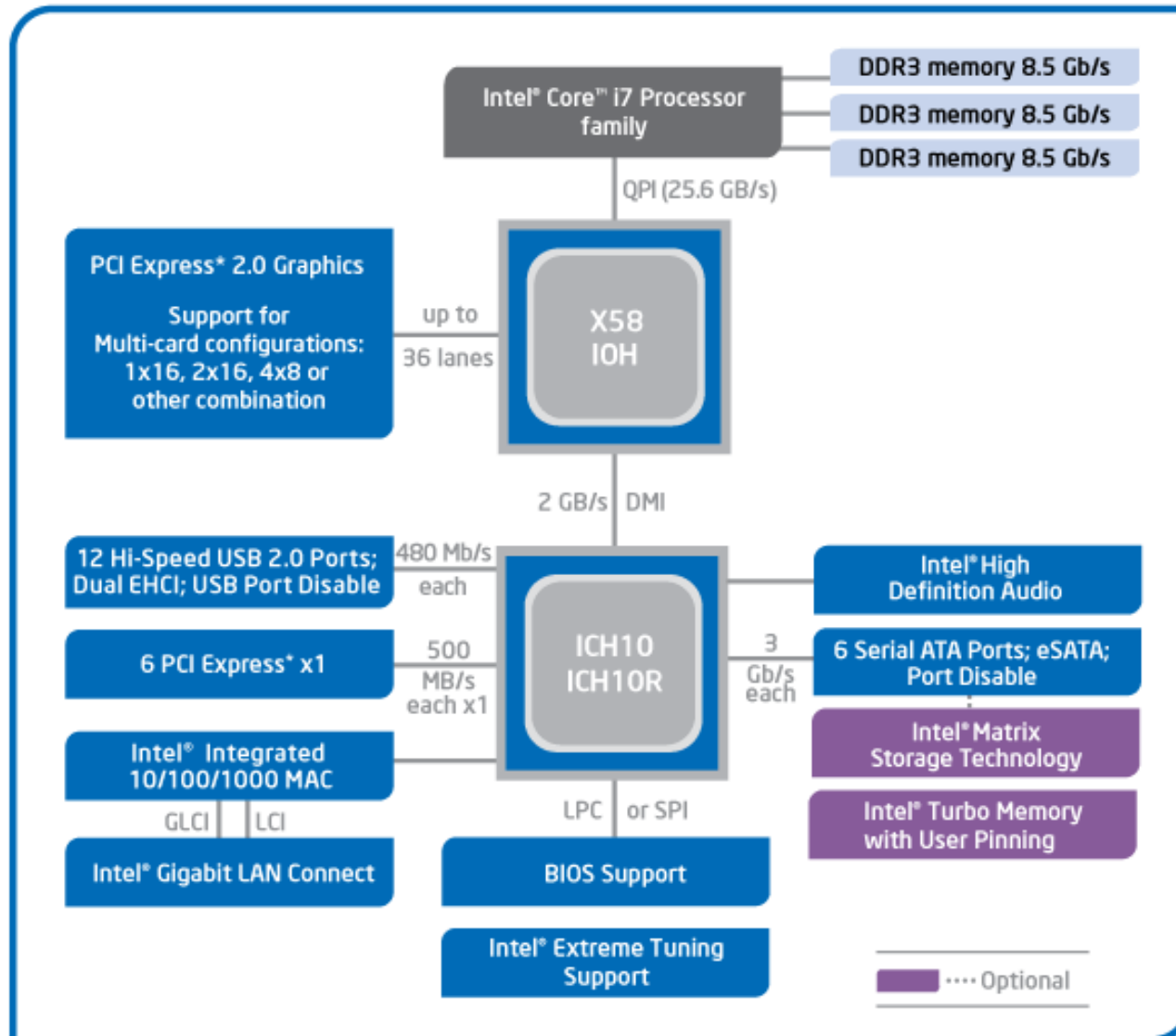
Evolución de jerarquía de bus (3)



Evolución de jerarquía de bus (4)

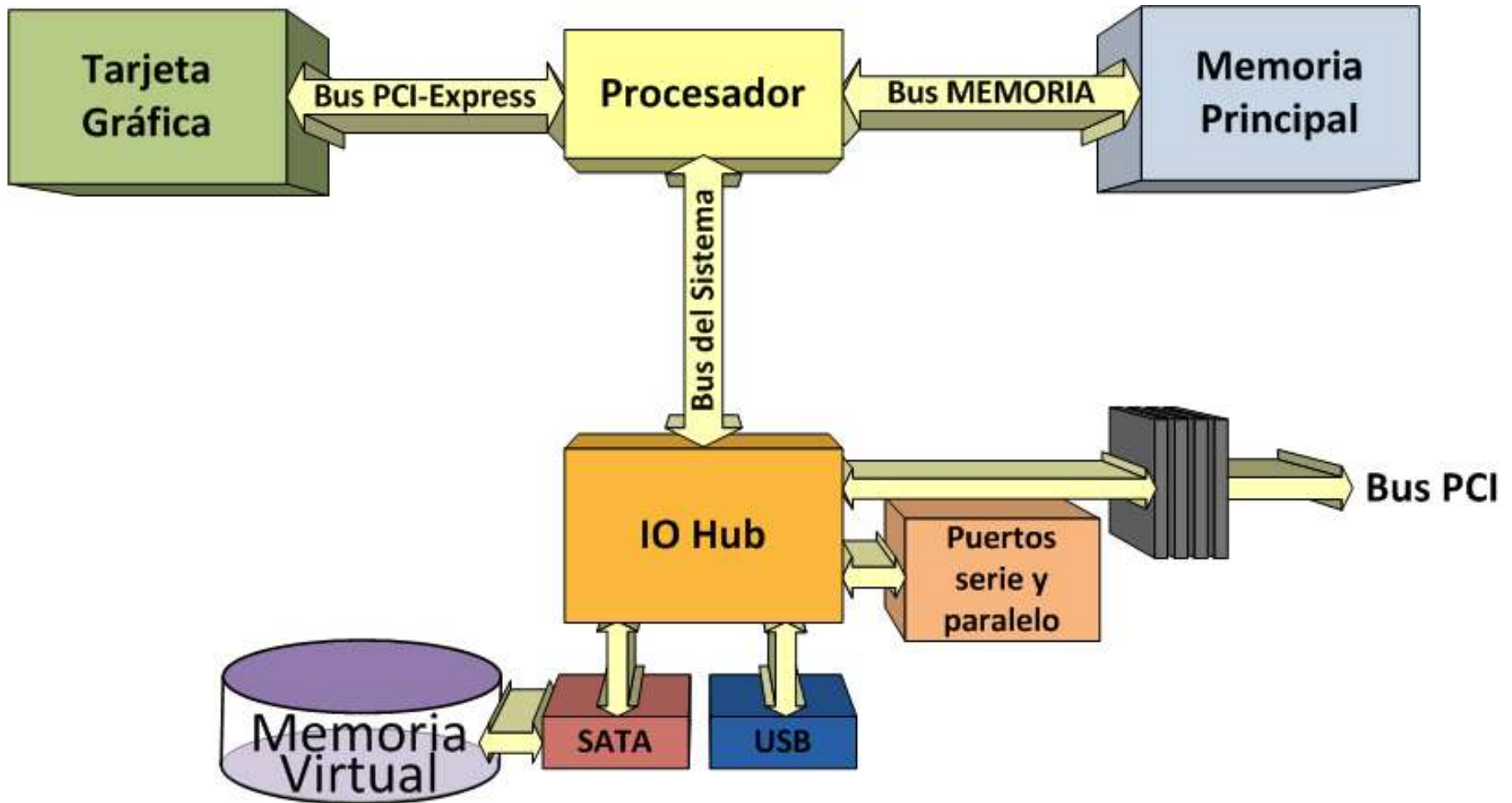


Intel core i7



Intel® X58 Express Chipset Block Diagram

Evolución de jerarquía de bus (5)



Lecturas recomendadas

- *Organización y Arquitectura de Computadoras*, William Stallings, Capítulo 3, 5^{ta} ed.
- *Diseño y evaluación de arquitecturas de computadoras*, M. Beltrán y A. Guzmán, Capítulo 2 Apartado 2.8, 1^{er} ed.
- www.pcguide.com/ref/mbsys/buses/
- Páginas de fabricantes

Arquitectura de Computadoras

CURSO 2022

Turno:
Clase 8

Resumen clase 8

2

Procesadores supersegmentados y superescalares

- Evolución de la segmentación : primeras generaciones
- Procesadores supersegmentados
- Procesadores superescalares
- Paralelismo, IPL, MPL
- Procesadores superescalares
- Políticas de emisión
- Renombrado de registros
- Ejecución superscalar
- Ejemplo de procesador superescalar
- Tratamiento de interrupciones

Evolución de la segmentación

Primera generación – ejecución secuencial

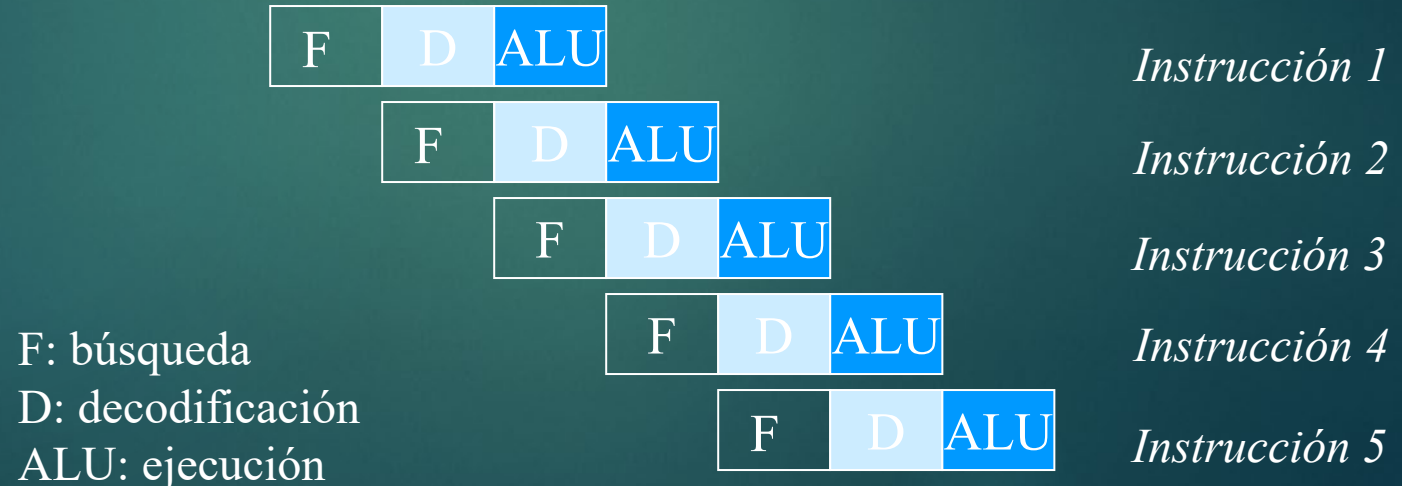
- La primer generación de microprocesadores se caracterizó por un proceso de ejecución completamente secuencial de instrucciones.
- No empezaba una nueva instrucción hasta que se completaba la corriente.
- Ejemplos: i4004, i8008/80, MC6800, MCS6502, Z80, F8 ...



Evolución de la segmentación

Segunda generación – ejecución segmentada

- En la segunda generación se empezó a segmentar el cauce, aunque en forma limitada y en muy pocas etapas.
- Nuevas instrucciones podían iniciarse mientras otras estaban en proceso.
- Ejemplos: MC68000, i8086/286, Z8000



Evolución de la segmentación

Tercera generación – ejecución segmentada aumentada

- En la tercera generación la segmentación se profundizó, aumentando significativamente la cantidad de etapas.
- Nuevas instrucciones podían iniciarse mientras otras estaban en proceso.
- Ejemplos: MC68020, i80386, R2000/3000



Instrucción 1



Instrucción 2



Instrucción 3



Instrucción 4



Instrucción 5

F: búsqueda

D: decodificación

EA: dirección efectiva

E: ejecución

M: memoria

WB: post escritura

Evolución de la segmentación

Limitaciones de procesadores segmentados

- En general, un procesador de k etapas tiene una productividad teórica máxima igual a k

$$\text{productividad teórica máxima} = k$$

- Pero esa productividad tiene varias limitaciones:
 - La máxima capacidad teórica es 1 instrucción por ciclo (CPI o IPC =1). Es decir, solo puede completar 1 instrucción por ciclo de reloj.
 - Hay 1 solo pipeline para los diferentes tipos (de datos).
 - Los atascos en el cauce producen burbujas innecesarias que reducen la productividad.

Evolución de la segmentación

Rendimiento de procesadores escalares

➤ El rendimiento de un procesador escalar segmentado depende de:

➤ **IPC:** número de instrucciones por ciclo

➤ **f:** Frecuencia del reloj ($T = 1 / f$)

➤ **k:** cantidad de etapas del cauce

➤ El tiempo de ejecución es igual a:

$$\text{CPU time} = N^{\circ} \text{ instr.} \times \text{IPC} \times T$$

donde: T = tiempo de ciclo

➤ Para estos procesadores, en el mejor de los casos $\text{IPC} = 1$, y solo se puede mejorar este tiempo si se aumenta f .

Evolución de la segmentación

Limitaciones en procesadores escalares segmentados

- Las limitaciones en el rendimiento de un procesador escalar segmentado se originan en 2 aspectos:
 1. El IPC es igual a 1, en el mejor de los casos.
 2. Los posibles conflictos que pueden producir atascos en el cauce debidos a:
 - Dependencias de datos: hay 3 tipos de dependencias de datos: Verdadera, de salida, y antidependencia.
 - Penalizaciones en saltos.
 - Conflictos por uso de recursos.

Evolución de la segmentación

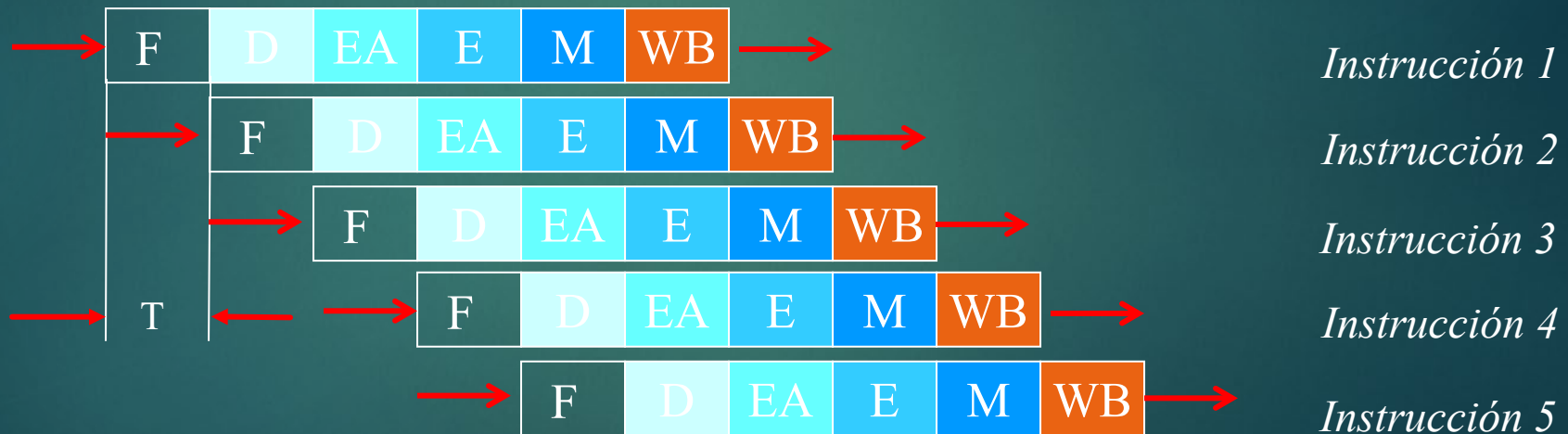
Limitaciones en procesadores escalares segmentados

En la figura siguiente se puede apreciar el proceso de ejecución de una secuencia de instrucciones. En cada ciclo T se ingresa una nueva instrucción, y también se resuelve solo una.

F: búsqueda
D: decodificación

EA: dirección efectiva
E: ejecución

M: memoria
WB: post escritura



Existen 2 técnicas para aumentar la performance usando segmentación.

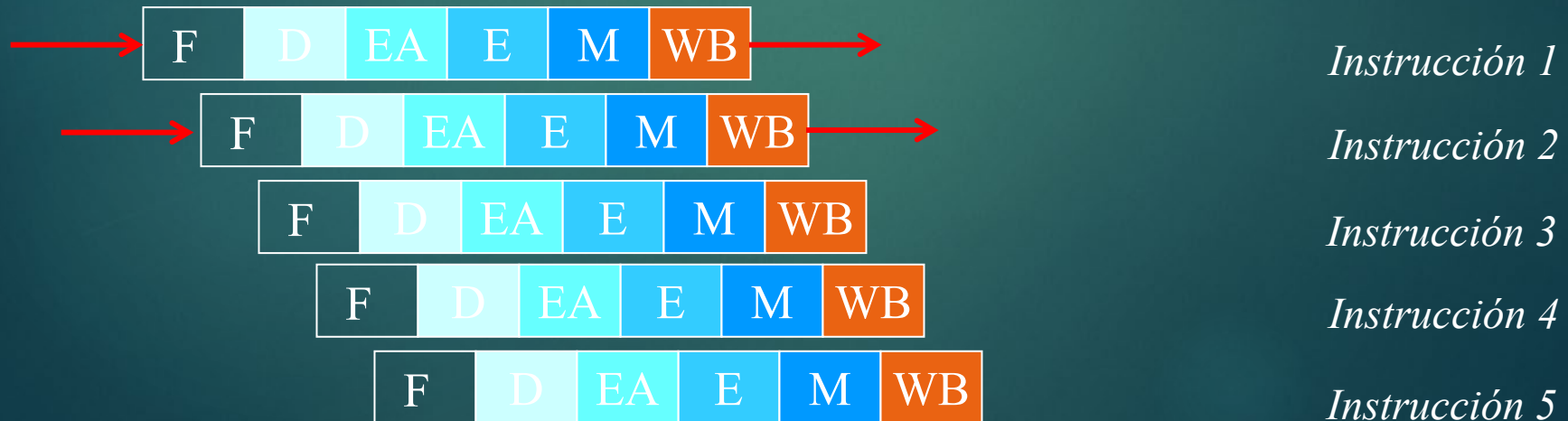
Procesadores supersegmentados y superescalares

1) Procesadores supersegmentados

En la ejecución supersegmentada de instrucciones cada ciclo se divide en fracciones más chicas, en las que se inician nuevas instrucciones.

Por ejemplo, un procesador supersegmentado de grado 2 acepta una nueva instrucción en cada semiciclo de reloj.

F: búsqueda EA: dirección efectiva M: memoria
D: decodificación E: ejecución WB: post escritura



Procesadores supersegmentados y superescalares

Procesadores supersegmentados

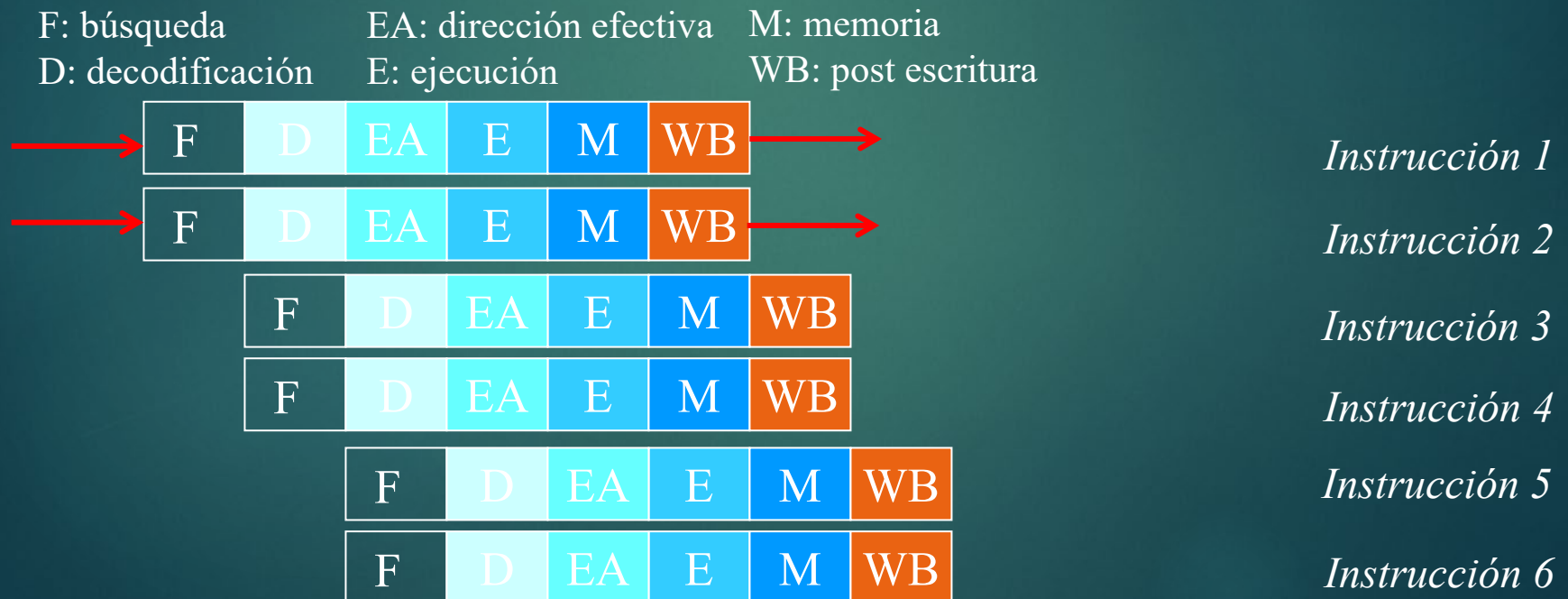
- La ejecución supersegmentada consiste en subdividir cada segmento en partes más pequeñas.
- Como muchas operaciones no necesitan todo un ciclo de reloj, se puede hacer más de una tarea en ese ciclo, subdividiendo el ciclo de reloj en sub-intervalos, lo que es equivalente a usar una mayor frecuencia de reloj (menor ciclo de reloj).
- El tiempo para las instrucciones individuales no varía, pero aumenta el grado del paralelismo de la máquina temporal.
- El pipeline se hace más profundo, pero está limitado por la tecnología (es decir, por la frecuencia máxima).

Procesadores supersegmentados y superescalares

2) Procesadores superescalares

En la ejecución superescalar de instrucciones, en cada ciclo se inician más de 1 instrucción.

Por ejemplo, un procesador superescalar de grado 2 inicia 2 nuevas instrucciones en cada ciclo de reloj.



Procesadores supersegmentados y superescalares

13

Procesadores superescalares

- La aproximación superescalar se basa en poder ejecutar varias instrucciones en diferentes cauces de manera independiente y concurrente.
- Algunos ejemplos de procesadores superescalares son: MC68040, i80486, MC88110, i80860, PA-RISC, Sparc, R6000.

Procesadores superescalares

14

Procesadores superescalares

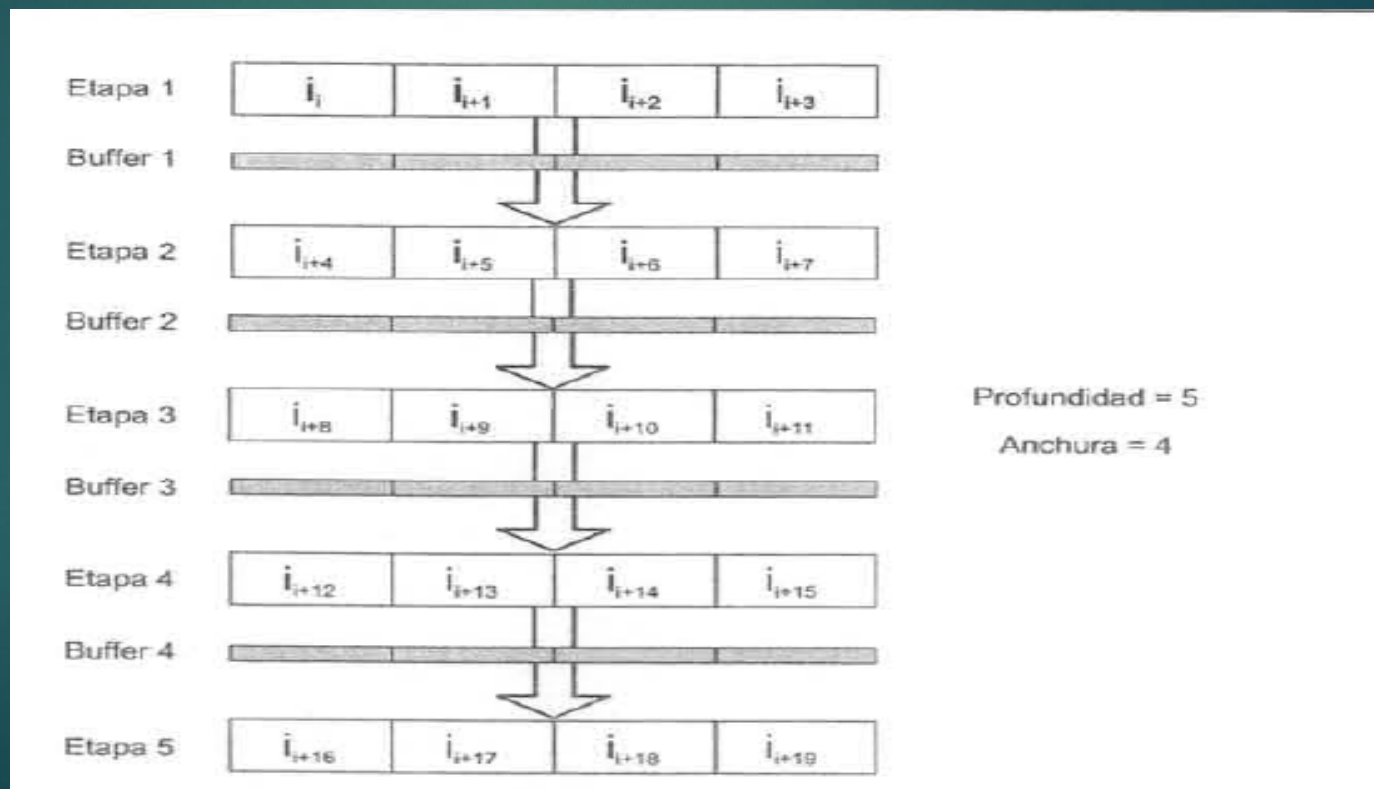
- Se pueden llevar a cabo (completar) 2 o más instancias de cada etapa de una instrucción simultáneamente.
- Para poder iniciar/ejecutar 2 o más instrucciones simultáneamente, se requiere la duplicación de algunas o todas las partes de la CPU/ALU, por ejemplo:
 - Captación de múltiples instrucciones al mismo tiempo.
 - Ejecución (sumas y multiplicaciones) simultánea.
 - Ejecución de carga/almacenamiento, mientras se lleva a cabo una operación en ALU.
- El grado de paralelismo y, por tanto, la aceleración de la máquina aumentan, ya que se ejecutan más instrucciones en paralelo.

Procesadores superescalares

15

La aproximación superescalar presenta paralelismo de máquina de 2 tipos: temporal y espacial.

Por ejemplo, una máquina supersegmentada de profundidad 5, y ancho 4 (capacidad de ejecutar hasta 20 instrucciones simultáneamente) tendría la siguiente arquitectura:

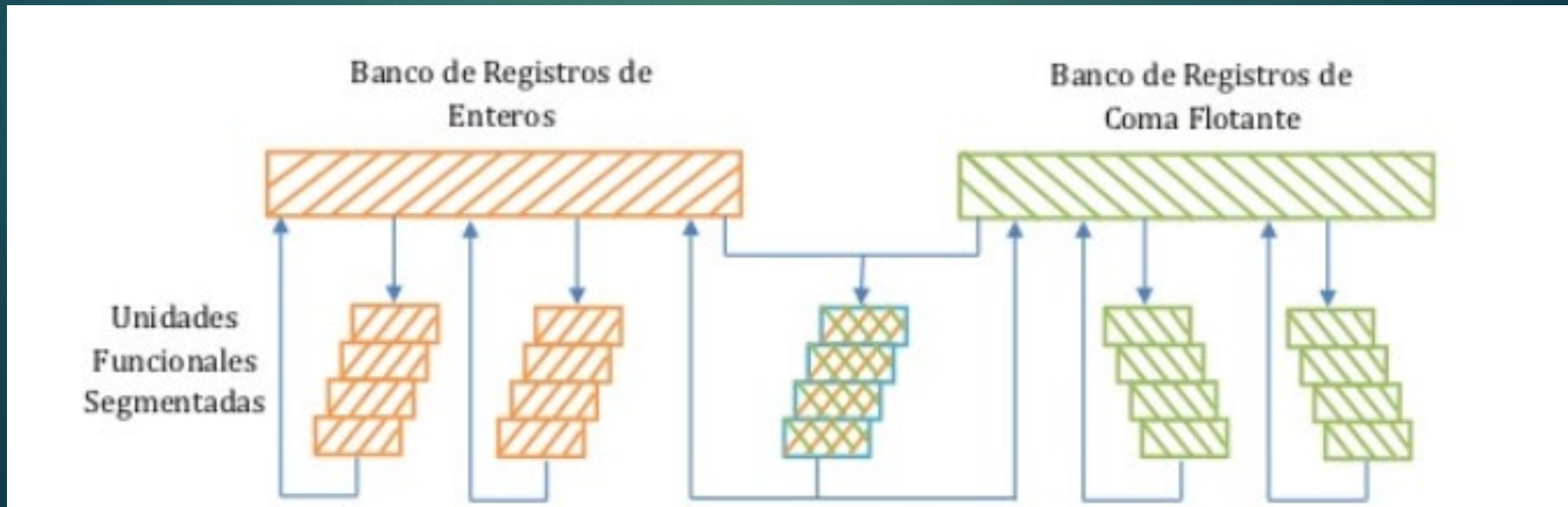


Procesadores superescalares

16

Esquema básico

Un procesador superescalar dispone, básicamente, de múltiples unidades funcionales, cada una implementada como un cauce segmentado, que admite la ejecución paralela de varias instrucciones.



Procesadores superescalares

Los conflictos en procesadores superescalares son los mismos que en procesadores segmentados.

I0 Add r1, r2

I1 Mov r3, r1



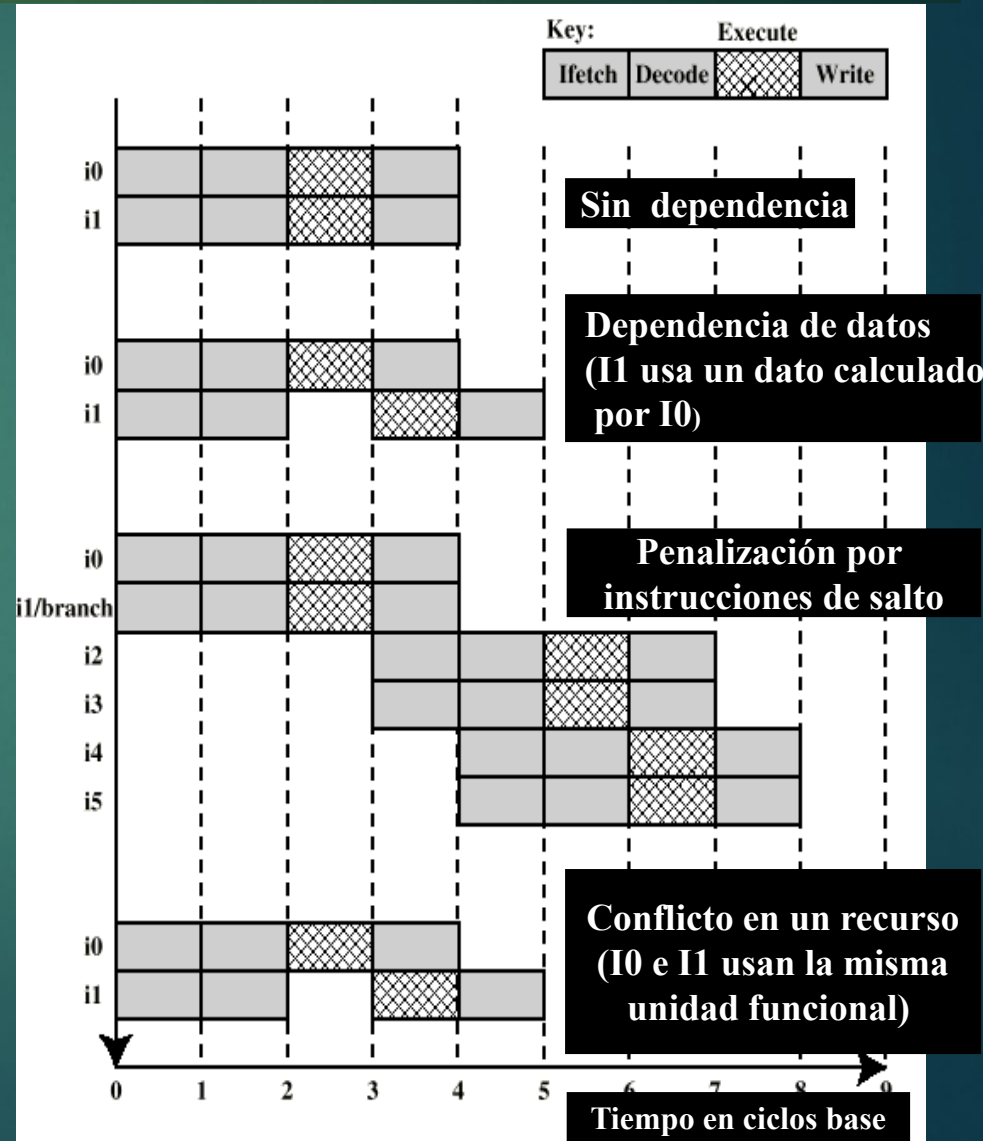
I0 Add r1, r2

I1 Imp etiqueta



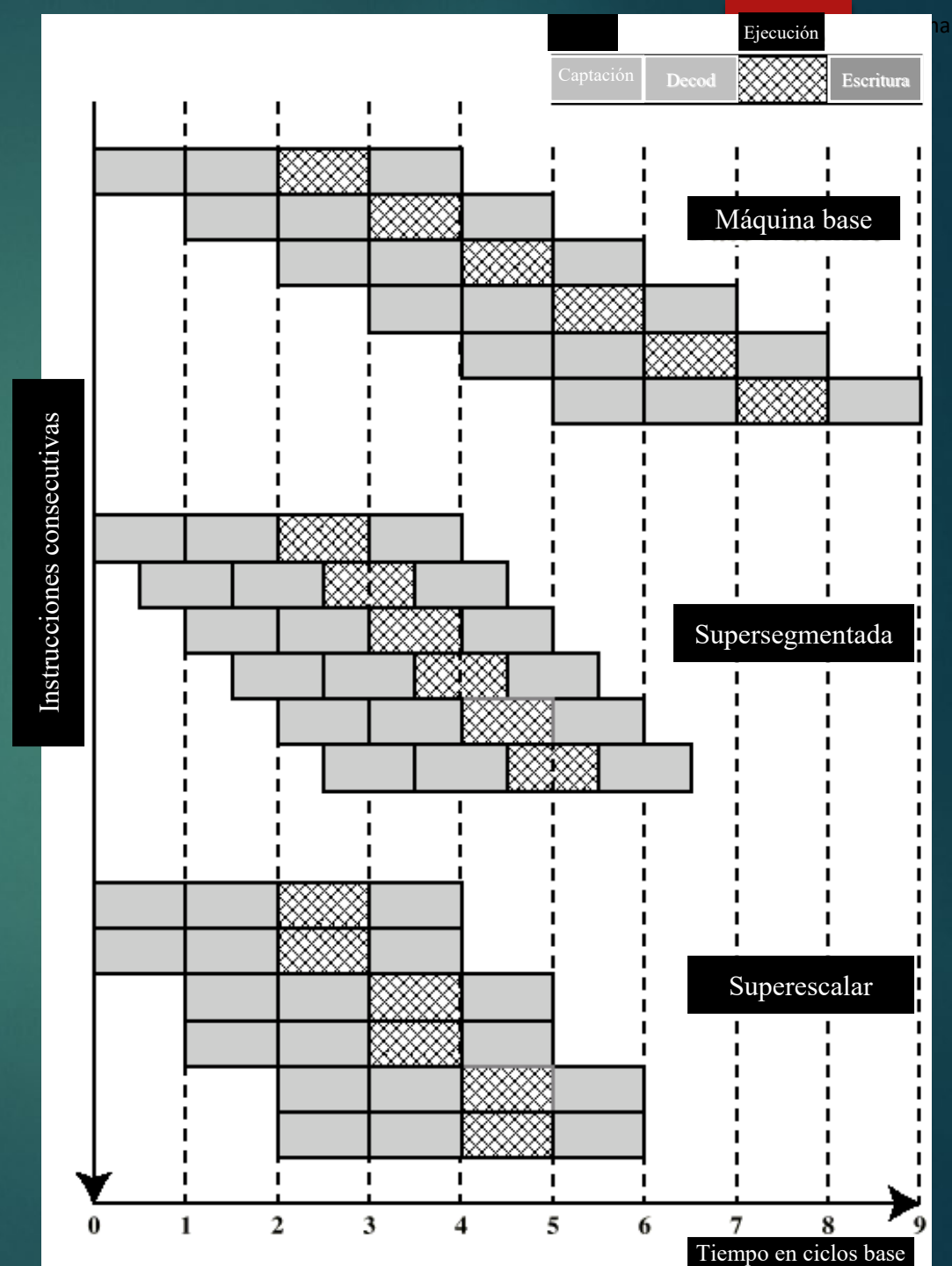
I0 Add r1, r2

I1 Add r4, r3



Procesadores superescalares

El gráfico siguiente muestra una comparación entre las ejecuciones segmentada, superscalar y la supersegmentada.



Paralelismo

19

- El paralelismo es la capacidad para ejecutar varias tareas en el mismo intervalo de tiempo (en “paralelo”).
- Hay 2 tipos de paralelismo:
 - Paralelismo a nivel de instrucciones: se refiere a las posibilidades que tiene un programa para ejecutar instrucciones en paralelo
 - Paralelismo a nivel de máquina: es la capacidad que tiene una máquina para encontrar y ejecutar tareas en paralelo.

Paralelismo

20

Paralelismo a nivel de programa

- El grado de paralelismo de un programa se mide usando un parámetro conocido como IPL.
- El IPL es la cantidad, en promedio, de instrucciones que pueden ejecutarse en paralelo.
- Las instrucciones pueden ejecutarse en paralelo si son independientes.

ADD	R1, R2, R3
SUB	R4, R5, R6
AND	R7, R8, R9

Paralelismo de grado 3

ADD	R1, R2, R3
SUB	R4, R5, R1
AND	R7, R8, R4

Paralelismo de grado 1

Paralelismo

21

Paralelismo a nivel de máquina

- El grado de paralelismo de una máquina se mide usando un parámetro conocido como MPL, que es la capacidad que tiene una máquina para aprovechar el paralelismo de un programa.
- El MPL es función de:
 - Número de instrucciones captadas por ciclo.
 - Número de unidades funcionales.
 - Mecanismos para localizar y ejecutar instrucciones independientes.

Procesadores superescalares

22

Tratamiento de las instrucciones

- En los procesadores superescalares el objetivo fundamental es localizar instrucciones que puedan ser introducidas al pipeline y ejecutadas. En este proceso de detección, es necesario distinguir:
 - El orden en que se captan las instrucciones.
 - El orden en que se ejecutan las instrucciones.
 - El orden en que las instrucciones actualizan los registros y las posiciones de memoria.
- Cuanto más sofisticado es el procesador, menos restricciones impone a estos ordenamientos. Incluso puede alterar cualquier ordenamiento, respecto del estrictamente secuencial. La única condición es que el resultado debe ser correcto.

Políticas de emisión

23

Tratamiento de las instrucciones

- Las políticas de emisión de instrucciones son los protocolos usados para el envío de las instrucciones a las unidades funcionales, es decir, definen la forma en que se captan, ejecutan y finalizan (escriben los resultados) las instrucciones.
- En función del orden en que se captan, ejecutan y terminan las instrucciones, existen 3 políticas de emisión de instrucciones:
 - Emisión y finalización ordenada
 - Emisión ordenada y finalización desordenada
 - Emisión y finalización desordenada

Políticas de emisión

24

Políticas de emisión de instrucciones

- Para analizar las políticas de emisión de instrucciones, vamos a considerar un procesador superescalar con un cauce de 3 segmentos compuesto por:
 - Búsqueda de instrucción y decodificación: IF+D
 - Ejecución: Ex
 - Escritura de resultados: W

Además, el procesador dispone de:

- 2 unidades de decodificación D1 y D2.
- 3 unidades de ejecución E1, E2 y E3.
- 2 copias de la etapa de escritura W1 y W2.

Políticas de emisión

25

Políticas de ejecución de instrucciones

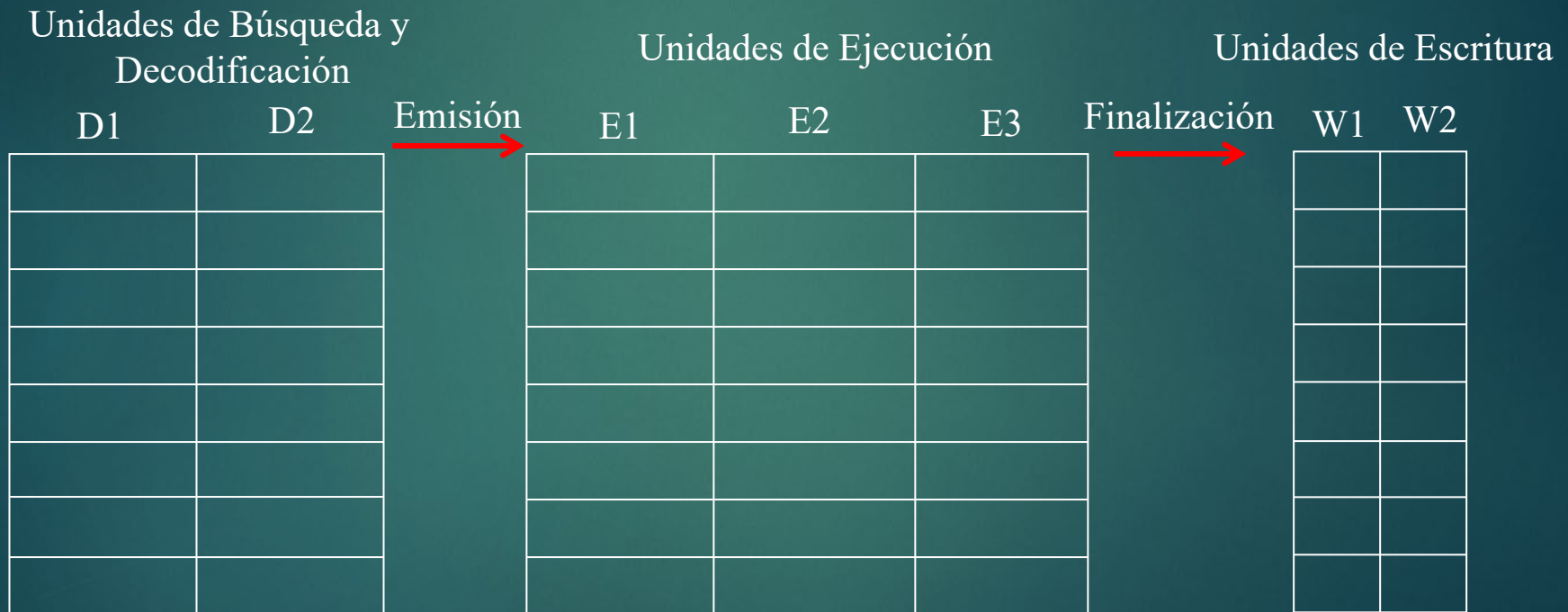
- Como tiene 2 unidades de búsqueda y decodificación D1 y D2, 2 instrucciones pueden ser leídas y decodificadas al mismo tiempo.
- Solo se puede leer un nuevo par de instrucciones cuando se liberan ambas unidades D1 y D2.
- Para garantizar la correcta ejecución, los conflictos se resuelven parando la instrucción (“stall”) hasta que el conflicto desaparece.

Políticas de emisión

26

Políticas de ejecución de instrucciones

Funcionalmente, el procesador superescalar que vamos a usar se puede visualizar de la siguiente manera:



En las filas se van a indicar las instrucciones en proceso, para cada ciclo de reloj.

Políticas de emisión

27

Políticas de ejecución de instrucciones

Se va a analizar el comportamiento del procesador superescalar para una secuencia de 6 instrucciones con las siguientes características:

- I1 tarda dos ciclos en ejecutarse
- I2 tarda 1 ciclo
- I3 e I4 tardan 1 ciclo y usan la misma unidad funcional (E3)
- I5 e I6 tardan 1 ciclo y usan la misma unidad funcional (E2)
- I5 depende del valor producido por I4

Políticas de emisión

28

Emisión y finalización ordenada

En esta política, las instrucciones se emiten exactamente como están en el programa (como si fuera “secuencial”), y se escriben los resultados en el mismo orden. El resultado de ejecutar las 6 instrucciones es el siguiente.

Decodificación		Ejecución			Escritura		Ciclo
D1	D2	E1	E2	E3	W1	W2	
I1	I2						1
I3	I4	I1	I2				2
I3	I4	I1					3
	I4			I3	I1	I2	4
I5	I6			I4			5
	I6		I5		I3	I4	6
			I6				7
					I5	I6	8

Políticas de emisión

29

Emisión y finalización ordenada

- En el ejemplo anterior se puede ver que:
 - Las instrucciones se captan y decodifican de a 2.
 - Las instrucciones I5 e I6 deben esperar hasta que se desocupen ambos decodificadores D1 y D2.
 - Dado que el orden en que se escriben los resultados debe ser el mismo en que se captan, I2 no puede ser escrita hasta que I1 no se haya completado y esté lista para guardarse (W1 o W2).
 - I4 tiene que esperar que I3 libere E3. Idem I5 e I6.
 - I5 tiene que esperar que se ejecute I4.
 - Se requieren 8 ciclos para ejecutar las 6 instrucciones.

Políticas de emisión

30

Emisión ordenada y finalización desordenada

En esta política, las instrucciones se emiten exactamente como está en el programa, pero los resultados se escriben en “cualquier orden”. Las mismas 6 instrucciones se ejecutan de la siguiente manera.

Decodificación		Ejecución			Escritura		Ciclo
I1	I2						1
I3	I4	I1	I2				2
	I4	I1		I3	I2		3
I5	I6			I4	I1	I3	4
	I6		I5		I4		5
			I6		I5		6
					I6		7

Políticas de emisión

31

Emisión ordenada y finalización desordenada

- En el ejemplo anterior se puede ver que:
 - Las instrucciones se captan y decodifican de a 2. Por eso, las instrucciones I5 e I6 deben esperar hasta que se desocupen ambos decodificadores D1 y D2.
 - Dado que el orden en que se escriben los resultados puede ser distinto del que se captan, I2 puede ser escrita antes que I1 se haya completado y esté lista para guardarse.
 - Al permitir la finalización desordenada, I3 puede ser ejecutada anticipadamente, ganándose 1 ciclo.
 - Ahora se requieren 7 ciclos para ejecutar las 6 instrucciones.

Políticas de emisión

32

Emisión ordenada y finalización desordenada - Conclusiones

- Las instrucciones entran a las unidades de ejecución a medida que se van decodificando y hay unidades de ejecución disponibles.
- Las emisiones de las instrucciones están limitadas por los posibles conflictos de recursos, dependencias de datos y de saltos.
- Al permitir la finalización desordenada, aparecen nuevas fuentes de conflictos por dependencia de datos.
- En el siguiente ejemplo se pueden apreciar los nuevos conflictos que aparecen por la política de finalización desordenada.

Políticas de emisión

33

Emisión ordenada y finalización desordenada - Conflictos

Consideremos el siguiente programa:

R3:= R2 op R5 (I1)

R4:= R3 + 1 (I2)

R3:= R5 + 1 (I3)

R7:= R3 op R4 (I4)

Los riesgos que aparecen en el ejemplo anterior son:

1) RAW (dependencia verdadera)

I1 escribe el resultado en R3, que es un operando de la I2.

2) WAW (dependencia de salida)

I1 escribe el resultado en R3 igual que la I3. Si I3 se escribe antes que I1, entonces la I4 usará un valor incorrecto de R3.

3) WAR (antidependencia)

I3 escribe en R3. Pero I2 necesita el valor previo de R3. I3 no puede escribir R3 hasta que I2 haya leído el valor.

Políticas de emisión

34

Emisión ordenada y finalización desordenada - Conflictos

- La finalización desordenada requiere detectar estas posibles nuevas dependencias, y evitar que se ejecuten instrucciones en forma desordenada que puedan alterar el resultado del programa.
- Por tal motivo, la lógica de emisión de instrucciones es más compleja que la empleada en máquinas con política de finalización ordenada.
- Con la política de emisión ordenada, el procesador solo puede decodificar instrucciones hasta el punto de dependencia o conflicto. Es decir, no analiza si hay nuevas instrucciones que pueden emitirse sin producir conflicto.

Políticas de emisión

35

Emisión y finalización desordenada

- Para permitir que el procesador busque más allá del punto de conflicto por nuevas instrucciones, se requiere usar una política de emisión desordenada.
- La política de emisión desordenada requiere separar, mediante un buffer, llamado Ventana de instrucciones, las unidades de decodificación y las de ejecución.
- Cada instrucción decodificada se coloca en un buffer intermedio, desde donde se emiten las que no presentan conflictos. Mientras no se llene el buffer, nuevas instrucciones son decodificadas y colocadas para su emisión a las unidades de ejecución. Las instrucciones emitidas no deben tener conflictos por recursos ni por dependencias de datos.

Políticas de emisión

36

Emisión y finalización desordenada

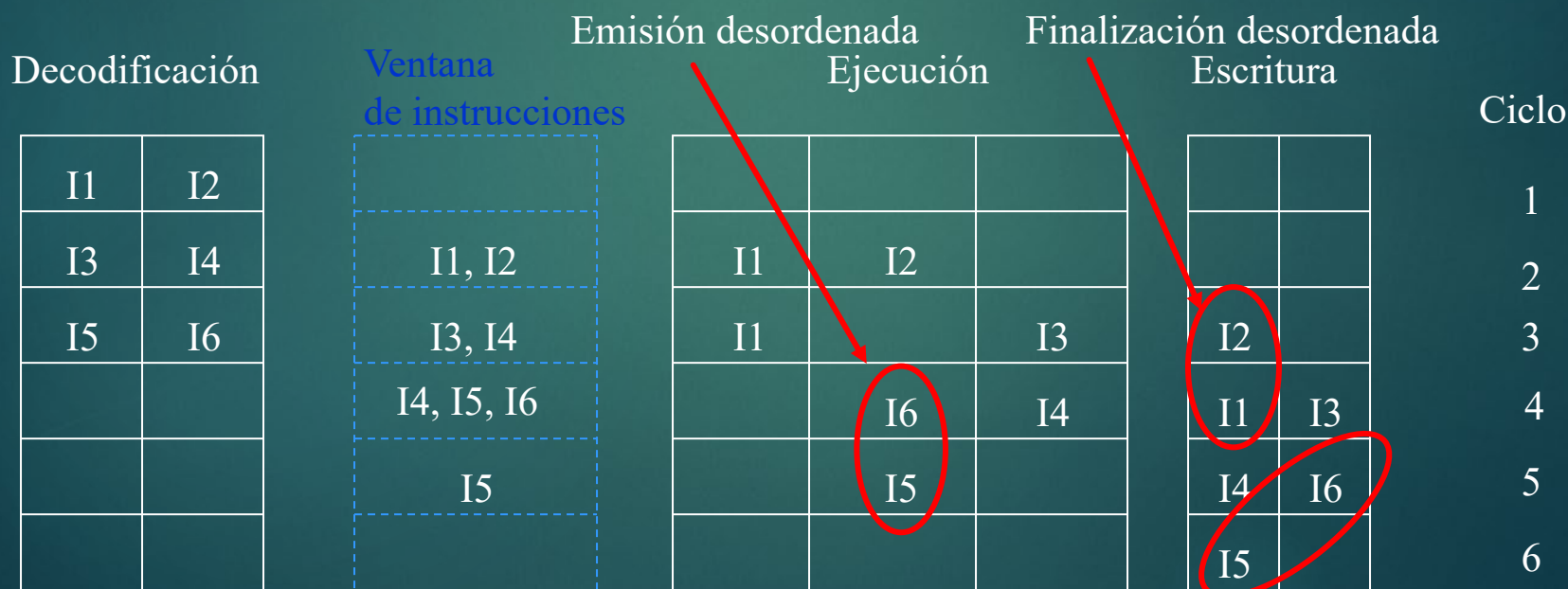
- La Ventana de instrucciones no es una etapa del cauce, es un buffer que retiene la información necesaria para emitirla.
- En el siguiente ejemplo se muestra el esquema funcional de una máquina con emisión y finalización desordenada.

Políticas de emisión

37

Emisión y finalización desordenada

- Se intercala una buffer (ventana de instrucciones) entre las unidades de decodificación y las de ejecución.
- Las instrucciones se emiten y los resultados se guardan en forma desordenada.



Políticas de emisión

38

Emisión y finalización desordenada

- En el ejemplo anterior se puede ver que:
 - Las instrucciones se captan y decodifican de a 2, pero ahora la ventana de instrucciones permite, en el ciclo 3, decodificar las instrucciones I5 e I6.
 - Como I6 no tiene dependencias y está lista para ejecutarse, puede ser emitida anticipadamente, incluso antes de I5. La ventana le permite “ver” al procesador más adelante de las instrucciones en conflicto, pudiendo detectar nuevas instrucciones (la I6) que no genera conflicto.
- Ahora se requieren 6 ciclos para ejecutar las 6 instrucciones.

Renombrado de registros

39

- Al permitir finalización y/o emisión desordenada surgen los nuevos problemas de dependencia de datos (WAW y WAR).
- Las dependencias de salida y antidependencias surgen porque los valores de los registros pueden no reflejar la secuencia de valores dictada por el flujo del programa (en realidad es un conflicto de almacenamiento: hay instrucciones que compiten por un mismo registro).
- Tal como se vio en clases anteriores, estos nuevos conflictos pueden resolverse con detenciones en la etapa del cauce, pero esa solución produce pérdidas de tiempo que pueden llegar a ser inaceptables.

Renombrado de registros

40

Supongamos la siguiente secuencia de instrucciones de un programa:

R3:= R3 op R5 (I1)

R4:= R3 + 1 (I2)

R3:= R5 + 1 (I3)

R7:= R3 op R4 (I4)

En la secuencia anterior se pueden observar las 3 dependencias:

RAW entre la I1 y la I2

WAR entre la I2 y la I3

WAW entre la I1 y la I3

Renombrado de registros

41

- Dado que en definitiva, es un conflicto de recursos (los registros), se puede resolver duplicando los recursos.
- Esta técnica se conoce como Renombrado de Registros.
- Los registros en las instrucciones del programa, son registros “lógicos”. Los registros físicos (es decir, los reales) los asigna el procesador.
- Cada vez que se guarda un dato en un registro el procesador le asigna un nuevo registro físico. Las instrucciones siguientes que referencien a ese registro, deben renombrarse para referenciar el registro físico que tiene el dato correcto.

Renombrado de registros

42

Consideremos el ejemplo anterior (donde los nombres de registros son referencias “lógicas”):

R3:= R3 op R5 (I1)

R4:= R3 + 1 (I2)

R3:= R5 + 1 (I3)

R7:= R3 op R4 (I4)

El procesador va asignando nuevos registros en cada escritura, quedando las siguientes referencias “físicas”:

R3b:= R3a op R5a (I1)

R4a:= **R3b** + 1 (I2)

R3c:= R5a + 1 (I3)

R7a:= **R3c** op R4a (I4)

Renombrado de registros

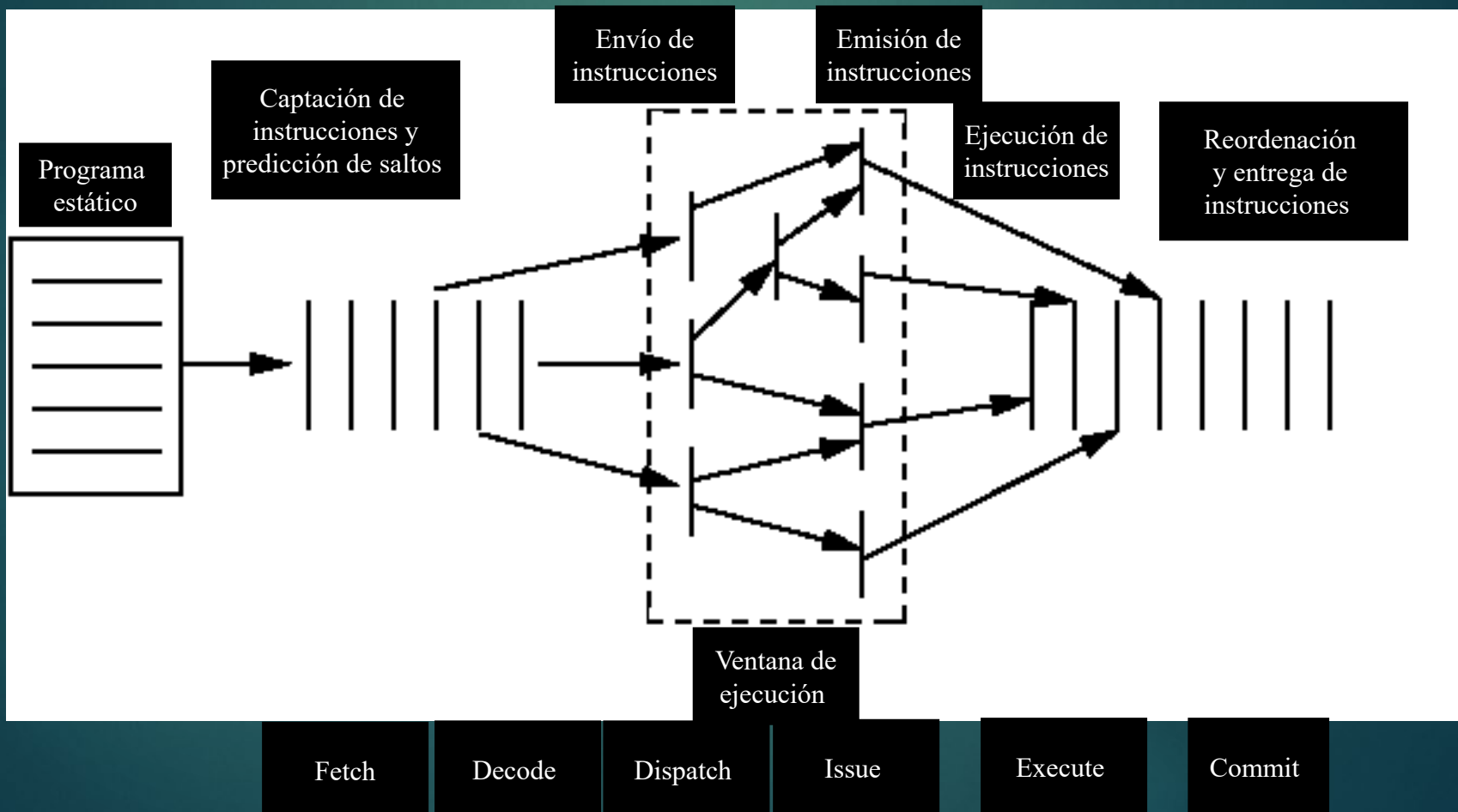
43

- En el ejemplo anterior, en la instrucción I1 se asigna un nuevo registro R3b distinto del usado anteriormente R3a.
- La I2 debe renombrarse para usar R3b (y no R3a), y crea R4a (que no existía previamente).
- La I3 asigna un nuevo registro R3c, distinto del usado anteriormente (R3b).
- La I4 debe renombrarse para usar R3c (y no R3b), y crea R7a (que no existía previamente).
- De esta manera se eliminan las dependencias WAW y WAR (antidependencia y dependencia de salida) quedando únicamente la dependencia verdadera RAW.

Ejecución superescalar

44

En resumen, el modelo de ejecución superescalar se puede esquematizar de la siguiente manera.



Ejecución superescalar

45

La imagen anterior representa una ejecución superescalar típica, donde:

- Las instrucciones son captadas a partir del programa estático, y decodificadas para la detección temprana de dependencias y predicción de saltos.
- Las instrucciones, todavía ordenadas, son enviadas a la ventana de ejecución, donde se las redistribuye en función de las dependencias verdaderas, esto es, las que están en condiciones de ser ejecutadas o las que necesitan disponer de datos de otras instrucciones.
- Las instrucciones son ejecutadas en un orden basado en la disponibilidad de unidades de ejecución y en las dependencias verdaderas.

Ejecución superescalar

46

- Las instrucciones ejecutadas son nuevamente reordenadas secuencialmente, y los resultados guardados en el orden que corresponde.
- Este último paso es para:
 - Reordenar el almacenamiento de acuerdo al programa original
 - Eliminar todas las ejecuciones inservibles (aquellas que se hicieron por predicción de salto o ejecución especulativa) y que se deben descartar.
 - Mejorar la respuesta a las interrupciones.
- De esta manera, las instrucciones ejecutadas producen resultados almacenados temporalmente, hasta que se graban los resultados válidos en los registros reales.

Ejecución superescalar

47

En resumen, la ejecución superscalar involucra:

- Diferentes estrategias de captación simultánea de múltiples instrucciones.
- Lógica para determinar dependencias verdaderas entre valores de registros y mecanismos para comunicar esos valores.
- Mecanismos para iniciar o emitir múltiples instrucciones en paralelo.
- Recursos para la ejecución en paralelo de múltiples instrucciones.
- Mecanismos para entregar el estado del procesador en un orden correcto.

Ejecución superescalar

48

Se han realizado diferentes estudios para determinar la influencia de algunas técnicas empleadas en procesadores superescalares. Un estudio es el que se muestra a continuación.

Base: sin duplicación y con emisión desordenada

Ld/st: duplicación ld/st y emisión desordenada

+alu: duplicación ALU y emisión desordenada

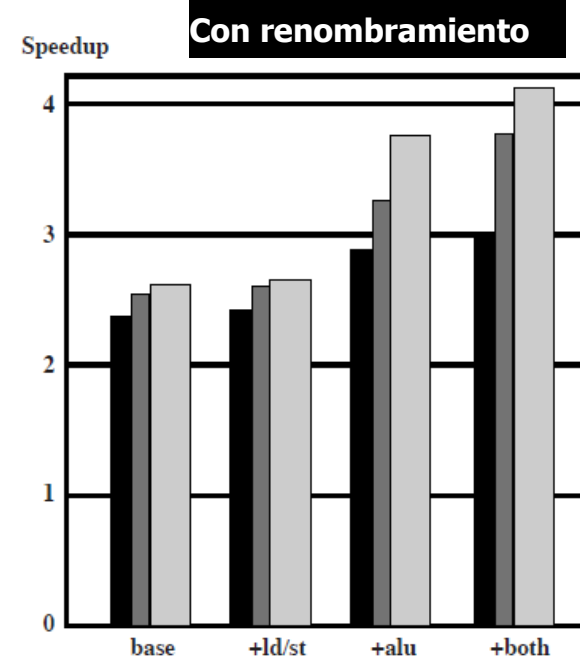
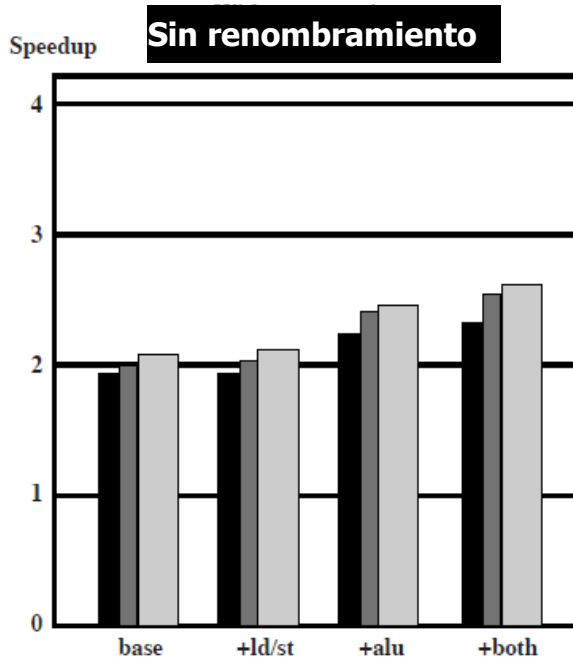
+both: duplicación ld/st y ALU y emisión desordenada

Tamaño de ventana
(instrucciones)

8

16

32



Ejecución superescalar

49

Estudios sobre ejecuciones superescalares

En el estudio anterior, se han comparado las mejoras obtenidas (o “speedup” en el eje vertical) para los siguientes casos:

- 2 situaciones: sin (gráfico de la izquierda), y con (gráfico de la derecha), renombrado de registros .
- 3 tamaños de ventanas de instrucciones: 8, 16 y 32 instrucciones.
- 4 tipos de máquinas:
 - Máquina base: sin duplicación de unidades funcionales
 - Máquina base + ld/st: duplicando las unidades de acceso a la memoria
 - Máquina base + ALU: duplicando las ALU
 - Máquina base + ld/st + ALU: duplicando las unidades de acceso a la memoria y las ALU

Ejecución superescalar

50

Conclusiones

- En la máquina sin renombrado de registros se observa que:
 - Duplicar las unidades funcionales de acceso a la memoria y las ALU (barras “both”) tiene poco impacto en la aceleración obtenida (el eje vertical pasa de 2 a 2.5).
 - Disponer de una Ventana de instrucciones mas grande (8 a 32) no modifica significativamente los resultados.
- En la máquina con renombrado de registros se observa que:
 - Duplicar las ALU y también las unidades de acceso a la memoria mejoran fuertemente la aceleración obtenida (el eje vertical pasa de 2.5 a 4).
 - El tamaño de la Ventana de instrucciones tiene un impacto significativo cuando cambia de 8 a 16 instrucciones.

Ejecución superescalar

51

Conclusiones

- La gran diferencia entre los resultados con y sin renombrado de registros se debe a un elemento fundamental en el proceso de ejecución de instrucciones:
 - En las máquinas sin renombrado de registros, todos los conflictos por las dependencias de datos (RAW, WAR, WAW) se resuelven mediante detenciones del cauce (“stall”).
 - En las máquinas con renombrado de registro se eliminan los conflictos por dependencias de salida y antidependencia (WAW, WAR) quedando únicamente la dependencia directa o verdadera (RAW).

Ejemplo procesador superescalar

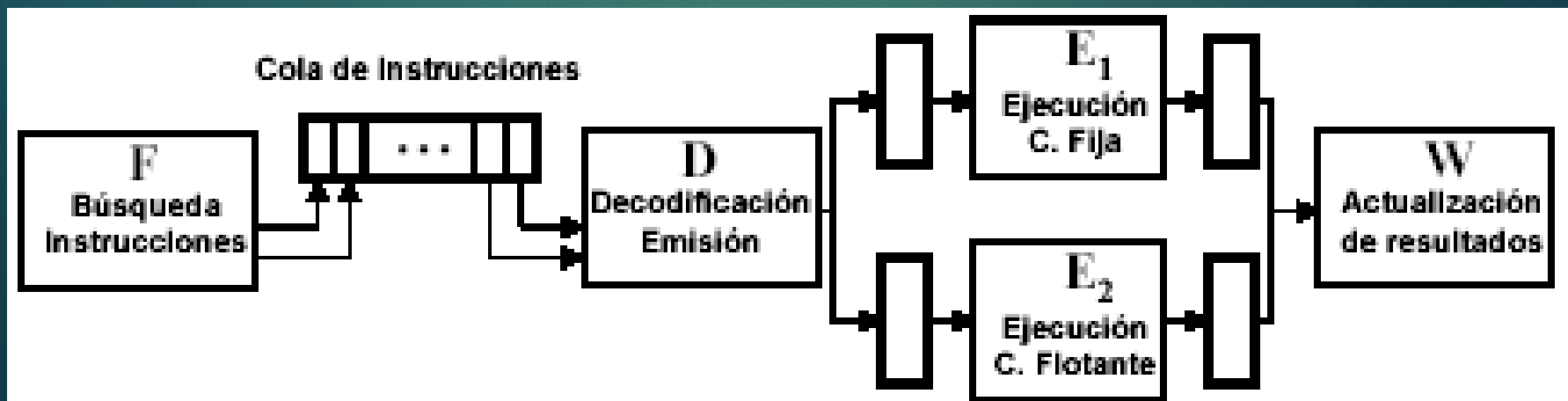
52

Un esquema de un procesador superescalar se muestra en la figura siguiente.

La unidad de búsqueda de instrucciones (F) busca y carga en un buffer (cola de instrucciones), las instrucciones leídas.

La unidad de decodificación/emisión puede tomar 2 instrucciones de la cola y decodificarlas.

Hay 2 unidades de ejecución (ALU), una de punto fijo y la otra de punto flotante.

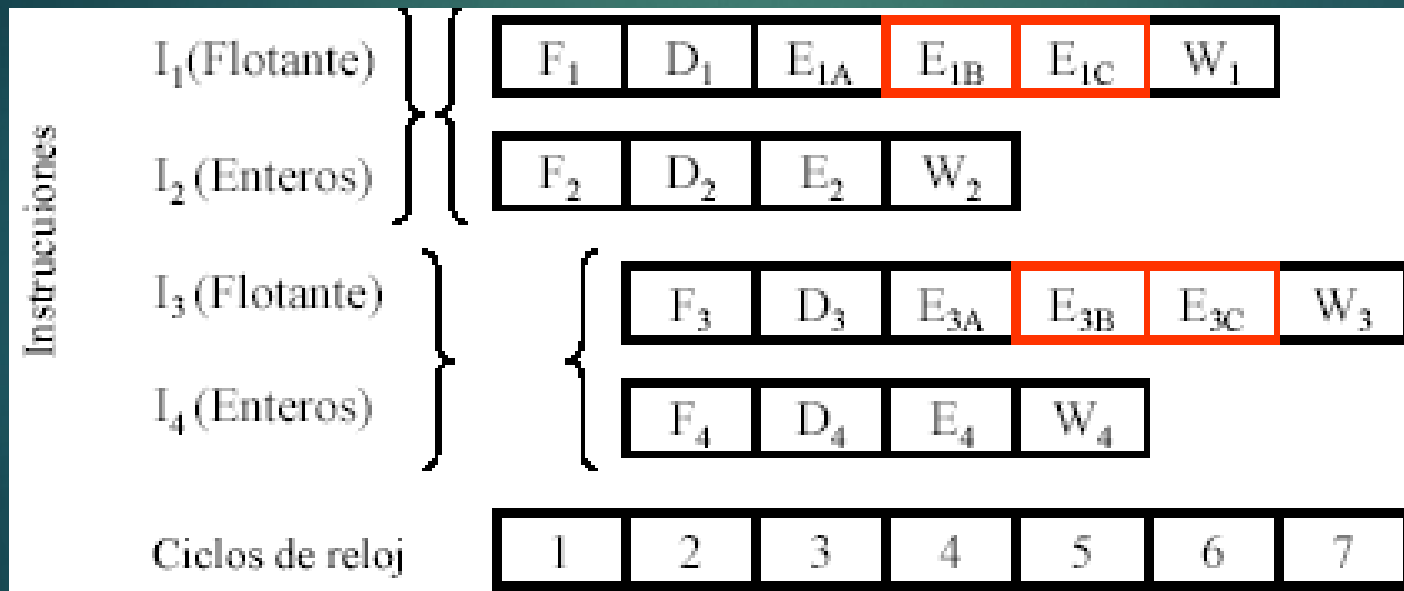


Ejemplo procesador superescalar

53

Si las 2 instrucciones que capta, una es de enteros y la otra de coma flotante (segmentada), y no existen riesgos, entonces pueden emitirse y ejecutarse a la vez.

En el diagrama siguiente, entran 2 instrucciones por ciclo:



Es responsabilidad del compilador generar el código apropiado para el máximo aprovechamiento del procesador superescalar.

Interrupciones en procesadores superescalares

54

- En los procesadores superescalares el tratamiento de interrupciones internas o excepciones (rupturas por ejecución de instrucciones) y externas (rupturas por eventos externos) es más complicado que en los procesadores convencionales.
- La razón es que en el momento en que se produce una interrupción hay varias instrucciones en ejecución (y además, desordenadas).
- El problema que se plantea es si producida una excepción en una instrucción dada I2, en qué estado quedaron las instrucciones anteriores (por ejemplo I1) y posteriores (por ejemplo I3) dado que con la política de ejecución desordenada pudieron haber quedado en distintos estados.

Interrupciones en procesadores superescalares

55

Interrupciones internas o excepciones

Existen 2 estrategias para el tratamiento de excepciones:

- Excepciones precisas: las instrucciones que preceden a la que produjo la excepción se completan, y las que le suceden se reinician. Es decir, el comportamiento es “idéntico” al que tendría la misma computadora no segmentada
- Excepciones imprecisas: no se respetan las condiciones exactas de la máquina no segmentada.

Interrupciones en procesadores superescalares

Interrupciones internas o excepciones

- Para garantizar un estado consistente (preciso) se requiere:
 - Instrucciones anteriores terminan correctamente
 - La que origina la excepción y siguientes se abortan
 - Tras la rutina de tratamiento se comienza por la que originó la excepción

Es decir:

Ejemplo: Fallo de página.



Interrupciones en procesadores superescalares

57

Interrupciones internas o excepciones

- Para que las interrupciones sean precisas los resultados se deben almacenar en el orden en que aparecen las instrucciones.
- Para ello, se requiere retardar las escrituras de los resultados de tal manera que queden en el orden en el que estaba el programa secuencial.

Interrupciones en procesadores superescalares

Interrupciones externas

- Las interrupciones externas pueden tener distinto grado de “precisión”, dado que no necesariamente tienen que ver con las instrucciones en ejecución (por ejemplo operaciones de I/O).
- **Interrupciones imprecisas:** se completa lo que estaba en ejecución.
- **Interrupciones “algo” imprecisas:** el software (la rutina de servicio de la interrupción) resuelve algunas inconsistencias.
- **Interrupciones precisas:** solo se completan las instrucciones previas a la interrupción, para lo cual se requiere implementar la escritura con buffer de salida. La unidad de emisión deja de emitir, se cancela la cola, todas las instrucciones pendientes se completan, y comienza el procesamiento de la interrupción.

Interrupciones en procesadores superescalares

59

Conclusiones

En general se tiene:

- En los sistemas con finalización desordenada:
 - Excepciones imprecisas
 - Mayor performance
- En los sistemas con finalización ordenada (es decir, con buffer de reordenamiento):
 - Excepciones precisas
 - Menor performance

Referencias

- Organización y Arquitectura de Computadoras, William Stallings, Capítulo 13 de 5ta edición ó Capítulo 14 de 7ma edición.

Arquitectura de Computadoras

CURSO 2022

Turno:
Clase 9

Resumen clase 9

2

Procesamiento paralelo

- Procesamiento paralelo
- Taxonomía de Flynn
- Multiprocesadores simétricos SMP
- Multiprocesadores NUMA
- Cluster
- Tipos de acceso a memoria en sistemas multiprocesadores
- Arquitecturas on-chip
- Procesamiento multihebra (Multithreading)

Procesamiento paralelo

3

Introducción

- La demanda de máquinas de mayor rendimiento es una exigencia que surgió desde la aparición de las primeras computadoras, y continúa en forma permanente.
- Existen 2 caminos para aumentar la capacidad de procesamiento:
 1. Mejorar el rendimiento de una máquina con un solo procesador.
 2. Disponer de sistemas con varios procesadores.

Procesamiento paralelo

4

Introducción

1. **Mejorar el rendimiento de una máquina con un solo procesador requiere:**
 - Explotar el paralelismo a nivel instrucción (ILP).
 - Optimizar la detección del paralelismo, a nivel de hardware (MPL).
2. **Mejorar la capacidad de procesamiento con varios procesadores, requiere:**
 - Explotar el paralelismo a nivel proceso.
 - Detección del paralelismo a nivel de sistema operativo, compilador, o programación.

Taxonomía de Computadoras

5

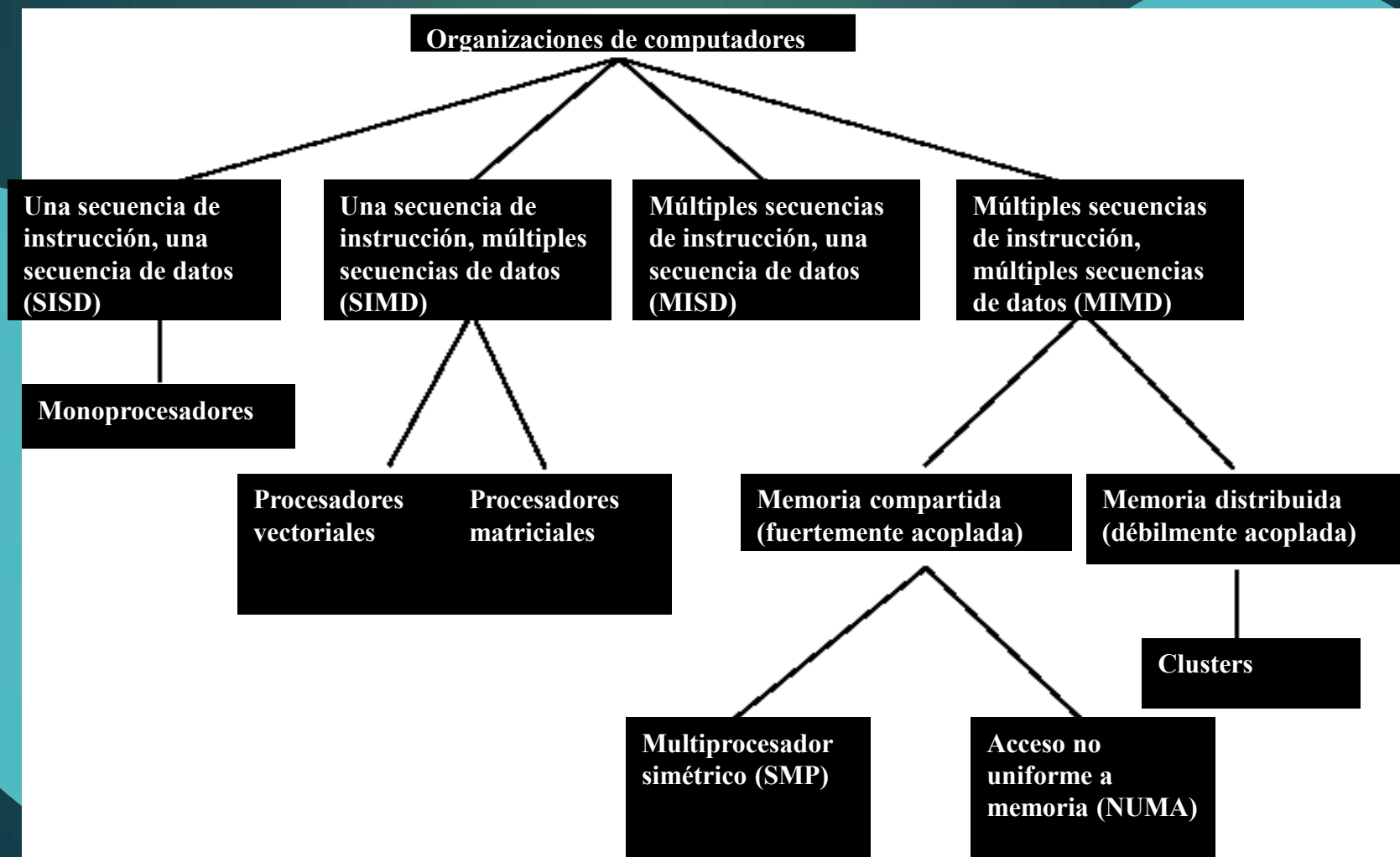
Taxonomía de Flynn

Existen varias formas distintas de clasificar los Sistemas de cómputo. Una de ellas se basa en determinar las cantidades de flujos de instrucciones y datos que se pueden transferir simultáneamente. En base a este concepto se tienen 4 tipos básicos de máquinas:

- SISD: una secuencia de instrucciones, una secuencia de datos.
- SIMD: una secuencia de instrucciones, múltiples secuencias de datos.
- MISD: múltiples secuencias de instrucciones, una secuencia de datos.
- MIMD: múltiples secuencias de instrucciones y múltiples secuencias de datos.

Taxonomía de Flynn

La imagen siguiente muestra los diferentes categorías de Computadoras de acuerdo a la taxonomía de Flynn.

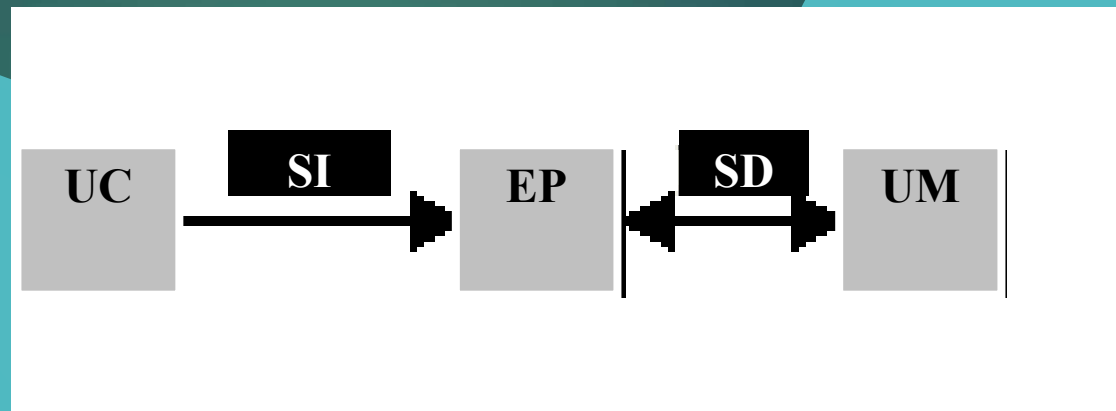


Taxonomía de Flynn

7

Máquinas SISD

Una máquina tipo **SISD (Single Instruction – Single Data)**, según Flynn, tiene la siguiente estructura funcional.



donde:

- **UC:** Unidad de control (captura de instrucciones)
- **EP:** Elemento de Proceso (ejecuta)
- **UM:** Unidad de Memoria (almacenamiento de datos)
- **SI:** secuencia de instrucciones
- **SD:** secuencia de datos

Taxonomía de Flynn

8

Máquinas SISD

Es un sistema básicamente compuesto por:

- una Unidad de Control (UC)
- un elemento de proceso (EP)
- una Unidad de Memoria (UM).

La UC interpreta (lee) una única secuencia de instrucciones (SI), que el EP resuelve sobre una única secuencia de datos (SD) proveniente de la UM.

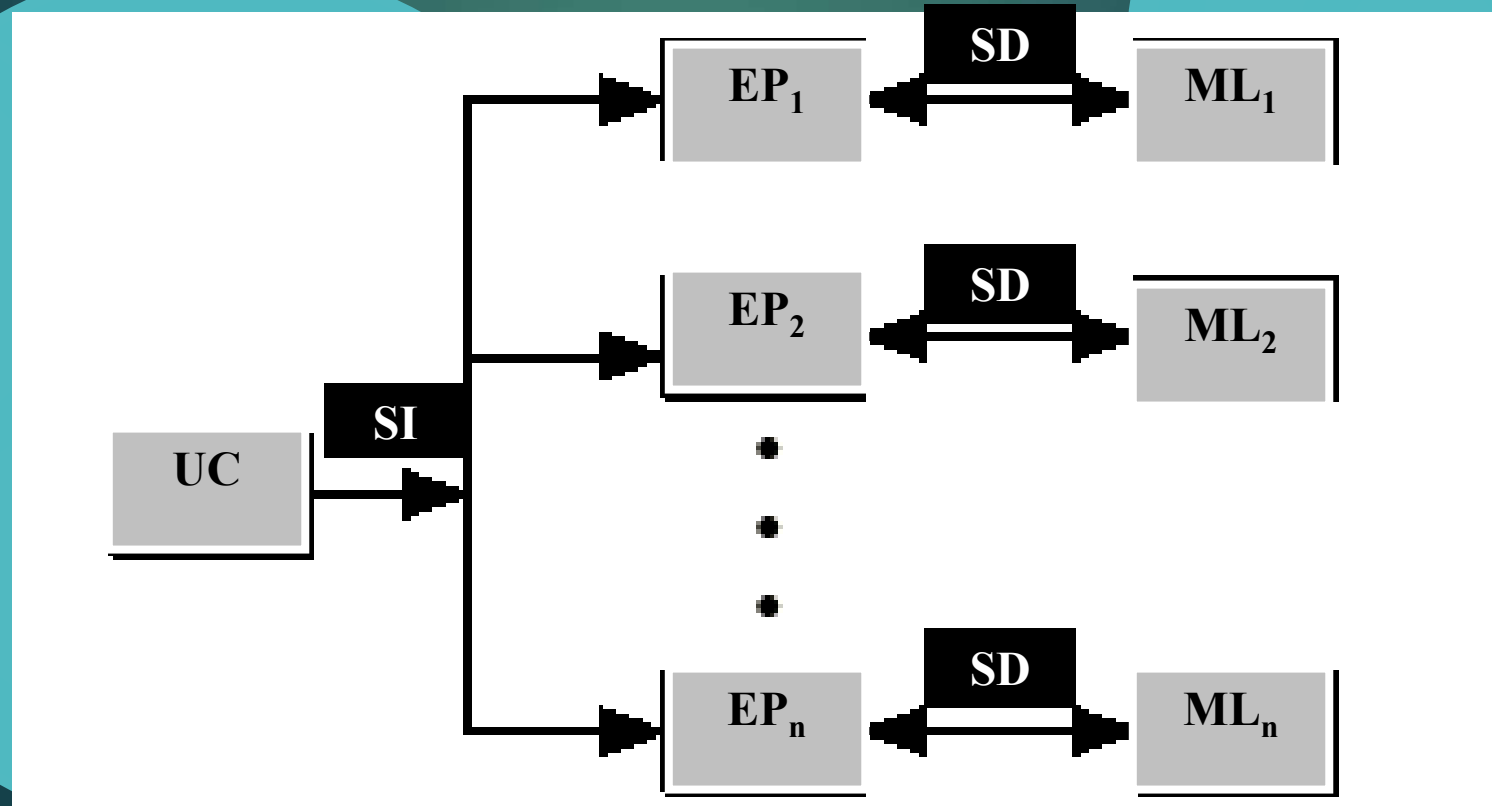
- Ejemplo máquina SISD: Computadoras monoprocesador.

Taxonomía de Flynn

9

Máquinas SIMD

La imagen siguiente muestra el esquema de una máquina tipo **SIMD (Single Instruction – Multiple Data)**.



Taxonomía de Flynn

10

SIMD

Es un sistema compuesto por:

- una Unidad de Control (UC).
- una matriz de elementos computacionales o elementos de proceso (EP).
- Una Unidad de memoria local (ML) por cada EP.

La UC interpreta una única secuencia de instrucciones (SI) que múltiples EP resuelven simultáneamente cada uno sobre una secuencia de datos (SD) proveniente de su propia ML.

Taxonomía de Flynn

11

Máquinas SIMD

Cada instrucción es ejecutada por varios elementos de proceso (EP), cada uno con sus propios datos (ML).

➤ Ejemplo máquina SIMD: Procesadores vectoriales y matriciales.

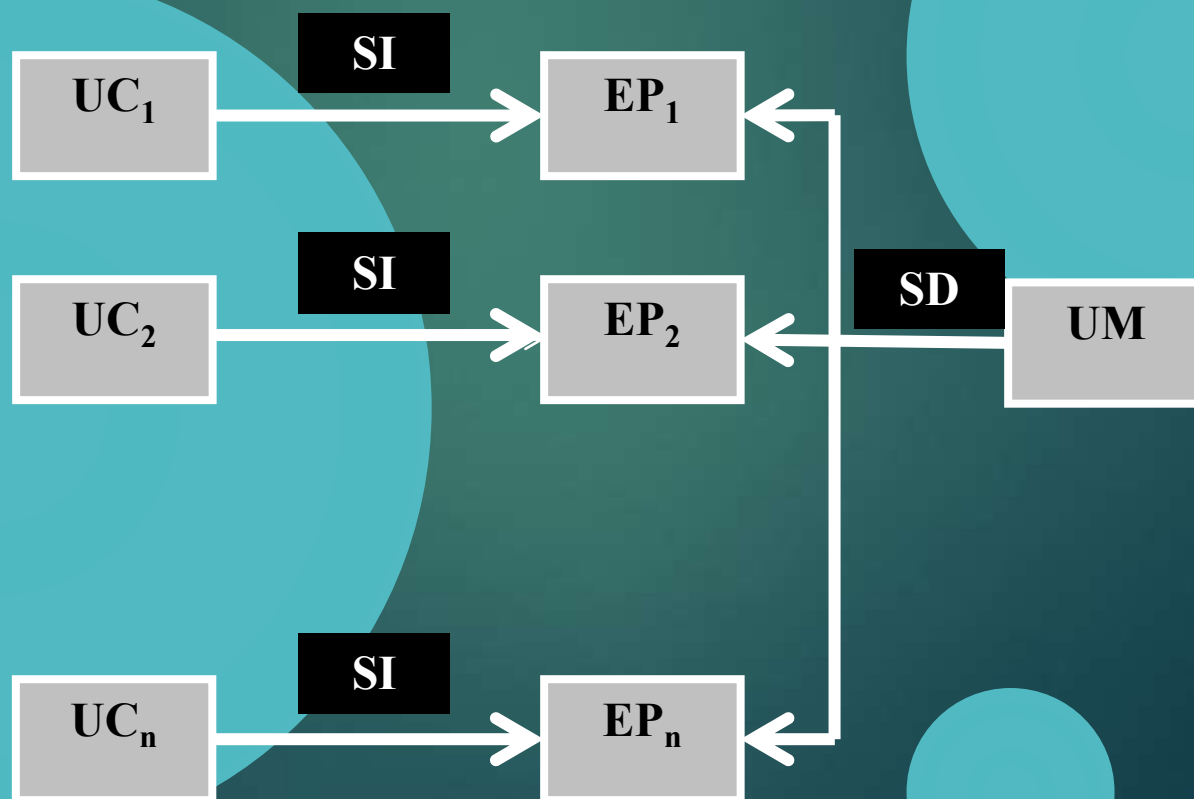
En estas máquinas se requiere disponer de:

- Un conjunto ampliado de instrucciones (además de las “convencionales” o escalares) para permitir manejar operaciones vectoriales. Es decir, el repertorio de instrucciones incluye operaciones de suma, almacenamiento, multiplicación, etc., de vectores, para poder asignar a los EPs.
- Se deben agregar instrucciones para transferir datos entre EPs, que es un atributo típico de un “lenguaje paralelo”.

Taxonomía de Flynn

Máquinas MISD

La imagen siguiente muestra el esquema de una máquina tipo MISD (Multiple Instruction – Single Data).



Taxonomía de Flynn

13

Máquinas MISD

Es un sistema compuesto por:

- una matriz de elementos computacionales o elementos de proceso (EP) cada uno con su propia Unidad de Control (UC).
- Una Unidad de memoria (UM) para todos los EP.

Cada UC interpreta una secuencia de instrucciones (SI) que cada EP resuelve simultáneamente sobre una única secuencia de datos (SD) proveniente de una sola memoria.

Taxonomía de Flynn

14

Máquinas MISD

Se transmite una única secuencia de datos (SD) a un conjunto de procesadores cada uno ejecutando su propia secuencia de operaciones (MI).

Cada procesador ejecuta una secuencia de instrucciones diferente.

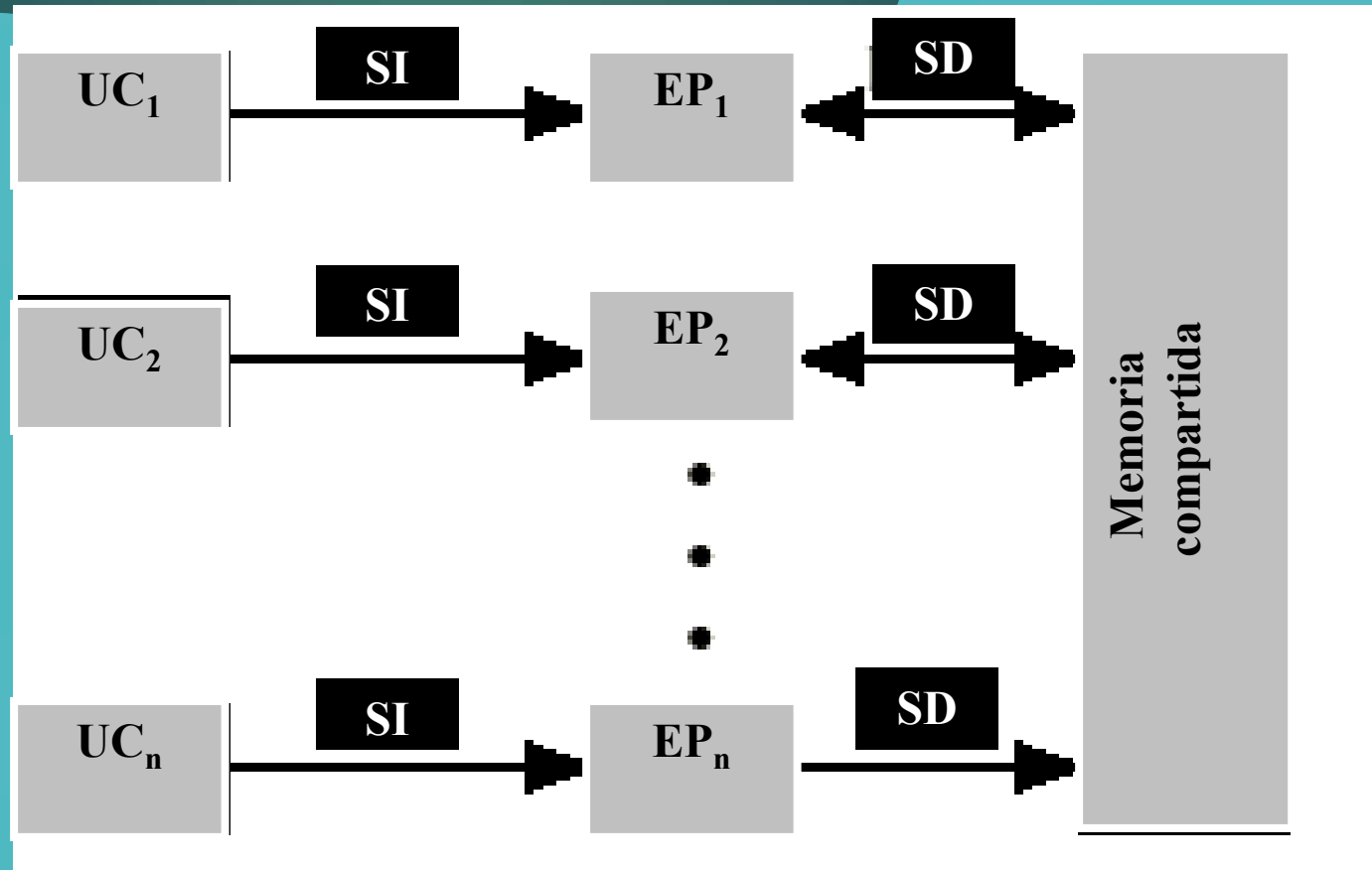
- Ejemplo: es un modelo teórico, no hay registros de máquinas de este tipo (hay algunas de características similares pero no cumplen los requisitos para ser exactamente tipo SIMD).

Taxonomía de Flynn

15

Máquinas MIMD

Existen 2 modelos de Computadoras tipo **MIMD** (Multiple Instruction – Multiple Data). Un modelo es **MIMD de Memoria compartida**.

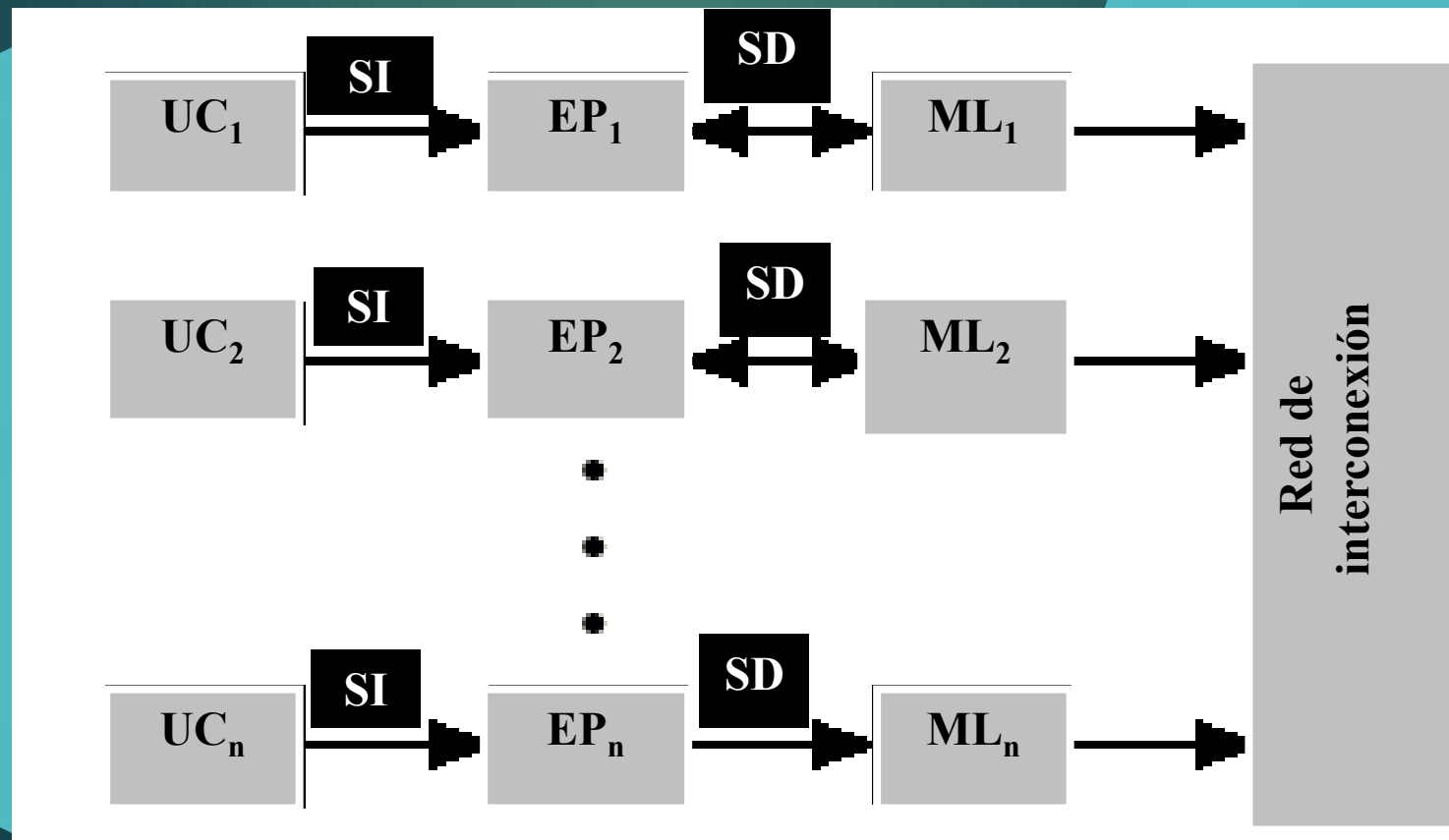


Taxonomía de Flynn

16

Máquinas MIMD

El segundo tipo de Computadoras tipo MIMD es MIMD de Memoria distribuida.



Taxonomía de Flynn

17

MIMD

Es un sistema compuesto por:

- Múltiples procesadores (UC), ejecutando
- Múltiples secuencias de instrucciones (MI), sobre
- Múltiples secuencias de datos (MD).

Se pueden dividir según la organización de la Memoria (que es también la forma de comunicarse) en:

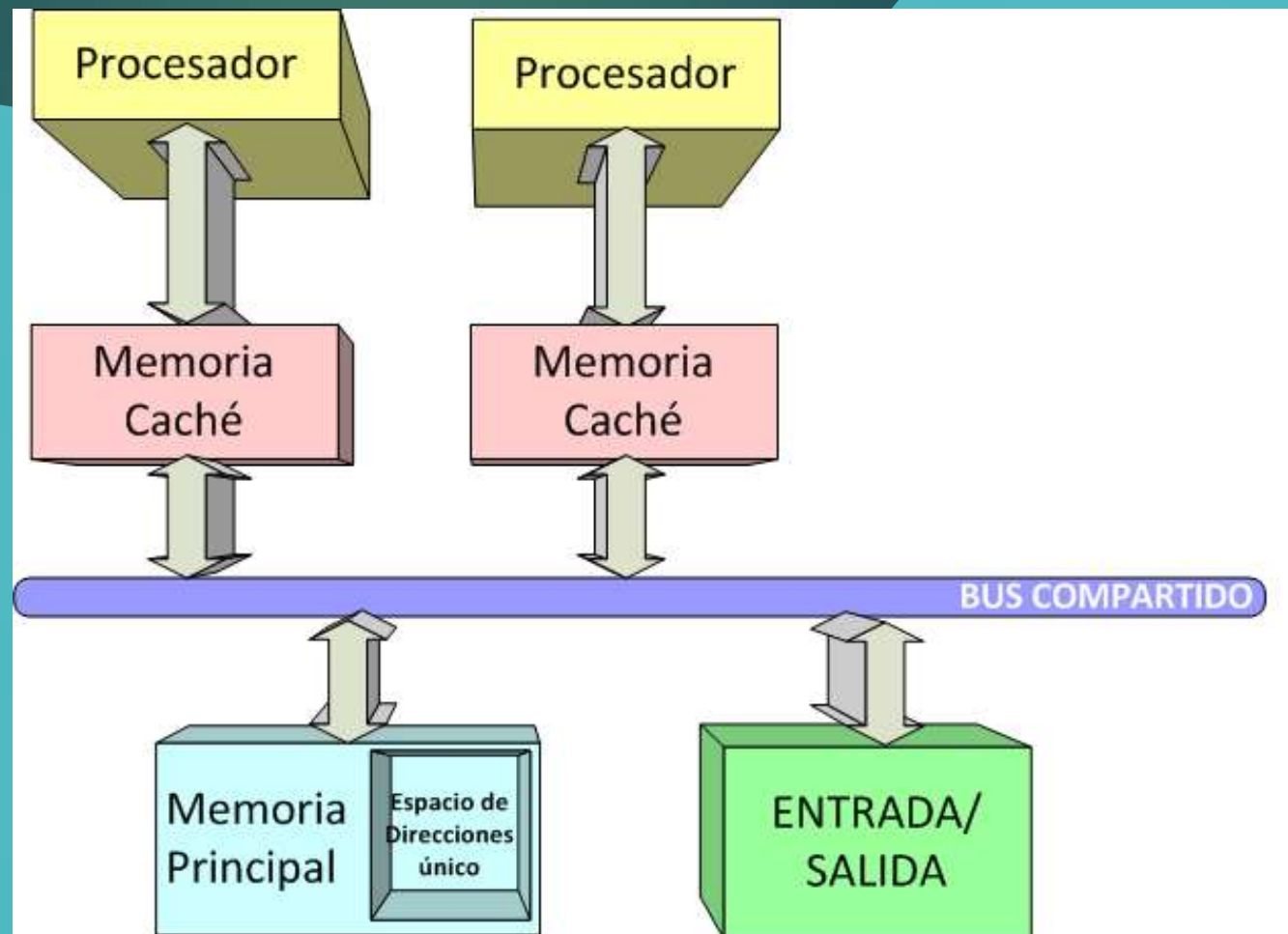
- MIMD de Memoria compartida, con 2 variantes:
 - SMP (multiprocesadores simétricos)
 - Sistemas NUMA
- MIMD de Memoria distribuida, comúnmente llamados Clusters.

Multiprocesadores Simétricos SMP

18

Multiprocesadores SMP

El modelo de computadora tipo SMP se muestra en la figura siguiente.



Multiprocesadores Simétricos SMP

19

Es un sistema con las siguientes características principales:

- Dos o más procesadores idénticos (o muy similares) de capacidades comparables (homogéneos).
- Memoria principal y E/S única (compartida) por todos los procesadores.
- Interconexión mediante un bus u otro tipo de medio similar (“fuertemente acoplados”).
- Igual tiempo de acceso a la memoria para todos los procesadores. Por eso se los identifica como del tipo “UMA”: Uniform Memory Access.
- Todos los procesadores pueden desempeñar las mismas funciones.
- Sistema operativo integrado, que proporciona la interacción entre los procesadores y sus programas.

Multiprocesadores Simétricos SMP

20

Ventajas

- **Mayores prestaciones:** en general tienen buenos resultados, si las tareas pueden organizarse en paralelo.
- **Buena disponibilidad:** un fallo en un procesador no detiene la operación del sistema, dado que todos los procesadores pueden hacer las mismas tareas.
- **Crecimiento:** pueden aumentarse las prestaciones añadiendo más procesadores, pero hay restricciones a este mecanismo.
- **Escalado:** normalmente limitado, en función de la cantidad de procesadores.

Multiprocesadores Simétricos SMP

21

Desventajas

Los principales problemas se originan en los conflictos por el bus compartido, la coherencia y consistencia de los datos, y la sincronización de tareas entre procesadores.

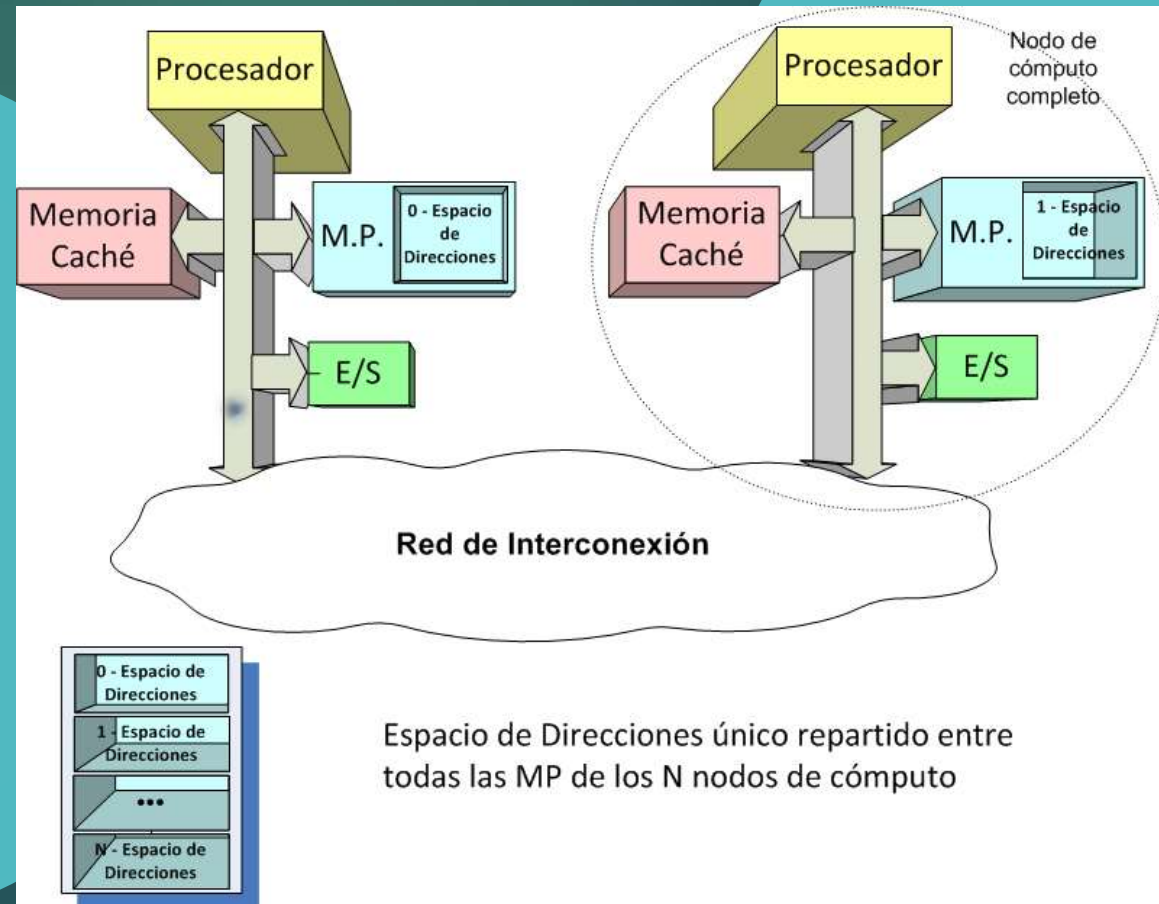
- Las prestaciones están limitadas por el tiempo de ciclo del bus.
- Cada procesador está equipado con una memoria cache, que reduce los accesos a memoria y mejora las prestaciones. Pero eso trae un problema.
- Al disponer una caché en cada procesador, se pueden producir problemas de coherencia de cache (datos que pueden estar en más de una caché). Por razones de velocidad, este problema debe ser resuelto por el hardware. Se requieren usar protocolos para la administración de los datos en las caché (protocolos de coherencia tipo sondeo o snoopy).

Multiprocesadores NUMA

22

Multiprocesadores con memoria compartida y distribuida - NUMA

El modelo de computadora tipo NUMA se muestra en la figura siguiente.



Multiprocesadores NUMA

23

Es un sistema con las siguientes características principales:

- Dos o más procesadores idénticos (o muy similares) de capacidades comparables (homogéneos) formando “nodos”.
- Cada nodo es un procesador completo con su propia memoria local y E/S. Los nodos tienen características muy similares.
- Los nodos están interconectados mediante una red de interconexión.
- El espacio de direcciones (memoria lógica) es común a todos los procesadores, por eso es un sistema de memoria compartida. Pero el tiempo de acceso al espacio de memoria es distinto para el acceso a la memoria local que para el acceso a las memorias de los otros nodos.
- Por eso se los llama NUMA: Non Uniform Memory Access.

Multiprocesadores NUMA

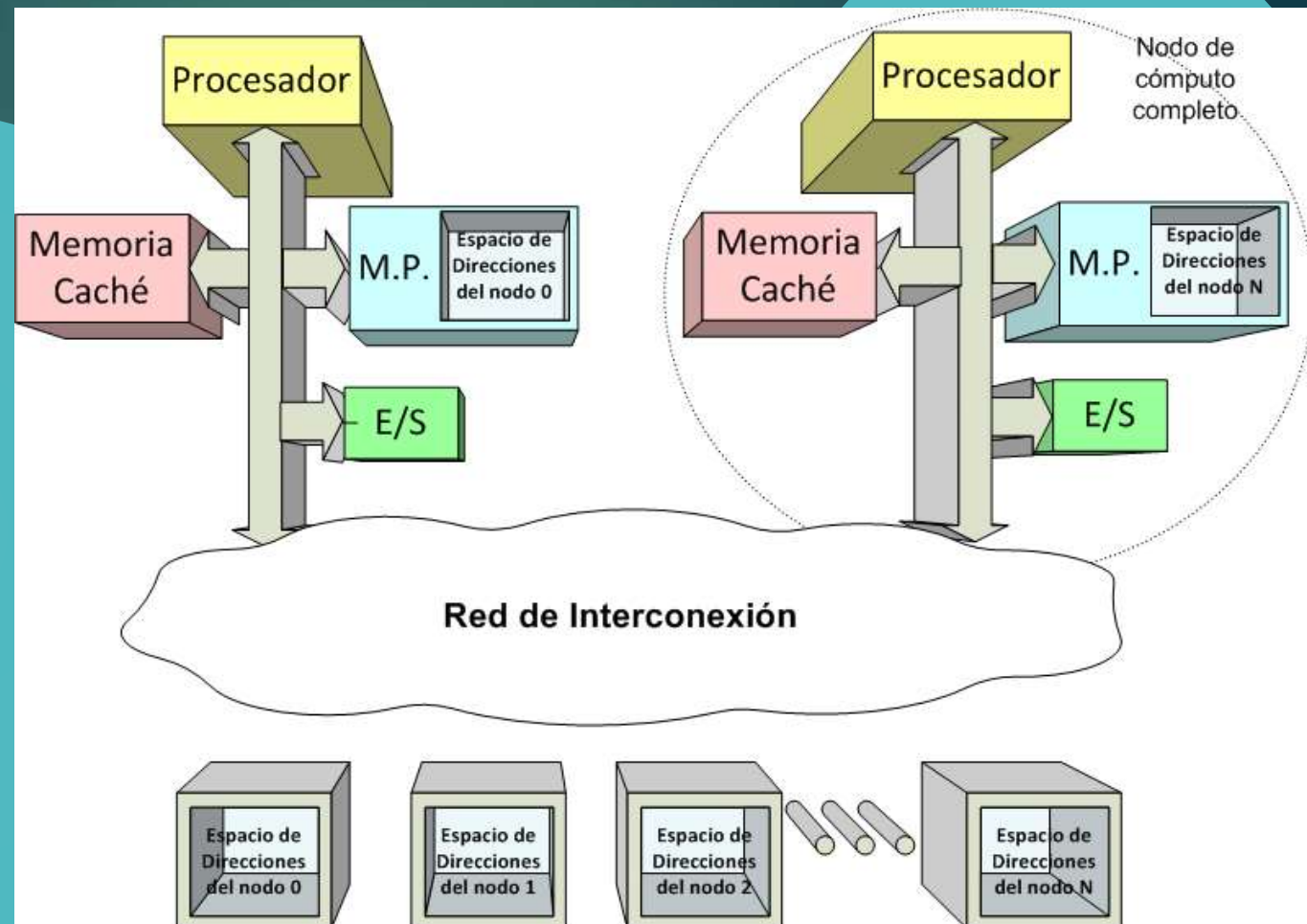
24

- Todos los procesadores pueden desempeñar las mismas funciones.
- El Sistema operativo está integrado, y proporciona la interacción entre los procesadores y sus programas.

Cluster

Multiprocesadores de memoria distribuida - Cluster

El modelo de una arquitectura tipo Cluster se muestra en la figura siguiente.



Cluster

26

Es un sistema con las siguientes características principales:

- Compuesto por 2 o más nodos.
- Cada nodo es un procesador completo con su propia memoria local y E/S. Los nodos pueden tener características similares (“homogéneos”) o distintas (“heterogéneos”).
- Interconectados mediante una red de interconexión. Debido al tipo de red y protocolo de comunicación se los considera del tipo “levemente acoplados”.
- Como los espacios de direcciones son independientes es un sistema de Memoria distribuida.

Cluster

27

- Todos los procesadores pueden desempeñar las mismas funciones.
- El Sistema operativo está integrado, y proporciona la interacción entre los procesadores y sus programas.
- La comunicación entre procesos es en base a mensajes (a resolver por el programa).

Cluster

- Los Cluster, básicamente, son computadoras completas, interconectadas, que trabajan conjuntamente como un único recurso.
- Es decir, para las tareas en ejecución se comportan como si fuera una única máquina.
- Como ya se dijo, cada computadora se denomina “nodo”.
- En general, presentan prestaciones y disponibilidad elevadas.
- Las aplicaciones son propias de un servidor, y constituyen una alternativa a los SMP.

Cluster

29

Ventajas

- **Escalabilidad absoluta:** dependiente de la cantidad de nodos incorporados se puede disponer de mayores prestaciones .
- **Escalabilidad incremental:** posibilidad de agregar nuevos nodos fácilmente.
- **Alta disponibilidad:** capacidad de seguir operando con nodos en falla.
- **Mejor relación precio/prestaciones:** porque se usan equipos de cómputo estandar (y, posiblemente, de bajo costo).

Cluster vs SMP

30

SMP

- Permiten dar soporte a aplicaciones de alta demanda de recursos.
- Disponibles comercialmente (SMP es más antiguo).
- Más fácil de administrar y configurar.
- Cercano a los sistemas de un solo procesador.
- La planificación (scheduling) es la diferencia principal
- Menos espacio físico / Menor consumo de potencia
- SMP tiene límite práctico en su número de procesadores: entre 16 y 64 por degradación de prestaciones.

Cluster vs SMP

Cluster

- Permiten dar soporte a aplicaciones de alta demanda de recursos.
- Disponibles comercialmente (SMP es mas antiguo).
- Superior escalabilidad incremental y absoluta
- Superior disponibilidad.
- Mayor redundancia
- En Clusters cada nodo tiene su propia memoria principal. Así las aplicaciones no 'ven' la memoria global.
- La coherencia es mantenida por software y no por hardware.
- Alternativa a sistemas SMP para brindar multiprocesamiento a gran escala. Por ejemplo, el SGI Origin de Silicon Graphics es un NUMA con 1024 procesadores MIPS R10000.

TIPOS DE ACCESOS A MEMORIA:

UMA, NUMA, CC-NUMA

32

Los sistemas multiprocesadores se pueden clasificar de acuerdo al tipo de acceso a memoria en:

1. UMA - Uniform memory access

- Igual tiempo de acceso a todas las regiones de memoria.
- Igual tiempo de acceso a memoria para los diferentes procesadores.

Ej: SMP

TIPOS DE ACCESOS A MEMORIA:

UMA, NUMA, CC-NUMA

33

2. NUMA - Non-uniform memory access

- El tiempo de acceso de un procesador difiere, dependiendo de la región de memoria que accede.
- Diferentes procesadores acceden a diferentes regiones de memoria a diferentes velocidades.

Ej: Cluster

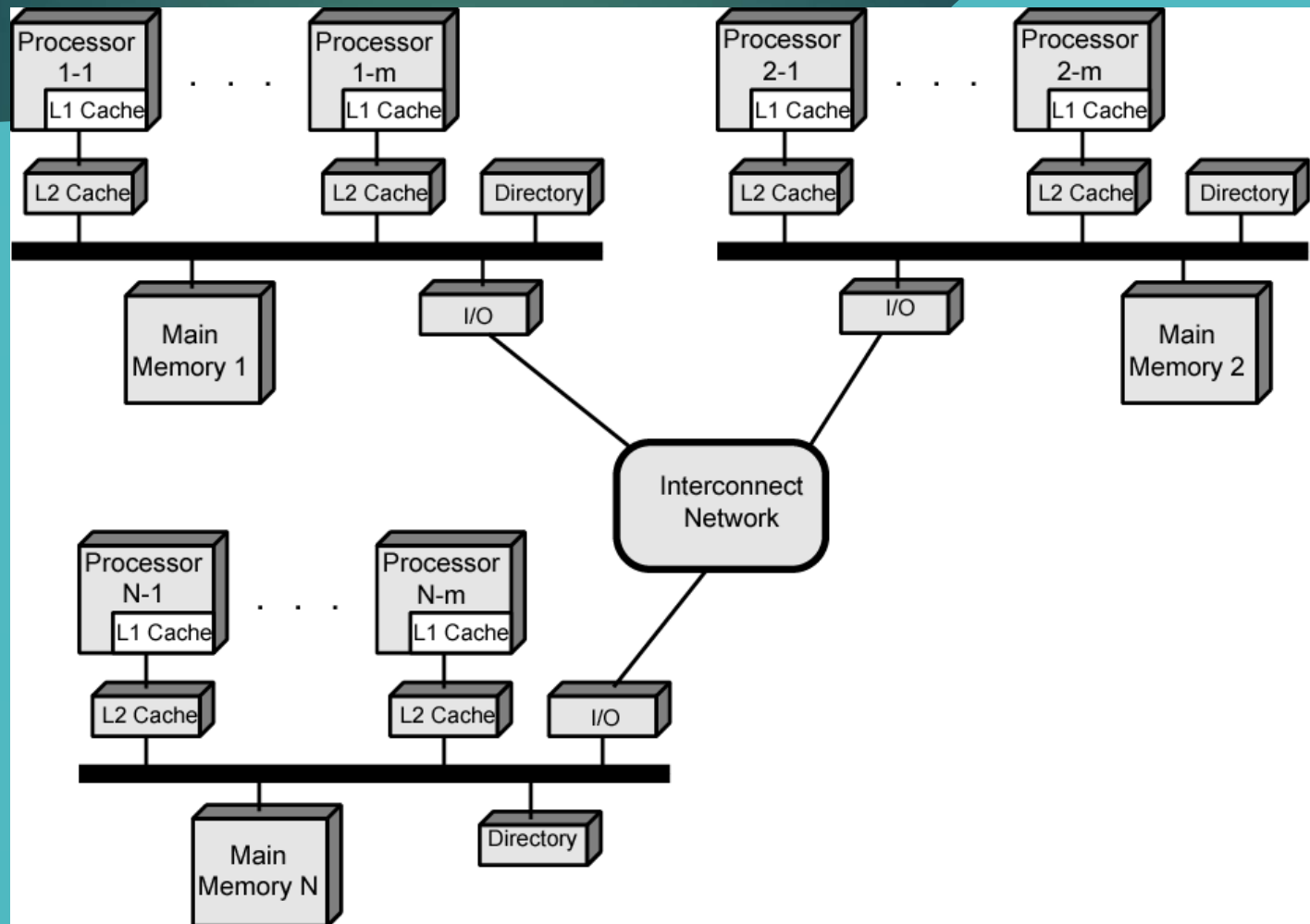
3. CC-NUMA - cache coherente NUMA

- Es un NUMA que mantiene coherencia de cache entre las cache de los distintos procesadores.

Ej: Sistemas con memoria compartida distribuida.

Sistemas CC-NUMA

En la imagen siguiente se muestra el **modelo de organización de un sistema NUMA con coherencia en las caché (CC-NUMA)**.



Sistemas CC-NUMA

35

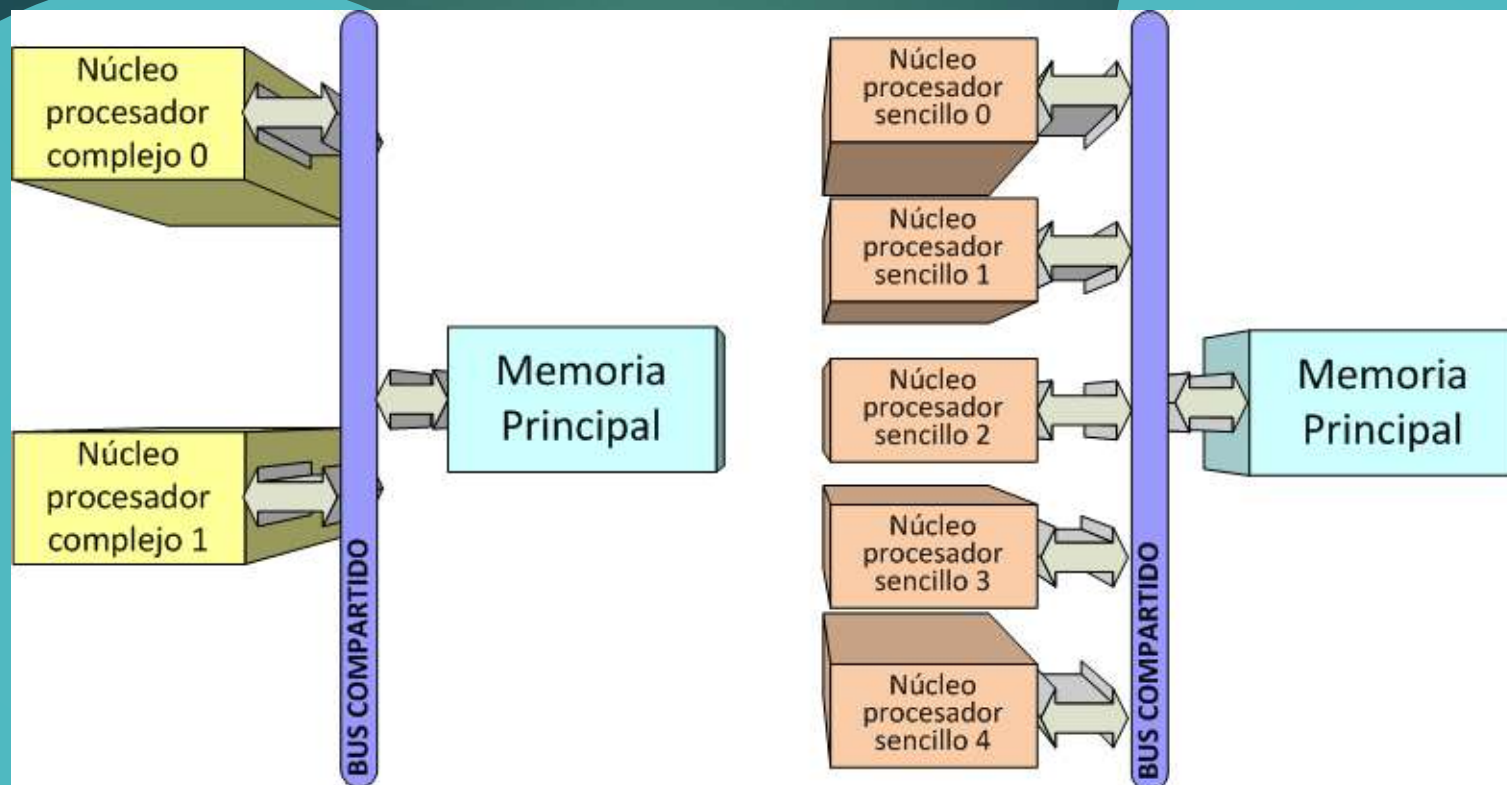
Los sistemas CC-NUMA presentan las siguientes características:

- Cada nodo tiene 2 o más procesadores (por ejemplo un SMP), cierta cantidad de memoria (principal) y E/S.
- Cada procesador tiene su cache (típicamente L1 y L2).
- Los nodos están interconectados por algún tipo de red.
- Existe un espacio de direcciones de memoria único para todos los procesadores de todos los nodos.
- Orden de acceso a memoria:
 - cache L1 (local al procesador).
 - cache L2 (local al procesador).
 - Memoria principal (local al nodo).
 - Memoria remota (petición por red).
- La coherencia se mantiene en forma automática y transparente

Arquitecturas on chip con Memoria compartida

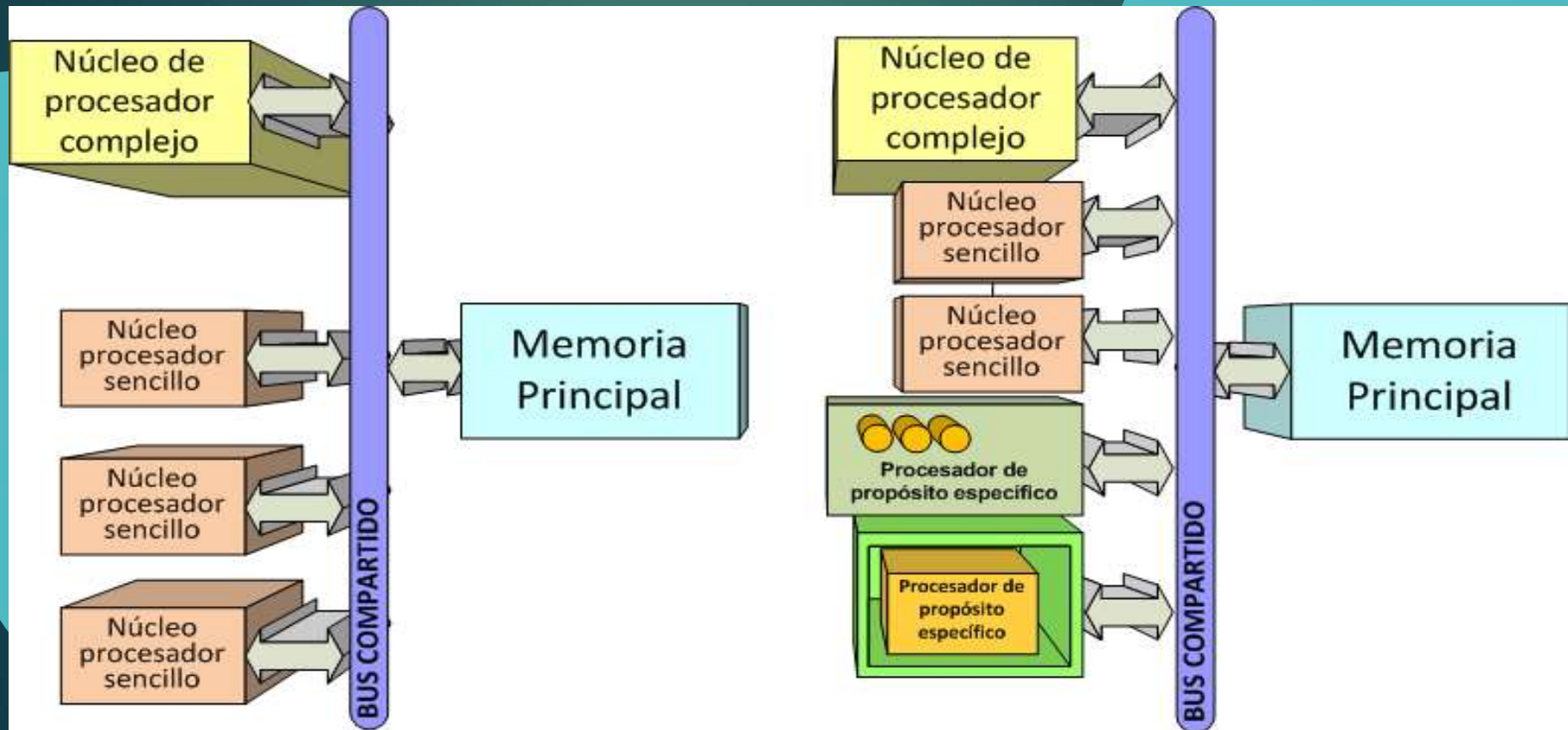
Las arquitecturas on-chip con memoria compartida pueden ser homogéneas o heterogéneas.

Son homogéneas si todos los procesadores son idénticos.



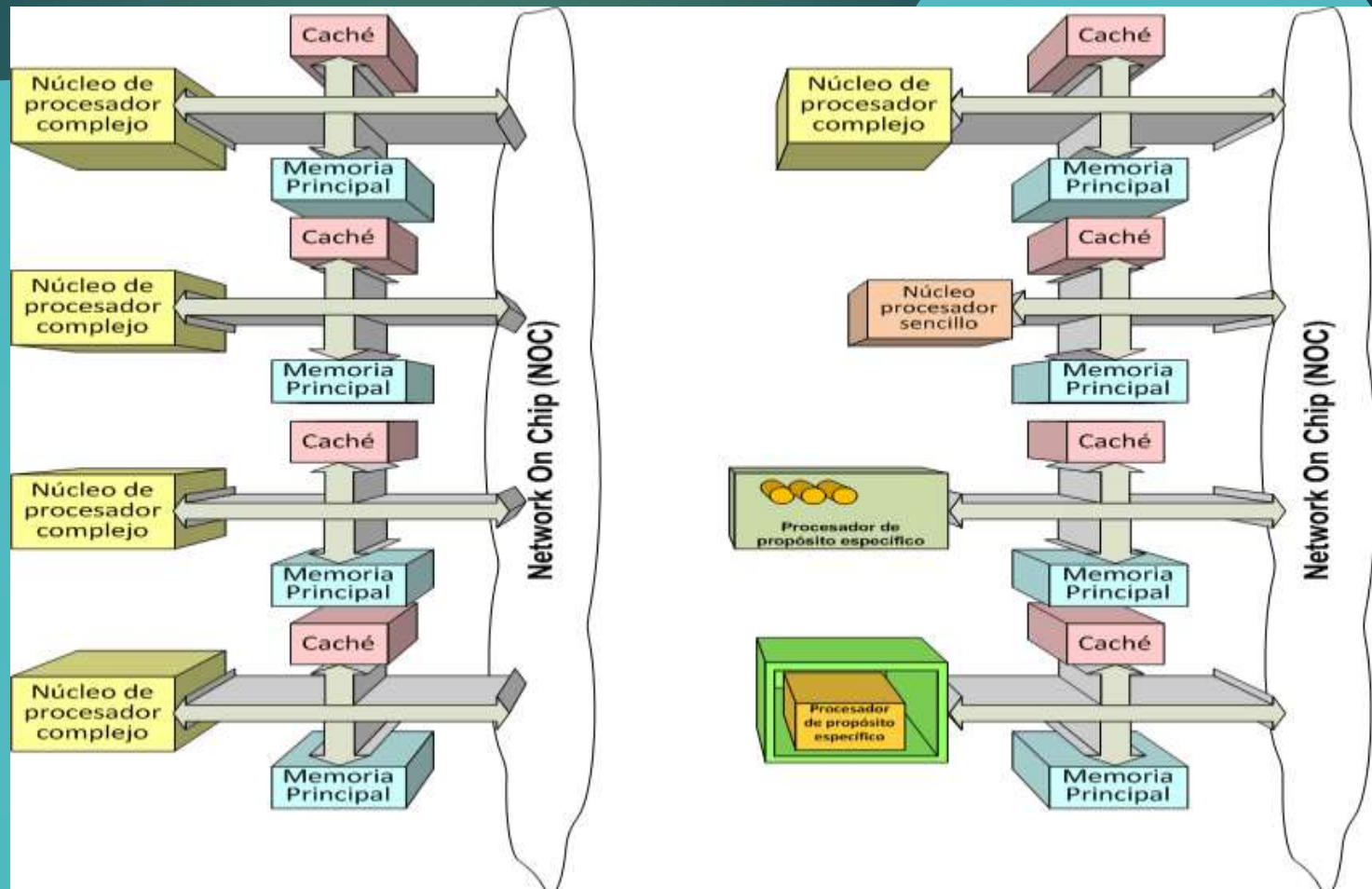
Arquitecturas on chip con Memoria compartida

Son heterogéneas si los procesadores tienen distintas prestaciones



Arquitecturas on chip con Memoria distribuida

Las arquitecturas on chip con memoria distribuida también pueden ser homogéneas o heterogéneas.



Procesamiento Multihebra

39

Proceso

Un Proceso es un programa ejecutándose (“corriendo”) en un Sistema que:

- Es “propietario” de recursos propios.
- Maneja un espacio de direcciones virtuales para almacenar la imagen de un proceso (code, data, stack, etc) con:
 - Información propia del estado del proceso.
 - Planificación/ejecución por el sistema operativo.
- La comunicación entre procesos es a través de mecanismos específicos.
- La conmutación entre procesos (“Process switch”) requiere un cambio de los recursos asignados (mediante mecanismos precisos y complejos).

Procesamiento Multihebra (Multithreading)

Hebra (thread) o hilo

Es una unidad de trabajo que:

- Tiene su propio contexto de procesador (incluido PC y SP) y área de datos para su pila (stack).
- Comparte con otras hebras, código, variables globales (espacio de memoria), archivos abiertos por el proceso al que pertenecen, etc.
- Se ejecuta secuencialmente.
- Relativamente fácil de interrumpir, el procesador cambia a otra hebra rápidamente si lo necesita.
- La conmutación de hebra ("Thread switch") es un cambio de control del procesador entre hebras de un mismo proceso.
- Un Proceso está compuesto de muchas Hebras o Hilos.

Procesamiento Multihebra (Multithreading)

Se pueden aumentar las prestaciones de un Sistema en 2 niveles:

- Nivel de instrucciones (IPL): aumentando la cantidad de instrucciones ejecutadas en paralelo, explotando el paralelismo de instrucciones secuenciales.
- Nivel de hebras (TPL): aumentando el nivel de hebras en paralelo, explotando el paralelismo entre instrucciones pertenecientes a distintos hilos de ejecución. Esto involucra el sistema operativo, el compilador y el hardware.

Procesamiento Multihebra (Multithreading)

Como ya se vió en clases anteriores, el IPL explota el paralelismo a nivel de instrucciones, pertenecientes a un hilo o hebra de ejecución. Si el programa no tiene mucho paralelismo, el resultado va a ser pobre.

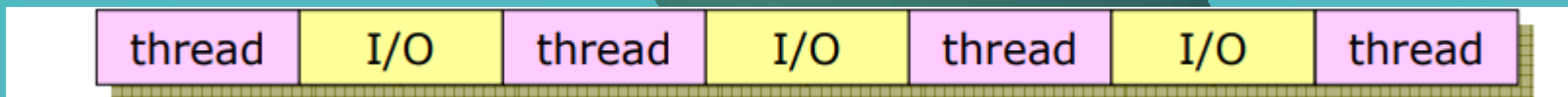
Un proceso puede tener muchos hilos o hebras. Entonces:

- Las instrucciones de diferentes hebras pueden ser paralelizadas.
- Se pueden mejorar las prestaciones generales si se ejecutan más de una hebra en paralelo (simultáneamente).
- La idea es que si hay recursos libres durante la ejecución de un hebra, ejecutando varias hebras se puede hacer un mejor aprovechamiento de los recursos.
- No se mejora el tiempo de ejecución de un hebra.

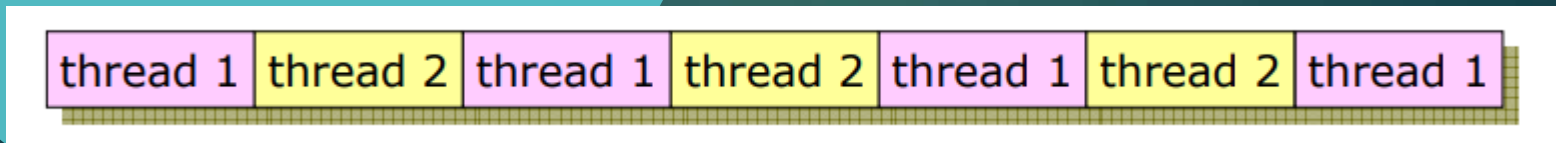
Procesamiento Multihebra (Multithreading)

El procesamiento de multiples hebras (multithreading) consiste en ejecutar 2 o más hebras en forma concurrente.

Un ejemplo claro es el siguiente. Se está ejecutando 1 hebra únicamente, y el procesador queda inactivo durante los períodos de tiempo donde se ejecutan acciones de I/O.



Pero si se pueden ejecutar múltiples hebras, durante los períodos de I/O puede ejecutar otras hebras.



Procesamiento Multihebra (Multithreading)

La ejecución de múltiples hebras puede ser de 2 tipos:

- Multihebras explícitas: cuando son definidas en el programa, tanto a nivel de usuario (aplicaciones) como a nivel de Sistema operativo (núcleo).
- Multihebras implícitas: cuando son definidas por el compilador (estático) o por hardware (dinámico).

Para poder implementar procesamiento multihebra es necesario, al menos:

- Un PC (contador de programa) distinto para cada hebra que pueda ejecutarse concurrentemente.
- HW para ejecución concurrente.

Procesamiento Multihebra

Cambios de contexto en ejecución Multihebra

El contexto de una hebra está formado por:

- El contador de programa (PC)
- Registros de datos, de direcciones (punteros), de estado, control, de segmentos
- Datos propios en memoria (eventualmente)
- Datos en caché (eventualmente)

Cuando se cambia de una hebra a otra, se produce un cambio de contexto (de hebras).

Los cambios de contexto de hebras pueden ser de 2 tipos:

- tradicional
- rápido.

Procesamiento Multihebra

46

Cambios de contexto (de hebras) tradicional

- Lo produce el SO mediante una interrupción, típicamente asociada a un timer (asignación temporal de hebras).
- La interrupción detiene la hebra en ejecución.
- El SO salva el contexto de la hebra en ejecución.
- El SO recupera el contexto de la hebra detenida anteriormente.
- La hebra detenida anteriormente se reinicia mediante un retorno de interrupción.
- La hebra detenida no se entera que fue interrumpida.

Procesamiento Multihebra

47

Cambios de contexto (de hebras) rápido (por hardware)

- Se produce porque la hebra queda en espera por alguna razón, típicamente por un fallo de acceso a la caché (tener en cuenta que la PF de acceso a la caché es de varios órdenes de magnitud respecto de un acceso a la caché).
- Si el procesador tiene duplicados, al menos, el PC y los registros (todos o gran parte), entonces el procesador conmuta de hebra sin necesidad de salvar el contexto de la hebra detenida por el fallo (que consume tiempo).
- Este cambio de contexto es mucho más rápido que el tradicional (del SO).

Procesamiento Multihebra

Políticas de ejecución multihebra

Las hebras en ejecución usan recursos. Los recursos pueden ser asignados de 2 formas:

- **Estática:** el particionado de recursos es fijo.
- **Dinámica:** el particionado de recursos cambia por algún mecanismo.

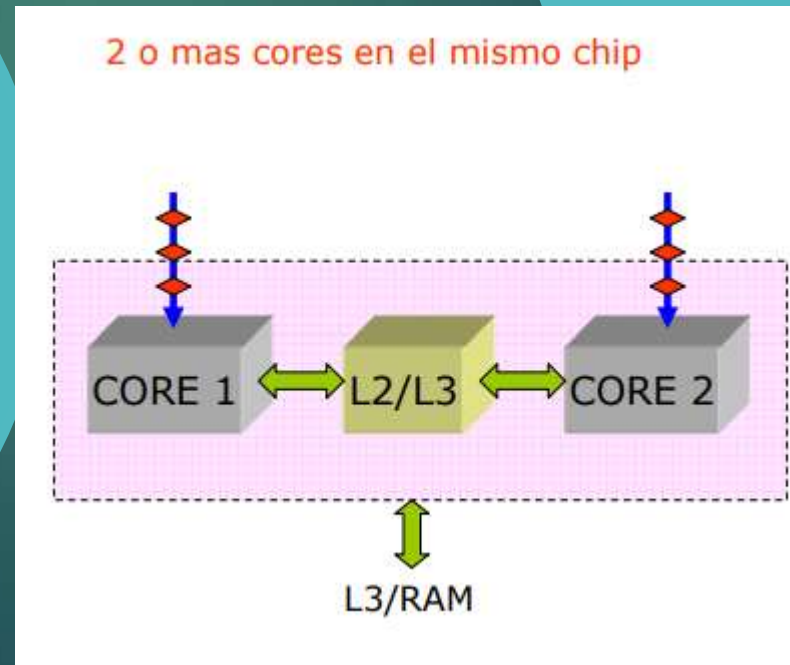
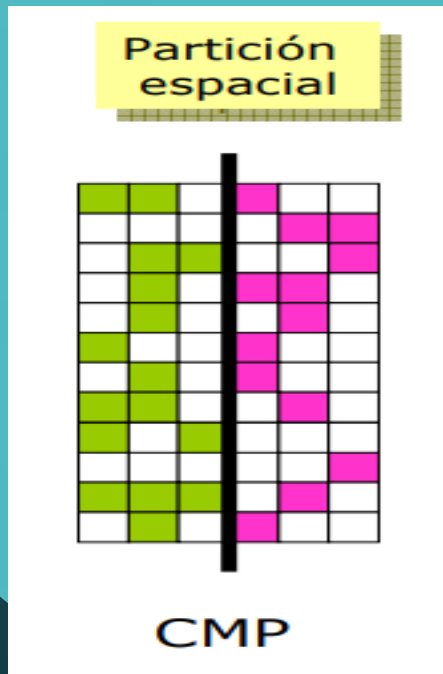
Existen 4 políticas de ejecución multihebra (MT):

- CMP (Chip Multi Processor)
- FGMT (Fine Grained Multi Threading)
- CGMT (Coarse Grained Multi Threading)
- SMT (Simultaneous Multi Threading)

Procesamiento Multihebra

Procesamientos multihebra tipo CMP

- El procesador dispone de 2 o más núcleos. A cada núcleo se le asigna una hebra en forma fija.
- La política de partición CMP es de tipo estática espacial.
- Típico de procesadores SMP de 2 o más núcleos.



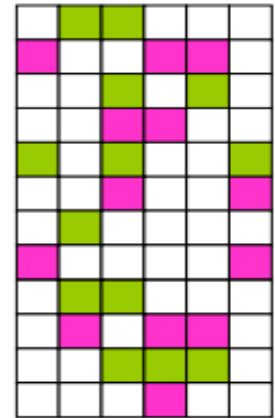
Procesamiento Multihebra

50

Procesamientos multihebra tipo FGMT

- El procesador conmuta de una hebra a otra en cada ciclo. La conmutación es rápida, 1 sola hebra por vez. El procesador dispone de 2 o mas contextos, para ejecutar 2 o más hebra concurrentemente (Ej: Cray CDC 6600 y Denelcor HEP).
- La política de partición FGMT es de tipo estática temporal.
- Al ser una conmutación por ciclo se dice que es de “grano fino” (FGMT : Fine grained Multi-threading)
- Reduce las latencias en memoria, y evita y resuelve dependencias de datos.
- Requiere que el compilador encuentre muchas hebras independientes.

Partición
temporal



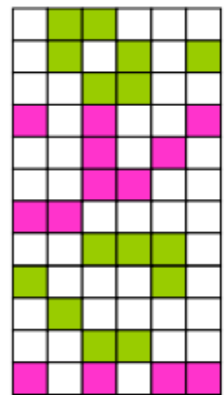
FGMT

Procesamiento Multihebra

Procesamientos multihebra tipo CGMT

- El procesador conmuta de una hebra a otra cuando una hebra se detiene por algún evento de gran latencia (por ejemplo, fallo de acceso a la caché).
- La política de partición CGMT es de tipo dinámica temporal.
- La conmutación por algún fallo se dice que es de “grano grueso” (CGMT : Coarse grained Multi-threading)
- En general requiere un manejo más complejo porque la conmutación es asincrónica.
- Se deben replicar registros para conmutar rápido.
- Se debe hacer una distribución equitativa de las hebras

Por ciclo de reloj



CGMT

Procesamiento Multihebra

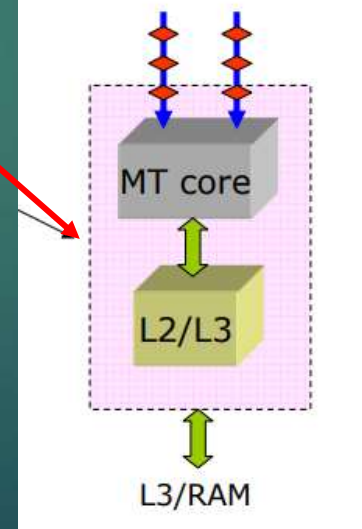
Procesamientos multihebra tipo SMT

- Hay varias hebras ejecutándose en varias unidades funcionales de un procesador superescalar (1 solo núcleo).
- La política de partición SMT es de tipo dinámica espacial.

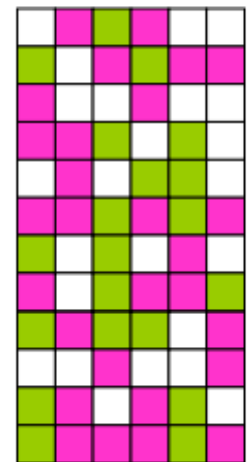
Ej: Pentium 4, core i3, i5, i7

(Intel lo llama Hyper-threading)

un único core en el chip



Por unidad funcional



Procesamiento Multihebra

53

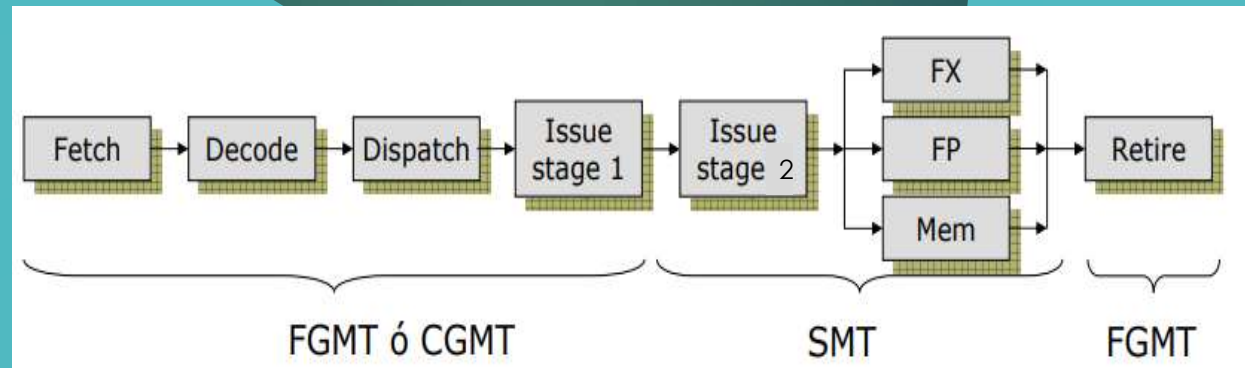
Pentium 4

- El Pentium 4 es un procesador con una arquitectura Multithreading (Hyper-threading o HT como la llama Intel) híbrida, que permite disponer de 2 procesadores lógicos compartiendo algunos de los recursos de ejecución del procesador físico (es decir, 1 solo núcleo).
- El SO reparte las tareas entre ellos.
- Cada procesador lógico puede actuar individualmente.
- La mejora depende fuertemente de grado de paralelización de las tareas. En algunos casos puede llegar a un 30% de mejora. En otros puede hacer la ejecución más lenta (por ello se puede deshabilitar el “modo HT”).

Procesamiento Multihebra

Pentium 4

El Pentium 4 tiene distintas políticas de tratamiento de las hebras dependiendo de las unidades funcionales involucradas (por eso es un híbrido). En la imagen siguiente se pueden ver las estrategias usadas en cada etapa.



- Etapas Fetch-Decode-Dispatch-Issue 1: FGMT, hasta que una hebra se frena, pasando a CGMT.
- Issue 2 – Ex – Mem: SMT, las hebras pueden mezclar instrucciones.
- Retire: FGMT

Referencias

- Organización y Arquitectura de Computadoras, William Stallings, Capítulo 16 de 5ta edición ó Capítulo 18 de 7ma edición.
- Diseño y evaluación de arquitecturas de computadoras, M. Beltrán y A. Guzmán, Capítulo 5 de 1ra edición.